

从 C++ 到 AI 计算：第三册

张量、算子、量化与本地大模型推理引擎

CPU Performance Study

本册接续第一册的程序执行证据链和第二册的现代 CPU 账本，围绕一个能推理开源 7B 级 decoder-only 大模型的本地引擎展开：模型加载、tokenizer、张量布局、GEMM、attention、KV cache、量化、CPU/GPU 后端、调度、验证和性能对标都必须落到可测量的机器事实。

前言

第一册把 C++ 程序追到二进制、ABI、汇编和可信测量；第二册把同一个程序继续放进现代 CPU、多核心、缓存、TLB、SIMD、线程、运行时、I/O 和分布式边界中。第三册从这里进入 AI 计算，但它不会把 AI 写成框架 API 清单。真正的问题不是“怎样调用一个模型”，而是：一个能推理开源 7B 级大模型的本地引擎，怎样把模型文件、tokenizer、权重张量、GEMM、attention、KV cache、量化参数、CPU/GPU kernel、调度器和测量报告组织成一条可解释的机器链路。

核心思想

第三册把大模型推理拆成可以验证的机器事实：张量是带形状、步长和生命周期的内存视图；算子是有输入合同和输出合同的 kernel；量化是整数表示、误差和吞吐之间的合同；KV cache 是跨 token 持续增长的状态；推理引擎是模型、内存、后端、调度和测量证据的组合。

正文以原理和工程证据为主，使用伪代码、关键 C++ 片段、抽象汇编路径、数据布局表、流图和测量边界解释机制；工程目录负责把这些机制落实到可构建、可测试、可 benchmark 的实现。

怎样阅读本册

阅读第三册时，不要先把注意力放在模型名和框架名上。模型名称会变，框架接口会变，硬件指令集也会扩展；但本地大模型推理的基本账本长期稳定：权重字节从哪里来，token 怎样变成 id，张量按什么步长读取，在哪个类型中累加，KV cache 写到哪里，CPU/GPU 后端各自承担哪些 kernel，怎样判断误差可接受，怎样证明时间数字对应正确工作。

如果你完全不懂编译原理，也可以从本册开始。先把“编译器”理解成一个严格的工程流水线：读懂输入，建立中间表示，检查不变量，在语义不变的前提下改写成更适合机器执行的形式。AI 编译器只是把这件事用在模型文件、张量、量化、推理图、后端 kernel 和运行时调度上。遇到 IR、verifier、lowering、pass、runtime 这些词，不要先背定义，而要问：它在本地推理引擎里检查了什么、生成了什么、节省了哪部分机器成本。

本册会频繁回扣第一册和第二册。第一册教你把 C++ 代码追到 ABI、汇编、对象布局和可信 benchmark；第二册教你把同一段机器工作放进现代 CPU、cache/TLB、SIMD、线程、NUMA、运行时和 I/O 边界里解释。第三册讲 AI 编译器和推理后端时，会把这两类能力当成工具，而不是重新孤立发明一套 AI 术语。

建议按四条线并行阅读。

- 1. 架构线：模型资产、编译前端、IR 优化、后端、运行时分别负责什么，哪些事实不能跨层乱猜。
- 2. 语义线：每个算子定义什么输入、输出和错误边界。
- 3. 表示线：token、张量、权重、scale、zero point、activation buffer 和 KV cache 怎样存在内存里。
- 4. 机器线：核心循环生成什么地址序列、使用哪些指令、触发哪些 cache、带宽、NUMA、PCIe 或显存问题。
- 5. 后端线：同一个算子合同怎样分别落到 CPU SIMD/多线程 kernel 和 GPU kernel/stream 上。
- 6. 证据线：reference、误差 oracle、反汇编、profiler、benchmark 和框架对标报告怎样组成闭环。

若某个优化看起来很诱人，先问五件事：它是否保持算子语义，是否改变了机器账本中真正等待的部分，是否适用于 prefill、decode 或两者，C++20 工程实现是否有清楚模块边界和资源所有权，是否有测量证据证明它相对当前基线和现有框架缩小了差距。只要这些问题没有说清，就不要把它当成可靠优化。

固定主案例：MiniLLM-CPU

第三册所有重要概念都应回到同一个主案例：MiniLLM-CPU（一个从 tiny fixture 到 7B shape 账本的本地推理引擎切片）。它有两个尺度。第一个尺度小到可以手算，用来训练正确性；第二个尺度接近真实 7B，用来训练容量、带宽和后端判断。

尺度	固定参数	用途
tiny fixture	<code>vocab=16,hidden=8,layers=2,heads=2,kv_heads=1,head_dim=4,seq=4</code>	手算 shape、stride、RoPE、GQA、KV cache、logits golden
kernel fixture	<code>T=2,K=8,N=6</code> 与 <code>H=4096,0=4096,K=4096</code>	对照 naive、packed、SIMD、tail、cache 和带宽上限
7B shape	<code>L=32,H=4096,heads=32,kv_heads=8,head_dim=128,ctx=4096</code>	估算权重字节、KV 容量、prefill/decode tokens/s 上限

tiny fixture 贯穿的 token 链路固定为：

Listing 1 MiniLLM-CPU 的固定 token trace

```
prompt ids
-> embedding [T,H]
-> RMSNorm [T,H]
-> Q [T,heads,head_dim], K/V [T,kv_heads,head_dim]
-> RoPE on Q/K
-> KV cache append [layer,position,kv_head,head_dim]
-> GQA attention output [T,H]
-> MLP SwiGLU [T,H]
-> lm_head logits [T,vocab]
-> sampler next token
```

每一章都必须说明自己接在这条链路的哪里。讲张量，要给 shape、stride 和地址公式；讲量化，要给 byte layout、scale 和误差；讲 compiler，要给 IR dump、pass 前后差异和 plan；讲 backend，要给 baseline、反汇编、perf 或替代计数器；讲 serving，要给 queue、TTFT、TPOT、取消和拒绝。

每章最低密度

第三册不接受只讲“应该有什么”的章节。每章至少要留下下面十类内容中的多数项，核心章节必须全部覆盖：

验收与评分标准

1. 一个具体问题，不用抽象口号开头。
2. 一个 tiny 数值例子，能手算或能人工检查。
3. 一个真实 shape 例子，至少说明 7B 级别的字节或延迟影响。
4. 一段可编译或可直接迁移到 C++20 项目的核心代码。
5. 一张地址流、内存布局、状态机或时间线图。
6. 一张 benchmark、ledger 或 oracle 表。
7. 一段 perf、反汇编、trace 或等价工具证据。
8. 一个错误案例，例如 shape 错、tail 错、KV offset 错、scale 错、fallback 静默发生。
9. 一个作业，要求读者改代码或复现实验。
10. 一个和 MiniLLM-CPU artifact 的连接：fixture、命令、CSV、report、CI 或 release gate。

学完第三册应能做什么

学完第三册不等于可以胜任所有 AI infra 岗位，但应达到一个明确出口：

- 能写出 tiny decoder 的 reference，并用 logits golden 定位 embedding、RMSNorm、QKV、RoPE、attention、KV cache、MLP 和 sampler 的错误。
- 能为 7B 级模型计算 fp16/int8/int4 权重字节、KV cache 容量、decode 最小读流和带宽 tokens/s 上限。
- 能实现一个 int8 GEMV/GEMM 路径：scalar baseline、packed layout、SIMD 或清晰后端接口、tail path、operator oracle 和 benchmark。
- 能设计 AI compiler 的最小 IR：TensorDesc、GraphNode、ShapeVerifier、MemoryPlan、BackendLowering、ExecutablePlan 和 pass verifier。
- 能把本地推理接成服务：admission、session、KV 预算、prefill/decode 调度、stream event、取消、backpressure、SLO trace。

- 能写一份工程证据报告：correctness、benchmark、perf/trace、工业框架对标、已知限制和下一步优化。

若这些做不到，说明只是读过 AI infra 名词；若这些能稳定完成，才说明第三册真正把第一册和第二册的系统能力用到了本地大模型推理上。

常见缩写和术语先导

第三册的缩写和工程词最密，因为它同时涉及模型、编译器、CPU/GPU 后端和服务运行时。下面这页不是让你背名词，而是把字母、形状词和工程对象先和本书主线对齐。

CPU/GPU/ISA/API/ABI/SDK/GDB CPU 是中央处理器；GPU 是图形处理器或通用并行加速器；ISA 是指令集架构；API 是源码接口；ABI 是二进制接口；SDK 是软件开发工具包；GDB 是 GNU 调试器。第三册会不断把模型算子落到这些机器和工具边界。

C++20 C++ 标准版本名。本册的 descriptor、RAII、span、oracle、runtime plan 等工程示例默认以 C++20 为实现边界。

SIMD/AVX/AVX2/AVX-512 SIMD 是一条指令处理多个数据；AVX/AVX2 是 x86 向量扩展；AVX-512 提供更宽寄存器和 mask 机制。后端选择要看 shape、tail、频率和平台覆盖。

FMA/VNNI/AMX/PMU/IPC FMA 是乘加指令；VNNI 是低精度 dot-product 相关指令能力；AMX 是 x86 tile 级矩阵扩展；PMU 提供硬件计数器；IPC 是每周期指令数。

TLB/L2/L3/LLC/HBM/DRAM/NUMA TLB 是地址翻译缓存；L2/L3/LLC 是缓存层级；HBM 是高带宽显存；DRAM 是主存；NUMA 表示不同内存节点访问成本不同。权重流、KV cache 和 activation arena 都会碰到这些成本。

JSON/CSV/CI/I/O JSON 常保存模型配置、tokenizer 或 manifest；CSV 常保存 benchmark 表格；CI 是持续集成；I/O 是输入输出。模型加载和性能验收离不开这些工程格式。

Unicode/UTF-8 Unicode 是统一字符集合；UTF-8 是它的常见变长字节编码。Tokenizer、prompt、stop string 和 golden case 都必须保存清楚的字节边界。

KB/KiB/MB/MiB/GB/GiB 容量单位。KB/MB/GB 常按十进制理解，KiB/MiB/GiB 明确按 1024 进位；模型权重、KV cache、arena 和显存预算必须写清单位口径。

mmap/perf/objdump mmap 把模型文件或匿名页映射到进程虚拟地址空间；perf 用采样和 PMU 事件看瓶颈；objdump 用来确认后端是否生成预期指令。它们分别服务加载、测量和机器码验证。

LLM Large Language Model, 大语言模型。本书关注它在本地推理时如何变成张量计算、KV cache 和请求调度。

KV cache Key/Value cache, Transformer 解码时保存历史 token 的 K/V 张量，避免每生成一个 token 都重算全部历史。

QKV Query、Key、Value。attention 会用 Q 查历史 K，并把权重作用到 V 上得到上下文表示。

BM/BN/BK GEMM tile 记号。BM 表示一次处理多少 batch 行，BN 表示多少输出通道，BK 表示沿 K 维一次推进多少元素。

TILE_N/TILE_K 后端实验里的 tile 常量。TILE_N 表示 N 方向一次处理的列块，TILE_K 表示归约维度块；它们是 schedule 参数，不是新模型结构。

QH/KVH/KVA QH 是 query head 数；KVH 或 KVA 是 key/value head 数。它们不是固定算法名，而是 shape verifier 里必须检查的一组形状变量。

MLP/BLAS MLP 是 Multi-Layer Perceptron，多层感知机；BLAS 是 Basic Linear Algebra Subprograms，基础线性代数接口族。Transformer 块里的 MLP 通常指 attention 后面的前馈网络。

GEMM/GEMV/FLOP/MAC GEMM 是矩阵乘矩阵，GEMV 是矩阵乘向量；FLOP 是浮点运算次数；MAC 是 multiply-accumulate，乘加次数。prefill 更像 GEMM，decode 中很多投影更像 GEMV。

FP16/BF16/TF32/INT8/MFLOP/GFLOP/TFLOP FP16、BF16、TF32、INT8 是常见低精度 dtype；MFLOP/GFLOP/TFLOP 是百万、十亿、万亿级浮点运算量或吞吐单位。报告必须写清 dtype 和 FLOP 口径。

RMSNorm/RoPE/SwiGLU RMSNorm 是均方根归一化；RoPE 是旋转位置编码；SwiGLU 是

一种门控前馈网络激活结构。

MHA/GQA/MQA MHA 是多头注意力；GQA 让多个 query head 共享一组 KV head；MQA 让所有 query head 共享一组 KV head。

QK/PV attention 中的两个主要矩阵乘。QK 计算 query 和 key 的相似度分数；PV 把 softmax 后的权重作用到 value 上。

FlashAttention 一类分块 attention 思路。数学结果仍是 attention，但通过在线 softmax 和 tile 复用减少中间矩阵写回。

TTFT/TPOT TTFT 是 Time To First Token，首 token 延迟；TPOT 是 Time Per Output Token，之后每个输出 token 的平均时间。

p50/p95/p99 延迟分位数。p50 表示典型请求，p95/p99 表示尾部请求；模型服务要同时看 TTFT/TPOT 的分位数，而不能只看平均值。

SLO Service Level Objective，服务等级目标，例如 TTFT、TPOT 或 p95 延迟阈值。

FIFO/FAQ/DDIA/RAG/BM25/HTML FIFO 是先进先出调度；FAQ 是常见问题；DDIA 常指《Designing Data-Intensive Applications》这类数据密集系统材料；RAG 是检索增强生成；BM25 是经典稀疏关键词检索打分；HTML 是网页标记语言。

GGUF llama.cpp 生态常见模型文件格式，通常把权重、量化信息和模型元数据打包在一个文件中。

ggml/llama.cpp/vLLM/oneDNN ggml 和 llama.cpp 是本地 LLM 推理生态中的 C/C++ 项目；vLLM 是高吞吐 LLM 服务系统；oneDNN 是 CPU 深度学习算子库。本册提到它们时，是把工业实现当作 shape、layout、kernel 和 runtime 的对照样本。

ONNX Open Neural Network Exchange，开放神经网络交换格式。它用于描述模型图、算子和权重，导入时必须经过 shape、dtype 和算子语义校验。

BPE/BOS/EOS/PAD/UNK/OOV BPE 是 Byte Pair Encoding，一类 tokenizer；BOS 是句首 token，EOS 是句尾 token，PAD 是填充 token；UNK 是未知 token；OOV 是 out-of-vocabulary，词表外输入。

组合缩写 BOS/EOS、GEMM/GEMV、MLIR/TVM、VNNI/AMX 这类斜杠组合表示“把几个相关概念并列讨论”。斜杠不是新术语，读的时候拆回每个条目即可。

MoE/LoRA MoE 是 Mixture of Experts，专家混合结构；LoRA 是 Low-Rank Adaptation，用低秩增量适配模型权重。

GPTQ/AWQ/NF4 常见大模型权重量化名词。GPTQ 偏二阶近似逐层量化，AWQ 强调保护重要 activation 通道，NF4 是非均匀 4-bit codebook 量化格式之一。

LCQI 本书项目里的 Local CPU Quantized Inference，指本地 CPU 量化推理实验线。它是教学工程代号，不是业界标准缩写。

LCQI_TOKENIZER_V1 本册教学工程中的 tokenizer manifest 版本名，表示“LCQI 项目的第 1 版 tokenizer 合同”。它不是外部标准。

IR/SSA/pass/lowering/UB IR 是中间表示；SSA 是静态单赋值形式；pass 是读取并分析或改写 IR 的步骤；lowering 是把高层表示逐步降到后端可执行形式；UB 是 undefined behavior，C++ 未定义行为。

AST Abstract Syntax Tree，抽象语法树。第三册提到它时，通常是为了把传统编译器概念和 AI graph IR 做类比。

MLIR/TVM/XLA 三类工业编译器或编译框架。本书提到它们时，重点是比较 IR、pass、schedule 和 lowering 的工程边界。

CUDA/HIP/NCCL/PCIe CUDA 是 NVIDIA GPU 编程接口；HIP 常用于 AMD GPU 生态；NCCL 是 NVIDIA Collective Communications Library，常用于多 GPU collective 通信；PCIe 是主机和设备之间常见互连。

ACL Arm Compute Library，Arm 生态常见 CPU/GPU 计算库。本书提到它时，重点是把算子合同映射到已有后端库。

SM/CU GPU 的基本执行资源组。NVIDIA 常称 SM，AMD 常称 CU。

HBM/GDDR GPU 常见显存类型。HBM 带宽高、封装近；GDDR 更常见于消费级 GPU。后端

报告要区分显存带宽和主机 DRAM 带宽。

OOM/RNG/NaN OOM 是内存不足；RNG 是随机数生成器；NaN 是 not-a-number，常提示数值计算已经失控。

KL/MSE/MMR/MRR/PII KL divergence 和 mean squared error 是量化校准中常见指标；MMR 是最大边际相关性，用来在检索中平衡相关性和多样性；MRR 是平均倒数排名；PII 是 personally identifiable information，个人可识别信息。

UI/PR UI 是用户界面；PR 是 pull request，代码合并请求。验收章节提到它们时，关注状态机、回归门禁和可审查证据。

KB/MB/GB 存储容量单位，分别表示千字节、兆字节、十亿字节量级。模型大小、KV cache 和 arena 报告必须写清单位口径。

token/tokenizer/prompt/vocab token 是模型处理的离散编号或片段；tokenizer 把文本字节映射成 token 序列；prompt 是输入给模型的上下文；vocab 是可用 token 集合。不要把用户里的一个汉字、一个字节和一个 token 混为一谈。

shape/dtype/layout/stride shape 是张量各维长度；dtype 是元素类型；layout 是物理排列；stride 是相邻下标在内存中的步幅。后端优化前必须先证明这四件事解释一致。

descriptor/metadata/schema/manifest descriptor 是结构化描述对象；metadata 是描述数据的数据；schema 是格式规则；manifest 是导入后保存模型、tokenizer、权重、量化和后端事实的机器可读清单。

reference/oracle/golden/fixture reference 是定义正确结果的清晰实现；oracle 是判定器；golden 是已确认的期望结果；fixture 是测试输入和环境搭好的样本。

backend/runtime/kernel/microkernel backend 是具体硬件实现层；runtime 是执行计划、内存和调度层；kernel 是某个算子的具体热路径实现；microkernel 是更小的 tile 级核心计算单元。

arena/workspace/packed weight arena 是预先规划的大块内存；workspace 是临时工作区；packed weight 是为后端读取顺序重排后的权重。它们决定热路径是否反复分配、拷贝和解包。

prefill/decode/logits/sampling prefill 是处理整段 prompt、建立 KV cache 的阶段；decode 是逐 token 生成阶段；logits 是下一个 token 的打分；sampling 是按 greedy、top-k、top-p 或温度等策略选 token。

tail/mask/lane/tile/block tail 是不能整除 tile、SIMD 宽度或 group size 的尾部；mask 标记有效 lane；lane 是向量槽；tile 和 block 是分块计算单位。低比特推理的大量 bug 都出在这些边界。

stream/event/callback/backpressure stream 是 GPU 或服务运行时的异步执行队列；event 是可记录和等待的完成点；callback 是完成后回调；backpressure 是过载时让上游减速、排队或失败。

baseline/benchmark/profile/trace baseline 是基线；benchmark 是受控性能测量；profile 是运行画像；trace 是事件流水。第三册的优化结论必须绑定这四类证据。

ReLU/ReLU6/activation ReLU 是把负数截到 0 的激活函数；ReLU6 进一步把结果截到 6；activation 是中间激活张量或激活函数。量化和融合时必须说明 clamp 发生在哪里。

SentencePiece/WordPiece/byte fallback SentencePiece 和 WordPiece 是常见 tokenizer 算法族；byte fallback 是词表不覆盖时退回到字节级表示。它们决定同一段 UTF-8 文本会变成哪些 token。

LangChain/LangGraph/LlamaIndex 常见上层 LLM 应用编排框架。它们不等于推理引擎，但会改变 prompt、tool、retrieval、stream、checkpoint 和 trace 合同。

OpenAI-style/URL/endpoint OpenAI-style 表示仿 OpenAI API 形态的请求、流式事件或兼容服务；URL 是统一资源定位符；endpoint 是服务入口地址。兼容接口不代表底层 tokenizer、预算和取消语义自动一致。

TensorRT-LLM/SmoothQuant/LM head TensorRT-LLM 是 NVIDIA 生态的大模型推理优化系统；SmoothQuant 是把 activation 和 weight 尺度迁移后再量化的一类方法；LM head 是

把 hidden state 投影到词表 logits 的输出层。

PDF/QA PDF 是固定版式文档格式；QA 在工程验收中常指质量保证，在问答应用中也可指 question answering。读到 QA 时要结合上下文判断是测试流程还是问答任务。

checkpoint/tool/chunk/bucket checkpoint 是可恢复状态；tool 是模型可调用的外部工具；chunk 是文档或输入切片；bucket 是按范围、长度或哈希归类的桶。

scheduler/pipeline/target/epoch scheduler 决定请求或 kernel 的执行顺序；pipeline 是多阶段处理链；target 是后端目标或质量/性能目标；epoch 是版本或轮次编号。

checksum/hash/contract/generation checksum 是输出摘要；hash 是摘要或哈希值；contract 是输入输出和错误边界；generation 是同一对象的版本编号。

register/heap/stack/interrupt/owner register 是寄存器；heap 是动态分配区域；stack 是调用栈；interrupt 是中断；owner 是当前拥有资源或写权限的一方。后端和服务代码里的所有权必须能被测试和日志说明。

roofline/throughput/bandwidth/coherence/shard/prefetch roofline 是用算术强度估计性能上界的模型；throughput 是吞吐；bandwidth 是带宽；coherence 是缓存一致性；shard 是数据分片；prefetch 是提前取数据以隐藏延迟。

LayerNorm/ALU/TensorRT LayerNorm 是按隐藏维做均值方差归一化的层；ALU 是算术逻辑单元；TensorRT 是 NVIDIA 推理优化生态，TensorRT-LLM 是其中面向大模型的系统。

sanitizer/worker sanitizer 是编译器插桩诊断工具族，用于发现越界、未定义行为或数据竞争；worker 是执行请求、kernel 或后台任务的线程/执行单元。

MiniLLM/TinyTrace/TensorView 这些是本册教学工程里的示例名：MiniLLM 表示最小本地推理线，TinyTrace 表示轻量事件追踪，TensorView 表示不拥有底层数据的张量视图。它们不是行业标准缩写，读到时按项目内对象理解。

Transformer 形状变量 `head_dim` 是每个 attention head 的隐藏维宽度；`kv_heads/kv_head` 是 Key/Value head 数或其中一个 head；`query_heads/q_head` 是 Query head 数或其中一个 head；`n_q/n_kv` 常分别表示 query head 数和 KV head 数。看到这些变量时，先检查 MHA、GQA、MQA 三种形状关系。

KV cache 字段 `kv_cache` 保存历史 K/V 张量；`prompt_tokens` 是输入上下文 token 数；`max_context` 和 `max_new_tokens` 分别限制上下文窗口和本次最多生成 token 数；`model_position` 是模型看到的绝对或相对位置。它们共同决定能不能继续解码、会不会越界。

量化字段 `scale_w` 和 `scale_a` 常表示权重、激活的量化比例；`zero_point` 表示整数零点；`group_size` 表示共享一组量化参数的元素数量；`acc_i32` 常表示 int8 乘加后用 32 位整数累加。量化 bug 往往来自这些字段的单位和广播方向不一致。

服务指标字段 `ttft_ms` 是首 token 延迟毫秒数；`prefill_tokens_per_s` 和 `decode_tokens_per_s` 分别表示 prefill、decode 吞吐；`finish_reason` 记录生成停止原因，例如到达 EOS、长度上限或取消请求。指标必须和请求大小一起解释。

后端实验名 `TILE_N`、`TILE_K`、`out_tile`、`packed_weight`、`tail_policy` 这类名字是后端 schedule 和布局选择。它们不是模型层名，而是为了让内层循环更接近 SIMD、cache 和寄存器约束。

项目字段名 `request_id`、`model_id`、`layer_id`、`block_id`、`tokenizer_hash`、`backend_plan_id` 这类 id/hash 字段用于重放和审计：id 定位对象，hash 确认内容版本，plan id 确认后端选择。模型工程不能只靠文件名猜版本。

数学边界短写 `K0/K1`、`V0/V1`、`W+1`、`T-1`、`G-1` 这类短写通常出现在 shape、窗口或循环边界推导中。字母本身没有固定全书含义，必须回到所在公式：K 常是 key 或归约维，V 常是 value，W 常是窗口，T 常是时间步或 token 位置，G 常是 group。

组合方向词 `GEMV/GEMM`、`QKV/MLP`、`ONNX/MLIR`、`TPOT/TTFT` 这类组合是在同一段比较几个已解释概念。斜杠表示并列，不表示一种新算法。

CMake 和转换 `cmake` 是构建系统命令；`static_cast` 是 C++ 显式类型转换。第三册的工程验收章节用它们记录“如何构建”和“类型边界如何写清”，不是模型概念。

目录

前言	ii
怎样阅读本册	iii
常见缩写和术语先导	vi
第 1 章 推理引擎全链路、张量账本与量化	1
1.1 端到端链路：从模型文件到下一个 token	1
1.2 张量布局、地址流与第一个矩阵向量内核	10
1.3 从矩阵向量乘走向 GEMM、packing 与 SIMD	17
1.4 量化系统总览：从实数模型到低比特推理合同	26
1.5 int8 量化、累加器与重新量化	32
1.6 低比特权重与 KV cache 量化：int4、group scale 与运行时状态	42
1.7 量化工程实现：C++20 descriptor、校准记录、oracle 与回归门禁	55
1.8 7B 推理计算账本：FLOPs、字节、KV cache 与延迟上限	70
第 2 章 Transformer、KV Cache、采样与服务运行时	81
2.1 AI 推理原理、KV cache 与计算账本：从一句 prompt 到本地 7B 引擎	81
2.2 状态机：一个 token 穿过 decoder	104
2.3 Reference decoder 实现：先把一层算对	106
2.4 推理算法：logits、softmax、采样、搜索与投机解码	116
2.5 KV cache 实现细节：布局、分页、位置、量化与故障模式	121
2.6 业务服务实战：请求、调度、队列、取消与资源预算	132
2.7 状态机：请求从到达到释放资源	143
2.8 上层 AI 应用编排：RAG、Agent、工具和状态图	149
第 3 章 模型导入、AI 编译器与内存计划	161
3.1 AI 编译器前端：模型导入、manifest 与 shape verifier	161
3.2 为什么真实模型加载是第三册的分水岭	174
3.3 Tokenizer 是模型合同的一部分	175
3.4 GGUF、safetensors 和分片权重	176
3.5 shape verifier 要从 config 推导，而不是照抄文件	177
3.6 mmap、lazy load 和 prepare 的边界	178
3.7 首 token trace：加载闭环的最终证据	178
3.8 错误案例库	179
3.9 验收标准	179
3.10 推理运行时与内存规划：typed graph 如何服务一次真实请求	179
3.11 AI 编译器：从推理图到机器执行计划	189
3.12 AI 编译器细化：IR、pass、lowering 与 executable plan	195
3.13 状态机：一个 pass 从候选到可上线	209
第 4 章 C++20 CPU/GPU 后端优化、工业对标与验收	217
4.1 CPU 后端优化细讲：地址流、SIMD、线程、NUMA 与 C++20 工程边界	217
4.2 GPU 硬件基础：SM、warp、显存层级、tensor core 与推理瓶颈	224
4.3 状态机：一个 GPU kernel 怎样消耗资源	232
4.4 GPU 后端优化细讲：device buffer、stream、kernel launch 与端到端延迟	235
4.5 CPU decode 实战：从标量 GEMV 到 packed 低比特微内核	242
4.6 GPU decode 端到端：常驻权重、KV 布局、sampler 与同步成本	251
4.7 工业 LLM 专项：投机解码、MoE、LoRA、多 GPU 与源码对照	259
4.8 为什么需要这一章	259
4.9 Speculative decoding：用验证换吞吐	260
4.10 MoE：算力稀疏，系统不稀疏	260
4.11 LoRA 和 adapter：低秩增量也是 runtime 状态	261
4.12 量化算法谱系：格式比论文名更重要	261
4.13 CUDA/HIP 与 Tensor Core：从 kernel 到 plan	261
4.14 多 GPU：并行不是把模型复制几份	262
4.15 工业源码对照：看设计理由，不贴 logo	262
4.16 专项能力的交付边界	263
4.17 验收标准	264
4.18 工程验收与回归体系：reference、测试、覆盖率、性能门禁和框架对标	264
附录 A 实践卷：本地 CPU 推理引擎切片	282
A.1 零、贯穿主线工程	282
A.2 一、工程入口	282
A.3 二、阶段 0：manifest 先说明字节怎样解释	283
A.4 三、阶段 1：tokenizer 合同固定输入身份	283

A.5 四、阶段 2: tiny MLP 是最短闭环	283
A.6 五、阶段 3: int8 kernel 从 scalar 对照到 AVX2	283
A.7 六、阶段 4: reference decoder trace 定位错误边界	284
A.8 七、阶段 5: 7B 账本不是实测吞吐	284
A.9 八、阶段 6: serving probe 把推理放回服务合同	284
A.10 九、项目阶段清单	284
A.11 十、每章代码任务矩阵	285
A.12 十一、递进大作业: LCQI 推理引擎	286
附录 B 完整源码清单: LCQI 本地 CPU 推理引擎	287
B.1 一、CMakeLists.txt	287
B.2 二、README	289
B.3 三、模型公共头	292
B.4 四、推理公共头	293
B.5 五、Reference Decoder 公共头	294
B.6 六、Int8 Kernel 公共头	297
B.7 七、Tokenizer 公共头	298
B.8 八、Safetensors 公共头	299
B.9 九、模型加载实现	300
B.10 十、Tiny MLP 推理实现	303
B.11 十一、命令行入口	305
B.12 十二、Reference Decoder 完整实现	307
B.13 十三、Int8 Kernel 基础实现	324
B.14 十四、Int8 Kernel AVX2 实现	331
B.15 十五、Tokenizer 实现	334
B.16 十六、Safetensors 实现	338
B.17 十七、Benchmark 程序	347
B.18 十八、Trace 程序	349
B.19 十九、Ledger 程序	352
B.20 二十、Serving Probe 程序	355
B.21 二十一、oneDNN 对标程序	359
B.22 二十二、Tiny MLP Fixture	362
B.23 二十三、Tiny Tokenizer Fixture	362
B.24 二十四、完整测试	363
第三册能力清单	373
术语表	374

第 1 章

推理引擎全链路、张量账本与量化

1.1 端到端链路：从模型文件到下一个 token

先把一条请求链路钉住

推理引擎里有很多对象：manifest、typed graph、量化、KV cache、CPU/GPU backend、oracle、profiler。若不先把它们放进同一条请求链路，读者很容易学到一堆孤立概念。这里用一个最小对话请求把全链路钉住：用户输入一段 prompt，引擎加载模型，编译计划，执行 prefill，进入 decode 循环，最后不断产生 token。

Listing 1.1 端到端推理链路的最小形态

```
model directory
-> manifest
-> verified typed graph
-> quantization and layout plan
-> CPU/GPU executable plan

request prompt
-> tokenizer
-> prefill plan
-> KV cache initialized
-> logits
-> sampler
-> decode plan loop
-> next token stream
```

先把这些词落到一个能检查的最小目录里。假设磁盘上有一个教学模型目录：

Listing 1.2 最小模型目录里每个文件的责任

mini-model/	
config.json	hidden size, layers, heads, vocab size
tokenizer.json	text <-> token id
tensors.bin	raw weight bytes
tensors.index.json	tensor name, dtype, shape, offset, checksum
manifest.json	canonical model contract

manifest 不是“又一个配置文件”。它把外部文件名、权重命名、shape、dtype、量化格式、tokenizer 版本和校验和整理成引擎内部可以相信的合同。没有 manifest，后端只能靠字符串猜 `model.layers.0.attn.q.weight` 到底是不是第 0 层 Q projection；一旦模型方言换名，错误会延迟到运行时。manifest 能证明“这些文件和权重按本引擎认识的规则组成同一个模型”，但它不能证明算子实现正确，也不能证明生成文本质量好。

typedgraph 是把 manifest 里的张量和配置连成有类型的计算图。最小 decoder 层里，它会记录：hidden 输入是 `[1, H]`，Q/K/V projection 输出是什么 shape，attention 读取和写入哪些 KV cache state，MLP 输出仍回到 `[1, H]`。typed graph 能证明 shape、dtype、layout 和状态边在图层面说得通；它不能证明某个 AVX2 kernel 足够快。

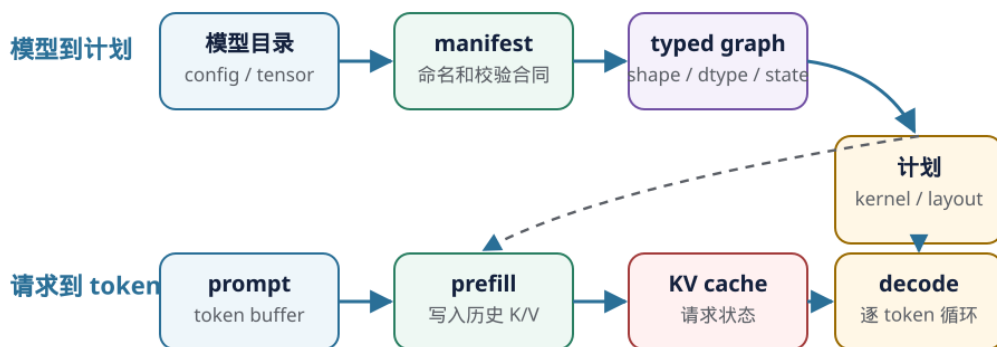
KVcache 是请求状态，不是模型权重。prefill 处理 prompt 时，把每一层每个历史 token 的 K/V

写进去；decode 第一步之后，每生成一个新 token，都要读取旧 K/V 并追加当前位置的 K/V。若把 KV cache 当成普通临时 activation，memory planner 可能复用或覆盖它，短 prompt 也许不炸，第二个 token 以后 logits 就会漂。

backend 是执行计划落到机器的选择：CPU scalar、AVX2、AVX-512、GPU kernel 或库调用。它只负责按合同执行 segment，不应该在热路径重新猜 shape。**oracle** 是裁判，用 reference 或分层检查比较结果；**profiler** 是账本，记录 prepare、prefill、decode、copy、sampler 和 fallback 的成本。oracle 证明“结果在误差合同内”，profiler 证明“时间花在哪里”；二者不能互相替代。

模型编译链与请求执行链

静态模型事实提前变成计划；每次请求只绑定会增长的运行时状态。



不要在 decode 热路径重新猜 shape 或重新 packing；计划可复用，KV cache 不能当普通临时张量复用。

图示：模型编译链提前固定静态事实，请求执行链绑定 prompt、KV cache 和 decode 状态。

这条链路有两个方向。第一条是“模型到计划”：它尽量在请求前完成，把能检查的语义、shape、dtype、量化、layout 和 kernel 选择提前固定。第二条是“请求到 token”：它发生在 runtime 中，绑定 session、token buffer、KV cache、arena、workspace 和 sampler。把这两条线分开，是推理引擎不退化成图解释器的关键。

核心思想

推理引擎的核心不是“写几个快 kernel”，而是把模型资产编成可执行计划，再把计划安全地绑定到一次会增长的请求状态上。优化只能发生在这条链路上的某个明确位置。

从一个坏的 `generate()` 看边界

最容易写出的推理程序，是一个巨大的 `generate()` 函数：读模型、解析 tokenizer、分配 activation、判断量化格式、选择 kernel、采样 token 全挤在一个循环里。

Listing 1.3 不可维护的推理主循环形态

```

generate(prompt):
    model = read_files()
    tokens = tokenize(prompt)
    for step in 0..max_new_tokens:
        for layer in model.layers:
            if layer.weight_format == "q4":
                unpack_and_run_q4()
            else:
                run_f32()
            maybe_allocate_more_memory()
            maybe_move_to_gpu()
  
```



```
next = sample(logits)
tokens.push_back(next)
```

问题不在于代码短，而在于它把四类生命周期混在一起：模型文件和权重属于模型生命周期；packed weight 和 executable plan 属于编译生命周期；token buffer、KV cache 和 sampler 属于请求生命周期；单个 kernel 的 workspace 和 profiler event 属于算子生命周期。边界混乱以后，错误定位、内存复用和性能测量都会失真。

生命周期	应该保存的对象	不应该做的事
模型生命周期	文件索引、manifest、原始权重视图	持有某次请求的 token 状态
编译生命周期	typed graph、packed weight、backend plan	每个 token 重新解析模型结构
请求生命周期	session、KV cache、sampler、trace	修改模型 manifest 或后端 capability
算子生命周期	局部 workspace、临时 tile、计数器	泄漏到下一层或下一次请求

常见误区

推理引擎里很多难查的问题，本质上不是数学错，而是生命周期错：静态对象在热路径重复构造，请求对象被全局共享，临时 buffer 被跨 step 使用，或者 profiler 把 prepare 成本算进 decode。

模型到计划：哪些事情应该提前做

模型加载阶段不应该只读文件。它要把外部资产变成编译器可以相信的事实。

阶段	产物	拦住的问题
读取资产	raw tensor index、config、tokenizer	文件缺失、大小不一致、checksum 错
manifest	canonical tensor roles	模型方言、权重命名、tied weight
verifier	typed graph 与诊断	shape/dtype/quant/KV 合同错误
优化与 lowering	executable plan	融合、layout、memory、backend 选择
prepare	packed weight、device buffer	热路径重复 packing 或上传

这张表的核心是“早拒绝”。如果 tokenizer 词表大小和 embedding 行数不一致，应该在前端报错；如果 int4 scale 数量不匹配，应该在量化 verifier 报错；如果 GPU backend 不支持某种低比特格式，应该在 plan 阶段显式 fallback 或拒绝。不要等到 decode 第 128 个 token 时才用错误 logits 暴露问题。

请求到 token：哪些事情只能运行时知道

运行时不是编译器的替代品。它处理的是请求期才知道的状态：prompt 内容、当前 position、最大生成长度、sampler 参数、KV cache 已写入长度、当前选择的 backend、是否打开 oracle/profiler。

Listing 1.4 一次请求的运行期状态

```
Session:
  token_buffer
  current_position
  kv_cache
```

```

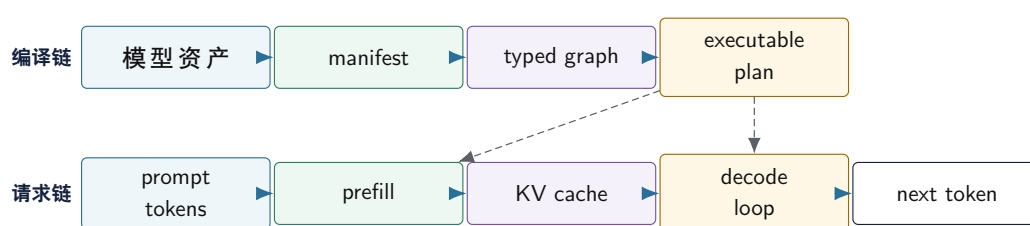
activation_arena
workspace
sampler_state
profiler_trace
oracle_policy

```

prefill 和 decode 的职责不同。prefill 读取整个 prompt，批量写入 KV cache，并产出最后一个 prompt token 的 logits。decode 每次只处理一个新 token，读取历史 KV cache，追加当前位置的 K/V，再产出下一步 logits。很多后端优化之所以失败，是因为把这两种形态混成一个计划。

同一个算子名字在不同阶段不一定是同一种机器工作。QKV projection 在 prefill 里更像大 GEMM，在 decode 里更像小 batch GEMV；attention 在 decode 里还要读取从 0 到当前 position 的历史 K/V；sampler 若每步强制 GPU 同步并拷回完整 logits，也会进入端到端延迟。

模型编译链路与请求执行链路



图示：本图要证明的命题是：模型编译链和请求执行链共享计划，但生命周期和错误边界不同。

计划不是抽象名词，而是一份可执行合同

本册反复使用 **可执行计划**（executable plan）这个词。它不是架构口号，而是一份后端和运行时都能执行的合同：语义 segment、kernel 选择、layout、packed weight、workspace、KV cache 读写、profiler event 和 fallback 规则都要落在这里。

Listing 1.5 executable plan 的最小合同字段

```

ExecutablePlan:
  segments:
    id
    operator_or_fused_region
    input_values
    output_values
    state_effects: reads_kv | writes_kv | none
    backend_choice
    kernel_descriptor
    workspace_bytes
    profiler_label
    oracle_checkpoint_policy

```

这里的 **state_effects** 很重要。decode attention 会读历史 KV cache，同时写入当前位置的 K/V；sampler 会读 logits 并更新随机数状态。若 IR 和 plan 不表达这些状态边，memory planner 可能错误复用 buffer，profiler 可能漏掉 cache 写入，oracle 也无法定位漂移点。

一个真实业务接口长什么样

说“能做业务开发”，不能只说能生成文本。最小可用接口要把请求参数、资源预算、流式输出、停止原因和诊断 trace 都暴露出来。一个本地推理服务的请求可以先设计成：

Listing 1.6 GenerateRequest 的最小字段

```
GenerateRequest:
  request_id
  model_id
  prompt
  max_new_tokens
  max_context_tokens
  temperature
  top_k
  top_p
  repetition_penalty
  stream: true | false
  quant_profile_id
  backend_policy: cpu | gpu | auto
  trace_level: none | summary | debug
```

这些字段不是接口装饰。`max_context_tokens` 决定 KV cache 容量；`max_new_tokens` 决定最坏 decode 时长；采样参数决定结果可复现性；`quant_profile_id` 决定误差和性能合同；`backend_policy` 决定 CPU/GPU plan 选择；`trace_level` 决定是否允许 materialize 中间结果和更细 profiler。

响应也不能只是一个字符串。流式输出至少要区分 token 事件、错误事件和结束事件：

Listing 1.7 流式响应事件

```
TokenEvent:
  request_id
  token_id
  token_text
  index
  model_position
  finish_reason_optional
  trace_event_id_optional

FinalResponse:
  request_id
  text
  generated_tokens
  finish_reason: stop_token | max_tokens | context_limit | cancelled | error
  usage:
    prompt_tokens
    completion_tokens
    kv_cache_bytes
  trace_summary_optional
```

这样设计后，业务层可以实现超时、取消、计费、审计、问题定位和重试。若接口只返回一个最终字符串，就无法判断卡在 prefill、decode、sampler、KV 容量还是后端 fallback。

流式生成不是 print 循环

流式生成的状态机要明确。prefill 完成之前，通常还没有第一个输出 token；prefill 之后每个 decode step 产生一个候选 token，sampler 决定是否结束，然后服务层把 token 事件推给客户端。

Listing 1.8 流式生成状态机

```

state = admitted
tokens = tokenizer.encode(prompt)

run prefill(tokens)
emit trace: first_token_ready

while not stopped:
    logits = decode_one_token(session)
    next = sampler.sample(logits)
    append next into session
    emit TokenEvent(next)
    if stop_condition(next):
        emit FinalResponse(stop_token)
        break

```

真实服务还要处理取消和背压。客户端断开时，runtime 必须停止后续 decode，并释放该 session 的 KV cache、arena 和 pending events。若下游消费 token 很慢，不能让输出队列无限增长；要设置队列上限，超过后暂停调度、丢弃请求或返回 backpressure 错误。否则模型算得再快，服务也会被 I/O 队列拖垮。

常见误区

流式输出是运行时协议，不是把每个 token 打印出来。它必须和 session 生命周期、KV cache 释放、取消、超时、trace 和 backpressure 绑定。

容量预算和 admission control

业务服务最先撞到的不是“能不能生成一句话”，而是能同时服务多少请求。admission control 的职责是在请求进入 decode 前判断资源是否足够。最小预算包括权重、packed weight、KV cache、activation arena、workspace、队列和系统余量。

Listing 1.9 请求进入前的容量判断

```

required_kv_bytes =
    layers * max_context_tokens * kv_heads * head_dim *
    2 /* K and V */ * kv_bytes_per_element

required_runtime_bytes =
    required_kv_bytes +
    activation_arena_bytes +
    workspace_bytes +
    trace_buffer_bytes

```

若使用前文 7B 级 GQA 参数， $L=32$ 、 $kv_heads=8$ 、 $head_dim=128$ 、fp16 KV、4096 context，一个 session 的 KV cache 约 512MiB。若 int4 权重约 3.7GiB，机器剩余可用内存 10GiB，理论上不能简单做 10GiB/512MiB。还要扣掉 packed weight、arena、workspace、allocator 碎片、线程栈、页表和操作系统余量。

admission decision 应该结构化：

Listing 1.10 AdmissionDecision 字段

```

AdmissionDecision:
    accepted: true | false
    reason:

```

```

ok | context_too_long | memory_budget_exceeded |
queue_overloaded | backend_unavailable
selected_plan_id
reserved_kv_bytes
reserved_workspace_bytes
estimated_ttft_ms
estimated_decode_tokens_per_s

```

这让业务层能给出明确错误：上下文太长、内存不够、队列满、GPU 后端不可用，而不是笼统返回“生成失败”。也让运维能知道是容量问题还是正确性问题。

SLO 要分首 token 和后续 token

推理服务的延迟目标不能只写一个平均响应时间。至少要拆成：

- **TTFT** (time to first token)：从请求进入到第一个 token 可输出。
- decode p50/p95：每个后续 token 的延迟分布。
- prefill tokens/s：prompt 处理吞吐。
- decode tokens/s：生成阶段吞吐。
- queue wait：进入模型前等待了多久。
- memory peak：请求期间峰值内存或显存。

prompt 很长但输出很短时，TTFT 主要由 tokenizer、prefill、KV 写入和首个 sampler 决定；prompt 较短但输出很长时，decode p95、KV 读流和 sampler 更重要。若服务端合批，平均 tokens/s 可能上升，但单请求 p95 可能变差，因为请求要等待同批次其他 session。

Listing 1.11 业务 SLO 报告字段

```

ServiceSloReport:
  prompt_bucket
  max_context_bucket
  backend_plan_id
  quant_profile_id
  queue_wait_p50_p95
  ttft_p50_p95
  decode_step_p50_p95
  tokens_per_second
  rejection_rate
  memory_peak_bytes
  fallback_count

```

有了这些字段，才谈得上把第三册内容落到一个本地助手、RAG 服务、代码补全服务或批量摘要服务里。不同业务目标会选择不同 profile，但底层证据字段是同一套。

排队模型：吞吐够不等于 p95 够

Serving 最容易被平均值骗。一个模型离线测得 **20tok/s**，不代表线上每个请求都能稳定低延迟。请求到达有波动，prompt 长度有波动，decode 长度有波动，prefill 和 decode 还会争同一组后端资源。只要利用率接近 1，排队等待就会迅速放大，p95/p99 会先崩。

先用最小模型描述单个服务台：

$$\rho = \lambda \times E[S]$$

其中 λ 是请求到达率， $E[S]$ 是平均服务时间， ρ 是利用率。若 $\rho \geq 1$ ，长期看队列必然增长；若 ρ 虽小于 1 但接近 1，稍微有长 prompt 或长输出，尾延迟就会明显上升。Little's Law 给出另一个必须检查的关系：

$$L = \lambda \times W$$

L 是系统中平均请求数， W 是请求在系统里的平均停留时间。这个式子不能直接告诉 p99，但能抓住一个硬约束：如果到达率不变而平均停留时间上升，系统内在途请求数一定上升；在推理服务里，这通常意味着 KV cache、队列内存和取消恢复压力一起上升。

LLM serving 还要把一次请求拆成 prefill 和 decode：

Listing 1.12 请求服务时间的最小拆分

```
service_time_ms =
    queue_wait_ms +
    tokenizer_ms +
    prefill_ms(prompt_tokens) +
    first_sample_ms +
    sum(decode_step_ms(context_i) for each generated token) +
    stream_flush_ms
```

这里的 `decode_step_ms(context_i)` 不是常数。前面 KV 账本已经说明，context 越长，每步至少要读更多历史 K/V。因此，同样生成 128 个 token，短 prompt 和长 prompt 的 decode 时间分布不同；同样 prompt，输出越长，后面的 token 也会越来越贵。

现象	排队模型解释	工程动作
平均 tokens/s 高，p95 TTFT 高	prefill 在队列里等待合批或后端空闲	分离 prefill/decode 队列，限制 batch 等待
decode p99 随并发突增	利用率接近 1，长输出占住 decode slot	admission control, max new tokens, 抢占或分片
长上下文请求拖慢全部请求 取消后内存仍高	单步 decode 服务时间随 S 上升 in-flight 请求未及时释放 KV/page	context 分桶调度，KV 预算隔离 cancellation barrier, session epoch, 回收审计
队列满但 GPU 利用率不高	CPU tokenizer、sampler、拷贝或锁成为服务台	分阶段 trace，拆服务台利用率

continuous batching 的取舍。 Continuous batching 把不同请求的 decode step 组合到同一批执行，提高后端利用率。它的代价是调度器每步都要决定哪些 session 进入 batch，如何处理刚完成 prefill 的请求，如何处理取消、超时和不同 context 长度。它改善吞吐，但可能增加单请求等待，尤其是短请求被长请求反复同批拖住时。

paged KV 的 SLO 意义。 Paged KV 不只是 kernel layout。它让 session 的 KV cache 可以按 block 分配、释放和复用，降低变长请求造成的碎片；也让 prefix cache 和迁移更容易。但它把一个连续地址公式换成 block table 查找，profile 必须同时记录 block 命中、page 分布、释放延迟和 slot epoch。否则，paged KV 的正确性 bug 会表现成偶发 logits 漂移，性能 bug 会表现成 p99 decode 抖动。

admission control 不是简单限流。 接收请求前要估算两类资源：静态资源是否够，例如 KV 最大容量、workspace 和队列长度；动态资源是否会把利用率推到危险区，例如预计 prefill 时间、预计 decode token 数、当前 in-flight decode step。一个实用的 admission report 至少要多写几行：

Listing 1.13 带排队模型的 admission report

```
AdmissionEstimate:
    arrival_rate_1m
    inflight_requests
    queue_depth
    estimated_prefill_ms
    estimated_decode_ms_p50
```



```

estimated_decode_ms_p95
estimated_kv_bytes
projected_utilization
projected_ttft_p95
decision: accept | reject | shed | retry_after

```

若报告没有 `projected_utilization` 和 `queue_depth`，它只能解释“内存够不够”，不能解释“接了这个请求后 p95 会不会炸”。若没有 `estimated_kv_bytes`，它只能解释排队，不能解释为什么取消恢复和长上下文会造成内存压力。

错误也要沿链路定位

链路完整的好处是错误能定位。若生成结果不对，不要直接怀疑 sampler 或模型质量，而要按链路缩小范围：

- tokenizer 输出是否和 reference 一致。
- manifest 中 embedding、lm head、Q/K/V、MLP 权重角色是否正确。
- shape/dtype/quant verifier 是否通过。
- packed weight 与 reference converter 是否一致。
- 单算子 oracle 是否通过。
- 层输出 oracle 从哪一层开始漂移。
- logits top-k 是否变化。
- sampler 固定 seed 后是否仍然分叉。

性能问题也一样。首 token 慢，不等于 decode 慢；decode 慢，不等于所有 kernel 都慢；GPU kernel 时间低，不等于端到端快。profiler 要沿链路拆：load、prepare、prefill、decode、sample、copy、sync、fallback。

用 trace 把链路变成证据

链路如果只存在于图里，仍然不能维护。每次运行都应该能输出一个轻量 trace，让 correctness 和 performance 共享同一套定位坐标。trace 不必一开始很复杂，但字段要稳定。

Listing 1.14 端到端推理 trace 的最小字段

```

RunTrace:
  run_id
  model_id
  manifest_hash
  backend_plan_id
  quant_profile_id
  prompt_tokens
  generated_tokens
  events:
    stage
    segment_id
    layer_id
    backend
    context_length
    start_ns
    end_ns
    bytes_read_estimate
    bytes_written_estimate
    oracle_status

```

`manifest_hash` 追溯模型语义; `backend_plan_id` 追溯 kernel、layout 和 fallback; `quant_profile_id` 追溯误差阈值和量化格式; `context_length` 让 decode 指标可分档。trace 坐标稳定, 后续 CPU SIMD、GPU kernel 和 KV cache 量化报告才可比较。

硬验收

读完本节, 应该能用一条链路解释全书:

- 模型资产先变成 manifest 和 typed graph, 再 lowering 成 executable plan。
- 请求运行时绑定 token、KV cache、arena、workspace、sampler、oracle 和 profiler。
- prefill 与 decode 是两种形态, 不能强行共用一个性能模型。
- 模型、编译、请求和算子生命周期必须分开, 否则内存、状态和测量都会串扰。
- executable plan 要显式记录 segment、backend、workspace、状态读写、oracle checkpoint 和 profiler label。
- 正确性和性能问题都应该沿链路定位, 而不是从最终文本倒猜。
- 端到端 trace 要让模型语义、后端计划、量化 profile、context length 和事件耗时可追溯。

1.2 张量布局、地址流与第一个矩阵向量内核

为什么先讲布局再讲算子优化

推理引擎最终会把 tokenizer、decoder graph、KV cache、CPU/GPU 后端和性能证据接在一起; 但不能从这些大词直接跳到最快 kernel。这里先拿一层矩阵向量乘做显微镜, 因为每个 Transformer projection、MLP 子层和 decode 阶段的小 batch 路径, 最终都会回到同一组问题: 张量在内存里到底怎样排布, 热循环每一步访问哪些地址, 累加器和输出缓冲怎样被机器指令更新。

本节先放大最小切片中的第一层:

Listing 1.15 本节先追踪这一层

```
hidden = relu(input * W0 + b0)
```

为了便于手工检查, 先把 batch 固定为 1, 把它看成一个矩阵向量乘:

Listing 1.16 矩阵向量乘的 reference 语义

```
for out in 0..N:
    acc = bias[out]
    for k in 0..K:
        acc += input[k] * weight[out, k]
    hidden[out] = max(acc, 0)
```

这段 reference 很慢也没关系, 它定义了结果。优化版本可以分块、展开、向量化、预打包权重、融合 ReLU, 也可以换成 int8 路径; 但每一次改造都必须回答同一组问题: `input[k]` 是否连续, `weight[out, k]` 是否连续, bias 何时加载, acc 用什么宽度保存, 输出写到哪里, activation 是分支还是条件数据流。

在 decoder-only Transformer 的 decode 阶段, 同样的形状会反复出现。当前 token 的 hidden state 是一条向量, `Wq`、`Wk`、`Wv`、`Wo`、`Wgate`、`Wup`、`Wdown` 都是巨大的权重矩阵。单 token decode 时, 很多 projection 看起来就是“一个向量乘很多权重行”; 只不过 `K` 可能是几千, `N` 也可能是几千到上万, 权重字节和 cache/TLB 压力会远大于教学切片。

核心想

AI 算子的第一层机器事实不是“矩阵乘很重要”, 而是: 循环次序决定地址流, 地址流决定 cache/TLB 和 SIMD 能否工作, 累加器宽度决定正确性边界, reference 和 oracle 决定优化是否可信。

张量元数据必须落到地址公式

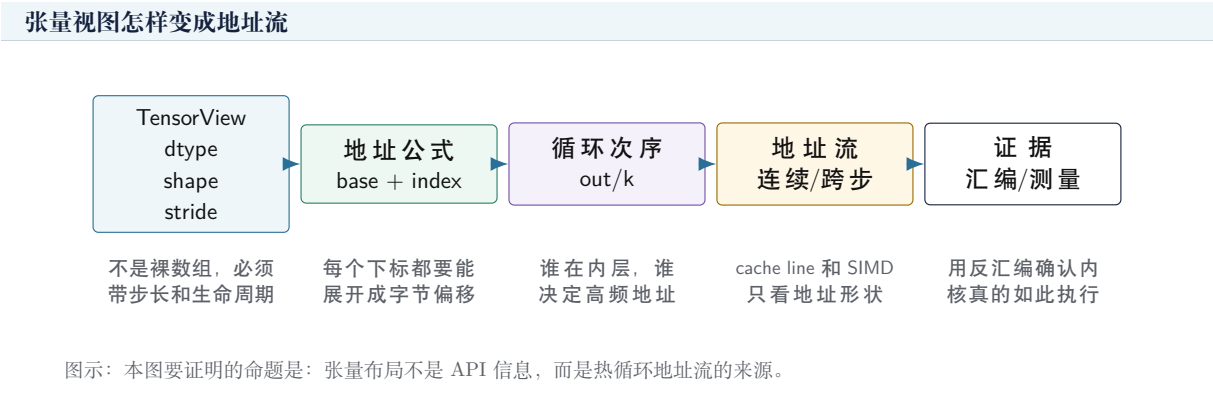
一个张量视图至少包含 dtype、shape、stride 和 data pointer。以二维权重矩阵 $W0[N,K]$ 为例，若按行主序连续存储，地址公式是：

Listing 1.17 行主序权重矩阵的地址公式

```
address(W[out, k]) =
    base_W + (out * stride_out + k * stride_k) * sizeof(element)

row-major contiguous:
    stride_out = K
    stride_k    = 1
```

若 $stride_k=1$ ，内层循环扫描 k 时权重地址连续；若权重来自转置视图， $stride_k$ 可能变成 N ，内层循环就变成跨步访问。源码里同样写 $W[out,k]$ ，机器看到的地址流却完全不同。第三册后面讲 packing，本质就是把“不适合内核的 stride”复制成“适合热循环的连续块”。



第一个 float32 矩阵向量内核

先写一个朴素但清楚的 float32 内核：

Listing 1.18 float32 reference 风格矩阵向量乘

```
1 void matvec_relu_f32(float const* input,
2                       float const* weight,
3                       float const* bias,
4                       float* hidden,
5                       int n,
6                       int k)
7 {
8     for (int out = 0; out < n; ++out) {
9         float acc = bias[out];
10        float const* row = weight + out * k;
11        for (int i = 0; i < k; ++i) {
12            acc += input[i] * row[i];
13        }
14        hidden[out] = acc > 0.0f ? acc : 0.0f;
15    }
16 }
```

这段代码的内层地址流很明确： $input[i]$ 连续读取， $row[i]$ 连续读取， acc 通常留在寄存器里， $hidden[out]$ 每个输出写一次。若 K 较大，权重流通常主导读取字节；若 N 较大，同一个

input 向量会被每个输出行复用。后续优化会围绕两件事展开：怎样让 input 和权重的复用更靠近核心，怎样让多个乘加并行进入 SIMD 或多个累加器。

一个抽象汇编账本可以写成：

Listing 1.19 float32 内层循环的典型机器形态

```
load  input[i]          ; stride-1 load
load  weight[row+i]     ; stride-1 load if row-major
mul   input, weight
add   product into acc
branch back to inner loop
```

这不是要求某个编译器必须生成标量 `mul/add`。在合适目标上，优化器可能生成向量 load、FMA、循环展开和水平归约。这里的重点是：无论标量还是向量，内核都必须消耗 input 地址流、weight 地址流和 acc 依赖链；SIMD 只是一次处理多个相邻元素，不能替代布局 and 地址分析。

把内层循环切成基本块

为了把第一册和第二册的方法接进 AI 算子，不能只停在 C++ 循环。下面这份伪反汇编把同一个 `matvec_relu_f32` 切成三个基本块：外层准备一行权重，内层做 dot product，尾部做 ReLU 和写回。

Listing 1.20 matvec 第一层的伪反汇编基本块

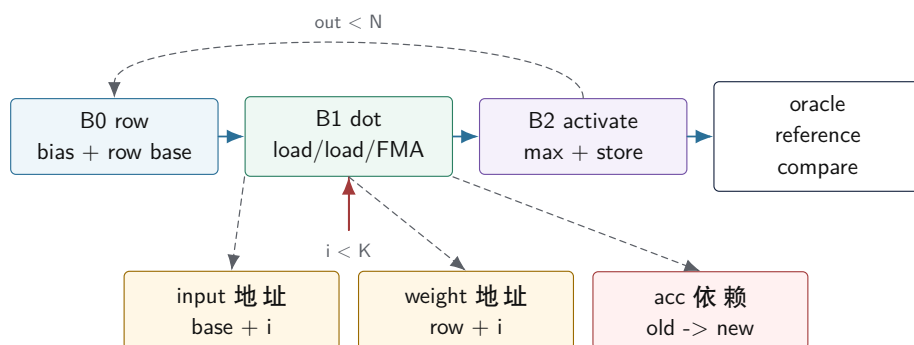
```
B0_row:
  acc = load bias[out]
  row = weight + out * K
  i = 0

B1_dot:
  x = load input[i]
  w = load row[i]
  acc = fma_or_mul_add(acc, x, w)
  i = i + 1
  branch i < K back to B1_dot

B2_activate_store:
  y = max(acc, 0)
  store hidden[out] = y
  out = out + 1
  branch out < N back to B0_row
```

这份账本故意不写具体寄存器名，因为寄存器分配、展开和向量化会随编译器变化。但它保留了性能分析必须稳定存在的边：`B1_dot` 的两条 load 由 `i` 产生地址，`acc` 是跨内层迭代的归约依赖，`i < K` 是内层回边，`hidden[out]` 写回发生在归约完成之后。若后续优化把 `B1_dot` 展开成四个累加器，变化的是依赖图；若把权重预打包，变化的是 `row[i]` 的地址流；若融合下一层，变化的是 `B2_activate_store` 的写回边界。

matvec 热区的控制流、地址流和证据闭环



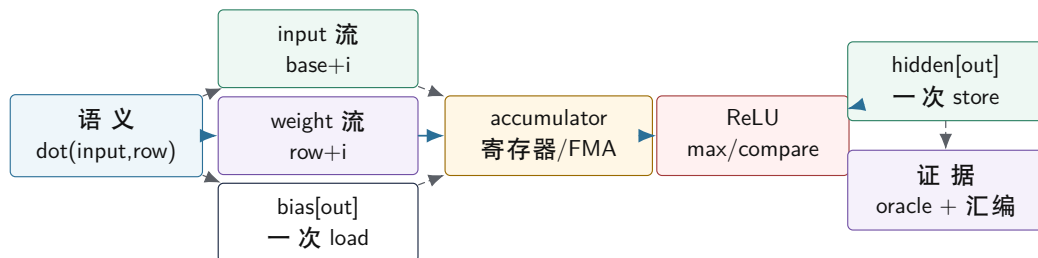
验证时先确认基本块和回边，再确认 load 地址是否连续，最后用 reference 输出和误差合同证明优化没有改变语义。

图示：本图要证明的命题是：AI 内核分析必须同时切出控制流、地址流、归约依赖和正确性证据。

按这张图读后续优化，很多术语会自然归位。循环展开不是“让代码更长”，而是让 **B1_dot** 中出现多条相互独立的 acc 链；SIMD 不是“自动快”，而是要求 **input** 和 **weight** 至少在热路径上形成可装载的相邻 lane；packing 不是“拷贝一份权重”，而是把 **weight** 的跨步或间接地址改造成内层连续流；ReLU 融合不是“少一个函数”，而是移动 **B2_activate_store** 的边界，减少中间张量的写回和读回。

用一张图同时看语义、地址和机器形态

矩阵向量内核的语义、地址流和硬件账本



本图先固定 float32 路径。int8 路径会把 input/weight load 改成窄整数，把 acc 改成 int32，并增加 requantize、round、clamp。

图示：本图要证明的命题是：一个 AI 算子要同时有语义账本、地址账本和机器账本，后续优化才能被验证。

按这张图读代码，bias 不是循环外的装饰，它是每个输出通道的一次初始 load；activation 不是数学符号，它要落成 **max**、条件移动、近似函数或查表路径；输出 store 不是性能主体，但它决定 hidden buffer 的生命周期和后续层的输入布局。若把 activation 与 matvec 融合，可以少一次写出再读回 hidden 的中间流；但融合后 oracle 必须仍能解释输出误差，调试路径也要能定位是哪一层出错。

布局错误怎样伪装成算子慢

常见误判是看到 **matvec** 慢，就直接想“需要 SIMD”。但如果权重不是行主序连续，或者每个输出行通过指针数组间接访问，SIMD 也只能在更差的地址流上工作。下面两种形状语义相同，机器成本不同：

Listing 1.21 两种语义相同但地址形状不同的权重访问

```

row-major packed:
    weight[out*K + 0], weight[out*K + 1], weight[out*K + 2] ...

pointer rows:
    rows[out] -> heap block
    rows[out+1] -> another heap block
    row pointers and weight bytes may live on many pages

```

第一种形状让内层循环沿着连续权重流前进；第二种形状先追一层指针，再进入某个堆块。若块很小、分配分散或页很多，TLB 和 cache line 利用率都会变差。对于 7B 级推理引擎，模型加载阶段把权重转换为内核友好的 packed layout 往往比在每次推理时容忍任意布局更合理。代价是加载时间、额外内存和格式版本；收益是热路径简单、可测、可向量化。CPU 后端可能选择 cache-friendly panel，GPU 后端可能选择显存连续、coalesced load 或 tensor-core 友好的 tile；两者都必须从同一份张量元数据出发。

TensorView 的 C++20 合同

工程里不能把张量参数写成一堆裸指针和整数。至少需要一个只观察内存、不拥有内存的 view；真正拥有内存的对象可以是 arena、mmap weight、DeviceBuffer 或 aligned buffer。view 的责任是描述地址公式。

Listing 1.22 TensorView 的最小字段

```

TensorView:
    data
    element_type
    rank
    shape[]
    stride[]
    byte_offset
    device
    layout_tag

```

它要满足几条不变量：

- `shape[d] >= 0`，rank 与 shape/stride 长度一致。
- view 不拥有内存，不负责释放 `data`。
- 地址计算必须检查或由 verifier 证明不会越界。
- `layout_tag` 只是快速路径提示，真正地址仍由 stride 决定。
- kernel 若要求 contiguous，必须显式检查，而不是假设 stride 合适。

Listing 1.23 contiguous 检查的语义

```

expected = 1
for dim from rank-1 down to 0:
    if stride[dim] != expected:
        return false
    expected *= shape[dim]
return true

```

这个检查看起来简单，但能拦住大量错误。比如一个 `[T,H]` 视图经过转置后 shape 仍然可能是二维，但 stride 不再是 row-major contiguous；若后端把它送进只支持连续输入的 GEMM kernel，输出就会错或性能极差。正确路线是：verifier 标出非连续，compiler 插入 layout conversion 或选择

支持 stride 的 reference/slow path。

layout conversion 也要进入 profiler

很多推理代码把 layout conversion 当成“准备工作”，没有计入耗时。实际它可能非常贵。若每个 decode step 都把某个 activation 从非连续转换成连续，转换成本会进入热路径；若模型加载时一次性把权重 pack 好，成本属于 prepare 阶段，应该和 decode 分开报告。

Listing 1.24 layout conversion event 字段

```
LayoutConversionEvent:
    source_value
    source_layout
    target_layout
    bytes_read
    bytes_written
    stage: prepare | prefill | decode
    reason: backend_requirement | fusion_boundary | debug_oracle
```

这能避免一种常见误判：kernel 本身很快，但前后各有一次 layout conversion，端到端反而慢。profiler 若只记录 kernel 时间，就会把瓶颈藏起来。

layout 测试矩阵

张量布局相关测试不能只测连续 happy path。最小测试矩阵应该包含：

- row-major contiguous 二维矩阵。
- 带 padding 的矩阵，例如每行 stride 大于列数。
- 转置 view，shape 正确但 stride 交换。
- 子视图，byte offset 非零。
- tail shape，K 或 N 不能整除 tile。
- 量化 packed layout，逻辑坐标与物理位置不同。

Listing 1.25 layout oracle 的基本做法

```
for each logical coordinate:
    reference = dense_reference[coordinate]
    actual = read_through_tensor_view(view, coordinate)
    compare actual with reference
```

这个测试不是性能测试，而是证明地址公式正确。只要地址公式错，后面 SIMD、GPU、量化都只是更快地读错数据。

从 float32 路径过渡到 int8 路径

量化路径不是把 float 替换成 int8_t 那么简单。同一层 matvec 进入 int8 后，机器账本变成：

Listing 1.26 int8 matvec 的机器账本变化

```
load int8 input values
load int8 packed weights
convert or widen to int16/int32 lanes
dot product into int32 accumulators
add int32 bias or folded bias
requantize acc to output dtype
apply clamp or activation
```

store output

权重字节变小, cache 能容纳更多参数; 某些 CPU 有专门的 dot-product 或 madd 指令, 能提高吞吐。但累加器必须更宽, 重新量化必须处理 scale、舍入和饱和, oracle 也必须从“逐元素几乎相同”变成“误差在合同范围内”。这就是第三册的基本节奏: 每个算子先有 float reference, 再有地址账本, 再有量化语义, 最后才进入特定 ISA 和 kernel 优化。

把这条账本接回 Transformer

现在把本节的 **matvec** 账本放回 decoder-only Transformer 的一个 decode 步骤。假设当前层输入是一条 hidden 向量 **x[hidden]**, 这一层至少要做下面几组投影:

Listing 1.27 decode 单 token 中反复出现的投影账本

```
q = matvec(x, Wq)
k = matvec(x, Wk)
v = matvec(x, Wv)
append k, v into KV cache at current position

attn = attention(q, K_cache[0..pos], V_cache[0..pos])
x = x + matvec(attn, Wo)

gate = matvec(norm(x), Wgate)
up = matvec(norm(x), Wup)
x = x + matvec(silu(gate) * up, Wdown)
```

这里的每个 **matvec** 都可以用本节的方法拆开: 输入向量地址、权重行地址、accumulator 依赖、输出 store、量化 scale 和 oracle。不同的是, 真实模型把尺寸和状态放大了。Q/K/V 的输出还要 reshape 成 **[heads, head_dim]**; K/V 不是普通临时输出, 而要写入第 **layer** 层、第 **position** 个 token 的 KV cache; attention 会在后续 token 中反复读取这些 cache 行。

从 matvec 切片接回 decoder 层状态



图示: 本图要证明的命题是: 最小 matvec 账本不是玩具结论, 它会直接进入每一层 Transformer decode 的投影、KV cache 和 MLP 路径。

prefill 阶段则把 **x** 从一条向量扩成多条 token 行。此时 **matvec** 账本会变成 GEMM/GEMV 问题: prompt token 越多, 权重复用越明显, packing、tile、SIMD 和 GPU kernel 的价值越容易体现; decode 阶段每次只处理一个或少数 token, 权重流和 KV cache 读写又会成为另一种瓶颈。因此性能报告必须把 prefill tokens/s、decode tokens/s 和首 token 延迟分开写。

硬验收

读完本节, 应该能对最小切片的一层 matvec, 以及 Transformer 中的 Q/K/V 或 MLP projection, 写出下面几件事:

- 用 shape 和 stride 推导 **input[k]**、**weight[out,k]**、**bias[out]**、**hidden[out]** 和 KV cache 写入位置的地址公式。
- 说明行主序连续权重为什么适合内层 **k** 循环, 转置或指针行为什么会改变地址流。

- 把 float32 matvec 写成 load、mul、add、branch、store 的机器账本。
- 解释 activation 融合减少了哪条中间内存流，也说明它怎样改变调试和 oracle 边界。
- 说明 int8 路径为什么需要 int32 accumulator、requantize 和误差合同。
- 区分 prefill 中的多 token GEMM 倾向和 decode 中的单 token GEMV 倾向。

这些输出看起来比“会调用矩阵乘 API”低级，但它们是后续 GEMM packing、SIMD、量化校准、算子融合、KV cache 规划、CPU/GPU 后端和推理图内存规划的共同地基。

1.3 从矩阵向量乘走向 GEMM、packing 与 SIMD

为什么不能直接从 matvec 跳到最快 GEMM

上一章把一层矩阵向量乘拆成了 shape、stride、地址公式、基本块、地址流和机器账本，并把它接回了 decoder-only Transformer 的 Q/K/V projection、KV cache 写入和 MLP 子层。那条路径故意很小，因为它让每个性能判断都有落点：input[i] 从哪里读，row[i] 是否连续，acc 为什么形成归约依赖，activation 和写回发生在什么边界。

本节继续沿同一条路径推进，但不直接背某个 BLAS 接口。真正的高性能矩阵乘不是一句“调用 GEMM”就结束。它要解决三个逐层出现的问题：

Listing 1.28 从 matvec 到 GEMM 的三个新问题

```
matvec:
  one input vector times many weight rows
  output is one hidden vector

prefill or batched inference:
  many token rows share the same weight matrix
  output becomes a token-by-channel matrix

GEMM kernel:
  compute a tile C[m,n] from panels A[m,k] and B[k,n]
  choose loop order, packing layout, SIMD lanes and accumulators
```

第一个问题是复用。matvec 中，同一个 input 会被每个输出行复用；prefill 中多个 token 行共享同一份权重，batch 服务时多个请求也共享权重。第二个问题是布局。数学上都是矩阵乘，机器上却要决定哪一块连续、哪一块预打包、哪一块留在 cache 或显存。第三个问题是寄存器和 SIMD。向量指令一次处理多个 lane，但只有当地址流、对齐、数据类型和累加器安排都配合时，它才会变成吞吐提升，而不是更复杂的写法。

核心思想

GEMM 优化的核心不是“矩阵乘很快”，而是把重复使用的数据切成能留在 cache 和寄存器里的小块，并让内层循环产生稳定、连续、可向量化的地址流。

把 token 维度加回模型

上一章把 token 行固定为 1。若 prefill 一次处理 T 个 prompt token，或者服务端把 B 个请求合批，线性层语义可以写成：

Listing 1.29 多 token 版线性层 reference 语义

```
for t in 0..T:
  for out in 0..N:
    acc = bias[out]
    for k in 0..K:
      acc += input[t, k] * weight[out, k]
    output[t, out] = activation(acc)
```

这仍然是 reference，不是最终内核。它暴露了两个复用方向。固定 **t**、遍历 **out** 时，一个 token hidden 行会被多次读取；固定 **out**、遍历 **t** 时，一个 weight 行会被多次读取。若 decode 每次只有一个 token，权重复用不明显，GEMV 或小 batch GEMM 可能是主形态；若 prefill token 多，权重矩阵成为多个 token 共享的热数据，GEMM 风格就有意义。

常见的矩阵表示可以写成：

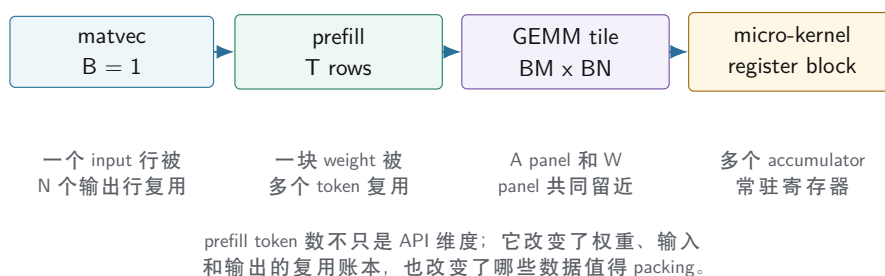
Listing 1.30 第一层的矩阵形状

```
A = input    shape [T, K]
W = weight   shape [N, K] row-major by output channel
C = output   shape [T, N]

semantic:
  C[t, out] = dot(A[t, :], W[out, :]) + bias[out]
```

如果把右侧矩阵看成 $W_T[K, N]$ ，语义就接近 $C=A*W_T$ 。这一步很容易在纸上完成，却不能在机器上忽略布局： $W[out, k]$ 行主序连续适合 matvec 的内层 **k**；但 GEMM 常希望在一个小 tile 里同时取多个 **out**，也就是读取 $W[out0, k]$ 、 $W[out1, k]$ 、 $W[out2, k]$ 。这些地址在原始行主序中相隔 **K** 个元素，不一定适合内层 SIMD。

prefill 把复用方向从一条行扩成一个 tile



图示：本图要证明的命题是：从 decode matvec 到 prefill GEMM 的关键变化是复用单位从一条向量扩大为一个可驻留的 tile。

三层循环不是任意交换

矩阵乘 reference 通常有三层循环。初学者容易把循环顺序理解成纯数学等价：只要结果相同，怎么换都可以。性能分析要更具体：谁在内层，谁就决定高频地址流；谁跨 tile 复用，谁就决定 cache 工作集；谁留在寄存器，谁就决定 accumulator 数量。

Listing 1.31 三种循环顺序的地址含义

```
t-out-k:
  for each token t:
    for each output out:
      scan A[t, k] and W[out, k]

t-k-out:
  for each token t:
    for each k:
      reuse A[t, k] across many out
      touch many W[out, k] and C[t, out]
```

```

tile-t-out-k:
  for blocks of tokens and out:
    keep C tile accumulators
    stream K panels of A and packed W

```

第一种像上一章的 matvec：每个输出行顺序扫描权重，简单、可验证，但每次只产生一个输出。第二种把 $A[b, k]$ 复用到多个 out ，但如果 $W[out, k]$ 在原始行主序中跨步很大，内层访问会变差。第三种把问题切成 tile：一次只处理 BM 个样本和 BN 个输出通道，用多个 accumulator 保存一个 C 小块，沿 K 扫描输入和权重面板。

这就是为什么 GEMM 教材经常出现 BM 、 BN 、 BK 。它们不是玄学参数，而是三个工作集边界：

- BM ：一次处理多少输入行，影响输入 panel 和输出 tile。
- BN ：一次处理多少输出列，影响 accumulator 数量和权重 panel。
- BK ：一次沿归约维度推进多少元素，影响 packing 块和 cache 复用。

Packing 的本质：把坏地址流变成后端友好的地址流

packing（预打包）指把原始张量中的一块数据复制成内核更喜欢的布局。它不是为了让模型文件好看，而是为了让热循环简单。以权重为例，原始行主序 $W[out, k]$ 对单个输出行很好；但一个微内核若想同时算 BN 个输出，内层可能需要在同一个 k 下读多条输出行的权重。原始布局给出的地址是：

Listing 1.32 原始权重在同一个 k 下访问多个 out

```

W[out0, k] = base + (out0 * K + k)
W[out1, k] = base + (out1 * K + k)
W[out2, k] = base + (out2 * K + k)

distance between neighboring out values: K elements

```

若把一个 $BN \times BK$ 的权重块打包成 k -major，内核看到的就可以是：

Listing 1.33 packed weight panel 的一种形状

```

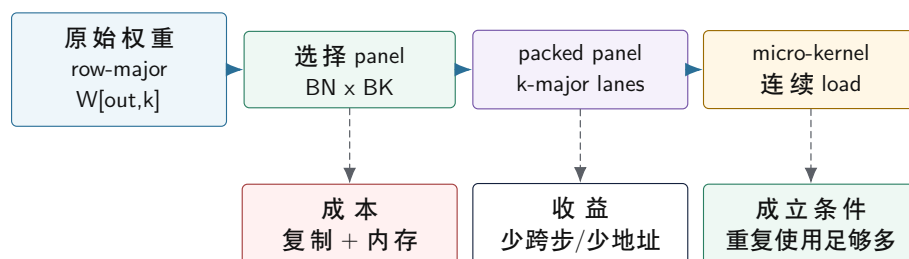
for k in 0..BK:
  packed[k, 0] = W[out0, k]
  packed[k, 1] = W[out1, k]
  packed[k, 2] = W[out2, k]
  ...

inner kernel reads packed[k, 0..BN)

```

这样做付出了一次复制、额外内存和格式管理成本，换来内层循环的连续读取、更少地址计算和更稳定的 SIMD lane。是否值得 packing，取决于这块权重会被复用多少次。7B 模型权重会在每个 token、每一层反复使用，所以在加载阶段或首次运行阶段打包常常合理；但不同后端喜欢的 packed layout 不一定相同。CPU 后端关心 cache panel、SIMD lane 和预取；GPU 后端关心 coalesced load、shared memory tile、tensor core 数据格式和显存对齐。runtime 因此不能把唯一一种 CPU packed layout 写死成模型格式。

packing 把原始布局改造成内核布局



packing 不是数学变换，而是把原始地址图复制成微内核更容易连续读取的地址图。

图示：本图要证明的命题是：packing 的价值来自热循环地址流改善，必须和复用次数一起判断。

SIMD 需要多条独立账本配合

SIMD（单指令多数据）可以理解为一个指令同时处理多个 lane。例如一个 float32 向量寄存器可以装多个相邻浮点数，FMA 指令可以对多个 lane 同时做乘加。但 SIMD 对地址流有要求：连续 load 最容易，固定可预测步长次之，随机 gather 或指针追踪通常更难。

在 GEMM 微内核里，SIMD 常见两种方向。第一种是沿 **K** 维度装载多个相邻元素，做向量乘加，最后水平归约到一个输出。第二种是同时维护多个输出通道，让一个输入标量广播到多个 lane，与 packed 权重 lane 相乘，更新多个 accumulator。哪种方向更好，取决于 ISA、寄存器数量、数据类型、矩阵尺寸和权重布局。

Listing 1.34 一个抽象 micro-kernel 账本

```

for k in 0..BK:
    a0 = load A[b0, k]
    a1 = load A[b1, k]
    wv = load packed_W[k, out0..outN]
    acc0[0..N] += broadcast(a0) * wv
    acc1[0..N] += broadcast(a1) * wv

store C[b0, out0..outN]
store C[b1, out0..outN]
  
```

这段账本看起来已经像高性能内核，但仍然不能脱离前两册的方法。要问：**packed_W** 是否真的连续；**A[b, k]** 是否能留在 cache；accumulator 数量是否超过寄存器预算；输出 store 是否连续；尾部 **N** 不能整除 SIMD 宽度时怎样处理；反汇编是否真的生成了向量 load、广播和 FMA，而不是退回标量循环。

从标量 reference 到 micro-kernel 的切分路线

真正写后端时，不应该一上来就写最大化的 SIMD intrinsic。更稳的路线是把同一个 linear 语义切成四个阶段，每个阶段都能被 oracle 验证。

Listing 1.35 linear 内核的四阶段演进

```

stage 0: scalar reference
  one output channel at a time
  easiest to audit, slowest

stage 1: scalar tiled
  compute a small C[BM, BN] tile
  
```



```

    prove tile boundary and tail are correct

stage 2: packed weight
    keep scalar arithmetic
    prove packed descriptor maps to the same weights

stage 3: SIMD micro-kernel
    replace inner loop with vector loads/FMA/dot
    keep the same descriptor and oracle cases

```

这个顺序有一个好处：当 stage 3 出错时，问题范围很小。若 stage 2 已经证明 packed descriptor 的字节解释正确，stage 3 的错误就更可能来自 SIMD lane 顺序、accumulator、尾部 mask 或 dtype 转换，而不是模型导入或 shape verifier。

以 **BM=2**、**BN=4** 的小 tile 为例，标量 tiled 版本可以先写成：

Listing 1.36 标量 tiled GEMM 的参考形态

```

for b in 0..B step 2:
  for out in 0..0 step 4:
    acc[2][4] = 0
    for k in 0..K:
      for bi in 0..1:
        for oi in 0..3:
          acc[bi][oi] += A[b + bi, k] * W[out + oi, k]
        store acc into C[b..b+1, out..out+3]

```

这段代码不快，但它已经有了微内核的形状：`acc[2][4]` 是寄存器账本的影子，`b/out/k` 是 tile 循环骨架，尾块可以独立测。后续把 `W[out+oi,k]` 换成 packed 地址，把 `oi` 方向换成 SIMD lane，就不会改变上层语义。

一个 micro-kernel 的 descriptor 至少要把这些事实写清：

Listing 1.37 micro-kernel descriptor 的最小字段

```

MicroKernelDesc:
  bm
  bn
  bk
  input_dtype
  weight_dtype
  accumulator_dtype
  output_dtype
  weight_packed_layout
  required_alignment
  tail_policy
  supported_isa

```

这让内核选择变成编译器问题，而不是一堆 `ifcpuhasavx` 散落在调用点。typed graph 给出 shape 和 dtype，backend capability 给出 ISA 和支持格式，lowering 选择 descriptor，oracle 用同一 descriptor 解释 packed bytes。

验收与评分标准

一个 linear micro-kernel 进入默认路径前，至少要通过四组测试：整齐尺寸、**K** 维尾部、**0** 维尾部、非默认 stride。若是量化权重，还要额外覆盖 group 边界和 scale 尾组。

尾块、对齐和小尺寸陷阱

工业 GEMM 内核会花大量代码处理边界。教材如果只展示整齐尺寸，会让读者误以为优化就是把主循环写出来。实际推理中，隐藏层大小、输出类别数、batch size 和量化 block 不一定刚好整除 tile。

尾块不是异常路径。 若 **N** 不能整除 **BN**，最后一个输出 tile 是窄 tile；若 **B** 不能整除 **BM**，最后一个 batch tile 是矮 tile；若 **K** 不能整除向量宽度，归约尾部需要标量补齐、mask load 或专门小内核。尾块很常见，不能只靠“输入尺寸都规整”逃避。

对齐是性能条件，不是语义条件。 非对齐 load 通常仍然能得到正确结果，但可能跨 cache line 或页，成本更高。packing 阶段可以选择对齐缓冲，减少内核里处理复杂地址的机会。报告中要写清：对齐要求是硬性合同，还是只是 fast path 条件；不满足时是复制、降级还是报错。

小矩阵可能被 packing 成本吞掉。 一个只有几行几列的小层，打包和调度的固定成本可能大于内核收益。第三册后续讲推理图和运行时时会反复遇到这个问题：大算子适合重内核，小算子更需要融合、批处理或减少中间内存流。

prefill、decode 与后端合同

把 GEMM 放回 7B 推理引擎后，性能不能只写一个“矩阵乘耗时”。同一个权重矩阵在 prefill 和 decode 中的形状不同：

Listing 1.38 同一层在 prefill 和 decode 中的矩阵形态

```
prefill:
  A = tokens x hidden
  W = out x hidden
  C = tokens x out
  weight reuse across many token rows

decode:
  A = 1 x hidden
  W = out x hidden
  C = 1 x out
  weight stream dominates, KV cache also grows
```

CPU 后端通常先把 decode 的单 token latency 做稳，再逐步优化 prefill 吞吐：标量 reference、普通线程切分、SIMD micro-kernel、packed weight、NUMA-aware weight placement、算子融合、量化内核。GPU 后端则更依赖批量和 tile：prefill 往往容易吃到大 GEMM 吞吐，decode 若 batch 太小就可能被 kernel launch、内存访问和 KV cache 读写限制。真实 runtime 要把这些差异放到 backend contract，而不是让上层 graph 直接知道每个后端的细节。

Listing 1.39 GEMM 后端合同的最小字段

```
op: linear
input: dtype, shape, layout, device
weight: dtype, shape, packed_layout, scale_policy
output: dtype, shape, layout, device
mode: prefill or decode
backend: cpu_scalar, cpu_simd, cpu_threaded, gpu
evidence: latency, tokens_per_second, error_report
```

这份合同让第三册后续能一步一步把性能提高，而不是一次跳到“对标成熟框架”。每次优化都必须说明它改变了哪一栏：是 dtype 从 fp32 变成 int8，还是 packed layout 变了；是 mode 只覆盖 prefill，还是 decode 也受益；是 CPU 后端新增 SIMD path，还是 GPU 后端新增 kernel；是降低了 latency，还是提高了 batch 吞吐。

用 $T=2, K=8, N=6$ 手算一次 GEMM

固定一个小矩阵，避免 GEMM 只停在符号层：

$$A[T, K] = A[2, 8], \quad W[N, K] = W[6, 8], \quad C[T, N] = C[2, 6]$$

这里 T 是 token 或 batch 行数， K 是 hidden/input 维度， N 是输出通道数。采用推理里常见的权重表示 $W[\text{out}, k]$ ，计算公式是：

$$C[t, n] = \sum_{k=0}^{K-1} A[t, k] \times W[n, k]$$

例如：

$$C[1, 4] = A[1, 0]W[4, 0] + A[1, 1]W[4, 1] + \cdots + A[1, 7]W[4, 7]$$

若原始 row-major 存储，地址 offset 为：

$$\text{offset}_A(t, k) = t \times K + k$$

$$\text{offset}_W(n, k) = n \times K + k$$

$$\text{offset}_C(t, n) = t \times N + n$$

计算 $C[0, 0..5]$ 时， $A[0, 0..7]$ 会被每个输出通道重复使用 6 次，权重则按行读取 $W[0, 0..7]$ 、 $W[1, 0..7]$ ，直到 $W[5, 0..7]$ 。计算 $C[1, 0..5]$ 时，权重完全相同，又被复用一次。这就是 prefill 或 batch GEMM 的核心机会： T 越大，权重复用越多。

原始 layout 到 packed layout 的地址变化

假设 micro-kernel 想一次算 $BN=4$ 个输出通道。对某个 k ，它想连续拿到：

$$W[0, k], W[1, k], W[2, k], W[3, k]$$

但原始 row-major $W[n, k]$ 中，相邻输出通道的地址差是 $K=8$ 个元素：

Listing 1.40 原始 W 在同一个 k 下跨输出通道

```
W[0,k] offset = 0 * 8 + k
W[1,k] offset = 1 * 8 + k
W[2,k] offset = 2 * 8 + k
W[3,k] offset = 3 * 8 + k

stride between W[n,k] and W[n+1,k] = 8 elements
```

packing 可以把 $\text{out}=0..3, k=0..7$ 的 panel 改成 $k\text{-major}$ ：

Listing 1.41 $BN=4$ 的 packed panel

```
packed offset for panel(out_base=0):
k=0: W[0,0], W[1,0], W[2,0], W[3,0]
k=1: W[0,1], W[1,1], W[2,1], W[3,1]
```

```
...
k=7: W[0,7], W[1,7], W[2,7], W[3,7]
```

此时 micro-kernel 每个 k 读取 `packed[k,0..3]` 是连续的。若 SIMD lane 正好能容纳 4 个 int8、int16 或 float 元素的一组处理，地址流就变简单了。第二个 panel `out=4..5` 是尾部 panel，只有 2 个输出通道；它可以走 mask/tail kernel，也可以在 packing 时补零，但补零必须让 oracle 知道真实 $N=6$ ，不能把补出来的输出当成有效通道。

decode GEMV 的字节账本

decode 时常见形状是：

$$A[1, K] \times W[N, K] \rightarrow C[1, N]$$

令 $K=4096$ 、 $N=4096$ 、fp16 权重和 fp16 activation。一次 GEMV 的主要账本是：

$$weight\ bytes = N \times K \times 2 = 4096 \times 4096 \times 2 \approx 32\ MiB$$

$$activation\ bytes = K \times 2 = 8\ KiB$$

$$output\ bytes = N \times 2 = 8\ KiB$$

每个输出做 K 次乘加，共：

$$flops = 2 \times N \times K \approx 33.6\ MFLOP$$

算术强度粗略为：

$$AI \approx \frac{33.6\ MFLOP}{32\ MiB} \approx 1.0\ FLOP/byte$$

这说明 decode GEMV 很容易被权重读取带宽限制。activation 很小，可以留在 cache 或寄存器附近；输出也小；真正大的是每个 token 都要扫一遍权重。int8 权重把权重字节减半到约 **16MiB**，int4 再减半到约 **8MiB**，但会引入 scale、unpack 和 dequant 成本。

prefill GEMM 为什么更容易吃满算力

prefill 若一次处理 $T=128$ 个 token，形状变成：

$$A[128, 4096] \times W[4096, 4096] \rightarrow C[128, 4096]$$

权重仍然约 **32MiB**，但被 128 行输入复用。FLOPs 变为：

$$2 \times 128 \times 4096 \times 4096 \approx 4.29\ GFLOP$$

只按权重字节估算，算术强度从约 **1FLOP/byte** 提升到约 **128FLOP/byte**。实际还要读 activation、写 output、处理 cache blocking 和 attention 中间量，但方向很清楚：prefill 更像大 GEMM，适合 tile、packing、线程并行、GPU tensor core；decode 单 token 更像 GEMV 或小 GEMM，权重流和调度固定成本更突出。

阶段	典型形状	权重复用	常见瓶颈
prefill	$T \times K$ by $N \times K$	高，复用到多 token	compute、tile、activation 峰值
decode	$1 \times K$ by $N \times K$	低，单 token 扫权重	memory bandwidth、launch、KV 读

register tiling 不是玄学

以 **BM=2, BN=4** 为例, micro-kernel 需要维护 8 个 accumulator:

$$acc[0, 0], acc[0, 1], acc[0, 2], acc[0, 3], acc[1, 0], acc[1, 1], acc[1, 2], acc[1, 3]$$

每个 **k**, 读取两个 activation:

$$a_0 = A[t + 0, k], \quad a_1 = A[t + 1, k]$$

再读取四个 packed 权重:

$$w_0, w_1, w_2, w_3 = W[out + 0..3, k]$$

然后执行 8 次乘加:

$$acc[0, i] += a_0 \times w_i, \quad acc[1, i] += a_1 \times w_i$$

这些 accumulator 最好留在寄存器里。如果 **BM** 和 **BN** 取得太大, accumulator、输入临时值、权重向量、指针和 loop counter 会超过寄存器预算, 编译器或手写汇编就会 spill 到栈, 性能反而下降。因此 tile 大小要同时满足 cache 工作集和寄存器账本, 不是越大越好。

反汇编要验证什么

GEMM/GEMV 优化报告里的反汇编不是装饰。至少要验证:

- 内层循环是否真的使用目标 SIMD/dot 指令, 而不是标量乘加。
- packed 权重是否连续 load, 是否出现意外 gather 或复杂地址。
- accumulator 是否留在寄存器, 是否有明显 spill/load/store。
- 尾部路径是否分离, 主循环是否被尾部判断污染。
- int8/int4 路径是否把 unpack、scale 读取和 dot-product 放在合理位置。

Listing 1.42 micro-kernel 报告必须贴出的材料

```
source:
  scalar reference and optimized kernel

descriptor:
  BM, BN, BK, dtype, packed layout, tail policy

assembly:
  inner loop disassembly
  load/FMA/dot/reduce instructions
  tail path disassembly

perf:
  elapsed ns, bytes, FLOPs, GB/s, GFLOP/s
  cycles, instructions, IPC if available
  cache/TLB counters if available
```

错误案例: packing 正确但解释错误

packing 最危险的 bug 是“字节确实被复制了, 但 kernel 和 oracle 对它的解释不同”。例如 packing 写入顺序是:

```
packed[k, lane] = W[out_base + lane, k]
```

但 kernel 读取时以为：

```
packed[lane, k] = W[out_base + lane, k]
```

对某些小矩阵，若数据恰好对称或测试只用了全 1 权重，这个错误可能不暴露。因此 GEMM/GEMV oracle 必须使用非对称、非重复、带尾部的测试数据。例如令：

$$W[n, k] = 100n + k$$

这样只要 n 和 k 被交换，输出会立刻错。测试数据要故意让地址维度可辨认，而不是只用随机数或全零全一。

把现有 lab 放进证据链

当前 [books/cpu-volume-3/labs/linux_cpu_inference/](#) 里的 `linear_i8` 是一个故意朴素的内核。它按输出行遍历 int8 权重，把每个权重乘以 scale 后与 float 输入相乘，并把结果累加到 float：

Listing 1.43 当前 lab 的线性层机器账本

```
for out in output_size:
    acc = bias[out]
    row_base = out * input_size
    for in in input_size:
        weight_i8 = weights[row_base + in]
        acc += float(weight_i8) * layer.scale * input[in]
    output[out] = acc
```

这段代码适合作为 reference 风格的本地推理入口，因为它的语义清楚，测试能手工验证。但它只是第一阶段切片，不是最终引擎。它没有 token tile，没有 packed weight panel，没有 int32 accumulator，没有 requantize，也没有 CPU/GPU 后端选择。因此报告必须明确它的量化范围：当前 lab 是“int8 权重 + float 输入 + float 累加”的简化路径；真实 7B 路径还要处理 per-channel/per-group quantization、packed weights、KV cache 量化和后端共享的误差报告。

硬验收

读完本节，应该能把“GEMM 很快”拆成下面几条可检查事实：

- 说明 batch 怎样改变 input、weight 和 output 的复用账本。
- 写出 BM、BN、BK 分别约束哪一块工作集。
- 解释 packing 改善的是哪条地址流，以及它为什么需要足够复用才能抵消复制成本。
- 区分 SIMD lane、accumulator 寄存器、连续 load、广播和尾块处理。
- 说明同一个 projection 在 prefill 中为什么更像 GEMM，在 decode 中为什么更像 GEMV 或小 GEMM。
- 能把当前 lab 的 `linear_i8` 定位成清晰 reference/教学内核，而不是最终 7B 推理引擎的 GEMM 微内核。

同一个 `linear_i8` 切片进入 7B 模型后，必须继续回答：权重为什么能用 int8 或 int4 保存，scale 怎样进入计算，accumulator 为什么要更宽，重新量化到底在做什么，KV cache 能不能量化，oracle 又应该怎样判定误差。

1.4 量化系统总览：从实数模型到低比特推理合同

量化为什么必须单独成章

量化很容易被误解成“把 float 改成 int8”。如果只是换类型名，确实几行代码就能写完；但真正的推理引擎量化是一套跨越模型文件、编译前端、IR、后端 kernel、runtime、oracle 和 profiler

的系统合同。它决定权重能不能装进内存，决定 decode 阶段要读多少 KV cache，决定 CPU/GPU kernel 怎样读取 scale，决定 logits 是否还能被 sampler 正确使用。

一个最小反例可以说明它为什么不能混在普通算子章节里。假设某个 linear 权重被保存成 int4，每 64 个权重共享一个 scale。若后端 packing 时只重排了 4-bit 权重，却没有按同样分组重排 scale，kernel 仍然会跑，也可能输出一堆“看起来像正常浮点数”的 logits。但这些 logits 已经被错误 scale 污染。这个错误不是普通矩阵乘错误，而是量化格式、metadata、packing 和 kernel 合同同时破裂。

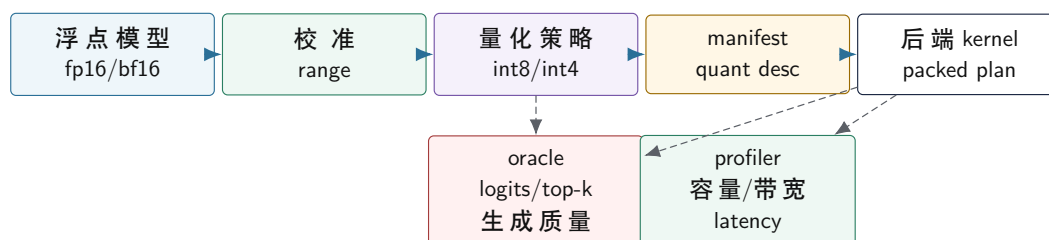
核心思想

量化是一条系统级合同：实数范围怎样选择，整数码点怎样解释，scale 和 zero point 怎样存储，accumulator 在哪里扩大，requantize 怎样舍入，packed layout 怎样保持 metadata 对齐，oracle 怎样判断误差是否可接受。

第三册里的量化流水线

本册把量化看成一条完整流水线，而不是一个局部函数。

量化从校准到后端执行的系统流水线



图示：本图要证明的命题是：量化不是单个 kernel，而是校准、策略、manifest、后端和验证共同组成的流水线。

校准阶段。 校准决定实数范围。权重可以离线扫描；activation 需要代表性输入；KV cache 是动态状态，通常要在真实 prompt/context 分布上观察。校准数据不代表真实分布时，scale 会偏，饱和会增多，生成质量会漂移。

策略阶段。 策略决定使用 int8、int4、per-tensor、per-channel、per-group、对称或非对称量化。策略不是越激进越好。int4 权重能省内存，但 kernel 要处理解包、scale 读取和更复杂误差；KV cache 量化能省长上下文内存，但 decode attention 的读路径会变复杂。

manifest 阶段。 量化信息必须进入 manifest。后端不能靠张量名字猜格式，必须读 quant descriptor：bit width、signedness、scale dtype、zero point policy、group size、axis、packed byte order、tail policy、checksum 和支持的 backend。

后端阶段。 后端负责把量化格式变成机器执行。CPU 可能选择 dequant-on-the-fly、int8 dot-product 或 int4 解包加 FMA；GPU 可能使用 vendor kernel、tensor core 支持格式或自定义 kernel。不同后端可以使用不同 packed layout，但必须共享同一逻辑量化合同。

验证阶段。 量化必须同时通过 oracle 和 profiler。oracle 证明误差在合同内；profiler 证明容量、带宽或 latency 的收益真实存在。只省文件大小但 decode 变慢，或 tokens/s 变快但 logits 漂移不可接受，都不是合格优化。

校准：scale 不是随便选出来的

线性量化的核心参数是 scale 和 zero point。scale 由范围决定，范围由校准决定。最简单的做法是扫描张量最小值和最大值：

Listing 1.44 最小最大校准的基本形状

```

min_value = min(x[i])
max_value = max(x[i])
  
```

```
scale = (max_value - min_value) / (qmax - qmin)
zero_point = round(qmin - min_value / scale)
```

这个算法容易理解，但不总是最好。若只有少数极端值，min/max 会把 scale 拉大，使大部分普通值的量化间隔变粗；若为了普通值缩小范围，极端值会被 clamp。两者都是误差，只是形态不同。量化校准的本质就是在舍入误差和饱和误差之间取舍。

校准策略	优点	风险
min/max	简单，可复现，适合第一版	极端值会拉大 scale
percentile	减少极端值影响	被截断值可能集中在关键通道
MSE/KL 搜索	更贴近分布误差	复杂，依赖校准集质量
per-channel 校准	减少通道间范围差异	元数据和 requantize 更复杂
动态校准	适合 activation 或 KV 状态	运行时成本和稳定性要验证

校准集必须写进报告。对图像模型，校准集可能是若干图片；对语言模型，校准集应该包含短 prompt、长 prompt、代码、中文、英文、对话、多轮上下文等场景。若只用很短英文 prompt 校准 KV cache，长中文对话的 activation 分布可能完全不同。校准不是离线脚本的附属品，而是模型合同的一部分。

线性量化的精确定义

先把最常见的线性量化写清楚。给定实数 x 、scale s 和 zero point z ，量化与反量化通常可以写成：

$$q = \text{clamp}(\text{round}(x/s) + z, q_{\min}, q_{\max})$$

$$\tilde{x} = s(q - z)$$

这里 q 是整数码点， $\tilde{x}$ 是反量化后的近似实数。若使用对称量化，通常 $z=0$ ，码点范围围绕 0；若使用非对称量化， z 让非零实数范围能映射到无符号整数区间。对称量化简单，适合很多权重；非对称量化能更好表达偏移分布，但后端计算和 zero point 修正更复杂。

形态	数学合同	工程影响
对称	$z=0$ ，正负范围近似对称	kernel 简洁，适合权重
非对称	z 可非零	activation 适配性强，但修正项更多
per-tensor	整个张量共用 scale	metadata 少，误差可能大
per-channel	每个输出通道一组 scale	精度好，scale 读流增加
per-group	每组权重一组 scale	int4 常见折中，packing 更复杂

per-group int4 的具体含义必须避免含糊。若 `group_size=64`，通常表示连续 64 个权重共享一个 scale。这个“连续”必须绑定到逻辑轴：是按输入通道连续，还是按 packed 后的物理顺序连续？正确做法是 descriptor 同时记录逻辑分组轴和 packed layout，并由 verifier 检查 scale 数量。

Listing 1.45 group scale 数量检查

```
groups_per_output =
    ceil(input_channels / group_size)

expected_scale_count =
    output_channels * groups_per_output
```

若 scale 少一个，kernel 可能越界；若 scale 多一个，说明 manifest、converter 或 packing 至少有一个环节对分组理解不同。不能让 kernel 用“能读多少读多少”的方式容错，因为这会把资产错误变成数值漂移。

校准算法怎样落到 C++20

校准不是只能依赖外部脚本。项目里至少应有可复现的校准结果读取和验证逻辑。对权重量化，离线 converter 可以扫描权重并写出 scale；对 activation 或 KV cache，运行时或校准工具需要收集统计量。

Listing 1.46 校准统计的最小结构

```
CalibrationStats:
  tensor_name
  axis
  granularity
  sample_count
  min_values
  max_values
  histogram_optional
  saturation_rate
  source_prompt_set_id
```

min/max 校准只需要最小值和最大值；percentile、MSE 或 KL 搜索通常需要直方图。直方图不是为了好看，而是为了回答“截断哪些值会让总体误差更小”。一个保守 MSE 搜索可以这样理解：枚举候选范围 $[-a, a]$ ，把范围外值 clamp，把范围内值量化再反量化，选择平均平方误差最小的 a 。

Listing 1.47 MSE calibration 的朴素形态

```
best_error = infinity
for candidate_absmax in candidates:
  scale = candidate_absmax / qmax
  error = 0
  for x in calibration_values:
    q = clamp(round(x / scale), qmin, qmax)
    x2 = q * scale
    error += (x - x2) * (x - x2)
  keep candidate with minimal error
```

这个朴素算法不一定是最高效实现，但它定义了语义。工程实现可以用直方图加速，而 oracle 应该能用小样本直接复现它。不要让校准工具和 runtime verifier 使用两套互相不透明的规则。

权重量化、activation 量化和 KV 量化不是一回事

三类量化对象生命周期不同，因此工程合同也不同。

对象	生命周期	关键风险
权重	离线静态，加载后常驻	packed layout、scale 对齐、后端格式支持
activation KV cache	每次推理临时产生 跨 token 状态，随上下文 增长	动态范围随输入变化，requantize 成本 误差会被未来 token 反复读取，meta- data 进入 attention 热路径

权重量化通常最先做，因为它直接降低模型容量和权重读流。activation 量化更难，因为每层中间值分布随 prompt 改变，且很多算子之间需要反量化/重量化边界。KV cache 量化介于两者之间：它不是静态权重，但每个 token 写入后会长期保存；它的容量收益很大，但误差会在长上下文里持续参与 attention。

保守推进顺序是：先做权重-only 量化，让 activation 仍用 fp16/fp32 近似计算；然后做局部 activation 量化实验；最后再做 KV cache 量化。这样每次只改变一个主要误差来源，oracle 更容易定位。

量化收益要同时算字节和指令

低比特并不自动更快。以 int4 weight-only GEMV 为例，热路径少读了权重字节，但增加了三个成本：解包 nibble、读取 scale、把整数码点乘回实数或进入整数 dot-product 路径。若当前机器瓶颈是内存带宽，少读字节可能收益明显；若瓶颈是解包指令或 scale 读流，int4 可能不如 int8。

Listing 1.48 weight-only 量化的粗略收益账

```
fp16 weight bytes:
  params * 2

int8 weight bytes:
  params * 1 + scale_bytes

int4 weight bytes:
  params / 2 + scale_bytes + packing_padding

extra work:
  unpack integer code
  load scale metadata
  dequantize or use integer dot path
```

对于 7B 参数量级，fp16 权重大约 14GB；int8 约 7GB 加 metadata；int4 约 3.5GB 加 metadata。这个容量收益很直观。但 decode tokens/s 是否按比例提升，要看 effective bandwidth、cache、TLB、SIMD 指令、group size 和线程调度。量化报告必须把容量收益和 latency 收益分开写。

业务开发中的量化边界

用户真正关心的是“学完能不能做业务开发”。量化在业务系统里不能只说模型更小，还要说明什么时候可以用、什么时候不该用。

- 搜索补全、草稿生成、离线摘要等任务，可能更能接受轻微 logits 漂移。
- 法律、医疗、财务、代码自动修改等高风险任务，不能只看生成文本是否通顺，要固定评测集和人工验收。
- RAG 场景要特别看引用和数字是否稳定，因为量化误差可能改变排序靠前的候选 token。
- 多轮对话要看长上下文质量，KV cache 量化的误差不能只在短 prompt 上验收。
- 如果使用工具调用或 JSON 输出，要验证格式稳定性，不能只看自然语言回答。

工程上可以把量化 profile 做成可选择配置：**quality** 使用更保守的权重格式和 fp16 KV；**balanced** 使用 int4 权重和 fp16/int8 KV 实验；**memory_saver** 使用更激进 KV 量化和较短窗口。每个 profile 都要有 oracle、benchmark 和适用说明。这样业务方选择的是有证据的配置，而不是一句“开 int4”。

误差来源：不是所有误差都一样

量化误差至少有五类。

舍入误差。 实数值落在两个整数码点之间，需要按某种规则舍入。舍入规则必须固定，否则 CPU reference、SIMD kernel 和 GPU kernel 可能出现系统性差异。

饱和误差。 值超出可表示范围时被 clamp 到边界。少量饱和可能可接受；某层、某通道或某类 prompt 上大量饱和，通常意味着 scale 或校准策略有问题。

累加误差。 低精度乘法后通常在 int32、fp32 或 fp16/bf16 中累加。不同累加器宽度、不同 reduction 顺序、不同 FMA 形态都会改变误差传播。

元数据误差。 scale、zero point、group scale、tail padding 或 packed byte order 被错误解释，会产生结构性错误。这类错误比随机舍入更危险，因为输出可能稳定但错误。

状态误差。 KV cache 量化会把误差写入跨 token 状态。一次写入后，未来很多 decode step 会读取它。它不是一次性中间 activation 误差，而是会随上下文参与后续注意力。

因此，量化 oracle 不能只比较某个算子一次输出。它要能按层、按通道、按 token position、按 prompt 类别和按生成步数定位误差。

量化在 AI 编译器中的位置

在 AI 编译器里，量化至少影响四个阶段。

Listing 1.49 量化事实在编译流水线中的流动

```
model import:
  parse quant metadata
  build QuantDescriptor

verifier:
  check shape, bit width, group size, scale count
  reject unsupported quant formats

IR passes:
  insert dequant, requant, quantized matmul
  decide fusion and materialization boundaries

backend lowering:
  choose packed layout and kernel
  bind scale/zero point/group metadata
```

如果导入阶段没有把量化格式读清楚，后端会被迫猜；如果 verifier 不检查 scale 数量，kernel 可能越界读 metadata；如果 IR pass 不知道某条边是量化张量，就可能错误融合或错误复用 arena；如果 lowering 不知道 group size，就无法生成正确 packed descriptor。

常见误区

量化不是后端私有小技巧。只在 CPU kernel 里临时解释 int4 字节，而不让 manifest、verifier、IR 和 oracle 知道量化合同，会让错误绕过编译器所有安全边界。

量化 lowering 的常见形态

从 IR 到后端，量化通常有三种 lowering 形态。

dequantize then compute。 先把低比特权重或 activation 还原成 fp16/fp32 临时块，再调用普通 GEMM/GEMV。这条路径容易实现和验证，适合作 reference 或第一版 GPU fallback；缺点是会产生额外中间写回，可能抵消低比特节省。

dequantize on the fly。 kernel 读取低比特码点和 scale，在寄存器里反量化后立即参与 FMA。CPU int4 weight-only GEMV 常用这种思路。它减少中间写回，但 kernel 要处理解包、scale group、tail 和 SIMD lane 对齐。

integer dot-product。 若硬件支持 int8 dot-product 或低比特矩阵指令，可以让整数码点直接进入 dot，再用 scale 和 requantize 还原输出。它可能更快，但对 zero point、accumulator、溢出和 scale 合并要求更严格。

lowering	优点	风险
dequant + compute	简单，容易对 oracle	额外内存流量大
on-the-fly dequant	少写中间值，适合 weight-only	kernel 复杂，scale 热路径
integer dot	可利用专门指令	zero point、溢出、requantize 复杂

同一量化格式可以有多个 lowering plan。executable plan 必须记录实际选择的是哪一种，否则 profiler 和误差报告无法解释差异。

量化在 C++20 项目中的边界

C++20 实现中，量化不应该散落成若干裸字段。建议至少有一个稳定的 descriptor：

Listing 1.50 QuantDescriptor 的字段边界

```
QuantDescriptor:
storage_dtype: int8 | uint8 | int4 | nf4 | fp8
logical_dtype: approximate_f16 | approximate_f32
scale_dtype: fp16 | fp32
zero_point_policy: none | symmetric | asymmetric
granularity: tensor | channel | group
axis
group_size
scale_count
packed_order
tail_policy
calibration_id
```

这个 descriptor 应该跟 tensor descriptor 一起流动，而不是藏在后端对象里。model loader 创建它，verifier 检查它，compiler pass 查询它，backend packing 消费它，oracle 报告它。这样一旦出错，日志能写清楚“某个张量使用 int4、group size 64、scale dtype fp16、tail padding zero、CPU AVX2 packed layout”，而不是只报 `invalidquantizedtensor`。

硬验收

读完本节，应该能说明：

- 量化是跨模型文件、编译器、后端、runtime 和验证的系统合同。
- scale 来自校准，校准策略会在舍入误差和饱和误差之间取舍。
- 量化误差至少包含舍入、饱和、累加、元数据和状态误差。
- QuantDescriptor 必须进入 manifest、verifier、IR pass、backend lowering 和 oracle。
- 量化优化必须同时有正确性报告和性能报告。

这套系统合同最终要落到低比特权重和 KV cache 上：int4/group quant 怎样存储，scale 怎样跟 packed weight 对齐，KV cache 量化为什么是运行时状态问题，C++20 后端怎样把这些信息变成可测 plan。

1.5 int8 量化、累加器与重新量化

从当前 lab 的简化路径开始，但落点是 7B 权重

第三册的 lab 里已经有一个最小切片。模型文件 `tiny_mlp_i8.txt` 保存 int8 权重、float bias 和每层一个 scale。当前 `linear_i8` 的核心路径可以概括为：

Listing 1.51 当前 lab 的简化量化路径

```
weight_i8 = load int8 weight
```



```
weight_f32 = float(weight_i8) * layer.scale
acc_f32 += weight_f32 * input_f32
output_f32 = acc_f32 + bias
```

这条路径只量化了权重，输入和输出仍然是 float32，累加器也是 float32。它非常适合教学的第一步：模型文件已经变小，权重地址流已经是 int8，结果又能用普通浮点误差比较。但它不是完整 int8 推理，更不是 7B 大模型的最终量化方案。真实本地推理要同时面对权重容量、内存带宽、CPU/GPU kernel、activation 精度、KV cache 容量和生成质量。完整路径通常还会量化 activation，用 int32 做 dot-product 累加，再把 int32 累加结果重新映射到目标输出类型；更激进的权重量化还会进入 int4、group-wise scale 和 dequant-on-the-fly。

核心理想

量化不是把类型名从 float 改成 int8。它是一份数值合同和后端合同：实数怎样映射到整数，乘加在哪个宽度里累加，bias 属于哪个尺度，输出怎样重新量化，KV cache 是否压缩，CPU/GPU kernel 怎样读 scale，误差怎样被 oracle 接受。

为什么 7B 推理绕不开量化

先只看权重容量，不看临时 activation、KV cache、allocator overhead 和 packed weight。一个 7B 参数模型若按不同 dtype 保存，大致账本是：

Listing 1.52 7B 权重容量的粗略账本

```
fp32: 7e9 * 4 bytes  ~= 28 GB
fp16: 7e9 * 2 bytes  ~= 14 GB
int8: 7e9 * 1 byte   ~= 7 GB, plus scales
int4: 7e9 * 0.5 byte ~= 3.5 GB, plus group scales
```

这个账本解释了为什么本地推理不会只讨论 fp32。许多普通机器装不下 fp32 权重；即使装得下，权重流也会压垮内存带宽。fp16/bf16 是常见精度基线，int8 和 int4 是本地部署的重要路径。量化的目标不是让文件更小这么简单，而是让模型能被加载、能在 cache/内存/显存带宽下持续供数，并且让 CPU/GPU kernel 能把压缩格式转化成可计算的乘加流。

线性量化的合同

最常见的线性量化把实数 x 映射到整数 q ：

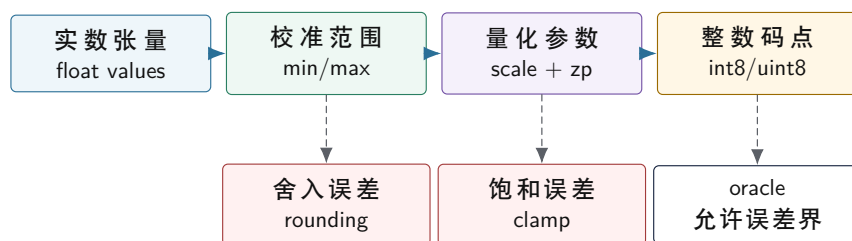
Listing 1.53 线性量化合同

```
q = clamp(round(x / scale) + zero_point, qmin, qmax)
x_approx = (q - zero_point) * scale
```

`scale` 是整数相邻刻度之间代表的实数间隔，`zero_point` 是实数零对应的整数位置。对称量化常让 `zero_point=0`，权重可以落在 `-128..127` 或 `-127..127` 附近；非对称量化允许零点偏移，适合 activation 范围不以零为中心的情况。

这个公式有三个必须写清的边界。第一，`round` 采用哪种舍入规则；不同实现可能选择 ties-to-even、away-from-zero 或固定点近似。第二，`clamp` 会吞掉超出范围的值；校准阶段若低估范围，极端值就会饱和。第三，反量化不是恢复原值，只是得到近似值；oracle 必须允许合同内误差。

实数、整数码点和误差合同



量化参数是数值合同的一部分；没有舍入、饱和和误差边界，int8 输出不能被可靠判定。

图示：本图要证明的命题是：量化误差来自范围选择、舍入和饱和，必须进入正确性合同。

为什么 accumulator 不能用 int8

两个 int8 数相乘，中间乘积已经不适合继续装在 int8。若 a 和 w 都接近 127，单次乘积约为 16129；再沿 K 累加，范围会继续扩大。因此 int8 dot-product 通常使用 int32 accumulator：

Listing 1.54 int8 dot-product 的整数账本

```

int32 acc = 0
for k in 0..K:
    int32 a = int32(input_i8[k]) - zero_input
    int32 w = int32(weight_i8[k]) - zero_weight
    acc += a * w
  
```

accumulator 的上界可以粗略估计：

Listing 1.55 accumulator 范围估计

```

worst_product ~= 127 * 127
worst_acc      ~= K * worst_product

if K = 4096:
    worst_acc is about 66 million
    int8 and int16 are not enough
  
```

真实模型不一定达到最坏情况，但内核不能依赖“通常不会”。溢出一旦发生，结果可能仍然像正常数字，不会崩溃，却会悄悄改变分类边界。使用 int32 accumulator 是把正确性边界先守住，再考虑指令吞吐和 requantize 成本。

Scale 怎样穿过乘加

若输入和权重都被量化，反量化语义是：

Listing 1.56 从量化值恢复 dot-product 语义

```

real_a[k] = (qa[k] - za) * scale_a
real_w[k] = (qw[k] - zw) * scale_w

real_acc = sum_k real_a[k] * real_w[k]
           = scale_a * scale_w *
             sum_k (qa[k] - za) * (qw[k] - zw)
  
```

这说明 int32 accumulator 保存的是无量纲整数和，真正的实数尺度在 `scale_a*scale_w` 上。bias 必须进入同一个尺度。常见做法是把 float bias 预先量化成 int32 bias：

Listing 1.57 bias 折叠到 int32 accumulator

```
bias_i32[out] = round(bias_f32[out] / (scale_a * scale_w[out]))
acc_i32 += bias_i32[out]
```

若使用 per-tensor weight scale，所有输出通道共用一个 `scale_w`；若使用 per-channel weight scale，每个 `out` 有自己的 scale，精度通常更好，但 requantize 和参数管理更复杂。第三册在这一章先讲清账本，后面再讨论不同 scale 粒度对模型精度和内核复杂度的影响。

重新量化不是简单右移

如果下一层还希望接收 int8 activation，就要把 int32 accumulator 映射回 int8 或 uint8：

Listing 1.58 重新量化的数学形状

```
real_out = acc_i32 * scale_a * scale_w
q_out = clamp(round(real_out / scale_out) + zero_out, qmin, qmax)

combined_multiplier = scale_a * scale_w / scale_out
q_out = clamp(round(acc_i32 * combined_multiplier) + zero_out, qmin, qmax)
```

这里的 `combined_multiplier` 很少刚好是 2 的幂。高性能实现通常把它近似成固定点乘法加 shift，再处理舍入和饱和。于是重新量化至少包含四步：乘以缩放因子、舍入、加输出零点、clamp 到目标整数范围。若还有 ReLU 或 ReLU6，activation clamp 也可以融合进最后的范围限制。

int32 accumulator 到 int8 输出的重新量化路径



图示：本图要证明的命题是：requantize 是一条数值和整数实现共同参与的路径，不是把 int32 截断成 int8。

固定点 requantize 的边界

实际 int8 kernel 里不希望每个输出都做慢速浮点乘法。常见做法是把 `combined_multiplier` 近似为整数乘法和右移：

Listing 1.59 固定点 requantize 的形态

```
scaled = round(acc_i32 * multiplier / 2^shift)
q = clamp(scaled + zero_out, qmin, qmax)
```

这里要固定三件事。第一，`multiplier` 和 `shift` 怎样从实数 `multiplier` 生成。第二，右移前怎样舍入，负数是否对称处理。第三，乘法中间结果需要多宽，通常要用 64 位临时值避免溢出。

Listing 1.60 requantize 测试必须覆盖的值

```
acc_i32:
0
```

```

1
-1
near positive saturation
near negative saturation
values around rounding half
large values requiring int64 intermediate

```

若这些边界没测，量化输出可能在普通样本上看起来正常，却在某些通道长期偏一。对分类模型，偏一可能不改变 top-1；对语言模型 logits，长期偏差可能改变 top-k 排名。

zero point 修正项为什么会拖慢内核

非对称量化中 dot-product 是：

$$\sum_k (q_a[k] - z_a)(q_w[k] - z_w)$$

展开后：

$$\sum_k q_a[k]q_w[k] - z_w \sum_k q_a[k] - z_a \sum_k q_w[k] + Kz_az_w$$

这条式子说明，非对称 zero point 不是免费字段。kernel 要么每次乘法前减 zero point，要么预计算某些和，例如每个权重行的 $\sum q_w[k]$ 。权重 zero point 若为 0，可以少掉一部分修正；activation zero point 若固定，也可以把某些项合并进 bias 或预处理。

选择	内核影响	常见用途
权重对称	少一个权重零点修正	weight-only 或 per-channel weight
activation 非对称	更好利用 activation 范围	int8 activation pipeline
二者非对称	精度可能好	修正项和测试更复杂

这也是很多推理后端先做对称权重量化的原因：收益明显，内核合同较清楚。等 reference、oracle 和 profiler 稳定后，再引入 activation 非对称量化更稳。

bias 的尺度错误会稳定污染输出

bias 常被忽略，但量化路径里它非常敏感。若 int32 accumulator 表示的是：

$$acc_i32 = \sum_k q_a[k]q_w[k]$$

则它对应的实数尺度是 `scale_a*scale_w`。bias 若仍是 float，需要先除以这个尺度再加入 accumulator，或在 accumulator 变回实数后加入。两种做法语义都可以，但不能混用。

Listing 1.61 两种 bias 路径不能混用

```

path A:
  acc_i32 += round(bias_f32 / (scale_a * scale_w))
  output = requantize(acc_i32)

path B:
  real = acc_i32 * scale_a * scale_w
  real += bias_f32
  output = quantize(real)

```

若 path A 的 bias 被当成 path B 使用，或者 per-channel `scale_w[out]` 被错用成 per-tensor scale，输出会出现稳定偏移。它不像随机舍入误差，而是每次都朝同一方向偏。oracle 应该单独测 bias-only、zero-input、one-hot-input 这些样本，便于定位。

对称、非对称、per-tensor、per-channel 与 per-group

量化参数的粒度会影响误差和内核复杂度。

对称权重量化。 权重通常正负都有，使用 `zero_point=0` 可以减少内核里的减零点操作，也让 dot-product 更简单。当前 lab 的模型就是这种教学形状：int8 权重乘以一个 float scale。

非对称 activation 量化。 activation 可能范围偏正，例如 ReLU 后没有负数。使用非对称 zero point 可以更充分利用整数码点，但内核要处理 `qa-za`，有时还要展开零点修正项。

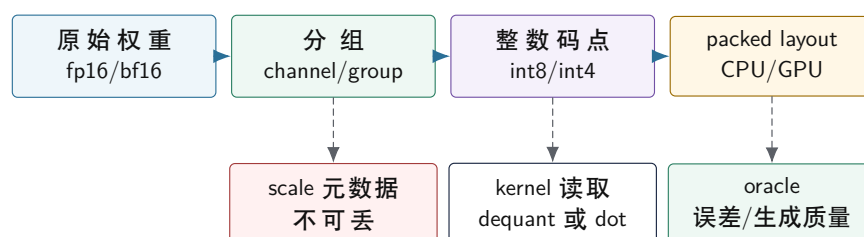
per-tensor scale。 整个张量共用一个 scale，实现简单，元数据少，但某些输出通道如果范围差异很大，小范围通道会损失精度。

per-channel scale。 每个输出通道拥有自己的权重 scale，常用于权重量化。它改善精度，但 requantize 阶段每个通道的 multiplier 不同，packing 和参数读取也要配合。

per-group scale。 大模型权重量化常按一小组连续权重共享 scale，例如每 32、64 或 128 个元素一组。它比 per-tensor 更能贴近局部范围，又比 per-element 元数据少。代价是 kernel 必须在读取权重块时同步读取 group scale，并且 packing 时不能把 scale 和数据关系打散。

工程上没有一个永远正确的选择。小型教学切片可以先使用 per-tensor scale，让模型文件、内核和 oracle 都清楚；7B 推理引擎要逐步引入 per-channel 或 per-group scale、校准数据集、后训练量化报告，以及 CPU/GPU 后端都能理解的 packed quantized layout。

7B 权重量化格式必须同时服务数值和后端



量化格式不是文件压缩格式：它必须同时让数值误差可解释、让 CPU/GPU kernel 能高效读取。

图示：本图要证明的命题是：7B 权重量化的 scale 粒度、整数码点和 packed layout 必须一起设计。

量化 oracle 要比较什么

浮点 reference 和 int8 实现不应该逐 bit 相等。对于分类切片，oracle 至少要比对三类结果：

- 数值误差：每个输出元素的绝对误差、相对误差或反量化误差是否在合同内。
- 排名稳定性：分类任务的 top-1 或 top-k 是否保持，边界样本是否被标记。
- 饱和统计：有多少值被 clamp 到最小或最大码点，是否集中在某一层或某通道。

如果只看 top-1，很多数值错误会被掩盖；如果只看逐元素误差，分类边界变化又可能被低估。对 7B 生成模型，还要增加层级 logits 检查和生成质量检查：固定 prompt、固定 sampler、固定随机种子时，logits 的 top-k 排名是否稳定；量化前后的困惑度或小验证集损失是否在阈值内；长上下文时 KV cache 量化是否导致后半段输出明显漂移。成熟报告应该同时写出 reference 版本、量化参数、输入样本集合、误差阈值、top-k 结果、饱和计数、logits 对比和生成样本摘要。

Listing 1.62 量化误差报告的最小字段

```
model_id
input_set_id
```

```

layer_name
scale_policy: per_tensor or per_channel
max_abs_error
mean_abs_error
top1_changed_count
clamped_min_count
clamped_max_count
logits_topk_changed_count
decode_token_mismatch_count
accepted: true or false

```

KV cache 能不能量化

KV cache 和权重不同。权重在加载后基本不变，可以离线量化、打包、校准；KV cache 是推理过程中不断写入的状态。每生成一个 token，每一层都会追加新的 K 和 V。若上下文很长，KV cache 会成为显著内存开销，也会成为 decode attention 的读取流。

Listing 1.63 KV cache 的状态账本

```

K_cache[layer, batch, head, position, head_dim]
V_cache[layer, batch, head, position, head_dim]

on each decoded token:
    compute k, v for current position
    write k, v into cache
    attention reads positions 0..current_position

```

是否量化 KV cache，要分开判断容量、带宽和误差。量化后 cache 更小，长上下文可能明显受益；但每次写入要量化，每次 attention 读取可能要反量化或使用支持低精度的 kernel。若 scale 按 token、head 或 block 变化，元数据读写也会进入热路径。KV cache 量化因此不能只看“省了多少字节”，还要看 decode tokens/s、logits 漂移和长上下文质量。

常见误区

权重量化成功不代表 KV cache 也能用同一套策略。权重是静态参数，KV cache 是动态状态；权重误差在每层固定存在，KV cache 误差会随着上下文读取方式进入后续 token。

三元素 dot-product 手算

量化如果不能手算，就很容易变成“把 float 压成 int8”的口号。固定一个最小 dot-product：

$$x = [0.2, -0.5, 1.0], \quad w = [0.1, -0.3, 0.7]$$

浮点 reference 是：

$$0.2 \times 0.1 + (-0.5) \times (-0.3) + 1.0 \times 0.7 = 0.02 + 0.15 + 0.7 = 0.87$$

先用对称 int8 量化。对称量化选择 `zero_point=0`，scale 由最大绝对值决定：

$$scale = \frac{\max(|values|)}{127}$$

对输入 `x`，最大绝对值是 `1.0`，所以：

$$scale_x = 1.0/127 \approx 0.007874$$

量化值：

$$q_x = \text{round}(x/\text{scale}_x) = [25, -64, 127]$$

对权重 w ，最大绝对值是 **0.7**：

$$\text{scale}_w = 0.7/127 \approx 0.005512$$

量化值：

$$q_w = \text{round}(w/\text{scale}_w) = [18, -54, 127]$$

整数 dot-product 用 int32 accumulator：

$$\text{acc} = 25 \times 18 + (-64) \times (-54) + 127 \times 127 = 450 + 3456 + 16129 = 20035$$

反量化：

$$y_q = \text{acc} \times \text{scale}_x \times \text{scale}_w \approx 20035 \times 0.007874 \times 0.005512 \approx 0.870$$

这个例子说明三件事。第一，int8 乘法的结果不能继续留在 int8，三个乘积相加已经到 20035，需要 int32 accumulator。第二，scale 的乘积是 accumulator 的真实单位。第三，误差来自 scale 选择、round、clamp 和有限码点，不是来自某个神秘的“量化损失”。

为什么 accumulator 要用 int32

最坏情况下，int8 乘积最大约为：

$$127 \times 127 = 16129$$

若 **K=4096**，全为同号最大值，则：

$$\text{acc}_{\max} = 4096 \times 16129 \approx 66,064,384$$

这个数远超 int16 的范围，但在 int32 范围内。若是 signed int8 的 **-128** 也进入计算，边界还要单独检查。量化 kernel 不能只在普通输入上测试；它必须测 near saturation、全正、全负、交替正负、大 **K** 和 bias 叠加后的边界。

Listing 1.64 int8 dot-product 的溢出边界

```
K = 4096
max_product = 127 * 127
max_acc = K * max_product

int16 max = 32767          // not enough
int32 max = 2147483647     // enough for this K
```

若后续引入 int4，单个乘积小很多，但 group scale、dequant-on-the-fly 和 packing 复杂度会上升。不要只按 bit width 判断速度；真正的内核要看 unpack 成本、scale 读取、向量指令支持和内存带宽。

非对称量化的手算边界

非对称量化用 **zero_point** 表示实数 0 落在哪个整数码点。常见公式是：

$$q = \text{clamp}(\text{round}(\text{real}/\text{scale}) + \text{zero_point})$$

$$\text{real} \approx (q - \text{zero_point}) \times \text{scale}$$

它适合范围明显偏正的 activation。例如 activation 范围是 $[0.0, 1.0]$ ，用 uint8 表示，scale 可以取：

$$scale = \frac{1.0 - 0.0}{255}, \quad zero_point = 0$$

如果范围是 $[-0.2, 1.0]$ ，zero point 会落在正整数附近，用来表达 0。问题是 dot-product 里每次都要处理 $(q_x - z_x)(q_w - z_w)$ ，否则整数积不是原始实数积。前文展开式中的修正项，就是非对称量化的工程成本来源。

per-channel scale 的真实作用

per-tensor scale 用整个权重矩阵的最大绝对值决定一个 scale。若某一行权重范围很小，它会被大范围行拖累。以两行权重为例：

$$w_0 = [0.01, -0.02, 0.03], \quad w_1 = [2.0, -1.5, 1.0]$$

per-tensor scale 由 **2.0** 决定：

$$scale = 2.0/127 \approx 0.01575$$

w₀ 里的 **0.01** 量化后约为 1，**-0.02** 约为 -1，**0.03** 约为 2，信息很粗。per-channel scale 则每个输出行单独选 scale，**w₀** 的 scale 由 **0.03** 决定：

$$scale_0 = 0.03/127 \approx 0.000236$$

小行的细节保留更多。代价是每个输出通道都要有自己的 scale，requantize 或 dequant 时不能只用一个 multiplier。CPU packed weight 也要保证第 **out** 行的 scale 与 packed 数据一起可定位；GPU kernel 则要避免 scale 读取破坏 coalescing。

从数值公式到 C++ quantizer

教学 quantizer 应该短到可以审查，但要把 round 和 clamp 写清。

Listing 1.65 对称 int8 量化的教学实现

```
1 struct SymmetricQuantParams {
2     float scale = 1.0F;
3 };
4
5 std::int8_t quantize_symmetric_i8(float value, SymmetricQuantParams params)
6 {
7     float scaled = std::round(value / params.scale);
8     float clamped = std::clamp(scaled, -127.0F, 127.0F);
9     return static_cast<std::int8_t>(clamped);
10 }
11
12 float dequantize_symmetric_i8(std::int8_t value, SymmetricQuantParams params)
13 {
14     return static_cast<float>(value) * params.scale;
15 }
```

这段代码故意不用 **-128**，是为了让正负范围对称，便于推导。工业实现可能使用 **-128**，也可能针对特定指令或格式选择不同边界；关键是格式合同必须写进模型 manifest 和 oracle，不能让不同 kernel 各自猜。

量化错误如何进入 logits

量化误差不是只停留在线性层输出。一个 transformer block 中, 投影误差会进入 attention score、softmax、attention output、MLP、residual 和下一层。K 的误差会影响 score:

$$score = \frac{q \cdot k}{\sqrt{d}}$$

若 score 的两个候选位置非常接近, 微小误差可能改变 softmax 排名。V 的误差进入加权和:

$$out = \sum_i softmax(score_i) \times v_i$$

MLP 权重量化误差则可能改变某些 hidden 维度的激活。最终 lm head 输出 logits, sampler 对 top-k、top-p、temperature 和随机数敏感。因此 7B 量化报告不能只写单层 `max_abs_error`。至少要同时看:

- 每层输出误差是否随层数放大。
- logits top-k 是否稳定。
- 固定 prompt、固定 seed 的 token 序列在哪一步开始分叉。
- 长上下文下 KV 量化是否比短上下文更容易漂移。
- 饱和值是否集中在某个层、某个 head 或某个 group。

量化章节的原理展开模板

以后每引入一种量化格式, 都按同一张表验收:

项目	必须回答的问题
问题来源	要省权重容量、activation 带宽、KV 容量, 还是提高某条 kernel 吞吐
抽象模型	real、q、scale、zero point、group、accumulator 分别是什么
数学公式	quant、dequant、dot、requant、bias fold 的公式
C++ 表示	manifest 字段、descriptor、quantizer、oracle 输入输出
内存布局	整数码点和 scale metadata 如何排列, packing 是否改变它们的对应关系
机器执行	使用 int32 dot、dequant-on-the-fly、固定点 requant, 还是查表/特化 kernel
性能模型	少读多少字节, 多做多少 unpack/scale 操作, 瓶颈从哪里迁到哪里
正确性边界	round、clamp、overflow、bias scale、top-k/logits 阈值
错误案例	scale 用错、zero point 漏减、group scale 错位、bias 尺度混用
工业处理	权重离线量化, activation 校准, KV 单独评估, 后端特化 layout

当前 lab 到 7B 量化路径的升级点

现在的 `linear_i8` 可以作为第一阶段验收: 模型加载、shape 检查、int8 权重读取、float scale 和 bias、activation、argmax 都已经清楚边界。要把它升级成 7B 推理引擎里的量化路径, 应分阶段推进, 而不是一次重写:

Listing 1.66 从教学线性层到完整 int8 kernel 的阶段

```
stage 1:
  int8 weights, float input, float accumulator
  easy oracle and model parsing
```

```

stage 2:
  int8 weights, int8 activation, int32 accumulator
  explicit input scale and zero point

stage 3:
  int32 bias folded into accumulator scale
  requantize output to int8 activation

stage 4:
  packed weights, SIMD dot-product, per-channel scales
  benchmark and error report tied to one model id

stage 5:
  group-wise int4 or int8 weights for transformer projections
  dequant-on-the-fly and backend-specific packed layout

stage 6:
  optional KV cache quantization
  long-context error and decode throughput report

```

每个阶段都必须保持同一件事：reference 语义不变。若引入 int8 activation 后 top-1 变化，或 7B 模型的 logits/top-k/token 序列变化，要能判断是量化误差超出合同，还是实现 bug，例如 scale 用错、zero point 漏减、bias 尺度不一致、requantize 舍入规则不匹配、group scale 读错、packed layout 和 kernel 解释不一致。

硬验收

读完本节，应该能完成下面几件事：

- 写出线性量化和反量化公式，并说明 scale、zero point、round 和 clamp 的作用。
- 解释 int8 dot-product 为什么需要 int32 accumulator。
- 把 `scale_a*scale_w/scale_out` 推导成 requantize 的 combined multiplier。
- 区分当前 lab 的简化路径和完整 int8 activation 路径。
- 解释 7B 权重在 fp16、int8、int4 下的容量压力，并说明 per-channel/per-group scale 的取舍。
- 说明 KV cache 量化为什么不同于权重量化。
- 设计一个量化 oracle，至少包含数值误差、top-k 稳定性、饱和统计、logits 对比和生成 token 变化。

后续内容会把这些量化账本放回算子实现：当权重被 packing、SIMD dot-product 指令被使用、GPU kernel 接入、多个层融合、activation buffer 被复用、KV cache 被压缩时，scale、zero point、accumulator、group metadata 和 oracle 仍然必须能被追踪。

1.6 低比特权重与 KV cache 量化：int4、group scale 与运行时状态

从 int8 走到 int4，问题变在哪里

int8 量化已经把 fp16 权重容量减半；int4 继续把 int8 再减半。对 7B 模型，容量差异非常直接：

Listing 1.67 7B 权重容量和元数据压力

```

fp16 weight: 7e9 * 2 bytes  ~= 14 GB
int8 weight: 7e9 * 1 byte   ~= 7 GB + scales
int4 weight: 7e9 * 0.5 byte ~= 3.5 GB + group scales

```

但 int4 不是“更小的 int8”。一个 int8 元素天然占一个字节；两个 int4 值通常打包在一个字节里。kernel 读取后要解包 nibble，处理符号扩展或 zero point，再乘 scale。为了精度，int4 权重很少整层共用一个 scale，通常按 group 保存 scale，例如每 32、64 或 128 个连续权重共享一个 scale。于是后端要同时处理两条流：压缩权重流和 scale metadata 流。

核心思想

int4 的主要难点不是 4-bit 算术本身，而是 packed byte order、group scale、尾部 padding、解包成本、scale 读取局部性和 oracle 合同必须同时正确。

先看一个会悄悄算错的实现

下面的伪代码看起来很自然，但它省略了 packed order、signedness、zero point、scale 粒度、tail policy 和后端重排合同。

Listing 1.68 容易算错的 int4 解包形态

```
for in in 0..input_channels:
    byte = packed[in / 2]
    q = (in % 2 == 0) ? (byte & 0xF) : (byte >> 4)
    w = (q - zero_point) * scale[in / group_size]
    acc += x[in] * w
```

常见误区

低比特实现最危险的错误，是 reference、converter、packing 和 kernel 各自持有一份隐含解释。正确做法是先定义一个可序列化的量化合同，再让 converter、verifier、packing、kernel 和 oracle 全部读取同一份合同。

group quant 的逻辑格式

以 group size 64 为例，一个输出通道的一段权重可以分成若干组。每组有 64 个 4-bit code 和一个 scale：

Listing 1.69 group-wise int4 权重的逻辑形状

```
weight[out, in]

group_size = 64
group_id = in / 64
offset_in_group = in % 64

q4_code = packed_weight[out, group_id, offset_in_group]
scale = group_scale[out, group_id]
real_weight ~= decode_int4(q4_code) * scale
```

若 in 维度不能整除 group size，就需要 tail policy。常见选择是补零、补中性码点或单独记录尾组长度。tail policy 必须进入 descriptor。否则不同 kernel 会对最后一组有不同解释。

字段	含义	错误后果
bit width	每个码点占几位	解包错位，全部权重错误
signedness	有符号还是无符号码点	零点和符号扩展错误
group size	多少权重共享 scale	scale 索引错误
scale dtype	scale 存 fp16/fp32 等	metadata 读取宽度错误
packed order	低 nibble 先还是高 nibble 先	每两个权重交换或错解
tail policy	尾组如何补齐	最后一块输出异常

量化合同要进入 manifest

如果 manifest 只写“这个权重是 int4”，后端仍然无法安全执行。manifest 需要把量化格式拆成可检查字段。

Listing 1.70 manifest 中的量化字段草图

```
QuantTensorManifest:
  tensor_name
  logical_shape
  role: q_proj | k_proj | v_proj | mlp_up | ...
  storage_bits
  code_signedness
  zero_point_policy
  scale_granularity
  group_size
  scale_tensor_name
  packed_order
  tail_policy
  calibration_record_id
```

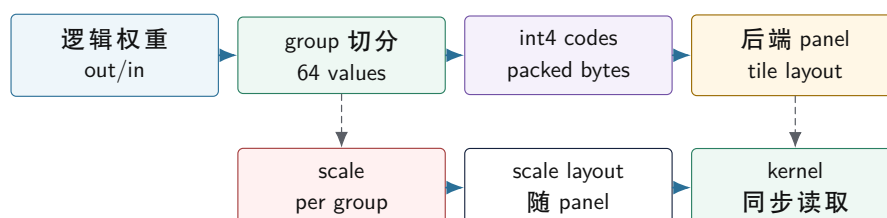
verifier 的任务不是重新做量化，而是拒绝不自洽的资产：packed byte 数、scale 数量、group size、tail policy、后端支持能力和角色一致性都要提前检查。

检查项	计算方式	失败时说明
packed bytes group count	$\text{ceil}(\text{elements} * \text{bits} / 8)$ $\text{ceil}(\text{input_channels} / \text{group_size}) * \text{output_channels}$	权重文件截断或格式解释错 scale 数量不匹配
backend support	capability 中是否支持该 profile	需要 fallback 或拒绝
tail policy	尾组是否有明确规则	最后一块输出不可验证
role consistency	同一逻辑角色是否 dtype/shape 一致	manifest 映射错误

packing 必须带着 scale 一起走

后端为了提高 cache locality 和 SIMD 利用率，通常会把权重从模型文件布局重排成 packed layout。对 int4/group quant，packing 不能只移动 4-bit 码点，还必须同步移动或重新索引 group scale。

int4 packing 同时重排权重码点和 scale



图示：本图要证明的命题是：int4 packing 不是只重排权重字节，scale metadata 必须跟着后端 panel 同步组织。

一个好的 packed descriptor 应该让 kernel 不需要查字符串、不需要理解原始文件格式，只需要按 descriptor 读取 panel：

Listing 1.71 int4 packed descriptor 草图

```
PackedInt4Weight:
  data: byte span
  scales: byte span
  output_channels
  input_channels
  group_size
  panel_rows
  panel_cols
  bytes_per_panel
  scales_per_panel
  nibble_order
  tail_policy
```

这个 descriptor 是第一册 ABI/对象布局知识和第二册 cache/SIMD 知识在第三册中的结合点。它既要在 C++20 中有明确对象生命周期和无拷贝视图，又要让后端按 cache line、SIMD 宽度和 tile 形状高效读取。

packing 要有字节级 oracle

只比较最终 logits，packing 错误定位会很晚。更可靠的办法是在 packed layout 形成后，抽取逻辑坐标，沿 descriptor 反查 packed byte、nibble、scale，再和 reference converter 比较。

Listing 1.72 packed int4 字节级 oracle

```
for sample in coordinate_samples:
    logical = {out, in}
    reference = convert_from_original_file(logical)

    packed_location = descriptor.locate(out, in)
    code = read_nibble(descriptor.data, packed_location)
    scale = read_scale(descriptor.scales, packed_location.group)
    actual = decode(code, scale, descriptor.zero_point_policy)

    compare(actual, reference, tolerance)
```

验收与评分标准

int4 packed weight 的最低门禁：

- verifier 能计算 packed bytes、group count 和 scale count。
- byte oracle 覆盖高/低 nibble、group 边界、tail 和 panel 边界。
- 单算子 oracle 至少覆盖 input channel 非 group size 整数倍的形状。
- profiler 分开报告 packing time、packed bytes、scale bytes 和 kernel time。

dequant-on-the-fly 还是预反量化

int4 权重进入计算时，有两种基本路线。第一种是 dequant-on-the-fly：kernel 每次读取 int4，解包、乘 scale，再参与 FMA 或 dot。第二种是预反量化：加载或 prepare 阶段把 int4 转回 fp16/bf16/fp32 的 packed buffer，热路径按浮点权重计算。

路线	优点	代价
dequant-on-the-fly 预反量化	内存带宽低，容量小 kernel 简单，可复用浮点路径	解包和 scale 乘法进入热路径 内存容量回升，prepare 成本高
混合路线	热层或热 tile 单独缓存	缓存策略和一致性更复杂

CPU decode 常常受权重读流和小 batch 形态影响，dequant-on-the-fly 可能有价值；但如果解包成本和 scale 读取打断 SIMD pipeline，收益会消失。prefill 阶段 GEMM 更大，预反量化或 vendor kernel 可能更合适。没有 profiler，不能靠直觉判断。

评估 dequant-on-the-fly 时，不要只看“每个权重少读多少字节”。还要看 scale metadata、unpack/extend/convert、shuffle、寄存器压力和 tail path。低比特节省的是带宽，也可能增加指令压力。

KV cache 量化是运行时问题

权重量化是静态问题：模型加载前后，权重不变。KV cache 量化是运行时问题：每个 token 生成后，所有层都会写入新的 K/V；后续 attention 会反复读取历史 K/V。

Listing 1.73 KV cache 量化写入和读取路径

```

on kv_append(layer, position, k_f16, v_f16):
    scale_k = choose_scale(k_f16)
    scale_v = choose_scale(v_f16)
    k_q = quantize(k_f16, scale_k)
    v_q = quantize(v_f16, scale_v)
    store k_q, v_q, scale_k, scale_v

on attention_decode(layer, q, context_length):
    for pos in 0..context_length:
        k = dequantize(load_k_q(pos), load_scale_k(pos))
        v = dequantize(load_v_q(pos), load_scale_v(pos))
        use k, v in attention

```

这段路径暴露三个关键问题。第一，scale 选择发生在 runtime，不能完全离线。第二，metadata 会随 position 增长，不能只统计 K/V 码点大小。第三，decode 每步会读取从 0 到当前 position 的历史；若每个 position 都有独立 scale，metadata 读取会变成热路径的一部分。

常见误区

KV cache 量化不能只报告“内存减半”。必须同时报告写入成本、metadata 成本、decode attention 耗时、logits 漂移和长上下文生成质量。

KV 量化误差先进入哪里

很多人会把 KV cache 量化想成“把历史 K/V 反量化回来，后面照常算 attention”。这句话只描述了数据流，没有描述误差流。真正先被扰动的不是最终输出文本，而是每个 head 的 attention score：

$$score_{i,j,h} = \frac{Q_{i,h} \cdot K_{j,g(h)}}{\sqrt{d}}$$

若量化后的 key 变成

$$\tilde{K}_{j,g(h)} = K_{j,g(h)} + \Delta K_{j,g(h)}$$

则 score 误差是：

$$\Delta score_{i,j,h} = \frac{Q_{i,h} \cdot \Delta K_{j,g(h)}}{\sqrt{d}}$$

这条式子比“量化会有误差”更重要，因为它说明了三件事。第一，误差会被当前 query 的方向放大或缩小，不是所有 token 位置都同样敏感。第二，同一个量化 profile，在不同 head、不同层、不同 query 上误差大小可能完全不同。第三，score 误差进入 softmax 后，会重新分配整行概率，而不是只影响一个位置。

如果 value 也量化，

$$\tilde{V} = V + \Delta V$$

则最终 attention 输出误差不只来自 ΔV ，还来自概率本身被改写。把概率写成

$$\tilde{P} = P + \Delta P$$

则：

$$\tilde{O} = \sum_j \tilde{P}_j \tilde{V}_j = \sum_j (P_j + \Delta P_j)(V_j + \Delta V_j)$$

展开后至少有三类误差项：

$$\Delta O \approx \sum_j P_j \Delta V_j + \sum_j \Delta P_j V_j + \sum_j \Delta P_j \Delta V_j$$

第一项是“value 被量化”的直接误差；第二项通常更危险，因为它表示 softmax 权重已经被改写，模型开始从历史中“看错位置”；第三项通常更小，但在长上下文和高压缩率下也不能忽略。

核心思想

KV cache 量化最敏感的地方往往不是 V 本身，而是 K 进入点积后对 softmax 排序和归一化的扰动。只看反量化后的 L2 误差，不足以判断生成质量。

softmax 为什么会放大量化误差

softmax 不是线性函数。若某一行 score 中本来有两个位置非常接近，比如：

$$s_a = 12.10, \quad s_b = 12.04$$

那么一个很小的量化误差就可能改变它们的排名。假设量化后变成：

$$\tilde{s}_a = 12.00, \quad \tilde{s}_b = 12.08$$

表面上每个位置只改了不到 **0.1**，但 attention 概率最大的那个位置已经从 **a** 变成 **b**。如果这两个位置对应的是两个不同语义片段，后续层读到的内容就会完全不同。长链条 decode 中，这种细小排序翻转会沿层数和 token 数累积。

再看稳定 softmax 的分母：

$$p_j = \frac{e^{s_j - m}}{\sum_t e^{s_t - m}}$$

当一行中存在几个接近最大值的位置时， $\Delta score$ 会通过指数放大为概率差异；当某个位置明显比其余位置大很多时，小误差反而可能不敏感。也就是说，量化误差的危害和“当前这行 attention 是尖锐分布还是接近平局”直接相关。

行分布形态	对 score 小误差的敏感性	典型现象
单峰明显领先	较低	top-1 不变，概率小幅扰动
多个位置接近	很高	排名翻转，读错历史片段
长尾很平	中等	概率整体重分配，输出更发散

因此 KV cache 量化评估不能只做 token 级平均误差。至少还要观察：

- attention score 的最大误差和分位数；

- top-1 attention 位置是否翻转；
- 重要 head/层是否在长上下文上更敏感；
- logits top-k 是否在长 context 档位下明显变化。

一个具体的 KV 量化误差数值例子

上面讲的是误差路径，现在把它压到可以手算的数字。设某个 head 的 query 和某一历史 key 为：

$$q = [0.60, -0.20, 0.50, 0.10]$$

$$K = [0.80, -0.35, 1.10, -0.50]$$

head 维度 $d=4$ ，所以原始 attention score 是：

$$score = \frac{q \cdot K}{\sqrt{4}} = \frac{0.60 \times 0.80 + (-0.20) \times (-0.35) + 0.50 \times 1.10 + 0.10 \times (-0.50)}{2} = 0.525$$

现在先看一种“局部范围较准”的 int4 对称量化。若这一个量化块的最大绝对值就是 **1.10**，则 scale 可取：

$$\alpha_{local} = \frac{1.10}{7} \approx 0.157$$

量化码点是：

$$q_K = \text{round}(K/\alpha_{local}) = [5, -2, 7, -3]$$

反量化后：

$$\tilde{K}_{local} = q_K \alpha_{local} \approx [0.786, -0.314, 1.100, -0.471]$$

新的 score 变成：

$$score_{local} = \frac{q \cdot \tilde{K}_{local}}{2} \approx 0.519$$

所以：

$$\Delta score_{local} \approx -0.006$$

再看一种更粗的情况。若这个 key 被塞进一个更大 block，block 内别的位置把最大绝对值拉到 **1.50**，则 scale 变成：

$$\alpha_{block} = \frac{1.50}{7} \approx 0.214$$

这时码点是：

$$q'_K = \text{round}(K/\alpha_{block}) = [4, -2, 5, -2]$$

反量化结果变成：

$$\tilde{K}_{block} \approx [0.857, -0.429, 1.071, -0.429]$$

新的 score 是：

$$score_{block} = \frac{q \cdot \tilde{K}_{block}}{2} \approx 0.546$$

于是：

$$\Delta score_{block} \approx +0.021$$

这组数字说明两个关键事实。第一，误差大小不只由 bit width 决定，还强烈依赖 scale 粒度和同块中的动态范围共享。第二，同一个原始 key，在两个都“合法”的 int4 配置下，score 偏移方向都可能相反：一个向下漂，一个向上漂。

现在把它放回 attention 排名。假设同一行里另一个候选位置的原始 score 是：

$$score_{rival} = 0.520$$

则原始情况下当前 key 领先：

$$0.525 > 0.520$$

局部量化后变成：

$$0.519 < 0.520$$

排名翻转；而粗 block 量化后又变成：

$$0.546 > 0.520$$

不但没翻转，领先还更大。

常见误区

这就是为什么很多人只看反量化后的均方误差会误判 KV 量化质量。真正决定 attention 行为的不是“整体误差小不小”，而是“关键候选位置之间的 score 排名会不会被改写”。

若继续往下一步看 softmax，排名翻转的影响会直接进入概率分配。尤其当一行里几个位置本来就接近时，0.01 到 0.03 级别的 score 扰动已经足以改写 top-1 attention 位置。对长上下文生成，这种翻转不是一次性的，它会把错误读出的上下文写回后续层 hidden，再影响下一步 query。

量化后的容量账本

KV cache 的基础容量公式已经在 Transformer/KV cache 部分建立。低比特章节只补充量化后新增的账本项：码点字节会下降，但 scale metadata、page table、padding、对齐浪费和临时 dequant buffer 可能抵消一部分收益。报告不能只写“fp16 变 int8 所以减半”，还要把这些附加项列出来。

项目	进入容量账本吗	说明
K/V 码点	是	最大主体，随 context 增长
scale metadata	是	粒度越细越大，也影响热路径
page table	是	分页 cache 或 sliding window 需要
padding/alignment	是	block、head 和设备对齐会产生浪费
临时 dequant buffer	视策略而定	若 attention 前反量化，峰值内存上升

把 metadata 字节也算出来

继续使用 7B 级 GQA 参数：layers=32, kv_heads=8, head_dim=128, context=4096, batch 为 1。fp16 KV 码点容量前面算过是 512MiB。如果把 K/V 码点改成 int8，码点主体约为：

Listing 1.74 int8 KV 码点主体容量

```
code_bytes =
    32 * 4096 * 8 * 128 * 2 * 1
    = 268435456 bytes
    = 256 MiB
```

但是 scale metadata 要看粒度。若每个 layer, position, kv_head 为 K 和 V 各存一个 fp16 scale，则：

Listing 1.75 per-token-per-head fp16 scale 容量

```
scale_bytes =
    layers * context * kv_heads *
    2 /* K and V */ *
    2 /* fp16 scale */

scale_bytes =
    32 * 4096 * 8 * 2 * 2
```

```
= 4194304 bytes
= 4 MiB
```

这个 metadata 相对 256 MiB 码点不大，但它在 attention 热路径中被反复读取。若改成 per-channel scale，每个 `head_dim` 都有 scale，则 metadata 变成：

Listing 1.76 per-token-per-channel fp16 scale 容量

```
scale_bytes =
    layers * context * kv_heads * head_dim *
    2 /* K and V */ *
    2 /* fp16 scale */

scale_bytes =
    32 * 4096 * 8 * 128 * 2 * 2
    = 536870912 bytes
    = 512 MiB
```

这就完全改变了账本：KV 码点降到 256 MiB，但 scale metadata 自己达到 512 MiB，容量反而比 fp16 KV 更差，还让 kernel 读取更多元数据。per-channel 可能精度好，但不能不算 metadata。

若使用 int4 KV，码点主体再减半到约 **128MiB**。但如果 scale 粒度太细，metadata 会吞掉大部分收益。因此 KV 量化报告必须同时列出 `code_bytes`、`scale_bytes`、`page_table_bytes` 和 `alignment_waste_bytes`。

核心思想

KV cache 量化的核心不是 bit width 越低越好，而是码点容量、scale metadata、读取热路径和误差共同闭合。metadata 不进入账本的量化报告没有工程意义。

KV dequant buffer 要不要物化

attention 读取量化 KV 时有两种路线。第一种是 on-the-fly dequant：读一个 block 的量化 K/V 和 scale，在寄存器或 shared memory 中还原并立即参与 score 和加权求和。第二种是先把一段历史 K/V 反量化到临时 buffer，再按普通 fp16 attention 处理。

路线	优点	风险
on-the-fly	不增加大临时 buffer，容量优势保留	kernel 复杂，scale 读流和解包进入热路径
物化 dequant buffer	可复用现有 attention kernel	峰值内存上升，额外写回和读取
分块物化	折中，适合 tiled attention	block 边界和 softmax 状态更复杂

如果物化整个 context 的 K/V，临时 buffer 大小接近 fp16 KV 的读取窗口。对 **4096** context、GQA 参数，单个 session 全层物化会非常大，通常不可接受。实际更可能按层、按 head 或按 block 物化，处理完立即释放。descriptor 和 memory planner 必须知道这个 workspace，否则峰值内存报告会少算。

KV cache scale 粒度

KV cache 的 scale 可以按不同粒度设计：

粒度	含义	风险
per-tensor	整个 cache 或整层共享 scale	范围过粗，误差可能大
per-head	每个 head 共享 scale	metadata 适中，但 position 差异被忽略
per-token	每个 position 一组 scale	精度好，metadata 热路径更重
per-block	每个 position block 一组 scale	精度与开销折中，需要 page/block 管理
per-channel	head_dim 内不同维度独立 scale	精度更细，metadata 和 kernel 复杂

对长上下文 decode，per-token 或 per-block 常更有解释力，因为不同 token 的 K/V 范围可能变化。但 scale 粒度越细，metadata 越多，attention kernel 的读取也越复杂。GPU 上还要考虑 coalesced load；CPU 上要考虑 cache line 和 TLB。这个选择必须由模型质量和 profiler 共同决定。

为什么 scale 粒度会改变误差分布

scale 粒度不是单纯的 metadata 体积选择，它直接决定误差怎样分布到 **head_dim** 和 **position** 上。设某个量化块的真实值向量为 $\mathbf{x}$ ，scale 为 α ，码点为 $\mathbf{q}$ ，则量化过程可以抽象成：

$$q = \text{clip}(\text{round}(x/\alpha)), \quad \tilde{x} = \alpha q$$

误差是：

$$\Delta x = \tilde{x} - x$$

若一个 scale 覆盖的数值范围太大，较小分量会被粗糙舍入；若 scale 覆盖范围太小，极值分量容易饱和。于是不同粒度的本质区别是：它们是在什么维度上共享动态范围。

per-head。一个 head 的所有位置或某个大块共用 scale。优点是 metadata 轻；缺点是如果这个 head 某几个 token 的 K/V 振幅特别大，其余位置的量化分辨率会被拖粗。

per-token。每个 position 各自选 scale。它能较好适应不同 token 的振幅变化，尤其适合长上下文中不同段落、代码块、表格或多语言文本混合时的局部范围变化；但每个 position 都要带 metadata，decode 热路径会反复读取这些 scale。

per-channel。**head_dim** 内不同维度各自选 scale。它能照顾某些维度长期幅值大、某些维度长期幅值小的情况，但 metadata 非常重，而且 kernel 需要更细粒度反量化。

per-block。在 position 和 dim 之间取一个折中块。它经常是工程上最实用的选择，因为既能比 per-head 更贴近局部范围，又不会像 per-token/per-channel 那么重。

可以把不同粒度理解成“谁共享同一个动态范围预算”。共享得越广，metadata 越省，但误差耦合越强；共享得越细，精度越好，但热路径越重。KV cache 和静态权重不同，它还要考虑历史长度增长后的累计读取成本，所以最优粒度往往不是权重量化里最优的那一个。

粒度	动态范围共享对象	metadata 成本	误差特征
per-head	整个 head 的较大区域	低	局部极值拖粗整体分辨率
per-token	每个 position	中高	适应 token 间范围变化
per-channel	dim 维度	很高	适应维度间长期振幅差异
per-block	局部块	中等	在局部适应性和成本间折中

长上下文为什么更容易暴露 KV 量化问题

短上下文中，decode 每步只看很少历史位置，attention 的候选集合小，很多误差还没有充分积累。长上下文中，问题会同时从三个方向放大。

第一，候选位置数变多。只要某些历史位置的 score 因量化误差发生排序翻转，softmax 就可能把注意力分到不同 token 上。第二，历史分布更复杂。长上下文里往往混有多段主题、代码、表格、指令头、自然语言解释，K/V 的局部动态范围比短 prompt 更不均匀。第三，误差会跨 token 累积。一次 decode 读错历史，不一定当步就输出错误 token，但它会把错误信息写入后续层 hidden，再生成下一步的 query。

这也是为什么 KV cache 量化报告必须按 context length 分桶，而不是只报一个全局平均值。一个合理的最小报告形态至少应包含：

Listing 1.77 长上下文量化报告的最小分桶

```
context buckets:
  0-512
  512-2048
  2048-8192
  8192+

for each bucket report:
  logits_topk_change
  attention_top1_flip_rate
  decode_tokens_per_second
  kv_metadata_bytes
```

常见误区

如果一个 KV 量化方案只在 256 或 512 token 上验证“几乎无损”，这个结论对 8K、32K 长上下文没有证明力。KV cache 是随上下文长度增长的运行时状态，不是一次性离线张量。

分页 KV cache 和 sliding window

真实请求不会总是从长度 0 开始生成固定短文本。服务端可能同时维护多个 session，每个 session 的 context length 不同，还可能触发上下文截断、prefix cache 复用或 sliding window。此时 KV cache 不再是一个简单的大连续数组，而更像一组 page 或 block。

Listing 1.78 分页 KV cache 的逻辑映射

```
logical position:
  session_id
  layer
  kv_head
  token_position
  head_dim_range

physical location:
  page_id = page_table[session_id, block_index]
  offset = position_in_block * stride_token + head * stride_head
```

分页设计便于增长、回收和 prefix 复用，但 attention kernel 要处理 page table，scale metadata 也要跟随 page 管理。sliding window 还需要逻辑 position 到物理 slot 的环形映射；descriptor 若不记录 epoch 或 base position，很容易读到旧 token。

工程案例

一个常见长上下文 bug 是：KV cache 物理 slot 被复用后，scale metadata 没有同步更新，或者 profiler 仍按旧 position 汇总。短 prompt 下这个 bug 不出现；只有 context 超过一个 page 或 sliding window 回绕后，logits 才开始漂移。

C++20 runtime 中的 KV cache descriptor

KV cache 量化需要 runtime descriptor，而不是只存在模型 manifest 中。

Listing 1.79 量化 KV cache 的运行时 descriptor

```
QuantizedKVCacheDesc:
  layers
  batch
  kv_heads
  head_dim
  capacity_tokens
  storage_dtype
  scale_dtype
  scale_granularity
  block_size
  layout
  page_table_policy
  dequant_policy
```

如果 KV cache 使用 page/block 增长，descriptor 还要指向 page table。若使用 sliding window，descriptor 必须记录逻辑 position 到物理 slot 的映射。若 CPU/GPU 各自维护 cache，runtime 必须知道数据在哪个设备上，什么时候需要同步或迁移。

Listing 1.80 KV cache append 的边界检查

```
append(layer, session, position, k, v):
  if position >= capacity_tokens:
    return context_limit_error
  slot = map_logical_to_physical(session, position)
  if slot.epoch != expected_epoch:
    return stale_slot_error
  quantize_and_store(k, v, slot)
  mark_written(layer, session, position)
```

KV cache 写入不是普通数组赋值。它至少要处理 capacity、逻辑到物理映射、epoch、量化数据、metadata 和 written 状态。

低比特量化的 oracle

低比特量化 oracle 至少要分四层：

- 权重还原层：随机抽查 packed int4 权重和 scale，确认反量化值匹配 reference converter。
- 单算子层：linear、attention、MLP 输出与 fp16/fp32 reference 比较。
- logits 层：比较 top-k、最大误差、KL 或分布差异。
- 生成层：固定 prompt、sampler 和 seed，比较 token 序列、困惑度或小验证集指标。

对 KV cache，oracle 还要增加 position 维度。短上下文正确不代表长上下文正确；前 32 个 token 正常不代表 2048 之后不漂移。报告必须包含 context length 分档。

Listing 1.81 低比特量化验收报告字段

```

quant_format:
  weight_bits
  group_size
  scale_dtype
  kv_cache_bits
  kv_scale_granularity

correctness:
  packed_weight_sample_errors
  layer_error_by_name
  logits_topk_change
  decode_mismatch_by_context_length

performance:
  model_load_time
  packing_time
  prefill_latency
  decode_tokens_per_second
  kv_cache_bytes
  metadata_bytes

```

量化决策要接回后端计划

量化 profile 最终必须影响 executable plan。后端可能支持 int8 GEMM，但不支持 int4 group size 128；可能支持 q4 权重，但要求 scale 连续对齐；可能支持 fp16 KV cache，但不支持 int8 KV cache attention。这些能力必须进入 plan。

Listing 1.82 量化 profile 与后端选择

```

QuantProfile:
  weight_format
  activation_format
  kv_cache_format
  error_tolerance_profile

BackendCapability:
  supported_weight_formats
  supported_kv_formats
  preferred_group_sizes
  requires_packed_scales

Plan decision:
  if backend supports profile:
    choose optimized kernel
  else if fallback allowed:
    choose reference or dequantized kernel
  else:
    reject with diagnostic

```

这让低比特量化成为编译器问题，而不是 kernel 小技巧：manifest 描述资产，verifier 检查自治，compiler 根据 backend capability 选择计划，runtime 执行计划，oracle 和 profiler 证明收益。

硬验收

读完本节，应该能说明：

- int4 的难点在 packed byte order、group scale、tail policy、解包和 scale 读取。
- 量化合同必须进入 manifest 和 verifier，而不是由 kernel 隐式猜测。
- packing 必须同时组织权重码点和 scale metadata。
- packed int4 要有字节级 oracle，覆盖 nibble、group、tail 和 panel 边界。
- dequant-on-the-fly 与预反量化各有容量、带宽和计算成本取舍。
- KV cache 量化是运行时状态问题，不等同于静态权重量化。
- KV cache 容量账本必须包含码点、scale metadata、page table、padding 和临时 buffer。
- KV cache scale 粒度会同时影响误差、metadata 大小和 attention kernel 热路径。
- 分页 KV cache 和 sliding window 需要显式记录逻辑到物理映射和 stale slot 防护。
- 低比特量化 oracle 必须覆盖 packed weight、单算子、logits、生成序列和 context length。
- 量化 profile 必须参与 backend capability 检查和 executable plan 选择。

后续进入具体 C++20 后端实现时，本节的 descriptor 和报告字段会直接变成代码结构、测试 fixture 和 benchmark 输出，而不是停留在文字说明。

1.7 量化工程实现：C++20 descriptor、校准记录、oracle 与回归门禁

为什么量化实现不能只写一个转换脚本

前两章已经说明，量化不是把 float 改成 int8，也不是把两个 int4 塞进一个字节就结束。真正的工程问题是：模型文件、编译前端、IR pass、后端 packing、runtime、oracle 和 profiler 必须对同一份量化语义达成一致。只要其中一层用自己的口径解释 scale、zero point、group size 或 tail policy，输出就可能稳定地错。

先看一个实际会发生的错误。转换脚本把某层 MLP 的 Wup 权重量化成 int4，每 64 个输入通道共享一个 scale。后端为了 SIMD 方便，把权重重排成 16x64 的 panel。若 packing 只移动 int4 code，没有把每个 group 的 scale 按同样 panel 顺序重排，kernel 会读取错误 scale。这个错误不一定崩溃，也不一定产生 NaN；它更可能让某些 token 的 logits 轻微漂移，直到长生成时才表现为质量变差。

核心思想

量化工程实现的核心不是某个低位宽格式，而是结构化元数据和验收链路。C++20 项目必须让 descriptor、verifier、packing、kernel、oracle 和 profiler 使用同一套量化合同。

本节把量化落到 C++20 项目结构。目标不是给出某个库的完整代码，而是讲清楚每个模块应该负责什么、哪些字段必须显式保存、哪些错误应该在编译前端拦住、哪些误差应该在 oracle 中报告、哪些性能收益必须由 profiler 证明。

QuantDescriptor：量化合同的最小载体

一个量化张量至少有三层身份。第一层是逻辑身份：它近似表达的是浮点权重、activation 还是 KV cache。第二层是存储身份：字节里到底是 int8、uint8、int4、nf4、fp8，scale 是 fp16 还是 fp32。第三层是计算身份：kernel 怎样反量化、累加和输出。把这三层都压进一个 dtype=q4 字符串，是后续混乱的源头。

Listing 1.83 QuantDescriptor 的工程字段

```
QuantDescriptor:
  logical_value: weight | activation | kv_cache
  storage_format: int8 | uint8 | int4 | nf4 | fp8
  bit_width
  signedness
  scale_dtype: f16 | f32
  zero_point_policy: none | symmetric | asymmetric
```

```
granularity: tensor | channel | group | token | block
axis
group_size
block_size
scale_shape
zero_point_shape
packed_order
tail_policy
calibration_id
error_contract_id
```

这些字段看起来多，但每一个都能对应一个真实错误。没有 **signedness**，int4 code 的符号扩展可能错；没有 **axis**，per-channel scale 会沿错维度读取；没有 **scale_shape**，verifier 无法判断 scale 数量是否匹配；没有 **packed_order**，高 nibble 和低 nibble 的解释会不一致；没有 **tail_policy**，最后一组不整除 group size 时会出现后端分歧。

常见误区

descriptor 的目标不是让代码显得正规，而是禁止后端猜测。任何需要 kernel 根据 tensor name、文件路径或隐含模型家族推断的量化事实，都应该提升为 descriptor 字段。

在 C++20 中，descriptor 应该是一个轻量、可拷贝或可引用的值对象。它不拥有大内存，不打开文件，不选择 kernel。它只描述合同，并能被 manifest、verifier、IR、packing、runtime 和报告共同引用。

Listing 1.84 C++20 中 descriptor 的责任边界

```
QuantDescriptor
  owns: small enum fields, shape metadata, ids
  does not own: tensor bytes, scale buffer, backend packed bytes

TensorDesc
  shape, stride, dtype, layout, device
  optional QuantDescriptor id

TensorStorage
  byte span or owned buffer
  file mapping or allocated memory
```

这样拆分以后，模型导入可以先建立 **TensorDesc** 与 **QuantDescriptor**；后端 prepare 阶段再用它们生成 **PackedWeightDesc**；runtime 执行时只绑定已经验证过的 descriptor 和 buffer。

CalibrationRecord：校准不是临时日志

校准选择 scale。scale 选择错了，后端再快也只是高效地产生错误近似。工程上不能只在转换脚本里打印一行 **calibrated**，而要保存可追溯的校准记录。

Listing 1.85 校准记录需要保存的事实

```
CalibrationRecord:
  calibration_id
  tensor_name_or_role
  dataset_id
  sample_count
  token_length_buckets
```



```
algorithm: minmax | percentile | mse_search | kl | runtime
clipping_policy
observed_min
observed_max
saturation_rate
scale_summary
created_by_tool_version
```

对权重，校准记录主要说明扫描范围和算法。对 activation，校准记录必须说明代表性输入。对 KV cache，校准更复杂，因为 K/V 是运行时状态；报告应该按 prompt 类型、context length 和层/头维度统计范围。如果一个量化模型只给出 scale 文件，却没有校准记录，后续质量问题很难定位：是模型本身不适合低比特，还是校准集太偏，还是后端解释错了 metadata？

对象	校准重点	必须警惕的偏差
权重	每层/每通道/每组范围	极端值拉大 scale, 尾组处理不一致
activation	代表性 prompt 和 batch 形态	校准集短、语言单一、没有代码或长上下文
KV cache	position、head、context length 分布	短上下文通过，长上下文漂移
logits	采样前分布稳定性	top-k 变化被均值误差掩盖

校准记录要进入 manifest 或伴随 manifest 存放。verifier 不一定重新校准，但它应该能检查记录是否存在、是否与当前张量匹配、scale 数量是否一致、后端是否支持这种量化策略。

verifier：把格式错误停在热路径之前

量化 verifier 至少分三层。第一层检查元数据完整性；第二层检查 shape 与 scale 数量；第三层检查后端支持。不要把这些检查推给 kernel，因为 kernel 位于最难给出好诊断的地方。

Listing 1.86 量化 verifier 的分层检查

```
metadata checks:
  bit_width is supported
  storage format and signedness are explicit
  scale dtype exists when format requires scale
  zero point policy matches signedness

shape checks:
  quantized axis is valid
  group count matches tensor shape and group size
  scale tensor shape equals expected scale_shape
  packed byte length matches bit_width and element count
  tail policy exists when dimension is not divisible

backend checks:
  CPU/GPU backend supports this format
  chosen kernel supports accumulator and output dtype
  alignment and packed order can be represented
```

诊断要像编译错误一样具体：

Listing 1.87 量化 verifier 诊断示例

```

error: tensor layers.21.mlp.wup.weight has invalid q4 scale shape
weight shape: [11008, 4096]
quant axis: input channel
group size: 64
expected scales per output row: 64
expected scale shape: [11008, 64]
actual scale shape: [11008, 63]
note: input dimension 4096 requires 4096 / 64 groups

```

这样的诊断能把问题定位到转换脚本或模型资产，而不是让用户面对 `segmentationfault`、`invalidargument` 或一堆错误 token。第一册强调过二进制和 ABI 的可信边界；这里的 verifier 就是在 AI 模型资产进入二进制热路径之前建立可信边界。

IR 中的量化节点：显式边界胜过隐式魔法

量化进入 IR 后，有两种常见表达。第一种是显式节点：`quantize`、`dequantize`、`requantize`、`quantized_linear`。第二种是在 tensor edge 上附加 quant descriptor，让某些算子直接消费量化边。两者都可以，但不能让量化完全隐藏在后端。

Listing 1.88 显式量化 IR 的形状

```

%wq = const_weight "layers.0.attn.wq.weight" : tensor<[4096,4096], q4_group>
%x  = rmsnorm(%hidden, %gamma)           : tensor<[T,4096], f16>
%y  = quantized_linear(%x, %wq)
      {accumulator=f32, output=f16, qdesc=#q4_g64}
      : tensor<[T,4096], f16>

```

显式表达带来三个好处。第一，pass 能看到量化边界，不会错误地把 dequant 移到不安全位置。第二，memory planner 能知道 scale 和 packed weight 的生命周期。第三，oracle 能在量化算子边界挂比较点。缺点是 IR 更复杂，需要 pass 明确处理量化节点。

对于初学者，可以先采用保守策略：权重量化用 `quantized_linear` 直接表达；activation 量化用显式 `quantize/dequantize/requantize` 表达；KV cache 量化用 runtime descriptor 表达。这样语义边界最清楚，后续再根据性能需求做融合。

常见误区

不要让优化 pass 看不见量化边界。一个融合 pass 如果不知道 linear 权重是 q4 group，就可能把需要 materialize 的 scale 读流藏进大 kernel，导致 oracle 难以定位误差。

packing: descriptor 到后端物理布局的转换

packing 是 lowering 的一部分，不是加载阶段的随手优化。它把逻辑量化张量转换成后端 kernel 喜欢的物理布局。一个 C++20 后端应该把 packing 输出描述清楚：

Listing 1.89 PackedQuantizedWeightDesc 的最小内容

```

PackedQuantizedWeightDesc:
  source_tensor_id
  backend_id
  isa_or_device
  panel_rows
  panel_cols
  tile_k
  data_offset
  data_bytes
  scale_offset

```

```

scale_bytes
bytes_per_panel
scales_per_panel
alignment
tail_policy
checksum

```

这个 descriptor 让 kernel 可以按 panel 读取, 不需要知道原始模型格式; 也让诊断可以从 packed bytes 追溯到原始 tensor。若 oracle 发现某层误差突然变大, 报告能指出它使用了哪个 packed descriptor、哪个 backend、哪个 ISA、哪个 group size, 而不是只写 “linear failed”。

packing 要单独计时。若 int4 packing 花了 30 秒, 但用户只生成 20 个 token, 热路径的加速可能无法抵消加载时间; 若服务端长期复用同一个模型, packing 成本就可以摊销。profiler 必须把 `pack_time_ms` 和 `decode_tokens_per_s` 分开, 避免把 prepare 成本藏起来。

reference converter: 先证明字节解释正确

低比特量化的第一层 oracle 不是比较最终生成文本, 而是证明 “字节到实数” 的解释正确。reference converter 应该用最清楚的标量实现, 不追求速度。它读取原始 int4/int8 bytes、scale、zero point 和 tail policy, 输出一段 fp32 或 fp16 参考值。

Listing 1.90 reference converter 的测试输入

```

test cases:
  one full group
  dimension not divisible by group size
  positive and negative int4 codes
  high-nibble-first and low-nibble-first packed order
  symmetric and asymmetric zero point
  f16 and f32 scale storage

```

只有 converter 通过, 才有资格测试 quantized linear。否则单算子误差可能来自字节解释、scale 数量、tail policy、SIMD 解包、累加顺序中的任何一个点。把验证分层, 是编译器工程的基本方法: 先验证低层事实, 再验证组合行为。

reference decoder layer: 把整层链条跑通

前面的量化、attention 和 KV cache 内容已经把局部原理拆开。真正让工程变稳的, 是有一条慢但可信的 reference decoder layer 路径, 能把 embedding 之后的一层完整计算串起来。它不追求快, 只追求语义清楚、边界明确、可逐点对比。

Listing 1.91 reference decoder layer 的最小输入

```

ReferenceLayerInput:
  x_in           // [T,H] or [1,H]
  rms_attn_gamma // [H]
  wq, wk, wv, wo
  rms_ffn_gamma  // [H]
  wgate, wup, wdown
  rope_cos, rope_sin
  causal_mask or visible_range
  kv_cache_in
  layer_config    // heads, kv_heads, head_dim, eps

```

一层 reference path 至少分成下面几步:

Listing 1.92 reference decoder layer 的语义步骤

```

1. h_attn = rmsnorm(x_in, rms_attn_gamma)
2. q = linear(h_attn, wq)
3. k = linear(h_attn, wk)
4. v = linear(h_attn, wv)
5. q, k = apply_rope(q, k, position)
6. kv_cache_out = append(kv_cache_in, k, v)
7. attn_out = attention(q, kv_cache_out, visible_range)
8. x_mid = x_in + linear(attn_out, wo)
9. h_ffn = rmsnorm(x_mid, rms_ffn_gamma)
10. g = linear(h_ffn, wgate)
11. u = linear(h_ffn, wup)
12. m = silu(g) * u
13. x_out = x_mid + linear(m, wdown)

```

这条路径的价值在于：它天然给出很多中间比较点。量化 kernel、fused kernel、paged KV cache、GQA kernel、packed int4 linear，都不需要一上来和最终文本比较；它们可以先和 reference path 的某一步比较。只要中间某一步开始漂，定位范围就会大幅缩小。

中间检查点应该怎么定

一个实用的 oracle 链，不会只在最后比较 **x_out**。至少应该暴露这些检查点：

检查点	张量形状	能定位的问题
h_attn	[N,H]	RMSNorm、输入 layout、dtype 转换
q/k/v	[N,heads,d]	linear、量化权重、head reshape
q_rope/k_rope	[N,heads,d]	RoPE、位置编号、左右 padding
scorerow	[visible]	GQA 映射、mask、量化 K 误差
probrow	[visible]	softmax、online normalization
attn_out	[N,H]	V 读取、KV 量化、head merge
x_mid	[N,H]	residual、Wo、accumulator dtype
g/u/m	[N,I]	MLP 权重、SiLU、逐元素乘
x_out	[N,H]	单层总输出合同

这样设计后，kernel 出现误差时就不会只得到一句“最终 logits 不一致”。例如：若 **q/k/v** 正确但 **q_rope/k_rope** 开始偏移，问题就在位置合同；若 **scorerow** 正确但 **probrow** 错，问题就在 softmax 或 mask；若 **attn_out** 正确但 **x_mid** 错，问题就在 **Wo** 或 residual。

单 token decode 的 reference 流程

对 decoder-only 推理，最有工程价值的不是整段 prefill 一次性 reference，而是单 token decode reference。因为大多数性能优化、KV cache 问题、长上下文问题，都发生在这个循环里。最小流程可以写成：

Listing 1.93 单 token decode 的 reference 流程

```

input:
  x_cur      // [1,H]
  kv_cache   // all previous positions
  pos        // absolute model position

for each layer:
  run reference decoder layer

```

```

update kv_cache

last_hidden = x_cur
logits = lm_head(last_hidden)

```

对这个流程，测试不能只断言“生成 token 对”。更好的做法是固定 prompt、固定 seed，把第 t 步 decode 的以下结果都落盘或比较：

Listing 1.94 decode oracle 的逐步证据

```

step t:
  layer 0 q checksum
  layer 0 score top entries
  layer 0 attn_out checksum
  layer L-1 x_out checksum
  logits top-k
  chosen next token

```

这样做虽然笨，但对工程极其有效。paged KV cache、GQA、int8/int4 KV、fused attention、speculative decode 验证失败时，都能回到第一个偏离的 step/layer/checkpoint。

把 reference decoder layer 落成 C++20 骨架

前面的步骤表已经说明“要算什么”，但工程上还需要回答“代码骨架怎么摆”。reference 路径最怕一开始就写成巨函数，把 tensor 视图、scratch buffer、checkpoint 和比较逻辑搅在一起。更稳的做法是先把数据边界钉死。

Listing 1.95 reference path 的最小数据结构骨架

```

enum class CheckpointKind {
    HAttn,
    Q,
    K,
    V,
    QRope,
    KRope,
    ScoreRow,
    ProbRow,
    AttnOut,
    XMid,
    G,
    U,
    M,
    XOut
};

struct TensorView final {
    const float* data = nullptr;
    std::array<int, 4> shape{};
    std::array<int, 4> stride{};
    int rank = 0;
};

struct MutableTensorView final {

```

```

float* data = nullptr;
std::array<int, 4> shape{};
std::array<int, 4> stride{};
int rank = 0;
};

struct LayerConfig final {
    int hidden_size = 0;
    int num_query_heads = 0;
    int num_kv_heads = 0;
    int head_dim = 0;
    int ffn_intermediate = 0;
    float rms_eps = 1.0e-5f;
};

struct CheckpointRecord final {
    CheckpointKind kind;
    std::vector<float> values;
    std::vector<int> shape;
};

```

TensorView 的目的不是炫技，而是强行把 reference path 和某个具体 runtime tensor 类解耦。reference 只需要“从哪里读、形状是什么、怎样走 stride”。这样 production path 用 arena、mmap、NUMA page、device buffer 都无所谓；reference 仍然能吃统一视图。

单层 reference 的输入输出可以进一步压成：

Listing 1.96 ReferenceLayer::run 的责任边界

```

struct ReferenceLayerInput final {
    TensorView x_in;
    TensorView rms_attn_gamma;
    TensorView wq, wk, wv, wo;
    TensorView rms_ffn_gamma;
    TensorView wgate, wup, wdown;
    TensorView rope_cos;
    TensorView rope_sin;
    int position = 0;
    int visible_begin = 0;
    int visible_end = 0;
};

struct ReferenceLayerOutput final {
    std::vector<float> x_out;
    std::vector<float> k_append;
    std::vector<float> v_append;
    std::vector<CheckpointRecord> checkpoints;
};

class ReferenceLayer final {
public:
    explicit ReferenceLayer(LayerConfig cfg) : cfg_(cfg) {}
    ReferenceLayerOutput run(const ReferenceLayerInput& in,
                           const KvCacheView& kv_in) const;
};

```



```
private:
    LayerConfig cfg_;
};
```

这里故意让 `run()` 返回 `k_append/v_append`, 而不是在内部偷偷写全局 cache。原因很直接: reference 的第一职责是把每一步算清楚, 第二职责才是和 runtime 状态衔接。若一开始就把 cache 写入隐藏在内部, `prefill/decode` 对齐和 `checkpoint` 定位都会变难。

真正的 `run()` 内部流程, 可以保持和数学合同一一对应:

Listing 1.97 ReferenceLayer::run 的最小伪代码

```
ReferenceLayerOutput ReferenceLayer::run(...) const {
    auto h_attn = rmsnorm(in.x_in, in.rms_attn_gamma, cfg_.rms_eps);
    save(HAttn, h_attn);

    auto q = linear(h_attn, in.wq);
    auto k = linear(h_attn, in.wk);
    auto v = linear(h_attn, in.wv);
    save(Q, q); save(K, k); save(V, v);

    auto q_rope = apply_rope(q, in.rope_cos, in.rope_sin, in.position);
    auto k_rope = apply_rope(k, in.rope_cos, in.rope_sin, in.position);
    save(QRope, q_rope); save(KRope, k_rope);

    auto score_row = attention_scores(q_rope, kv_in, k_rope,
                                      in.visible_begin, in.visible_end);
    save(ScoreRow, score_row);

    auto prob_row = softmax_stable(score_row);
    save(ProbRow, prob_row);

    auto attn_out = attention_reduce(prob_row, kv_in, v);
    save(AttnOut, attn_out);

    auto x_mid = residual_after_wo(in.x_in, attn_out, in.wo);
    save(XMid, x_mid);

    auto h_ffn = rmsnorm(x_mid, in.rms_ffn_gamma, cfg_.rms_eps);
    auto g = linear(h_ffn, in.wgate);
    auto u = linear(h_ffn, in.wup);
    auto m = silu_mul(g, u);
    save(G, g); save(U, u); save(M, m);

    auto x_out = residual_after_wdown(x_mid, m, in.wdown);
    save(XOut, x_out);
    return {...};
}
```

这段骨架要点不在“写了多少 helper 函数”, 而在于每一步都能独立落 checkpoint。生产后端可以融合 `QKV`、融合 `Wo+residual`、做 online softmax、做 paged KV、做低比特量化; 但 reference 层故意不融合, 因为它的职责是给每一条优化提供一个可对照的语义基线。

有了 layer 级骨架, 还需要一层专门做比较和报告:

Listing 1.98 decode oracle 的比较与报告骨架

```

struct CheckpointDiff final {
    CheckpointKind kind;
    float max_abs = 0.0f;
    float mean_abs = 0.0f;
    bool topk_changed = false;
};

struct DecodeStepReport final {
    int step = 0;
    int layer = 0;
    std::vector<CheckpointDiff> diffs;
    bool accepted = false;
    std::string first_failure;
};

class DecodeOracle final {
public:
    DecodeStepReport compare(const ReferenceLayerOutput& ref,
                           const ProductionLayerTrace& got,
                           const ToleranceProfile& tol) const;
};

```

这时 reference 和 oracle 的分工就非常明确：

- **ReferenceLayer** 只负责“算出可信结果并导出检查点”。
- **DecodeOracle** 只负责“按检查点规则比较并生成诊断”。
- production backend 只负责“按性能目标运行并暴露必要 trace”。

这个拆法在后面接 paged KV、GQA、int4 weight、int8 KV、FlashAttention 风格 softmax 时都不会塌。因为无论优化路径怎样变，最小闭环都还是：

同一输入 → reference checkpoints ↔ production checkpoints → diff report

没有这条骨架，所谓“oracle”往往只剩一句最终 logits 不一致，调试价值很低。

paged KV、GQA 和量化 KV 的 oracle 闭环

reference layer 只保存连续逻辑 KV 还不够。真实后端会把 KV cache 放进 page/block；GQA 会让多个 query head 共享一个 K/V head；KV 量化又会让 K/V 码点和 scale metadata 分开存。三者叠加后，错误不一定出现在 attention 算术本身，而可能出现在逻辑坐标到物理地址的映射。

先把生产路径必须暴露的 trace 字段列出来：

Listing 1.99 paged quantized KV 的最小 trace 字段

```

KvAccessTrace:
    step
    layer
    q_head
    kv_head
    model_position
    visible_begin
    visible_end
    page_id
    page_offset

```

```

slot_epoch
k_code_bytes
v_code_bytes
k_scale_id
v_scale_id
rope_position

```

这些字段看起来多，但每一个都对应真实故障。若 `q_head->kv_head` 映射错，是 GQA 错；若 `model_position` 和 `rope_position` 不一致，是位置合同错；若 `page_id/page_offset` 指向旧 slot，是 paged cache 或 sliding window 错；若 `k_scale_id` 指错，是 KV 量化 metadata 错。

reference 侧可以继续使用连续逻辑坐标：

Listing 1.100 reference KV 逻辑读

```

for q_head in query_heads:
    kv_head = q_head / gqa_group_size
    for pos in visible_begin..visible_end:
        k_ref = K_ref[layer, kv_head, pos]
        v_ref = V_ref[layer, kv_head, pos]
        score_ref[pos] = dot(q_ref[q_head], k_ref) / sqrt(head_dim)

```

production 侧可以使用 page table、量化码点和 scale：

Listing 1.101 production KV 物理读

```

for q_head in query_heads:
    kv_head = q_head / gqa_group_size
    for pos in visible_begin..visible_end:
        slot = page_table.lookup(session, pos)
        k_q = read_k_codes(slot, layer, kv_head)
        k_scale = read_k_scale(slot, layer, kv_head)
        k = dequantize(k_q, k_scale)
        score_got[pos] = dot(q_got[q_head], k) / sqrt(head_dim)

```

oracle 的闭环不是要求这两段代码长得一样，而是要求它们在同一逻辑坐标上给出可解释的差异。比较顺序应当从“坐标和字节解释”开始，而不是直接比较 logits：

Listing 1.102 paged KV oracle 的比较顺序

```

1. check visible set:
    reference visible positions == production visible positions

2. check GQA mapping:
    kv_head == q_head / group_size

3. check page mapping:
    logical position -> page_id/page_offset is valid
    slot epoch matches expected sequence

4. check quantized bytes:
    code bytes and scale ids are present
    dequantized K/V match reference tolerance

```

- ```

5. check attention:
 score row, prob row, attn_out

6. check layer/logits/token:
 x_out, logits top-k, chosen token

```

这套顺序能把错误压到第一个可解释边界。比如：

| 首个失败点           | 典型原因                              | 不该先怀疑什么     |
|-----------------|-----------------------------------|-------------|
| visible set     | mask、sliding window、prefix 保留策略   | 量化精度        |
| GQA mapping     | query head 到 KV head 映射错          | softmax     |
| page mapping    | page table、slot epoch、回绕复用        | linear 权重   |
| quantized bytes | scale id、nibble order、tail policy | RoPE        |
| score row       | K 量化、RoPE、head dim stride         | sampler     |
| prob row        | softmax、mask、online merge         | Wq/Wk       |
| attn_out        | V 读取、head merge、Wo 前布局            | logits head |

对 KV 量化，oracle 还要同时保存两类误差：反量化误差和 attention 行为误差。反量化误差回答“bytes 是否按合同解释”；attention 行为误差回答“这些误差是否已经改变 score 排名和概率分布”。一个报告可以这样组织：

Listing 1.103 KV quant oracle report 的核心字段

```

KvQuantOracleReport:
 context_length
 layer
 q_head
 kv_head
 visible_count
 max_k_dequant_error
 max_v_dequant_error
 score_max_abs_error
 score_top1_flipped
 prob_kl_divergence
 attn_out_max_abs_error
 first_bad_position
 first_bad_page
 first_bad_scale_id

```

这份报告比“logits mismatch”更能指导修复。若 `max_k_dequant_error` 很小但 `score_top1_flipped` 很多，说明问题不是字节解释，而是量化 profile 对该层/head 太激进。若 `first_bad_page` 总在 page 边界，优先查 page table、block size 和 scale metadata 布局。若只在 sliding window 回绕后失败，优先查 `slot_epoch`、base position 和 RoPE position。

### 核心思想

paged KV、GQA、KV 量化同时存在时，oracle 的第一任务不是证明最终文本像不像，而是证明“逻辑位置、K/V head、物理 page、量化 bytes、scale metadata、RoPE position”指向的是同一个历史 token。

### 一个失败定位例子：不是 softmax，是 scale page 错

假设某个优化后端在 4096 token 以内全部通过，到了 4097 token 后 logits 开始漂。只看最终输出，很容易误判为长上下文量化精度不够。按上面的 oracle 链排查，会得到更具体的证据：

Listing 1.104 失败报告示例

```
step: 4097
layer: 18
q_head: 12
kv_head: 3
first_failure: quantized bytes
logical_position: 4096
page_id: 64
page_offset: 0
slot_epoch: expected 1, got 0
k_scale_id: expected 524288, got 516096
score_max_abs_error: 0.184
score_top1_flipped: true
```

这个报告已经把方向压得很窄：第 4096 位置刚好落到新 page，page\_offset=0，slot\_epoch 和 k\_scale\_id 都错。此时不应该去改 softmax，也不应该调大 logits 容忍度；应该检查 page 分配、scale metadata page stride、以及 sliding window 或 prefix cache 是否复用了旧 slot。

如果修复后同一 case 变成：

Listing 1.105 修复后报告应该收敛到数值误差

```
visible set: pass
GQA mapping: pass
page mapping: pass
quantized bytes: pass
score_max_abs_error: 0.012
score_top1_flipped: false
attn_out_max_abs_error: 0.006
logits_topk_changed: false
```

这才说明错误从“合同错误”退化成“可接受的数值近似”。量化优化必须先通过合同层，再讨论误差阈值。

### prefill reference 不能只看最后一列

prefill 常被简化成“只看最后一个 token 的 logits”。这不够。prefill 一次性处理整个 prompt，任何一个中间位置的 causal mask、RoPE 位置或 attention 分块若出错，都可能在最后一列被部分掩盖。reference prefill 至少应该能检查：

- 第一个位置是否只看自己；
- 中间位置是否只看左侧历史；
- 最后位置的 logits 是否匹配；
- prefill 写入的 KV cache 是否与逐 token reference append 一致。

最后这一点很关键。一个正确的 prefill 实现，写出的  $K/V[0..T-1]$  应该和“把同一段 prompt 逐 token 跑 decode-style reference append”得到的结果一致。若两者不一致，后面的 decode 再快也建立在错误状态上。

## 常见误区

prefill 和 decode 不能各自有一套“差不多对”的 reference。它们必须在共享的 KV cache 逻辑合同上闭合，否则很容易出现 prefill 通过、decode 通过、但 prefill 后接 decode 的真实路径失败。

## 容忍度应该按检查点分层

reference oracle 不是要求所有路径 bitwise 相等。不同后端、不同累加顺序、fused kernel、online softmax 都可能带来合法的微小差异。关键是容忍度要按检查点分层，而不是用一个统一阈值糊过去。

| 检查点           | 更合理的比较方式                         | 原因                      |
|---------------|----------------------------------|-------------------------|
| 字节解释<br>q/k/v | 严格相等<br>max abs / relative error | 合同层，不应漂移<br>累加顺序和量化会有微差 |
| score row     | top entries + max error          | 小误差可能改变 softmax 排名      |
| prob row      | KL 或 top-1/top-k 位置              | 概率分布比逐元素更有意义            |
| x_out         | max abs / cosine similarity      | 层输出是连续值                 |
| logits        | top-k 变化 + max error             | 直接关系采样                  |
| token 序列      | 固定 seed 下完全一致或记录首个偏离步            | 离散行为最终验收                |

工程上最糟糕的情况是：前面所有连续值比较都在阈值内，但 logits 的 top-k 排名在某些长上下文点上开始翻转。这个时候不能简单说“误差很小所以没问题”，而应该把它报告成采样敏感点。

## C++20 项目里 oracle 链怎么落地

reference path 不应该塞进生产 kernel 文件里，更不应该被宏条件塞得到处都是。更好的结构是：

Listing 1.106 reference/oracle 的项目结构草图

```
reference/
reference_linear.cpp
reference_rmsnorm.cpp
reference_rope.cpp
reference_attention.cpp
reference_decoder_layer.cpp

oracle/
quant_byte_oracle.cpp
operator_oracle.cpp
layer_oracle.cpp
decode_oracle.cpp
```

这样做的好处是边界清楚。reference 模块只负责算出可信结果；oracle 模块只负责比较和报告；生产后端只负责跑快。需要时，CI 或本地 debug 可以把同一 prompt 同时送进 production path 和 reference path，在某个 checkpoint 失败后立即停止。

## 验收与评分标准

reference decoder layer 的最低门禁：

- 单层 decode 能输出 q/k/v、score、prob、attn\_out、x\_mid、x\_out。
- prefill 写出的 KV cache 能与逐 token reference append 对齐。
- 容忍度按字节解释、连续值、概率分布、logits 和 token 序列分层。



- 报告能指出首个偏离的 step、layer 和 checkpoint。

**oracle：量化误差要按层次报告**

量化 oracle 不能只给一个 **pass/fail**。它应该按层次报告误差，因为不同层次定位的问题不同。

| oracle 层次      | 比较对象                                                  | 能定位的问题                                         |
|----------------|-------------------------------------------------------|------------------------------------------------|
| 字节解释           | packed bytes 反量化值                                     | nibble order、scale、zero point、tail             |
| 单算子<br>层输出     | linear/attention/MLP 输出<br>decoder block hidden state | kernel、累加器、requantize<br>融合、arena 覆盖、layout 转换 |
| logits<br>生成序列 | top-k、分布、最大误差<br>固定 seed 的输出 token                    | 质量风险和采样敏感点<br>端到端可见行为变化                        |

初学者最容易困惑的是：为什么 logits 有小误差，生成序列却完全不同？原因是采样是离散决策。若两个 token 的概率非常接近，轻微 logits 变化就可能改变 top-k 排序或采样结果。因此量化报告不能只看平均误差，还要看 top-k 变化、KL 或分布差异，以及固定 sampler 下的序列差异。

Listing 1.107 量化 oracle 报告字段

```
QuantOracleReport:
 model_id
 quant_profile_id
 prompt_set_id
 context_length_buckets
 tensor_decode_failures
 operator_error_by_layer
 layer_output_error_by_position
 logits_max_abs_error
 logits_topk_changed
 generated_token_mismatch
 tolerance_profile
```

这份报告应该和 profiler 报告一起出现。一个 int4 优化如果速度提升 35%，但 top-k 在某些中文长上下文 prompt 中大量变化，就不能直接算合格；反过来，如果 logits 几乎不变但 decode 变慢，也不能叫优化。

**profiler：量化收益要拆成容量、带宽和计算**

量化常被宣传为“模型变小，所以更快”。这句话只说对了一部分。量化可能减少权重带宽，也可能增加解包、scale 读取和 requantize 成本。KV cache 量化可能减少 cache 容量和 attention 读带宽，也可能增加 metadata 读取和反量化成本。profiler 必须拆开这些项。

Listing 1.108 量化 profiler 的关键字段

```
QuantProfilerReport:
 original_weight_bytes
 quantized_weight_bytes
 scale_metadata_bytes
 packed_weight_bytes
 kv_cache_bytes_by_context
 kv_scale_metadata_bytes
 pack_time_ms
```

```

prefill_latency_ms
decode_latency_ms_by_context
dequant_time_ms
requant_time_ms
kernel_time_ms
memory_bandwidth_estimate

```

对 CPU，量化收益经常体现在减少内存读流，但如果解包逻辑让 SIMD pipeline 停顿，收益会下降。对 GPU，低比特格式若不能进入高效 tensor core 或 vendor kernel，可能会因为自定义解包 kernel 和同步开销而变慢。第二册讲过 cache、TLB、SIMD 和线程；本节的 profiler 字段就是把那些硬件事实拉回量化优化。

### 回归门禁：哪些测试必须常跑

量化改动风险高，不能靠人工看几条生成文本。C++20 项目至少应该有四类回归门禁：

- descriptor/verifier 单元测试：非法 group size、scale shape、tail policy、packed byte length 必须被拒绝。
- converter 测试：固定小张量的 int4/int8 bytes 反量化结果必须匹配 reference。
- kernel oracle 测试：常见 shape、tail shape、对齐/非对齐、不同 group size 都要覆盖。
- 端到端小模型测试：固定 prompt、固定 seed、固定 sampler，比较 logits 和生成序列合同。

性能也要有门禁，但不能用单次运行的绝对数字硬卡所有环境。更实用的是保存硬件信息、线程数、模型片段、prompt set 和相对基线，在同一机器上做回归判断。若一次改动让 decode latency 明显上升，CI 或本地 benchmark 应该提醒，而不是等到整本引擎集成后才发现。

### 验收与评分标准

量化功能进入默认路径前，至少要满足四条线：descriptor/verifier 能拒绝坏格式；reference converter 能解释 bytes；kernel oracle 能覆盖误差；profiler 能证明容量或速度收益。缺一条，就只能算实验分支。

### 硬验收

读完本节，应该能把量化从“脚本转换”提升为工程系统：

- QuantDescriptor 要显式记录 bit width、signedness、scale、zero point、granularity、axis、group、packed order 和 tail policy。
- CalibrationRecord 要保存校准数据、算法、范围、饱和和工具版本，不能只留下 scale 文件。
- verifier 要在热路径之前检查 metadata、shape、scale 数量、packed byte length 和后端支持。
- IR 要显式表达量化边界，让 pass、memory planner、lowering 和 oracle 都看得见。
- packing 是后端 lowering 的物理产物，必须有 packed descriptor 和诊断映射。
- oracle 要按字节解释、单算子、层输出、logits 和生成序列分层报告。
- profiler 要拆开容量、metadata、packing、dequant、requant、prefill 和 decode 成本。

到这里，量化大章不再只是数值公式，而是已经能进入 C++20 项目的模块设计、测试设计和验收设计。后续讲后端 micro-kernel 时，本节的 descriptor、oracle 和 profiler 会直接成为 kernel 合同的一部分。

## 1.8 7B 推理计算账本：FLOPs、字节、KV cache 与延迟上限

### 为什么必须算账

讨论本地 7B 推理时，最常见的空话是“优化矩阵乘”“减少显存”“提高 tokens/s”。这些话如果不落到 FLOPs、字节、cache、带宽和延迟，就无法指导实现。本节用一个典型 7B decoder-only GQA 模型建立账本。具体模型可能略有差异，但方法通用。

```

layers L = 32
hidden H = 4096
query_heads n_q = 32
kv_heads n_kv = 8
head_dim d = 128
intermediate I = 11008
vocab V = 32000
context S = 4096

```

这些数字不是装饰。它们决定每个 token 要读多少权重、KV cache 占多少内存、attention 随上下文怎样增长、CPU/GPU 后端为什么在 prefill 和 decode 阶段表现不同。

### 参数量账本

先算一层主要权重。Q projection 输出  $n\_q*d=4096$ ，K/V projection 各输出  $n\_kv*d=1024$ 。

| 权重    | 形状         | 参数量    |
|-------|------------|--------|
| Wq    | 4096x4096  | 16.78M |
| Wk    | 1024x4096  | 4.19M  |
| Wv    | 1024x4096  | 4.19M  |
| Wo    | 4096x4096  | 16.78M |
| Wgate | 11008x4096 | 45.09M |
| Wup   | 11008x4096 | 45.09M |
| Wdown | 4096x11008 | 45.09M |

一层线性权重大约：

$$16.78 + 4.19 + 4.19 + 16.78 + 45.09 + 45.09 + 45.09 \approx 177.2M$$

32 层约 **5.67B** 参数。加上 embedding、lm head、norm 和少量元数据，就接近 7B 级别。这个账本直接解释了 decode 为什么常常被权重读流支配：每生成一个 token，主要线性权重基本都要参与一次。

### 权重字节账本

参数量乘以每个权重的字节数，就是权重读流的粗略下界。

| 格式   | 7B 权重字节        | 备注                       |
|------|----------------|--------------------------|
| fp16 | 约 <b>14GB</b>  | 精度高，带宽压力大                |
| int8 | 约 <b>7GB</b>   | 还要加 scale/zero point     |
| int4 | 约 <b>3.5GB</b> | metadata 和 packing 会增加一些 |

若每个 decode token 都要流过大部分权重，那么 CPU 上限首先受内存带宽约束。假设有效内存带宽是 **100GB/s**，int4 权重加 metadata 后每 token 读 **3.7GB**，只看权重的理论上限也只有：

$$\frac{100}{3.7} \approx 27 \text{ tokens/s}$$

这还没有算 KV cache 读取、activation、线程同步、解包、scale、sampler、allocator 和调度开销。真实速度低于这个上限并不奇怪。这个公式比单纯说“CPU 慢”更有用，因为它告诉我们应该优先减少权重字节、改善 packing、提高带宽利用率，而不是盲目堆线程。

### decode 每层 FLOPs 账本

一次 decode 只处理一个新 token。线性层 MAC 数大约等于权重参数量。每层主要线性 MAC：

$$QKV = H(H + 2n_{kv}d) = 4096(4096 + 2048) \approx 25.17M$$

$$O = H^2 \approx 16.78M$$

$$MLP = 3HI = 3 \times 4096 \times 11008 \approx 135.27M$$

总计每层约：

$$25.17 + 16.78 + 135.27 \approx 177.22M \text{ MACs}$$

若把一次乘加按 2 FLOPs 计，每层约 **354MFLOPs**，32 层约 **11.36FLOPs/token**。attention 的点积和加权求和还要另算，随上下文长度  $S$  增长：

$$attention\_macs \approx 2 \times n_q \times S \times d$$

$S=4096$  时，一层 attention 约：

$$2 \times 32 \times 4096 \times 128 \approx 33.55M \text{ MACs}$$

32 层约 **1.07BMACs**。所以在长上下文下，attention 计算也很可观；但在很多 CPU decode 场景，低比特权重的解包和权重/KV 字节流仍然是更早撞到的墙。

### KV cache 容量账本

KV cache 每个 token、每层要保存 K 和 V。GQA 下保存的是  $n_{kv}$  个 head，不是  $n_q$  个 head。

$$kv\_bytes\_per\_token\_per\_layer = 2 \times n_{kv} \times d \times bytes$$

fp16 下每个元素 2 字节：

$$2 \times 8 \times 128 \times 2 = 4096 \text{ bytes}$$

32 层就是每个 token **128KiB**。上下文 **4096** 时，一个 session 的 KV cache 是：

$$4096 \times 128KiB = 512MiB$$

这个数字对业务开发非常关键。一个本地服务如果同时接 8 个 **4096** context 的会话，仅 KV cache 就可能接近 **4GiB**，还不包括权重、packed weight、activation arena、workspace、线程栈和系统余量。

| context     | 单 session fp16 KV | 8 session fp16 KV |
|-------------|-------------------|-------------------|
| <b>1024</b> | <b>128MiB</b>     | <b>1GiB</b>       |
| <b>2048</b> | <b>256MiB</b>     | <b>2GiB</b>       |
| <b>4096</b> | <b>512MiB</b>     | <b>4GiB</b>       |
| <b>8192</b> | <b>1GiB</b>       | <b>8GiB</b>       |

这就是为什么 admission control 必须在请求进入前估算 KV cache，而不是等到 decode 过程中分配失败。

### KV cache 读取账本

容量只是第一半。decode 每步还要读取历史 K/V。当前 token 在每层 attention 中读取长度为  $S$  的 K 和 V：

$$kv\_read\_bytes\_per\_layer = S \times 2 \times n_{kv} \times d \times bytes$$

$S=4096$ 、fp16 时每层约 **16MiB**，32 层约 **512MiB/token**。也就是说，在长上下文下，即使用 int4 权重，每生成一个 token 还要额外读约半 GB 的 KV cache。这解释了两个现象：

- 上下文越长，decode token 越慢，不只是 attention FLOPs 增加，KV 读字节也线性增加。
- KV cache 量化可能有效，因为它减少的是每步反复读取的历史状态，而不是只减少静态权重。

但 KV 量化比权重量化更敏感。权重是固定的，可以离线校准；KV 是动态 activation，每个请求、每个位置都不同。若 scale 选择不好，attention score 会被放大后进入 softmax，少量误差可能改变 top-k。后面的低比特 KV 章节会专门展开。

### Attention 每 token 的字节账本

上一节的 **512MiB/token** 是“唯一 K/V 元素至少要读到”的下界。真正 kernel 的流量还要看实现是否能在 query head 之间复用 K/V、是否物化 score、是否把 softmax 临时写到内存、是否分页读取 KV。先把 decode attention 拆成四段：

1. 读取历史 K，计算  $q \cdot k$  score。
2. 对 score 加 mask、scale，并做 softmax。
3. 读取历史 V，做加权求和。
4. 写当前 token 的 K/V 到 cache，写 attention 输出。

对本节 7B 形状：

Listing 1.110 7B attention 参数

```
L = 32
n_q = 32
n_kv = 8
d = 128
S = 4096
KV dtype = fp16, bytes = 2
```

每层唯一 K 读取下界是：

$$K_{unique} = S \times n_{kv} \times d \times bytes = 4096 \times 8 \times 128 \times 2 = 8MiB$$

V 也是 **8MiB**，所以每层唯一 K/V 读取下界是 **16MiB**，32 层就是 **512MiB/token**。但一个朴素 per-query-head kernel 可能按 **n\_q** 重新读取 K/V：

$$KV_{logical\_per\_q} = S \times 2 \times n_q \times d \times bytes = 4096 \times 2 \times 32 \times 128 \times 2 = 64MiB/layer$$

32 层就是 **2GiB/token**。两者差了 4 倍，正好是  $n_q/n_{kv} = 4$ 。这不是数学语义差异，而是 kernel 数据复用差异：GQA 降低了唯一 KV 容量；kernel 还要真正把同一个 KV head 在同组 query heads 之间复用，才能接近下界。

| 账本项              | 每层      | 32 层    | 说明              |
|------------------|---------|---------|-----------------|
| 写当前 K/V          | 4 KiB   | 128 KiB | 每 token 新增状态    |
| 唯一 K 读           | 8 MiB   | 256 MiB | score 阶段下界      |
| 唯一 V 读           | 8 MiB   | 256 MiB | weighted sum 下界 |
| 唯一 K/V 合计        | 16 MiB  | 512 MiB | 理想复用下界          |
| per-query 逻辑 K/V | 64 MiB  | 2 GiB   | 朴素 head 循环可能接近  |
| score 临时 fp32    | 512 KiB | 16 MiB  | 若物化 $[n_q, S]$  |

这张表解释了为什么“KV cache 已经用了 GQA”并不自动等于 attention 带宽最优。容量账本按 **n\_kv** 算；坏 kernel 的实际读流可能更接近 **n\_q**。因此 attention benchmark 不能只报 KV cache 总容量，还要报告 kernel 的读流组织：按 query head 单独扫历史，按 KV head 分组扫历史，还是 tile 后把 K/V 放入 cache/shared memory 再给同组 query heads 复用。

### context length 的斜率

decode attention 的关键不是某个固定数，而是随上下文长度  $S$  线性增长的斜率。对 fp16 GQA 7B，下界是：

$$\text{bytes/token} = L \times S \times 2 \times n_{kv} \times d \times \text{bytes}$$

代入  $L = 32, n_{kv} = 8, d = 128, \text{bytes} = 2$ ：

$$\text{bytes/token} = S \times 131072 \text{ bytes}$$

也就是上下文每增加 1024 token，decode 每个新 token 至少多读约 **128MiB** 的 KV。

| context $S$ | fp16 GQA KV 读下界 | 只按 100 GB/s 带宽的时间下界 |
|-------------|-----------------|---------------------|
| 1024        | 128 MiB/token   | 1.34 ms/token       |
| 4096        | 512 MiB/token   | 5.37 ms/token       |
| 8192        | 1 GiB/token     | 10.7 ms/token       |
| 16384       | 2 GiB/token     | 21.5 ms/token       |

这个时间下界只算 KV 读，还没有算权重读取、解包、RMSNorm、MLP、sampler、线程同步、分页 block table 和调度排队。若长上下文 TPOT 随  $S$  接近线性增长，首先要检查这个 KV 斜率，而不是直接怀疑 sampler 或 tokenizer。

### MHA、GQA、MQA 的容量和读流差异

GQA/MQA 的直接收益是减少 KV heads。若其他参数不变， $n_{kv}$  从 32 降到 8，KV 容量和唯一 KV 读下界都减少 4 倍；降到 1，则减少 32 倍。

| 形态  | $n_{kv}$ | 4096 context 单 session KV | decode 唯一 KV 读/token |
|-----|----------|---------------------------|----------------------|
| MHA | 32       | 2 GiB                     | 2 GiB                |
| GQA | 8        | 512 MiB                   | 512 MiB              |
| MQA | 1        | 64 MiB                    | 64 MiB               |

但这不是免费午餐。更少的 KV heads 改变模型结构和质量边界；更少的唯一 KV 元素也不保证 kernel 实际读流等比例下降。报告必须同时写“模型结构账本”和“kernel 实测账本”：前者来自 descriptor，后者来自 benchmark、profile 或至少来自代码地址流审计。

### 三种 decode attention 读流：朴素、分组复用和分页

上一节的表只给出容量和唯一元素下界。真正做后端时，还要把 kernel 的读取顺序写出来。对 GQA 7B， **$n_q=32, n_{kv}=8$** ，每个 KV head 服务 4 个 query head。三种实现的字节账本完全不同。

**朴素 per-query-head 扫描。** 最直接的写法是外层遍历 query head，内层扫全部历史位置：

Listing 1.111 朴素 GQA decode attention 读流

```

for qh in 0..n_q:
 kvh = qh / (n_q / n_kv)
 for pos in 0..S:
 read K[layer, pos, kvh, :]
 score[pos] = dot(Q[qh, :], K[pos, kvh, :])
 probs = softmax(score)
 for pos in 0..S:
 read V[layer, pos, kvh, :]
 out[qh, :] += probs[pos] * V[pos, kvh, :]

```



这个版本数学上正确，却可能把同一个 KV head 的 K/V 为同组 4 个 query head 重读 4 次。模型结构用了 GQA，kernel 读流却接近 MHA。

**按 KV head 分组复用。** 更好的 CPU/GPU kernel 会把同一个 KV head 下的 query head 分成一组。每次读取一个历史 K/V 向量后，给 4 个 query head 同时使用。这样唯一 K/V 读流接近下界，但代价是寄存器、accumulator 和 softmax 状态更多：

Listing 1.112 按 KV head 分组的读流

```
for kvh in 0..n_kv:
 q_group = query_heads_mapped_to(kvh) // 4 heads
 for pos in 0..S:
 k = read K[layer, pos, kvh, :]
 update score for all qh in q_group
 online_softmax_or_materialized_softmax(q_group)
 for pos in 0..S:
 v = read V[layer, pos, kvh, :]
 update out for all qh in q_group
```

分组复用不是免费。若 4 个 query head 的 score 全部物化，临时内存仍是  $[group, S]$ ；若用 online softmax，要维护每个 query head 的 running max、running sum 和 partial output。它节省的是历史 K/V 重读，不是消除 attention 计算。

**paged KV attention。** Paged KV 把连续逻辑位置映射到物理 block/page。它解决的是多 session、变长上下文、prefix 共享和内存碎片问题，不会让历史 K/V 本身消失。读流会多一层 block table：

Listing 1.113 paged KV 的额外地址流

```
for logical_pos in visible_range:
 block_id = block_table[logical_pos / block_size]
 block_offset = logical_pos % block_size
 physical = kv_pages[block_id] + block_offset * kv_stride
 read K/V from physical
```

如果 block table 常驻 cache，这层开销可能很小；如果 session 很多、block table 和 KV page 分散、访问跨 NUMA 或跨 GPU page，paged layout 会增加 TLB、cache 和地址计算压力。paged attention 的正确评价不是“分页一定快”，而是看它用少量间接寻址换来了更高并发和更低碎片。

| 实现形态                               | K/V 主读流                       | 额外读写                                  | 适合场景                                      |
|------------------------------------|-------------------------------|---------------------------------------|-------------------------------------------|
| per-query 扫描<br>按 KV head 分组       | 接近 $n_q$ 重读<br>接近 $n_{kv}$ 下界 | score 临时简单<br>多 head softmax 状态       | 教学 baseline<br>GQA/MQA decode             |
| online softmax                     | 不物化完整 score                   | running max/sum/out                   | 长上下文、显存紧张                                 |
| paged KV<br>FlashAttention 风格 tile | K/V 仍按历史读取<br>减少 score/HBM 往返 | block table、page stride<br>tile 状态和同步 | 多 session serving<br>prefill、大块 attention |

这里要小心一个常见误解：FlashAttention 风格的在线 softmax/tile 技术主要减少 score 矩阵和中间概率的物化、提高片上复用。对 decode 单 token，它通常不能突破“历史 K/V 必须被读到”的下界；它能做的是避免把  $[n_q, S]$  score/prob 写回大内存，并改善 query head 分组、tile 和 cache/shared memory 复用。长上下文 decode 的第一性瓶颈仍然要从 KV bytes/token 开始算。

## 把 attention 读流写进 profiler schema

Attention benchmark 只报 **tokens/s** 不够。至少要让 profiler 输出下面这些字段，才能区分“模型结构账本”和“kernel 实际读流”：

**Listing 1.114** decode attention profile 的关键字段

```
DecodeAttentionProfile:
 layer
 context_length
 q_heads
 kv_heads
 head_dim
 kv_dtype
 layout: contiguous | paged
 kernel_mode: per_query | grouped_kv | online_softmax | library
 kv_unique_bytes_lower_bound
 kv_estimated_read_bytes
 score_temp_bytes
 block_table_bytes
 q_group_size
 tile_size
 elapsed_us
 effective_kv_bandwidth_gbps
 correctness_oracle: score | prob | attn_out | logits
```

若 **kv\_estimated\_read\_bytes/kv\_unique\_bytes\_lower\_bound** 接近  $n_q/n_{kv}$ ，说明 GQA 的结构收益没有被 kernel 读流吃到。若 **score\_temp\_bytes** 随  $S$  很大，说明没有使用 online softmax 或 tile 内归约。若 **block\_table\_bytes** 很小但耗时仍高，问题更可能在 K/V 主读流、TLB、NUMA、GPU page locality 或线程组织，而不是 block table 本身。

### 常见误区

Paged KV、FlashAttention 风格 online softmax 和 GQA 是三类不同优化：paged KV 管内存管理和并发，online softmax 管中间矩阵物化，GQA 管模型结构里的 KV heads 数量。把它们混成一句“attention 优化了”，会让性能报告失去诊断能力。

### prefill 和 decode 不是同一账本

prefill 处理整个 prompt。若 prompt 长度  $T=512$ ，一层 linear 变成  $[512, H] \times [H, 0]$  的矩阵乘。权重被同一批 token 复用，GEMM 能更好利用计算单元。decode 每步  $T=1$ ，权重复用少，容易被权重读流限制。

| 阶段      | 主要形态                | 常见瓶颈                   |
|---------|---------------------|------------------------|
| prefill | 大 GEMM、批量 attention | 算力、显存带宽、attention tile |
| decode  | GEMV/小 GEMM、KV 读取   | 权重带宽、KV 带宽、launch/同步   |

这就是为什么一个后端可能 prefill 很快、decode 一般；也可能 CPU decode 经过 int4 和线程优化后可用，但长 prompt prefill 明显慢。报告必须分开写 TTFT、prefill tokens/s 和 decode tokens/s。

用一个  $4096 \times 4096$  fp16 projection 看 arithmetic intensity，差异会更直观。decode 时  $T=1$ ：

$$flops = 2 \times 1 \times 4096 \times 4096 \approx 33.6M$$

只读权重就约 **32MiB**，输入和输出各约 **16KiB**，所以强度约 **1FLOP/byte**。prefill 若  $T=512$ ，同一个权重矩阵被 512 个 token 复用：

$$flops = 2 \times 512 \times 4096 \times 4096 \approx 17.2G$$

权重约 **32MiB**，输入和输出各约 **4MiB**，粗略强度超过 **400FLOP/byte**。真实 GEMM 会因为 cache、tiling、layout 和 batch 细节打折，但方向不会变：prefill 更容易吃满算力，decode 更容易被字节流、launch 和同步支配。

#### roofline：用带宽和算力约束 tokens/s

粗略上限可以用 roofline 思路判断。对每个 decode token，假设需要读：

Listing 1.115 decode token 的粗略字节项

```
weight_bytes_per_token
kv_read_bytes_per_token(context)
activation_bytes
metadata_bytes
temporary_workspace_bytes
```

带宽上限：

$$tokens/s \leq \frac{effective\_bandwidth}{bytes\_per\_token}$$

算力上限：

$$tokens/s \leq \frac{effective\_flops}{flops\_per\_token}$$

实际上限取二者较小者，再扣除线程调度、解包、sampler、同步和 cache miss。举例：int4 权重 **3.7GB/token**，长上下文 KV **0.5GB/token**，总计约 **4.2GB/token**。若有效带宽 **100GB/s**：

$$\frac{100}{4.2} \approx 23.8 \text{ tokens/s}$$

若同一机器由于解包、NUMA、TLB、线程调度只能达到 **55GB/s** 有效带宽，上限就是 **13tokens/s**。这比“开更多线程”更能解释问题：线程只能提高并行利用率，不能凭空减少每 token 字节数。

#### decode tokens/s 上限表

把上面的账本压成一张表。这里使用十进制 GB 粗算，**S=4096** 时 fp16 KV 读取约 **0.5GB/token**；权重按 **fp16=14GB**、**int8=7GB**、**int4+metadata=3.7GB** 估算。因此 decode 每 token 最少读：

| 格式   | 权重 GB/token | KV GB/token | 合计 GB/token |
|------|-------------|-------------|-------------|
| fp16 | 14.0        | 0.5         | 14.5        |
| int8 | 7.0         | 0.5         | 7.5         |
| int4 | 3.7         | 0.5         | 4.2         |

只按带宽算，理论上限如下：

| 有效带宽                    | fp16 tok/s | int8 tok/s | int4 tok/s |
|-------------------------|------------|------------|------------|
| <b>50GB/s</b> CPU       | 3.4        | 6.7        | 11.9       |
| <b>100GB/s</b> CPU      | 6.9        | 13.3       | 23.8       |
| <b>200GB/s</b> 高端 CPU   | 13.8       | 26.7       | 47.6       |
| <b>600GB/s</b> 入门 GPU   | 41.4       | 80.0       | 142.9      |
| <b>1000GB/s</b> GPU     | 69.0       | 133.3      | 238.1      |
| <b>1600GB/s</b> 高带宽 GPU | 110.3      | 213.3      | 381.0      |

这张表不是性能承诺，而是上限。实际系统会继续扣掉：低比特解包、scale metadata、cache miss、TLB miss、线程同步、sampler、batch 调度、GPU launch、host-device staging 和 fallback。若实测 int4 decode 只有 **8tok/s**，不能直接说实现失败；要先看有效带宽是不是只有 **35GB/s**，以及时间是否被解包或调度吃掉。

### 算力上限通常不是 decode 的第一堵墙

前面估算  $S=4096$  时，线性层和 attention 合起来约 **13.5GFLOPs/token**。若只按算力上限：

| 有效算力                  | 理论 tok/s | 解释                   |
|-----------------------|----------|----------------------|
| <b>0.5TFLOP/s</b> CPU | 37       | 标量/低效 SIMD 可能达不到     |
| <b>1TFLOP/s</b> CPU   | 74       | 已经高于很多带宽上限           |
| <b>10TFLOP/s</b> GPU  | 740      | decode 小 batch 未必能吃满 |
| <b>50TFLOP/s</b> GPU  | 3700     | launch/KV/带宽可能先限制    |

如果同一配置的带宽上限只有 **24tok/s**，而算力上限是 **74tok/s**，首要问题就不是“FLOPs 不够”，而是每 token 读了太多字节或没有把带宽用起来。CPU micro-kernel 章节里的 **4096x4096** 实测会继续把这个差距拆开：同样是 int8 权重，标量、packed scalar 和 AVX2 的有效带宽差异很大。

### prefill 上限要按 prompt 维度重算

prefill 不能套用 decode 的 **GB/token**。若  $T=512$ ，权重读流被 512 个 token 分摊，单 token 的权重字节近似变成：

| 格式   | 权重总字节        | 摊到每 prompt token  |
|------|--------------|-------------------|
| fp16 | <b>14GB</b>  | <b>27MB/token</b> |
| int8 | <b>7GB</b>   | <b>14MB/token</b> |
| int4 | <b>3.7GB</b> | <b>7MB/token</b>  |

但 prefill 的 attention 是  $O(T^2)$ ，而且 TTFT 还包含 tokenizer、模型 prepare、KV cache 初始化、可能的 packing 和调度排队。因此 prefill report 至少要拆成：

Listing 1.116 prefill report 最少字段

```
prompt_tokens
tokenizer_ms
prepare_or_pack_ms
prefill_compute_ms
prefill_tokens_per_s
kv_write_bytes
ttft_ms
```

只报“首 token 很慢”没有诊断价值。若慢在 **prepare\_or\_pack\_ms**，优化 kernel 没用；若慢在 **prefill\_compute\_ms**，要看 GEMM/attention；若慢在 **tokenizer\_ms** 或排队，问题已经进入 serving 层。

### 量化为什么有效，为什么也危险

权重量化主要减少静态权重字节。int8 把 fp16 权重从 2 字节变成 1 字节；int4 变成 0.5 字节，再加 scale metadata。若 decode 被权重带宽限制，低比特能直接提高上限。

但量化不是免费午餐。计算中会出现：

- 解包成本：int4 需要从 byte 中取 nibble。
- scale 成本：per-group 或 per-channel scale 要参与反量化。
- accumulator 精度：低比特乘法需要更宽累加或合理 requantize。

- 误差传播：Q/K/V、MLP、lm head 对最终 logits 的敏感度不同。
- metadata 读流：scale、zero point 和 group table 也占带宽。

所以量化报告不能只写“模型从 14GB 变成 3.7GB”。它至少要写：

Listing 1.117 量化收益报告字段

```
original_weight_bytes
packed_weight_bytes
scale_metadata_bytes
decode_tokens_per_s
prefill_tokens_per_s
operator_error
logits_topk_overlap
generation_diff_under_fixed_seed
```

低比特格式能不能进入业务服务，必须由容量、速度和质量三者共同决定。

### 并发容量账本

业务服务要算的是总内存，不是单请求。一个粗略公式：

Listing 1.118 服务内存预算

```
total =
 model_weight_bytes
+ packed_weight_bytes
+ backend_static_workspace
+ concurrent_sessions * kv_cache_bytes_per_session
+ concurrent_sessions * activation_arena_bytes
+ trace_buffers
+ safety_margin
```

假设 int4 packed 权重和 metadata 约 **4.0GB**，静态 workspace **0.5GB**，每个 **4096** context session 的 KV 为 **512MiB**，每个 session arena 和 trace 估计 **128MiB**，机器可给进程使用 **12GB**。最多并发不能简单算  $12/0.5=24$ ，而应先扣静态项：

$$12 - 4.0 - 0.5 - \text{safety} \approx 6.5GB$$

每 session 约 **640MiB**，最多约 10 个，还要考虑 allocator 碎片和峰值。因此 admission control 应该保守，并按 **max\_context\_tokens** 预留，而不是按当前 prompt 长度乐观估计。

### 把账本写进 trace

计算账本如果只在书里，工程上仍然不可用。trace event 应该包含估计字节和实际耗时：

Listing 1.119 账本相关 trace 字段

```
TraceEvent:
 stage
 layer_id
 segment_id
 context_length
 estimated_flops
 estimated_weight_bytes
 estimated_kv_read_bytes
 estimated_kv_write_bytes
```

```
elapsed_ns
backend
quant_profile_id
```

有了这些字段，报告就能回答：

- decode 变慢是上下文变长导致 KV 读增加，还是后端 fallback。
- int4 变快是否符合权重字节下降的预期。
- prefill 慢是 GEMM 吞吐问题，还是 tokenizer/prepare 被算进了 TTFT。
- 某层异常慢是否因为 layout 转换或 workspace 分配。

### 把账本绑定到可运行 artifact

本章的公式不能只停在正文里。当前 lab 新增了 `lcqi_ledger`，固定本节使用的 7B 形状，直接导出 FLOPs、KV 容量、每 token 字节和带宽上限：

Listing 1.120 7B ledger 复现命令

```
books/cpu-volume-3/build/lcqi-release/labs/linux_cpu_inference/lcqi_ledger \
> books/cpu-volume-3/results/lcqi-sevenb-ledger-2026-06-24.csv
```

CSV 字段很少，故意保持稳定：

Listing 1.121 lcqi ledger CSV 字段

```
section,item,value,unit,notes
```

真实输出中的关键行如下：

Listing 1.122 lcqi ledger CSV 摘要

```
compute,linear_macss_per_layer,177209344.000,macs,decode token
compute,attention_macss_per_layer,33554432.000,macs,context=4096
compute,total_decode_flops_per_token,13488881664.000,flops,linear plus ↵
↵ attention
kv,fp16_capacity_context_4096,512.000,MiB,single session
decode_bytes,int4_total_gb,4.237,GB/token,weight plus KV
bandwidth_limit,int4_at_100_gbps,23.602,tokens/s,bandwidth-only upper bound
```

这几行把本章的核心判断变成了回归对象：`context=4096` 的 fp16 KV 单 session 是 **512MiB**，int4 权重加 fp16 KV 的粗略读流是 **4.237GB/token**，有效带宽 **100GB/s** 的带宽上限是 **23.602tok/s**。CTest 会检查 `int4_at_100_gbps` 这一行；若后续有人改了模型形状、KV dtype 或权重字节假设，账本和测试会一起暴露变化。

### 硬验收

读完本节，应该能自己给一个 7B 模型算粗账：

- 根据 `L,H,n_q,n_kv,d,I,S` 估算每层参数量和 decode FLOPs。
- 估算 fp16、int8、int4 权重字节，并用带宽上限推 tokens/s。
- 计算单 session KV cache 容量和多 session 并发内存。
- 区分 prefill 的 GEMM 账本和 decode 的 GEMV/KV 读账本。
- 解释为什么长上下文会增加 KV 读字节和 attention 计算。
- 把 FLOPs、权重字节、KV 字节和耗时写进 trace，而不是只报最终速度。



## 第 2 章

# Transformer、KV Cache、采样与服务运行时

## 2.1 AI 推理原理、KV cache 与计算账本：从一句 prompt 到本地 7B 引擎

先把问题讲清楚：模型不是在“理解一句话”

本节从最小问题开始：用户输入一句话，本地大模型怎样生成下一个 token？如果这件事讲不清，张量、量化、KV cache、CPU/GPU 后端和 benchmark 都会变成孤立名词。

假设用户输入：

Listing 2.1 一个最小生成请求

```
prompt: "C++ 为什么需要内存模型？"
```

推理引擎不会直接处理这串字符。第一步是 tokenizer，把文本切成 token id。token id 是整数，不一定对应一个汉字、一个英文单词或一个完整词。第二步是 embedding，把每个 token id 映射成一个 hidden 向量。第三步是多层 decoder block，不断改写每个位置的 hidden。第四步是 `lm_head`，把最后一个位置的 hidden 投影成词表大小的 logits。第五步是 sampler，从 logits 里选出下一个 token id。然后把这个新 token 追加到序列尾部，继续下一轮。

Listing 2.2 从 prompt 到 next token 的主链路

```
text
-> tokenizer.encode(text)
-> token ids
-> embedding vectors
-> repeated decoder blocks
-> final hidden at last position
-> lm_head logits over vocabulary
-> sampler chooses next token id
-> append token and repeat
```

### 核心思想

decoder-only 大模型推理的核心任务不是一次输出整篇文章，而是不断重复同一个步骤：给定已有 token 序列，预测下一个 token 的分数分布，再由 sampler 选出一个 token。

这句话决定了本节后面的所有机器账本。因为一次只追加一个 token，decode 阶段天然有强状态性；因为每一步都要看历史 token，attention 会读取越来越长的上下文；因为历史 K/V 可以复用，KV cache 变成推理引擎的核心运行时状态；因为每步都要过所有层，权重读流和矩阵向量乘会支配 decode 性能。

### 训练目标和推理目标：为什么模型会预测下一个 token

decoder-only 语言模型在训练时常用一个 token 预测任务。给定一段 token 序列：

Listing 2.3 训练样本的移位关系

```
input: t0, t1, t2, t3, ...
```

```
target: t1, t2, t3, t4, ...
```

模型在位置  $i$  只能看见  $0..i$  的历史，然后预测  $i+1$ 。这叫 **因果语言建模** (causal language modeling)。因果的意思是当前位置不能偷看未来 token。推理时也沿用同一规则：已有 token 是上下文，模型预测下一个 token。

训练和推理的区别在于，训练知道整段 target，可以一次计算很多位置的损失；推理不知道未来，只能生成一个 token 后再把它加入上下文。因此推理有两个阶段：

| 阶段      | 输入形态                | 目标                                  |
|---------|---------------------|-------------------------------------|
| prefill | 用户 prompt 的多个 token | 批量处理 prompt，填好 KV cache，得到首个 logits |
| decode  | 每次一个新 token         | 追加 K/V，读取历史 cache，生成下一个 logits      |

这不是 API 名称差异，而是计算形态差异。prefill 能把很多 token 一起送进矩阵乘，更像 GEMM；decode 每步只有一个 token，更像 GEMV 或小 GEMM，并且要读历史 KV cache。很多人以为“模型推理就是跑一遍网络”，这句话只对 prefill 的直觉接近，对 decode 不够准确。decode 是一个带状状态的循环。

### token、embedding、hidden、logits 和 sampler

先把常见对象定义清楚。

**token id**。tokenizer 把文本映射成整数序列。一个中文词、英文词、标点、空格、子词片段都可能对应一个或多个 token。业务开发中经常要关心 token 数，因为计费、上下文长度、KV cache 容量和 latency 都按 token 变化。

**embedding**。embedding 表可以理解成一个矩阵  $E[\text{vocab}, H]$ 。token id 查表后得到长度为  $H$  的向量。若 prompt 有  $T$  个 token，embedding 输出形状就是  $[T, H]$ 。

**hidden**。hidden 是模型内部的向量表示。它不是概率，也不是文本。每个 decoder block 接收 hidden，输出新的 hidden。hidden size  $H$  是计算量和内存量的核心参数之一。

**logits**。`lm_head` 把最后位置的 hidden 投影成  $[\text{vocab}]$  分数。logits 没有归一化，softmax 后才变成概率。做贪心解码时选最大 logits；做温度、top-k、top-p 采样时会改变采样分布。

**sampler**。sampler 不是模型主体。它是从 logits 选择 token 的策略。工程上必须把“模型算错”和“采样策略不同”分开定位。固定 seed、temperature、top-k、top-p 后，同一 logits 应该产生可复现的 token 序列。

| 对象        | 形状示例                | 归属                      |
|-----------|---------------------|-------------------------|
| token ids | $[T]$               | tokenizer/runtime       |
| embedding | $[T, H]$            | 模型权重 + activation       |
| hidden    | $[T, H]$ 或 $[1, H]$ | decoder block 中间状态      |
| logits    | $[\text{vocab}]$    | <code>lm_head</code> 输出 |
| next id   | 标量整数                | sampler 输出              |

### decoder block 的骨架

多数 7B 级本地模型可以先按 decoder-only Transformer 解释。不同模型会在 RMSNorm/LayerNorm、RoPE、SwiGLU、GQA/MQA、量化格式和词表上有差异，但一层的骨架大致相同：

**Listing 2.4** 一个 decoder block 的高层账本

```
x = input hidden

h = norm(x)
```

```

q = h * Wq
k = h * Wk
v = h * Wv
q, k = apply_position_encoding(q, k, position)
append k, v to KV cache
a = attention(q, K_cache[0..position], V_cache[0..position])
x = x + a * Wo

h = norm(x)
g = h * Wgate
u = h * Wup
m = activation(g) * u
x = x + m * Wdown

```

这段伪代码里，最重的计算通常是线性投影：**Wq/Wk/Wv/Wo/Wgate/Wup/Wdown**。它们本质上是矩阵乘或矩阵向量乘。RMSNorm、RoPE、激活函数和残差加法算术量小一些，但会产生 activation 读写；如果这些读写没有融合，也会消耗内存带宽。

这里第一次出现了 KV cache。当前 token 产生的 **k** 和 **v** 不能像普通临时 activation 一样丢掉，因为未来 token 的 attention 会读取它们。KV cache 把推理引擎从“无状态算子流水线”变成“跨 token 状态机”。

### 单层 decoder 的数学合同

把上面的伪代码写成数学合同，才能判断一个推理引擎有没有算对。设第 **l** 层输入为 **X<sub>l</sub>**，形状是 **[N, H]**。prefill 时 **N=T**，decode 时 **N=1**。RMSNorm 先对每个 token 的 hidden 做归一化：

$$\hat{x} = \frac{x}{\sqrt{\frac{1}{H} \sum_{i=1}^H x_i^2 + \epsilon}} \odot \gamma$$

然后做 Q/K/V 投影。若 query head 数是 **n<sub>q</sub>**，K/V head 数是 **n<sub>kv</sub>**，每个 head 的维度是 **d**，通常有 **H=n<sub>q</sub>\*d**：

| 权重        | 形状                           | 输出                               |
|-----------|------------------------------|----------------------------------|
| <b>Wq</b> | <b>[H, n<sub>q</sub>*d]</b>  | <b>Q: [N, n<sub>q</sub>, d]</b>  |
| <b>Wk</b> | <b>[H, n<sub>kv</sub>*d]</b> | <b>K: [N, n<sub>kv</sub>, d]</b> |
| <b>Wv</b> | <b>[H, n<sub>kv</sub>*d]</b> | <b>V: [N, n<sub>kv</sub>, d]</b> |
| <b>Wo</b> | <b>[n<sub>q</sub>*d, H]</b>  | attention 输出回到 <b>[N, H]</b>     |

GQA 的关键是 **n<sub>q</sub>** 可以大于 **n<sub>kv</sub>**。若 **n<sub>q</sub>=32**、**n<sub>kv</sub>=8**，每 4 个 query head 共享同一个 K/V head。可以写成：

$$g(h) = \left\lfloor \frac{h}{n_q/n_{kv}} \right\rfloor$$

其中 **h** 是 query head 编号，**g(h)** 是它读取的 K/V head 编号。这个映射必须进入 KV cache 的逻辑合同，否则 head 读错不会立刻崩溃，只会让 logits 慢慢偏掉。

对第 **h** 个 query head，attention 的核心公式是：

$$score_{i,j,h} = \frac{Q_{i,h} \cdot K_{j,g(h)}}{\sqrt{d}} + mask_{i,j}$$

$$P_{i,:,h} = \text{softmax}(score_{i,:,h}), \quad O_{i,h} = \sum_j P_{i,j,h} V_{j,g(h)}$$

prefill 时  $i$  和  $j$  都在 prompt 内,  $\text{mask}_{\{i,j\}}$  会把未来位置置为负无穷; decode 时  $i$  只有当前 token,  $j$  扫描历史 cache。RoPE 通常在算 score 前作用到  $Q$  和  $K$ , 它不改变张量形状, 但会把位置编号注入到每个 head 的二维旋转对里。位置编号错一位, shape 仍然正确, logits 会错。

attention 输出拼回  $[N, n_q \times d]$  后乘  $W_o$ , 再和残差相加。随后进入 MLP。许多 7B 级模型使用 SwiGLU:

$$\text{MLP}(x) = (\text{silu}(xW_{\text{gate}}) \odot xW_{\text{up}}) W_{\text{down}}$$

其中  $W_{\text{gate}}$  和  $W_{\text{up}}$  的形状通常是  $[H, I]$ ,  $W_{\text{down}}$  是  $[I, H]$ 。 $\odot$  是逐元素乘法。SwiGLU 不是“一个激活函数小算子”这么简单, 它让 MLP 有三块大矩阵, 也解释了为什么前面 7B 单层账本里 MLP FLOPs 是大头。

### 核心思想

单层 decoder 的正确性合同可以压成一句话: norm 改尺度,  $Q/K/V$  建立检索空间, RoPE 写入位置, mask 保证因果性, softmax 把相似度变成权重,  $V$  汇聚内容, MLP 做逐 token 非线性变换, residual 保留主干信息。

### RoPE 到底把位置写到哪里

RoPE 的全名是 rotary positional embedding。它不是把一个位置向量加到 hidden 上, 而是在每个 attention head 内, 把  $Q$  和  $K$  的相邻两个维度当成一个二维平面, 对它们按位置编号做旋转。设某个二维对为  $[x_0, x_1]$ , 位置为  $p$ , 这个维度对的角频率为  $\theta$ , 旋转后是:

$$\begin{bmatrix} x'_0 \\ x'_1 \end{bmatrix} = \begin{bmatrix} \cos(p\theta) & -\sin(p\theta) \\ \sin(p\theta) & \cos(p\theta) \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \end{bmatrix}$$

一个 head 的  $\text{head\_dim}=128$ , 就有 64 个这样的二维旋转对。低频维度转得慢, 高频维度转得快。推理实现通常会预先准备  $\cos/\sin$  表, 或者在 kernel 中即时计算/递推。无论实现方式怎样, 位置  $p$  必须和 token 的真实绝对位置一致。

RoPE 的关键效果体现在点积里。若 query 在位置  $i$ , key 在位置  $j$ , 二者分别旋转后再点积, 分数会携带相对位置  $i-j$  的信息。也就是说, 模型不只知道“这两个 token 内容像不像”, 还知道“它们相隔多远”。这正是 KV cache 必须保存已经加过位置的  $K$  的原因之一: 未来 query 来时, 会用自己的位置旋转  $Q$ , 再和历史位置旋转过的  $K$  做点积。

Listing 2.5 RoPE 在 prefill 和 decode 中的位置合同

```
prefill:
 for position p in 0..prompt_tokens-1:
 q[p], k[p] = rope(q[p], k[p], p)
 store k[p] into KV cache

decode:
 p = prompt_tokens + generated_tokens
 q_new, k_new = rope(q_new, k_new, p)
 append k_new at KV cache position p
 attend q_new to cached K[0..p]
```

这里最容易错的是 position 计数。左 padding、prefix cache、分页 cache、被截断的滑动窗口、批量合并请求, 都会让“数组下标”和“模型位置编号”不再天然相等。实现可以把 KV 放在任意物理 page, 但 RoPE 用的位置编号必须仍然是模型语义位置。

### 常见误区

RoPE 错误通常不是 shape 错误。程序会正常运行, KV cache 大小也对, 甚至前几个 token 看起来还行; 但长上下文、prefix 复用、续写请求会出现 logits 漂移。调试时要记

录每个 token 的 model position，而不是只打印 cache slot。

| 场景              | 容易犯的错                  | 后果             |
|-----------------|------------------------|----------------|
| 左 padding batch | 把 batch 内数组列号当位置       | 短句和长句位置错开      |
| prefix cache    | 复用 K/V 但新 token 从 0 计数 | 续写 logits 偏移   |
| 滑动窗口            | cache slot 回绕后位置也回绕    | RoPE 相对距离语义损坏  |
| 分页 KV           | page offset 当全局位置      | 跨页 attention 错 |

## Q、K、V 到底是什么意思

自注意力要解决的问题是：当前位置应该从历史哪些 token 中取信息。每个位置经过三个投影得到 **Q**、**K**、**V**：

- **Q**：query，可以理解成当前位置提出的问题。
- **K**：key，可以理解成历史位置用于匹配的问题索引。
- **V**：value，可以理解成历史位置真正提供的内容。

对当前 token 的一个 attention head，计算可以写成：

Listing 2.6 单 head decode attention

```
for pos in 0..current_position:
 score[pos] = dot(q_current, k_cache[pos]) / sqrt(head_dim)

prob = softmax(score)

out = 0
for pos in 0..current_position:
 out += prob[pos] * v_cache[pos]
```

第一轮循环计算“当前 token 和每个历史 token 有多相关”。第二步 softmax 把分数变成权重。第三轮循环把历史 **V** 按权重加起来，得到当前位置从历史上下文中取回的信息。

如果是 prefill，模型会同时处理 prompt 中多个位置。为了保持因果约束，位置 **i** 只能看 **0..i**，不能看后面的 token。这就是 causal mask。decode 阶段每次只处理最后一个新 token，它天然只能看历史 cache，因此 mask 形态更简单，但读历史 K/V 的长度会随着上下文增长。

## 一个能手算的 attention 例子

用一个二维 head 看清 attention。设当前 query 和两个历史 key/value 为：

$$q = [1, 0], \quad k_0 = [1, 0], \quad k_1 = [0, 1]$$

$$v_0 = [2, 0], \quad v_1 = [0, 4], \quad d = 2$$

两个 score 是：

$$s_0 = \frac{q \cdot k_0}{\sqrt{2}} = 0.707, \quad s_1 = \frac{q \cdot k_1}{\sqrt{2}} = 0$$

softmax 后：

$$p_0 = \frac{e^{0.707}}{e^{0.707} + e^0} \approx 0.67, \quad p_1 \approx 0.33$$

输出就是：

$$out = p_0 v_0 + p_1 v_1 \approx 0.67[2, 0] + 0.33[0, 4] = [1.34, 1.32]$$

这个小例子说明三件事。第一，**Q/K** 决定从历史位置拿多少权重；第二，**V** 决定真正被汇聚的内容；第三，softmax 不会只选一个 token，它通常产生一个加权混合。实际模型只是把二维 head 换成 **128** 维 head，把两个历史位置换成长上下文，把一个 head 换成几十个 head。

### 常见误区

不要把 attention 解释成“模型查找最相关的一个词”。更准确地说，它是在每个 head 内对可见历史位置做加权汇聚。某些 head 可能很尖锐，某些 head 可能很分散；最终还要经过 **Wo** 和 MLP 混合。

### 一个能手算的最小 decoder block

只看 attention 还不够，因为读者真正要实现的是整层 decode。下面给一个极小、可以手算完的单层单 head 例子。设 hidden size **H=4**，query head 数和 KV head 数都为 **1**，**head\_dim=2**，当前是 decode 第 **1** 步，历史里已有 1 个 token。

先给输入和权重。为了把账本压到最小，这里故意选近似单位选择矩阵：

**Listing 2.7** 最小 decoder block 的输入与权重

```
x_in = [1, 0, 1, 0]
```

```
Wq =
[[1,0],
 [0,1],
 [0,0],
 [0,0]]
```

```
Wk = Wq
```

```
Wv =
[[2,0],
 [0,4],
 [0,0],
 [0,0]]
```

```
Wo =
[[1,0,1,0],
 [0,1,0,1]]
```

```
Wgate =
[[1,0],
 [0,1],
 [0,0],
 [0,0]]
```

```
Wup =
[[1,0],
 [0,1],
 [1,0],
 [0,1]]
```

```
Wdown =
[[1,0,1,0],
 [0,1,0,1]]
```



假设 RMSNorm 的  $\gamma = [1, 1, 1, 1]$ ,  $\epsilon$  足够小可忽略; 历史 cache 中已经有

$$k_0 = [1, 0], \quad v_0 = [2, 0]$$

当前 token 经过  $W_q/W_k/W_v$  后得到:

$$q_1 = [1, 0], \quad k_1 = [1, 0], \quad v_1 = [2, 0]$$

为了和前一节的二维 attention 例子保持一致, 再假设历史里还有一个位置

$$k_2 = [0, 1], \quad v_2 = [0, 4]$$

于是当前 query 实际可见的是三个位置  $0, 1, 2$ , 其中  $0$  和  $1$  都与  $q_1$  同向,  $2$  与  $q_1$  正交。  
score 是:

$$s_0 = s_1 = \frac{1}{\sqrt{2}} \approx 0.707, \quad s_2 = 0$$

稳定 softmax 后:

$$p_0 = p_1 = \frac{e^{0.707}}{2e^{0.707} + 1} \approx 0.401, \quad p_2 \approx 0.198$$

因此 attention 输出是:

$$attn\_out = 0.401[2, 0] + 0.401[2, 0] + 0.198[0, 4] \approx [1.604, 0.792]$$

$W_o$  把二维 head 拼回  $[4]$ :

$$attn\_proj = [1.604, 0.792, 1.604, 0.792]$$

做第一次残差:

$$x_{mid} = x_{in} + attn\_proj = [2.604, 0.792, 2.604, 0.792]$$

现在进入 MLP。先做第二次 RMSNorm:

$$r = \sqrt{\frac{2.604^2 + 0.792^2 + 2.604^2 + 0.792^2}{4}} \approx 1.939$$

$$h_{ffn} = \frac{x_{mid}}{r} \approx [1.343, 0.408, 1.343, 0.408]$$

然后按上面的  $W_{gate}/W_{up}$  投影:

$$g = [1.343, 0.408]$$

$$u = [1.343 + 1.343, 0.408 + 0.408] = [2.686, 0.816]$$

SwiGLU 的逐元素结果是:

$$\text{silu}(1.343) \approx 1.064, \quad \text{silu}(0.408) \approx 0.245$$

$$m = \text{silu}(g) \odot u \approx [1.064 \times 2.686, 0.245 \times 0.816] \approx [2.858, 0.200]$$

最后经  $W_{down}$  回到  $[4]$ :

$$ffn\_out = [2.858, 0.200, 2.858, 0.200]$$

第二次残差后的层输出是:

$$x_{out} = x_{mid} + ffn\_out \approx [5.462, 0.992, 5.462, 0.992]$$

这个例子虽然极小, 但已经包含了单层 decode 的完整合同:

Listing 2.8 最小 decoder block 的逐步账本

```

1. x_in -> RMSNorm -> h_attn
2. h_attn -> Wq/Wk/Wv -> q/k/v
3. append current k/v into KV cache
4. q dot historical K -> score row
5. softmax(score) -> prob row
6. prob weighted sum historical V -> attn_out
7. attn_out -> Wo -> x_mid
8. x_mid -> RMSNorm -> h_ffn
9. h_ffn -> Wgate/Wup -> g/u
10. silu(g) * u -> m
11. m -> Wdown -> ffn_out
12. x_mid + ffn_out -> x_out

```

### 核心思想

只要你能把这个最小例子逐标量复现，就已经具备了写 reference decoder layer 的能力。大模型真实工程只是把维度、head 数、层数、GQA 映射、RoPE 和量化合同扩展出去，而不是换了一套完全不同的数学。

### causal mask 不是一个随手写的 if

attention 能保持“当前位置不能偷看未来”，靠的不是口头约束，而是 mask 的数学合同。对 prefill 中第  $i$  个位置看第  $j$  个位置，最基本的 causal mask 是：

$$mask_{i,j} = \begin{cases} 0, & j \leq i \\ -\infty, & j > i \end{cases}$$

它加到 score 上之后，未来位置经过 softmax 的概率就会变成 0。于是：

$$P_{i,j} = \frac{\exp(score_{i,j} + mask_{i,j})}{\sum_t \exp(score_{i,t} + mask_{i,t})}$$

这条式子看起来简单，但实现时有三个常见坑。第一，mask 是语义合同，不是某个 kernel 私有约定；prefill、decode、reference 和 optimized kernel 的可见位置集合必须一致。第二，很多高性能实现不会真的存一个完整  $[T, T]$  mask 矩阵，而是在 kernel 中按  $i, j$  规则即时判断。第三，若 softmax 在 fp32 accumulator 中做，可以直接使用 `-inf`；若某条路径在低精度上直接做指数，常会改用足够小的有限负数，让它在指数后安全下溢到 0。

一旦从 full attention 改到局部窗口，mask 公式也会变化。若窗口宽度是  $W$ ，则第  $i$  个位置只看最近  $W$  个可见历史：

$$mask_{i,j}^{(W)} = \begin{cases} 0, & \max(0, i - W + 1) \leq j \leq i \\ -\infty, & \text{otherwise} \end{cases}$$

这已经不是“同一个模型只是省点内存”，而是改变了可见上下文集合。任何 sliding window、sink token、prefix LM 或者 block sparse attention，都必须先把“谁能看见谁”写成明确合同，再去实现 kernel。

| 形态              | 第 $i$ 个位置的可见集合      | 复杂度直觉         |
|-----------------|---------------------|---------------|
| full causal     | $0..i$              | 长度随 $i$ 线性增长  |
| window $W$      | $\max(0, i-W+1)..i$ | 每行最多 $W$ 个位置  |
| prefix + window | 固定前缀并加最近窗口          | 语义更复杂，缓存复用更敏感 |

## 为什么没有 KV cache 会慢得离谱

先想一个没有 KV cache 的 decode。已经生成了  $S$  个 token，现在要生成第  $S+1$  个。如果没有 cache，模型为了让当前 token attend 到历史，必须重新把  $0..S$  这些 token 跑过每一层，重新计算每一层每个历史位置的 K/V。下一步  $S+2$  又要再算一遍  $0..S+1$ 。

这会造成重复计算。历史 token 的 K/V 一旦算出来，在后续 decode 中不会因为新 token 到来而改变。它们应该保存下来。KV cache 做的正是这件事：

**Listing 2.9** 有无 KV cache 的 decode 差异

```
without KV cache:
 step S:
 recompute K/V for tokens $0..S$ in every layer
 compute attention for current token

with KV cache:
 prefill:
 compute and store K/V for prompt tokens
 step S:
 compute only current token K/V
 append K/V at position S
 read cached K/V[$0..S$] for attention
```

## 核心思想

KV cache 省掉的是历史 token 的重复 K/V 投影和中间层重算；它没有省掉读取历史 K/V 的成本。decode 仍然要随上下文长度读取越来越多的 K/V，所以 KV cache 同时带来性能收益和内存压力。

这句话非常重要。KV cache 不是让 attention 变成常数时间；它让“重算历史网络”变成“读取历史 K/V”。前者会让每步重复大量矩阵乘，后者会让每步产生随上下文增长的内存读流。后续优化 KV cache 的核心，就是围绕容量、布局、读带宽、量化和分页管理做取舍。

## KV cache 的逻辑形状和物理布局

KV cache 的逻辑形状通常可以写成：

**Listing 2.10** KV cache 的逻辑形状

```
K_cache[layer, batch, kv_head, position, head_dim]
V_cache[layer, batch, kv_head, position, head_dim]
```

每一层都有自己的 cache；每个 batch/session 有自己的 cache；每个 K/V head 有自己的历史；每个 position 保存一个 `head_dim` 向量。这里用 `kv_head` 而不是 `head`，是因为一些模型使用 multi-query attention 或 grouped-query attention。它们让多个 query head 共享较少的 K/V head，以降低 KV cache 容量和读流。

逻辑形状不等于物理布局。CPU 后端可能希望 `position, head_dim` 连续，便于按历史顺序扫描；GPU 后端可能希望按 page、block 或 head 分组，让 warp 读取得到更好的 coalescing；分页 KV cache 还会引入 page table。无论物理布局怎样变，逻辑合同不能变：给定 `layer, batch, kv_head, position, head_dim`，读到的 K/V 必须一致。

| 布局选择        | 适合的访问             | 风险                   |
|-------------|-------------------|----------------------|
| position 连续 | CPU 长上下文顺序扫描      | 多 session/page 管理较麻烦 |
| head 连续     | 按 head 分工的 kernel | position 读流可能跨步      |
| page/block  | 动态增长、释放、prefix 复用 | page table 和边界处理复杂   |
| 量化 block    | 降低容量和带宽           | scale metadata 进入热路径 |

## KV cache 的容量账本

KV cache 容量可以用一条公式估算：

Listing 2.11 KV cache 容量估算

```
bytes =
 layers *
 batch *
 context_tokens *
 kv_heads *
 head_dim *
 2 /* K and V */ *
 bytes_per_element
```

举一个接近 7B 模型的账本。假设 `layers=32`, `kv_heads=8`, `head_dim=128`, 上下文长度 `S=4096`, batch 为 1, KV 用 fp16, 也就是每个元素 2 字节：

Listing 2.12 GQA 模型的 KV cache 例子

```
bytes = 32 * 1 * 4096 * 8 * 128 * 2 * 2
 = 536870912 bytes
 = 512 MiB
```

如果 `kv_heads` 不是 8, 而是 32, 容量会变成 4 倍, 也就是约 2 GiB。若 batch 从 1 变成 4, 再乘 4。若上下文从 4096 变成 32768, 再乘 8。KV cache 不是小 buffer, 它经常决定本地推理能不能跑长上下文、能跑多少并发 session、能不能放进 GPU 显存。

## 常见误区

权重量化解决的是静态模型容量和权重读带宽；KV cache 量化解决的是运行时状态容量和历史 K/V 读带宽。两者对象不同、生命周期不同、误差来源不同, 不能混为一谈。

KV cache 量化也不是免费。比如 int8 KV cache 会减少 K/V 码点字节, 但需要 scale metadata; per-token scale 精度较好, 但 metadata 和读取更重; per-block scale 开销小一些, 但误差和 block 边界要验证。报告不能只写“KV cache 减半”, 必须同时写 decode latency、metadata bytes、logits 误差和长上下文质量。

## MHA、GQA、MQA 为什么影响 KV cache

attention 的 query head 数决定输出表达能力的一部分, 但 KV cache 的容量由 K/V head 数决定。三种常见形态可以这样区分：

| 形态  | K/V head 数                    | 含义                          |
|-----|-------------------------------|-----------------------------|
| MHA | <code>n_kv=n_q</code>         | 每个 query head 有自己的 K/V head |
| GQA | <code>1&lt;n_kv&lt;n_q</code> | 多个 query head 共享一组 K/V head |
| MQA | <code>n_kv=1</code>           | 所有 query head 共享一个 K/V head |

继续用 `n_q=32`, `head_dim=128`, `L=32`, `S=4096`, fp16 KV。不同 K/V head 数的 cache 容量是：

| 形态  | <code>n_kv</code> | KV 容量     | 每 token KV 读流 |
|-----|-------------------|-----------|---------------|
| MHA | 32                | 约 2 GiB   | 约 2 GiB       |
| GQA | 8                 | 约 512 MiB | 约 512 MiB     |
| MQA | 1                 | 约 64 MiB  | 约 64 MiB      |

这张表解释了为什么很多本地模型选择 GQA。它保留多个 K/V head，比 MQA 更有表达能力；同时把 KV cache 容量和读流从 MHA 的四分之一甚至更低。注意，GQA 降低的是  $Wk/Wv$  输出、KV cache 容量和历史 K/V 读流；它不降低  $Wq/Wo$  和 MLP 的主计算量。

实现上，GQA 不能靠简单复制 K/V 到 32 个 query head 来“方便计算”。复制会把 KV cache 容量和读流重新放大，抵消 GQA 的主要收益。正确做法是在 attention kernel 中让多个 query head 映射到同一个 K/V head：

**Listing 2.13** GQA head 映射不能靠物理复制糊弄

```
group_size = num_query_heads / num_kv_heads

for q_head in 0..num_query_heads-1:
 kv_head = q_head / group_size
 k = K_cache[layer, batch, kv_head, position, :]
 v = V_cache[layer, batch, kv_head, position, :]
 compute_attention(q[q_head], k, v)
```

### 核心想

GQA 是模型结构和运行时账本同时成立的设计：模型训练时学会共享较少的 K/V head，推理时才能少存、少读历史 K/V。没有这个前提，运行时强行合并 K/V head 会改变模型语义。

### KV cache 缓存的不是 hidden

一个常见误解是：既然历史 token 已经算过，保存 hidden 就够了。实际不是这样。每一层 attention 需要的是这一层自己的 **K** 和 **V**，它们来自当前层输入 hidden 经过  $Wk/Wv$  投影，再叠加 RoPE 等位置信息。第 3 层的 **K/V** 不能替代第 18 层的 **K/V**；token embedding 也不能替代任意层的 **K/V**。

| 对象        | 是否长期保存 | 原因                                 |
|-----------|--------|------------------------------------|
| token ids | 是      | tokenizer 结果和上下文合同，需要用于位置、日志和重放    |
| hidden    | 通常否    | 每层临时状态，下一层用完即可释放或复用 buffer         |
| K/V       | 是      | 后续 token 的每一层 attention 都要读取历史 K/V |
| logits    | 通常否    | sampler 用完即可丢弃，除非调试或审计需要保存         |

因此 KV cache 的正确性不能只看最终文本是否“差不多”。更严格的 oracle 应该能固定 prompt、权重、采样参数和位置编号，比较某层某位置的 **K/V** 或最终 logits。KV cache 布局、分页、量化、batch 合并都可以改，但逻辑坐标和数值合同不能漂移。

### 每生成一个 token 要读多少 KV

KV cache 的容量公式只回答“占多少内存”，还没有回答“每步读多少”。full attention decode 下，当前 token 在每一层都要扫描历史 **K** 算 score，再扫描历史 **V** 做加权求和。粗略读流就是：

**Listing 2.14** decode 单 token 的 KV 读写粗账

```

kv_read_bytes_per_token =
 layers *
 context_tokens *
 kv_heads *
 head_dim *
 2 /* K and V */ *
 bytes_per_kv_element

kv_write_bytes_per_token =
 layers *
 kv_heads *
 head_dim *
 2 /* new K and V */ *
 bytes_per_kv_element

```

继续使用前面的 7B 级参数， $L=32$ ， $kv\_heads=8$ ， $head\_dim=128$ ，fp16 KV：

| 上下文长度     | 每 token 读 KV | 每 token 写 KV |
|-----------|--------------|--------------|
| $S=4096$  | 约 512 MiB    | 约 128 KiB    |
| $S=32768$ | 约 4 GiB      | 约 128 KiB    |

这张表比“KV cache 很大”更有工程意义：长上下文 decode 的热路径不是写入新 K/V，而是反复读取历史 K/V。KV int8 大致把码点读流减半；KV int4 大致降到四分之一，但 scale metadata、反量化和误差验证会进入热路径。滑动窗口、稀疏 attention、prefix cache、page attention 的价值，也要放在这条读流上判断。

若 prompt 长度为  $P$ ，继续生成  $G$  个 token，KV 读取不是简单的  $G \times P$ 。因为上下文每步增长 1，累计读流近似为：

Listing 2.15 生成  $G$  个 token 的累计 KV 读流

```

total_kv_read =
 layers * kv_heads * head_dim * 2 * bytes_per_kv *
 (G * P + G * (G - 1) / 2)

```

这解释了为什么同一个模型在短回答和长回答中的平均 tokens/s 会不同。不是 sampler 变慢了，而是 decode 后半段每步要扫描更长的历史。

### prefix cache 和 sliding window 改变的是语义

很多工程讨论会把 prefix cache、window attention、KV eviction 都叫成“cache 优化”。这不够准确。prefix cache 在语义上仍然是 full attention，它只是复用已经算过、并且逻辑位置完全一致的前缀 K/V；sliding window 则真的改变了每一步的可见历史集合；KV eviction 若没有模型层面的对应设计，往往直接改变 logits。

设用户 prompt 长度为  $P$ ，已经生成了  $t$  个 token。full attention decode 的可见集合是：

$$Visible(t) = \{0, 1, \dots, P + t\}$$

如果某次请求复用了前缀缓存，只要前缀文本、tokenization 结果、RoPE 位置编号和模型权重都一致，那么前  $P-1$  个位置的 K/V 可以直接重用；新 token 仍然会看见完整前缀。这时变的是“算不算前缀”，不是“能不能看前缀”。

若改成窗口大小为  $W$  的 sliding window，并保留完整前缀  $0..P-1$ ，后续第  $t$  步可见集合会变成：



$$Visible_{prefix+window}(t) = \{0, \dots, P-1\} \cup \{\max(P, P+t-W+1), \dots, P+t\}$$

若没有保留完整前缀，而是纯窗口 attention，则更激进：

$$Visible_{window}(t) = \{\max(0, P+t-W+1), \dots, P+t\}$$

这两者的差异非常大。前者还能长期保留系统提示词、指令头和固定 schema；后者在上下文够长时会把最早的系统指令也裁掉。实现上如果只是“把最旧 page 释放掉”，却没有同步修改 mask、position 合同和 benchmark 口径，那么报告里的模型已经不是原来的 full attention 模型。

### 常见误区

prefix cache 的复用条件比“前缀文本一样”更严格。tokenization 版本、special token 规则、chat template、RoPE 位置起点、模型权重版本，只要有一项不同，前缀 K/V 就不能安全复用。

### 计算量账本：线性层、attention 和 sampler

现在把计算量讲清。设 hidden size 为  $H$ ，层数为  $L$ ，prompt token 数为  $T$ ，decode 当前上下文长度为  $S$ 。一个线性层从  $H$  投影到  $O$ ，处理  $N$  个 token，乘加量大约是：

Listing 2.16 线性层 FLOPs 粗账

```
linear_flops ~= 2 * N * H * O
```

系数 2 来自一次乘法和一次加法。这个公式足够解释 prefill 和 decode 的差别：

- prefill:  $N=T$ ，一次处理很多 token，线性层更像 GEMM。
- decode:  $N=1$ ，一次处理一个 token，线性层更像 GEMV 或小 GEMM。

attention 的账本分两部分。第一，当前 query 和历史 K 做点积，大约是  $2*S*head\_dim$  每个 head。第二，用 softmax 权重加权历史 V，也大约是  $2*S*head\_dim$  每个 head。所有层和所有 head 加起来，decode attention 的读流和计算量都随  $S$  线性增长。

sampler 也不能忽略。若词表大小是  $V$ ，贪心采样至少要扫描  $V$  个 logits；top-k/top-p 还会有排序、选择或前缀概率累加。对于很大的 vocab 或 GPU 后端，如果每步都把完整 logits 从 GPU 拷回 CPU，sampler 和同步也会进入端到端瓶颈。

| 阶段      | 主要工作                         | 常见瓶颈                            |
|---------|------------------------------|---------------------------------|
| prefill | 多 token 线性层、prompt attention | GEMM 吞吐、activation 峰值、workspace |
| decode  | 单 token 投影、读取历史 KV           | 权重读带宽、KV cache 读流、小 kernel 开销   |
| sampler | logits 后处理                   | 词表扫描、top-k/top-p、设备同步           |

### 同一组 7B 参数下的 prefill/decode 对账

前面给了公式，现在把它们放进同一张账本里。仍取  $H=4096$ 、 $L=32$ 、 $I=11008$ 、 $n_q=32$ 、 $n_{kv}=8$ 、 $head\_dim=128$ 。对 decode 单 token，单层主要线性部分大约是：

$$Q : 2H^2, \quad K/V : 2H(n_{kv}d) \times 2, \quad O : 2H^2, \quad MLP : 2HI \times 3$$

代入后：

| 项目             | 单层 FLOPs | 32 层 FLOPs |
|----------------|----------|------------|
| Q projection   | 约 33.6M  | 约 1.07G    |
| K/V projection | 约 16.8M  | 约 0.54G    |
| O projection   | 约 33.6M  | 约 1.07G    |
| MLP 三矩阵        | 约 270.5M | 约 8.66G    |
| 合计             | 约 354.4M | 约 11.34G   |

这只是线性层，不含 attention 读历史 K/V、softmax、sampler 和 runtime 调度。若 prompt 长度  $T=2048$ ，prefill 线性层会把这 **11.34G** 乘上 **2048**，得到约 **23.2TFLOPs**；若  $T=4096$ ，就是约 **46.5TFLOPs**。但是 prefill 的权重 tile 能被多个 token 行复用，所以它不能按“每 token 读完整模型权重”粗暴相乘。

prefill attention 的二次项也要算。单 head 的  $QK^T$  和  $PV$  合起来约  $4T^2d$ ；所有 query head 和所有层合起来约：

$$F_{prefill\_attn} \approx L \times n_q \times 4T^2d$$

代入后得到：

| prompt 长度     | 线性层 FLOPs | attention 二次项 FLOPs |
|---------------|-----------|---------------------|
| <b>T=2048</b> | 约 23.2T   | 约 2.2T，因果半矩阵约 1.1T  |
| <b>T=4096</b> | 约 46.5T   | 约 8.8T，因果半矩阵约 4.4T  |

这张表说明：长 prompt 的 TTFT 不是只由线性层决定。prompt 从 **2048** 到 **4096**，线性层约翻倍，attention 二次项约翻四倍。若一个 benchmark 只测 **128** token prompt，它不能证明 **8K** prompt 的首 token 延迟。

再看 decode 长生成。假设 prompt 长度  $P=2048$ ，继续生成  $G=256$  个 token。线性层 FLOPs 近似是：

$$F_{decode\_linear} \approx 11.34G \times 256 \approx 2.90TFLOPs$$

decode attention 的累计历史长度不是 **256\times2048**，而是：

$$\sum_{t=0}^{G-1} (P+t) = GP + \frac{G(G-1)}{2} = 556928$$

attention 计算量约：

$$F_{decode\_attn} \approx L \times n_q \times 4d \times 556928 \approx 0.292TFLOPs$$

注意这个 **0.292TFLOPs** 看起来不如线性层大，但它伴随大量 KV 读流。fp16 GQA KV 的累计读字节是：

$$32 \times 8 \times 128 \times 2 \times 2 \times 556928 \approx 68GiB$$

所以长生成的 decode 后半段常常表现为两条压力叠加：每个 token 都要读权重并做线性层，同时还要随上下文增长扫描历史 KV。权重量化主要压低第一条；KV 量化、paged KV、sliding window、GQA/MQA 主要压低第二条。但任一项只要改变 mask、position 或 K/V 数值解释，就必须回到 oracle 验证。

### 核心思想

prefill 的核心问题是“多 token 大矩阵 + 长 prompt 二次 attention”；decode 的核心问题是“每 token 权重流 + 随上下文增长的 KV 读流”。优化方向不同，验收指标也不能混在一个平均 tokens/s 里。

### batch decode 不是免费吞吐

服务端常把多个 session 合并成 batch decode，希望让单 token GEMV 变成小 GEMM，提高权重复用。这个方向是对的，但它不是免费吞吐。设 batch 中有  $B$  个 session，线性层从  $[1, H]$  变成  $[B, H]$ ，权重 tile 可以被  $B$  行复用；但是每个 session 的上下文长度可能不同，KV 读流仍然要分别扫描：

$$KVRead_{batch} \approx L \times n_{kv} \times d \times 2 \times bytes \times \sum_{b=0}^{B-1} S_b$$

若 batch 内有一个 32K 长上下文 session 和多个短上下文 session，attention kernel、page table 和调度可能被长请求拖慢。若为了凑 batch 等待太久，吞吐提高但 p95 延迟变差。真正的服务端账本至少要同时报告：

- batch size 分布，而不是只报最大 batch；
- batch 内 context length 分布；
- 每步实际扫描的 KV token 总数  $\sum S_b$ ；
- 合批等待时间和 decode kernel 时间；
- 是否出现 padding 计算、page table 间接访问和长短请求互相阻塞。

这也是 paged KV cache 的价值之一：它让不同 session 的 KV 可以分散增长和回收，调度器可以按 page 管理上下文。但 page table 引入间接寻址，kernel 必须把逻辑位置、物理 page、RoPE position、scale metadata 一起处理对。

### logits 到 token 的算法细节

模型主体输出的是 logits，不是最终文字。设词表大小为  $V$ ，最后位置的 hidden 是  $h$ ，`lm_head` 权重是  $W_{vocab}$ ，则：

$$z = hW_{vocab}, \quad z \in \mathbb{R}^V$$

$z_i$  是第  $i$  个 token 的未归一化分数。softmax 概率是：

$$p_i = \frac{\exp((z_i - m)/\tau)}{\sum_j \exp((z_j - m)/\tau)}, \quad m = \max_j z_j$$

这里减去  $m$  是为了避免指数溢出；`\tau` 是 temperature。temperature 越小，分布越尖，越接近贪心；temperature 越大，尾部 token 概率会被抬高。若实现没有先减最大值，长尾 logits 在 fp16/fp32 下都可能出现数值问题。

Listing 2.17 sampler 的常见执行顺序

```
logits = lm_head(last_hidden)
logits = apply_repetition_penalty(logits, generated_tokens)
logits = logits / temperature
logits = apply_top_k_filter(logits, k)
logits = apply_top_p_filter(logits, p)
prob = softmax_stable(logits)
next_id = sample(prob, rng_state)
```

top-k 是保留分数最高的  $k$  个 token，其他 token 置为负无穷。top-p 是先按概率从高到低排序，保留累计概率达到阈值  $p$  的最小集合。两者可以叠加，但顺序会影响结果。工程测试时必须固定顺序、随机数算法和 seed；否则同一个 logits 也会生成不同文本。

| 策略             | 算法含义                | 典型风险               |
|----------------|---------------------|--------------------|
| greedy         | 取最大 logits          | 稳定但容易重复和死板         |
| temperature    | 缩放 logits 分布        | 过高会放大低质量尾部 token   |
| top-k          | 只在前 <b>k</b> 个候选中采样 | <b>k</b> 太小会截断合理候选 |
| top-p          | 保留累计概率核             | 排序和前缀和成本进入热路径      |
| repeat penalty | 惩罚已生成 token         | 过强会破坏术语和代码一致性      |

sampler 的 oracle 要分两层。第一层比较 logits：同一 prompt、同一权重、同一 KV cache 状态下，优化前后的 logits 误差应在阈值内。第二层比较 token 序列：固定 sampler 参数和随机数状态后，next token 应一致。只看最终自然语言文本是不够的，因为采样随机性会掩盖模型计算错误。

### 为什么 7B decode 常常像“读权重”

对 dense decoder-only 模型，decode 每生成一个 token 都要经过所有层。粗略说，每步会触碰模型的大部分权重。若权重是 fp16，7B 参数约 14GB；若是 int8，约 7GB 加 scale；若是 int4，约 3.5GB 加 group scale。实际 kernel 会有 cache、packing 和分块，但 decode batch 小，权重复用有限，内存带宽往往很显眼。

这解释了为什么量化能显著影响本地 CPU 推理：它不仅降低模型容量，也降低每步权重读流。但低比特会引入解包、scale 读取和反量化成本。收益来自“少读字节”，代价来自“多做转换”。是否更快，要看当前瓶颈是带宽、指令、cache、TLB 还是线程调度。

prefill 不同。prefill 有多个 token 行，权重能被多个 token 复用，GEMM kernel 更容易吃到算术吞吐。于是同一个模型可能表现为：prefill 很快，decode 慢；或者 GPU prefill 很强，但单用户 decode tokens/s 提升不如峰值算力那样夸张。

### 核心理想

不要用一个“模型速度”概括推理性能。至少要分开报告：加载时间、packing 时间、time to first token、prefill tokens/s、decode tokens/s、p50/p95 单 token 延迟、KV cache bytes 和峰值内存。

### 7B 级模型的一层粗账

为了让账本落地，取一组常见 7B 级参数做估算：**H=4096**，层数 **L=32**，MLP 中间维度近似 **I=11008**，query head 数 **32**，GQA 下 **kv\_heads=8**，**head\_dim=128**。不同模型数值会变，但量级判断相同。

decode 单 token 时，一层的主要线性层 FLOPs 近似如下：

| 子层               | FLOPs 公式                       | 量级      |
|------------------|--------------------------------|---------|
| Q projection     | <b>2*H*H</b>                   | 约 33.6M |
| K/V projection   | <b>2*H*kv_heads*head_dim*2</b> | 约 16.8M |
| O projection     | <b>2*H*H</b>                   | 约 33.6M |
| MLP gate/up/down | <b>2*H*I*3</b>                 | 约 270M  |

这张表说明两个事实。第一，MLP 往往是单层 FLOPs 大头；第二，GQA 会显著降低 K/V projection 和 KV cache 容量，但不会降低 Q projection 和 MLP。粗略相加，一层 decode 线性部分约 350M FLOPs，32 层就是 11G FLOPs 量级。这个数字不是精确 benchmark，但足以解释为什么每 token latency 对本地机器很敏感。

再看权重字节。如果一层权重按 fp16 读，Q、O 各约 **H\*H\*2** 字节，MLP 三个矩阵约 **H\*I\*3\*2** 字节。仅这些线性层，一层就接近数百 MB 级权重流；32 层接近完整模型权重规模。int8、int4 的意义首先是减少这条读流，但它们会引入 scale、解包和反量化成本。

Listing 2.18 单 token decode 的粗略机器判断

```
if batch == 1:
 projection shape is mostly GEMV
```

```

weight reuse is low
weight bandwidth and cache/TLB behavior are critical
else:
 projection becomes small/batched GEMM
 weight reuse improves
 arithmetic throughput becomes more visible

```

### 把权重字节和带宽下限算出来

单 token decode 的线性层 FLOPs 很大，但在 batch 为 1 时，另一个更硬的约束是权重字节。可以先用最朴素的下限估算：

Listing 2.19 单 token decode 的字节下限

```

weight_bytes_per_token ~= parameter_count * bytes_per_weight
lower_bound_ms =
 1000 * (weight_bytes_per_token + kv_read_bytes) /
 effective_memory_bandwidth_bytes_per_second

```

对 7B 参数模型，忽略少量非矩阵参数和格式头，权重读流大致如下：

| 权重格式 | 码点字节     | 加上 scale 后的直觉   |
|------|----------|-----------------|
| fp16 | 约 14 GB  | 几乎就是完整权重流       |
| int8 | 约 7 GB   | 还要读 group scale |
| int4 | 约 3.5 GB | scale 和解包成本更显眼  |

把它和 KV 读流合在一起看。若 int4 权重实际要读约 **3.7GB**，上下文 **S=4096** 的 fp16 KV 每 token 约 **512MiB**，那么单 token 至少要搬运约 **4.2GB** 热数据。若有效内存带宽是 **100GB/s**，光搬数据的下限就是约 **42ms/token**，也就是最多约 **24token/s**。若有效带宽是 **300GB/s**，下限约 **14ms/token**，最多约 **71token/s**。

### 常见误区

这里的数字不是 benchmark 结果，而是反证工具。如果实测声称 batch 1、7B、长上下文、低带宽机器能远超这个下限，就必须解释权重是否常驻更高层 cache、是否跳过了部分层、是否用了 speculative decoding、是否测的是 prefill，或者统计口径是否把等待时间排除了。

### 从账本推到可服务容量

推理引擎能不能落地，不取决于一句“7B 可以本地跑”，而取决于容量预算是否闭合。最小预算可以先拆成三项：静态权重、运行时 KV cache、临时 workspace。

Listing 2.20 单机推理服务的内存预算

```

usable_memory >=
 model_weight_bytes +
 max_sessions * max_context_tokens * kv_bytes_per_token +
 workspace_bytes +
 runtime_overhead_bytes

```

前面例子里，fp16 KV 的 **kv\_bytes\_per\_token** 是：

Listing 2.21 每个 session 每个 token 的 KV 容量

```

kv_bytes_per_token =
 layers * kv_heads * head_dim * 2 * bytes_per_kv
 = 32 * 8 * 128 * 2 * 2
 = 131072 bytes
 = 128 KiB

```

因此一个 4096 token 上下文的 session 需要约 512MiB KV；16 个并发 session 就是约 8GiB KV。若模型权重 int4 后约 3.7GiB，机器可用内存 16GiB，再扣掉 workspace、页表、线程栈、allocator 碎片和操作系统余量，能稳定承载的并发 session 不会等于 16GiB/512MiB=32。这个除法忽略了权重和运行时开销，是错误容量评估。

吞吐预算也要分 prefill 和 decode。一个请求的首 token 延迟可以粗略拆为：

Listing 2.22 请求延迟拆分

```

TTFT =
 queue_wait +
 tokenize_time +
 prefill_time(prompt_tokens) +
 first_sampler_time

per_output_token_latency =
 decode_compute_time +
 kv_read_time(current_context) +
 sampler_time +
 scheduler_overhead

```

这组公式能直接指导工程取舍。prompt 很长而输出很短，优先看 prefill；prompt 不长但输出很长，优先看 decode 后半段和 KV 读流；并发 session 多，优先看 KV 容量、分页和调度；batch 合并能提升权重复用，但会增加排队等待，p95 延迟可能变差。没有这些数字，只说“部署一个问答服务”没有工程意义。

### prefill 的账本为什么不同

prefill 假设 prompt 长度是  $T$ 。同一个 Q projection 从  $[1, H] * [H, H]$  变成  $[T, H] * [H, H]$ 。FLOPs 乘以  $T$ ，但权重不是读  $T$  份独立副本；优秀 GEMM 会让一块权重被多个 token 行复用。于是 prefill 的算术强度比 decode 高，GPU tensor core 或 CPU 高质量 GEMM 更容易发挥作用。

| 阶段      | 数据复用                 | 典型优化方向                                         |
|---------|----------------------|------------------------------------------------|
| prefill | 多个 token 复用同一权重 tile | GEMM、tile、batch、tensor core、activation planner |
| decode  | 单 token 复用弱，逐步读历史 KV | packed weight、低比特、GEMV、小 kernel、KV layout      |

attention 也不同。prefill 的 attention 处理  $T$  个位置，因果 mask 下每个位置看左侧历史，总体有近似  $T^2$  的交互；decode 每步只处理当前 token，看长度为  $S$  的历史，单步是  $S$  级别。若 prompt 很长，prefill attention 可能显著；若生成很长，decode 的累计 KV 读取会显著。

### prefill attention 的矩阵账本

decode attention 是当前 query 扫描历史 K/V；prefill attention 是 prompt 内所有 query 同时看左侧历史。对单个 head，prefill 可以写成三个矩阵步骤：

$$S = \frac{QK^T}{\sqrt{d}} + M, \quad P = \text{softmax}(S), \quad O = PV$$



其中  $Q/K/V$  的形状都是  $[T, d]$ ,  $S/P$  的形状是  $[T, T]$ ,  $M$  是 causal mask。第  $i$  行只能保留  $0..i$  列, 未来列被设为负无穷。这个  $[T, T]$  分数矩阵解释了为什么长 prompt 的 prefill attention 会突然变重。

| 步骤      | 形状                | 粗略 FLOPs        |
|---------|-------------------|-----------------|
| $QK^T$  | $[T, d] * [d, T]$ | $2 * T * T * d$ |
| softmax | 每行长度 $T$          | 指数、求和、归一化       |
| PV      | $[T, T] * [T, d]$ | $2 * T * T * d$ |

若  $T=4096$ ,  $d=128$ , 单个 head 的  $QK^T$  就约 4.3GFLOPs,  $PV$  也是同量级。32 个 query head 合起来是数百 GFLOPs 级别。实际高性能实现不会天真地把完整  $[T, T]$  矩阵长期写到内存里, 而会分块计算、在线 softmax、尽量减少 HBM/DRAM 流量。这就是 FlashAttention 这类算法的核心方向: 数学结果不变, 改变中间矩阵的物理存取方式。

CPU 本地推理也要理解这一点。短 prompt 时, prefill 主要看线性层 GEMM; 长 prompt 时, attention 的二次项会变明显。若 benchmark 只用 32 token prompt, 就不能推断 8K prompt 的 TTFT。反过来, decode 单步没有  $[T, T]$  分数矩阵, 但它会随着上下文增长持续读 KV cache。

### 常见误区

不要把 prefill tokens/s 和 decode tokens/s 混在一起平均。prefill 的主项包含批量线性层和  $T^2$  attention; decode 的主项包含单 token 权重流和  $S$  级 KV 读取。两个数代表不同问题。

### softmax 为什么能在线分块算

前面写了 prefill attention 的主式:

$$S = \frac{QK^T}{\sqrt{d}} + M$$

若天真实现, 先把完整  $[T, T]$  score 矩阵写到内存, 再逐行做 softmax, 再把概率矩阵乘  $V$ 。这样数学上没错, 但内存流量很大。高性能 attention 的关键观察是: softmax 可以在不物化整行的情况下分块计算。

先看一行 score  $s_1, \dots, s_n$  的稳定 softmax:

$$p_i = \frac{e^{s_i - m}}{\sum_j e^{s_j - m}}, \quad m = \max_j s_j$$

若把一整行拆成多个 block, 可以对每个 block 先算局部最大值和局部指数和, 再合并。设 block A 的局部最大值和局部和是  $m_A, l_A$ , block B 的是  $m_B, l_B$ , 则合并后的全局值是:

$$m = \max(m_A, m_B)$$

$$l = e^{m_A - m} l_A + e^{m_B - m} l_B$$

输出向量也能按同样的缩放关系合并。若 block A 的加权值和 block B 的加权值分别是  $o_A, o_B$ , 则:

$$o = \frac{e^{m_A - m} l_A}{l} o_A + \frac{e^{m_B - m} l_B}{l} o_B$$

这说明 attention 不需要把完整概率矩阵  $P$  永久落到内存里。只要每处理一个 block 时维护“当前行的最大值、归一化分母和累计输出”, 就能得到与稳定 softmax 相同的数学结果。FlashAttention 这一类算法的核心就是这个思想: 不改变结果, 改变中间矩阵的物理存取路径。

```

row_max = -inf
row_sum = 0
row_out = 0

for each score/value block:
 block_max = max(scores)
 new_max = max(row_max, block_max)

 row_sum = exp(row_max - new_max) * row_sum +
 sum(exp(scores - new_max))

 row_out = exp(row_max - new_max) * row_out +
 sum(exp(scores - new_max) * values)

 row_max = new_max

output = row_out / row_sum

```

### 核心思想

online softmax 的价值不是“少做了 softmax”，而是避免把巨大中间矩阵写回再读回。attention 的数学没有变，变的是内存交通组织方式。

### roofline 判断：算力瓶颈还是带宽瓶颈

第二册讲过 roofline 思路：性能取决于算术强度，也就是每读写一个字节做多少计算。把它放到推理里：

Listing 2.24 推理阶段的算术强度直觉

```

prefill GEMM:
 many FLOPs per reused weight byte
 more likely compute-bound on strong backend

decode GEMV:
 fewer FLOPs per weight byte
 often bandwidth/cache/TLB sensitive

KV attention:
 reads K/V proportional to context length
 bandwidth grows with S

```

所以“加 GPU”不是万能解释。GPU 对 prefill 大 GEMM 很强；但单用户 decode 若 batch 小，可能被 launch、KV 读流、logits 拷贝和 sampler 同步限制。CPU 优化也一样：SIMD 指令很重要，但若主要时间在权重读流和 KV cache 读流上，单纯增加 FMA 吞吐不会线性提升 tokens/s。

### 验收与评分标准

一个高质量推理报告至少要回答：

- 当前测的是 prefill、decode，还是混合端到端。
- 模型参数、量化格式、上下文长度、batch、线程/GPU 配置是什么。
- 每 token 权重读流和 KV cache 读流大致是多少。
- 主要瓶颈更像算力、带宽、cache/TLB、同步，还是 sampler。

- 优化后 logits/oracle 是否仍然通过。

### 数值稳定性：Transformer 不是只看 shape

shape 正确只能证明张量尺寸对得上，不能证明数值正确。Transformer 推理里最容易出现“程序不崩但 logits 漂”的位置，是 RMSNorm、RoPE、softmax、SwiGLU、量化反量化和采样边界。它们的错误常常不会立刻变成 NaN，而是在长上下文、低温采样或 prefix 复用时放大。

**RMSNorm 的稳定边界** RMSNorm 的分母是 hidden 向量均方根加 `\epsilon`：

$$r = \sqrt{\frac{1}{H} \sum_i x_i^2 + \epsilon}$$

实现时至少有三个选择会影响误差。第一，平方和用 fp32 还是 fp16/bf16 累加；第二，`\epsilon` 加在 sqrt 前还是其他位置；第三，scale `\gamma` 与除法的顺序是否改变舍入。对 7B 级 hidden `H=4096`，若用低精度累加平方和，误差会被 4096 项累计放大。reference path 应使用 fp32 累加，optimized path 的误差要相对 reference 验证。

Listing 2.25 RMSNorm oracle 应记录的中间量

```
input hidden checksum
sum_square_fp32
rms_denominator
normalized_hidden checksum
gamma_applied_hidden checksum
max_abs_error_vs_reference
```

**softmax 必须先减最大值** attention score 进入 softmax 前可能有很大正负范围。直接计算 `exp(score)` 容易 overflow；全是很小负数又可能 underflow。稳定写法是先减去本行最大值：

$$\text{softmax}(x_i) = \frac{e^{x_i - m}}{\sum_j e^{x_j - m}}, \quad m = \max_j x_j$$

causal mask 或 padding mask 通常把非法位置设为很大的负数。实现要保证 masked lane 不参与最大值，也不参与分母。若把 mask 值设得不够小，非法位置可能获得非零概率；若整行全 mask，分母为 0，需要由上层合同禁止或定义结果。

| 错误          | 表面现象           | 检测输入                   |
|-------------|----------------|------------------------|
| 不减最大值       | NaN/inf 或概率全 0 | 大正 score、大负 score      |
| mask 参与 max | 合法概率被压扁        | 左 padding、短序列 batch    |
| 分母低精度累加     | 长上下文概率漂移       | context=1、128、4096     |
| 全 mask 未定义  | NaN 扩散到 logits | sweep<br>空上下文或非法 batch |

**RoPE 的误差来自位置和三角函数表** RoPE 数值错误有两类。一类是位置语义错：把 physical slot 当 model position，把滑动窗口回绕位置当绝对位置，把 left padding 列号当 token position。另一类是三角函数表或 dtype 错：`cos/sin` 表精度不足、索引错、head dim 对偶维度配错。前者 shape 完全正确但语义错；后者短上下文可能不明显，长上下文会扩大。

RoPE oracle 不应只比较最终 logits。它要能 dump 某个 layer、head、position 的旋转前后 `q/k` 片段，并记录 `model_position`、`physical_slot` 和 `rope_table_index`。这三个数必须分开。

**SwiGLU 的溢出和量化边界** SwiGLU 中 `silu(g)*u` 是逐元素乘法。若 `g` 很大，`silu(g)` 接近 `g`；若 `g` 很小负，`silu(g)` 接近 0。低精度路径要注意激活值范围、乘法精度和 down projection 前的 dtype。量化 MLP 时，`Wgate/Wup/Wdown` 的 scale 误差会穿过非线性，不能简单把三块线性层的误差相加。

### 核心思想

Transformer 的数值正确性要按中间合同验证: RMSNorm 的平方和、RoPE 的位置、softmax 的 max/sum、SwiGLU 的激活范围、量化的 accumulator 和 scale。最终 logits 是结果, 不是足够细的诊断。

#### attention 数据移动账本: 从 $O(S^2)$ 到 bytes/token

只说 attention 复杂度是  $O(S^2)$  不够。推理系统更关心每个阶段读写多少字节。设 hidden 为  $H$ , query heads 为  $n_q$ , KV heads 为  $n_{kv}$ , head dim 为  $D$ , 上下文长度为  $S$ , 元素字节为  $b$ 。decode 单 token 时, 当前 token 的 Q 只是一小块, 但 attention 要读取历史 K/V:

$$Bytes_{KV/read} \approx S \times n_{kv} \times D \times 2 \times b$$

其中 2 表示 K 和 V。以 7B 常见形状  $n_{kv} = 8, D = 128, S = 4096, b = 2$  估算:

$$4096 \times 8 \times 128 \times 2 \times 2 = 16 \text{ MiB}$$

这只是单层 decode attention 读取 KV 的量级; 32 层就是约 512MiB/token 的 KV 读流, 还没算权重、激活、softmax 和 logits。这个数量级解释了为什么长上下文 decode latency 会随 context 增长, 为什么 KV layout、paging、quantization 和 batching 会成为系统问题。

| 阶段                | 主要读写                          | 典型瓶颈                           |
|-------------------|-------------------------------|--------------------------------|
| prefill attention | QK 分数矩阵、softmax、PV, token 间并行 | 算力、workspace、activation 带宽     |
| decode attention  | 读历史 K/V, 写当前输出                | KV 带宽、cache/TLB、softmax 归约     |
| GQA/MQA           | 减少 KV heads 数量                | head 映射、layout、质量/模型约束         |
| paged KV          | 非连续 slot 映射                   | page table、跨页 coalescing、调度复杂度 |

若使用 MHA,  $n_{kv} = n_q$ 。GQA 把多个 query head 共享到较少 KV head, KV cache 容量和读流按  $n_{kv}/n_q$  下降。若  $n_q = 32, n_{kv} = 8$ , KV 字节降为 MHA 的四分之一。这个节省不是“免费优化”: 模型结构必须支持 GQA, kernel 必须正确实现  $q\_head \rightarrow kv\_head$  映射, oracle 必须覆盖 head mapping 错误。

#### prefill 和 decode 的 FLOPs/bytes 分阶段上限

对一个线性层  $y = xW$ , prefill 输入是  $[T, H]$ , 权重是  $[H, O]$ , FLOPs 约为  $2THO$ 。权重读入后可被  $T$  个 token 复用;  $T$  越大, 算术强度越高。decode 输入是  $[1, H]$ , FLOPs 约为  $2HO$ , 同一层权重通常只服务一个 token 或小 batch, 权重复用弱。

$$AI_{prefill} \approx \frac{2THO}{bytes(W) + bytes(X) + bytes(Y)}$$

$$AI_{decode} \approx \frac{2HO}{bytes(W) + bytes(x) + bytes(y)}$$

当  $H = O = 4096$ , fp16 权重约 32MiB, 单 token decode 约 33.5M FLOPs。若只看权重读, 算术强度约:

$$\frac{33.5M}{32MiB} \approx 1 \text{ FLOP/byte}$$

这很容易被内存带宽限制。int8 把权重字节减半, 算术强度提高; int4 再减半, 但 scale、解包、group 元数据和 kernel 支持会吃掉一部分收益。真正报告要同时列权重 bytes、metadata bytes、dequant 成本和实际带宽。

Listing 2.26 decode token 的最小带宽账本

```

for each layer:
 read RMSNorm weights
 read QKV/O projection weights
 read MLP gate/up/down weights
 read KV cache for current context
 write updated K/V
 write hidden/logits intermediates

token lower bound:
 max(weight_bytes / weight_bandwidth,
 kv_bytes / kv_bandwidth,
 flops / compute_throughput)

```

### RoPE 位置缩放和长上下文错误

RoPE 的基础公式可以看成对偶维度旋转。对一对维度  $(x_{2i}, x_{2i+1})$ ，位置  $p$  和频率  $\theta_i$  给出：

$$x'_{2i} = x_{2i} \cos(p\theta_i) - x_{2i+1} \sin(p\theta_i)$$

$$x'_{2i+1} = x_{2i} \sin(p\theta_i) + x_{2i+1} \cos(p\theta_i)$$

错误常发生在三个位置。第一，把 batch 内列号当作真实 model position，left padding 时错。第二，把 paged KV 的 physical slot 当作 position，cache 回收或滑窗时错。第三，长上下文扩展使用 position scaling 或不同 base，却没有和训练/模型配置一致。短 prompt 下这些错误可能几乎看不出来，长上下文才表现为质量下降。

Listing 2.27 RoPE trace 必须拆开的三个位置

```

token_index_in_request
model_position
kv_physical_slot
rope_table_index
sliding_window_base

```

因此 RoPE 的验收不应只比较一句短 prompt logits。要固定一个带 left padding、prefix reuse、sliding window 或 paged slot 的 trace，dump 旋转前后 Q/K 的小片段，并验证 `model_position` 与 `physical_slot` 分离。

### 采样数值边界：temperature、top-k、top-p 不应掩盖 logits 错误

采样把 logits 转成 token，但它不是模型 correctness 的替代品。temperature 会缩放 logits：

$$z'_i = \frac{z_i}{T}$$

当  $T$  很小，最大 logit 的微小误差可能改变 argmax；当  $T$  很大，分布变平，低概率 token 更容易出现。top-k 截断保留前  $k$  个 token；top-p 保留累计概率达到阈值的最小集合。它们都对排序和概率归一化敏感。

| 采样参数          | 数值敏感点            | 测试方式                                 |
|---------------|------------------|--------------------------------------|
| temperature 小 | logit 排名微小变化影响输出 | 比较 logits/top candidates, 不只比较 token |
| top-k         | 第 k/k+1 名边界敏感    | 构造接近分数                               |
| top-p         | 累计概率和排序敏感        | 固定 logits, 验证保留集合                    |
| 随机采样          | RNG 状态影响复现       | seed、counter、sampler state dump      |

工程上应先用 greedy 或固定 sampler 验证 decoder logits, 再验证 sampler。若优化后 token 不同, 第一步不是争论“采样随机”, 而是比较 logits、top candidates、概率集合和 RNG 状态。

## 2.2 状态机：一个 token 穿过 decoder

Transformer 章节不能只列公式, 还要让读者跟踪同一个 token 的状态。下面这台状态机是 reference decoder、KV cache、compiler state edge 和 serving scheduler 的共同底图。

| 状态            | 张量形状示例                               | 关键合同                             |
|---------------|--------------------------------------|----------------------------------|
| TokenId       | [1] int32                            | tokenizer 输出稳定, 位置已知             |
| Embedding     | [1, H] fp16/fp32                     | token id 不越界, dtype 转换明确         |
| NormedHidden  | [1, H]                               | RMSNorm epsilon 和精度合同            |
| QKVPProjected | $q: [n_q, d], k/v: [n_{kv}, d]$      | GQA head 映射明确                    |
| RopeApplied   | 同 Q/K                                | model position 不等于 physical slot |
| KvAppended    | cache 写入<br>[layer, pos, kv_head, d] | state write 不可删除                 |
| AttentionOut  | [1, H]                               | 读取历史 [0..pos], mask 正确           |
| MlpOut        | [1, H]                               | SwiGLU/activation 数值稳定           |
| Logits        | [V]                                  | lm head 和 tolerance 明确           |
| SampledToken  | [1] int32                            | sampler state 和 RNG 可复现          |

这张表的训练价值在于：每一步都能 dump、比较和定位。若最终 token 错, 不要从自然语言文本猜原因, 而是沿状态机比较 embedding、RMSNorm、Q/K/V、RoPE 后 Q/K、KV append、attention score、MLP、logits、sampler state。

### 小模型手算：GQA head 映射和 KV 地址

设  $n_q = 4, n_{kv} = 2, d = 2, seq = 3$ 。GQA 映射通常是：

$$kv\_head = \left\lfloor q\_head \times \frac{n_{kv}}{n_q} \right\rfloor$$

因此  $q\_head = 0, 1$  读  $kv\_head = 0$ ,  $q\_head = 2, 3$  读  $kv\_head = 1$ 。若 KV cache layout 是 [layer, position, kv\_head, dim], 地址偏移为：

$$offset = (((layer \times S + position) \times n_{kv} + kv\_head) \times d + dim)$$

当  $layer = 1, position = 2, kv\_head = 1, dim = 0, S = 3, n_{kv} = 2, d = 2$ ：

$$offset = (((1 \times 3 + 2) \times 2 + 1) \times 2 + 0) = 22$$

这个手算例子能暴露两类真实 bug：把  $q\_head$  直接当  $kv\_head$  会越界或读错；把 physical slot 当 model position 会在 paged cache、prefix reuse 或 sliding window 下旋转错误。



## 数值边界：RMSNorm、softmax 和低精度误差传播

RMSNorm 的核心是：

$$y_i = x_i \cdot \frac{w_i}{\sqrt{\frac{1}{H} \sum_j x_j^2 + \epsilon}}$$

$\epsilon$  不是装饰。它决定接近零向量时的稳定性，也影响低精度路径和 reference 的误差界。softmax 也必须用 max-subtraction：

$$\text{softmax}(z_i) = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}, \quad m = \max_j z_j$$

若直接对大 logits 求 exp，会 overflow；若低精度 accumulation 太粗，top-k 边界可能变化。量化误差、RoPE 三角表误差、attention score 误差和 MLP 误差会层层传播；因此第三册的 oracle 要比较中间 tensor，而不是只看最终 token。

### 实验：固定 token trace

#### 习题与作业

实验一：固定 tiny transformer 参数  $\text{vocab} = 16, H = 8, \text{layers} = 2, n_q = 2, n_{kv} = 1, d = 4, \text{seq} = 4$ 。输出每个 token 在每层的 shape、dtype、stride、KV offset 和 checksum。  
 实验二：故意把 GQA 映射写成  $\text{kv\_head} = \text{q\_head}$ 。要求测试在  $n_q > n_{kv}$  时失败，并打印错误 head、position 和 offset。  
 实验三：构造 left padding 或 prefix reuse trace。验证 `model_position`、`kv_physical_slot` 和 `rope_table_index` 三者分离。  
 实验四：比较 naive softmax 和 max-subtraction softmax。输入包含大正数、大负数和接近 top-k 边界的 logits，报告 overflow、underflow 和 token 变化。

### CPU 后端、GPU 后端和同一份算子合同

理解原理后，再谈后端。CPU 和 GPU 不应该各自定义一套模型语义。它们应该共享同一份算子合同：输入 shape、dtype、layout、量化 profile、误差界、KV cache 逻辑坐标一致。不同的是物理实现。

Listing 2.28 共享算子合同，分离后端实现

```
OperatorContract:
 op_name
 input_shape
 output_shape
 dtype
 quant_profile
 logical_layout
 accuracy_tolerance

CpuKernel:
 packed_weight_layout
 simd_width
 thread_plan
 numa_policy

GpuKernel:
 device_layout
 block_shape
 stream
```

```
workspace
copy_policy
```

CPU 后端的优势是可观察：可以看汇编、cache miss、TLB miss、线程扩展曲线和 NUMA 行为。GPU 后端的优势是吞吐：prefill 大 GEMM、attention kernel 和显存带宽更强，但 kernel launch、host-device 同步和小 batch decode 会带来固定成本。工程上应先让 CPU reference 和 CPU optimized path 把语义、oracle、profiler 建稳，再把同一合同挂到 GPU 后端。

## 硬验收

读完本节，应该能回答下面问题：

- 文本怎样变成 token id，token id 怎样变成 hidden，hidden 怎样变成 logits。
- logits 和 sampler 的关系是什么，为什么 sampler 不应该掩盖模型计算错误。
- decoder block 中 Q/K/V、attention、MLP、residual、norm 分别做什么。
- KV cache 省掉了哪些重复计算，又引入了哪些内存和读带宽压力。
- 写出 KV cache 的逻辑形状和容量公式，并能做 7B 级粗略估算。
- 解释 prefill 为什么更像 GEMM，decode 为什么更像 GEMV 加历史 KV 读取。
- 说明为什么 decode 常常受权重带宽、KV cache 读流、小 kernel 和 sampler 同步影响。
- 用一组 7B 级参数粗算单层投影、MLP、KV cache 容量和 decode 权重读流。
- 用 roofline 思路判断当前问题更像算力瓶颈、带宽瓶颈还是同步/采样瓶颈。

本节之后再看量化、memory planner、AI 编译器和后端优化，才有稳定落点：不是为了堆名词，而是为了让这条从 prompt 到 next token 的链路更正确、更快、更可测、更能落地成服务。

## 2.3 Reference decoder 实现：先把一层算对

### 为什么必须先写 reference decoder

很多推理项目失败，不是因为缺少 SIMD 或 GPU，而是因为一开始没有可信 reference。后端越快，越难调；量化越低，越难判断误差来源；KV cache 越复杂，越容易把位置、head 映射和缓存槽混在一起。reference decoder 的作用是把数学合同写成最朴素、最容易审计的 C++20 代码，让后续所有优化都有比较对象。

### 核心思想

reference decoder 不是低质量版本，而是正确性基准。它可以慢，但必须清楚、可复现、少隐藏状态、少复用复杂优化代码。

本节目标不是实现完整 7B 框架，而是实现一个小型 decoder slice：token embedding、RMSNorm、Q/K/V linear、RoPE、causal attention、MLP、lm head 和 sampler 前 logits。这个 slice 可以用极小维度，例如 **H=16**、**heads=2**、**head\_dim=8**、**layers=1**，也可以用真实 shape 的小权重 fixture。重点是每一步都能输出中间张量并被 oracle 比较。

为了让每一步都能手算，本节使用固定的 TinyTrace-16：

**Listing 2.29** TinyTrace-16 decoder 参数

```
vocab = 16
hidden H = 8
layers L = 2
query_heads = 2
kv_heads = 1
head_dim = 4
intermediate I = 16
prompt token ids = [3, 1, 4, 2]
```

真实 7B 模型只是把这些维度放大: **H** 从 8 变 4096, **L** 从 2 变 32, **I** 从 16 变 11008, **vocab** 从 16 变几万。reference decoder 的价值在于: TinyTrace 上每一步能手算, 真实 shape 上同一套 shape 合同仍然成立。

### reference 的边界

reference decoder 应该故意避开复杂优化:

- 不做 packed weight, 直接读普通 row-major 权重。
- 不做 SIMD intrinsic, 使用普通循环。
- 不做线程并行, 避免浮点归约顺序变化。
- 不复用优化后端的 int4 解包或 GPU kernel。
- 不把多个算子融合成一个不可观察的大函数。

它应该保留的能力是检查和观察:

- 每个张量都有 shape、dtype 和 layout。
- 每个阶段能选择性 dump 中间结果。
- 每个算子能独立运行 fixture。
- 每个错误能指出哪一层、哪一个 head、哪一个 position。
- 固定 seed 的 sampler 能复现。

### 最小类型: 先让 shape 说话

不要让裸指针和裸长度在 reference 中到处流动。最小实现也需要一个张量视图。

Listing 2.30 reference TensorView 的语义字段

```
TensorViewF32:
 data: span<float>
 shape: vector<int64>
 stride: vector<int64>
 name: string_view

requirements:
 shape.size == stride.size
 every index is checked in debug mode
 view does not own memory
 owner outlives view
```

reference 里可以接受 debug 检查稍慢, 因为它换来定位能力。优化后端可以在 verifier 通过后走 unchecked fast path, 但 reference 不应该这样做。每次 oracle 报错时, 张量名、shape 和索引比一个段错误有用得多。

模型参数也要结构化, 不要用字符串 map 在热路径查。

Listing 2.31 一层 decoder 的 reference 权重集合

```
DecoderLayerWeights:
 rms_attn_weight: [H]
 wq: [Q, H]
 wk: [KV, H]
 wv: [KV, H]
 wo: [H, Q]
 rms_mlp_weight: [H]
 w_gate: [I, H]
 w_up: [I, H]
 w_down: [H, I]
```

```
ModelWeights:
 token_embedding: [V, H]
 layers: array<DecoderLayerWeights>
 final_norm_weight: [H]
 lm_head: [V, H]
```

这里使用 `[out,in]` 形式表达线性层权重，和前面 `matvec` 章节一致：输出通道是行，输入通道是列。若模型文件来自别的布局，导入阶段应该转换或记录 `layout`，不要让 `reference` 每次猜。

### RMSNorm：第一个数值合同

RMSNorm 的公式是：

$$y_i = x_i \cdot \frac{1}{\sqrt{\frac{1}{H} \sum_j x_j^2 + \epsilon}} \cdot w_i$$

`reference` 实现要固定三件事：累加精度、`epsilon`、输出 `dtype`。建议 `reference` 用 `fp32` 累加，即使真实后端用 `fp16/bf16`。原因是 `reference` 要更接近数学定义，而不是模仿某个硬件的舍入。

Listing 2.32 RMSNorm `reference` 伪代码

```
sum_sq = 0
for i in 0..H:
 sum_sq += x[i] * x[i]

scale = rsqrt(sum_sq / H + epsilon)

for i in 0..H:
 y[i] = x[i] * scale * weight[i]
```

RMSNorm 的测试不要只测随机数。至少包含全零输入、极小值、大值、负值和 `H` 不能被 SIMD 宽度整除的形状。虽然 `reference` 不用 SIMD，但后续 `optimized path` 会用这些 `fixture`。

### Linear：所有大算子的共同骨架

`decoder` 中多数重计算都是 `linear`。`reference` 版本应尽量朴素：

Listing 2.33 Linear `reference` 语义

```
for out in 0..OUT:
 acc = bias_or_zero(out)
 for in in 0..IN:
 acc += weight[out, in] * x[in]
 y[out] = acc
```

这段代码要配 `shape` 检查：

Listing 2.34 Linear `shape` 合同

```
x.shape == [IN]
weight.shape == [OUT, IN]
y.shape == [OUT]
if bias exists:
 bias.shape == [OUT]
```

后续 CPU 后端的 packed weight、int8、int4、SIMD micro-kernel 都要和这个 reference 比较。若 reference 自己也绕过 shape 或复用 packed descriptor，oracle 就失去意义。

### RoPE：位置编号必须作为参数传入

RoPE 不能从数组下标偷偷推导位置。reference 函数应该显式接收 `model_position`：

Listing 2.35 RoPE reference 合同

```
apply_rope(q_or_k, head_count, head_dim, model_position, rope_config):
 for head in 0..head_count:
 for pair in 0..head_dim/2:
 theta = frequency(pair, rope_config)
 angle = model_position * theta
 rotate two adjacent dimensions by angle
```

测试要覆盖下面几类错误：

- position 为 0 时旋转应保持不变。
- 同一向量在不同 position 输出不同。
- prefix 长度为 `P` 时，新 token 的 position 是 `P`，不是 0。
- 滑动窗口或分页 cache 改变物理槽时，position 不随槽回绕。

RoPE 的 oracle 不只比较最终 logits。最好在 tiny fixture 里直接比较旋转后的 Q/K。这样一旦长上下文漂移，可以更早定位到位置合同。

### KV cache：reference 也要严格建模状态

即使 reference 很慢，也不能每步重新计算所有历史 K/V 后假装有 KV cache。那样无法测试真实运行时状态。reference 的 KV cache 可以用普通连续数组：

Listing 2.36 KV cache reference 字段

```
KVCache:
 layers
 max_context
 kv_heads
 head_dim
 key: [layers, max_context, kv_heads, head_dim]
 value: [layers, max_context, kv_heads, head_dim]
 filled_tokens
```

append 合同必须检查：

- 写入 position 小于 `max_context`。
- K/V 的 head 数是 `kv_heads`，不是 query head 数。
- 写入后 `filled_tokens` 和 session position 一致。
- debug 模式下不能覆盖已经写入但未被窗口策略淘汰的槽。

GQA 读取时要从 query head 映射到 KV head：

Listing 2.37 GQA head 映射

```
group_size = query_heads / kv_heads
kv_head = query_head / group_size
```

这个公式必须由 verifier 保证整除。若 `query_heads` 不能整除 `kv_heads`，模型资产应在加载阶段被拒绝，而不是让 attention 函数自己补救。

**Causal attention: 先写能看懂的版本**

decode 阶段的单 token attention 最适合作为 reference 起点:

**Listing 2.38** 单 token decode attention reference

```
for q_head in 0..query_heads:
 kv_head = map_query_to_kv(q_head)

 for pos in 0..current_position:
 score[pos] = dot(q[q_head], key_cache[pos, kv_head]) / sqrt(head_dim)

 prob = softmax(score[0..current_position])

 out[q_head] = 0
 for pos in 0..current_position:
 out[q_head] += prob[pos] * value_cache[pos, kv_head]
```

softmax 要做数值稳定处理:

**Listing 2.39** 稳定 softmax

```
max_score = max(score)
sum = 0
for i:
 e[i] = exp(score[i] - max_score)
 sum += e[i]
for i:
 p[i] = e[i] / sum
```

prefill reference 可以先用同一函数逐位置执行,再逐步加批量版本。这样容易证明 causal mask: 第  $i$  个 prompt token 只能看  $0..i$ 。后续优化可以把 prefill attention 做成矩阵形式或 fused kernel, 但 oracle 仍然回到这个语义。

**MLP: SwiGLU 的中间值不能丢**

SwiGLU 的 reference 要保留 gate、up 和乘积中间值:

**Listing 2.40** SwiGLU MLP reference

```
gate = linear(w_gate, x)
up = linear(w_up, x)

for i in 0..I:
 hidden[i] = silu(gate[i]) * up[i]

down = linear(w_down, hidden)
```

$\text{silu}(x)=x/(1+\exp(-x))$ 。这里的中间值很重要,因为 MLP 往往是 FLOPs 大头。若最终 logits 漂移,保留 gate/up/down 的 oracle checkpoint 能迅速判断是第一段线性、激活、逐元素乘,还是 down projection 出错。

**单层 forward: 把状态边写出来**

一层 decoder forward 可以写成下面的形态:

**Listing 2.41** 单层 decoder reference forward



```

forward_layer(layer_id, x, model_position, kv_cache):
 h = rms_norm(x, rms_attn_weight)

 q = linear(wq, h)
 k = linear(wk, h)
 v = linear(wv, h)

 apply_rope(q, query_heads, head_dim, model_position)
 apply_rope(k, kv_heads, head_dim, model_position)

 kv_cache.append(layer_id, model_position, k, v)

 attn = attention_decode(q, kv_cache, layer_id, model_position)
 x = x + linear(wo, attn)

 h = rms_norm(x, rms_mlp_weight)
 mlp = swiglu_mlp(h)
 x = x + mlp

 return x

```

注意 `kv_cache.append` 是状态写入，`attention_decode` 是状态读取。reference 也要把这两步分开记录。若后续 memory planner 错误复用 buffer，或者 decode step 提前读取未写完的 K/V，trace 才能定位。

### TinyTrace 的完整 shape trace

下面只跟踪一个 token 的一层 forward。设当前 token 是 prompt 中第 2 个 token，`model_position=2`，进入第 0 层前 hidden 形状是 `[H]=[8]`。

| 阶段            | 输出 shape       | 必须检查的合同                                          |
|---------------|----------------|--------------------------------------------------|
| embedding     | [8]            | token id 在 [0,16) 内                              |
| RMSNorm(attn) | [8]            | weight 是 [8]，epsilon 固定                          |
| Q linear      | [8]            | $W_q=[8,8]$                                      |
| K linear      | [4]            | $W_k=[4,8]$ ，因为 <code>kv_heads*head_dim=4</code> |
| V linear      | [4]            | $W_v=[4,8]$                                      |
| RoPE(Q)       | [2,4]          | 使用 <code>position=2</code> ，不是 cache slot        |
| RoPE(K)       | [1,4]          | KV head 数不同于 query head 数                        |
| KV append     | cache[0,2,0,:] | 写第 0 层、逻辑位置 2、KV head 0                          |
| attention     | [2,4]->[8]     | 每个 query head 读 pos0..2                          |
| O linear      | [8]            | $W_o=[8,8]$                                      |
| RMSNorm(MLP)  | [8]            | residual 后再 norm                                 |
| gate/up       | [16], [16]     | $W_{gate}/W_{up}=[16,8]$                         |
| SwiGLU hidden | [16]           | $\text{silu}(\text{gate}) * \text{up}$ 逐元素       |
| down          | [8]            | $W_{down}=[8,16]$                                |
| layer output  | [8]            | 第二次 residual                                     |

跑完整个 prompt 后，最后一个 token 的 hidden 进入 final norm 和 lm head：

Listing 2.42 从最后 hidden 到 logits 的 shape trace

```
last_hidden: [8]
final_norm_weight: [8]
normed: [8]
lm_head weight: [16, 8]
logits: [16]
predicted_token = argmax(logits)
```

这条 trace 是 reference decoder 的最低门禁。若某个实现只告诉你“logits shape 是 [16]”，但不能说明第 0 层 position 2 的 K 写到了哪个 cache 坐标、query head 1 映射到哪个 KV head，它还不是可调试的 decoder。

### 把 shape trace 绑定到仓库代码

本仓库的 reference slice 落在 `books/cpu-volume-3/labs/linux_cpu_inference/src/reference_decoder.cpp`。它不是完整生产 runtime，但已经覆盖了这条链路的关键函数：

- `rms_norm`：用 fp32 累加定义 norm 合同。
- `linear_f32`：定义 row-major [out,in] 权重语义。
- `apply_rope`：显式接收 `model_position`。
- `ReferenceKVCache::append/key/value`：按 `layer, position, kv_head, dim` 寻址。
- `attention_decode`：实现 GQA head 映射和稳定 softmax。
- `run_reference_decode`：从 token ids 跑到 logits 和 argmax。
- `run_reference_decode_with_trace`：输出逐 checkpoint 的 shape、stride、dtype、layout、checksum、max abs 和 values 摘要。

测试入口在 `books/cpu-volume-3/labs/linux_cpu_inference/tests/lcqi_tests.cpp`。目前它覆盖 RMSNorm、RoPE、KV attention、reference decode end-to-end、reference trace contract、packed int8 kernel 与 scalar 对齐。trace contract 会检查关键 checkpoint 的 shape、stride、dtype、layout 和 logits golden，避免 reference decoder 只验证最终 token。

当前仓库已经把 tiny reference decoder 的最终 logits 固化成 golden，而不是只检查输出范围。固定输入 `tokens=[1,2,3]`，`make_tiny_reference_decoder_model()` 的输出是：

Listing 2.43 lcqi reference decoder golden logits

```
predicted_token = 0
logits[0] = 0.352516979
logits[1] = -0.126884326
logits[2] = 0.0797837451
logits[3] = -0.29797405
logits[4] = 0.127030417
```

对应测试常量在 `lcqi_tests.cpp` 中记录为 `LCQI_REFERENCE_DECODER_LOGITS`。这仍然不是完整 oracle，但它比“predicted token 在 vocab 范围内”强得多：任何 RMSNorm、RoPE、KV append/read、attention、SwiGLU 或 lm head 的语义漂移，都会先撞到这个 logits golden。

### 逐张量 checkpoint 已经能导出

最终 logits 只能说明端到端错了，不能说明错在哪里。当前 lab 新增了 `lcqi_trace` 命令，固定 prompt fixture `tok1tok2tok3`，tokenizer 输出 `tokens=[1,2,3]`，然后导出逐 checkpoint CSV：

Listing 2.44 reference trace 复现命令

```
books/cpu-volume-3/build/lcqi-release/labs/linux_cpu_inference/lcqi_trace \
```

```
> books/cpu-volume-3/results/lcqi-reference-trace-2026-06-23.csv
```

CSV 字段如下：

**Listing 2.45** TinyTrace checkpoint 记录

```
name,token_position,layer_id,shape,stride,dtype,layout,checksum,max_abs,values
```

前几行真实输出如下：

**Listing 2.46** lcqi reference trace CSV 摘要

```
prompt,0,-1,scalar,scalar,utf8,text,0,0,tok1 tok2 tok3
tokenizer,0,-1,3,1,i32,contiguous,6,3,1;2;3
embedding,0,-1,4,1,f32,row_major,-0.25,0.4,-0.4;0.1;0.25;-0.2
attn_norm,0,0,4,1,f32,contiguous↵
↵ ,-0.900291,1.53241,-1.53241;0.344792;1.05353;-0.766205
q_before_rope,0,0,2x2,2x1,f32,contiguous↵
↵ ,-0.769078,0.518146,-0.518146;0.459723;-0.367778;-0.342877
k_before_rope,0,0,1x2,2x1,f32,contiguous,0.317017,0.544963,-0.227946;0.544963
kv_cache_key_slot,0,0,1x1x1x2,2x2x2x1,f32,kv_contiguous↵
↵ ,0.317017,0.544963,-0.227946;0.544963
```

这里已经包含 shape、stride、dtype、layout 和少量 values。比如 `q_before_rope` 的 shape 是 `2x2`，stride 是 `2x1`，说明它按 `[query_head, head_dim]` 连续布局解释；`kv_cache_key_slot` 的 shape 是 `1x1x1x2`，对应 `[layer, position, kv_head, dim]` 的单槽切片，layout 显式标成 `kv_contiguous`。

CSV 末尾会把 logits 接到 sampler 和新 token：

**Listing 2.47** lcqi reference trace 的输出尾部

```
final_norm,2,-1,4,1,f32,contiguous↵
↵ ,0.760889,1.72225,1.72225;-0.745899;0.342036;-0.557496
logits,2,-1,5,1,f32,contiguous↵
↵ ,0.134473,0.352517,0.352517;-0.126884;0.0797838;-0.297974;0.12703
sampler_argmax,2,-1,5,1,f32,argmax,0,0,token=0
generated_token,3,-1,scalar,scalar,i32,generated,0,0,0
```

这就把 `prompt->tokenizer->embedding->RMSNorm->QKV->RoPE->attention->KVcache->MLP->logits->sampler->token` 放进同一个可复现实验。它仍然是 tiny fixture，不是生产 tokenizer 或采样器；但它已经足够用来训练读者逐步定位 shape、layout 和数值漂移。

例如 position 2、layer 0 的关键 checkpoint：

| checkpoint    | shape                 | 定位能力                       |
|---------------|-----------------------|----------------------------|
| emb           | [8]                   | token id 和 embedding 行     |
| attn_norm     | [8]                   | RMSNorm epsilon 和权重        |
| q_before_rope | [2, 4]                | Q projection 和 reshape     |
| k_after_rope  | [1, 4]                | K projection、RoPE position |
| kv_slot       | [layer, pos, kv_head] | cache 地址和 GQA head         |
| attn_prob_h1  | [3]                   | causal mask 和 softmax      |
| mlp_gate      | [16]                  | gate projection            |
| mlp_hidden    | [16]                  | silu 和逐元素乘                 |
| layer_out     | [8]                   | residual 和 buffer 覆盖       |

这些 checkpoint 不要求每次 CI 都存完整向量。常跑测试可以只比 `checksum/max_abs/topk`；漂移调试时再打开完整 dump。关键是 checkpoint 名称和 shape 要稳定，否则两次报告无法对齐。

### 误差边界要随 checkpoint 分层

不同 checkpoint 对误差的容忍度不同。reference 对 reference 应该接近完全一致；优化后端若只改变循环顺序，可以用很小阈值；量化后端必须看更高层语义。

| 比较对象                        | 推荐阈值                | 失败解释                    |
|-----------------------------|---------------------|-------------------------|
| f32 reference vs f32 scalar | <b>1e-5</b> abs/rel | shape、权重布局、代码错误         |
| f32 scalar vs SIMD f32      | <b>1e-4</b> abs/rel | 归约顺序或 SIMD lane 错       |
| int8 operator               | profile dependent   | scale、zero point、累加策略   |
| RoPE checkpoint             | <b>1e-5</b> abs/rel | position、pairing、base 错 |
| attention prob              | <b>1e-4</b> abs/rel | mask、softmax 稳定性、KV 映射  |
| logits top-k                | overlap gate        | 量化质量和可见输出漂移             |

不要用一个全局 `epsilon` 盖住所有层。RMSNorm 的小误差可能还能接受；attention score 的小误差经过 softmax 后可能改变概率；logits 的小差异若正好发生在 top-1/top-2 附近，会直接改变采样结果。

### 端到端 reference decode loop

端到端 reference 可以先只输出 logits，不做复杂采样：

Listing 2.48 reference decode loop

```
tokens = tokenizer_fixture.encode(prompt)

for pos in 0..tokens.size:
 x = embedding[tokens[pos]]
 for layer in 0..layers:
 x = forward_layer(layer, x, pos, kv_cache)

last_hidden = x
normed = rms_norm(last_hidden, final_norm_weight)
logits = linear(lm_head, normed)
```

prefill 的最后一个 token 产生首个 logits。之后 decode 追加 sampler 选出的 token：

Listing 2.49 decode 生成循环 reference

```

for step in 0..max_new_tokens:
 next_id = sampler(logits, seed, params)
 tokens.push_back(next_id)

 pos = tokens.size - 1
 x = embedding[next_id]
 for layer in 0..layers:
 x = forward_layer(layer, x, pos, kv_cache)

 logits = linear(lm_head, rms_norm(x, final_norm_weight))

```

这里故意没有 batch、线程和 fusion。它的价值是每个 token、每层、每个中间值都能被检查。

### Golden fixture 怎么设计

reference 必须配 golden fixture。一个高质量 fixture 至少包含：

**Listing 2.50** tiny decoder fixture 内容

```

model_dims:
 vocab
 hidden
 layers
 query_heads
 kv_heads
 head_dim
 intermediate
 max_context

weights:
 token_embedding
 layer weights
 final_norm
 lm_head

input:
 prompt_token_ids
 sampler_params
 seed

expected:
 checkpoints
 logits
 generated_token_ids

```

权重不要全用随机数。建议混合使用小整数、正负数、接近零的值和非对称值。全随机 fixture 容易覆盖到计算路径，却不容易人工定位错误。比如 RoPE 可以用只有两个维度非零的向量，attention 可以用能手算 softmax 排序的 Q/K/V。

### checkpoint 策略

不要只保存最终 logits。推荐 checkpoint 分层：

| checkpoint     | 目的                    | 常见错误               |
|----------------|-----------------------|--------------------|
| embedding      | tokenizer 和查表         | token id 错、词表错     |
| norm output    | RMSNorm               | epsilon、累加精度       |
| q/k/v          | linear 与 head reshape | 权重转置、shape 错       |
| rope q/k       | 位置编码                  | position、维度配对      |
| attention prob | mask 与 softmax        | 未来泄漏、溢出            |
| attention out  | KV cache 读取           | head 映射、cache slot |
| mlp hidden     | SwiGLU                | 激活、逐元素乘            |
| layer output   | residual              | 加法位置、覆盖 buffer     |
| logits top-k   | 可见输出                  | lm head、采样差异       |

checkpoint 太多会增加文件体积, 可以通过 `trace_level` 控制。日常测试比较关键 checkpoint; 调试漂移时打开 debug 全量输出。

### reference 和优化后端怎样比较

比较不能只看完全相等。不同 dtype 和不同归约顺序会造成微小差异。oracle 应按层级使用不同阈值:

**Listing 2.51** oracle tolerance profile

```
f32_scalar:
 absolute: 1e-5
 relative: 1e-5

f16_or_bf16:
 absolute: 2e-2
 relative: 2e-2

int8_weight:
 per_operator_absolute: profile dependent
 logits_topk_overlap: required

int4_weight:
 compare logits, top-k and generation under fixed prompt set
```

阈值必须随 profile 记录到报告里。不要让测试代码里一个魔法数决定所有后端。特别是量化后端, operator 误差、logits top-k 和生成序列要一起看; 单个最终文本“看起来不错”不能证明模型没有漂移。

### 硬验收

读完本节, 应该能动手写一个小型 reference decoder:

- 用 `TensorView` 和结构化权重表达 shape, 而不是裸指针乱传。
- 分别实现 RMSNorm、Linear、RoPE、KV append/read、attention、SwiGLU 和 lm head。
- 显式传入 `model_position`, 不把物理 cache slot 当 RoPE 位置。
- 用 tiny fixture 保存 checkpoint 和 logits golden。
- 用 oracle profile 比较 reference 与优化后端。
- 把 reference 当正确性基准, 而不是被优化路径污染的慢版本。

## 2.4 推理算法: logits、softmax、采样、搜索与投机解码



## 模型输出的不是文字，而是分数分布

decoder 最后一层输出 hidden 后，**lm\_head** 会得到词表大小的 logits。logits 是每个 token 的未归一化分数，不是概率，也不是最终文字。推理算法的任务是把 logits 变成下一个 token id。

### 核心思想

“模型会不会回答得好”不只由权重决定，也由解码算法决定。贪心、温度、top-k、top-p、重复惩罚、beam search 和投机解码都会改变输出、延迟、可复现性和服务成本。

如果把模型主体看成 **hidden->logits**，采样器就是 **logits->nexttoken**。它在数学上很小，在业务上很重要：同一组 logits，贪心可能稳定但死板，高温采样可能多样但不稳定，top-p 可能减少低质量尾部 token，重复惩罚可能降低复读，但也可能破坏术语一致性。

### softmax 和数值稳定

softmax 把 logits 转成概率：

$$p_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

直接计算容易溢出。稳定做法是减去最大值：

$$p_i = \frac{e^{z_i - z_{max}}}{\sum_j e^{z_j - z_{max}}}$$

这不会改变概率，因为分子分母同时乘了同一个常数。采样器必须使用稳定 softmax，尤其在低温或 logits 幅度较大时。

Listing 2.52 稳定 softmax 伪代码

```
max_logit = max(logits)
sum = 0
for i in vocab:
 prob[i] = exp(logits[i] - max_logit)
 sum += prob[i]
for i in vocab:
 prob[i] /= sum
```

在工程实现中，完整 softmax 会扫描整个词表。若词表有三万到十几万 token，sampler 的成本并非完全可以忽略。GPU 后端若每步把完整 logits 拷回 CPU 再采样，会引入同步和传输成本；CPU 后端则要注意 top-k 选择和概率归一化的复杂度。

### 贪心解码

贪心解码直接选择最大 logit：

$$next = \arg \max_i z_i$$

它的优点是确定、简单、可复现，适合测试、oracle 和某些结构化任务。缺点是缺少多样性，容易陷入重复，且不一定得到全局最优序列。贪心解码不需要 softmax，因为最大 logit 对应最大概率。

Listing 2.53 贪心解码

```
best_id = 0
best_value = logits[0]
for i in 1..vocab:
 if logits[i] > best_value:
 best_value = logits[i]
 best_id = i
```

```
return best_id
```

reference 和回归测试常用贪心，因为它去掉了随机性。若贪心下 logits top-1 都不一致，先不要讨论采样策略，应该回到模型计算、量化或后端 oracle。

### 温度：控制分布尖锐程度

温度采样先把 logits 除以温度  $T$ ：

$$p_i = \text{softmax}(z_i/T)$$

$T$  越小，分布越尖锐，模型更接近贪心； $T$  越大，分布越平，低分 token 更容易被采到。温度不能简单理解成“创造力”。它改变的是概率分布熵，过高会增加胡言乱语，过低会增加模板化和重复。

| 温度   | 效果         | 常见用途        |
|------|------------|-------------|
| 接近 0 | 接近贪心，稳定    | 测试、代码、结构化输出 |
| 中等   | 保留多样性      | 问答、摘要、普通对话  |
| 较高   | 分布更平，随机性更强 | 创意任务，但风险高   |

实现上要防止  $T=0$  直接除零。工程中通常把  $T \leq 0$  解释成贪心，或者拒绝参数。

### top-k：只在前 k 个候选中采样

top-k 先保留 logit 最大的  $k$  个 token，把其他 token 概率置零，再归一化采样。它减少低质量尾部 token 的机会，也让采样成本从全词表 softmax 变成 top-k 选择加小集合 softmax。

Listing 2.54 top-k 采样流程

```
selected = find_top_k(logits, k)
for token not in selected:
 probability[token] = 0
softmax over selected logits
sample one token from selected distribution
```

$k=1$  等价于贪心。较大的  $k$  给更多多样性。top-k 的工程关键是选择算法。完整排序是  $O(V \log V)$ ，只取前  $k$  可以用 heap、partial selection 或后端专用 kernel。对每 token decode 来说，词表扫描成本会累积。

### top-p：按累计概率截断

top-p 又叫 nucleus sampling。它先按概率从大到小排序，选择最小集合，使累计概率超过  $p$ ：

$$\sum_{i \in S} p_i \geq p$$

然后只在集合  $S$  中重新归一化采样。top-p 的直觉是：当模型很确定时，候选集合小；当模型不确定时，候选集合大。相比固定 top-k，它能根据分布形态自适应。

代价是需要概率和排序或近似排序。若实现每步对全词表完整排序，sampler 可能成为短 decode 的显著成本。高性能实现通常会结合 top-k 上限、分块选择或 GPU sampler。

### 重复惩罚和频率惩罚

语言模型容易重复。重复惩罚会根据已生成 token 调整 logits。最简单的策略是：若 token 已出现，把正 logit 除以 penalty，把负 logit 乘以 penalty。这样已出现 token 的相对分数下降。

Listing 2.55 重复惩罚示意

```
for token in generated_tokens:
 if logits[token] > 0:
```

```

logits[token] /= penalty
else:
 logits[token] *= penalty

```

频率惩罚则按出现次数扣分。存在惩罚只看是否出现过。它们都是采样策略，不是模型本体。业务上要谨慎：代码、法律、医学、术语解释中，重复某些 token 可能是正确行为；过强惩罚会破坏格式、变量名和专有名词。

### 停止条件

生成循环不能只靠 `max_new_tokens` 停止。常见停止条件包括：

- 生成了模型的 EOS token。
- 生成了业务指定 stop token 或 stop sequence。
- 达到 `max_new_tokens`。
- 达到 `max_context_tokens`。
- 请求超时或被取消。
- sampler 发现非法分布，例如全部概率为零。

stop sequence 比 stop token 更难，因为它可能跨多个 token。服务端要在 token 流里维护一个小窗口，检查最近 token 是否匹配 stop sequence。若流式输出已经把 stop sequence 的一部分发给客户端，可能需要延迟输出或在业务协议中接受这种行为。不同产品会选择不同策略，但必须明确。

### 随机数和可复现性

随机采样必须有可控随机数源。固定 seed、固定 sampler 参数、固定 logits，应该得到同一 token 序列。多线程服务中不要使用全局共享随机数直接采样，否则请求之间会互相影响。

Listing 2.56 SamplerState 的最小字段

```

SamplerState:
 seed
 step
 temperature
 top_k
 top_p
 repetition_penalty
 generated_token_counts

```

把 `step` 放进状态是为了让 trace 可重放。若第 37 步出现异常，可以用同一 logits、同一 sampler state 复现该步选择。业务上，完全可复现有助于审计和回归；创意任务可以使用随机 seed，但也应该把 seed 记录到 trace。

### beam search：搜索多个候选序列

beam search 不是每步只保留一个 token，而是保留多个候选序列。每个候选有累计 log 概率。每一步扩展所有候选，保留得分最高的 `beam_width` 条。

Listing 2.57 beam search 伪代码

```

beams = [empty sequence with score 0]
for step in 0..max_new_tokens:
 candidates = []
 for beam in beams:
 logits = run_model(beam.sequence)
 for token in top_candidates(logits):
 candidates.add(beam + token, updated_score)
 beams = best beam_width candidates

```

beam search 常用于翻译、较短结构化任务或需要高概率序列的场景。但对聊天式大模型，它可能输出更保守、更模板化的文本，而且计算成本明显更高。若没有共享 KV cache 和批量扩展，beam width 为  $B$  就接近把 decode 成本放大  $B$  倍。实现 beam search 时，每个 beam 都有自己的 token 序列和 KV cache 分支，不能简单共享一个 session 状态。

### 投机解码：用小模型减少大模型步数

投机解码的目标是减少大模型 decode 次数。思路是用一个小模型先快速提出多个候选 token，再让大模型一次验证这些候选。若候选被接受，就相当于大模型一次前进多个 token；若被拒绝，则回退到大模型给出的分布。

简化流程：

Listing 2.58 投机解码简化流程

```
draft_tokens = small_model.generate_k_tokens(context)
big_model evaluates context + draft_tokens
for each draft token:
 accept if verification rule passes
 otherwise sample from corrected distribution and stop draft chain
```

投机解码不是简单“用小模型代替大模型”。它的关键是验证规则要保持目标大模型的分布不变或可控近似。工程上还要处理两套模型的 tokenizer 一致性、KV cache、显存/内存占用、调度和 trace。它适合在 decode 被大模型单步开销限制时使用，但会增加系统复杂度。

### 采样器放 CPU 还是 GPU

如果 logits 在 GPU 上，采样器有两种选择。把 logits 拷回 CPU 采样，实现简单，但每步有设备到主机拷贝和同步。把 sampler 放在 GPU，能减少同步，但实现 top-k/top-p、随机数和 stop 检查更复杂。

| 位置          | 优点                    | 风险                          |
|-------------|-----------------------|-----------------------------|
| CPU sampler | 简单、易调试、业务逻辑方便         | 每步 copy/sync, GPU decode 受阻 |
| GPU sampler | 减少同步，适合高吞吐            | 实现复杂, trace 和可复现性更难         |
| 混合策略        | top-k 在 GPU，最终选择在 CPU | 仍需管理拷贝和候选集合                 |

第一版可以先用 CPU sampler，但 trace 必须记录 logits copy 和 sampler 时间。否则接入 GPU 后，会误以为 kernel 很快就代表端到端快。

### 采样错误怎样定位

若生成结果异常，不要先改温度。按下面顺序定位：

1. 固定 sampler 为贪心，检查 logits top-1 是否和 reference 一致。
2. 固定 seed，检查同一 logits 下 sampler 是否复现。
3. 检查 temperature、top-k、top-p 的应用顺序。
4. 检查重复惩罚是否只作用于已生成 token，而不是 prompt 或全部词表。
5. 检查 stop token 和 stop sequence 是否在 token 边界处理。
6. 检查 GPU/CPU sampler 是否使用同一随机数和同一排序规则。

采样器看起来是“小组件”，但它决定最终可见输出。工程报告应该把 `sampler_profile` 记录到 trace，否则两次输出不同可能根本不是模型或后端问题。

### 硬验收

读完本节，应该能实现一个可控采样器：

- 知道 logits、softmax、概率和 token 选择的关系。
- 能实现稳定 softmax、贪心、温度、top-k、top-p 和重复惩罚。
- 能解释采样策略对确定性、多样性、质量和延迟的影响。
- 能设计 `SamplerState`，固定 seed 后复现生成序列。
- 能区分 beam search 和普通采样的 KV cache 成本。
- 能说明投机解码为什么需要验证，而不是直接相信小模型。
- 能把 sampler 时间、logits copy、seed 和参数写进 trace。

## 2.5 KV cache 实现细节：布局、分页、位置、量化与故障模式

### KV cache 是运行时数据结构，不只是公式

前面已经讲过 KV cache 的数学作用：历史 token 的 K/V 被保存下来，未来 decode step 可以复用。真正写引擎时，KV cache 不是一句“缓存 K/V”，而是一个运行时数据结构。它要回答：内存按什么布局放，逻辑 position 如何映射到物理槽，GQA head 怎样映射，多个 session 如何分配，超过上下文怎么办，量化后 scale 放哪里，取消时怎样释放。

#### 核心理想

KV cache 的正确性由三个坐标共同决定：layer、model position、KV head。物理内存可以连续、分页或回绕，但不能破坏这三个语义坐标。

下面用一个固定 tiny trace 连接 KV cache、reference decoder、memory planner 和 oracle：

Listing 2.59 TinyTrace-16 的 KV 参数

```
vocab = 16
hidden H = 8
layers L = 2
query_heads = 2
kv_heads = 1
head_dim = 4
sequence tokens = [3, 1, 4, 2]
max_context = 4
```

这个例子故意小到可以手算，但它保留了真实 decoder 的关键结构：`query_heads!=kv_heads`，所以有 GQA；`seq=4`，所以每一步 decode 读到的历史长度不同；`layers=2`，所以地址公式必须带 layer 维度，不能只看一个数组。

最朴素的 fp16 KV cache 形状是：

Listing 2.60 连续 KV cache 形状

```
key: [layers, max_context, kv_heads, head_dim]
value: [layers, max_context, kv_heads, head_dim]
```

这个布局容易理解，适合 reference 和早期实现。但生产级 runtime 常会遇到变长 session、并发、内存碎片、长上下文和 prefix 复用，连续大块内存不一定够灵活。

### 从第 0 个 token 手算到第 3 个 token

在 TinyTrace-16 中，每层、每个 token 写入一条 K 和一条 V。因为 `kv_heads=1, head_dim=4`，一条 K 是 4 个元素，一条 V 也是 4 个元素。fp16 下每元素 2 字节，所以每层每 token 的 KV 字节是：

$$2 \times kv\_heads \times head\_dim \times 2 = 2 \times 1 \times 4 \times 2 = 16 \text{ bytes}$$

两层就是每 token 32 字节；4 个 token 的整个 session KV cache 是 128 字节。这个数字很小，

但公式和 7B 完全一样。

| decode step | 写入                                  | attention 可读逻辑位置 | 每层读 KV 字节 (fp16) |
|-------------|-------------------------------------|------------------|------------------|
| 0           | $K_0, V_0 \rightarrow \text{pos}_0$ | 0                | 16               |
| 1           | $K_1, V_1 \rightarrow \text{pos}_1$ | 0, 1             | 32               |
| 2           | $K_2, V_2 \rightarrow \text{pos}_2$ | 0, 1, 2          | 48               |
| 3           | $K_3, V_3 \rightarrow \text{pos}_3$ | 0, 1, 2, 3       | 64               |

这里的关键不是 128 字节，而是增长规律：decode 第  $p$  步会读取  $p+1$  个历史位置的 K/V。长上下文变慢，不只是因为 softmax 更长，也是因为 KV 读字节随上下文线性增长。

### 连续布局地址公式

连续布局下，`key[layer, position, kv_head, dim]` 的元素偏移是：

$$\text{offset} = (((\text{layer} \times \text{max\_context} + \text{position}) \times \text{kv\_heads} + \text{kv\_head}) \times \text{head\_dim} + \text{dim})$$

TinyTrace-16 中 `max_context=4, kv_heads=1, head_dim=4`，所以：

$$\text{offset} = ((\text{layer} \times 4 + \text{position}) \times 1 + 0) \times 4 + \text{dim}$$

| 坐标                        | 元素偏移 | fp16 byte offset | 含义                      |
|---------------------------|------|------------------|-------------------------|
| <code>key[0,0,0,0]</code> | 0    | 0                | 第 0 层第 0 个 token 的 K 起点 |
| <code>key[0,1,0,0]</code> | 4    | 8                | 第 0 层第 1 个 token 的 K 起点 |
| <code>key[1,0,0,0]</code> | 16   | 32               | 第 1 层第 0 个 token 的 K 起点 |
| <code>key[1,3,0,2]</code> | 30   | 60               | 第 1 层第 3 个 token 的第 2 维 |

value 数组用同一套偏移公式，但 base pointer 不同。真实 runtime 可以把 K 和 V 分开，也可以交错存；无论哪种布局，都必须能把 `layer, position, kv_head, dim` 映射回确定地址。

### GQA head 映射的最小例子

TinyTrace-16 有 2 个 query head、1 个 KV head，因此：

$$\text{group\_size} = \text{query\_heads} / \text{kv\_heads} = 2$$

$$\text{kv\_head} = \text{query\_head} / \text{group\_size}$$

| query head | kv head | 读哪些 K/V                               |
|------------|---------|---------------------------------------|
| 0          | 0       | <code>key[layer, 0..pos, 0, :]</code> |
| 1          | 0       | <code>key[layer, 0..pos, 0, :]</code> |

这说明 GQA 不是少算 attention head，而是多个 query head 共享同一组 K/V。若代码把 `kv_head=query_head`，在 TinyTrace-16 的 query head 1 上就会越界；在某些更大的模型上则可能读到错误 head，程序不一定崩，但 logits 会漂移。

### 常见误区

KV cache 最危险的 bug 往往不是 shape mismatch，而是语义坐标错位：RoPE 用了物理 slot、GQA head 映射错、prefix 后 position 从 0 重启、或 sliding window 读到了已经淘汰的槽。这些错误都可能输出“看起来像中文/英文”的文本，但长上下文质量会突然下降。



## 逻辑 position 和物理 slot 分开

模型语义使用 `model_position`。内存使用 `slot`。连续布局里二者常常相等，但实现不要依赖这个巧合。

Listing 2.61 KV 逻辑到物理映射

```
KvAddress:
 layer_id
 model_position
 kv_head
 head_offset

PhysicalAddress:
 block_id
 slot_in_block
 byte_offset
```

RoPE 使用的是 `model_position`，attention mask 使用的是历史逻辑范围，内存读写使用的是物理地址。若滑动窗口把物理 slot 回绕，`model_position` 不能跟着回到 0。若 prefix cache 复用前 1024 个 token，新 token 的 position 应该从 1024 开始，而不是从新 session 的局部数组下标开始。

## 连续布局的优点和限制

连续布局可以按 session 一次分配最大上下文：

Listing 2.62 连续 KV 分配

```
bytes =
 layers * max_context * kv_heads * head_dim *
 2 /* K,V */ * bytes_per_element
```

优点是地址计算简单，cache locality 可预测，debug 容易。缺点是浪费：很多请求实际不会用满 `max_context`，但 admission 为了安全会预留整块。并发多时，大量未使用 KV 空间会占住内存。

连续布局适合本册早期项目，因为它能先把正确性和账本讲清。等到并发和长上下文成为主要问题，再引入分页 KV。

## 分页 KV：把上下文拆成 block

分页 KV 把每个 session 的上下文拆成固定大小 block，例如每个 block 容纳 16 或 32 个 token。session 维护一个 block table：

Listing 2.63 分页 KV 元数据

```
KvBlock:
 block_id
 capacity_tokens
 used_tokens
 device_or_host
 ref_count

SessionKvTable:
 logical_block_index -> block_id
```

逻辑 position 到 block：

Listing 2.64 分页 KV 地址计算

```
logical_block = model_position / block_tokens
slot = model_position % block_tokens
block_id = session_table[logical_block]
address = block_base(block_id, layer, kv_head, slot, head_offset)
```

分页的优点是按需增长，便于 prefix block 共享，也便于释放。缺点是 attention 读历史 K/V 时不再是一个完全连续区间，kernel 要读取 block table。GPU 上还要考虑 block table 访问、coalescing 和 shared memory staging。

### prefix cache 和引用计数

多个请求可能共享同一段 prompt，例如系统提示词或 RAG 模板前缀。prefix cache 可以复用这段 K/V。实现上，prefix block 应该只读共享，引用计数记录多少 session 使用它。

Listing 2.65 prefix cache 生命周期

```
build prefix:
 run prefill for shared prompt
 store KV blocks
 mark blocks read-only

new session:
 attach prefix blocks
 increase ref_count
 model_position starts after prefix length

session release:
 decrease ref_count
 free block when ref_count becomes zero
```

prefix cache 最容易错的位置是 RoPE 和 model position。复用 prefix K/V 时，新 token 的 position 不能从 0 开始；attention 也必须把 prefix token 算进历史长度。

### 滑动窗口和上下文截断

有些模型或服务会限制最近窗口，例如只保留最近  $W$  个 token 的 KV。此时物理内存可以回绕，但语义要明确：

- attention 只能读窗口内 token。
- RoPE position 是否继续增长，取决于模型的长上下文策略。
- 被淘汰 token 的 K/V 不能被读到。
- stop sequence 和业务 token buffer 可能仍需保留完整文本。

不要把滑动窗口实现成“数组下标取模”就结束。取模只是物理槽复用；语义上还要维护窗口起点、逻辑 position、可读范围和 RoPE 策略。

### KV cache 量化

KV cache 量化的目标是减少容量和读带宽。fp16 KV 一个 4096 context session 约 512 MiB；int8 可接近减半；int4 可进一步降低。但 KV 是动态 activation，比权重量化更敏感。

Listing 2.66 KV 量化 descriptor

```
KvQuantDesc:
 bit_width
 scale_policy:
```

```

 per_token_head
 per_block_head
 per_channel
group_size
scale_dtype
store_k_quantized
store_v_quantized
dequant_stage:
 before_score
 inside_attention_kernel

```

K 的误差会进入 dot product，再被 softmax 放大；V 的误差会在线性加权中传播。二者敏感度不同。有些实现可能只量化 V，或对 K 使用更高精度。不能简单把权重量化策略照搬到 KV。

### KV 量化的 scale 放哪里

量化不只是压缩数据，还要保存 scale。若按 token/head 存 scale，metadata 随上下文增长；若按 block/head 存 scale，metadata 少，但误差可能增大。

| scale 策略       | 优点              | 风险              |
|----------------|-----------------|-----------------|
| per token/head | 精度较好，适配动态范围     | metadata 多，读流增加 |
| per block/head | metadata 少，分页友好 | block 内动态范围差异大  |
| per channel    | 某些维度更稳定         | 实现和访问更复杂        |

报告不能只写 KV 从 512 MiB 变成 256 MiB，还要写 scale metadata 字节、attention 误差、logits top-k 和 decode latency。若 metadata 访问破坏 coalescing，实际速度可能不升反降。

### KV cache 的 oracle

KV cache 需要专门 oracle，不要只看最终文本。

Listing 2.67 KV oracle 层次

```

append oracle:
 written K/V equals reference at model_position

address oracle:
 logical position maps to expected physical block and slot

read oracle:
 attention reads exactly positions in visible range

quant oracle:
 dequantized K/V within tolerance

score oracle:
 q dot k score drift within profile

```

score oracle 很重要。K 的小误差经过 dot product 和 softmax 后，可能改变 attention 分布。直接比较最终 logits 会太晚。

### KV cache descriptor

为了让编译器、runtime 和后端使用同一套合同，KV cache 应该有 descriptor，而不是把参数散落在构造函数里。

Listing 2.68 KVCacheDesc 字段

```

KVCacheDesc:
 layers
 max_context
 query_heads
 kv_heads
 head_dim
 element_dtype
 layout:
 contiguous
 paged
 sliding_window
 block_tokens_optional
 quant_desc_optional
 rope_position_policy
 device_residency:
 host
 device
 unified

```

`query_heads` 和 `kv_heads` 同时出现，是为了让 verifier 检查 GQA 映射；`layout` 决定地址计算；`block_tokens` 只对分页布局有意义；`quant_desc` 决定 K/V 的存储 dtype 和 scale；`rope_position_policy` 说明 position 是否绝对递增、滑动窗口内重映射或使用模型特定 scaling；`device_residency` 决定后端是否需要 copy。

这份 descriptor 要进入 plan report。后端 kernel 不应该从 session 对象里临时读取一堆字段再猜 layout。若 CPU 和 GPU 使用不同 KV layout，也应该产生两个不同 plan，而不是一个 descriptor 被不同后端解释成不同含义。

### KV append 和 read 的接口

KV cache 至少需要两个热路径接口：append 当前 token 的 K/V，读取历史可见范围。

Listing 2.69 KV append/read 合同

```

append(layer_id, model_position, key, value):
 require model_position < max_context or window policy allows it
 require key.shape == [kv_heads, head_dim]
 require value.shape == [kv_heads, head_dim]
 write into physical storage
 update logical length

read_range(layer_id, query_head, visible_begin, visible_end):
 kv_head = map_query_head(query_head)
 return K/V view for visible logical positions

```

`append` 是写状态，`read_range` 是读状态。它们要分别打 trace。比如一个 decode step 中，某层应该先 append 当前 token 的 K/V，再让当前 query 读 `0..position`。有些实现会先读历史再写当前 token，再单独把 self attention 加进去；这也可以，但 plan 必须明确。不要让调用顺序成为隐式约定。

### 多 session 分配器

业务服务里不止一个 session。KV cache manager 要负责分配、释放和统计。

Listing 2.70 KVCacheManager 职责

```
KVCacheManager:
 reserve(request_id, KVCacheDesc, max_tokens)
 attach_prefix(request_id, prefix_id)
 append(request_id, layer, position, k, v)
 release(request_id)
 stats()
```

**reserve** 要和 admission control 绑定; **release** 要在 completed、cancelled、timeout 和 backend error 路径都执行; **stats** 要输出已预留、已使用、空闲、碎片和共享 prefix block。若只在正常完成路径释放, 长时间服务一定会出现内存只涨不降。

### KV cache 与 memory planner 的边界

activation arena 可以按一层或一个 step 复用; KV cache 不能这样复用, 因为它跨 token 存活。memory planner 必须知道哪些 value 是普通临时 activation, 哪些是 session state。

| 内存对象          | 生命周期             | planner 策略          |
|---------------|------------------|---------------------|
| 临时 activation | 一个 segment 或一层内  | last-use 后复用        |
| workspace     | 一个 step 或一个 plan | 后端独占或按 stream 分配    |
| packed weight | 模型/plan 生命周期     | prepare 后常驻         |
| KV cache      | session 生命周期     | 不参与普通 activation 复用 |
| trace buffer  | 请求生命周期           | 有大小上限               |

如果 planner 把 K/V 当普通 activation, 单步测试可能通过, 第二个 decode token 就会读到被覆盖的数据。这类 bug 很隐蔽, 必须通过 state effect 和 layer oracle 抓住。

### 容量压力下的策略

当内存不足时, 有几种策略:

- 直接拒绝新请求, 最简单也最可控。
- 降低 max context, 但需要业务明确接受。
- 使用 KV 量化, 减少每 session 容量。
- 使用分页 KV, 减少未使用预留。
- 使用 prefix 共享, 减少重复 prompt。
- 把部分 session 排队, 而不是立即分配。

这些策略都要体现在 admission decision 中。不要在 runtime 中途偷偷把 KV 从 fp16 改成 int8, 也不要静默截断上下文。容量策略改变的是语义或质量边界, 必须对业务可见。

### 常见故障模式

| 故障                 | 表现                     | 定位证据                |
|--------------------|------------------------|---------------------|
| position 重置        | prefix 后输出漂移           | RoPE checkpoint     |
| GQA head 映射错       | 某些 head 输出异常           | head-level oracle   |
| 物理 slot 覆盖         | 长文本中途突然漂移              | address oracle      |
| context off-by-one | 第一个新 token 或最后 token 错 | visible range trace |
| scale metadata 错位  | 量化 KV 长上下文变差           | quant oracle        |
| 取消未释放              | 并发后内存只涨不降              | resource counter    |

这些故障都可能在 shape 正确、程序不崩溃的情况下发生。因此 KV cache 必须有自己的 trace 和 oracle。

### KV trace 示例

一条 KV trace 不需要记录完整张量, 但要记录坐标和容量。

Listing 2.71 KV trace event

```

KvTraceEvent:
 request_id
 layer_id
 operation: reserve | append | read | release
 model_position
 visible_begin
 visible_end
 kv_head_count
 head_dim
 layout
 physical_block_optional
 bytes_reserved
 bytes_used
 quant_profile_optional

```

当长上下文性能变差时，可以按 `visible_end-visible_begin` 分桶；当内存泄漏时，可以看 `reserve` 和 `release` 是否配对；当 prefix 复用错位时，可以看 `model_position` 是否从 prefix 长度后继续。

### 从 4 个 token 手算 KV cache

把 KV cache 讲清楚，不能只说“避免重复计算历史 token”。先固定一个小模型：

Listing 2.72 KV cache 手算模型

```

layers = 2
query_heads = 2
kv_heads = 1
head_dim = 2
sequence_positions = 0, 1, 2, 3
dtype = fp16

```

每一层、每个 token 都会生成当前 token 的 **K** 和 **V**。因为 `kv_heads=1`，两个 query head 共享同一个 KV head。这就是 GQA 的最小形状。第 0 个 token decode 时，cache 为空，当前层计算出 **K0, V0**，写入 position 0，然后 attention 只能读 position 0。第 1 个 token 计算出 **K1, V1**，写入 position 1，然后读 position 0 和 1。第 2 个 token 读 0、1、2。第 3 个 token 读 0、1、2、3。

| step | 写入         | 本 step 可读范围 | 读 token 数 |
|------|------------|-------------|-----------|
| 0    | K[0], V[0] | 0..0        | 1         |
| 1    | K[1], V[1] | 0..1        | 2         |
| 2    | K[2], V[2] | 0..2        | 3         |
| 3    | K[3], V[3] | 0..3        | 4         |

这张表解释了 decode latency 为什么随 context 增长。每个新 token 只写一个位置，但它在要读越来越长的历史。prefill 和 decode 的差异也在这里：prefill 一次处理一段 prompt，可以把多个 token 的 Q、K、V 当作矩阵批量计算；decode 每次只有当前 token 的 Q，但 attention score 要和历史所有 K 做点积。

### GQA 的 head 映射公式

当 `query_heads > kv_heads` 时，不是每个 query head 都有独立 K/V。常用映射是：

$$kv\_head = \left\lfloor \frac{query\_head \times kv\_heads}{query\_heads} \right\rfloor$$



对上面的最小模型，`query_heads=2`，`kv_heads=1`，所以：

| query head | kv head | 含义                            |
|------------|---------|-------------------------------|
| 0          | 0       | 第 0 个 query head 读唯一 KV head  |
| 1          | 0       | 第 1 个 query head 也读唯一 KV head |

更真实的 7B 形状常见 `query_heads=32`、`kv_heads=8`，则每 4 个 query head 共享 1 个 KV head：

$$kv\_head = query\_head / 4$$

这不是性能细节，而是语义合同。若映射写错，shape 仍然能对上，程序也可能不崩溃，但某些 head 会读到错误历史，表现为 logits 漂移、长上下文回答变差、特定 prompt 复现不稳定。KV oracle 必须能按 `query_head` 打印映射结果。

### 连续布局的地址公式

假设连续 KV cache 使用 layout：

**Listing 2.73** 连续 KV cache layout

```
K[layer, position, kv_head, head_dim]
V[layer, position, kv_head, head_dim]
```

令 `L` 是层数，`S` 是最大 context，`Hkv` 是 KV head 数，`D` 是 head dim。若元素是 fp16，单元素 2 字节。K 的线性 offset 是：

$$offset_K = (((layer \times S + position) \times Hkv + kv\_head) \times D + dim)$$

地址是：

$$addr_K = base_K + offset_K \times sizeof(element)$$

对手算模型，`S=4`、`Hkv=1`、`D=2`。第 1 层、position 3、kv head 0、dim 1 的 K offset 为：

$$(((1 \times 4 + 3) \times 1 + 0) \times 2 + 1) = 15$$

若 fp16，则地址距离 `base_K` 为 30 字节。这个小数值例子很重要：它逼着读者区分 layer、position、head 和 dim。很多 KV bug 不是公式很难，而是把 `position` 和物理 slot、把 `query_head` 和 `kv_head` 混在一起。

### layout A/B 为什么会影响 decode

连续布局至少有两种常见方向：

**Listing 2.74** 两种 KV layout

```
layout A:
 K[layer, position, kv_head, dim]

layout B:
 K[layer, kv_head, position, dim]
```

对单个 query head 做 decode attention 时，通常固定 `layer` 和 `kv_head`，沿 `position` 读取所有历史 K，再沿 `dim` 做 dot product。layout A 中，相邻 position 之间距离是 `Hkv*D` 个元素；layout B 中，固定 head 后 position 更连续，距离是 `D` 个元素。若 `Hkv` 较大，layout A 的历史扫描会跨更大步长；若 append 当前 token 要同时写所有 KV heads，layout A 的写入更接近一段连续块。

| layout                                                                       | 优点                                           | 风险                                                                     |
|------------------------------------------------------------------------------|----------------------------------------------|------------------------------------------------------------------------|
| <code>[layer, pos, head, dim]</code><br><code>[layer, head, pos, dim]</code> | 当前 token append 多 head 较自然<br>单 head 历史扫描更连续 | 单 head 扫历史时 position 跨步为 $H_{kv} \times D$<br>append 所有 head 可能分散到多个区域 |

所以 layout 选择不能脱离 kernel。CPU decode attention 若按 head 扫历史，可能偏好 head-major；GPU kernel 若按 block 组织 token 和 head，可能选择分页或分块布局，让一组线程读取连续位置。成熟系统的关键不是“某个 layout 永远最好”，而是 descriptor、kernel 和 oracle 必须使用同一地址解释。

### 每 token KV 字节账本

7B 形状取 `layers=32`、`kv_heads=8`、`head_dim=128`、fp16。每个 token 每层写 K 和 V：

$$bytes\_per\_layer\_token = 2 \times kv\_heads \times head\_dim \times 2$$

代入：

$$2 \times 8 \times 128 \times 2 = 4096 \text{ bytes} = 4 \text{ KiB}$$

所有 32 层：

$$4096 \times 32 = 131072 \text{ bytes} = 128 \text{ KiB/token}$$

4096 context：

$$128 \text{ KiB} \times 4096 = 512 \text{ MiB}$$

这只是一个 session 的 KV，不含权重、workspace、logits、allocator 碎片和 metadata。若 10 个并发长上下文 session 同时存在，KV 容量就是数 GB 级。这个账本解释了为什么 KV cache 是 serving 的 admission control 问题，而不是 attention kernel 的一个局部数组。

### decode 读写字节和 latency 下界

每生成一个新 token，写入当前 token 的 K/V 是固定的 **128KiB/token**。但 attention 读取历史 KV 与 context 长度成正比。对 position 为 `t` 的 token，粗略读取量是：

$$read\_bytes(t) \approx layers \times 2 \times kv\_heads \times head\_dim \times sizeof(dtype) \times (t + 1)$$

在上面的 7B 形状下，每个历史 token 对所有层贡献约 **128KiB** 的可读 K/V。context 为 4096 时，单步 attention 可能面对约 **512MiB** 的 KV 读流。实际 kernel 会分层执行、分块读取、复用 cache 或 shared memory，并且不一定每一步都把全部字节以最坏方式经过 DRAM；但数量级告诉我们：长上下文 decode 很容易被内存带宽和访存组织限制。

若设备有效 KV 读取带宽只有 **100GB/s**，单步读 **512MiB** 的理论下界约为：

$$0.5 \text{ GB} / 100 \text{ GB/s} = 5 \text{ ms}$$

这个下界还没有算 QKV projection、MLP、lm head、softmax、采样和调度。若报告声称 4096 context 的单 session decode 达到极高 tokens/s，就必须解释 KV 读流如何被减少、压缩、分页复用、合批或落在更高有效带宽上。

### C++ descriptor 和 append/read 验证代码

KV cache 的 C++ 表示应把地址合同放在一个地方，而不是散落在 kernel 调用点。一个教学版 descriptor 可以写成：

Listing 2.75 连续 KV cache descriptor 的教学形态

```

1 enum class KvLayout {
2 LayerPositionHeadDim,
3 LayerHeadPositionDim,
4 };
5
6 struct KvCacheDesc {
7 std::int32_t layers = 0;
8 std::int32_t max_context = 0;
9 std::int32_t query_heads = 0;
10 std::int32_t kv_heads = 0;
11 std::int32_t head_dim = 0;
12 std::int32_t element_bytes = 2;
13 KvLayout layout = KvLayout::LayerPositionHeadDim;
14 };
15
16 std::int64_t map_query_head(const KvCacheDesc& desc, std::int32_t query_head)
17 {
18 return static_cast<std::int64_t>(query_head) * desc.kv_heads / desc.↵
 ↵ query_heads;
19 }
20
21 std::int64_t k_offset_lpdh(
22 const KvCacheDesc& desc,
23 std::int32_t layer,
24 std::int32_t position,
25 std::int32_t kv_head,
26 std::int32_t dim)
27 {
28 return (((static_cast<std::int64_t>(layer) * desc.max_context + position)
29 * desc.kv_heads + kv_head)
30 * desc.head_dim + dim);
31 }

```

这段代码不是最终高性能实现，但它能成为 address oracle。任何分页、量化、GPU layout 都应该有类似的地址解释函数，用于在 debug 模式下检查 **append** 是否写到预期位置，**read\_range** 是否读了正确历史。

### 错误案例：position、head 和 slot 三种混淆

KV cache 最常见的错误可以用同一个 4 token 例子复现。

Listing 2.76 KV cache 三类典型错误

```

position bug:
 prefix length = 2
 next decoded token should use model_position = 2
 wrong implementation restarts at position = 0

GQA bug:
 query_head = 5, query_heads = 32, kv_heads = 8
 expected kv_head = 1
 wrong implementation uses kv_head = 5

```

```
slot bug:
 paged cache physical slot wraps to 0
 logical position is still 4096
 wrong implementation feeds slot id into RoPE position
```

第一类错误会让 prefix 后输出突然漂移。第二类错误通常只污染部分 head，最终 logits 变化不一定每次都巨大，但长文本会越来越差。第三类错误最隐蔽：物理 slot 可以复用，模型 position 不能随便回绕。RoPE 使用的是语义位置，不是存储槽编号。

### 工业系统如何处理这类问题

成熟推理系统通常把 KV cache 当成 session state，而不是普通 activation。它们会把 KV layout、block table、prefix 共享、context limit、量化策略和设备位置写进 runtime plan 或 session metadata。分页 KV 的核心思想是把长上下文切成固定 token block，session 保存逻辑 position 到物理 block 的表；这样可以减少大块连续预留，支持 prefix 共享和更灵活的回收。代价是 attention kernel 必须读 block table，地址计算更复杂，oracle 也必须能解释 logical-to-physical 映射。

本书的工程标准是：即使暂时不实现完整 paged attention，也要把 descriptor 和 trace 设计到能表达它。否则连续 KV 写死在接口里，后面再接 batching、prefix cache、KV 量化或 GPU backend，会出现大面积重构。

### 硬验收

读完本节，应该能设计一个可靠 KV cache：

- 区分 layer、model position、KV head 和物理 slot。
- 能实现连续 KV，并知道它在并发下的浪费。
- 能解释分页 KV 的 block table、地址计算和 prefix 共享。
- 能处理 prefix cache、滑动窗口、取消释放和 context limit。
- 能设计 KV 量化 descriptor，并估算 metadata 成本。
- 能用 append、address、read、quant、score oracle 定位 KV 错误。

## 2.6 业务服务实战：请求、调度、队列、取消与资源预算

### 业务开发不是把 generate() 暴露成接口

很多本地大模型 demo 的接口只有一个输入 prompt 和一个输出 text。它能演示模型效果，却不能承载真实业务。业务开发要面对的问题更具体：多个用户同时请求怎么办，长 prompt 是否允许，用户断开后是否继续烧算力，队列满了怎样拒绝，CPU/GPU 后端不可用怎样降级，怎么知道慢在排队、prefill、decode 还是 sampler。

#### 核心思想

推理服务的核心对象不是一个字符串函数，而是请求状态机、资源预留、调度策略、session 生命周期、流式事件和可追溯 trace。没有这些对象，模型能输出文本也不能算可交付业务服务。

本节从服务视角设计一个最小但完整的本地推理服务：它接收请求，验证参数，估算资源，决定是否接纳，运行 prefill 和 decode，流式返回 token，处理取消和超时，最后输出 usage 与 trace。

### 请求字段要表达资源和语义

一个业务接口必须把会影响资源、质量和延迟的字段显式暴露出来。

Listing 2.77 GenerateRequest 的业务字段

```
GenerateRequest:
 request_id
 user_id_optional
```

```

model_id
prompt
max_new_tokens
max_context_tokens
temperature
top_k
top_p
repetition_penalty
stop_token_ids
stream
timeout_ms
backend_policy
quant_profile_id
trace_level

```

`max_context_tokens` 不是普通参数，它决定 KV cache 上限；`max_new_tokens` 决定最坏 decode 步数；`timeout_ms` 决定调度器能否中止；`backend_policy` 决定 CPU/GPU plan 选择；`quant_profile_id` 决定质量和容量；`trace_level` 决定是否允许记录更多中间证据。

请求验证要分两层。第一层是 API 参数验证，例如 `top_p` 是否在合法范围、`max_new_tokens` 是否为正。第二层是模型相关验证，例如请求的 context 是否超过模型最大上下文，选择的量化 profile 是否存在，后端是否支持该 profile。不要把这两类错误都混成 `badrequest`。

### 响应也要表达停止原因

响应不能只返回文本。最小响应应该能告诉业务层为什么结束。

**Listing 2.78** GenerateResponse 的业务字段

```

TokenEvent:
 request_id
 index
 token_id
 text_fragment
 model_position
 trace_event_id_optional

FinalEvent:
 request_id
 finish_reason:
 stop_token | max_tokens | context_limit |
 timeout | cancelled | backend_error | internal_error
 prompt_tokens
 completion_tokens
 kv_cache_bytes
 prefill_ms
 decode_ms
 trace_summary_optional

```

业务逻辑会依赖 `finish_reason`。若是 `max_tokens`，可以提示用户继续生成；若是 `context_limit`，应该让用户缩短输入；若是 `timeout`，可能需要重试或调小输出长度；若是 `backend_error`，要看 fallback 和后端日志。没有停止原因，服务层只能猜。

## Admission control: 先预算，再接纳

服务端最重要的保护机制是 admission control。它在请求进入模型前决定是否接纳。最小预算包括 KV cache、activation arena、workspace、trace buffer 和队列 slot。

Listing 2.79 AdmissionRequest 字段

```
AdmissionRequest:
 model_id
 prompt_tokens
 max_context_tokens
 max_new_tokens
 backend_policy
 quant_profile_id
 trace_level
```

决策输出要结构化：

Listing 2.80 AdmissionDecision 字段

```
AdmissionDecision:
 accepted
 rejection_reason_optional:
 context_too_long
 memory_budget_exceeded
 queue_full
 backend_unavailable
 quant_profile_unsupported
 selected_plan_id
 reserved_kv_bytes
 reserved_workspace_bytes
 estimated_ttft_ms
 estimated_decode_tokens_per_s
```

一个常见错误是按当前 prompt 长度分配 KV cache，而不是按请求允许的最大上下文预算。若 prompt 是 500 token，`max_new_tokens` 是 3500，最终可能增长到 4000 token。admission 若只看 500，会在 decode 中途撞内存，业务体验比直接拒绝更差。

## 队列：排队时间也是用户延迟

推理服务中的队列至少有两类：等待进入模型的请求队列，以及流式输出事件队列。二者都必须有限。

| 队列              | 风险             | 控制方式              |
|-----------------|----------------|-------------------|
| 请求队列            | 排队时间吞掉 SLO     | 上限、优先级、超时、拒绝      |
| decode ready 队列 | 某些 session 饿死  | 公平调度、step budget  |
| 流式事件队列          | 客户端慢导致内存膨胀     | backpressure、断开检测 |
| trace 队列        | debug 模式产生日志洪峰 | 采样、等级、大小上限        |

排队时间要进入 trace。否则用户看到 TTFT 很慢时，工程师可能错误优化 prefill kernel，而真正的问题是请求在队列里等了 800ms。

## 队列模型：SLO 不是平均值

服务化推理需要一个最小 queueing model。即使不做复杂排队论，也必须知道三个量：到达率  $\lambda$ 、服务率  $\mu$ 、利用率  $\rho = \lambda / \mu$ 。当  $\rho$  接近 1 时，平均服务时间只增加



一点，排队时间可能急剧上升。LLM serving 更复杂，因为 prefill 和 decode 服务时间差异大，长输出请求会占用很多 step；但基本判断不变：系统不能长期以接近满载运行还期待稳定 p95。

$$TTFT = queue\_wait + admission + prefill + first\_decode + first\_emit$$

$$TPOT \approx decode\_step\_wait + decode\_kernel + sampler + emit$$

这两个公式把“慢”拆开。TTFT 慢可能是排队，也可能是长 prompt prefill；TPOT 慢可能是 decode kernel，也可能是为了合批等待、sampler 回传或客户端 backpressure。若只报端到端平均延迟，工程师无法知道该优化 kernel、调度、队列上限还是 I/O。

| 指标                           | 主要含义              | 常见错误解释               |
|------------------------------|-------------------|----------------------|
| <code>queue_wait_p95</code>  | admission 后等待模型资源 | 误判为 prefill 慢        |
| <code>ttft_p95</code>        | 首 token 用户体验      | 忽略 prompt 长度分桶       |
| <code>tpot_p95</code>        | 持续生成速度            | 忽略合批等待和 backpressure |
| <code>active_sessions</code> | decode 竞争程度       | 不等于 batch size 一定大   |
| <code>rejection_rate</code>  | 容量保护触发频率          | 不等于服务失败，可能是正确保护      |

**批处理的收益和代价** 合批提高吞吐，因为多个 session 可以共享权重读流和 kernel launch。但合批需要等待更多请求凑齐，这会增加 queue wait 或 decode step wait。保守调度器应有 `max_batch_size` 和 `max_wait_ns` 两个边界。吞吐报告必须同时给 tokens/s 和 p95/p99，否则很容易用长尾换吞吐。

Listing 2.81 合批调度的最小数字字段

```
BatchingPolicy:
 max_batch_size
 max_wait_ns
 prefill_token_budget
 decode_session_budget
 deadline_ordering

BatchingMetrics:
 formed_batch_size
 wait_ns_before_launch
 tokens_per_second
 ttft_p95
 tpot_p95
```

**SLO 需要分桶** prompt=32 和 prompt=4096 的 TTFT 不应放在同一个平均值里；context=128 和 context=8192 的 decode attention 也不应混在一起。最小分桶维度是 prompt tokens、current context、max new tokens、backend、quant profile 和 batch size。分桶后才能判断某个优化改善了谁、伤害了谁。

### 调度粒度：一次跑完整请求还是一 token 一调度

最简单的调度方式是一个请求跑到结束再处理下一个请求。它容易实现，但对长输出请求不公平：一个用户生成 2048 token，会阻塞其他短请求。更合理的最小策略是 step-level 调度：每个活跃 session 每次执行一个或几个 decode step，然后让出。

Listing 2.82 step-level decode 调度

```
while active_sessions not empty:
```

```

session = pick_next_session()
if session.need_prefill:
 run_prefill(session)
else:
 run_decode_step(session)
 emit_token_event(session)

if session.finished:
 release_resources(session)
else:
 requeue(session)

```

这不是完整动态 batching，但已经把“一个请求独占模型”的问题拆开。后续可以在 `pick_next_session` 和 `run_decode_step` 之间加入小批量合并：把多个 session 的当前 token 合成 batch，执行同一层 decode，再拆回各自 session。合批会提高吞吐，也会引入等待和复杂状态，因此必须由 trace 报告收益和 p95 代价。

### prefill 调度和 decode 调度不同

prefill 通常更重，且 prompt 长度差异大。一个长 prompt prefill 可能占用较长时间，影响其他请求 TTFT。decode 每步较短，但会重复很多次。调度器至少要区分：

- 新请求的 prefill 是否允许插队。
- 长 prompt 是否拆块 prefill。
- decode session 是否按公平轮转。
- 是否允许为了 batch 吞吐牺牲单请求 p95。
- CPU 和 GPU 后端是否使用不同队列。

最保守的实现可以先不做复杂合批，但要把调度决策记录下来：

Listing 2.83 SchedulerEvent 字段

```

SchedulerEvent:
 request_id
 state
 queue_wait_ns
 selected_backend
 batch_size
 prompt_tokens
 context_length
 reason

```

这样后续优化调度时，能判断吞吐提升是否以 p95 变差为代价。

### 取消和超时：资源释放是合同

用户关闭页面、网络断开、业务层超时，都应该触发取消。取消不是设置一个标志就完事。runtime 必须让正在运行的 session 到达安全点，然后释放资源。

Listing 2.84 取消处理的资源清单

```

on_cancel(request_id):
 mark session cancel_requested
 stop scheduling new decode steps
 drain or discard stream events
 release KV cache reservation

```

```

release activation arena
release workspace
write final event with cancelled
write trace summary

```

如果正在 GPU kernel 中，不能随意销毁底层 buffer。可以把取消点放在 step 边界：当前 kernel 完成后不再调度下一步。CPU 后端也类似，线程池任务要检查取消标志，但不能在持有共享结构时直接抛出异常导致资源计数不平衡。

### 常见误区

取消路径必须有测试。只测正常完成，会漏掉最真实的服务问题：客户端断开后继续生成、KV cache 不释放、事件队列持续增长、trace 文件缺 final event。

### 流式输出与 backpressure

流式 token 事件通常比模型计算慢得多，也可能被网络、客户端渲染或上游代理阻塞。服务端不能让事件队列无限增长。最小策略：

- 每个 session 设置事件队列上限。
- 队列满时暂停该 session 的 decode 调度。
- 超过最大阻塞时间后取消请求或返回 backpressure 错误。
- 客户端断开时立即进入取消路径。

backpressure 要进入调度器，而不是只在 I/O 层处理。否则模型会继续生成 token，I/O 层只是堆积事件，内存仍然失控。

### 错误分类：给业务可处理的错误

错误要分层返回。下面是一套最小分类：

| 错误类别  | 例子                         | 业务处理            |
|-------|----------------------------|-----------------|
| 请求错误  | 参数范围错、prompt 为空            | 直接提示用户修改        |
| 容量错误  | context 超限、队列满、内存不足        | 稍后重试或缩短输入       |
| 模型错误  | manifest 不合法、tokenizer 不匹配 | 运维修模型资产         |
| 后端错误  | GPU 不可用、dtype 不支持          | fallback 或切 CPU |
| 运行时错误 | KV 越界、arena 冲突             | 记录 trace，阻断发布   |

不要把所有错误都变成 500。业务系统要根据错误类型决定重试、降级、提示、告警还是阻断部署。

### 服务级 SLO 报告

服务的 SLO 不只是一条平均延迟。最小报告：

Listing 2.85 ServiceSloReport 字段

```

ServiceSloReport:
 model_id
 backend_plan_id
 quant_profile_id
 prompt_bucket
 context_bucket
 request_count
 rejection_rate
 queue_wait_p50_p95
 ttft_p50_p95
 decode_step_p50_p95

```

```
tokens_per_second
cancel_count
timeout_count
memory_peak_bytes
fallback_count
```

这份报告能指导业务选择。例如 RAG 问答更关注 TTFT 和长 prompt prefill；代码补全更关注短 prompt 低延迟 decode；批量摘要更关注吞吐；离线任务可以接受更高延迟换取更大 batch。不同业务可以使用同一引擎，但 SLO 和调度策略不同。

### 真实 workload trace：不要只测单请求

单请求 demo 无法证明服务可用。最小 workload trace 应该同时包含短 prompt、长 prompt、取消、超限、慢客户端和后端 fallback。这样才能暴露队列、KV 预算、backpressure 和错误分类。

Listing 2.86 MiniLLM-CPU serving workload fixture

```
workload minillm_serving_smoke:
 req_short:
 prompt_tokens=32
 max_new_tokens=4
 stream=true

 req_rag:
 prompt_tokens=512
 max_new_tokens=8
 stream=true

 req_cancel:
 prompt_tokens=128
 max_new_tokens=128
 cancel_after_decode_steps=0

 req_too_long:
 prompt_tokens=4096
 max_new_tokens=512
 expect_reject=memory_budget_exceeded

 req_slow_client:
 prompt_tokens=64
 max_new_tokens=16
 event_consume_delay_ms=200
 expect_backpressure=true
```

这组 workload 很小，但覆盖真实服务最容易出问题的路径。它不是为了模拟所有线上流量，而是为了保证服务对象存在：admission、queue、session、KV reservation、stream event、cancel、backpressure、final event 和 trace summary。

### 线上故障演练：每个故障都要有字段证明

服务化章节必须训练读者处理故障，而不是只讲正常路径。下面四个演练应该进入回归：

| 故障      | 注入方式                                       | 必须证明的字段                                                      |
|---------|--------------------------------------------|--------------------------------------------------------------|
| 上下文超限   | <code>prompt+max_new&gt;max_context</code> | <code>rejection_reason=context_too_long</code> , 没有分配 KV     |
| KV 预算不足 | 降低 serving KV budget                       | <code>memory_budget_exceeded</code> , reserved bytes 不增加     |
| 客户端取消   | decode 前或 decode 中取消                       | <code>finish_reason=cancelled</code> , KV/arena/workspace 归还 |
| 慢客户端    | 限制 event queue 消费速度                        | backpressure 计数增加, 调度器暂停或取消 session                          |
| 后端降级    | 禁用目标 ISA/GPU kernel                        | fallback event 可见, plan/report 标出原因                          |

故障演练的重点是“字段能不能证明”。如果取消后只看到函数返回, 不能证明资源释放; 如果 fallback 没有事件, 不能证明性能数字对应哪个后端; 如果慢客户端只堆积队列, 不能证明 backpressure 真的进入调度器。

### 连续合批的保守边界

生产 LLM serving 常用 continuous batching, 把多个 session 的当前 decode step 合成小 batch。第三册可以先实现 step-level 调度和 trace, 不必立即做生产级 PagedAttention; 但要把连续合批的接口留对。

Listing 2.87 连续合批需要的最小调度对象

```
DecodeReadySession:
 request_id
 model_position
 kv_cache_binding
 input_token_id
 sampler_state
 deadline_ns
 priority

DecodeBatchPlan:
 batch_id
 sessions[]
 max_wait_ns
 backend_plan_id
 kv_bindings[]
 expected_shape: [B,H]
```

合批必须保护两个边界。第一, batch 只共享权重和 kernel, 不共享 session 状态; 每个 session 的 KV binding、position、stop 条件和 sampler state 都独立。第二, 合批等待时间要进入 SLO。吞吐变高但 `queue_wait_p95` 和 `decode_step_p95` 失控, 不是服务优化。

### 把服务化 SLO 绑定到可运行 probe

本节不能只停在接口字段。当前 lab 新增了 `lcqi_serving_probe`, 用固定请求集模拟 admission、batch 合并、排队、prefill、decode、取消和拒绝, 并导出 CSV:

Listing 2.88 serving SLO probe 复现命令

```
books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_serving_probe↵
↵ \
> books/cpu-volume-3/results/lcqi-serving-slo-probe-2026-06-24.csv
```

CSV 字段如下:

Listing 2.89 serving probe CSV 字段

```
row_type,request_id,accepted,rejection_reason,prompt_tokens,reserved_tokens,↵
↵ kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,decode_step_p95_ms,↵
↵ tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,metric_name,↵
↵ metric_value
```

真实输出的请求行:

Listing 2.90 serving probe request 摘要

```
request,req_short,1,none,32,36,4.500,0.000,2.560,16.560,14.204,14.204,4,↵
↵ max_tokens,0.000,,
request,req_rag,1,none,512,520,65.000,8.000,40.960,62.960,14.322,14.322,8,↵
↵ max_tokens,0.000,,
request,req_cancel,1,none,128,144,18.000,16.000,10.240,40.240,14.230,14.230,0,↵
↵ cancelled,2.000,,
request,req_too_long,0,memory_budget_exceeded↵
↵ ,4096,4608,576.000,0.000,0.000,0.000,0.000,0.000,0,rejected,0.000,,
```

这四行展示了生产闭环的三个关键合同。第一，admission 使用 `prompt_tokens+max_new_tokens` 预留 KV，而不是只看当前 prompt。第二，取消请求有 `finish_reason=cancelled` 和 `cancel_reclaim_ms=2.000`，说明取消路径必须释放资源并留下 final event。第三，超长请求在进入模型前被拒绝，原因是 `memory_budget_exceeded`，不是运行到一半 OOM。若要和连续合批对上，probe 还应记录 `batch_id`、`batch_size` 和 `state_path`；这三项说明请求是如何从 queued 进入 admitted、prefill、decode、cancelled 或 rejected 的。batch 只是调度合同，不是业务事实本身，业务事实仍然是 request id、KV 预算和最终 finish reason。

probe 还输出聚合 SLO:

Listing 2.91 serving probe summary 摘要

```
summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,accepted_count,3.000
summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,rejected_count,1.000
summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,rejection_rate,0.250
summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,queue_wait_p95_ms,16.000
summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,ttft_p95_ms,62.960
```

这不是生产调度器，只是最小 probe。但它已经把 DDIA/MLSys 风格的问题压到了 artifact：请求分布、队列等待、TTFT、TPOT、KV 预算、取消回收和拒绝率必须被记录。后续真实 serving runtime 可以替换 probe 的模拟耗时，但 CSV 合同不应随意改变。

### 排队模型：SLO 先被利用率击穿

服务化推理的第一个定量模型不是 Transformer，而是排队。设请求到达率为  $\lambda$ ，服务率为  $\mu$ ，单服务台利用率为：

$$\rho = \frac{\lambda}{\mu}$$



当  $\rho$  接近 1, 平均等待时间会非线性上升; 即使单次推理 kernel 没变慢, 队列等待也会把 TTFT 和 p95/p99 打爆。多 worker、多 GPU、多 batch 会改变有效  $\mu$ , 但不会取消这个规律。

Listing 2.92 一个最小 SLO 账本

```
arrival_rate_rps = 20
mean_service_time_ms = 40
service_rate_rps = 25
utilization = 20 / 25 = 0.80

if mean_service_time_ms grows to 48:
 service_rate_rps = 20.8
 utilization = 0.96
 queue wait becomes unstable
```

这说明服务报告不能只报模型 tokens/s。tokens/s 可能很好, 但请求到达率、prompt 分布、max new tokens、取消率、慢客户端和 fallback 会把有效服务率拉低。Admission control 的任务是在进入不稳定区之前拒绝、排队、降级或缩短请求。

### Little's Law: 队列长度不是孤立指标

Little's Law 给出一个简单约束:

$$L = \lambda W$$

$L$  是系统中平均请求数,  $\lambda$  是吞吐,  $W$  是平均停留时间。若平均系统内请求数持续增长, 而吞吐没有增长, 平均停留时间必然增长。服务监控里队列长度、in-flight、TTFT 和吞吐必须一起看。

| 现象              | 可能解释                  | 应检查                        |
|-----------------|-----------------------|----------------------------|
| 队列增长, 吞吐不变      | 到达率超过服务率              | admission、请求桶、fallback     |
| 队列稳定, TTFT 高    | prefill 或 batch 等待过长  | prompt length、batch wait   |
| TPOT 高, TTFT 正常 | decode step 慢或 KV 读流重 | context、KV layout、back-end |
| 吞吐高, p99 高      | 长请求或慢客户端拖尾            | 分桶、取消、backpressure         |

这张表把线上症状连回引擎对象: queue、batch、session、KV cache、sampler、stream event。没有这层模型, 服务化章节会变成字段清单。

### TTFT 和 TPOT 的分解公式

TTFT 是用户看到第一个 token 的时间, 通常包含排队、admission、tokenizer、prefill、第一次 decode、sampler 和 streaming flush:

$$TTFT = T_{queue} + T_{admit} + T_{tokenize} + T_{prefill} + T_{decode0} + T_{sample0} + T_{flush0}$$

TPOT 是后续每个 token 的平均或分位时间:

$$TPOT \approx T_{schedule} + T_{decode\_step} + T_{sample} + T_{stream}$$

优化必须针对分量。如果 TTFT 主要来自 prefill, 优化 KV decode 没用; 如果 TPOT 主要来自 stream backpressure, 优化 GEMV 也不会改善用户体验; 如果 queue wait 占一半, 模型 kernel 再快也无法满足 SLO。

Listing 2.93 SLO trace 必须拆分的阶段

```
queue_wait_ms
admission_ms
```

```

tokenize_ms
prefill_ms
decode_step_ms
sampler_ms
stream_flush_ms
kv_reserve_ms
fallback_ms

```

### continuous batching 的数学取舍

连续合批提高吞吐，是因为多个 session 共享一次权重读取和更大的矩阵形态；但它引入等待。设 batch 等待窗口为  $w$ ，合批后 decode step 从单请求  $t_1$  降到 batch 平均  $t_b$ 。对某个 session，若等待成本大于节省成本，SLO 反而变差。

$$\Delta = w + t_b - t_1$$

若  $\Delta < 0$ ，该请求受益；若  $\Delta > 0$ ，该请求变慢。系统可能仍因吞吐提高而整体受益，但低延迟业务不一定接受。调度器必须按业务桶设置不同 `max_wait_ns`：代码补全短等待，离线摘要长等待，RAG 长 prompt 可能优先 prefill throughput。

| 业务桶    | 合批策略                   | 风险            |
|--------|------------------------|---------------|
| 代码补全   | 小窗口，优先 TTFT/TPOT       | 吞吐低，GPU 利用率不足 |
| RAG 问答 | prefill 分桶，decode 适中窗口 | 长 prompt 占用资源 |
| 批量摘要   | 大窗口，优先吞吐               | 单请求延迟高        |
| 交互聊天   | 限制上下文和队列等待             | 长上下文用户被拒绝或降级  |

### admission control 的资源不等式

请求进入模型前，至少要满足 KV、workspace、队列和截止时间约束。以 KV 为例：

$$KV_{needed} = layers \times reserved\_tokens \times kv\_heads \times head\_dim \times 2 \times bytes$$

若当前已预留  $KV_{used}$ ，总预算  $KV_{budget}$ ，则：

$$KV_{used} + KV_{needed} \leq KV_{budget}$$

这条不等式要在 prefill 前检查，而不是 decode 到一半才 OOM。类似地，workspace、event queue、stream buffer 和 backend slot 都要有预算。拒绝请求不是坏事；没有字段证明的中途失败才是坏事。

Listing 2.94 admission 失败应给出可行动原因

```

reject:
 context_too_long
 kv_budget_exceeded
 queue_deadline_exceeded
 backend_unavailable
 unsupported_quant_profile
 request_cancelled_before_admit

```

### 线上故障复盘字段

生产级 serving 不是没有故障，而是故障可解释。一次 p99 抖动复盘至少要能回答：请求属于哪个桶，排队多久，进入哪个 batch，prefill/decode 各多久，KV 是否触顶，是否 fallback，是否遇到慢客户端，取消是否释放资源，后端当时的 selected plan 是什么。

Listing 2.95 ServingIncidentTrace 最小字段

```

request_id
prompt_bucket
max_new_tokens
arrival_time
admit_time
batch_id_sequence
selected_backend_plan
kv_reserved_bytes
queue_wait_ms
prefill_ms
decode_step_p95_ms
stream_backpressure_ms
fallback_reason
finish_reason
resource_released

```

这份 trace 把第三册的 AI Infra 和生产系统连接起来。没有它，模型能跑只是 demo；有它，才开始接近线上工程。

## 2.7 状态机：请求从到达释放资源

推理服务必须把请求生命周期写成状态机。否则 admission、取消、超时、stream backpressure 和 KV 释放都会散落在 if 分支里，线上事故很难复盘。

| 状态             | 持有资源                              | 合法转移                             |
|----------------|-----------------------------------|----------------------------------|
| Arrived        | request metadata                  | AdmitQueued 或 Rejected           |
| AdmitQueued    | 队列位置、deadline                     | Admitted 或 Rejected              |
| Admitted       | KV 预算、workspace 预算、session id     | PrefillRunning 或 Cancelled       |
| PrefillRunning | prefill batch slot、KV write range | DecodeReady 或 Failed             |
| DecodeReady    | session KV、sampler state          | DecodeRunning、Finished、Cancelled |
| DecodeRunning  | decode batch slot、backend plan    | DecodeReady 或 Streaming          |
| Streaming      | output buffer、client flow-control | DecodeReady、Finished、Client-Gone |
| Released       | 无运行时资源                            | 终态                               |

这张表要求每个状态都回答“谁释放资源”。若客户端断开时 session 仍停在 **DecodeReady**，KV cache 会泄漏；若 **Streaming** 被慢客户端堵住，decode scheduler 可能继续生成 token，输出队列膨胀；若 **PrefillRunning** 失败没有回滚 KV range，后续 session 可能读到脏状态。

### 排队模型：SLO 不是平均 tokens/s

设请求到达率为  $\lambda$ ，平均服务时间为  $S$ ，利用率粗略为：

$$\rho = \lambda S$$

当  $\rho$  接近 1，排队等待会非线性上升。Little's Law 给出长期平均关系：

$$L = \lambda W$$

$L$  是系统内平均请求数， $W$  是平均停留时间。它不能替代 tail latency 模型，但能提醒一个事实：吞吐接近极限时，队列一定涨。continuous batching 能提高吞吐，降低单位 token 成本，但会给单个请求引入 batching wait；prefill 大请求还会阻塞 decode 小步，影响 TPOT。

Listing 2.96 SLO 报告必须分阶段

```

TTFT =
 queue_wait_before_admit
+ batching_wait_prefill
+ prefill_compute
+ first_decode_wait
+ first_decode_compute
+ first_token_stream_wait

TPOT =
 decode_batch_wait
+ decode_compute
+ sampler
+ stream_backpressure

```

因此“GPU 利用率提高”不自动等于 SLO 改善。若连续合批把 TPOT p50 降低，却让 TTFT p95 暴涨，就要按业务目标重新设置 admission 和 batching window。

### 证据包：一份可复查的 serving CSV

服务化章节必须落到 artifact。下面的 CSV 不是为了模拟完整线上系统，而是规定最小证据字段：队列、prefill、decode、stream、KV 预算和结束原因必须分开。

Listing 2.97 最小 serving trace CSV

```

request_id,prompt_tokens,max_new,arrival_ms,admit_ms,finish_ms,ttft_ms,↵
↵ tpot_p95_ms,kv_reserved,kv_written,queue_wait_ms,prefill_ms,decode_steps,↵
↵ finish_reason
r1,32,16,0,0,185,42,8.9,1572864,393216,0,31,16,completed
r2,2048,8,3,18,620,440,18.5,100663296,8421376,15,401,8,completed
r3,64,128,4,0,97,0,0,12582912,0,0,0,0,rejected_kv_budget
r4,128,64,9,21,260,73,12.1,12582912,2097152,12,52,17,client_gone

```

这份表能训练三个判断。第一，**r2** 的 TTFT 高主要来自长 prompt 的 prefill，而不是 decode step 慢；第二，**r3** 在 admission 阶段被拒绝，比 decode 中途 OOM 更可控；第三，**r4** 必须能证明客户端断开后释放了未写入的 KV 预留。若 trace 没有 **kv\_reserved** 和 **kv\_written**，事故复盘会退化成猜。

### 队列验收：用公式压住“平均值幻觉”

给定一个 60 秒窗口，若到达 1200 个请求，则  $\lambda = 20$  requests/s。若平均端到端停留时间  $W = 180ms = 0.18s$ ，Little’s Law 给出平均系统内请求数：

$$L = \lambda W = 20 \times 0.18 = 3.6$$

这个数若和 trace 中平均 in-flight 请求数差很远，说明窗口、时间戳或采样口径有问题。再看 tail：如果 p50 TTFT 是 40ms，p95 TTFT 是 440ms，不能用平均 tokens/s 宣称服务健康。服务报告至少要包含：

Listing 2.98 SLO 验收字段

```

arrival_rate_rps
admission_reject_rate
queue_wait_p50_p95_p99
ttft_p50_p95_p99

```

```
tpot_p50_p95_p99
prefill_batch_size_histogram
decode_batch_size_histogram
kv_reserved_peak
kv_written_peak
client_cancel_count
fallback_backend_count
```

continuous batching 的正确目标不是“batch 越大越好”，而是在业务 SLO 下最大化吞吐。若 batching window 从 2ms 增到 20ms，让吞吐提升 15%，但 TTFT p95 从 120ms 变成 700ms，这可能对离线任务合理，对交互式聊天不合理。调度器必须把业务目标写进 policy，而不是只追硬件利用率。

### 工程事故：KV 预算只算已用，不算已预留

一个常见线上事故是：系统只统计已经写入的 KV bytes，没有统计 admitted request 未来保留的 token。短时间内多个长上下文请求同时进入，decode 到一半才发现 KV 不够，只能中止请求或触发 OOM。正确做法是在 admission 阶段按最大上下文或保守 token budget 预留，并在取消、结束或缩短输出时归还。

Listing 2.99 KV 预算事故复盘

```
bad accounting:
 kv_used counts written tokens only
 long requests pass admission
 OOM happens during decode

fixed accounting:
 kv_reserved counts admitted future slots
 kv_written tracks actual progress
 cancellation releases reserved - written
 incident trace records release reason
```

这个事故说明 serving 的“状态”不只是 tensor。预算、预留、队列位置、deadline、客户端连接和 sampler RNG 都是状态。AI Infra 如果不讲这些，系统只能停留在本地 demo。

### LangChain 类上层编排不是推理引擎

到这里为止，本章主要从推理服务内部看请求：admission、prefill、decode、stream、KV 预算和 SLO。真实应用里，请求通常不会直接打到一个裸的 decoder，而是先经过 LangChain、LangGraph、LlamaIndex 或自研 orchestration layer。这一层负责编排 prompt template、chat history、retriever、tool、agent loop、callback、trace、structured output 和 guardrail。它不是推理引擎本身，但它会决定推理引擎收到什么 token、什么时候收到、允许用多少 context、什么时候取消、如何流式返回。

因此第三册不能把上层框架当作“Python 应用细节”略过。对 AI Infra 工程师来说，关键不是背某个框架 API，而是识别它传下来的系统合同。一个 LangChain 类请求进入本地引擎前，至少要被规范化成下面的边界对象：

Listing 2.100 上层编排传给推理服务的最小合同

```
OrchestratedRequest:
 request_id
 conversation_id
 model_policy:
 model_id
 quant_profile
```

```

 backend_preference
 max_context
tokenizer_policy:
 tokenizer_hash
 chat_template_hash
 add_bos
 stop_token_ids
prompt_budget:
 history_tokens
 retrieved_tokens
 tool_result_tokens
 reserved_output_tokens
retrieval_context:
 chunk_ids
 chunk_token_counts
 citation_ids
 rerank_scores
tool_policy:
 allowed_tools
 max_tool_calls
 per_tool_timeout_ms
 idempotency_required
slo:
 deadline_ms
 streaming_required
 cancel_token
trace:
 trace_id
 parent_span_id

```

这份合同把“应用层好不好用”落到系统字段。若 `chat_template_hash` 变了，同样的 messages 会变成不同 token；若 retriever 多塞了 3000 个 token，admission 的 KV 预算会改变；若 tool loop 没有 `max_tool_calls`，一个请求可能反复调用工具，最后把队列和上下文窗口拖垮；若 trace 没有父子 span，p95 抖动时无法区分是检索慢、工具慢、prefill 慢还是 streaming 慢。

### RAG 状态机：检索也是资源账本

RAG 不是“把几段文档拼到 prompt 里”。它是一条会消耗 CPU、网络、存储、向量库、reranker、token budget 和 KV cache 的流水线。最小状态机如下：

| 状态             | 输出事实                                  | 失败或预算边界             |
|----------------|---------------------------------------|---------------------|
| QueryReceived  | user query、conversation id            | 空输入、权限、deadline     |
| QueryRewritten | rewritten query、rewrite version       | 改写漂移、额外 LLM call 成本 |
| Embedded       | query embedding、embedding model id    | embedding 服务慢、维度不匹配 |
| Retrieved      | chunk ids、raw scores                  | 向量库超时、召回不足          |
| Reranked       | ordered chunk ids、rerank scores       | reranker 成本、排序漂移    |
| PromptBuilt    | prompt bytes、token count、citation map | context 超限、引用丢失     |
| Admitted       | KV 和 workspace 预留                     | KV 预算、队列 deadline   |
| Generated      | streamed tokens、finish reason         | stop 规则、慢客户端、取消     |
| Audited        | trace、citations、quality flags         | 引用不匹配、schema 不合法    |

每个状态都要产生可复查字段。尤其是 `PromptBuilt`，它必须保存 prompt template version、chat template version、tokenizer hash、每个 chunk 的 token span 和 citation id。否则“模型乱答”可能只是检索 chunk 被截断、引用编号错位、中文全角符号被 tokenizer 拆得更多、或者 stop token



和模板冲突。

**Listing 2.101** RAG prompt budget 报告

```
rag_budget:
 max_context: 8192
 system_tokens: 186
 history_tokens_before_compression: 3120
 history_tokens_after_compression: 880
 query_tokens: 42
 retrieved_chunks:
 - chunk_id: doc17:0042
 tokens: 512
 score: 0.82
 citation_id: C1
 - chunk_id: doc91:0007
 tokens: 384
 score: 0.79
 citation_id: C2
 reserved_output_tokens: 1024
 total_prompt_tokens: 2004
 admission_context_tokens: 3028
```

这张账本直接连接底层推理。Prefill FLOPs、KV 预留、TTFT 和 long-context 带宽都由 `admission_context_tokens` 推导；引用可靠性由 `citation_id` 到 prompt span 的映射保证；压缩是否破坏上下文，要靠压缩前后 token 和引用覆盖率报告判断。上层框架越灵活，越不能让 prompt 构造成为不可见黑盒。

### Agent/tool loop: 每一步都是一次小型事务

Agent 常见执行形态是“模型思考，决定调用工具，工具返回，模型继续”。系统风险来自循环：每次模型调用消耗 token 和 KV，每次工具调用消耗网络或外部系统，每次工具结果又会被写回上下文。没有上限和事务身份，agent 很容易制造无法复盘的长尾。

**Listing 2.102** tool call 事务字段

```
ToolCall:
 tool_call_id
 request_id
 step_index
 tool_name
 input_schema_hash
 input_json_checksum
 idempotency_key
 start_ms
 timeout_ms
 output_bytes
 output_token_estimate
 status: ok | timeout | cancelled | schema_error | tool_error
```

工具调用的返回值不能直接拼进 prompt。它必须先过 schema、大小、敏感字段、编码和 token 预算检查；长结果要截断或摘要；外部动作要有幂等键；取消要能传到工具层；迟到工具结果不能写入已经结束或已经重试的新 step。工具循环的 admission 不只看 LLM 的 max context，还要看未来步骤预算：

$$T_{history} + T_{retrieved} + T_{tool\_result} + T_{reserved\_output} \leq T_{max\_context}$$

$$N_{tool\_calls} \leq N_{max\_steps}, \quad \sum_i timeout_i \leq deadline_{request}$$

这些约束看似应用层，实际会影响底层 scheduler。一个未限步的 agent 可能让 decode slot 长时间被同一 conversation 占用；一个超长 tool result 可能把下一轮 prefill 推成大请求；一个慢工具会让用户看到 TTFT 高，但问题根本不在模型 kernel。Serving trace 必须能把 `tool_wait_ms`、`prompt_build_ms`、`queue_wait_ms`、`prefill_ms` 和 `decode_ms` 拆开。

### 上层框架错误如何变成底层事故

| 上层错误                                                | 底层表现                                                           | 必须补的证据                                                                                     |
|-----------------------------------------------------|----------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| chat template 版本漂移<br>retrieval chunk 无 token 预算    | tokenizer ids 全变, logits oracle 失效<br>长 prompt 拉高 TTFT 和 KV 预留 | template hash、tokenizer golden<br>prompt budget、chunk token<br>count                       |
| tool result 未截断<br>agent 无最大步数<br>stop string 按字符匹配 | prefill 突然变成长上下文请求<br>请求占用队列和外部工具过久<br>streaming 可能漏停或误停       | tool output bytes/tokens<br>step index、max steps、deadline<br>stop token ids、decoder buffer |
| 引用 id 和 prompt span 脱节<br>取消只停客户端连接                 | RAG 答案看似有引用但不可审计<br>KV 和 tool call 继续占资源                       | citation map、retrieval trace<br>cancel propagation trace                                   |

这张表给出一个判断标准：LangChain 类框架接入本地引擎，不是“把 URL 改成本地 OpenAI-compatible endpoint”就结束。真正的接入要把 prompt、retrieval、tool、trace、预算、取消、stream backpressure 和质量回归都接进同一套证据系统。否则上层灵活性会把底层推理引擎重新变成不可解释的黑盒。

### OpenAI-style streaming 和回调事件

上层框架通常通过 streaming/callback 观察中间状态：新 token、tool call、tool result、retrieval event、error、finish。底层服务不要只输出 token 字符串，而应输出可映射到状态机的事件流。

Listing 2.103 统一 streaming event schema

```
StreamEvent:
 trace_id
 request_id
 event_index
 event_type:
 retrieval_started | retrieval_finished |
 tool_call_started | tool_call_finished |
 token_delta | finish | error | cancelled
 span_id
 parent_span_id
 timestamp_ms
 token_id_optional
 text_delta_optional
 tool_call_id_optional
 usage_delta:
 input_tokens
 output_tokens
 kv_reserved_bytes
 backpressure_ms
```

这套事件不是为了取代框架 callback，而是为了让 callback 有系统含义。若客户端很慢，`backpressure_ms` 会增长；若工具超时，`tool_call_finished` 应带 `timeout`；若用户取消，后续 token event 必须停止，并出现资源释放证据。这样 LangChain、LangGraph、LlamaIndex 或自研前端都可以消费同一种底层 trace，而不是各自解析日志。

### 上层编排的 release gate

把上层框架接入第三册项目后，发布门禁要增加几类测试：

#### 验收与评分标准

- 固定 `messages`、chat template、tokenizer hash，验证 prompt token ids 不漂移。
- 固定 retrieval fixture，验证 chunk 排序、citation map、prompt token span 和答案引用。
- 固定 tool schema，验证合法 JSON、非法 JSON、超长输出、超时、取消和迟到结果。
- 固定 agent 最大步数，验证 step budget、deadline 和 token budget 会拒绝失控循环。
- 固定 streaming trace，验证 token delta、tool event、finish reason 和取消释放资源。
- 固定质量回归集，分别记录 answer、citation、tool-call JSON、TTFT、TPOT 和 queue wait。

这类测试不会替代 kernel oracle，但它能防止另一个常见失败：底层模型没有变，kernel 没有变，只因为 prompt template、retriever、tool schema 或 callback 行为变了，线上质量和 SLO 就漂移。第三册若要覆盖真实 AI Infra，必须把“上层编排的可测性”也纳入工程验收。

### 实验：调度策略的反证输入

#### 习题与作业

实验一：构造三类请求：短 prompt 短输出、长 prompt 短输出、短 prompt 长输出。比较 FIFO、prefill/decode 分离、continuous batching 三种策略的 TTFT、TPOT、p95、拒绝率和 KV 使用曲线。

实验二：模拟慢客户端。让 streaming 队列持续阻塞，验证 scheduler 是否停止继续生成、是否释放取消请求的 KV、是否记录 `stream_backpressure_ms`。

实验三：把 admission 从 `kv_used` 改成 `kv_reserved`。构造多个长上下文请求，证明中途 OOM 变成 admission reject，并给出可行动错误原因。

### 硬验收

读完本节，应该能把第三册项目接成业务服务：

- 请求和响应字段能表达资源、采样、后端、量化、trace 和停止原因。
- admission control 在请求进入模型前预算 KV、workspace、arena 和队列。
- 调度器区分 prefill 与 decode，并能记录 queue wait、batch size 和 context。
- 取消、超时和客户端断开都能释放 KV cache 与事件队列。
- backpressure 会影响调度，而不是只在 I/O 层堆积。
- 错误分类能让业务决定重试、降级、提示或告警。
- SLO 报告拆开 queue、TTFT、decode、内存、fallback 和拒绝率。

## 2.8 上层 AI 应用编排：RAG、Agent、工具和状态图

本册前面把本地推理引擎拆成 tokenizer、模型导入、KV cache、scheduler、backend 和 profiler。真实应用还会在引擎上面再叠一层：prompt template、retrieval、reranker、tool calling、agent loop、memory、stream callback、trace 和 evaluation。LangChain 类框架提供 agent harness 和模型/工具抽象，LangGraph 类运行时强调长时间、可恢复的状态图，LlamaIndex 类系统强调数据导入、索

引、检索和 RAG 查询。名字会变，API 会变，但它们共同暴露的工程问题不会变：上层编排会改变 token、上下文、队列、KV 预算、质量和失败语义。

因此 AI Infra 工程师不能只会写 decode kernel。一个 production request 到达模型前，可能已经执行过文档解析、chunk、embedding、向量检索、权限过滤、rerank、prompt packing、tool schema 验证和 agent checkpoint。若这些环节没有证据，底层 tokens/s 再高，也无法解释“为什么这次回答慢、为什么引用错、为什么工具重复执行、为什么上下文超限”。

### 常见误区

本节是第三册的扩展层，不改变主线。主线仍然是本地推理引擎、模型加载、KV cache、compiler/runtime 和 CPU/GPU 后端。应用编排只保留和底层引擎接口有关的合同：token、span、budget、trace、取消、质量回归和资源释放。不要把本节读成某个上层框架 API 教程。

### 三类上层框架对应三类系统合同

上层框架可以先按职责拆，而不是按库名记。

| 类别             | 典型职责                                               | 推理引擎必须看到的合同                                            |
|----------------|----------------------------------------------------|--------------------------------------------------------|
| Agent harness  | messages、prompt、tools、middleware、structured output | token budget、tool schema、stop 规则、trace id              |
| 状态图 runtime    | durable execution、checkpoint、human-in-the-loop、恢复  | state version、step id、checkpoint id、取消和重放边界            |
| RAG/data layer | ingestion、chunk、embedding、index、retrieve、rerank    | chunk id、embedding version、citation span、ACL、freshness |

这个表的关键是：上层框架不是“业务代码之外的装饰”。它们输出的字段会进入第三册所有底层模型。Prompt 变长会提高 prefill 成本；retrieval chunk 变多会提高 KV 预留；tool loop 变深会提高请求停留时间；状态图 checkpoint 不正确会导致重复工具调用；stream callback 背压会让 decode 结果堆积。

### 源码级对标：不要只写“某框架支持 RAG/Agent”

应用层对标必须落到源码对象和接口边界。不同版本目录会变，所以报告要固定 commit/tag，并用符号搜索确认路径。最低阅读对象如下：

| 系统类别                        | 应阅读的源码对象                                                    | 要迁移到本地引擎的合同                                              |
|-----------------------------|-------------------------------------------------------------|----------------------------------------------------------|
| LangChain 类 agent harness   | message/model/tool 抽象、callback、structured output parser     | messages hash、tool schema hash、stream event、error class  |
| LangGraph 类状态图 runtime      | graph state、checkpoint saver、step transition、resume/cancel  | step id、checkpoint id、state version、迟到结果拒绝               |
| LlamaIndex 类 RAG/data layer | document/node、index、retriever、query engine、citation mapping | doc/chunk id、embedding version、index epoch、citation span |
| 向量库/检索层                     | collection schema、metadata filter、distance metric、top-k API | tenant ACL、metric、normalized flag、recall trace           |

报告不需要复制大段源码，但必须写清四件事：入口对象叫什么，状态字段在哪里，失败或取消如何传播，哪些字段必须下传给本地推理引擎。若一个框架只把最终 prompt 字符串传给模型，而不暴露 token span、citation map 和 template hash，本地引擎仍能生成文本，但很难做引用 oracle、token 预算回归和质量复盘。若状态图 runtime 有 checkpoint，但工具 side effect 没有 idempotency key，恢复后仍可能重复执行外部操作。

Listing 2.104 上层框架源码阅读报告字段

```
framework_source_report:
 framework_name:
 version_or_commit:
 entry_objects:
```

```

state_objects:
cancellation_path:
retry_or_resume_path:
stream_callback_path:
trace_fields_exposed:
fields_missing_for_local_engine:
adaptation_boundary:

```

这份报告把“上层生态了解”变成工程证据。它也防止应用层喧宾夺主：本地引擎只吸收必要合同，不复制整个框架。

### 从用户请求到模型输入的完整状态机

一个上层 AI 请求可以写成下面的状态机。它比裸 serving 状态机更长，因为模型调用只是其中一段。

| 状态                | 产物                               | 主要错误边界            |
|-------------------|----------------------------------|-------------------|
| MessageReceived   | messages、tenant、deadline         | 格式、权限、超时          |
| TemplateRendered  | prompt bytes、template hash       | 模板漂移、角色丢失         |
| QueryPlanned      | retrieval query、tool plan        | 查询改写漂移、计划失控       |
| ContextRetrieved  | chunk ids、scores、ACL proof       | 召回不足、越权、过期索引      |
| ContextPacked     | token spans、citation map         | context 超限、引用错位   |
| AdmittedToModel   | KV 预算、backend plan               | 队列超时、资源不足         |
| ModelRunning      | token stream、tool call proposal  | stop 错误、schema 错误 |
| ToolExecuting     | tool result、side-effect receipt  | 重复执行、迟到结果         |
| StateCheckpointed | graph state、step result          | 崩溃恢复、重放一致性        |
| Finalized         | answer、usage、trace、quality flags | 引用不成立、输出不合规       |

这张表要求每个状态有可保存产物。尤其是 **TemplateRendered**、**ContextPacked** 和 **StateCheckpointed**。如果只保存最终 answer，之后无法判断错误来自模板、检索、上下文拼接、模型、工具，还是状态恢复。

### 文档导入不是普通文件读取

RAG 的第一条链路是 ingestion。输入可能是 PDF、HTML、Markdown、代码仓库、数据库行、工单、聊天记录或日志。它们不能直接变成 embedding。导入器必须先建立文档身份、解析版本、权限、时间和可复现内容。

Listing 2.105 DocumentRecord 最小字段

```

DocumentRecord:
 doc_id
 source_uri
 source_type
 tenant_id
 acl_policy_id
 content_hash
 parser_version
 created_at
 updated_at
 visibility_epoch
 raw_bytes
 parsed_text
 parse_warnings

```

**content\_hash** 是数据事实，**parser\_version** 是解释事实，**acl\_policy\_id** 是权限事实，**visibility\_epoch** 是索引可见性事实。缺任意一个，恢复和审计都会变弱。例如 PDF 解析器升

级后，同一个文件可能抽出不同换行；权限系统更新后，旧索引里的 chunk 可能不该再被检索；文档更新后，旧 chunk 的 citation 可能指向过期内容。

### chunking 的合同：token、语义和引用必须同时成立

Chunk 不是随便按字符切。它必须同时满足 token 预算、语义边界、引用可回溯和检索粒度。太短会召回碎片，太长会挤占 context；只按字符切会断开代码块、表格和定义；只按语义切可能产生过长 chunk。

**Listing 2.106** ChunkRecord 最小字段

```
ChunkRecord:
 chunk_id
 doc_id
 source_span_bytes
 source_span_lines
 text
 token_count
 tokenizer_hash
 chunker_version
 overlap_prev_tokens
 overlap_next_tokens
 section_path
 citation_label
 acl_policy_id
```

Chunker 的验收要有反例。代码文档要保持函数签名和说明在同一 chunk；FAQ 要避免把问题和答案切开；表格要保存列名；中文文档要按 tokenizer 统计 token，而不是按字符猜；多租户数据要让每个 chunk 带 ACL，而不是只给文档带 ACL。

| 错误切法         | 线上表现                | 应补证据                        |
|--------------|---------------------|-----------------------------|
| 按字符固定长度切     | prompt 中引用缺上下文      | source span、section path    |
| 忽略 tokenizer | 实际 token 超预算        | tokenizer hash、token count  |
| 重叠过大         | 重复 chunk 挤占 context | overlap tokens、dedup report |
| 表格行列拆散       | 答案引用无法验证            | table parser version        |
| 权限只在 doc 层检查 | chunk 被迁移后越权        | chunk ACL、visibility epoch  |

### Embedding 和 vector store 是数据库问题

Embedding 不是“调用一次模型得到向量”。它是一个带版本、维度、归一化、距离度量、批处理、索引构建和可见性切换的数据系统。

**Listing 2.107** EmbeddingRecord 最小字段

```
EmbeddingRecord:
 chunk_id
 embedding_model_id
 embedding_model_version
 embedding_dim
 distance_metric: cosine | dot | l2
 normalized: true
 vector_dtype: f32 | f16 | int8
 vector_checksum
 index_partition
 visible_epoch
```



如果 embedding 模型升级，旧向量和新向量通常不能混在同一个距离空间里直接比较；如果 cosine 检索要求单位向量，却忘了归一化，分数排序会漂移；如果向量量化成 int8，召回率和存储带宽都要重新评估。Vector store 的 schema 必须把这些事实写出来。

Listing 2.108 IndexBuild 状态机

```
RawDocument
-> ParsedDocument
-> ChunkedDocument
-> EmbeddedChunks
-> IndexBuilding
-> ShadowIndexReady
-> VisibleIndex
-> RetiredIndex
```

这条状态机避免半成品索引对线上可见。新索引应先在 shadow 状态完成 build、校验、抽样查询和权限检查，再通过 `visible_epoch` 切换；旧索引保留一段时间用于回滚。删除文档也不能只删原文件，要向索引写 tombstone 或推进 visibility epoch，保证旧 chunk 不再被召回。

### 检索路径：召回、过滤、重排和打包是四个阶段

Query-time RAG 不应写成一个 `retrieve(prompt)`。至少拆成四个阶段：

Listing 2.109 RAG query pipeline

```
1. query rewrite:
 messages -> retrieval query

2. candidate recall:
 dense vector / sparse BM25 / metadata filter -> candidates

3. rerank:
 candidates -> ordered evidence with scores

4. context packing:
 evidence -> prompt spans under token budget
```

召回阶段追求覆盖，重排阶段追求精度，打包阶段追求上下文预算和引用可审计。把这三件事混在一起，会出现很难解释的质量问题：召回不足被误判为模型幻觉，reranker 慢被误判为 prefill 慢，context packing 截断导致引用看起来存在但答案用不到。

Listing 2.110 RetrievalTrace 最小字段

```
RetrievalTrace:
 query_id
 rewrite_version
 embedding_model_version
 index_epoch
 recall_top_k
 candidates_before_acl
 candidates_after_acl
 rerank_model_id
 rerank_top_k
 packed_chunk_ids
 packed_token_count
```

```
dropped_relevant_candidates
citation_map_checksum
```

这份 trace 让质量评估可以定位。若 `candidates_before_acl` 很多但 after ACL 很少，可能是权限或租户过滤导致召回不足；若 `dropped_relevant_candidates` 不为 0，说明 token budget 太紧或 packing 策略有问题；若 rerank top-k 改变但 final answer 变差，要看 reranker 版本和分数分布。

### Context packing：别把长上下文当垃圾桶

长上下文不是免费的。第三册前面已经算过 KV bytes/token 和 context length 对 TPOT 的线性影响。上层 context packing 要把检索、历史、工具结果和输出预留放进同一条不等式：

$$T_{system} + T_{history} + T_{query} + T_{retrieved} + T_{tools} + T_{reserved} \leq T_{max}$$

如果预算不够，不能简单截断尾部。更合理的策略是按优先级压缩：保留 system 和安全策略；保留当前 query；保留高分证据和 citation span；压缩旧历史；截断低分 chunk；对超长 tool result 摘要。每一次压缩都要留下报告。

Listing 2.111 ContextPackingReport

```
max_context: 8192
system_tokens: 180
query_tokens: 44
history_original_tokens: 3200
history_packed_tokens: 900
retrieved_original_tokens: 4096
retrieved_packed_tokens: 1536
tool_original_tokens: 1200
tool_packed_tokens: 300
reserved_output_tokens: 1024
dropped_chunks: [doc8:0012, doc9:0003]
compression_policy: summarize_history_keep_recent_4
```

这份报告直接解释 TTFT、KV 预算和质量边界。若 answer 缺少某个事实，先看该 chunk 是否被召回、是否通过 ACL、是否被 rerank 保留、是否被 packing 丢弃，而不是立刻怀疑模型。

### Hybrid retrieval 和 rerank 的系统取舍

很多生产 RAG 会组合 dense retrieval、sparse retrieval、metadata filter 和 reranker。Dense 向量擅长语义相似，BM25 类稀疏检索擅长关键词和专有名词，metadata filter 处理租户、时间和类型，reranker 用更贵模型在小候选集上重排。

| 阶段              | 购买的能力      | 代价                  |
|-----------------|------------|---------------------|
| Dense recall    | 语义召回、同义表达  | embedding 成本、向量索引内存 |
| Sparse recall   | 精确词、编号、错误码 | 词法匹配弱语义             |
| Metadata filter | 权限、时间、类型约束 | 过滤太早可能伤召回           |
| Rerank          | 小集合精排      | 额外模型延迟和成本           |
| MMR/diversity   | 减少重复 chunk | 可能牺牲最高相关分           |

这张表要和 SLO 合起来看。若 RAG 查询 p95 高，不能只看模型 TPOT。可能是 embedding 服务慢、vector store p95 高、reranker batch 不稳定、或者 context packing 太大导致 prefill 变慢。RAG 的 SLO 报告必须分段。

Listing 2.112 RAG SLO 分段

```
rag_slo:
 rewrite_ms
 embedding_ms
 vector_search_ms
 sparse_search_ms
 acl_filter_ms
 rerank_ms
 context_pack_ms
 model_queue_wait_ms
 prefill_ms
 decode_tpot_p95_ms
```

### Agent 状态图：持久化比循环代码重要

Agent loop 最小形态是 LLM -> tool -> LLM，但 production agent 更接近状态图。每一步都可能失败、暂停、等待人工确认、重试或恢复。状态图的核心对象不是函数，而是 **AgentState**。

Listing 2.113 AgentState 最小字段

```
AgentState:
 thread_id
 run_id
 step_id
 messages
 scratchpad_summary
 tool_calls_pending
 tool_results_committed
 memory_refs
 budget:
 max_steps
 remaining_steps
 deadline_ms
 remaining_tokens
 checkpoint_id
 state_version
```

这里的 **thread\_id** 表示长期会话，**run\_id** 表示一次执行，**step\_id** 表示状态图中的一步。Checkpoint 不能只保存 messages；还要保存预算、pending tool、已经提交的 tool result、memory 引用和 state version。否则崩溃恢复时可能重复调用有副作用的工具，或把迟到工具结果写到错误步骤。

Listing 2.114 Agent step 状态机

```
Planned
-> ModelCalling
-> ToolProposed
-> ToolValidated
-> ToolExecuting
-> ToolCommitted
-> Checkpointed
-> NextStepOrFinished
```

每个转移都要有幂等身份。尤其是 **ToolExecuting->ToolCommitted**，必须区分“工具返

回了”和“工具结果被状态图接受”。外部支付、发邮件、改数据库这类工具必须有 idempotency key 或补偿策略；没有幂等键的工具应默认禁止自动重试。

### Tool calling 的 schema、权限和副作用

Tool calling 的风险不是 JSON 解析，而是把模型输出当成可信指令。工具执行前至少要通过四类检查：

- schema：字段类型、枚举、范围、必填项和默认值。
- permission：当前用户、租户、工具和资源是否匹配。
- budget：工具耗时、输出字节、输出 token 和重试次数。
- side effect：是否幂等，失败后是否可补偿，是否需要人工确认。

Listing 2.115 ToolPolicy

```
ToolPolicy:
 tool_name
 input_schema_hash
 output_schema_hash
 allowed_tenants
 requires_human_approval
 idempotency_mode: required | optional | forbidden
 max_output_bytes
 max_output_tokens
 timeout_ms
 retry_policy
 redaction_policy
```

工具输出也不能无限进入上下文。长输出要摘要，敏感字段要脱敏，二进制或 HTML 要转成安全文本，错误信息要分类。否则一次工具异常可能把巨大的 stack trace 塞进 prompt，既浪费 KV，又泄漏内部信息。

### Memory：短期上下文和长期记忆不是一回事

Agent 框架常说 memory，但 memory 至少分三类。短期 memory 是当前 messages 和 scratchpad；压缩 memory 是旧对话摘要；长期 memory 是跨会话保存的用户偏好、事实或任务状态。三者的生命周期、权限和召回方式不同。

| memory 类型  | 用途             | 风险                     |
|------------|----------------|------------------------|
| Short-term | 当前推理步骤上下文      | token 膨胀、scratchpad 泄漏 |
| Compressed | 长对话摘要          | 摘要丢失细节、不可审计            |
| Long-term  | 用户偏好、项目事实、任务状态 | 隐私、过期、错误记忆污染           |

长期 memory 应像 RAG 文档一样有来源、时间、权限和删除机制。不能因为模型说“用户喜欢 X”就永久写入。写 memory 需要 evidence：来自哪条用户消息，是否需要确认，何时过期，如何删除。否则 memory 会成为不可控的隐形 prompt。

### Human-in-the-loop 是状态转移，不是弹窗

需要人工确认时，系统不应把请求卡在某个线程里等待。正确做法是把 agent state 转到 **WaitingForHuman**，释放模型和 KV 资源，保存 checkpoint，并让外部事件恢复执行。

Listing 2.116 HumanInterrupt

```
HumanInterrupt:
 run_id
 step_id
```

```

reason
proposed_action
required_role
expires_at
checkpoint_id
resume_policy

```

恢复时要检查 state version、权限、deadline 和工具副作用。若人工批准发生在旧 checkpoint 上，而状态图已被取消或重试，批准不能直接生效。Human-in-the-loop 的难点不是 UI，而是让暂停、恢复和取消都保持一次性语义。

### 结构化输出：schema 是接口，不是提示词愿望

Structured output 常被写成“请按 JSON 输出”。这不够。Schema 必须成为 release gate。模型输出要经过解析、校验、修复策略和失败分类；修复不能无限循环；错误要能回传给业务。

Listing 2.117 StructuredOutputReport

```

schema_hash
raw_output_bytes
parse_status: ok | json_error | schema_error | repaired | rejected
repair_attempts
fields_missing
fields_out_of_range
enum_violations
final_object_checksum

```

如果 schema 是外部 API 的输入，还要做语义检查。例如金额不能为负，日期不能在过去，用户 id 必须属于当前租户。JSON 合法不代表业务安全。工具调用尤其如此：schema gate 只能证明形状，permission gate 和 side-effect gate 才证明能执行。

### 质量评估：RAG 和 agent 不能只看最终文本

上层链路的 evaluation 要拆成多层，不同层用不同 oracle。

| 层级         | 指标                                       | 失败定位              |
|------------|------------------------------------------|-------------------|
| Retrieval  | recall@k、MRR、nDCG、ACL pass               | 索引/召回/过滤          |
| Rerank     | pairwise preference、top-k overlap        | reranker 版本或特征    |
| Context    | packed relevant tokens、citation coverage | packing/压缩        |
| Generation | answer exactness、refusal、format validity | 模型/模板/sampler     |
| Citation   | citation precision/recall、span match     | 引用映射/截断           |
| Tool       | schema pass、side-effect once、timeout     | agent/tool policy |
| SLO        | TTFT、TPOT、tool wait、retrieval p95        | 系统瓶颈              |

RAG 的一个合格 fixture 应保存 query、gold relevant chunk、允许答案要点、禁止答案要点、citation 要求和 token budget。Agent fixture 应保存工具响应、期望 tool call 序列、最大步数和取消点。这样才能判断是召回错、引用错、工具错，还是模型生成错。

Listing 2.118 RAGEvalCase

```

case_id
query
tenant_id
gold_relevant_chunk_ids
forbidden_chunk_ids

```

```

expected_answer_facts
expected_citation_ids
max_prompt_tokens
expected_failure_mode_optional

```

### 安全和稳定性：上层链路的硬边界

上层编排要有明确拒绝路径。常见硬边界包括 prompt 长度、检索权限、工具权限、输出 schema、PII、越权引用、外部请求速率、模型热更新和 OOM 降级。

Listing 2.119 上层请求拒绝原因

```

reject:
 prompt_too_long
 retrieval_acl_denied
 index_epoch_unavailable
 tool_not_allowed
 tool_schema_invalid
 tool_requires_human_approval
 context_budget_exceeded
 structured_output_failed
 unsafe_memory_write
 tenant_rate_limited

```

拒绝不是失败，模糊继续才是失败。若 retrieval ACL 不确定，不能把未过滤 chunk 塞进 prompt；若 tool schema 不合法，不能让模型“猜着修”；若 context budget 超限，不能静默截掉 system policy；若 memory 写入不安全，不能把未确认偏好永久保存。

### 对接本地推理引擎的接口边界

上层框架最终要调用第三册的本地推理引擎。接口应避免把上层框架对象直接泄漏进后端。后端只接受规范化请求：

Listing 2.120 NormalizedGenerationRequest

```

NormalizedGenerationRequest:
 request_id
 model_id
 tokenizer_hash
 prompt_token_ids
 token_spans:
 - source: system | history | retrieval | tool
 start_token
 end_token
 evidence_id_optional
 sampling_policy
 stop_token_ids
 max_new_tokens
 kv_budget_hint
 deadline_ms
 stream_event_policy

```

这样做的好处是后端不需要理解 LangChain、LangGraph、LlamaIndex 或某个业务 SDK。它只理解 token、span、预算、停止规则和 trace。框架可以换，底层合同不换；模型可以换，tokenizer



hash 和 template hash 会暴露差异；RAG 可以换检索器，citation span 仍然可审计。

### 一份完整上层 trace 应该长什么样

最终证据不应散在日志里。一次请求的 trace 至少要能串起：

**Listing 2.121** UpperAITrace 最小结构

```
UpperAITrace:
 request_id
 conversation_id
 trace_id
 messages_hash
 template_hash
 tokenizer_hash
 retrieval:
 query_id
 index_epoch
 candidate_counts
 packed_chunk_ids
 citation_map_checksum
 agent:
 run_id
 step_count
 checkpoint_ids
 tool_call_ids
 model:
 prompt_tokens
 max_new_tokens
 ttft_ms
 tpot_p95_ms
 finish_reason
 quality:
 citation_pass
 schema_pass
 safety_flags
```

这份 trace 让团队能做三类复盘：质量复盘看 retrieval、citation 和 schema；性能复盘看 embedding、rerank、prefill、decode 和 streaming；可靠性复盘看 checkpoint、tool idempotency、取消和资源释放。没有这条 trace，上层框架越强，系统越像黑盒。

## 本章实验和验收

### 习题与作业

实验一：构造一个 20 篇文档的小 RAG 集合。为每篇文档生成 **DocumentRecord**、**ChunkRecord**、**EmbeddingRecord** 和 **RetrievalTrace**，故意加入一个过期 chunk、一个越权 chunk 和一个 tokenizer 版本变化样例，验证它们被拒绝或标记。

实验二：实现一个最小 hybrid retrieval pipeline：dense candidate、sparse candidate、metadata filter、rerank、context packing。报告 recall@k、packed token count、citation coverage、embedding latency、vector search latency 和 rerank latency。

实验三：实现一个有两个工具的 agent 状态图。一个工具无副作用，一个工具需要 idempotency key。注入超时、迟到 completion、人工确认和取消，验证 checkpoint 恢复不会重复提交副作用。

实验四：建立 10 条 RAG/agent release fixtures。每条保存 query、gold chunk、tool

response、expected citation、structured output schema 和 SLO 阈值。任何 prompt template、retriever、tool schema 或 tokenizer 改动都必须跑过这些 fixtures。

### 验收与评分标准

- 能区分 agent harness、状态图 runtime、RAG data layer 对底层引擎的不同合同。
- 文档导入、chunk、embedding、index build 有版本、权限、hash 和可见性状态。
- 检索报告能拆开 recall、ACL、rerank、context packing 和 citation map。
- Agent tool call 有 schema、权限、幂等、预算、checkpoint 和迟到结果处理。
- Memory 写入有来源、权限、过期和删除策略。
- 结构化输出和工具调用进入 release gate，而不是只靠提示词。
- 本地推理接口只消费规范化 token/span/budget/trace，不依赖上层框架对象。

## 第 3 章

# 模型导入、AI 编译器与内存计划

### 3.1 AI 编译器前端：模型导入、manifest 与 shape verifier

#### 先从一个会悄悄跑错的模型开始

推理引擎最危险的失败，不是程序立刻崩溃，而是模型能跑、速度也正常、生成结果却已经被污染。假设一个 7B 模型目录里有这些文件：

Listing 3.1 一个本地模型目录的最小文件组

```
config.json
tokenizer.model
tokenizer_config.json
weights-00001.bin
weights-00002.bin
quantization.json
```

如果 `config.json` 写着 `hidden_size=4096`，但某个 `Wq` 权重实际按 `[4096, 4096]` 以外的形状保存，后端 `linear kernel` 仍然可能读到连续字节并给出一个输出。若 `tokenizer` 的词表大小和 `lm_head` 的输出维度差一个特殊 token，采样器也可能只在极少数 prompt 上出错。若某个 `int4` 权重缺少 group scale，kernel 甚至能按照错误 scale 反量化出一串“看起来像数”的浮点值。

这就是 AI 编译器前端存在的理由。它不只是把文件读进内存，而是要把模型文件转换成一个已检查的 **manifest**（清单）和 `typed graph`。manifest 不是随手写的说明文档，而是“后端可以相信哪些事实”的合同：有哪些张量、每个张量在哪里、形状是什么、`dtype` 是什么、量化参数是什么、哪个算子消费它、输出应该是什么。

#### 核心思想

模型导入阶段的目标不是“加载成功”，而是建立可验证的语义合同。没有 manifest、shape verifier 和 `typed graph`，后端 kernel 只能在热路径里猜测输入是否正确。

#### manifest：把文件事实变成编译器事实

传统编译器的源文件需要词法和语法分析；AI 编译器的模型文件需要 manifest。manifest 至少要收集四类事实：

- 模型结构：层数、hidden size、intermediate size、head 数、KV head 数、head dimension、最大上下文长度。
- tokenizer 合同：词表大小、特殊 token、BOS/EOS/PAD 规则、是否需要加入前缀空格、byte fallback 或 normalizer。
- 权重目录：每个权重张量的名字、文件偏移、字节长度、dtype、shape、逻辑 layout 和 checksum。
- 量化元数据：量化格式、group size、scale dtype、zero point 策略、per-tensor/per-channel/per-group 边界。

下面是一个压缩后的 manifest 草图。真实工程里可以来自 JSON、二进制索引或模型格式内置 metadata；教材先写成文本，是为了看清不变量。

Listing 3.2 模型 manifest 的核心字段

```
model:
 architecture: decoder_only_transformer
```

```

layers: 32
hidden_size: 4096
intermediate_size: 11008
attention_heads: 32
kv_heads: 8
head_dim: 128
max_context: 4096

tokenizer:
 vocab_size: 32000
 bos_id: 1
 eos_id: 2

tensor:
 name: layers.0.attn.wq.weight
 role: linear_weight
 shape: [4096, 4096]
 dtype: q4_group
 group_size: 64
 scale_dtype: f16
 file: weights-00001.bin
 offset: 1048576
 bytes: 9437184

```

manifest 的作用不是增加格式复杂度，而是把“文件里有什么”变成“编译器知道什么”。后续 graph import 不再直接扫文件名；它查询 manifest。后续 shape verifier 不再靠字符串约定；它读 tensor descriptor。后续 backend lowering 不再猜某个权重是不是 int4；它读 dtype 和 quantization descriptor。

### 导入不是字符串匹配

许多教学代码会用简单规则找权重名，例如看到 `wq` 就当作 Q projection，看到 `w1` 就当作 MLP gate。真实模型格式会让这种做法很快崩掉：不同模型使用不同命名，某些权重已经被合并为 QKV packed projection，某些模型有 grouped-query attention，某些模型把 `lm_head` 和 embedding tied 在一起，某些模型的 RoPE scaling 规则也不同。

前端应该把导入分成三步：

Listing 3.3 模型导入的三阶段

```

parse_files:
 read config, tokenizer config, tensor index, quant metadata
 build raw manifest entries

resolve_schema:
 map model-specific names to canonical roles
 detect tied weights and packed projections
 attach tokenizer and generation contracts

verify_and_import:
 run shape/dtype/quant checks
 produce typed graph nodes and tensor descriptors

```

第一步只读事实，不解释太多。第二步把模型方言映射到本引擎的 canonical roles，例如 `q_proj.weight`、`attention.wq.weight`、`layers.0.attention.wq` 都可能被解析为同一个角色。第

三步才产生 typed graph。这个顺序很重要：如果一边扫文件一边直接创建 graph node，错误通常会散落在导入代码、运行时代码和后端代码里。

schema resolution：把模型方言翻译成统一角色

不同模型家族最烦人的地方不是数学完全不同，而是同一数学对象有不同名字、不同转置约定和不同融合方式。前端必须把它们翻译成本引擎内部的统一角色。否则每个后端 kernel 都会开始识别模型名字，系统很快失控。

| 外部命名示例                                 | canonical role    | verifier 要确认                     |
|----------------------------------------|-------------------|----------------------------------|
| <code>model.embed_tokens.weight</code> | token embedding   | 行数等于 vocab size, 列数等于 hidden     |
| <code>self_attn.q_proj.weight</code>   | attention Q       | 输出维等于 <code>n_q*head_dim</code>  |
| <code>self_attn.k_proj.weight</code>   | attention K       | 输出维等于 <code>n_kv*head_dim</code> |
| <code>mlp.gate_proj.weight</code>      | MLP gate          | 输出维等于 intermediate               |
| <code>lm_head.weight</code>            | logits projection | 是否 tied embedding, 词表大小一致        |

有些模型把 Q/K/V 合并成一个大张量，例如 `qkv_proj.weight`。这时 schema resolution 不能简单把它当成普通 linear，而要记录切片规则：

Listing 3.4 QKV packed projection 的导入合同

```
PackedQKV:
 source_tensor
 q_range: [0, n_q * head_dim)
 k_range: [q_end, q_end + n_kv * head_dim)
 v_range: [k_end, k_end + n_kv * head_dim)
 storage_layout: row_major | col_major | backend_packed
```

如果这个合同不显式保存，后端就可能把 Q/K/V 的切片顺序写死在 kernel 里。等另一个模型把顺序改成 Q/V/K，程序仍然能跑，但 attention 会稳定地错。

一层 decoder 的 shape 推导例子

把 shape verifier 写成代码之前，要能在纸上推导一层。设配置为：

Listing 3.5 一组 7B 级 decoder 配置

```
hidden_size = 4096
intermediate_size = 11008
attention_heads = 32
kv_heads = 8
head_dim = 128
```

则 verifier 应该推出：

| 张量角色          | 期望形状          | 推导来源                   |
|---------------|---------------|------------------------|
| Q weight      | [4096, 4096]  | $n_q * d = 32 * 128$   |
| K weight      | [1024, 4096]  | $n_{kv} * d = 8 * 128$ |
| V weight      | [1024, 4096]  | 同 K                    |
| O weight      | [4096, 4096]  | query heads 拼回 hidden  |
| MLP gate/up   | [11008, 4096] | intermediate by hidden |
| MLP down      | [4096, 11008] | hidden by intermediate |
| RMSNorm gamma | [4096]        | 每个 hidden 维度一个权重       |

这个例子很重要，因为它能立即抓住 GQA 配置错误。若 K/V 权重被导入成 [4096, 4096]，但配置写着 `kv_heads=8`，前端必须拒绝或明确标记这是非 GQA 模型；不能让 attention kernel 到运行时才发现 `kv_head` 索引和权重输出维度对不上。

### shape verifier: 让每条边都有可证明的形状

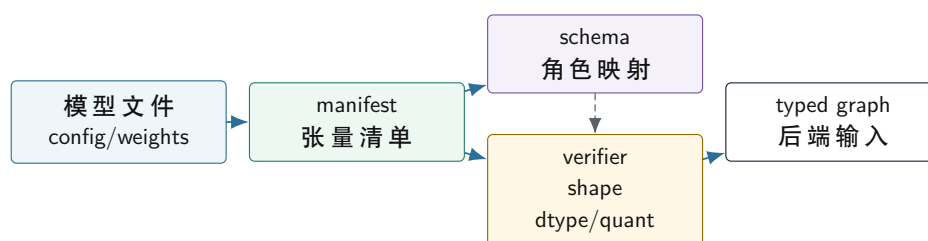
shape verifier 是 AI 编译器前端的语义分析器。它的输入是 manifest 和 graph schema，输出是带 shape 的 typed graph，或者一组明确诊断。以一层 decoder block 为例，若 hidden size 记为  $H$ ，head 数为  $A$ ，KV head 数为  $KVA$ ，head dimension 为  $D$ ，那么至少有：

Listing 3.6 decoder 层 shape 约束

```
H == A * D
K_shape == [KVA * D, H]
V_shape == [KVA * D, H]
Q_shape == [A * D, H]
Wo_shape == [H, A * D]
Wgate_shape == [I, H]
Wup_shape == [I, H]
Wdown_shape == [H, I]
RMSNorm_gamma_shape == [H]
KV_cache_shape == [layers, batch, KVA, max_context, D]
```

这些约束决定的不只是正确性，还决定性能路径。若  $KVA < A$ ，这是 grouped-query attention，decode attention kernel 要知道多个 query head 共享一组 K/V head；若  $D$  不是 SIMD-friendly 的倍数，CPU 后端要准备尾部处理；若  $I$  和 tile size 不对齐，MLP kernel 需要 remainder path。shape verifier 把这些事实提前暴露出来，而不是让后端在热循环里临时发现。

### manifest 到 typed graph 的前端流水线



前端先把文件事实稳定成 manifest，再把模型方言解析成角色，最后由 verifier 产出可被后端相信的 typed graph。

图示：本图要证明的命题是：模型导入不是直接读权重跑 kernel，而是 manifest、schema 和 verifier 共同建立编译器事实。



### dtype verifier: 不是每个字节数组都能进 kernel

shape 对了, dtype 仍然可能错。一个 linear 权重可能是 fp16、bf16、int8、int4 group-wise, 也可能是某个后端专用 packed format。dtype verifier 要检查三层事实:

- 逻辑 dtype: 算子语义中权重代表什么数值类型, 例如 f16 weight 或 q4 group weight。
- 存储 dtype: 文件中每个元素或压缩块怎样编码, 例如 4-bit pair、int8、f16 scale。
- 计算 dtype: kernel 中使用什么累加器和输出类型, 例如 fp32 accumulator、int32 dot accumulator、f16 output。

这三层不能混为一谈。int4 权重的逻辑值是近似浮点权重, 存储上可能两个 4-bit value 打包在一个字节, 计算时可能先反量化到寄存器再做 FMA, 也可能走整数 dot 再 requantize。若 manifest 只写 **dtype=int4**, 后端还不知道 group size、scale layout、zero point、padding 和尾部怎么处理。

**Listing 3.7** 量化权重 verifier 要检查的边界

```
for each quantized weight tensor:
 require group_size > 0
 require input_channels % group_size == 0 or tail_policy exists
 require scale tensor exists
 require scale length matches groups
 require packed byte length matches shape and bit width
 require backend supports this quant format
```

这些检查看似繁琐, 但比在 kernel 中读越界 scale 或错误解码权重便宜得多。前端拒绝模型是清晰错误; 后端悄悄算错 logits 是高成本错误。

### verifier 负例矩阵

前端测试不能只加载一个正确模型。必须构造坏 manifest, 证明 verifier 会在正确层次拒绝它。最小负例矩阵如下:

| 负例                    | 应报位置                         | 典型风险                  |
|-----------------------|------------------------------|-----------------------|
| embedding 行数少于 vocab  | tokenizer/embedding verifier | token id 越界           |
| K/V head 数与权重维度不一致    | shape verifier               | GQA attention 读错 head |
| QKV packed 切片顺序缺失     | schema verifier              | Q/K/V 语义交换            |
| int4 scale 数量不足       | quant verifier               | kernel 越界或误反量化        |
| group size 为 0 或不支持   | quant verifier               | 除零或 unsupported plan  |
| RoPE head dim 为奇数     | position verifier            | 二维旋转对不完整              |
| max context 小于 prompt | runtime admission            | KV cache 越界           |
| GPU 不支持目标 dtype       | backend capability           | 静默 fallback 或失败       |

每个负例都应该检查诊断内容, 而不只是“失败了”。比如 K/V head 维度错误的诊断要写出 **kv\_heads\*head\_dim** 的推导; int4 scale 数量错误要写出 expected scale count; GPU dtype 不支持要写出 requested profile 和 backend capability。

### RoPE 和位置参数也要验证

RoPE 参数如果错, shape 仍然可能全部正确。前端至少要检查:

- **head\_dim** 是否能拆成二维旋转对, 通常要求偶数。
- RoPE base、scaling 类型、原始最大位置和扩展规则是否存在。

- prefill 与 decode 使用同一 position 语义。
- sliding window 或 prefix cache 是否记录 base position。

Listing 3.8 RoPE verifier 字段

```
RopeDesc:
 head_dim
 base_theta
 scaling_policy
 original_max_position
 effective_max_position
 position_base_policy
```

如果模型使用扩展上下文策略，而 manifest 没记录 scaling policy，引擎可以选择拒绝，而不是默认用基础 RoPE。默认继续跑会得到 shape 正确但长上下文错误的结果。

### tokenizer 也是编译合同的一部分

很多人把 tokenizer 当作预处理，但本地推理引擎不能这样切开。tokenizer 决定 token id 序列，token id 决定 embedding 访问和位置编码，最终影响每一步 logits。若 tokenizer 的 BOS/EOS 规则、normalizer 或特殊 token 与模型训练时不一致，后端再快也只是高效地产生错误输入。

前端至少要验证：

- tokenizer 词表大小等于 embedding 行数，或有明确扩展规则。
- `bos_id`、`eos_id`、`unk_id` 等特殊 token 在词表范围内。
- generation config 中的 stop token 与 tokenizer 合同一致。
- prompt encode 后的 token 数不会超过当前 session 的 max context。
- tokenizer 输出 dtype 能被 embedding kernel 接受，通常是 32 位整数索引。

这部分和编译器的符号解析有相似性。源代码里的名字必须解析到某个声明；prompt 里的文本必须解析到模型词表中的 token id。若解析规则错了，后面所有 IR 和 kernel 都建立在错误输入上。

### tokenizer.json、SentencePiece 和 chat template 的细节合同

真实 tokenizer 不是一张 `string->id` 的 map。常见 tokenizer 至少包含 normalizer、pre-tokenizer、model、post-processor、added tokens 和 special token 规则。BPE、WordPiece、SentencePiece unigram 或 byte fallback 的内部算法不同，但导入器最终要产出同一种合同：给定一段 UTF-8 字节和一组 encode 选项，输出确定的 token id 序列、特殊 token 插入位置和反解码边界。

| 组件             | 决定的事实                            | 常见漂移                      |
|----------------|----------------------------------|---------------------------|
| normalizer     | Unicode 规范化、大小写、空白处理             | 可见文本相同，UTF-8 字节或 token 不同 |
| pre-tokenizer  | 如何按空格、标点或字节切分                    | 中文、emoji、全角符号切分变化         |
| model          | BPE merge、unigram score、vocab id | merge 表版本或 vocab 顺序漂移     |
| post-processor | BOS/EOS、role token、模板片段          | chat template 改版改变 prompt |
| added tokens   | 新增特殊 token 和普通 token             | token id 超过 embedding 行数  |
| byte fallback  | 未登录字符如何落到字节 token                | OOV 文本不再可逆                |

chat template 尤其不能靠人工直觉处理。用户给的是 messages: system、user、assistant、tool；模型吃的是一串带角色标记、换行、终止标记和特殊 token 的文本或 token。不同模板可能都“看起来合理”，但 token ids 完全不同。导入器要把模板版本纳入 manifest：

Listing 3.9 chat template 合同字段

```
ChatTemplateContract:
 template_hash
 roles_supported: [system, user, assistant, tool]
```

```

bos_policy
eos_policy
assistant_prefix
tool_call_marker
tool_result_marker
stop_token_ids
stop_string_fallbacks

```

这也解释了为什么上层 LangChain 类框架接入本地引擎时，不能只传一个最终 prompt 字符串。如果框架负责渲染 chat template，引擎仍要记录 `template_hash` 和 tokenizer golden；如果引擎负责渲染，框架必须传结构化 messages 和模板策略。两边都默认自己“懂模板”，最容易造成训练格式和推理格式不一致。

**Listing 3.10** tokenizer 负例应覆盖的输入

1. same visible text, different UTF-8 normalization
2. leading space vs no leading space
3. Chinese punctuation and full-width characters
4. emoji or byte fallback character
5. added special token in prompt body
6. tool-call JSON with braces, quotes and newlines
7. stop `string` crossing token boundary

这些负例的目标是让错误停在 tokenizer verifier。若 stop string 只按字符扫描，可能跨 token 边界误停；若 added token id 没检查 embedding 行数，只有包含该特殊 token 的少数 prompt 会越界；若中文 prompt 不保存 UTF-8 hex，编辑器或网页复制产生的不可见变化会让 golden 失效。

#### 诊断：错误要停在前端，而不是后端崩溃

shape verifier 的诊断要像编译器错误，而不是像运行时崩溃。一个有用诊断应该包含：哪个 tensor、期望形状、实际形状、这个期望来自哪个 config 字段或上游算子，以及可能的修复方向。

**Listing 3.11** 好的模型导入诊断示例

```

error: tensor layers.12.attn.wk.weight has incompatible shape
expected: [1024, 4096]
 reason: kv_heads(8) * head_dim(128), hidden_size(4096)
actual: [4096, 4096]
note: this looks like full multi-head K projection,
 but config declares grouped-query attention

```

这类诊断让使用者知道是模型文件、config、转换脚本还是引擎 schema 出了问题。反过来，如果只在后端报 `invalidargument` 或 `segmentationfault`，调试会被迫从机器码和二进制文件倒推语义，成本完全不对等。

#### 前端产物：typed graph 必须足够给后端使用

前端通过后，输出不应该只是“一个 graph 指针”。它至少要包含：

- Graph nodes：算子类型、输入输出边、状态边和执行阶段标签。
- Tensor descriptors：shape、stride、dtype、layout、quantization、file location、ownership。
- Symbol table：canonical tensor name 到 descriptor 的映射。
- State descriptors：KV cache、position、session buffer 和 sampler state。
- Verification report：通过了哪些约束，哪些路径需要 fallback 或 tail handling。

这些产物会直接进入后续流水线：Graph IR rewrite、Tensor IR layout 选择、memory planning、

CPU/GPU lowering 和 runtime dispatch。前端如果偷懒，后端就会被迫补课；而后端补课通常发生在最难调试、最要求性能稳定的地方。

### 真实模型资产闭环：从目录到第一条 token trace

一个本地推理引擎不能只验证 toy fixture。真正有工程说服力的前端报告，应能把一个真实模型目录推进到第一条可解释 token trace。这里的“真实”不是必须一开始支持所有格式，而是至少支持一种明确资产形态，并把文件事实、tokenizer、权重索引、typed graph 和 reference decoder 连接起来。

**Listing 3.12** 真实模型导入的最小验收路径

```
model_asset
-> parse metadata and tokenizer contract
-> build tensor index with offsets, byte lengths, dtype
-> verify shapes, quant descriptors, checksums
-> create typed graph and state descriptors
-> mmap or load weights under ownership rules
-> run tokenizer on fixed prompt
-> run one reference decode step
-> dump checkpoint trace and logits checksum
```

这条路径比“模型能加载”更严格。它要求每个阶段都有可保存的产物：`manifest.json`、`tensor-index.csv`、`verify-report.txt`、`graph-ir.txt`、`trace.csv` 和 `logits-golden.txt`。后端优化可以晚一点做，但前端资产闭环必须先稳住，否则后面的 kernel benchmark 很可能在错误输入上跑得很快。

### GGUF 和 safetensors：两类不同边界

真实模型常见资产形态至少有两类。GGUF 更像一个把 metadata、tensor directory 和权重字节放在同一容器里的本地推理资产；safetensors 更强调安全的张量存储和 metadata，tokenizer/config 通常以旁路 JSON 文件存在。教材不要求一开始同时实现两者，但要把它们的工程边界讲清楚。

| 资产形态                 | 前端要提取                                              | 常见错误边界                                        |
|----------------------|----------------------------------------------------|-----------------------------------------------|
| GGUF                 | metadata、tensor name、offset、dtype、shape、quant type | metadata 方言、量化块解释、tokenizer 字段不完整             |
| safetensors          | JSON header、tensor offsets、dtype、shape、metadata    | config/tokenizer 分离，权重名方言需要 schema resolution |
| tokenizer JSON/model | vocab、normalizer、pre-tokenizer、special tokens      | BOS/EOS 规则、byte fallback、词表和 embedding 不一致    |
| generation config    | stop token、max length、sampling defaults            | sampler 和 tokenizer 合同不一致                     |

这张表的结论是：模型格式不是普通文件读取问题。GGUF 的 tensor directory 能帮你快速建立权重索引，但你仍要解释量化块；safetensors 的张量边界清晰，但模型结构和 tokenizer 合同往往要从其他文件合并。无论哪种格式，前端最终都必须产出同一种内部 manifest，否则后端会被格式细节污染。

### tensor index：mmap 之前先证明字节范围

权重文件最好不要一开始就读成一堆 `std::vector<float>`。真实模型可能十几 GB，前端需要先建立 tensor index：每个张量的文件、偏移、字节长度、对齐、dtype、shape、checksum 和逻辑角色。只有 index 通过验证后，runtime 才能决定 mmap、按需加载、预取或复制到 backend packed cache。

**Listing 3.13** tensor index 的最低字段

```
TensorIndexEntry:
```

```
canonical_name
source_name
file_path
offset_bytes
length_bytes
alignment
shape
storage_dtype
logical_dtype
quant_descriptor_id
checksum
mmap_allowed
```

验证规则要非常机械。张量字节范围不能越过文件大小；两个张量范围不能意外重叠，除非格式明确允许 alias 或 tied weight；offset 要满足格式和后端对齐要求；packed quant tensor 的字节长度要能由 shape、bit width、block size 和 scale metadata 推导出来。若这些规则不通过，错误必须停在导入阶段。

**Listing 3.14** 字节范围 verifier 的拒绝条件

```
reject if offset + length > file_size
reject if range overlaps another tensor without alias record
reject if length != expected_storage_bytes(shape, dtype, quant)
reject if checksum exists and does not match
reject if backend requires alignment and offset violates it
```

这一步能阻止很多“后端崩溃”。例如 int4 权重少了最后一个 scale block，kernel 可能只在某个 output tile 读到越界；如果 tensor index 阶段已经计算 expected bytes，错误会变成一条清楚诊断，而不是随机 logits 漂移。

### tokenizer golden: 真实 prompt 的第一道 oracle

真实模型链路的第一个 golden output 不应该是最终生成文本，而应该是 tokenizer 输出。固定一个短 prompt，例如 **"Hello"** 或中文短句，保存 token id 序列、BOS/EOS 插入规则和特殊 token 处理。若 tokenizer 都不一致，后续 embedding、RoPE、KV cache 和 logits 全部没有比较意义。

**Listing 3.15** tokenizer golden 报告

```
tokenizer_golden:
 model_id
 tokenizer_hash
 prompt_utf8_hex
 add_bos: true
 add_eos: false
 tokens: [1, 15043]
 token_count: 2
 vocab_size: 32000
 embedding_rows: 32000
```

报告里保存 `prompt_utf8_hex` 是为了避免不可见空格、换行、全角字符、Unicode normalization 和编辑器编码把实验搞乱。中文 prompt 尤其要保存原始 UTF-8 字节，因为不同 tokenizer 对空格、字节 fallback 和 normalization 的处理差异会直接改变 token id。

### 第一条真实 trace：不要直接对最终文本

前端闭环的第一条 trace 可以只跑一层或一个 decode step，但必须包含中间 checkpoint。建议固定字段如下：

**Listing 3.16** 真实模型 first-token trace schema

```
trace_row:
 stage
 layer
 position
 tensor_name
 shape
 stride
 dtype
 layout
 checksum
 max_abs
 max_rel_error_vs_reference
```

最小 checkpoint 顺序为：

**Listing 3.17** first-token trace checkpoint

```
tokenizer
embedding
layer0.input_rmsnorm
layer0.qkv_linear
layer0.rope_q
layer0.rope_k
layer0.kv_append
layer0.attention_scores
layer0.attention_out
layer0.mlp_gate_up
layer0.block_out
final_norm
lm_head_logits
sampler_selected_token
```

如果项目暂时没有完整真实模型后端，也可以先只执行 reference path 和 shape trace。关键是产物要固定下来：同一模型资产、同一 prompt、同一 commit 应输出同一批 checkpoint checksum。后续 SIMD kernel、paged KV、int4 weight 或 GPU backend 只要在某一步开始偏离，就能定位到具体阶段。

### mmap 权重加载的所有权合同

真实权重导入通常会使用 mmap 或分块读取。mmap 的好处是避免一次性复制整个文件，并允许按需触页；代价是生命周期、page fault、文件替换和错误边界更复杂。前端 manifest 必须把“张量视图”和“文件映射”分开建模。

**Listing 3.18** mmap 权重所有权状态机

```
Mapped:
 file descriptor validated
 mapping created
 tensor index ranges point into mapping
```



**PinnedForPlan:**  
executable plan holds references to tensor views  
file identity and checksum recorded

**Packed:**  
backend-specific packed weights created  
packed cache has its own ownership

**Released:**  
no live tensor view may point into unmapped bytes

错误案例很具体：如果 runtime 在 packing 后释放 mmap，但某个 fallback kernel 仍指向原始 weight view，短测试可能不触发，长请求或错误路径才崩溃。解决办法不是“别释放”，而是让 executable plan 明确记录每个 segment 使用原始 view、packed cache 还是 device buffer。所有权状态必须能被 verifier 检查。

真实模型导入的负例包

toy 模型负例不够。真实资产导入要构造能模拟转换器错误的负例包：

| 负例                               | 应该被谁拒绝                       | 若漏掉会怎样                     |
|----------------------------------|------------------------------|----------------------------|
| tokenizer vocab 比 embedding 多 1  | tokenizer/embedding verifier | 特殊 token 偶发越界              |
| QKV packed 顺序写错                  | schema resolver              | attention 语义交换但形状正确        |
| tensor offset 少 16 字节对齐          | tensor index verifier        | SIMD load 或 mmap view 边界异常 |
| q4 scale block 少一个               | quant verifier               | 某个 tile 反量化读错              |
| RoPE scaling metadata 缺失         | position verifier            | 短上下文通过, 长上下文漂移             |
| lm head tied weight 未声明 alias    | alias verifier               | memory planner 误释放或重复加载    |
| config max context 大于 KV plan 容量 | admission/runtime verifier   | decode 中途越界或拒绝太晚           |

这些负例训练的是“错误停在哪一层”。如果 QKV 顺序错被 attention kernel 才发现，说明 schema 解析太弱；如果 vocab mismatch 到 sampler 才发现，说明 tokenizer 合同没有前置；如果 q4 scale 缺失到 AVX2 kernel 才崩，说明 quant verifier 没有完成字节账本。

导入报告和回归证据

模型导入报告不需要夸大支持所有模型，但必须诚实且可复现：

Listing 3.19 真实模型导入报告目录

```
model-import-report/
 asset-summary.txt
 commands.txt
 environment.txt
 manifest.json
 tensor-index.csv
 verifier-report.txt
 tokenizer-golden.txt
 graph-ir.txt
 first-token-trace.csv
 logits-checksum.txt
 unsupported-features.txt
```

**unsupported-features.txt** 很重要。若当前只支持 fp16 safetensors，不支持 GGUF q4；只支持独立 Q/K/V，不支持 packed QKV；只支持基础 RoPE，不支持某种 scaling；都应该明确写出来。生产级工程不是永远支持一切，而是知道自己支持什么、拒绝什么、如何诊断。

### 真实 LLM 工业链条：从资产到第一条 token trace

第三册不能只停在“manifest 应该有什么”。真实 LLM 工业链条至少跨过 tokenizer、权重格式、metadata、mmap、shape verifier、graph import、memory plan、backend plan、KV cache 和 sampler。任何一环靠猜，后面的 kernel 再快也只是把错误更快地扩散。

| 链条阶段              | 必须建立的事实                                      | 典型工业格式或系统                                   |
|-------------------|----------------------------------------------|---------------------------------------------|
| asset discovery   | 文件集合、分片、checksum、版本                          | safetensors index、GGUF metadata             |
| tokenizer import  | vocab、normalizer、special token、byte fallback | tokenizer.json、SentencePiece、GGUF tokenizer |
| tensor index      | name、offset、bytes、dtype、shape、checksum       | safetensors header、GGUF tensor table        |
| schema resolution | 外部名字映射到 canonical role                       | Llama/Mistral/Qwen 等方言                      |
| quant descriptor  | group size、scale、zero point、packing order    | GGUF q4/q5/q8、GPTQ/AWQ 产物                   |
| typed graph       | 每个 op 的 shape/dtype/layout/state edge        | 本书 Graph IR、ONNX/MLIR 类系统                   |
| memory plan       | activation、workspace、KV、packed cache         | arena、mmap、device buffer                    |
| backend plan      | CPU/GPU kernel、ISA、fallback、packing          | ggml/llama.cpp、oneDNN、TensorRT-LLM          |
| first token trace | 每层 checkpoint、logits、top-k、sampler           | oracle CSV、profiler event                   |

这张表给了工程边界。safetensors 的优势是头部记录 tensor 名、dtype、shape 和 data offsets，适合安全地 mmap 和校验范围；GGUF 的优势是把模型 metadata、tokenizer、量化格式和 tensor table 放在一个文件里，适合本地推理分发。无论选择哪一种，本引擎内部都不应该让后端直接依赖外部文件格式。外部格式先转换成 manifest 和 tensor descriptor，后端只消费已验证合同。

### 工业导入的负载顺序

真实导入流程应按“先轻后重”组织，避免读了几十 GB 权重后才发现 tokenizer 或 shape 不匹配。

Listing 3.20 真实模型导入的推荐顺序

1. read small metadata:  
config, tokenizer metadata, tensor index, quant metadata
2. verify global contracts:  
vocab\_size, hidden\_size, layers, heads, kv\_heads,  
head\_dim, rope config, max\_context, tied weights
3. verify tensor table:  
names, canonical roles, offsets, byte ranges,  
dtype, shape, quant scale ranges, checksums
4. build typed graph:  
op nodes, tensor descriptors, state edges,  
tokenizer contract, generation contract
5. build memory and backend plan:  
mmap views, packed weight cache, KV cache budget,

CPU/GPU kernel selection, fallback rules

```
6. run first-token trace:
 tokenizer ids, embedding, each decoder checkpoint,
 logits checksum, top-k, sampled token
```

这个顺序的关键是：metadata 和 tensor index 先闭合，权重字节后进入热路径。若 `lm_head` 和 tokenizer vocab 不一致，应在读取 tensor table 时拒绝；若 RoPE scaling metadata 缺失，应在 graph import 时拒绝；若 q4 scale 数量不对，应在 quant descriptor 验证时拒绝；不能等到 AVX2 kernel 读错 scale 后才崩。

GGUF 和 safetensors 的导入差异

两类格式的工程取舍不同。safetensors 更像“安全 tensor 容器”：它强调 header、shape、dtype 和 data offsets，不执行任意代码，适合 HuggingFace 生态权重分片。GGUF 更像“本地推理包”：它把模型 metadata、tokenizer、量化类型和 tensor table 一起打包，适合 llama.cpp/ggml 风格的本地加载。

| 维度           | safetensors                           | GGUF                                     |
|--------------|---------------------------------------|------------------------------------------|
| metadata     | 通常还要结合 config/tokenizer 文件            | 文件内包含大量模型和 tokenizer metadata            |
| tensor table | header 中有 name/shape/dtype/offset     | header 和 tensor info 记录 name/type/offset |
| 量化           | 常见 fp16/bf16 或外部量化产物                  | 内建多种 ggml quant type                     |
| tokenizer    | 通常来自 tokenizer.json 或 tokenizer.model | 可随文件存储 tokenizer 字段                      |
| mmap         | 可按 data offsets 建 view                | 可按 tensor offset 建 view，但需处理 alignment   |
| 主要风险         | 多文件版本不一致、tokenizer/config 漂移          | 量化类型解释、metadata 兼容、版本差异                  |

本书项目可以先支持一种格式，但报告必须写明支持边界。只支持 safetensors fp16，并不丢人；声称支持 GGUF 却不解释 q4 block layout、scale 元数据和 tokenizer 字段，才是危险。工业链条的第一原则是可拒绝、可诊断、可复现。

第一条 token trace 是导入闭环的验收

导入成功不能只看“模型文件打开了”。最小验收是一条固定 prompt 的 first-token trace。它应该记录从 prompt 到 sampled token 的每个关键边界：

Listing 3.21 first-token trace 的核心字段

```
FirstTokenTrace:
 model_asset_id
 tokenizer_id
 prompt_text
 token_ids
 graph_digest
 memory_plan_digest
 backend_plan_id
 checkpoints:
 embedding_checksum
 layer0_rmsnorm_checksum
 layer0_qkv_checksum
 layer0_rope_checksum
 layer0_attention_checksum
 layer0_mlp_checksum
```

```

 final_logits_checksum
topk:
 ids
 values
sampled_token
tolerance_profile

```

这条 trace 同时服务正确性和回归定位。它能证明 tokenizer、模型 metadata、张量 shape、layout、KV state、后端计划和 sampler 确实连上了。若以后换量化格式、换 packed layout 或换 GPU backend, trace 能指出漂移从哪一层开始。

### 硬验收

读完本节, 应该能说明 AI 编译器前端在本地推理引擎中的职责:

- manifest 把模型目录、权重文件、tokenizer 和量化 metadata 转换成编译器事实。
- schema resolution 把不同模型的命名方言映射到统一算子角色。
- shape verifier 用 config 和算子合同推导每条边的形状, 不让后端猜。
- dtype 和 quant verifier 区分逻辑 dtype、存储 dtype 和计算 dtype。
- tokenizer 合同必须和 embedding、generation config、max context 一起检查。
- 好诊断应该指出 tensor、期望、实际、推导来源和可能错因。
- 真实模型导入要产出 manifest、tensor index、tokenizer golden、typed graph 和 first-token trace。
- mmap 权重视图、packed weight cache 和 device buffer 要有明确所有权状态。
- 负例包要证明错误停在前端, 而不是落到后端 kernel 或 sampler 才暴露。
- 真实 LLM 工业链条要从外部资产导入到 first-token trace, 而不是只加载一个二进制权重数组。

下一步进入 runtime 和 memory planner 时, 这些前端产物会成为所有优化的前提: 只有 typed graph 的边界稳定, 内存生命周期、kernel selection 和 CPU/GPU 后端计划才有可信输入。

### 真实模型加载闭环: Tokenizer、权重格式与首 token trace

#### 3.2 为什么真实模型加载是第三册的分水岭

第三册已经有 tensor、KV cache、量化、IR、runtime 和 backend 账本。但如果模型资产仍然停留在手写 manifest 或 tiny fixture, 读者还没有真正进入 AI Infra。真实推理系统的第一道工业边界, 是把磁盘上的 tokenizer、config、权重分片、量化 metadata 和 chat template 变成一个可验证的 executable plan。任何一个字段错了, 后面的 kernel 再快也只是更快地产生错误 logits。

本章目标是建立“真实模型加载闭环”的验收标准:

Listing 3.22 真实模型加载闭环

```

model directory or package
-> tokenizer contract
-> config and architecture contract
-> tensor table and weight shards
-> quant descriptor
-> name mapping and shape verifier
-> typed graph
-> memory map or staged load
-> first token trace
-> logits oracle

```

完成这条链路以后, 推理系统才从原理说明进入可训练、可复查的工程闭环。

### 3.3 Tokenizer 是模型合同的一部分

Tokenizer 不是服务边缘的小工具。它决定 token id、special token、BOS/EOS、padding、chat template、位置编号和 embedding 行索引。tokenizer 版本变化，可能让同一个 prompt 变成不同 token 序列；chat template 变化，可能让角色、换行和特殊标记不同；special token id 错，可能直接访问错误 embedding 行。

| 字段               | 必须验证什么                 | 错误后果           |
|------------------|------------------------|----------------|
| vocab size       | embedding/lm head 行数匹配 | 越界或 logits 维度错 |
| token id mapping | special token id 稳定    | BOS/EOS/UNK 错误 |
| normalizer       | 文本归一化规则                | 同一文本 token 不同  |
| pre-tokenizer    | 空格、字节、unicode 处理       | prompt 重放失败    |
| chat template    | role 到文本的展开规则          | 对话语义漂移         |
| padding side     | position id 和 mask     | RoPE/KV 位置错    |

报告不能只写“加载 tokenizer 成功”。它要固定一个 prompt fixture，输出 token ids、special token 位置、chat template 展开文本 hash 和 tokenizer 配置 hash。若 tokenizer.json、SentencePiece model 或 GGUF 内嵌 tokenizer 互相冲突，verifier 必须拒绝或明确选择优先级。

#### 当前 lab 的 tokenizer 合同

当前 `books/cpu-volume-3/labs/linux_cpu_inference` 已经把 tokenizer 从“正文要求”推进到可运行 fixture。它不是完整 BPE 或 SentencePiece 实现，而是一个最小合同：BOS/EOS、UNK 策略、chat template hash、词表、prompt fixture 和合同 hash。

**Listing 3.23** tiny tokenizer fixture 的核心字段

```
LCQI_TOKENIZER_V1
bos_id 0
eos_id 4
unk_id -1
chat_template_hash tiny-chat-template-v1
vocab_size 5
vocab:
 0 <bos>
 1 tok1
 2 tok2
 3 tok3
 4 <eos>
```

运行 `lcqi_trace` 时，trace 会先输出完整 tokenizer 结果，再把 decoder 输入单独传给 reference decoder：

**Listing 3.24** tokenizer trace 的最小硬证据

```
tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4
tokenizer_contract,0,-1,scalar,scalar,u64,fnv1a,13452902845388333734,0,tiny-↵
↵ chat-template-v1
```

这个设计故意把两层分开。Tokenizer 合同包含 BOS/EOS、template 和词表身份；decoder 合同只接收本次 tiny 模型需要的正文 token。真实 LLaMA/Qwen 类模型也要做同样分层：chat template、special token、padding、position id 和 decoder input ids 都要在 trace 中可见。否则换了 tokenizer 或 template 后，后端 kernel 仍然能运行，却已经不再处理同一个请求。

下一步从 fixture 走向真实格式时，不应删除这个合同，而是扩展它：

**Listing 3.25** 从 fixture 扩展到真实 tokenizer 的门禁

```
tokenizer_gate:
 tokenizer_source: tokenizer.json / sentencepiece.model / gguf metadata
 version_or_hash:
 normalizer:
 pre_tokenizer:
 special_tokens:
 chat_template_hash:
 prompt_fixture_ids:
 detokenize_roundtrip:
 embedding_vocab_match:
```

完整 BPE/SentencePiece 的复杂度在分词算法本身；工程风险在合同漂移。实验代码先固定合同，是为了让后续替换真实 tokenizer 时有可回归的出口。

### 3.4 GGUF、safetensors 和分片权重

GGUF 更像本地推理包：metadata、tokenizer、tensor table 和量化类型常放在同一文件中。safetensors 更像安全 tensor 容器：header 描述 dtype、shape 和 data offsets，权重可能分片，config 和 tokenizer 通常在旁边文件中。两者都不是“读一堆 float”这么简单。

Listing 3.26 模型资产清单

```
model_asset:
 format: gguf / safetensors / custom_fixture
 architecture: llama_like / qwen_like / other
 config:
 layers
 hidden
 intermediate
 attention_heads
 kv_heads
 rope_theta
 vocab_size
 tokenizer:
 source
 vocab_hash
 chat_template_hash
 tensors:
 name
 dtype
 shape
 data_offset
 shard_id
 quant_type
 checksum
```

真实加载器最容易错的是名字映射。不同生态对同一权重可能命名不同：`layers.0.attention.wq.weight`、`model.layers.0.self_attn.q_proj.weight`、`blk.0.attn_q.weight`。加载器不能靠字符串猜完就算成功；它要把每个外部 tensor 映射到内部 graph value，并记录 unmapped、duplicate、shape mismatch、dtype mismatch 和 transpose requirement。



### 当前 lab 的 safetensors manifest gate

当前 lab 已经补了一个很小的 safetensors header 解析器。它不声称能加载完整 LLaMA/Qwen 权重,也不处理所有 JSON 细节;它只把真实模型格式最关键的目录事实落到可测试代码:header 长度、`__metadata__`、tensor name、dtype、shape、`data_offsets`、payload 边界,以及 dtype/shape 推导出的字节数是否等于 offset 区间。

Listing 3.27 safetensors manifest gate 的最小证据

```
SafeTensorManifest:
 header_size
 data_start_offset
 metadata:
 format -> lcqi-fixture
 tensors:
 model.embed_tokens.weight:
 dtype = F32
 shape = [2, 3]
 data_offsets = [0, 24]
 model.layers.0.attn.q_proj.weight:
 dtype = F16
 shape = [4, 3]
 data_offsets = [24, 48]
```

测试会运行时生成一个合法 fixture,并验证两个 tensor 的名称、dtype、shape 和 offset;还会生成 `data_offsets` 超出 payload、dtype/shape 字节数不匹配的坏文件,要求加载器拒绝。这个 gate 的意义是把“权重格式导入”从口号压到第一层工程边界:任何后续 name mapping、shape verifier、mmap 或分片加载,都必须先建立可信 tensor table。若连 offset 范围和字节数都不检查,后端 kernel 读到的只是任意字节流。

下一阶段应在这个 gate 上继续扩展,而不是绕开它:

Listing 3.28 从 safetensors header 到真实加载器的下一层

```
safetensors_manifest
-> shard index and tensor table merge
-> external name to canonical role mapping
-> config-derived expected shape
-> dtype and quant descriptor verification
-> mmap/read policy
-> first token trace
```

这样写能保持边界诚实:当前 lab 已经验证真实格式的目录合同,但还没有完成完整权重解释和首 token 真实模型 oracle。

### 3.5 shape verifier 要从 config 推导,而不是照抄文件

验证器不能只检查“文件里写了 shape”。它要从模型 config 推导每个 tensor 的期望 shape,再和文件形状比对。例如 decoder-only GQA 模型中:

Listing 3.29 LLaMA-like shape 推导

```
hidden = H
intermediate = I
layers = L
query_heads = QH
```

```

kv_heads = KVH
head_dim = H / QH

tok_embeddings.weight: [vocab, H]
layers.i.attn.wq.weight: [QH * head_dim, H]
layers.i.attn.wk.weight: [KVH * head_dim, H]
layers.i.attn.wv.weight: [KVH * head_dim, H]
layers.i.attn.wo.weight: [H, H]
layers.i.ffn.w1.weight: [I, H]
layers.i.ffn.w2.weight: [H, I]
layers.i.ffn.w3.weight: [I, H]
output.weight: [vocab, H]

```

如果  $H\%QH \neq 0$ , `head_dim` 不合法; 如果 `KVH` 不能整除或被 `QH` 映射, GQA 关系不合法; 如果 `vocab` 与 `tokenizer` 不一致, `embedding` 和 `logits` 语义不合法。verifier 应输出诊断, 而不是让后端 kernel 在运行时越界。

### 3.6 mmap、lazy load 和 prepare 的边界

真实模型通常很大。加载策略至少有三类: 一次性读入内存, mmap 文件并按需触页, mmap 后 prepare 阶段主动触碰和打包。mmap 成功不代表权重已经进入内存; 第一次 decode 若触发大量缺页, 会把 cold fault 混进 TPOT。prepare 阶段若做 packed weight、NUMA first-touch、checksum 和量化 metadata 解析, 也必须和 request latency 分开报告。

Listing 3.30 加载阶段边界

```

OpenPackage
-> ParseMetadata
-> VerifyTokenizerAndConfig
-> BuildTensorTable
-> MapOrReadWeights
-> VerifyShapesAndDTypes
-> BuildTypedGraph
-> PreparePackedWeights
-> WarmPagesIfRequired
-> ReadyForPrefill

```

服务报告要区分 load time、prepare time、first prefill time、steady decode time。若把 prepare 放进第一个请求, TTFT 会很高; 若把 prepare 从报告中删掉, 系统容量和冷启动成本又被隐藏。正确做法是两个数字都保留。

### 3.7 首 token trace: 加载闭环的最终证据

真实加载闭环的第一份硬证据, 不是回答自然语言问题, 而是固定 prompt 的首 token trace。它应该记录每层关键中间值的 shape、dtype、layout、checksum 或小片段, 以及最终 logits/top-k。trace 不需要保存巨大 tensor, 但必须足以定位第一个偏离点。

Listing 3.31 first token trace 字段

```

first_token_trace:
 model_id
 asset_hashes
 tokenizer_hash
 config_hash

```

```

prompt_text_hash
token_ids
backend_plan
for each layer:
 rmsnorm_checksum
 qkv_shape_dtype_layout
 rope_position
 kv_append_slot
 attention_score_summary
 mlp_checksum
logits:
 shape
 dtype
 checksum
 top_k
sampler:
 temperature
 seed
 selected_token

```

如果首 token trace 能稳定复现，后续优化才有基准。换 tokenizer、换权重格式、换量化 profile、换 backend，都应先对齐这个 trace。否则“生成结果看起来差不多”无法支撑工程判断。

### 3.8 错误案例库

| 错误                              | 现象                     | 应有诊断                       |
|---------------------------------|------------------------|----------------------------|
| tokenizer vocab 与 embedding 不一致 | 访问越界或 logits 维度错       | 报告两者来源和实际值                 |
| chat template 缺失                | 对话 prompt 语义不同         | 报告模板 hash 和展开文本 hash       |
| Q/K/V shape 映射错                 | shape 可能仍能乘，但 head 语义错 | 指出 layer、tensor、期望 shape   |
| RoPE theta 缺失或默认错               | 短 prompt 近似，长上下文漂移     | 报告 config 来源和默认策略          |
| weight transpose 方向错            | logits 大幅偏离            | name mapping 中记录 layout 转换 |
| 量化 group size 错                 | 输出可运行但误差大              | QuantDesc scale count 失败   |
| mmap 冷触页混入请求                    | 首次 TTFT 异常高            | load/prepare/request 分段报告  |

### 3.9 验收标准

第三册的真实模型加载闭环至少要交付：

1. tokenizer fixture：文本、展开文本 hash、token ids、special token 位置。
2. asset manifest：config、tensor table、dtype、shape、offset、checksum。
3. name mapping report：外部 tensor 到内部 graph value 的映射。
4. shape verifier：正例通过，坏例准确拒绝。
5. load/prepare/request 分段时间。
6. first token trace：中间 checkpoint、logits top-k 和 sampler 状态。
7. 回归门禁：tokenizer、config、权重、量化和 backend 任一变化都触发 trace 对比。

#### 核心思想

真实模型加载不是附属功能，而是 AI Infra 的入口合同。没有 tokenizer、权重格式、shape verifier 和首 token trace，后端优化没有可信基线。

### 3.10 推理运行时与内存规划：typed graph 如何服务一次真实请求

## 先从一个会抖动的请求开始

模型导入和 verifier 把外部资产固定成 typed graph；AI 编译器把 typed graph 降成 executable plan。现在看计划真正落到一次请求时会遇到什么问题。假设用户输入 512 个 token 的 prompt，希望继续生成 128 个 token，本地引擎已经通过 verifier，权重也能被 kernel 读取。一个最朴素的 runtime 可能这样写：

Listing 3.32 朴素 runtime 的热路径

```
tokens = tokenizer.encode(prompt)

for op in graph:
 input_tensors = lookup_inputs(op)
 output_tensor = allocate(op.output_shape, op.output_dtype)
 run_kernel(op, input_tensors, output_tensor)

for step in 0..max_new_tokens:
 for op in graph:
 input_tensors = lookup_inputs(op)
 output_tensor = allocate(op.output_shape, op.output_dtype)
 run_kernel(op, input_tensors, output_tensor)
 next_token = sample(logits)
```

这段代码很容易在小模型上“跑通”，也很容易在 7B 级模型上暴露几个实际问题。第一，每个算子动态分配输出，会把 allocator、缺页、清零和碎片化放进热路径；decode 每生成一个 token 都重复这些成本，尾延迟会抖。第二，每次遍历 graph 时还在解释 op、查 dtype、查 layout、判断 shape、分支和间接调用压进最热循环。第三，所有 activation 都当作独立对象存活，内存峰值会被中间张量堆起来。第四，KV cache 是跨 token 状态，却被朴素代码混在普通临时张量里，容易出现覆盖、重复分配或错误增长。

真实推理运行时要做的不是“按图执行一遍”，而是把第 6、7 章的 typed graph、shape 合同、layout 合同和后端计划绑定到一个 session 状态上。编译期已经能决定的事情，不能留到每个 token 再判断；每个 token 才知道的事情，也不能假装编译期已经固定。

### 核心思想

推理运行时的核心任务，是把编译器产出的 executable plan 绑定到一次会增长、会采样、会持有 KV cache 的真实请求。内存规划不是运行时小优化，而是让计划稳定执行的前提。

### runtime state: 请求不是一串独立 kernel

一次本地对话请求至少包含下面这些运行期状态：

- session：模型实例、后端实例、编译计划、线程池、设备句柄和错误报告上下文。
- token buffer：prompt token、已生成 token、当前位置、最大上下文和 stop 条件。
- KV cache：每层已经写入的 K/V、当前可读长度、容量、布局和可能的 page table。
- activation arena：prefill 或 decode 当前阶段可复用的临时张量区域。
- workspace：某些后端 kernel 需要的临时区，例如 attention scratch、GEMM packing scratch 或 GPU workspace。
- sampler state：temperature、top-k、top-p、seed、重复惩罚和 logits 后处理状态。
- profiler/oracle：记录事件、内存峰值、kernel 耗时和可选 reference 对比。

这些状态不属于同一种生命周期。权重从模型加载活到模型卸载；packed weight 从后端准备完成活到后端销毁；activation arena 通常只活在一个 plan invocation 内；KV cache 活在整个 session 内，并且随 token 增长；sampler state 随生成循环推进；profiler 事件则是运行期证据。把它们全部塞进一个“tensor map”会让内存所有权模糊，后续优化也没有安全边界。

Listing 3.33 推理 runtime 的核心对象关系

```

Engine
 ModelManifest
 TypedGraph
 ExecutablePlans
 BackendContext

Session
 TokenBuffer
 KVCache
 ActivationArena
 WorkspacePool
 SamplerState
 ProfilerTrace

```

注意这里的分层。Engine 持有模型级事实和编译计划，多个 session 可以共享它；Session 持有请求级状态，不能被另一个请求随意复用。CPU 后端的 packed weight、线程池和 NUMA 放置属于 Engine/BackendContext；某个用户这次对话的 KV cache 和 token buffer 属于 Session。这个边界一旦错了，多请求并发时会出现更隐蔽的问题：一个 session 覆盖另一个 session 的 cache，或者一个用户的 profiler/oracle 开关拖慢所有用户的热路径。

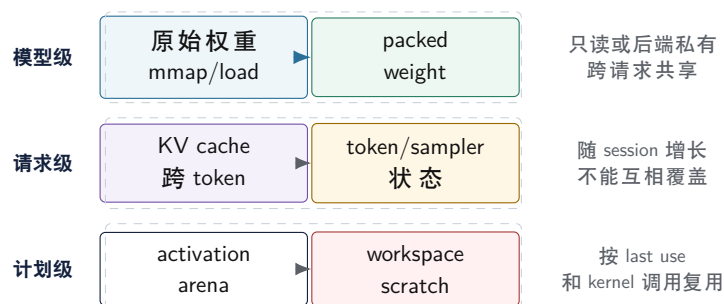
### 内存分类：先分清谁能复用

内存 planner 的第一步不是写分配器，而是给所有数据分类。下面这张表比“都叫 tensor”更接近真实运行时：

| 类别                      | 生命周期                               | 典型位置                                    | 复用规则                                   |
|-------------------------|------------------------------------|-----------------------------------------|----------------------------------------|
| 原始权重<br>packed weight   | 模型加载到卸载<br>后端准备到卸载                 | mmap 文件或主内存<br>CPU 对齐内存或 GPU 显存         | 只读，不能当 scratch<br>后端私有布局，按 kernel 合同读取 |
| activation<br>workspace | 一段执行计划内部<br>单个 kernel 或 segment 内部 | arena 或 device buffer<br>workspace pool | last use 后可复用<br>按算法临时借用，调用后归还         |
| KV cache                | session 生命周期                       | 主内存、显存或分层缓存                             | 追加写入，历史可读，<br>不能被普通 arena 覆盖           |
| logits                  | 每步采样前后                             | 小 buffer 或 device-to-host staging       | 可能需要保留 top-k<br>或完整分布                  |

这张表背后是编译器理论里的 liveness analysis。一个 activation 的定义点是某个算子的输出，最后一次使用点是最后一个消费者；最后一次使用之后，它占用的内存槽可以给别的 activation。原始权重和 KV cache 不满足这个条件：权重会被每层、每步反复读取；KV cache 的旧 position 虽然不会再被写入，却会在后续 attention 中反复读取。因此它们不能进入普通 activation arena。

## 推理内存的生命周期分类



图示：本图要证明的命题是：推理内存不能按“都是张量”统一处理，模型级、请求级和计划级生命周期必须分开。

底层机器后果也直接来自这张分类。packed weight 需要对齐到 cache line、页或 GPU backend 要求的边界；activation arena 希望少量大块内存重复使用，减少 allocator 和 TLB 抖动；KV cache 希望读流连续，避免 decode attention 每步跨太多页面；workspace pool 要能处理不同算法的最大临时需求，而不是每次 kernel 调用分配释放。

## activation arena：把生命周期分析落到地址

activation arena 的目标很明确：预先申请一块或少数几块大内存，让所有短生命周期 activation 在里面复用。它类似传统编译器里的寄存器分配和栈槽复用，只是单位从标量变量变成了大张量。

planner 至少要知道每个中间值的这些字段：

Listing 3.34 arena planner 使用的 value descriptor

```
ValueDesc:
 id
 shape
 dtype
 layout
 byte_size
 alignment
 defined_at
 last_used_at
 materialization_required
 backend_location
```

`defined_at` 和 `last_used_at` 来自 typed graph 或 lowered plan 的拓扑顺序。若一个值被 residual 分支、oracle dump 或多个后续算子消费，它的 `last_used_at` 必须延长。若某个 debug 模式要求保存层输出用于 reference 比较，`materialization_required` 会阻止 planner 提前覆盖它。若某个值在 GPU device 上，不能被 CPU arena 的地址复用。

一个简化的 arena planner 可以用线性扫描实现：

Listing 3.35 线性扫描式 activation arena 规划

```
events = values sorted by defined_at
free_list = []
active = []

for v in events:
 expire all active values with last_used_at < v.defined_at
 if v is stateful or materialization escapes plan:
```



```

 allocate dedicated storage or mark external
 continue
 block = find_free_block(free_list, v.byte_size, v.alignment)
 if no block:
 block = grow_arena(v.byte_size, v.alignment)
 assign v.offset = block.offset
 add v to active

```

工业实现会更复杂：要处理 in-place 更新、别名、跨 stream 同步、不同 device、best-fit/first-fit 策略、碎片回收和峰值优化。但最小原则先抓住生命周期：只要 typed graph 告诉我们每个值何时产生、何时最后使用，就可以把多个不重叠生命周期的 activation 放在同一段物理内存上。

**对齐不是语义细节。** CPU kernel 往往希望输入和输出至少按 32 或 64 字节对齐，便于向量 load/store 和 cache line 行为稳定。GPU 后端可能要求更大的 alignment 或特定 pitch。arena planner 若只按 byte size 紧密拼接，最后会把复杂的 unaligned 处理推给每个 kernel。

**清零不是免费操作。** 有些临时 buffer 不需要初始化，因为 kernel 会完整写入；有些 accumulator 或 mask buffer 必须清零。planner 应该把 zero-fill 作为 buffer 属性，而不是让每个算子默认 `memset`。否则一个看起来很小的 scratch 可能在长 prompt prefill 时变成可观带宽成本。

**debug/oracle 会改变生命周期。** 打开 oracle 对比时，某些中间结果需要物化并保留到 reference 比较之后。生产模式下能复用的 arena slot，在验证模式下可能不能复用。这不是“debug 模式慢一点”这么简单，而是 planner 必须显式支持不同 materialization policy。

### 常见误区

不要让 kernel 自己偷偷缓存输入指针。arena planner 会复用地址，如果某个异步 kernel 或 debug hook 在生命周期结束后仍持有旧指针，就会读到另一个张量的数据。异步后端必须把 stream/event 同步也纳入生命周期边界。

### 一个小图的 liveness 手算例子

内存规划不是抽象名词。看一个最小 residual block：

Listing 3.36 最小 residual block 的值流

```

v0 = input_hidden
v1 = rmsnorm(v0)
v2 = linear_qkv(v1)
v3 = attention(v2, kv_cache)
v4 = linear_wo(v3)
v5 = add(v0, v4)
v6 = rmsnorm(v5)
v7 = mlp(v6)
v8 = add(v5, v7)

```

若只看最后输出，`v1/v2/v3/v4/v6/v7` 都是短生命周期 activation；`v0` 必须活到第一次 residual；`v5` 必须活到第二次 residual；`kv_cache` 是外部状态，不进入 arena。可以把生命周期写成：

| 值               | 定义点       | 最后使用                     | 规划含义              |
|-----------------|-----------|--------------------------|-------------------|
| <code>v0</code> | 输入        | <code>add(v0, v4)</code> | 不能在 attention 前覆盖 |
| <code>v1</code> | RMSNorm   | <code>linear_qkv</code>  | QKV 后可复用          |
| <code>v2</code> | QKV       | <code>attention</code>   | attention 后可复用    |
| <code>v3</code> | attention | <code>linear_wo</code>   | Wo 后可复用           |
| <code>v5</code> | residual  | <code>add(v5, v7)</code> | MLP 期间必须保留        |
| <code>v7</code> | MLP       | <code>add(v5, v7)</code> | 输出后可复用            |

于是 planner 可以让 **v1** 和 **v3** 复用同一块 arena，让 **v2** 和 **v6** 复用另一块，只要它们的 shape/alignment 兼容。这个手算例子也解释了为什么 oracle/debug 会改变峰值内存：若要求保存 **v2** 的 Q/K/V 检查点，它的 last use 就不再是 attention，而是 oracle compare。

**Listing 3.37** debug policy 对生命周期的影响

```
production:
 v2 last_use = attention

oracle mode:
 v2 last_use = compare_checkpoint_after_layer
 materialization_required = true
```

所以运行时不能把“打开 oracle”当成单纯多打印日志。它会改变 materialization policy、arena 峰值和调度同步点。

### packed weight: 把文件布局变成内核布局

权重是只读的，但这不代表它可以直接以文件布局进入热路径。模型文件常按通用格式存储：张量名、shape、dtype、连续字节。高性能 kernel 需要的却是物理布局：按 tile 重排、按 SIMD 宽度分组、把 scale 和 weight 放在合适距离、补齐尾部、甚至为不同后端生成不同 packed format。

**Listing 3.38** 从 manifest 到 packed weight 的准备过程

```
for each linear weight tensor:
 verify logical shape and quantization metadata
 choose backend packing format
 allocate aligned packed buffer
 reorder or decode metadata into kernel-friendly panels
 record mapping: logical tensor -> packed descriptor
 keep checksum and source offset for diagnostics
```

packed descriptor 不是简单指针。它至少要告诉 kernel：每个 panel 的行列范围、stride、group scale 位置、tail padding、原始 dtype、计算 dtype、对齐和是否允许并行读取。对 int4/group quant 权重来说，scale 与 packed nibble 的关系尤其关键：如果 packing 只重排了 4-bit 数据，却没有同步重排 scale，kernel 会用错量化比例，输出仍然可能看起来“数值正常”。

这一步和传统编译器的后端 lowering 很像。高层 IR 中的 `linear(h, Wq)` 是语义；packed weight 是目标机器上的物理表示。语义值相同，不代表物理布局相同。CPU AVX2、AVX-512 和 GPU tensor core 可能需要完全不同的 packed descriptor；runtime 选择 plan 时必须同时选择对应的 packed weight。

### KV cache planner: 跨 token 状态不能当 scratch

KV cache 是 decoder-only 推理的核心状态。prefill 会批量写入 prompt 中所有 position 的 K/V；decode 每步追加一个 position，并让 attention 读取从开头到当前位置的历史 K/V。它的生命周期跨越整个 session，不能像 activation 一样在 last use 后复用，因为“last use”会随着未来 token 增长而不断后移。

**Listing 3.39** KV cache 的逻辑合同

```
append(layer, batch, kv_head, position, k_vector, v_vector)
read(layer, batch, kv_head, range[0..position])

invariants:
 position is monotonically increasing inside one session
 every readable position has been written exactly once
```

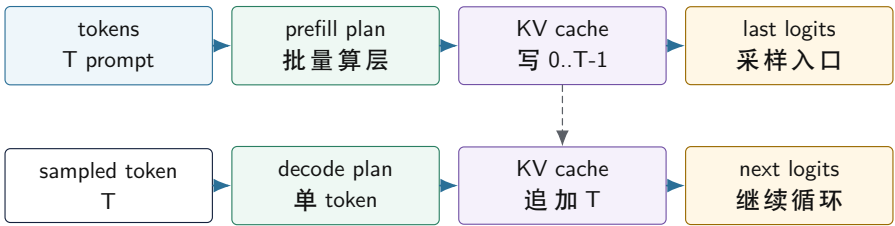
layout maps logical (layer, head, position, dim) deterministically  
cache capacity is checked before write

planner 需要先回答容量问题。若直接按最大上下文一次性分配，地址简单，性能稳定，但峰值内存高。若按 page 或 block 增长，短对话更省内存，但 attention kernel 要通过 page table 读取，读流可能变碎。若使用 sliding window 或 KV eviction，必须把语义写进模型合同：并不是所有模型都允许丢弃远处 token；即使允许，position encoding 和 attention mask 也要同步处理。

| 策略                        | 优点                            | 风险                                      |
|---------------------------|-------------------------------|-----------------------------------------|
| 固定最大容量<br>按 block/page 增长 | 地址简单，kernel 容易优化<br>多请求场景更省内存 | 空间峰值高，短请求浪费大<br>page table、碎片和跨页读取增加复杂度 |
| 滑动窗口                      | 长对话空间可控                       | 改变可见上下文，必须匹配模型语义                        |
| KV 量化                     | 降低 cache 带宽和容量                | attention 精度、scale 存储和反量化成本要验证          |

KV cache 的物理 layout 要围绕后端访问方式设计。CPU decode 常常让一个线程或一组线程处理部分 head/output 行，连续读取某个 head 的历史 position；此时 position 和 head\_dim 的局部连续性很重要。GPU kernel 可能按 block/warp 分摊 position 或 head，需要 coalesced load。两者可以使用不同物理 layout，但必须由同一个 logical descriptor 描述，不能让业务层知道后端私有排列。

prefill/decode 与 KV cache 的状态迁移



last logits 经 sampler 产生下一行的 decode 输入；prefill 写入的历史在 decode 阶段继续可读，decode 每步再追加一个位置。

图示：本图要证明的命题是：prefill 和 decode 不是同一形状下的两次执行，而是围绕 KV cache 读写合同形成的两个计划。

paged KV allocator: 不是 vector push back

固定最大 KV cache 简单，但并发请求一多就浪费。paged KV cache 把每个 session 的上下文切成固定 token 数的 block，例如每 page 存 16 或 32 个 token 的所有层/head K/V。allocator 管理空闲 page，session 的 page table 把逻辑 block 映射到物理 page。

Listing 3.40 paged KV allocator 的状态

```
KvPage:
 page_id
 capacity_tokens
 owner_session
 logical_block
 epoch
 written_mask
 k_bytes
```

```

v_bytes
scale_bytes_optional

SessionPageTable:
 logical_block -> page_id
 base_position
 window_size_optional

```

append 不能只是 **push\_back**。它要先计算逻辑 block 和 block 内 offset，缺 page 时向 allocator 申请，写入 K/V 和 scale metadata，然后标记 written。读取时必须检查 page 属主、epoch 和 written bit，否则回收后的旧 page 会被当成当前上下文读取。

**Listing 3.41** paged KV append/read 的核心检查

```

append(position, k, v):
 block = position / page_tokens
 offset = position % page_tokens
 page = page_table.get_or_allocate(block)
 require page.owner_session == session_id
 require page.epoch == expected_epoch(block)
 store k/v/scale at offset
 page.written_mask.set(offset)

read(position):
 block = position / page_tokens
 offset = position % page_tokens
 page = page_table.lookup(block)
 require page.written_mask.test(offset)
 return K/V view at offset

```

这套检查在 debug/oracle 模式必须严格执行；生产模式可以把部分检查移到 page 分配边界，但不能取消语义。否则一旦出现 sliding window 回绕、请求取消、page 复用或 prefix cache 共享，错误会表现成“长上下文偶发漂移”，极难定位。

### prefill 和 decode：两套计划，两套指标

prefill 和 decode 的差异不是参数不同，而是计算形态不同。prefill 输入多个 prompt token，Q/K/V projection 和 MLP 更像 GEMM，attention 要处理较大的 token 块；decode 通常一次只处理一个新 token，线性层更像 GEMV 或 small-batch GEMM，attention 主要压力变成读取越来越长的 KV cache。一个 plan 若试图同时覆盖两者，通常会在其中一个阶段浪费机器资源。

| 维度           | prefill                       | decode                       |
|--------------|-------------------------------|------------------------------|
| 输入形状         | 多 token prompt                | 通常 1 个新 token                |
| 主要矩阵形态       | GEMM / batched GEMM           | GEMV / small-batch GEMM      |
| attention 压力 | 块内计算和写 cache                  | 读历史 KV，随 context 增长          |
| 关键指标         | time to first token、prompt 吞吐 | tokens/s、单 token latency、尾延迟 |
| 内存行为         | activation 大，arena 峰值高        | activation 小，KV cache 带宽关键   |

runtime scheduler 需要把请求分成两个阶段，并在每个阶段选择对应 executable plan：

**Listing 3.42** prefill/decode 调度主循环

```

plan_prefill = select_plan(stage=prefill, tokens=T, backend)
bind(plan_prefill, session.kv_cache, session.arena)
logits = run(plan_prefill, prompt_tokens)

while not stop:
 next_id = sample(logits, sampler_state)
 session.tokens.append(next_id)
 plan_decode = select_plan(stage=decode, context=session.position, backend)
 bind(plan_decode, session.kv_cache, session.arena)
 logits = run(plan_decode, next_id)

```

这里有两个容易忽略的细节。第一，`select_plan` 不应该在热路径里重新做 shape inference、fusion 和 packing；它只是在已经编译好的候选计划里选一个。第二，decode plan 可能按 context 长度分档，例如短上下文使用一个简单 kernel，长上下文使用分块 attention 或不同线程划分。分档选择要足够便宜，否则优化收益会被 dispatch 开销吃掉。

**单请求调度和多请求调度不同。** 教材前半部分可以先把一个 session 跑稳；服务端批量推理还要处理多个 session 的 prefill/decode 交错、batching、优先级、取消、超时和 backpressure。无论是否做连续批处理，底层不变量相同：每个 session 的 KV cache 和 token position 不能混，跨请求共享的只有模型级计划和权重。

**线程池不是越满越好。** CPU decode 的单 token latency 常被小 kernel、同步和内存带宽限制。盲目让每个小算子都抢满线程池，可能增加 barrier 和 cache 竞争。scheduler 应该知道哪些 segment 值得并行，哪些 segment 用单线程或少量线程更稳。

**GPU launch 也是成本。** GPU 对大 prefill 很有吸引力，但 decode 的单 token 阶段可能被 kernel launch、host-device 同步和 logits 取回成本影响。runtime 不能只看峰值 FLOPS，必须记录每个阶段的真实事件时间。

### 调度边界：segment 比单算子更适合热路径

如果 runtime 仍然按单算子逐个 dispatch，就算内存已经规划好，也会留下大量固定开销：查表、虚调用、线程池 barrier、设备同步和 profiler 事件过细。编译器可以把多个已经确定顺序和 buffer 绑定的节点合成 segment。segment 不是融合成一个 kernel，而是一个更粗的调度单元。

Listing 3.43 executable plan 中的 segment 草图

```

Segment layer_0_attention_prefill:
 inputs: hidden, position, kv_cache
 arena_bindings:
 norm_out -> arena + 0x000000
 q_buffer -> arena + 0x410000
 k_buffer -> arena + 0x820000
 v_buffer -> arena + 0xC30000
 kernels:
 rmsnorm_linear_qkv_fused
 rope_and_kv_append
 attention_prefill
 linear_wo
 sync:
 none on CPU
 stream event before host logits read on GPU

```

segment 的价值是把“计划已经决定”这件事落到数据结构里。runtime 执行 segment 时，不再解析 graph，也不重新分配 activation；它只绑定 session 中的外部状态，按预定 kernel 序列推进。

profiler 也可以按 segment 汇总，再在需要时打开 kernel 级事件。这样既保留调试能力，又不让热路径永远停留在解释器形态。

这 and 传统编译器里的基本块、trace 或机器指令调度有相似之处。单条 IR 指令适合分析，最终执行时却希望形成更稳定的机器代码或调度块。AI 推理计划也一样：Graph IR 适合表达语义，segment 适合表达运行期调度。

### profiler 和 oracle：运行时要留下证据

没有 profiler，内存 planner 的改动很容易变成主观优化；没有 oracle，融合、量化和 layout 改动很容易悄悄改变 logits。运行时应该把证据作为一等输出，而不是出了问题才临时加日志。

一个最低可用的 profiler 事件可以这样设计：

Listing 3.44 推理 profiler 事件字段

```
event:
 session_id
 stage: load | prefill | decode | sample
 segment_id
 layer
 backend
 start_time_ns
 end_time_ns
 input_tokens
 context_length
 arena_peak_bytes
 workspace_bytes
 kv_cache_bytes
 kernel_name_optional
```

这些字段能回答几个关键问题：首 token 慢在 tokenizer、prefill 还是权重 packing；decode 慢在 linear、attention 还是 sampler；上下文变长后 attention 是否按预期变慢；arena 峰值是否符合 memory plan；某个后端切换是否真的改善了当前阶段。

oracle 则回答正确性问题。生产模式不可能每步都跑 reference，但开发和回归测试应该支持可控开关：

- 单算子 oracle：比较 RMSNorm、linear、RoPE、attention、MLP 的输出误差。
- 层级 oracle：比较某一层输入/输出 hidden state，定位误差扩散起点。
- logits oracle：比较 top-k、最大绝对误差、相对误差和采样前分布变化。
- 序列 oracle：固定 seed 和 sampler 后，比较生成 token 序列是否满足合同。
- 内存 oracle：检查 arena offset 是否重叠错误、KV position 是否越界、debug materialization 是否被提前覆盖。

oracle 不是简单打印浮点数。它要知道误差合同：fp32 reference 对 fp16/bf16/int8/int4 后端的容忍度不同；不同 reduction 顺序可能不 bitwise identical；量化 KV cache 会改变 attention 输出，需要单独设误差边界。把这些合同写进测试和报告，才能让优化有可追溯的安全线。

### 硬验收

读完本节，应该能把 typed graph 到真实请求之间的运行时边界讲清楚：

- runtime state 至少包含 session、token buffer、KV cache、activation arena、workspace、sampler 和 profiler/oracle。
- 权重、packed weight、activation、workspace、KV cache 和 logits 的生命周期不同，不能放进同一种分配策略。
- activation arena 依赖 liveness analysis、last use、alignment、materialization policy 和后端位置。



- packed weight 是后端 lowering 的物理产物，必须保存 layout、scale、padding、对齐和诊断映射。
- KV cache 是跨 token 状态，容量、layout、page/block、sliding window 和量化策略都必须受模型语义约束。
- prefill 和 decode 应该使用不同 executable plan，并用不同指标评估：time to first token、prompt 吞吐、tokens/s 和尾延迟。
- segment 把图解释开销移出热路径，profiler 和 oracle 则把性能与正确性证据留在运行时。

进入具体执行阶段时，就可以用这份账本判断每个优化是否站得住：它缩短的是哪段生命周期，减少的是哪条地址流，改变的是哪个后端 layout，profiler 应该看到什么，oracle 又必须保证什么。

### 3.11 AI 编译器：从推理图到机器执行计划

#### 推理引擎里为什么会出现编译器

前一章把本地 7B 级推理引擎拆成模型加载、tokenizer、decoder graph、KV cache、CPU/GPU 后端和证据报告。这里要再向下压一层：真实引擎不能每次遇到一个算子就临时解释执行。它必须把模型结构、张量形状、量化格式、设备能力和运行时状态编成一个可执行计划。这个过程就是 AI 编译器在推理引擎中的位置。

不要把 AI 编译器理解成“把 Python 变成机器码”的窄概念。推理场景里，输入通常已经不是普通源代码，而是模型文件、配置、tokenizer 规则、权重量化元数据和一张 decoder-only 计算图。编译器要做的事情是把这张图降到后端能高效执行的 kernel 序列，同时保持 logits、KV cache 和采样状态的语义不变。

Listing 3.45 推理引擎中的 AI 编译流水线

```
model/config/tokenizer
-> graph import and shape checking
-> typed tensor IR
-> graph rewrite and operator fusion
-> memory planning and lifetime analysis
-> backend lowering
-> kernel selection and packed layout generation
-> executable plan
-> runtime dispatch for prefill/decode
```

这条流水线和传统编译器很像。模型文件相当于源程序，图导入相当于 parsing，shape/dtype/quantization 检查相当于 semantic analysis，typed tensor IR 相当于中间表示，fusion 和 layout 选择相当于优化，CPU/GPU lowering 相当于后端代码生成，executable plan 则相当于目标程序。区别在于它编译的对象不是标量控制流为主的 C++ 函数，而是张量、状态、量化参数和设备 kernel 组成的推理程序。

#### 核心思想

AI 编译器在推理引擎中的核心职责，是把稳定的模型语义编成稳定的机器执行计划。它既不是单个 kernel，也不是普通 runtime 调度器，而是连接图语义、张量布局、内存生命周期、后端能力和正确性证据的编译流水线。

#### 前端：导入图时先建立语义合同

普通编译器前端会把源代码变成 token、AST 和符号表。AI 编译器的前端面对的是模型结构。以 decoder-only Transformer 为例，前端至少要确认这些事实：

- 每层有哪些算子：RMSNorm、Q/K/V projection、RoPE、attention、Wo projection、MLP gate/up/down、残差。
- 每个张量的 dtype、shape、stride、量化参数和驻留位置。

- 每个权重是否已经按某种 group size、scale 类型和物理布局保存。
- prefill 和 decode 是否共享同一张图，还是需要两套 specialization。
- KV cache 的逻辑 shape、最大上下文、当前位置和写入规则。

这些检查不是文档整理，而是编译器不变量。若 **Wq** 的输入维度和 hidden size 对不上，后端 kernel 不应该继续猜；若某个 int4 权重缺少 group scale，量化 lowering 不能把它当 int8；若 KV cache 的 head 数和当前 layer 的 attention head 数不一致，decode 会在很晚的位置产生错误 logits。前端要么拒绝模型，要么产出满足合同的 typed tensor IR。

Listing 3.46 typed tensor IR 的最小信息

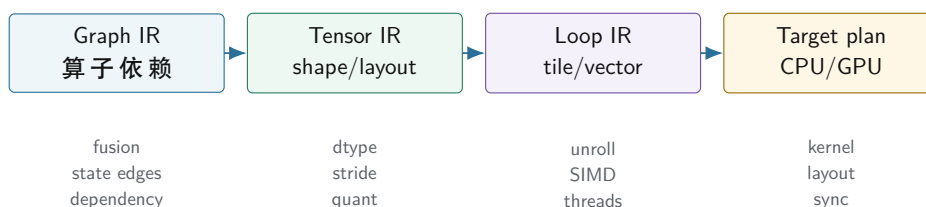
```
%0 = input_tokens : tensor<[T], i32>
%1 = embedding(%0) : tensor<[T,H], f16>
%2 = rmsnorm(%1, gamma) : tensor<[T,H], f16>
%3 = linear(%2, Wq_i4) : tensor<[T,Q], f16>
%4 = linear(%2, Wk_i4) : tensor<[T,K], f16>
%5 = linear(%2, Wv_i4) : tensor<[T,V], f16>
%6q, %6k = rope(%3, %4, pos) : tensor<[T,Q], f16>, tensor<[T,K], f16>
%7 = kv_append(%6k, %5) : state<KVCache>
%8 = attention(%6q, %7) : tensor<[T,H], f16>
```

这段 IR 不是为了好看。它让每条边都带上类型、形状和状态含义。没有这些信息，后面的融合、内存复用、layout 转换和后端选择都只能靠脆弱约定。

### IR：图 IR、张量 IR 和循环 IR 要分层

AI 编译器最容易犯的错误，是用一层表示承载所有事情。图 IR 擅长表达算子依赖：RMSNorm 后接 Q/K/V projection，RoPE 后写 KV cache，attention 后接 Wo projection。张量 IR 擅长表达 shape、stride、dtype、broadcast、reduction 和 layout。循环 IR 或 kernel IR 才适合表达 tile、向量宽度、展开、线程划分和寄存器累加器。

#### AI 编译器的三层 IR



图示：本图要证明的命题是：AI 编译器不能把算子图、张量布局 and 机器循环混成一层，否则优化会互相污染。

分层的好处是边界清楚。Graph IR 可以做无关后端的代数改写，例如把连续的 reshape 消掉，或把 RMSNorm 与后续线性层之间的临时 buffer 标成可融合候选。Tensor IR 可以决定某个 **[tokens, hidden]** 张量是否连续、某个 int4 权重是否要按 group 解码、某个 broadcast 是否会制造隐式 stride 0。Loop IR 再决定 CPU 上一个 tile 处理多少列、用几个累加器、尾部怎样 mask，或者 GPU 上一个 block 负责多少 token 和 head。

传统编译器也有类似分层。AST 不应该直接塞寄存器分配细节；SSA IR 不应该直接保存源代码排版；机器 IR 才关心调用约定和寄存器。AI 编译器同理：高层 graph 不应该知道 AVX2 的寄存器数，低层 kernel 也不应该重新推断模型层数和 KV cache 语义。

### 优化：融合不是越多越好

算子融合是 AI 编译器最常见的优化，但它经常被误讲成“把算子合在一起就快”。实际判断必须回到内存流和生命周期。若两个算子之间的中间张量很大、只被消费一次、且融合后不会增加过

多寄存器压力，那么融合可以减少一次写回和一次读取。若中间结果还被 residual、debug oracle、另一个分支或 KV cache 使用，盲目融合会破坏语义或让 buffer 生命周期变复杂。

Listing 3.47 融合决策要检查的事实

```
candidate: RMSNorm -> Linear

check:
 output of RMSNorm has exactly one consumer
 no oracle boundary requires materialized RMSNorm output
 fused kernel can stream input and gamma without spilling too much
 quantized Linear still has access to scale/group metadata
 backend supports the fused dtype/layout combination

if all true:
 lower to fused_rmsnorm_linear
else:
 keep two operators and expose the materialized buffer
```

attention 更能说明问题。prefill attention 可能是大块矩阵计算，decode attention 是一个 query 读取越来越长的 K/V cache。它们的形状不同，适合的 kernel 不同，融合边界也不同。prefill 可以倾向于高吞吐 tile；decode 更在意小 batch latency、KV cache 读流和 softmax 的数值稳定。一个“通用 attention 融合”若不区分这两种模式，很容易在其中一种场景下退化。

### 常见误区

融合优化必须由数据依赖、生命周期、数值误差和后端能力共同决定。只看算子名字相邻，就把它们写进一个大 kernel，是把编译器优化退化成手工拼接。

### 内存规划：生命周期分析是推理编译器的核心

本地推理最贵的不只是算术，还有内存容量和带宽。权重、packed weight、activation、KV cache、临时 workspace 和输出 logits 的生命周期完全不同。AI 编译器要像传统编译器做寄存器分配一样，为张量做内存分配。

Listing 3.48 张量生命周期的简化账本

```
weights:
 live from model load to session destroy
 read-only, often mmap or packed once

packed weights:
 live after backend preparation
 backend-specific physical layout

activation:
 live inside one layer or a small operator group
 can often reuse arena slots

KV cache:
 live across all decode steps
 grows by position and cannot be reused as scratch

workspace:
 live during one kernel or one scheduled segment
```

**size depends on backend algorithm**

这就是为什么内存规划不能靠 `new/delete` 随手分配。activation 可以进入 arena 并按生命周期复用；KV cache 必须按最大上下文或可增长策略预留；packed weight 可能比原始权重多占一份内存，但换来热路径布局稳定；GPU workspace 要考虑 stream 同步和设备内存峰值。若编译器没有显式生命周期，runtime 只能在执行时保守分配，结果是峰值内存上升、cache locality 变差、调试也更困难。

这一步和传统编译器的 liveness analysis 很接近。变量最后一次使用后可以释放寄存器；张量最后一次消费后可以释放 arena slot。区别在于张量尺寸大得多，KV cache 还跨 token 存活，后端 layout 可能要求额外对齐和 padding。

**后端 lowering: CPU 和 GPU 的目标不同**

同一个 Tensor IR 降到 CPU 和 GPU 时，目标函数不同。CPU 后端关心 cache line、TLB、SIMD 宽度、指令吞吐、分支、线程划分、NUMA 和 false sharing。GPU 后端关心 device memory、coalesced load、shared memory、warp/block 划分、kernel launch、stream 同步和 tensor core 格式。AI 编译器的后端 lowering 不能假装它们只是两个函数指针。

**Listing 3.49** 同一 linear 算子的两个 lowering 方向

```
linear(input[T,H], weight[0,H]):

CPU decode lowering:
 choose GEMV or small-batch GEMM
 use packed weight panels
 select SIMD width and tail handling
 partition output rows across worker threads

GPU prefill lowering:
 choose batched GEMM kernel
 ensure device-resident input and weight
 select vendor layout or internal tile layout
 schedule stream dependencies
```

这里的关键是 specialization。prefill 的 `T` 可能较大，decode 的 `T` 通常是 1；长上下文 attention 和短上下文 attention 也不该强行共享同一个计划。编译器可以为不同 shape、batch、上下文上限和后端生成多个 executable plan，再让 runtime 在请求进入时选择。这样做比每次动态判断更复杂，但能把热路径从“解释图”变成“执行计划”。

**运行时：编译计划仍然需要调度**

编译器生成计划后，runtime 仍然有工作。它要管理 session，接收 prompt，维护当前 position，选择 prefill 或 decode 计划，推进 KV cache，处理 sampler，记录 profiler 事件，并在必要时切换后端或回退到 reference。编译器负责把能提前决定的事情提前决定，runtime 负责处理请求期才知道的状态。

**Listing 3.50** 编译期和运行期的责任划分

```
compile time:
 validate model
 build typed graph
 choose layouts and fusion
 allocate static arenas
 prepare packed weights
 build backend executable plans
```

```

run time:
 tokenize prompt
 choose prefill/decode plan by shape and backend
 bind session buffers and KV position
 dispatch kernels
 sample next token
 collect correctness and performance evidence

```

这个划分也解释了为什么“推理引擎”和“AI 编译器”不是二选一。没有编译器，引擎会在每次请求中重复解释图、判断 layout、临时选择 kernel；没有 runtime，编译器生成的计划没有 session、KV cache 和 sampler 可以驱动。成熟系统通常是二者配合：前者把模型变成计划，后者把计划用在一个个真实请求上。

### runtime 调度器：执行计划怎样变成 token 流

编译器产出 executable plan 后，runtime 调度器要把它用在真实请求上。调度器至少处理五件事：请求进入、prefill 排队、decode 循环、取消/超时、资源释放。它不是简单循环调用 `plan.run()`。

Listing 3.51 runtime 调度器的最小状态

```

Scheduler:
 loaded_models
 plan_cache
 ready_prefill_queue
 active_decode_sessions
 memory_reservations
 backend_contexts
 trace_sink

```

prefill 和 decode 队列要分开。prefill 可能是长 prompt 的大块计算，decode 是很多 session 的单 token 步进。若长 prefill 一直占住所有工作线程或 GPU stream，已经在交互中的请求会断流；若 decode 过度优先，长 prompt 首 token 会等太久。调度策略要根据业务目标取舍。

| 策略         | 优点             | 风险               |
|------------|----------------|------------------|
| prefill 优先 | 首 token 更快进入模型 | 活跃会话 decode 抖动   |
| decode 优先  | 流式输出更平滑        | 新请求 TTFT 上升      |
| 分配时间片      | 两者较平衡          | 实现复杂，profile 更难读 |
| 按 deadline | 符合 SLO         | 需要更准的耗时估计        |

### 取消、超时和资源释放

业务服务必须支持取消。用户关闭连接、上游超时、内容策略中止、队列过载，都可能要求停止某个 session。取消不是只设置一个 bool。runtime 要保证：

- 不再为该 session 提交新的 decode step。
- 已提交到 GPU stream 的工作要么等待完成再释放 buffer，要么走后端支持的安全取消路径。
- KV cache page、activation arena、workspace reservation 和 trace buffer 被释放。
- sampler state 和 token buffer 不再被其他线程访问。
- FinalResponse 标出 **cancelled** 或 **timeout**，方便业务层区分。

Listing 3.52 取消路径的资源顺序

```

cancel(session):
 mark session as cancelling

```



```

stop scheduling new decode steps
wait or fence backend work using this session buffers
release KV pages and arena reservations
close stream with final cancelled event
write trace summary

```

这个顺序和第二册讲过的异步生命周期一致：释放资源之前，必须证明没有异步执行还会读写它。CPU 后端也有类似问题，只是表现为线程池任务；GPU 后端表现为 stream/event。

### plan cache 和 shape bucket

如果每个请求都重新编译 plan，热路径会很慢。runtime 通常需要 plan cache。cache key 至少包含模型、backend、量化 profile、stage、shape bucket 和 debug/oracle 配置。

**Listing 3.53** PlanCacheKey 字段

```

PlanCacheKey:
 model_id
 manifest_hash
 backend
 quant_profile_id
 stage: prefill | decode
 prompt_length_bucket
 context_length_bucket
 oracle_policy
 trace_level

```

shape bucket 是折中。完全为每个 prompt length 编译一个计划，cache 会爆；所有长度共用一个计划，kernel 可能低效。可以把 prompt length 分成 1-32、33-128、129-512、513+ 等桶，把 decode context 也按 0-512、512-2048、2048+ 分桶。每个桶对应不同 attention plan、workspace 和 profiler baseline。

debug/oracle 配置也要进入 key。因为打开 layer oracle 可能强制 materialize 某些中间张量，导致 fusion 和 memory planning 不同。不能把 debug plan 和 production plan 混用。

### 正确性：优化后的计划必须能被验证

AI 编译器的优化会改变执行顺序、buffer 物化点、累加精度、量化缩放和后端 kernel。它不一定能做到 bitwise identical，但必须有可解释的误差合同。对一个本地推理引擎来说，至少要比这些层次：

- 单算子输出：RMSNorm、linear、RoPE、attention、MLP 与 reference 的误差。
- 层输出：一个 decoder block 前后 hidden state 的误差。
- logits：最大绝对误差、top-k 排名变化和采样前分布变化。
- 生成序列：固定 seed 和固定 sampler 下 token 序列是否在合同内。
- 性能证据：每个优化计划的 prefill/decode 分项耗时和峰值内存。

传统编译器有 verifier、IR dump、debug info 和测试套件；AI 编译器也需要对应工具。Graph IR dump 用来查融合是否正确，memory plan dump 用来查 buffer 生命周期，backend plan dump 用来查 kernel 选择，oracle report 用来查误差，profiler report 用来查优化是否真的改善瓶颈。

### 硬验收

读完本节，应该能把“AI 编译器”和“推理引擎”的关系讲清楚：

- AI 编译器把模型、图、张量合同和后端能力编成 executable plan。
- 推理引擎在运行期维护 prompt、session、KV cache、sampler 和 profiler。
- Graph IR、Tensor IR 和 Loop IR 分别负责算子依赖、张量布局和机器循环。



- 融合优化必须检查消费者、生命周期、误差边界和后端支持。
- 内存规划本质上是张量级生命周期分析，KV cache 不能当普通 scratch buffer。
- CPU/GPU lowering 共享语义合同，但按不同硬件目标选择不同物理布局和 kernel。

后续进入真实模型加载、tokenizer 和具体算子时，这套编译视角会反复出现：每个优化都要说明它在哪一层 IR 生效，改变了哪条地址流或生命周期，后端计划如何验证，以及 runtime 怎样在 prefill 和 decode 中使用它。

### 3.12 AI 编译器细化：IR、pass、lowering 与 executable plan

#### 先把 IR 讲清楚：它是可检查的中间账本

如果没有学过编译原理，**IR**（中间表示）这个词容易显得抽象。可以先把它理解成“机器可读、可检查、可改写的中间账本”。源代码、模型文件或 JSON 配置都不是优化的好对象，因为它们包含太多外部格式细节。最终机器代码或 kernel 调用又太低层，已经失去了很多语义。IR 站在中间：保留足够语义让优化安全发生，又足够结构化让程序可以分析和改写。

传统 C++ 编译器不会直接从字符生成机器指令。它先把字符切成 token，组成 AST，做语义检查，降成 IR，再进入优化和后端。AI 编译器也是同一个思路，只是输入换成模型资产，输出换成 executable plan。

#### 核心理想

IR 的价值不是“多一层抽象”，而是让每个优化都有检查对象。没有 IR，融合、layout 选择、内存复用、量化 lowering 和后端选择都会退化成分散在运行时代码里的猜测。

本节把 AI 编译器从“概念流水线”细化到工程结构：Graph IR 表示算子依赖，Tensor IR 表示 shape/layout/dtype/quant，Loop IR 表示 tile/vector/thread，pass pipeline 表示改写顺序，lowering 表示逐步靠近 CPU/GPU 后端，executable plan 表示 runtime 真正执行的结果。

#### MiniLLM-CPU 的最小 IR dump

先把 tiny fixture 的一层 decoder block 写成 IR dump，而不是只讲 IR 定义。固定参数为 `hidden=8, heads=2, kv_heads=1, head_dim=4, T=3`。导入后，Graph IR 至少应能打印出类似下面的结构：

Listing 3.54 MiniLLM-CPU 一层 decoder 的 Graph IR dump

```
graph mini_layer0_decode:
 %tok : i32[3]
 %x : f32[3,8] = embedding(%tok, @tok_embeddings)
 %x_norm : f32[3,8] = rmsnorm(%x, @layers.0.attn_norm)
 %q : f32[3,2,4] = linear(%x_norm, @layers.0.wq)
 %k : f32[3,1,4] = linear(%x_norm, @layers.0.wk)
 %v : f32[3,1,4] = linear(%x_norm, @layers.0.wv)
 %q_rot,%k_rot = rope(%q, %k, position=0..2)
 state_write kv[layer=0,pos=0..2] = kv_append(%k_rot, %v)
 %attn : f32[3,8] = attention(%q_rot, state_read kv[0,0..2])
 %h : f32[3,8] = residual_add(%x, %attn)
 %h_norm : f32[3,8] = rmsnorm(%h, @layers.0.ffn_norm)
 %ffn : f32[3,8] = swiglu_mlp(%h_norm, @w1, @w3, @w2)
 %out : f32[3,8] = residual_add(%h, %ffn)
```

这段 dump 已经包含三个关键事实。第一，shape 是 IR 的一部分，不是注释。第二，KV cache 是 state edge，不是普通 activation。第三，**linear** 节点仍是语义节点，尚未决定 scalar、AVX2、AVX-512、GPU 或库调用。

**Tensor IR dump: 把地址和 layout 固定下来**

Graph IR 之后要能打印每个 value 的 Tensor IR。以 **%q**、**%k** 和 KV cache 为例:

**Listing 3.55** Tensor IR dump 示例

```
value %q:
 dtype=f32
 shape=[3,2,4]
 stride=[8,4,1]
 layout=token_head_dim
 device=host
 lifetime=[linear_q, attention]

value %k_rot:
 dtype=f32
 shape=[3,1,4]
 stride=[4,4,1]
 layout=token_kvhead_dim
 device=host
 lifetime=[rope, kv_append]

state kv_cache.layer0:
 dtype=f32
 logical_shape=[max_seq=4, kv_heads=1, head_dim=4]
 logical_stride=[4,4,1]
 physical_layout=position_kvhead_dim
 state_lifetime=session
```

这个 dump 直接服务后面的错误定位。若 GQA head 映射错，通常会出现在 **shape=[T,kv\_heads,head\_dim]** 或 attention 的 read binding；若 KV offset 错，通常会出现在 **logical\_stride** 或 physical layout；若 fusion 错，lifetime 和 materialization 点会暴露问题。

**Graph IR: 先表达依赖和状态**

Graph IR 关心的是“谁依赖谁”。在 decoder-only Transformer 中，RMSNorm 的输出进入 Q/K/V projection，RoPE 的输出写入 KV cache，attention 读取 KV cache 并输出 hidden state，MLP 读取 residual 后的 hidden state。Graph IR 不应该关心 AVX2 有几个寄存器，也不应该关心 GPU block 有多少线程。

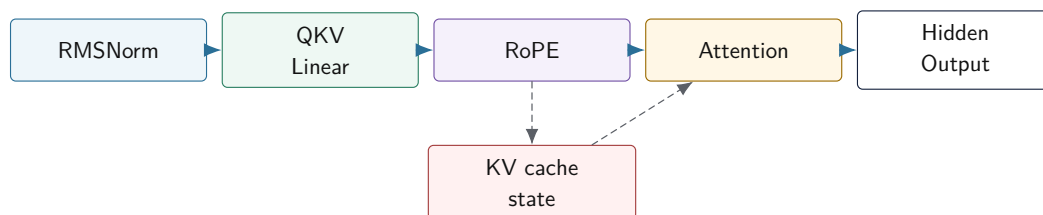
**Listing 3.56** Graph IR 节点的最小字段

```
GraphNode:
 id
 opcode: rmsnorm | linear | rope | attention | mlp | sampler
 inputs
 outputs
 state_reads
 state_writes
 stage: load | prefill | decode | both
 debug_name
```

**state\_reads** 和 **state\_writes** 很重要。KV cache 不是普通张量边；它跨 token 存活，会被 decode 反复读取。若 Graph IR 把 KV cache 当成普通 activation，memory planner 可能错误复用它的存储。传统编译器里，内存写、异常、volatile 和函数调用都需要特殊建模；AI 编译器里，KV

cache、sampler state 和 profiler/oracle materialization 也需要特殊建模。

#### Graph IR 表达数据边和状态边



图示：本图要证明的命题是：Graph IR 不只记录张量数据流，还要显式记录 KV cache 这样的跨 token 状态边。

#### Tensor IR: shape、stride、layout 和 dtype 是语义的一部分

Graph IR 知道有一个 **linear** 节点，但还不够。后端需要知道输入是  $[T, H]$  还是  $[H]$ ，是否连续，stride 是多少，权重是 fp16 还是 q4 group，输出要放在 CPU 内存还是 GPU 显存。这些属于 Tensor IR。

Listing 3.57 Tensor IR value descriptor

```

Value:
 id
 element_type
 shape
 stride
 logical_layout
 physical_layout
 quant_descriptor_id
 device
 alias_info
 lifetime_info

```

初学者常把 shape 当作数组长度，把 stride 当作底层细节。实际不是。shape 决定算子数学语义；stride 决定地址公式；layout 决定 cache 和 SIMD 读流；dtype 决定数值误差和 kernel 选择；quant descriptor 决定 scale/zero point 和反量化规则。它们都是编译器需要理解的合同。

例如连续的  $[T, H]$  hidden state 和转置视图可能拥有同样 shape，但地址流完全不同。一个 pass 如果只看 shape，不看 stride，就可能把一个本来连续的 kernel 应用到非连续视图上。第一册讲过对象布局 and 指针地址；这里就是把那些知识用于张量。

#### Loop IR: 机器优化从循环账本开始

当 Tensor IR 已经确定某个 **linear** 的输入、权重和输出，后端还要决定怎样循环。Loop IR 表示 tile、向量化、展开、线程划分和 memory access。它不再关心模型层名，而是关心循环变量和地址流。

Listing 3.58 一个 GEMV lowering 后的循环账本

```

for out_tile in output_channels step TILE_N:
 accum[TILE_N] = 0
 for k in input_channels step TILE_K:
 x_vec = load input[k : k + TILE_K]
 w_panel = load packed_weight[out_tile, k]
 accum += dot_or_fma(x_vec, w_panel)

```

```
store output[out_tile]
```

这里的每个选择都对应第二册的硬件账本。TILE 的大小影响 cache line、SIMD 寄存器和预取；`out_tile` 的划分影响线程任务；`packed_weight` 的布局影响 TLB 和顺序读；tail path 影响分支和 mask；accumulator 数量影响 dependency chain 和寄存器压力。

Loop IR 不一定要做成复杂编译器基础设施。对于本册 C++20 项目，可以先用结构化 descriptor 和少量 lowering 规则表达它：

Listing 3.59 LoopPlanDesc 的保守工程形态

```
LoopPlanDesc:
 operator_id
 backend
 tile_m
 tile_n
 tile_k
 vector_width
 unroll_k
 tail_policy
 thread_partition
 packed_weight_desc
 accumulator_dtype
```

重点不是形式，而是让后端选择可见、可测试、可报告。不要把 tile 和 tail 都埋在某个巨大 `matmul.cpp` 里。

### Pass 是带前置条件和后置条件的改写

Pass（编译遍）不是“随便遍历图改一改”。一个合格 pass 应该声明它读什么、要求什么、产出什么、会让哪些分析失效。传统编译器这样管理优化顺序；AI 编译器同样需要。

Listing 3.60 pass 合同的最小字段

```
PassContract:
 name
 input_ir
 required_analyses
 preconditions
 transformations
 produced_analyses
 invalidated_analyses
 verifier_after_pass
```

例如 `FuseRmsNormLinearPass` 的前置条件可能是：RMSNorm 输出只有一个消费者；consumer 是 linear；debug/oracle 不要求 materialize RMSNorm 输出；后端支持 fused dtype/layout；量化权重的 scale 可以在 fused kernel 中正确读取。后置条件是：原来的两个节点替换成 fused node；输出 shape 不变；lifetime 信息失效，需要重新计算。

### 常见误区

pass 如果不声明前置条件，就会把 bug 伪装成优化。特别是量化、KV cache 和 oracle materialization 相关 pass，必须明确哪些边界不能跨。

### 一个具体 pass pipeline

把 pass 讲成抽象概念还不够。一个能支撑本册推理引擎的保守 pipeline 可以先写成下面这样：

Listing 3.61 第三册项目的保守 pass pipeline

```

ImportCanonicalization
ShapeDtypeInference
QuantContractAttach
GraphVerification
SimpleCanonicalize
UseDefAndLivenessAnalysis
ConservativeFusion
MemoryPlanning
BackendLowering
PlanVerification

```

**ImportCanonicalization。** 模型导入后，先把外部格式里的名字、轴顺序、常量表示和算子别名统一成项目内部格式。比如某些模型文件里 Q/K/V 是三个权重，某些是一个 packed QKV 权重；某些图里 RMSNorm 写成多个基础算子，某些直接写成一个算子。canonicalization 的目标不是优化，而是让后续 pass 面对稳定输入。

**ShapeDtypeInference。** 这一遍推导每条 value 的 shape、dtype、layout 和 device。它必须能解释 prefill 的  $[T, H]$  和 decode 的  $[1, H]$ ，也必须能解释 GQA 中  $n_q$  与  $n_{kv}$  不同的情况。若 shape 推导失败，不能让后端自己猜。

**QuantContractAttach。** 量化不是后端小技巧。导入器读到的 int8、int4、group scale、zero point、packed order、tail policy，要在这一阶段附到 Tensor IR 的 descriptor 上。后续 pass 做 fusion、memory planning 和 lowering 时都要能查到。

**GraphVerification。** 在任何优化前先检查图：输入存在、输出唯一、状态读写合法、KV cache position 合法、sampler 只消费 logits、debug/oracle 要求的 materialization 点存在。这个阶段发现的问题通常是模型资产或导入逻辑问题。

**SimpleCanonicalize。** 做不会改变语义的小整理，比如删除恒等 reshape、合并连续 view、把等价 dtype 标记统一。它不应该跨过状态边，也不应该改量化解释。

**UseDefAndLivenessAnalysis。** 收集每个 value 的定义点、使用者和最后使用位置。memory planner 和 fusion 都要消费这些事实。不要在 fusion pass 里临时扫描一遍再自己做决定；这样会让测试粒度变差。

**ConservativeFusion。** 只做前置条件足够明确的融合。比如 RMSNorm 与 Linear 融合可以减少一次 activation 写回，但前提是 RMSNorm 输出没有被 oracle 要求物化、没有被多个消费者使用、后端支持对应 dtype/layout、量化 scale 读取顺序仍然正确。

**MemoryPlanning。** 根据 liveness、alignment、backend workspace 和 debug/oracle materialization 点分配 arena offset。KV cache、packed weight、workspace 和 activation arena 不属于同一类生命周期，不能混在一个简单线性 buffer 里。

**BackendLowering。** 把语义节点降到 CPU/GPU plan。CPU 计划包含 packed weight、tile、SIMD kernel、线程划分；GPU 计划包含 device buffer、stream、workspace、staging 和 sync。

**PlanVerification。** 最后一遍检查 executable plan：segment 顺序是否满足依赖，arena offset 是否冲突，kernel 是否支持实际 dtype/layout/quant，KV cache state binding 是否完整，profiler schema 是否能覆盖端到端指标。

这个 pipeline 故意保守。它没有一开始就做复杂图重写、自动调参或跨层大融合。原因是第三册的目标不是堆优化名词，而是让每个阶段都有可验证产物。等 reference、oracle、profiler 和回归门禁稳定后，再增加更激进的 pass 才有意义。

### Pass 前后 diff：优化必须可审计

每个 transform pass 都应该能输出前后差异。下面用 `EliminateRedundantReshape` 做一个最小例子。导入器有时会把 Q projection 写成 `linear->reshape->view`，但 reshape 只是把  $[T, 8]$  解释成  $[T, 2, 4]$ ，没有改变底层连续布局。

Listing 3.62 pass 前的 IR

```
%q_flat : f32[3,8] = linear(%x_norm, @wq)
%q_view : f32[3,2,4] = reshape(%q_flat)
%q_rot : f32[3,2,4] = rope_q(%q_view, position=0..2)
```

Listing 3.63 pass 后的 IR

```
%q_flat : f32[3,8] = linear(%x_norm, @wq)
%q_view : f32[3,2,4] = view(%q_flat, shape=[3,2,4], stride=[8,4,1])
%q_rot : f32[3,2,4] = rope_q(%q_view, position=0..2)
```

这次改写没有删除语义 value，只是把分配型 reshape 降成 view。pass verifier 要检查元素数相同、源 tensor 连续、目标 stride 正确、没有新增写入、oracle 若要求 dump %q\_view 仍然能 materialize。若这些条件不满足，pass 必须拒绝，而不是为了少一次分配硬改。

### Memory plan 示例：arena 不是随手分配

以 tiny decode 的单层为例，忽略 KV cache 和权重后，activation arena 可以复用部分 buffer。一个可读的 memory plan 至少要打印 value 生命周期和 offset：

Listing 3.64 MiniLLM-CPU tiny memory plan 摘要

```
arena host_activation:
 size_bytes = 768
 alignment = 64
```

| value   | bytes | defined_at | last_used_at | offset               |
|---------|-------|------------|--------------|----------------------|
| %x      | 96    | embedding  | residual1    | 0                    |
| %x_norm | 96    | rmsnorm0   | qkv_linear   | 128                  |
| %q      | 96    | q_linear   | attention    | 256                  |
| %k_rot  | 48    | rope       | kv_append    | 384                  |
| %v      | 48    | v_linear   | kv_append    | 448                  |
| %attn   | 96    | attention  | residual1    | 512                  |
| %h_norm | 96    | rmsnorm1   | mlp          | 128 # reuses %x_norm |
| %ffn    | 96    | mlp        | residual2    | 256 # reuses %q      |

这张表比“使用 arena 减少分配”有用得多。它说明哪些 offset 可以复用，哪些不能复用。若打开 debug oracle 要 dump %q, %ffn 不能复用 %q 的 offset；若 %k\_rot 写入 KV cache 后仍被 attention 直接读取，last use 也要延长。memory verifier 检查的就是这些事实。

### RMSNorm + Linear 融合的前置条件

用一个具体融合说明 pass 合同。RMSNorm 后面常接 QKV projection 或 MLP projection。若把 RMSNorm 输出写回 arena，再由 linear 读出，会多一次  $[N, H]$  的内存读写。融合后的 kernel 可以一边算归一化，一边把结果喂给矩阵乘。

Listing 3.65 融合前后的图形态

```
before:
 y = rmsnorm(x, gamma)
 z = linear(y, packed_weight)

after:
 z = fused_rmsnorm_linear(x, gamma, packed_weight)
```



这个融合的前置条件至少包括：

- **y** 只有一个真实消费者，或者额外消费者只是可关闭的 profiler 统计。
- oracle 当前配置不要求比较 **y** 的逐元素输出。
- **x**、**gamma** 和 **packed\_weight** 的 dtype/layout 被后端 fused kernel 支持。
- quantized weight 的 scale metadata 能在 fused kernel 中按相同逻辑顺序读取。
- RMSNorm 的 **epsilon**、累加类型和舍入规则与 reference profile 一致。
- 融合后 **z** 的 shape、dtype、device 和 quant contract 不变。

后置条件也要写清：原 **rmsnorm** 和 **linear** 被一个 fused node 替代；**z** 的逻辑 value id 可以保持或映射到新 id；use-def、liveness、cost analysis 和 memory plan 失效；shape/dtype 推导可以复用，但 pass verifier 必须重新检查输出合同。

### 核心思想

融合不是把两个函数塞进一个大函数。融合是一个受 use-def、oracle、dtype、layout、quant metadata 和后端能力共同约束的 IR 改写。

### 一个错误 fusion 的反例

再看一个不能随便融合的例子。假设图里有 RoPE、KV append 和 attention：

**Listing 3.66** RoPE、KV append 与 attention 的状态边

```
q_rot, k_rot = rope(q, k, position)
kv_cache.append(layer, position, k_rot, v)
out = attention(q_rot, kv_cache.read(layer, 0..position))
```

有人可能想把它们全部融合成一个 **rope\_attention** kernel，减少中间张量写回：

**Listing 3.67** 错误融合：丢掉了状态写入

```
out = fused_rope_attention(q, k, v, kv_cache, position)
```

如果这个 fused kernel 只在内部使用 **k\_rot** 参与当前 attention，却没有把 **k\_rot/v** 写入 KV cache，那么当前 token 可能看起来没错，但下一个 token 会读不到这一位置的 K/V。这个 bug 不一定立刻崩溃，因为 cache slot 里可能有旧值或零值；它会表现为后续 logits 漂移。

正确的融合必须把状态写入保留下来：

**Listing 3.68** 正确融合必须保留 state write

```
out = fused_rope_kv_append_attention(
 q, k, v,
 kv_cache_write_binding,
 kv_cache_read_binding,
 position)
```

pass verifier 要检查两件事。第一，融合前后的状态读写集合等价：**KVCache[layer, position]** 仍然被写入，attention 仍然读取 **0..position**。第二，执行顺序等价：当前位置写入必须在未来 token 读取前可见；若 GPU 上异步执行，还要有 stream/event 或 segment 顺序保证。

### 常见误区

AI 编译器里的状态边不能被当成普通 activation 边优化掉。KV cache、sampler state、随机数状态、debug/oracle materialization 都属于会影响后续行为的边界。

**分析 pass：先收集事实，再做决定**

很多优化应该先跑分析 pass，再跑变换 pass。分析 pass 不改 IR，只产出事实。变换 pass 使用这些事实做决定。把两者混在一个大函数里，后面很难调试和测试。

| 分析                | 产出事实                   | 后续用途                  |
|-------------------|------------------------|-----------------------|
| Shape analysis    | 每条边 shape/dtype/layout | verifier、kernel 选择    |
| Use-def analysis  | 每个 value 的消费者          | fusion、liveness       |
| Liveness analysis | defined/last use       | activation arena      |
| Alias analysis    | 是否共享存储或视图              | in-place、内存复用         |
| Quant analysis    | scale/group/format 支持  | quant lowering、oracle |
| Cost analysis     | 估算内存流和计算量              | pass 决策、plan 选择       |

例如 fusion pass 不应该自己顺手扫描所有消费者并猜内存生命周期。它应该消费 use-def 和 liveness 结果。这样测试可以分别验证 use-def 是否正确、liveness 是否正确、fusion 是否尊重这些事实。

**lowering：从语义逐步靠近后端**

lowering 是把高层语义变成低层执行计划的过程。它不是一步到位，也不是直接把 graph node 换成函数指针。更稳妥的做法是分阶段：

**Listing 3.69** AI 编译器 lowering 阶段

```

Graph IR:
 decoder block, attention, linear, mlp

Tensor IR:
 concrete shapes, layouts, quant descriptors

Backend-independent plan:
 fusion boundaries, liveness, memory plan

Backend-specific plan:
 CPU packed panels, SIMD kernel desc, thread partition
 GPU device buffers, kernel adapter, stream dependencies

Executable plan:
 segments, kernel calls, arena offsets, state bindings

```

分阶段的好处是：高层 verifier 可以检查语义，低层 verifier 可以检查机器合同。比如在 backend-independent plan 中，attention 仍然是语义操作；到了 CPU plan，decode attention 可能降低为按 head 和 position 分块读取 KV cache 的 kernel；到了 executable plan，runtime 只看到 segment、buffer offset、kernel handle 和 state binding。

**CPU lowering：从 tensor 到 SIMD plan**

以 decode 阶段的 quantized linear 为例，CPU lowering 要做的事情包括：

- 判断当前 shape 更像 GEMV 还是 small-batch GEMM。
- 查询 CPU feature，选择 scalar、AVX2、AVX-512 或其他 ISA plan。
- 确认权重是否已经按目标 ISA packed，必要时触发 prepare。
- 根据 output channel、group size、tail 和 cache 选择 tile。
- 选择单线程或多线程分区策略。
- 生成 kernel 调用参数和 profiler 标签。

Listing 3.70 CPU lowering 输出的计划片段

```
CpuKernelCall:
 kernel_id: q4_gemv_avx2_g64
 input_binding: arena.hidden_norm
 packed_weight: packed.layers.12.mlp.wup
 output_binding: arena.mlp_up
 tile_n: 16
 tile_k: 64
 threads: partition_by_output_rows
 tolerance_profile: q4_weight_f16_output
```

这里和第一册、第二册的连接很直接。第一册帮助我们理解 ABI、函数调用、指针和反汇编；第二册帮助我们理解 AVX、cache、TLB、线程和 NUMA；第三册把它们组织成 lowering 计划。

### GPU lowering: 把同步和 staging 写进计划

GPU lowering 不应该只输出一个 `launch(kernel)`。它还要表达 device buffer、stream、workspace、host-device staging 和同步点。特别是 logits 通常需要回到 CPU 采样，KV cache 可能常驻 GPU，某些 fallback 可能回到 CPU，这些边界都要在计划里显式出现。

Listing 3.71 GPU lowering 输出的计划片段

```
GpuSegment:
 stream: decode_stream
 device_inputs:
 hidden_buffer
 packed_weight_buffer
 kv_cache_buffer
 workspace: attention_workspace
 kernels:
 rmsnorm_linear_fused
 rope_kv_append
 attention_decode
 mlp_fused
 staging:
 copy logits top-k to host
 sync:
 wait before sampler
```

如果计划不记录 staging 和 sync, profiler 就会只看到 kernel 时间, 看不到端到端 decode latency。第二册讲过异步 I/O、背压和运行时调度；GPU 后端里的 stream 和 host-device 同步就是同类问题在 AI infra 中的具体形态。

### Executable Plan: runtime 执行的是计划，不是重新解释图

executable plan 是 AI 编译器交给 runtime 的产物。它应该把已经决定的事情固化下来：segment 顺序、kernel 选择、arena offset、packed weight、KV cache 绑定点、workspace 需求、profiler 标签和 oracle 边界。

Listing 3.72 ExecutablePlan 的核心结构

```
ExecutablePlan:
 plan_id
 stage: prefill | decode
 backend
```

```

assumptions:
 max_tokens
 context_bucket
 dtype_profile
 quant_profile
memory_plan:
 arena_size
 value_offsets
 workspace_requirements
segments:
 Segment[]
oracle_points:
 layer_output
 logits
profiler_schema

```

runtime 拿到 plan 后, 不再做 shape inference, 不再做 fusion, 不再选择 tile, 不再 packing。它只绑定 session 状态, 例如当前 token、KV cache position、arena 基址、backend context, 然后执行 segment。这样热路径才不会退回解释器形态。

### Executable plan 实例: decode 一个 token

下面是 tiny 模型 decode 第 3 个 position 时的 plan 摘要。它故意只展示一层中的几个 segment, 但字段足够说明 runtime 应该执行什么。

**Listing 3.73** MiniLLM-CPU decode executable plan 摘要

```

ExecutablePlan id=mini_cpu_decode_v1 stage=decode backend=cpu
assumptions:
 hidden=8
 max_context=4
 quant_profile=f32_reference
 isa=scalar

segments:
 s00 embedding:
 inputs: token_id
 outputs: arena.%x
 kernel: embedding_scalar
 profiler_label: layer0.embedding

 s01 qkv_projection:
 inputs: arena.%x_norm, weights.wq,wk,wv
 outputs: arena.%q, arena.%k, arena.%v
 kernel: linear_f32_scalar
 workspace_bytes: 0
 oracle_point: operator

 s02 rope_kv_append:
 inputs: arena.%q, arena.%k, arena.%v, position=3
 state_writes: kv[layer=0,pos=3]
 outputs: arena.%q_rot
 kernel: rope_and_kv_append_scalar

```

```
s03 attention_decode:
 inputs: arena.%q_rot
 state_reads: kv[layer=0,pos=0..3]
 outputs: arena.%attn
 kernel: attention_gqa_scalar
 profiler_label: layer0.attn.ctx4
```

这个 plan 让 runtime 的职责变窄：检查 session 还有 KV 容量，绑定 arena 基址，按 segment 执行，写 trace。它不应该临时决定 `attention_gqa_scalar` 还是 `attention_gqa_avx2`，也不应该重新推导 `kv[layer=0,pos=0..3]` 的范围。若后端换成 AVX2，变化应该体现在新的 plan id、kernel desc、packed weight desc 和 tolerance profile 中。

### IR、plan 与报告 artifact

AI compiler 章节的最终产物不应该只存在于书里。项目至少要能导出四份文本或 JSON artifact：

**Listing 3.74** AI compiler 最小 artifact

```
ir-before.json:
 imported graph and tensor descriptors

ir-after-passes.json:
 canonical graph, fused nodes, view/lifetime facts

memory-plan.json:
 arena size, value offsets, workspace, materialization blocks

executable-plan.json:
 segments, backend kernels, packed weights, state bindings, profiler labels
```

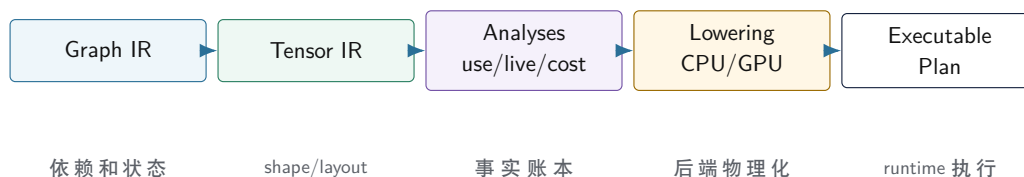
验收命令可以先很小：

**Listing 3.75** AI compiler artifact 验收命令

```
ai-compile-plan --model fixtures/minillm_tiny --backend cpu \
 --dump-ir-before ir-before.json \
 --dump-ir-after ir-after-passes.json \
 --dump-memory-plan memory-plan.json \
 --dump-executable-plan executable-plan.json
```

负例也必须存在：坏 embedding 行数、坏 QKV shape、坏 `kv_heads`、坏 int4 group size、debug oracle 要求被 fusion 删除的 value，都应该被 verifier 拒绝并给出对象名、期望、实际、来源和下一步检查。

## IR 到 executable plan 的编译流水线



图示：本图要证明的命题是：AI 编译器的最终产物不是一个抽象图，而是 runtime 可以直接绑定和执行的计划。

## Verifier 要贯穿每一层

前端有 verifier，IR pass 之后也要有 verifier，lowering 后还要有 verifier。不同层检查不同不变量：

- Graph verifier：节点输入输出存在，状态边合法，没有非法循环依赖。
- Tensor verifier：shape、stride、dtype、layout、quant descriptor 一致。
- Pass verifier：改写前后输出语义和边界没有被破坏。
- Memory verifier：arena offset 不错误重叠，alignment 满足后端要求。
- Backend verifier：kernel 支持 dtype/layout/tail/quant，packed descriptor 完整。
- Plan verifier：segment 顺序、state binding、workspace 和 sync 边界完整。

这些 verifier 不一定都很复杂，但必须存在。它们让错误在编译期或 prepare 阶段暴露，而不是进入运行时热路径后才以崩溃或质量漂移形式出现。

## 合法性规则：alias、liveness、fusion 和 layout propagation

AI compiler 的难点不只是“能不能把两个算子合起来”。真正的问题是：这个改写在所有合法输入、layout、dtype、quant profile、debug/oracle 和状态边下是否保持语义。为此，需要把 alias analysis、liveness、buffer reuse、fusion legality 和 layout propagation 写成规则，而不是写成后端里的 if。

**Alias analysis：谁共享同一块 storage** Tensor view、reshape、slice、transpose、KV cache state 和 in-place output 都可能共享底层 storage。Alias analysis 要给每个 value 分配 alias set，并标记只读、写入、view、state 和 external buffer。若两个 value 可能别名，memory planner 不能随意重叠它们；fusion pass 也不能把会改变读写顺序的节点合并。

Listing 3.76 AliasAnalysis 输出示例

```

AliasSet 7:
 %x read_write storage=arena0 offset=0 bytes=96
 %x_view read_only view_of=%x stride=[8,4,1]

AliasSet 12:
 state kv_cache.layer0 session_lifetime
 state_read attention0
 state_write kv_append0

```

KV cache 的 alias set 必须是 session lifetime，不能被当成普通 activation。若 memory planner 把它和临时 hidden 复用，短测试可能通过，第二个 token 就会读到被覆盖的历史 K/V。

**Liveness：buffer reuse 的唯一安全入口** Buffer reuse 只能在生命周期不重叠时发生。生命周期不是源码顺序，而是 executable plan 中定义点到最后使用点。debug/oracle materialization 会延长生命周期；异步 GPU kernel 和 host-device copy 也会延长生命周期；state edge 更是跨 token 存活。Liveness analysis 要输出每个 value 的 `defined_at`、`last_used_at` 和 barrier。



Listing 3.77 buffer reuse 的拒绝条件

```
cannot reuse if:
 live ranges overlap
 values may alias
 alignment requirements differ and cannot be met
 oracle requires both dumps
 async kernel may still read old buffer
 one value is session state
```

**Fusion legality: 减少内存写不等于合法** RMSNorm+Linear 融合看起来简单：少写一次 normalized hidden。但若 RMSNorm 输出有两个消费者，融合会重复计算或改变物化点；若 oracle 要 dump RMSNorm 输出，融合必须保留可 materialize 结果；若量化 linear 需要特定 scale 读取顺序，融合 kernel 要证明误差边界仍成立；若 backend 不支持当前 layout，不能为了融合强行转 layout 又不计成本。

| 检查项             | 为什么需要                  | 失败时动作           |
|-----------------|------------------------|-----------------|
| 单消费者            | 避免重复计算或语义改变            | 拒绝融合            |
| 无状态边跨越          | KV/sampler state 不能被隐藏 | 切断 fusion 边界    |
| oracle 允许       | 保留调试和回归证据              | materialize 或拒绝 |
| dtype/layout 支持 | 后端 kernel 有能力执行        | 选择未融合路径         |
| 误差合同            | 数值变化在阈值内               | 降级或扩大 oracle 检查 |

**Layout propagation: layout 是会传播的类型** 如果 QKV linear 输出选择 `token_head_dim`，RoPE 和 attention 就要消费这个 layout；如果 attention kernel 需要 `head_token_dim`，中间需要 transpose/view/copy。Layout propagation 要像类型推导一样，把每个 op 的支持 layout、偏好 layout 和转换成本传播出来。不能只在某个 kernel 里发现 stride 不合适再临时 copy。

Listing 3.78 layout propagation 的规则形态

```
RMSNorm:
 preserves input layout

Linear:
 input layout: token_hidden
 output candidates: token_hidden | token_head_dim

RoPE:
 requires head_dim contiguous

AttentionDecode:
 prefers kv layout: layer_position_kvhead_dim
 requires q layout: token_head_dim
```

这些规则让 compiler 能解释“为什么插入 layout conversion”。如果转换成本大于融合收益，plan verifier 应能指出优化被拒绝的原因，而不是静默生成慢路径。

### 展开：shape inference、in-place 和 stateful op 的严格规则

AI compiler 章节要避免停在“有一个 verifier”。真正的 verifier 要有可写成规则的推导。以 decoder block 为例，shape inference 至少要覆盖 linear、split、reshape、RoPE、attention、MLP 和 lm head。

| op            | 输入条件                         | 输出规则                                    |
|---------------|------------------------------|-----------------------------------------|
| Linear        | $X = [T, K], W = [O, K]$     | $Y = [T, O]$                            |
| QKV split     | $Y = [T, n_q D + 2n_{kv} D]$ | $Q = [T, n_q, D], K/V = [T, n_{kv}, D]$ |
| RoPE          | last dim $D$ even            | shape 不变, position edge 必须存在            |
| GQA attention | $n_q \% n_{kv} = 0$          | $O = [T, n_q, D]$ 或 merge 后 $[T, H]$    |
| SwiGLU        | gate/up 同 shape              | elementwise 后 shape 不变                  |
| LM head       | $X = [T, H], W = [V, H]$     | logits $[T, V]$                         |

规则失败要给出诊断,而不是让后端崩溃。例如  $n_q = 32, n_{kv} = 7$  时, GQA head mapping 不整除; `head_dim` 为奇数时, RoPE 对偶维度不成立; QKV packed weight 的输出维度对不上时, 导入阶段就应拒绝模型。

### in-place legality: 省 buffer 不能破坏读写顺序

in-place 优化看起来诱人,例如把 RMSNorm 输出写回 hidden, 或让 residual add 覆盖输入。但它必须满足读写顺序、alias、liveness 和 oracle 条件。若某个输入后续还要被另一个节点读取, in-place 会提前破坏值; 若 debug 要 dump 原输入, in-place 会丢证据; 若后端异步执行, host 以为可复用时 device 可能还没读完。

Listing 3.79 in-place 改写的拒绝条件

```
reject in-place if:
 input has another live consumer
 input aliases a state buffer
 oracle/debug requires original value
 op is not mathematically safe for overwrite
 backend may read input after writing output
 dtype/layout conversion changes element size
```

in-place pass 的输出不能只是“复用 offset”。它应输出一个原因表: 哪些 value 被覆盖, 覆盖发生在哪个 segment, 哪些原值最后使用点在覆盖之前, 哪个 verifier 证明了 alias 不冲突。没有原因表的 in-place 优化很难调试。

### stateful op: KV cache 和 sampler 是副作用边

传统图优化容易假设节点是纯函数: 输入相同, 输出相同, 没有副作用。LLM decode 不完全满足。KV append 会修改 session state; sampler 会消费和更新 RNG state; profiler 会记录事件; streaming 会写 event queue。这些边必须进 IR, 否则 pass 会错误重排或删除它们。

Listing 3.80 stateful op 的 IR 边

```
kv_append:
 data_inputs: k_current, v_current
 state_read: kv_cache[layer, session]
 state_write: kv_cache[layer, session]
 ordering: after rope, before attention_read_next

sampler:
 data_inputs: logits
 state_read: sampler_state
 state_write: sampler_state
 output: next_token
```

有了 state edge, compiler 才知道 KV append 不能被死代码删除, sampler 不能跨 logits 改写, 两个 token 的 decode 不能随意交换。第三册把 KV cache 当 state edge, 是 AI compiler 从“矩阵图”走向“推理 runtime”的关键。

### pass pipeline 的失效分析

每个 pass 都应声明会使哪些 analysis 失效。Fusion 改 use-def 和 liveness; layout conversion 改 stride 和 cost; quant lowering 改 dtype、scale 和 kernel selection; memory planning 消费 liveness 但不应改变 Graph IR。没有失效分析, 后续 pass 可能拿旧事实做错误决定。

| pass               | 保留的事实                  | 失效的事实                               |
|--------------------|------------------------|-------------------------------------|
| Shape inference    | graph topology         | tensor dtype/layout facts 被更新       |
| Fusion             | 部分 shape               | use-def、liveness、cost、memory plan   |
| Layout propagation | graph semantics        | stride、cost、kernel capability match |
| Quant lowering     | logical shape          | dtype、scale path、backend support    |
| Memory planning    | graph/tensor semantics | arena offset 旧结果                    |

这和 C++ 编译器优化是同一个工程原则: analysis 和 transform 分离, transform 后明确 preserved analyses。否则 pass pipeline 会变成一串不可局部验证的脚本。

### 工业实现参照: 不要只发明自己的词

第三册项目可以自研教学 runtime, 但要知道工业系统的设计谱系。

| 系统             | 关键设计                                                 | 本书借鉴点                               |
|----------------|------------------------------------------------------|-------------------------------------|
| ggml/llama.cpp | tensor 描述、quant 格式、backend kernel、轻量 graph           | 本地推理、packed weight、CPU/GPU dispatch |
| oneDNN         | primitive、memory descriptor、post-op、layout 选择        | 算子合同和 layout 是一等对象                  |
| TVM/XLA/MLIR   | IR、pass、lowering、schedule/codegen 分层                 | analysis/transform/verifier 分离      |
| vLLM           | paged KV cache、continuous batching、serving scheduler | KV state 和调度是系统核心                   |
| TensorRT-LLM   | engine、plugin、kernel fusion、KV cache 和多 GPU          | 后端 plan 与运行时强绑定                     |

借鉴不等于复制。教学项目不必实现完整 MLIR 或 vLLM, 但必须回答相同问题: tensor 如何描述, layout 如何传播, state 如何建模, memory 如何复用, kernel 如何选择, serving 如何满足 SLO, 错误如何复现。工业参照的作用是防止自研词汇脱离真实战场。

### 3.13 状态机: 一个 pass 从候选到可上线

AI compiler 最危险的写法, 是把 pass 当作“遍历图然后改一改”。生产级 pass 应该有状态机:

| 状态          | 必须持有的事实                            | 失败时动作                     |
|-------------|------------------------------------|---------------------------|
| Candidate   | 匹配到可改写子图                           | 记录不匹配原因                   |
| Legal       | shape、dtype、layout、state、alias 均合法 | 拒绝改写，保留原图                 |
| Profitable  | 成本模型显示值得改                          | 保留原图并记录原因                 |
| Rewritten   | 新节点、新边和 metadata 生成                | 立即 verifier               |
| Invalidated | use-def、liveness、cost 等失效集合明确      | 禁止后续 pass 使用旧事实           |
| OracleOK    | 小输入或 golden trace 通过               | 回退该 pass 或标记 experimental |
| Enabled     | backend、shape、debug flag 允许        | 运行时选择原路径或新路径              |

这台状态机让 pass 的失败变得可解释。一个 fusion 没触发，不应该只剩“pattern not found”；报告要说明是 shape 不匹配、layout 不兼容、state edge 阻止、debug materialization 要求保留，还是成本模型不划算。

### 严格规则：fusion legality

以 RMSNorm + Linear fusion 为例，合法性至少要检查：

$$\text{shape}(\text{out}(\text{RMSNorm})) = \text{shape}(\text{in}(\text{Linear}))$$

$$\text{dtype/path}(\text{RMSNorm}) \rightarrow \text{dtype/path}(\text{Linear}) \quad \text{在误差界内}$$

还要检查 layout、alias、debug 和 state：

Listing 3.81 Fusion legality checklist

```

shape compatible
dtype and quant profile compatible
layout stride supported by fused kernel
no state edge crossed
no oracle materialization required between ops
no alias requiring in-place preservation
error tolerance declared
fallback kernel exists

```

若中间 tensor 被 oracle dump 要求保存，就不能简单删除；若 Linear 需要 packed weight layout，而 RMSNorm 输出是非 contiguous view，fusion 可能需要额外 copy；若量化 scale 在两者之间有语义，fusion 必须保持 scale path。

### alias/liveness/buffer reuse 的数学模型

Memory planner 可以把每个 value 看成区间：

$$I_v = [\text{first\_use}(v), \text{last\_use}(v)]$$

两个 buffer 复用的必要条件是：

$$I_a \cap I_b = \emptyset$$

但 AI runtime 还要加三条约束：

$$\neg \text{alias}(a, b) \wedge \neg \text{state}(a) \wedge \neg \text{materialized}(a)$$

`state(a)` 表示它属于 KV cache、sampler state、RNG 或跨 token 状态；`materialized(a)` 表示调试、oracle、profile 或外部 API 要求保留。这个模型能解释为什么 activation 可以激进复用，KV cache 不能当普通临时 tensor 复用。

可手算的 liveness 与 buffer reuse 例子

只写公式还不够。以 tiny decode 的一层为例，把 segment 编号固定下来：

Listing 3.82 tiny decode segment 顺序

```
s00 embedding
s01 rmsnorm_attn
s02 qkv_linear
s03 rope_kv_append
s04 attention_decode
s05 residual_add
s06 rmsnorm_mlp
s07 swiglu_mlp
s08 residual_add
```

若没有 debug dump，部分 activation 的生命周期可以写成：

| value                | defined | last use | reuse 判断              |
|----------------------|---------|----------|-----------------------|
| <code>%x</code>      | s00     | s05      | 不能和 s05 前的输出复用        |
| <code>%x_norm</code> | s01     | s02      | s03 后可复用              |
| <code>%q</code>      | s02     | s03      | s04 后可复用，若不保留 q dump  |
| <code>%k_rot</code>  | s03     | s03      | 写入 KV 后可释放临时 view     |
| <code>%attn</code>   | s04     | s05      | s06 后可复用              |
| <code>%h_norm</code> | s06     | s07      | s08 前可复用旧短生命周期 buffer |

现在打开 oracle，要求 dump `%q` 和 `%attn`。生命周期立刻改变：

Listing 3.83 oracle materialization 会延长生命周期

```
dump %q after s04:
 %q live range becomes [s02, s04]

dump %attn after layer:
 %attn live range becomes [s04, s08]
```

这会拒绝原本合法的复用。若 memory planner 不知道 oracle materialization，它可能让 `%h_norm` 覆盖 `%attn`，结果最终 logits 仍可能正确，但中间 dump 和回归定位全部错。可靠的 AI compiler 不是只让模型跑起来，还要让证据点和优化共同存在。

buffer reuse 冲突矩阵

实际 planner 不应只比较两个区间。它要同时检查生命周期、alias、alignment、device、async barrier、state 和 materialization。

| 冲突类型                | 例子                                                | planner 动作                |
|---------------------|---------------------------------------------------|---------------------------|
| live range overlap  | <code>%x</code> 在 residual 前仍要读                   | 拒绝复用                      |
| alias set overlap   | <code>%x_view</code> 与 <code>%x</code> 共享 storage | 按同一 storage 处理            |
| alignment mismatch  | AVX2 kernel 要 32B, AMX tile 要 64B                 | 提升 offset alignment 或分配新槽 |
| device mismatch     | CPU host tensor 与 GPU device tensor               | 不在同一 arena 复用             |
| async barrier       | GPU kernel 仍可能读取旧 buffer                          | 等 event 或拒绝复用             |
| state lifetime      | KV cache 跨 token 存活                               | 从 activation arena 排除     |
| oracle materialized | 中间张量要 dump 到报告                                    | 延长生命周期                    |

这张矩阵直接连接工程事故。很多“内存优化后偶发错”的问题，不是 malloc/free 错，而是 planner 漏掉了一个隐含读者：debug dump、异步 kernel、host copy、KV cache state 或 profiler。

### layout propagation 的 worklist 规则

Layout propagation 应该像类型推导一样跑到不动点，而不是后端临时猜。每个 op 声明支持 layout、偏好 layout、保留 layout 和转换成本。编译器用 worklist 在图上传播约束：

**Listing 3.84** layout propagation 的 worklist 伪代码

```
initialize each value.layout_candidates from tensor descriptor
push all ops into worklist

while worklist is not empty:
 op = pop(worklist)
 input_layouts = meet(producer_candidates, op requirements)
 output_layouts = infer_outputs(op, input_layouts)
 if any candidate set changed:
 push users(output values)

if any candidate set is empty:
 report layout conflict with producer, consumer, and required layout
```

例如 RoPE 要求 `head_dim` 连续，AttentionDecode 偏好按 `position, kv_head, dim` 顺序读 KV，Linear 输出既可以是 `token_hidden`，也可以直接 view 成 `token_head_dim`。如果 Linear 后面只有 RoPE，直接生成 `token_head_dim` view 可能最便宜；如果同一个输出还被一个需要 `token_hidden` 的 oracle dump 消费，就可能需要 materialize 或拒绝融合。

**Listing 3.85** layout conflict 诊断应包含的字段

```
value: %q
producer: q_linear
producer_candidates: token_hidden, token_head_dim
consumer: rope_q
consumer_requires: head_dim_contiguous
other_consumer: oracle_dump
materialization_required: true
decision: keep view plus dump materialization, no destructive layout rewrite
```

好的诊断告诉读者“为什么没有融合/为什么插入 copy”。差的诊断只说 `layoutmismatch`，无法指导修复。



## shape inference 的错误诊断边界

Shape inference 失败时，不要把错误推给 backend。下面这些错误都应该在 compiler 阶段拒绝：

| 错误                                     | 诊断字段                           | 为什么不能放到 runtime |
|----------------------------------------|--------------------------------|-----------------|
| $n_q \% n_{kv} \neq 0$                 | query heads、kv heads、模型路径      | GQA 映射无定义       |
| RoPE head dim 为奇数                      | head dim、op name、position edge | sin/cos 对偶维度不成立 |
| QKV packed 输出维度不等于 $n_q D + 2n_{kv} D$ | expected、actual、weight name    | split 会读错权重     |
| int4 group size 不整除 K 且无 tail policy   | K、group、tail policy            | dequant 地址流不合法  |
| state write 缺失 position                | layer、session、model position   | KV cache 语义不完整  |

这些诊断必须包含对象名、期望值、实际值和来源。AI compiler 的用户不是只想知道“模型坏了”，而是要知道坏在 `layers.12.attention.wk`、manifest、shape verifier、quant descriptor 还是后端 capability。

## 反例：错误 fusion 丢掉了 oracle 和 state

Listing 3.86 错误 pass 的事故形状

```
before:
 q = linear_q(hidden)
 k = linear_k(hidden)
 k_rope = rope(k, model_position)
 kv_append(layer, position, k_rope)
 dump("k_rope", k_rope)

bad fusion:
 fused_qk = fused_linear_rope(hidden)
 attention(fused_qk.q, fused_qk.k)
 // kv_append disappeared
 // dump disappeared
```

这个 pass 可能在单 token 测试里看似正确，因为当前 attention 仍拿到了 K；但下一个 token 读取历史 KV 时会错，debug oracle 也丢失。正确 verifier 必须检查融合前后 state read/write 集合和 materialization 集合等价。

## 工业取舍：为什么不直接上 MLIR/TVM

教学项目自研轻量 IR 的价值，是让读者看清 shape、layout、state、memory 和 lowering 的因果；工业系统使用 MLIR/TVM/XLA 的价值，是成熟 pass 基础设施、dialect、legalization、codegen 和生态。两者不矛盾。第三册可以先用轻量 IR 完成可解释闭环，再在源码阅读中对比 MLIR 的 dialect/operation/pass manager、TVM 的 schedule/lowering、oneDNN 的 primitive descriptor。

| 选择           | 优点           | 代价              |
|--------------|--------------|-----------------|
| 教学轻量 IR      | 易读、可控、适合教学引擎 | codegen 和优化能力有限 |
| MLIR/TVM/XLA | 基础设施强，工业谱系清楚 | 学习和集成成本高        |
| 直接调库         | 快速得到性能       | 编译器原理和错误边界不可见   |

## 工业源码阅读：要看合同，不只看名字

工业系统对比不能停在“某框架也有 graph”。源码阅读要抽取同一组问题，才能和本书轻量 IR 对齐。

| 系统             | 阅读入口问题                                 | 应抽取的证据                                                        |
|----------------|----------------------------------------|---------------------------------------------------------------|
| ggml/llama.cpp | tensor、quant block、backend op 怎样描述     | tensor descriptor、quant layout、kernel dispatch、backend buffer |
| oneDNN         | primitive descriptor 如何选择 layout       | memory desc、primitive attr、post-op、reorder 条件                 |
| MLIR           | operation、dialect、pass 如何声明合法性         | op verifier、type inference、pass pipeline、pattern rewrite      |
| TVM            | schedule 如何从 tensor expression 走到 loop | lowered IR、tile、vectorize、memory scope                        |
| vLLM           | paged KV 和 scheduler 如何绑定              | block table、sequence group、admission、continuous batching      |

阅读报告必须回答三件事。第一，它把语义事实放在哪里：shape、dtype、layout、quant、state、device。第二，它把机器事实放在哪里：tile、vector width、workspace、backend capability、memory descriptor。第三，它如何失败：verifier 拒绝、fallback、runtime error、profile 标记还是 silent wrong answer。只列类名和文件名，不算工业拆解。

Listing 3.87 工业对照报告模板

```

system:
 name
 commit_or_version
 inspected_files

semantic_contract:
 tensor_shape_dtype_layout
 quant_descriptor
 stateful_edges

machine_contract:
 backend_dispatch
 memory_descriptor
 kernel_or_schedule_descriptor

failure_contract:
 verifier_or_check
 fallback_path
 debug_or_profile_artifact

lesson_for_minillm_cpu:
 adopt
 reject
 reason

```

这个模板能防止“工业对标”变成贴 logo。比如 oneDNN 的重点不是名字叫 primitive，而是 primitive descriptor 把 layout、post-op、reorder 和 kernel 选择变成可检查合同；vLLM 的重点不是“有 KV cache”，而是 block table、sequence state 和 continuous batching 共同定义了 serving 状态。

## 实验：pass 前后 IR dump

### 习题与作业

实验一：实现或设计一个 `EliminateRedundantReshape` pass。给出 pass 前 IR、pass 后 IR、preserved analyses、invalidated analyses 和 verifier 输出。

实验二：实现或设计 RMSNorm+Linear fusion 的 legality checker。构造四个拒绝用例：shape 不匹配、layout 非 contiguous、debug dump 要求 materialize、跨 KV state edge。

实验三：给 memory planner 一个 value lifetime 表，手工做 interval coloring。标出哪些 buffer 可复用，哪些因为 oracle/state/alias 不可复用。

实验四：阅读 ggml/llama.cpp、oneDNN 或 MLIR 中一个相近概念。报告必须回答：它如何表达 tensor/layout/state/pass 或 primitive，本书轻量 IR 借鉴哪一点，拒绝哪一点。

## C++20 实现边界

工程上，AI 编译器模块可以按下面方式组织：

Listing 3.88 AI 编译器模块组织建议

```
include/ai/graph/
 graph_ir.hpp
 tensor_desc.hpp
 verifier.hpp

include/ai/compile/
 pass.hpp
 pass_pipeline.hpp
 analyses.hpp
 memory_plan.hpp
 lowering.hpp
 executable_plan.hpp

src/compile/analysis/
 use_def_analysis.cpp
 liveness_analysis.cpp
 cost_analysis.cpp

src/compile/passes/
 fuse_rmsnorm_linear.cpp
 eliminate_redundant_reshape.cpp
 quant_lowering.cpp

src/compile/lowering/
 cpu_lowering.cpp
 gpu_lowering.cpp
```

接口上要保持几条纪律。pass 接收已验证 IR，返回改写结果和诊断；analysis 产出不可变事实，不偷偷改图；lowering 消费 descriptor 和 analysis，不重新解析模型文件；executable plan 是 runtime 的稳定输入，不暴露临时编译器内部指针。C++20 中可以用 `std::span` 表示节点序列和 value 序列，用 RAII 管理 arena 和 backend 资源，用显式 enum 和小值对象表达 opcode、dtype、layout 和 backend capability。

## 硬验收

读完本节，应该能真正理解 AI 编译器的内部结构：

- IR 是可检查、可改写的中间账本，不是为了抽象而抽象。
- Graph IR 表达算子依赖和 KV cache 等状态边。
- Tensor IR 表达 shape、stride、layout、dtype、device 和 quant descriptor。
- Loop IR 或 LoopPlanDesc 表达 tile、vector、tail、thread 和 packed weight。
- pass 必须声明前置条件、后置条件、依赖分析和失效分析。
- lowering 要分阶段从语义计划走到 CPU/GPU 后端物理计划。
- executable plan 固化 segment、kernel、arena、workspace、state binding、oracle 和 profiler。
- verifier 要贯穿 Graph、Tensor、Pass、Memory、Backend 和 Plan 每一层。

这就是把编译原理落到 AI infra 的方式：不是背名词，而是用 IR 和 pass 把模型语义、量化合同、内存生命周期和后端机器事实串成一条可验证的工程流水线。

## 第 4 章

# C++20 CPU/GPU 后端优化、工业对标与验收

## 4.1 CPU 后端优化细讲：地址流、SIMD、线程、NUMA 与 C++20 工程边界

### 后端优化的第一原则：先解释慢在哪里

CPU 后端优化很容易被写成一串术语：AVX、packing、thread pool、NUMA、prefetch。对初学者来说，这些词如果没有机器账本支撑，只会变成新的黑箱。真正的优化要先回答一个朴素问题：这段推理为什么慢？

对 7B 级 decoder-only 模型，慢可能来自不同位置。prefill 阶段可能慢在大 GEMM、attention 和 activation 峰值；decode 阶段可能慢在 GEMV、KV cache 读流、小 kernel 调度和线程 barrier；量化路径可能慢在 int4 解包、scale 读取和 requantize；多线程可能慢在 false sharing、内存带宽和 NUMA 远端访问。优化不能一上来就写 intrinsic，而要先用 profiler 拆开这些来源。

### 核心思想

CPU 后端优化不是“把代码写得像汇编”，而是建立地址流、指令流、内存层级、线程调度和数值合同之间的证据链。每个优化都要能说明它减少了哪种机器成本。

CPU 后端的实现层同时受三类合同约束：C++ 对象生命周期和 ABI 边界，cache/TLB/SIMD/线程/NUMA 的机器成本，以及张量、量化、KV cache 和 executable plan 的推理语义。后端优化必须把这三类合同同时写进报告。

### C++20 后端对象：所有权和视图必须分开

后端代码最先要避免的是裸指针混乱。权重、packed weight、activation、KV cache 和 workspace 生命周期不同。C++20 中应该把所有权对象和非拥有视图区分开。

Listing 4.1 后端对象的所有权边界

```
Owned objects:
AlignedBuffer owns CPU aligned memory
MappedFile owns file mapping
PackedWeight owns backend-specific packed bytes
ThreadPool owns worker threads
ProfilerTrace owns event storage

Non-owning views:
TensorView span + shape + stride + dtype
ByteSpan std::span<std::byte>
KernelArgs views bound for one call
```

这条边界可以避免两个常见错误。第一，kernel 把某个临时指针缓存到对象里，arena 下一次复用地址后读到错误数据。第二，某个 descriptor 以为自己拥有 scale buffer，实际 scale buffer 已经随加载临时对象释放。所有权对象用 RAII 管理生命周期；kernel 只拿当次调用有效的 view。

### 常见误区

高性能代码也必须有清楚所有权。越是 SIMD、线程和异步后端，越不能依赖“这个指针应该还活着”的隐含约定。

### TensorView: 地址公式要显式

一个 tensor view 至少要包含元素类型、shape、stride 和数据指针。连续内存只是特殊情况。后端 kernel 可以要求某些输入连续,但这个要求必须写在 kernel descriptor 中,由 verifier 检查。

Listing 4.2 TensorView 的语义字段

```
TensorView:
 data
 dtype
 shape
 stride
 layout
 device
 alignment
```

地址公式要能从 view 推导出来:

Listing 4.3 二维 view 的地址公式

```
address(i, j) = base + (i * stride0 + j * stride1) * sizeof(element)
```

为什么这很重要? 因为很多优化只对连续内存成立。若 `stride1=1`, 内层循环可以顺序 load; 若 `stride1` 很大, SIMD load 可能变成 gather 或标量读; 若输出 stride 不连续, store 也会变慢。第一册讲过指针和对象布局; 在张量后端中, 这些知识直接决定 kernel 合同。

### GEMV 和 GEMM: prefill 与 decode 的不同账本

prefill 通常处理多个 prompt token, 线性层形态接近 GEMM:

Listing 4.4 prefill linear 的抽象形态

$$X[T, H] * W[0, H]^T \rightarrow Y[T, 0]$$

decode 通常一次处理一个新 token, 形态更接近 GEMV:

Listing 4.5 decode linear 的抽象形态

$$x[H] * W[0, H]^T \rightarrow y[0]$$

这两个形态的优化重点不同。GEMM 有更多数据复用机会, tile 可以同时复用 X 和 W; GEMV 中输入向量  $x$  较小, 可以驻留 cache, 但权重每步都要读很多, 常常受权重带宽和 packed layout 影响。一个只为 GEMM 优化的 kernel, 不一定适合 decode; 一个只为 GEMV 优化的 kernel, 也不一定吃满 prefill。

| 阶段        | 主要形态                | 优化重点                             |
|-----------|---------------------|----------------------------------|
| prefill   | GEMM / batched GEMM | tile 复用、cache blocking、线程吞吐      |
| decode    | GEMV / small GEMM   | 权重读流、packing、低调度开销、KV cache      |
| MLP       | 大线性层组合              | gate/up/down 的融合边界和 activation 流 |
| attention | softmax + KV 读取     | 数值稳定、历史读流、context 分档             |

所以 executable plan 应该区分 prefill plan 和 decode plan。后端优化报告也应该分别给出 prefill



tokens/s、time to first token、decode tokens/s 和单 token latency。

### packing: 让热循环读到它想读的顺序

模型文件布局通常服务于通用存储，不服务于某个 SIMD micro-kernel。packing 的目标是把权重重排成热循环顺序读取的布局。对 decode GEMV，常见目标是让一个 output tile 的若干行权重按  $K$  方向连续读取，便于向量 load 和累加多个输出。

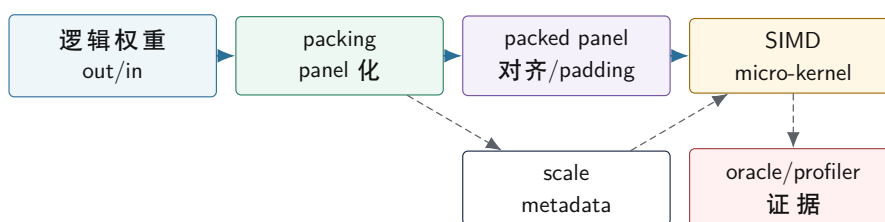
Listing 4.6 packing 前后的思维差异

```
logical:
 weight[out, in]

packed for kernel:
 panel[out_tile, in_tile, lane]
 scales[out_tile, group]
 tail padding for vector width
```

packing 的收益来自减少热路径地址计算、增加顺序读、避免跨 cache line 的随机访问，并让 scale metadata 与权重 code 靠近。但 packing 也有成本：准备时间、额外内存、可能的 TLB 压力、诊断复杂度。因此报告要区分 `pack_time_ms`、`packed_bytes` 和热路径收益。

#### CPU packed weight 与 micro-kernel 的关系



图示：本图要证明的命题是：packing 是 kernel 合同的一部分，权重 panel、scale metadata 和 profiler/oracle 必须一起设计。

### SIMD micro-kernel: 先设计寄存器账本

SIMD 优化不是把循环改成 intrinsic。要先设计寄存器账本：哪些寄存器放输入向量，哪些放权重，哪些放 accumulator，scale 在哪里乘，尾部怎样处理。以 fp32 GEMV 为例，一个 micro-kernel 可能同时累加 4 个输出通道：

Listing 4.7 GEMV micro-kernel 的寄存器账本

```
acc0, acc1, acc2, acc3 = 0
for k in 0..H step vector_width:
 x = load input[k]
 w0 = load weight[out0, k]
 w1 = load weight[out1, k]
 w2 = load weight[out2, k]
 w3 = load weight[out3, k]
 acc0 += x * w0
 acc1 += x * w1
 acc2 += x * w2
 acc3 += x * w3
horizontal_reduce acc0..acc3
```

对 int8 或 int4，账本更复杂。int8 可能使用 dot-product 指令或扩展到更宽类型累加；int4 要先解包 nibble，做符号扩展或 zero point 处理，再乘 scale 或进入整数累加。若 scale 每 64 个输入共享一次，micro-kernel 要在 group 边界加载 scale，并保证 packed layout 与 group 边界一致。

| 路径           | 热点                 | 主要风险                 |
|--------------|--------------------|----------------------|
| fp32/fp16 风格 | load、FMA、reduce    | accumulator 依赖链、tail |
| int8 dot     | dot 指令、int32 累加    | requantize、饱和、scale  |
| int4 dequant | nibble 解包、scale 乘法 | 解包开销、metadata 读流     |
| KV attention | 历史 K/V 读取、softmax  | context 增长、数值稳定      |

micro-kernel descriptor 必须声明这些合同：ISA、tile、alignment、tail、accumulator dtype、输出 dtype、quant group size 和误差容忍。否则 lowering、verifier、oracle 和 benchmark 都无法正确选择和解释它。

### int8 linear 的算术合同

int8 权重量化的 CPU 内核不能只写“把 int8 乘回 float”。至少有两条路线。第一条是教学和 reference 友好的路线：输入仍是 fp32/fp16，权重 int8 反量化成浮点参与 FMA。第二条是性能路线：输入也量化或分块量化，使用整数 dot 指令累加到 int32，再按 scale 还原到目标 dtype。

Listing 4.8 float input + int8 weight 的合同

```
real_w[out, k] = int8_w[out, k] * scale[out or group]
acc_float += input_float[k] * real_w[out, k]
```

这条路线容易验证，但每个权重都要转换或乘 scale。若 scale 是 per-channel，scale 读取较轻；若 scale 是 per-group，内层每到 group 边界要换 scale。若 scale 与 packed weight 不同布局，cache locality 会变差。

整数 dot 路线可以写成：

Listing 4.9 int8 dot + int32 accumulator 的合同

```
q_x = quantize(input_float, input_scale)
q_w = int8_w
acc_i32 += int32(q_x) * int32(q_w)
real_acc = acc_i32 * input_scale * weight_scale
output = cast_or_requantize(real_acc)
```

整数 dot 的难点更多：输入 scale 从哪里来，activation 是否逐 token 量化，zero point 是否为 0，accumulator 是否溢出，输出是 fp16/fp32 还是再量化。它可能更快，但必须有更强的 verifier 和 oracle。工程实现应先让 float-input/int8-weight 路径稳定，再引入 activation quantization 和整数 dot。

### 常见误区

不要把 int8 weight-only 和 int8 activation 混成一个词。前者主要节省权重容量和读带宽；后者改变输入表示、accumulator 合同和 requantize 路径，正确性风险更高。

### 用硬件计数器判断问题类型

CPU 优化不能只看 wall time。至少要能把慢分成几类：前端指令供应不足、后端执行端口饱和、cache miss、TLB miss、分支或同步。不同机器可用事件不同，但思路一致。

| 现象                  | 可能原因                       | 下一步                      |
|---------------------|----------------------------|--------------------------|
| IPC 低且 cache miss 高 | 权重/KV 读流不连续                | 查 layout、packing、page 大小 |
| IPC 低但 miss 不高      | 依赖链、指令混乱、解包开销              | 看反汇编和 uops               |
| 多线程不扩展              | 带宽饱和、barrier、false sharing | 改分区和线程数                  |
| 长上下文尾延迟高            | KV page 分散、TLB miss        | 查 paged KV 和大页策略         |
| int4 不如 int8 快      | 解包和 scale 读取压过带宽收益         | 重排 scale 或改 kernel       |

benchmark 报告里最好同时写“性能数字”和“解释数字”。例如：`decodetokens/s` 提升 18%，同时 LLC miss 降低、pack time 增加多少、logits top-k 是否变化。只有这样，优化才不是偶然跑快一次。

### tail path: 不要把边界条件当小事

真实模型 shape 不总是整齐满足 vector width 或 tile size。即使 hidden size 常常是 4096，intermediate size、vocab size、head 数、group size、batch 和 prompt 长度也可能产生尾部。tail path 写错会导致越界读、漏算或错误 padding。

Listing 4.10 tail path 需要覆盖的情况

```
K dimension not divisible by vector width
output channels not divisible by tile_n
group size does not divide logical tail
prompt token count not divisible by tile_m
vocab size not divisible by sampler block
```

优化代码常见坏味道是：主路径很快，tail path 用一堆临时判断散在热循环里。更好的做法是把 tail policy 写进 descriptor，并在 lowering 阶段选择专门主路径和尾部路径。测试必须覆盖 tail shape，不能只测漂亮的 `4096x4096`。

### 线程划分：decode 不一定适合细粒度并行

CPU 线程优化要先看任务粒度。prefill 的 GEMM 和 attention 块通常有足够工作量，可以按 token、output channel、head 或 tile 划分。decode 的单 token 路径包含许多小 kernel，若每个 kernel 都投递一批小任务，barrier 和线程唤醒可能抵消计算收益。

Listing 4.11 线程调度要记录的字段

```
ThreadPlan:
 stage: prefill | decode
 partition_axis: token | output_channel | head | layer | batch
 min_work_per_task
 barrier_count
 scratch_per_thread
 output_write_policy
 affinity_policy
```

false sharing 是一个具体风险。若多个线程写相邻很小的输出或 profiler counter，它们可能反复抢同一 cache line。第二册讲过 cache coherence；在推理后端里，它会表现为多线程不升反降。输出分区、per-thread scratch、counter padding 和归约策略都要考虑 cache line。

### 常见误区

不要用“CPU 利用率接近 100%”证明推理快。线程都忙可能是在抢 cache line、等内存带宽或反复 barrier。真正指标是端到端 latency、tokens/s、硬件计数器和可解释 scaling 曲线。

## NUMA、TLB 与大页：长上下文会放大内存事实

当模型权重、packed weight 和 KV cache 变大，NUMA 和 TLB 开始影响尾延迟。若线程跑在一个 NUMA node，而权重页主要分配在另一个 node，远端访问会拖慢。若 KV cache 分散在很多小页，长上下文 attention 会放大 TLB miss。若多请求同时增长 KV cache，内存分配策略还会影响碎片和局部性。

后端报告至少要记录：

- CPU 型号、核心数、NUMA node、线程亲和策略。
- 权重和 KV cache 的分配策略，是否 first-touch。
- 是否使用大页或特定页大小。
- 长 prompt 和长 decode 下的 tail latency。
- cache miss、TLB miss、远端访问等可用硬件计数器。

这部分不能凭空优化。先测，再做策略。比如固定线程亲和可能改善稳定性，也可能降低操作系统调度弹性；大页可能减少 TLB miss，也可能增加分配失败或内存浪费。教材强调的是把这些选择纳入报告，而不是给出一个永远正确的开关。

## KV cache CPU 路径：读流比写人更关键

KV cache 每生成一个 token 写一次当前 K/V，但 decode attention 每步要读取历史 K/V。上下文越长，读流越大。CPU 后端要围绕读取模式设计 layout。

Listing 4.12 decode attention 的读流

```
for head in heads:
 for pos in 0..context_length:
 k = load K[layer, head, pos, dim]
 score[pos] = dot(q[head], k)
 probs = softmax(score)
 for pos in 0..context_length:
 v = load V[layer, head, pos, dim]
 out[head] += probs[pos] * v
```

这里至少有三类优化。第一，layout 让 **pos, dim** 读取尽量连续，减少跨页和跨 cache line。第二，按 context length 分档，小上下文用简单路径，长上下文用分块路径。第三，若 KV cache 量化，需要把 dequant 和 attention 结合，但不能让 scale metadata 破坏读流。

数值上，softmax 需要稳定实现，通常要先减最大值。优化 attention 时不能只看速度，还要比较 logits 和生成质量。一个更快但数值不稳定的 attention kernel，会在长上下文下放大误差。

## benchmark harness：测量要能复现

后端优化最终要落到 benchmark。一个合格 harness 至少固定这些变量：

Listing 4.13 CPU 后端 benchmark 输入

```
model_or_fixture_id
quantization_profile
backend_plan_id
cpu_feature_path
thread_count
affinity_policy
prompt_set_id
prompt_lengths
decode_lengths
context_length_buckets
warmup_iterations
```

measurement\_iterations

输出要同时包含性能和正确性：

Listing 4.14 CPU 后端 benchmark 输出

```
load_time_ms
pack_time_ms
prefill_tokens_per_s
time_to_first_token_ms
decode_tokens_per_s
p50_decode_latency_ms
p95_decode_latency_ms
arena_peak_bytes
kv_cache_bytes
max_logit_abs_error
topk_changed_count
hardware_counters_optional
```

测量本身也要避免常见误区：没有 warmup、把模型加载混入 decode、把 packing 成本藏进首次 token、只测一个短 prompt、没有固定线程数、没有记录 CPU 频率状态、只报最好一次。第一册已经讲过可信测量；这里要把同样纪律应用到 AI 后端。

自研后端怎样对标成熟框架

自研后端不需要一开始超过成熟框架，但必须能分阶段解释差距。对标可以按下面的层次组织：

| 阶段           | 本项目目标       | 对标重点                |
|--------------|-------------|---------------------|
| reference    | 正确、可解释      | logits 与参考一致        |
| baseline     | 能端到端跑固定模型片段 | 找到最大瓶颈差距            |
| packing/SIMD | 单算子接近合理吞吐   | microbenchmark 和反汇编 |
| 线程化          | 多核扩展稳定      | scaling 曲线和尾延迟      |
| 量化           | 容量和速度同时改善   | 质量报告和 tokens/s      |
| runtime plan | 端到端稳定       | prefill/decode 分项差距 |

对标报告应该同时写“我们还差在哪里”。例如：prefill 接近成熟框架 70%，decode 只有 40%，原因可能是 KV cache attention 读流没有优化；或者 int4 权重容量下降明显，但 scale metadata 布局导致解包路径慢。这样的报告比一个孤立 tokens/s 数字更有工程价值。

优化判定表：看到现象后查什么

后端优化最需要的是排查顺序。下面这张表把常见现象、优先假设和验证动作连起来。

| 现象                | 优先假设                                   | 验证动作                                        |
|-------------------|----------------------------------------|---------------------------------------------|
| decode batch 1 慢  | 权重读流或小 kernel 调度                       | 算 weight bytes、看 pack layout、拆 kernel count |
| 长上下文变慢明显          | KV cache 读流/TLB/page table             | 按 context bucket 画 latency, 查 TLB/cache     |
| int4 不如 int8      | 解包和 scale metadata 太重                  | 分开测 unpack、scale load、FMA 时间                |
| 多线程不扩展            | 内存带宽、barrier、false sharing             | 画线程数曲线, 记录 barrier wait                     |
| prefill 快但 TTFT 慢 | tokenizer、prepare、packing 或 queue wait | 分拆 load、prepare、prefill、sampler             |
| GPU kernel 快但端到端慢 | launch、copy、sync 或 sampler             | 看 StepTrace, 不只看 kernel profiler            |

这张表的用法很直接：不要先改代码，先证明假设。若 decode 慢但 weight bytes 下限已经接近实测，继续优化 FMA 指令可能收益有限；若 int4 慢在 scale 读取，换更激进的位宽没有意义；若 p95 主要是 queue wait，kernel micro-optimization 不会解决业务 SLO。

### 硬验收

CPU 后端优化的验收点可以压成下面几项：

- 后端对象要分清 RAII 所有权和 `std::span`/TensorView 非拥有视图。
- TensorView 的 shape、stride、layout 和 alignment 决定地址公式和 kernel 合同。
- prefill 更像 GEMM，decode 更像 GEMV，两者应使用不同 plan 和指标。
- packing 要同时组织权重 panel、scale metadata、padding、对齐和诊断映射。
- SIMD micro-kernel 要先设计寄存器账本，再声明 dtype、tile、tail、accumulator 和误差合同。
- tail path 是必须测试的正确性边界，不是附属小分支。
- 线程、NUMA、TLB 和 KV cache 读流必须用 profiler 和硬件事实解释。
- benchmark harness 要固定模型、量化、线程、prompt、context、warmup 和正确性指标。

这章的核心结论很简单：高质量 AI infra 后端不是某段神秘快代码，而是一套从 C++20 对象生命周期到机器地址流、从 SIMD micro-kernel 到 runtime plan、从 oracle 到 profiler 的完整证据系统。

## 4.2 GPU 硬件基础：SM、warp、显存层级、tensor core 与推理瓶颈

### 为什么第三册必须讲 GPU 硬件

只讲 GPU API 和 stream 还不够。推理引擎的 GPU 后端最终跑在硬件上：线程如何成组执行，访存如何合并，shared memory 为什么快，tensor core 适合什么形态，显存带宽和 PCIe 传输怎样限制 decode，这些都会决定 AI 编译器 lowering 的选择。如果不了解这些硬件事实，GPU 后端很容易停在“能 launch kernel”的层面。

可以先把 GPU 和第二册里的 CPU 对比。CPU 少量强核心，靠大 cache、乱序执行、分支预测和 SIMD 处理复杂控制流；GPU 大量较简单执行单元，靠海量线程隐藏内存延迟，适合规则、密集、数据并行的工作。两者都要关心内存层级和并行，但并行粒度和瓶颈不同。

#### 核心思想

GPU 快不快，取决于工作是否能匹配它的执行模型：大量线程、规则访存、高算术强度、少同步、少 host-device 往返。prefill 往往更匹配，decode 单 token 往往更难匹配。

### SM/CU：GPU 的基本执行岛

不同厂商术语不同，NVIDIA 常说 SM，AMD 常说 CU。本册用 SM/CU 表示 GPU 上一组局部执行资源：它包含调度器、寄存器文件、若干向量/标量执行单元、shared memory 或 local data



share、load/store 单元，以及可能的 tensor core 或 matrix core。

一个 kernel launch 后，工作会被切成许多 thread block 或 workgroup。每个 block 被调度到某个 SM/CU 上执行。一个 block 内的线程共享一块 shared memory，并能做 block 内同步；不同 block 通常不能直接同步，只能通过全局内存和 kernel 边界间接协作。

Listing 4.15 GPU 执行层级的简化账本

```
kernel launch
 grid: many blocks
 block/workgroup
 warps/wavefronts
 lanes/threads
 registers
 shared memory
 global memory
```

这解释了为什么 GPU lowering 要选择 block 形状。一个 attention kernel 可以让一个 block 处理一个 head 的一段 position，也可以让一个 block 处理一组 token 和 head。选择不同，shared memory 用量、寄存器压力、访存合并和 occupancy 都会变。

#### warp/wavefront：线程不是完全独立执行

GPU 上的线程通常以 warp 或 wavefront 为单位执行。可以把它理解成“一组 lane 同时执行同一条指令，但每个 lane 处理不同数据”。这和 CPU SIMD 有相似性：CPU SIMD 是一条指令处理多个 lane；GPU warp 是多个线程 lockstep 执行同一指令。区别是 GPU 编程模型把 lane 暴露成线程。

如果 warp 内线程走不同分支，会发生 divergence：一部分 lane 先执行分支 A，另一部分 lane 再执行分支 B，吞吐下降。若相邻 lane 访问连续地址，硬件可以合并内存请求；若访问分散，global memory 事务变多。

| 现象                          | 类比                             | 推理后端影响                                              |
|-----------------------------|--------------------------------|-----------------------------------------------------|
| warp lockstep<br>divergence | SIMD lane 同步执行<br>lane mask 分裂 | 分支越规则越好<br>sampler/稀疏路径不适合热 kernel                  |
| coalescing<br>occupancy     | 连续向量 load<br>同时驻留的 warp 数      | KV cache 和权重 layout 要顺序<br>寄存器/shared memory 过多会降并发 |

第二册讲 SIMD 时强调“相邻 lane 和规则控制流”。GPU 这里是同一原则的放大版。prefill 的大矩阵乘通常规则；decode 中某些小 batch、变长 context、条件路径和 host 同步更容易破坏规则性。

#### coalescing：连续地址决定一次访存能带回多少有效数据

GPU global memory 带宽很高，但前提是 warp/wavefront 里的 lane 访问足够连续。若 32 个 lane 分别访问相邻的 fp16 元素，硬件可以用较少内存事务带回一整段数据；若 32 个 lane 访问相隔很远的位置，就会产生更多事务，实际带宽下降。

Listing 4.16 coalesced 与 scattered 访问的差异

```
coalesced:
 lane 0 reads base + 0
 lane 1 reads base + 1
 lane 2 reads base + 2
 ...
```

```

scattered:
 lane 0 reads page_table[a]
 lane 1 reads page_table[b]
 lane 2 reads page_table[c]
 ...

```

KV cache layout 直接影响 coalescing。若一个 warp 负责同一个 head 的连续 `head_dim`，那么 `head_dim` 连续布局有利；若一个 warp 负责多个 position 的同一维度，那么 position 连续更有利。paged KV 会引入 page table，长上下文和多 session 调度更灵活，但 kernel 必须小心把 page 内连续读和跨页边界分开处理。

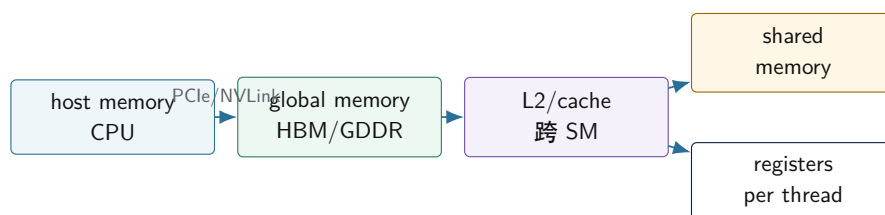
### 常见误区

GPU 上“逻辑上连续”不等于“lane 访问连续”。descriptor 只写 shape 不够，lowering 必须知道 warp mapping 和物理 stride。

### 显存层级：global、shared、register 和 cache

GPU 的 global memory 容量大、带宽高，但延迟也高。shared memory 在 SM/CU 内部，容量小但速度快，适合 tile 复用。register 是每个线程私有，最快但数量有限。L2 cache 连接多个 SM/CU，L1 或 texture cache 细节随架构不同。

#### GPU 推理中的内存层级



图示：本图要证明的命题是：GPU kernel 性能不只取决于算力，还取决于数据在哪一层内存里移动。

对推理来说，权重和 KV cache 通常在 global memory；tile 化 GEMM 会把输入/权重片段搬到 shared memory 或寄存器；accumulator 放在寄存器；logits 可能需要 staging 到 host。若每个 decode step 都把完整 logits 从 global memory 拷回 host，PCIe 或同步开销可能比某个小 kernel 更显眼。

### shared memory：它解决的是复用，不是容量

shared memory 适合保存一个 block 内多个线程都会反复使用的小 tile。矩阵乘中，一个 input tile 和 weight tile 被多个 lane 复用，搬到 shared memory 后可以减少 global memory 读流。attention 中，也可以把 K/V 的一段 block 或 softmax 中间量放到 shared memory，减少重复读写。

但 shared memory 容量小，而且占用越多，每个 SM/CU 能同时驻留的 block 可能越少。用它之前要问三个问题：

- 这个 tile 会被多少次复用，是否值得从 global memory 搬进来。
- tile 大小是否会让 occupancy 明显下降。
- shared memory 的访问是否会发生 bank conflict。

例如 decode attention 读取历史 K/V 时，如果每个 K/V 元素只被一个 query head 用一次，把整段历史搬进 shared memory 可能不划算；若同一 K/V block 会被多个 query head 或多个 batch row 复用，就有价值。GQA 的共享 K/V head 也会改变这个判断：多个 query head 读同一个 K/V head，理论上有复用机会，但具体能否利用取决于 kernel mapping。

**tensor core: 不是所有矩阵都天然吃满**

现代 GPU 常有 tensor core 或 matrix core，专门加速小矩阵块乘加。它们对 dtype、矩阵形状、对齐和 layout 有要求，常见于 fp16/bf16/tf32/int8/fp8 等路径。prefill 的大 GEMM 更容易使用 tensor core；decode 单 token 的 GEMV 不一定能有效使用。

| 场景                   | tensor core 适配性 | 关键条件                      |
|----------------------|-----------------|---------------------------|
| prefill QKV/MLP GEMM | 通常较好            | batch/token 足够大，layout 对齐 |
| decode GEMV          | 通常较难            | M 维太小，launch 和访存占比高       |
| int4 权重              | 取决于格式支持         | packed layout 和 scale 处理  |
| attention            | 取决于 kernel 设计   | softmax、mask、KV 读流复杂      |

这解释了一个常见现象：GPU prefill 很快，但 decode tokens/s 未必按峰值算力提升。decode 阶段每次只有一个 token，矩阵形态小，KV cache 读历史，logits 还要进 sampler。算力不是唯一瓶颈。

**用 roofline 判断 GPU 推理瓶颈**

roofline 的核心是算术强度：

$$AI = \frac{FLOPs}{Bytes}$$

若 AI 很低，即使 GPU 峰值 FLOPS 很高，也会被显存带宽限制；若 AI 很高，才更可能接近计算单元上限。prefill GEMM 的权重 tile 被多个 token 行复用，算术强度高；decode GEMV 的 batch 通常很小，权重复用弱，算术强度低。KV attention 还要随上下文读取历史 K/V，常常更像带宽问题。

| 路径               | 算术强度直觉         | 常见瓶颈                              |
|------------------|----------------|-----------------------------------|
| prefill GEMM     | 高，tile 复用好     | tensor core、workspace、activation  |
| decode GEMV      | 低到中，权重复用弱      | 显存带宽、launch、小矩阵效率                 |
| decode attention | 随 context 读 KV | KV 布局、带宽、softmax 归约               |
| logits staging   | 计算少，传输和同步明显    | host-device copy、sampler boundary |

一个简单反证方法是先算字节下限。若某个 decode step 至少要读 4GB 权重和 KV，而有效显存带宽是 800GB/s，那么单步光读数据就至少 5ms，还没算 kernel launch、softmax、MLP、staging 和 sampler。若 benchmark 报告低于这个数量级，就要检查是否统计了完整 step、是否用了 batch、是否跳过了某些层、是否缓存命中口径不同。

**核心思想**

GPU 性能分析不能只问“有没有 tensor core”。先算 FLOPs、bytes、launch、copy、sync，再判断瓶颈更像计算、带宽、调度还是 host-device 边界。

**occupancy: 并发多不等于一定快**

occupancy 指一个 SM/CU 上同时驻留的 warp/block 相对硬件上限的比例。高 occupancy 可以帮助隐藏 global memory 延迟，但不是越高越好。寄存器用太多、shared memory 用太多、block 太大，都会降低 occupancy；反过来，为了提高 occupancy 把 tile 做得太小，也可能降低数据复用。

推理 kernel 要平衡：

- 寄存器：accumulator 多，复用好，但 occupancy 可能降。
- shared memory：tile 大，global memory 读少，但可驻留 block 少。

- block 大小：并行度高，但同步和资源压力上升。
- 算术强度：每读一个字节做多少计算，决定更像算力瓶颈还是带宽瓶颈。

因此 GPU 后端的 profiler 不应只报告 occupancy。一个 kernel occupancy 很高但访存不合并，仍然慢；一个 tensor core kernel occupancy 不高但计算吞吐高，也可能很快。指标要和具体瓶颈一起看。

### warp 映射和 tensor core 账本

GPU kernel 设计必须写清 warp mapping: lane 维度对应 token、head、head\_dim、output channel 还是 K tile。不同映射会改变 coalescing、shared memory 复用、reduction 方式和 divergence。只写 `blockDim=256` 没有解释力。

| warp mapping                    | 优点                   | 风险                           |
|---------------------------------|----------------------|------------------------------|
| lane 覆盖连续 <code>head_dim</code> | K/V 读连续，coalescing 好 | head_dim 小时并行度有限             |
| lane 覆盖 position                | 长上下文并行度高             | softmax reduction 和 mask 更复杂 |
| warp 覆盖 output channels         | GEMV 权重读规则           | batch=1 时复用弱                 |
| warp 覆盖 batch rows              | 权重复用好                | 小 batch 或变长 session 难凑齐      |

**shared memory bank conflict** shared memory 虽然快，但不是“无成本数组”。它由多个 bank 组成，若一个 warp 的多个 lane 同时访问同一 bank 的不同地址，访问会被串行化或降效。Tile layout 要同时满足 global memory coalescing 和 shared memory bank 访问。一个从 global 读连续的布局，搬到 shared 后若按错误 stride 读取，仍可能慢。

**tensor core 的形状合同** Tensor core 通常执行固定小矩阵块的乘加，例如某种 `mxnxk` tile。后端要把大 GEMM 切成这些 tile，并满足 dtype、对齐、layout 和 accumulator 要求。若 decode 是 `M=1` 的 GEMV，很多 tensor core tile 的 M 维利用率很低；把多个 session 合批可以提高 M，但会引入 queue wait 和 KV binding 复杂度。

Listing 4.17 tensor core 使用前的账本

```
matrix shape:
 M = batch_or_tokens
 N = output_channels
 K = hidden

requirements:
 dtype supported by matrix core
 M/N/K aligned to tile or tail handled
 input/weight layout matches fragment loader
 accumulator dtype and scale path defined
 batch wait cost included in SLO
```

**register pressure 和 occupancy 的交换** 更大的 tile 会增加寄存器 accumulator，可能提升复用，也可能让每个 SM 驻留 warp 变少。更高 occupancy 能隐藏内存延迟，但如果每个 warp 都在读不合并的 global memory，occupancy 只是让更多慢请求同时排队。GPU 报告要同时写 registers per thread、shared memory per block、achieved occupancy、memory throughput 和 tensor core utilization。

### GPU profiler 指标怎样读

不同工具名字不同，但读法相近。不要孤立看一个百分比，而要把指标放回 kernel 目标。

| 指标                      | 可能说明什么         | 需要结合什么看                       |
|-------------------------|----------------|-------------------------------|
| achieved occupancy      | 驻留 warp 是否足够   | 寄存器、shared memory、block size  |
| memory throughput       | 是否接近带宽上限       | coalescing、cache hit、字节估算     |
| tensor core utilization | 矩阵单元是否忙        | dtype、layout、tile、GEMM 形态     |
| warp stall              | 等内存、同步或依赖      | global load、barrier、寄存器依赖     |
| launch count            | 小 kernel 数量    | decode step latency、fusion 机会 |
| copy time               | host-device 传输 | staging 大小、sampler 位置         |

例如 memory throughput 很高而 tensor core utilization 低，不一定是坏事；decode attention 可能本来就是读 KV 的带宽路径。反过来，occupancy 很高但 throughput 很低，可能是访问不合并、分支发散或 page table 间接访问导致。报告必须把这些指标和 executable plan 中的 kernel desc 对上。

### prefill 和 decode 的 GPU 瓶颈不同

把前面硬件事实放回推理链路，可以得到一个清晰判断：

| 阶段       | GPU 优势                          | 主要风险                                                    |
|----------|---------------------------------|---------------------------------------------------------|
| prefill  | 大 GEMM、并行 token、tensor core     | activation 峰值、attention workspace、显存占用                  |
| decode   | 权重常驻、KV cache 常驻、可融合小算子         | launch、同步、GEMV 小形态、KV 读流、logits staging                 |
| 采样<br>量化 | 可做 top-k 或局部 sampler<br>降低显存和带宽 | 完整 logits 回传和 CPU 同步<br>格式不支持、scale metadata、dequant 开销 |

这张表是 GPU lowering 的基础。prefill plan 应优先选择高吞吐 GEMM/attention；decode plan 应优先减少 launch 和 host-device 同步，优化 KV cache layout，并按 context length 分档。若把 prefill 的大矩阵思路直接套到 decode，通常会失望。

### GPU 硬件事实怎样进入 AI 编译器

GPU 硬件不应该散落成后端里的魔法数字。它应该进入 capability 和 kernel descriptor：

**Listing 4.18** GPU capability 和 kernel descriptor 字段

```
GpuCapability:
 backend_name
 max_threads_per_block
 warp_or_wavefront_size
 shared_memory_per_block
 register_file_limits
 supported_matrix_dtypes
 supported_quant_formats
 host_device_link

GpuKernelDesc:
 operator
 stage: prefill | decode
```

```

block_shape
warp_mapping
shared_memory_bytes
registers_estimate
supported_layouts
sync_points

```

编译器 lowering 使用这些字段选择 plan; verifier 用它们拒绝不支持的 shape 或 dtype; profiler 用它们解释瓶颈; benchmark 报告用它们保证可复现。这样 GPU 硬件知识就进入了工程合同，而不是停留在“这个 kernel 在我机器上更快”的经验。

### coalescing 的地址公式: warp 访问不是自然合并

GPU global memory 喜欢一个 warp 的 lane 访问连续、对齐、少量内存事务能覆盖的地址。若 lane  $i$  访问:

$$addr_i = base + (offset + i) \times sizeof(T)$$

通常更容易合并。若 lane  $i$  访问:

$$addr_i = base + index_i \times sizeof(T)$$

且  $index_i$  随机，硬件仍要发出多个内存事务。coalescing 不是“用了 GPU 就自动发生”，它来自 layout、warp mapping 和地址公式。

**Listing 4.19** KV cache 两种 warp 地址流

```

good for lane covers head_dim:
 lane i reads kv[layer][pos][kv_head][dim + i]
 addresses are contiguous inside head_dim

bad if lane covers random sessions:
 lane i reads kv[session_i][page_i][slot_i][dim]
 addresses may scatter across pages

```

paged KV cache 提高调度灵活性，但会让地址公式多一次 page table 间接。高质量 kernel 通常要把 page 内连续读和跨页边界分开处理，避免每个 lane 都走完全随机路径。

### shared memory bank conflict 的手算模型

shared memory 被拆成多个 bank。简化地说，若有  $B$  个 bank，地址按 word 映射到:

$$bank = word\_index \bmod B$$

若一个 warp 中多个 lane 同时访问同一 bank 的不同地址，就可能冲突。连续访问通常较好；按某个 stride 访问可能让所有 lane 落到少数 bank。

**Listing 4.20** bank conflict 的 stride 直觉

```

bank_count = 32

lane i reads shared[i]:
 bank = i mod 32
 lanes spread across banks

lane i reads shared[i * 32]:
 bank = 0 for all lanes

```



**worst conflict shape**

实际硬件有广播、bank 宽度和访问粒度等细节，但这个模型能解释为什么 shared memory tile layout 需要 padding 或 swizzle。把 global memory 读连续搬进 shared 只是第一步；shared 内部消费也要看 bank。

**occupancy calculator 的最小公式**

occupancy 受 threads per block、registers per thread、shared memory per block 和硬件上限共同限制。简化账本如下：

**Listing 4.21** occupancy 限制项

```
blocks_by_threads = max_threads_per_sm / threads_per_block
blocks_by_registers = registers_per_sm /
 (registers_per_thread * threads_per_block)
blocks_by_shared = shared_memory_per_sm / shared_memory_per_block
resident_blocks = min(blocks_by_threads,
 blocks_by_registers,
 blocks_by_shared,
 hardware_block_limit)
```

优化时常见交换是：增大 tile 提高复用，但增加 registers/shared，降低 resident blocks；减小 tile 提高 occupancy，但每个 global load 被复用次数下降。报告里只写 occupancy 百分比不够，必须写是哪一项限制了驻留。

| 限制项                 | 表面现象                      | 可能改法                     |
|---------------------|---------------------------|--------------------------|
| registers/thread 高  | resident warp 少，可能 spill  | 缩小 tile，分阶段累加            |
| shared/block 高      | resident block 少          | 压缩 tile，重用 shared buffer |
| threads/block 不合适   | 并行度或调度粒度差                 | 调整 block shape           |
| memory coalescing 差 | throughput 低、stall memory | 改 warp mapping/layout    |

**decode 小 batch 为什么难吃满 GPU**

LLM decode 常见痛点是 batch 小、每步依赖上一步 token、kernel 多、同步多。单 session decode 的 GEMV 形态  $M = 1$ ，很难像 prefill GEMM 那样提供大矩阵给 tensor core。即使权重常驻显存，每一步仍要调度多个 kernel，读大量权重和 KV，最后 sampler 可能还要同步 logits。

**Listing 4.22** 单 session decode 的 GPU 固定成本

```
for each token:
 launch or dispatch many small kernels
 read layer weights
 read KV cache up to context
 run reductions for attention
 produce logits
 synchronize sampler or copy top candidates
 feed next token to next step
```

连续合批能提高  $M$ ，让 GEMV 更像小 GEMM，也能摊薄 launch 成本；代价是 queue wait、KV binding、变长 session 和 SLO 复杂度。GPU backend 不能只追求吞吐，它必须和 serving scheduler 一起设计。

### host-device 边界和 sampler 位置

如果每步把完整 logits 从 GPU 拷回 CPU 再采样，词表大小  $V$  很大时会产生明显传输和同步成本。更好的路径可能是在 GPU 上做 logits processor、top-k/top-p 或至少取 top candidates，再只传小结果；但这会把采样语义、RNG 状态和可复现性带进 GPU plan。

| 采样位置              | 优点                | 风险                  |
|-------------------|-------------------|---------------------|
| CPU 完整 logits     | 易调试，复现简单          | 大拷贝、同步、TPOT 增加      |
| GPU top-k/top-p   | 传输少，减少同步          | kernel 复杂, RNG/排序合同 |
| 混合 top candidates | 折中，保留 CPU sampler | 需要误差和截断边界           |

这也是为什么 executable plan 要记录 sampler boundary。否则 benchmark 的 TPOT 可能测的是 GPU kernel，而用户体验卡在 logits staging 和 CPU sampler。

### GPU 报告的最小字段

GPU benchmark 报告至少包含：

Listing 4.23 GPU backend report 最小字段

```
model_shape
stage: prefill | decode
batch_size
context_length
kernel_count_per_token
selected_kernels
block_shape
warp_mapping
registers_per_thread
shared_memory_per_block
achieved_occupancy
memory_throughput
tensor_core_utilization
host_device_copy_bytes
sampler_boundary
correctness_oracle
```

这些字段让 GPU 优化回到证据链：kernel 是否真的对应目标阶段，shape 是否足够吃满硬件，访存是否合并，tensor core 是否被使用，host-device 是否拖尾，结果是否仍对。

## 4.3 状态机：一个 GPU kernel 怎样消耗资源

GPU 章节不能只背 thread、block、warp、SM。一次 kernel 从 launch 到完成，也是一台资源状态机：

| 状态            | 资源账本                            | 典型瓶颈                        |
|---------------|---------------------------------|-----------------------------|
| Launched      | grid/block、参数、stream            | launch overhead、依赖同步        |
| Resident      | block 占用 SM，分配寄存器/shared memory | occupancy 受寄存器或 shared 限制   |
| WarpScheduled | warp 被调度发射指令                    | divergence、指令供给             |
| MemoryIssued  | global/shared/register 访问发出     | coalescing、bank conflict、带宽 |
| ComputeIssued | ALU/tensor core 指令发出            | tile shape、dtype、数据准备       |
| Writeback     | 结果写回 global 或下一 kernel          | store 合并、同步、拷贝              |
| Completed     | event 可见，后续依赖解除                 | host-device 同步拖尾            |

这张表让 GPU 性能报告不再只写 “occupancy 低”。occupancy 低可能是寄存器太多，也可能是 shared memory 太多；occupancy 高也可能因为访存不合并而慢。必须把 resident blocks、warp scheduling、memory path 和 compute path 分开解释。

### 数学模型：occupancy 不是越高越好

设每个 SM 最大线程数为  $T_{max}$ ，最大 block 数为  $B_{max}$ ，寄存器总数为  $R_{sm}$ ，shared memory 为  $S_{sm}$ 。若每个 block 有  $T_b$  个线程，每线程用  $R_t$  个寄存器，每 block 用  $S_b$  shared memory，则 resident block 上限近似为：

$$B_{res} \leq \min \left( B_{max}, \left\lfloor \frac{T_{max}}{T_b} \right\rfloor, \left\lfloor \frac{R_{sm}}{T_b R_t} \right\rfloor, \left\lfloor \frac{S_{sm}}{S_b} \right\rfloor \right)$$

occupancy 是可驻留 warp 与硬件最大 warp 的比值。它的作用是隐藏 latency；但若 kernel 主要受 global memory 带宽限制，或者寄存器减少导致 spill，盲目提高 occupancy 可能更慢。GPU 报告要同时列 occupancy、register spill、memory throughput 和 achieved compute。

### coalescing 和 bank conflict 的具体反例

一个 warp 中第  $i$  个 lane 访问地址：

$$addr_i = base + (lane_i \times stride) \times sizeof(T)$$

当 stride=1 且地址对齐时，访问容易合并；当 stride 很大时，一个 warp 可能触碰许多 cache line 或 memory transaction。shared memory 也类似，若多个 lane 落到同一个 bank，访问会序列化或产生冲突。

Listing 4.24 GPU 访存事故复盘字段

```
symptom:
 kernel occupancy acceptable but memory throughput low

address model:
 lane i reads base + i * stride * sizeof(float)
 stride = hidden_size
 warp touches many cache lines

counter evidence:
 low global load efficiency or fallback timing evidence
 shared bank conflicts if available

fix:
 change layout or warp mapping
 tile into shared memory
 make lane contiguous on innermost dimension
```

这个反例对应 AI 里的真实问题：tensor layout 若让 warp lane 跨 head 或跨 token 跳跃访问，GPU 后端会把显存带宽浪费在不合并访问上。layout propagation 不是 compiler 装饰，而是 GPU 性能条件。

### 证据包：没有 GPU profiler 时也能先算地址

GPU 工程不能等到有完整 profiler 才开始严谨。先用地址账本排除明显错误。假设 warp 有 32 个 lane，元素是 fp16，cache/memory transaction 粗略按 128B 对齐段理解。若 lane  $i$  读取：

$$addr_i = base + i \times 2$$

32 个 lane 共读取 64B，通常落在一个 128B 段内，有效载荷至少在同一数量级。若 lane  $i$  读取：

$$addr_i = base + i \times 4096 \times 2$$

每个 lane 跨 8192B，可能触碰 32 个不同段。两者计算量相同，访存事务数量却不是同一级别。即使没有 Nsight 或 rocprof，这个地址模型也足以判定第二种 warp mapping 很可疑。

Listing 4.25 GPU 地址账本检查表

```
lane_mapping:
 lane id -> tensor index

address_formula:
 addr_i = base + logical_offset(i) * sizeof(dtype)

transaction_estimate:
 touched_segments
 useful_bytes
 loaded_bytes
 useful_ratio

required_profiler_fields:
 global load efficiency
 memory throughput
 dram transactions
 shared bank conflicts
```

这份账本和 AI compiler 的 layout propagation 直接相连。若 IR 只保存 shape，不保存 layout 和 stride，lowering 阶段无法判断 lane 是否连续；若 compiler 随意插入 transpose 或 view，GPU 后端可能从 coalesced 变成 scattered。

### occupancy 账本的可手算示例

设一个 SM 最多 2048 threads、最多 32 resident blocks、寄存器总数 65536，每个 block 256 threads，每线程 64 registers，不用 shared memory。resident block 上限为：

$$\left\lfloor \frac{2048}{256} \right\rfloor = 8$$

$$\left\lfloor \frac{65536}{256 \times 64} \right\rfloor = 4$$

因此寄存器把 resident blocks 限到 4。若把每线程寄存器降到 32，寄存器上限变成 8，但如果因此发生 spill，把局部值溢出到 global/local memory，性能可能更差。报告不能只写“occupancy 从 50% 提到 100%”，还要写 spill、memory throughput 和端到端阶段收益。

Listing 4.26 occupancy 报告中的拒绝条件

```
reject optimization if:
 occupancy increases but local memory spill appears
 achieved memory throughput drops
 correctness oracle cannot dump intermediate tensor
 TTFT/TPOT SLO worsens despite kernel microbenchmark win
```

### 工业取舍：tensor core、launch 和可复现性

tensor core 适合形状足够大、dtype/layout 符合要求、tile 能铺满的矩阵。prefill 常能满足；单 session decode 常常是小 batch GEMV，利用率差。把多个 session 合批可以提高 tile 利用，但会引

入排队等待、KV gather、变长 mask 和 SLO 取舍。若把 sampler 也搬到 GPU，还要处理 RNG 状态、top-k/top-p 排序稳定性和跨设备复现。

| 优化                   | 收益               | 新风险                             |
|----------------------|------------------|---------------------------------|
| continuous batching  | 提高矩阵形状和吞吐        | queue wait、KV 绑定复杂              |
| GPU sampler          | 减少 logits 拷贝和同步  | RNG/排序/复现合同复杂                   |
| fused kernel         | 降低 launch 和读写中间值 | debug/oracle materialization 变难 |
| tensor core lowering | 高吞吐              | dtype/layout/tile 限制，误差边界       |

## 实验：GPU 报告必须有反证输入

### 习题与作业

实验一：设计 prefill 与 decode 两组 shape。prefill 使用  $T = 128$  或更大，decode 使用  $T = 1$ 。记录 kernel count、host-device copy bytes、batch size、context length 和 TPOT/TTFT。解释为什么两者不能用同一个 tokens/s 结论。

实验二：构造 stride=1 和 stride=hidden 的两种 lane 映射。即使没有 GPU，也要写出地址公式、cache line transaction 预期和应采集的 profiler 字段。

实验三：比较 CPU sampler、GPU top-k 和混合 top candidates 三种边界。固定 logits、seed 和 sampler state，说明如何验证 token 一致或误差可接受。

实验四：填写一次 GPU kernel 的 occupancy 账本。至少包含 block size、registers/thread、shared memory/block、resident blocks、achieved occupancy 和是否发生 spill。

## 硬验收

读完本节，应该能把 GPU 硬件和推理后端连接起来：

- SM/CU 是 GPU 的局部执行资源，block/workgroup 被调度到其上。
- warp/wavefront 类似放大的 SIMD，喜欢规则控制流和连续访存。
- global memory、shared memory、register、cache 和 host-device 链路共同决定数据移动成本。
- tensor core 适合满足 dtype/layout/shape 条件的大矩阵块，不自动解决 decode GEMV。
- occupancy 要和寄存器、shared memory、访存合并、算术强度一起看。
- prefill 和 decode 的 GPU 瓶颈不同，lowering 和 profiler 必须分开。
- GPU capability 应进入 descriptor、verifier、lowering 和报告。

## 4.4 GPU 后端优化细讲：device buffer、stream、kernel launch 与端到端延迟

### GPU 后端不是把 CPU 指针换成 device pointer

GPU 对 AI 推理很重要，但它也很容易被误解。初学者常以为只要把权重放到显存、把算子换成 GPU kernel，就一定更快。真实情况更细：大 prefill 可能非常适合 GPU，因为矩阵乘规模大、并行度高；decode 单 token 却可能被 kernel launch、同步、KV cache 读流、logits 回传和 sampler 边界限制。GPU 峰值 FLOPS 很高，不等于一次本地对话的端到端 latency 一定低。

从编译器角度看，GPU 后端是另一个 lowering 目标。它和 CPU 后端共享模型语义、shape、dtype、quant descriptor、KV cache 合同和 oracle，但物理执行完全不同。CPU 后端关心 cache line、SIMD 寄存器和线程池；GPU 后端关心 device memory、coalesced load、shared memory、warp/block、stream、kernel launch 和 host-device staging。

### 核心思想

GPU 后端优化的核心是端到端计划，不是单个 kernel 时间。device buffer、stream、workspace、staging、同步点和 fallback 都必须进入 executable plan 和 profiler。

本节不会依赖某个具体 GPU API。无论使用 CUDA、HIP、Metal、Vulkan compute 还是厂

商库，工程边界都类似：资源要 RAII 管理，host/device 所有权要明确，异步执行要有 event 证据，kernel launch 要进入 profiler，fallback 不能悄悄打断语义合同。

### GPU 后端的 C++20 RAII 边界

GPU 后端首先要把资源生命周期写清楚。下面这些对象不应该以裸句柄和裸指针散落在 runtime 中：

Listing 4.27 GPU 后端 RAII 对象

```
GpuContext:
 owns device selection and backend capability query

DeviceBuffer:
 owns device memory allocation and free

Stream:
 owns asynchronous execution queue

Event:
 owns timestamp/synchronization marker

Workspace:
 owns temporary device memory for algorithms

StagingBuffer:
 owns pinned or mapped host memory for transfers

KernelAdapter:
 binds executable plan descriptors to launch calls
```

这些对象的价值不是包装 API，而是建立工程不变量。DeviceBuffer 的析构释放显存；Stream 的生命周期覆盖异步 kernel；Event 让 profiler 能测真实 device 时间；StagingBuffer 显式表示 host-device 边界；KernelAdapter 把高层 descriptor 转换成具体 launch 参数。没有这些边界，GPU 后端很快会变成一堆到处传递的 `void*` 和句柄。

### 常见误区

异步后端最怕生命周期伪装。CPU 代码返回不代表 GPU kernel 已经读完输入；arena 或 staging buffer 不能在相关 stream/event 完成前复用。

### host/device 驻留：数据在哪里比函数在哪里更重要

GPU 推理的第一条账本是数据驻留。权重、packed weight、activation、KV cache、logits、token id、sampler state 分别在 host 还是 device？什么时候传输？传输是否同步？这些问题往往比单个 kernel 代码更影响端到端延迟。

| 数据            | 理想驻留             | 风险                          |
|---------------|------------------|-----------------------------|
| packed weight | device 常驻        | 首次上传/显存峰值/多 GPU 分片          |
| activation    | device arena     | 跨 stream 生命周期和 workspace 冲突 |
| KV cache      | device 常驻或分层     | 长上下文显存占用、page table、量化      |
| logits        | device 计算，少量回传   | 完整 vocab 回传太贵，sampler 边界同步  |
| token id      | host/device 双方可见 | 每步小传输和同步放大 latency          |



一个常见坏路径是：每个 decode step 在 GPU 上算 logits，然后把完整 vocab logits 拷回 CPU，再在 CPU 上做 sampler，再把下一个 token 拷回 GPU。对于大词表，这个回传可能成为固定开销。更好的计划可能是在 GPU 上做 top-k 或 sampler 的一部分，只回传 next token 或少量调试数据。但这会改变 sampler 的执行位置，必须进入 executable plan 和 oracle。

### stream 和 event：异步不是自动并行

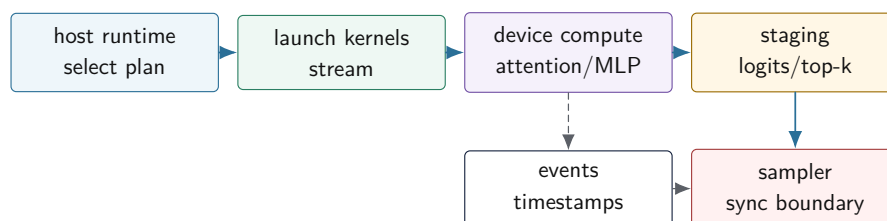
GPU API 往往是异步的。向 stream 提交 kernel 后，CPU 线程可以继续走；但这不等于工作已经完成，也不等于不同操作可以随便重排。stream 和 event 是计划中的同步语义。

Listing 4.28 GPU plan 中需要记录的同步边界

```
GpuPlan:
 stream decode_stream
 events:
 after_kv_append
 after_attention
 before_logits_staging
 dependencies:
 attention waits for kv_append
 sampler waits for logits staging
 arena slot reuse waits for producing kernels
```

profiler 也必须区分 CPU 提交时间和 GPU 执行时间。只测 kernel 内部时间，会漏掉 launch overhead、host-device copy、stream wait、CPU sampler 和计划选择。只测 CPU 墙钟时间，又难以定位慢在 device kernel 还是同步。两者都要记录。

### GPU executable plan 中的 stream、event 与 staging



图示：本图要证明的命题是：GPU 后端的端到端延迟由 launch、device compute、event、staging 和 sampler 同时决定。

### kernel launch：小算子会被固定开销吞掉

GPU kernel launch 有固定成本。prefill 的大 GEMM 能摊薄 launch 成本；decode 的许多小算子不一定能。如果每层都单独 launch RMSNorm、QKV、RoPE、KV append、attention、Wo、MLP gate、activation、down projection，decode 每个 token 会产生大量 launch 和同步。

解决方向有几类：

- 算子融合：把 RMSNorm 与 linear、RoPE 与 KV append 等合并。
- segment 调度：减少 host 侧逐算子解释和 profiler 细粒度开销。
- persistent 或批处理策略：对服务端多请求场景减少重复 launch。
- GPU sampler：减少 logits 完整回传和 host 同步。

融合仍然要受语义约束。比如 oracle 需要某层中间输出时，debug plan 不能完全隐藏物化点；量化权重需要 scale metadata，融合 kernel 必须正确读取；KV append 写入状态，后续 attention 必须看到正确 position。GPU fusion 不是“把越多函数塞进一个 kernel 越好”，而是减少内存流和 launch 开销，同时控制寄存器、shared memory 和调试边界。

**prefill GPU plan: 吞吐和 workspace**

prefill 的输入 token 多，矩阵乘和 attention 更大，GPU 通常更有优势。编译器 lowering 要为 prefill 选择合适的 batched GEMM、attention kernel 和 workspace。workspace 是后端算法的临时内存，不能和 activation arena 随意混用。

**Listing 4.29** prefill GPU plan 的主要字段

```
PrefillGpuPlan:
 prompt_length_bucket
 device_weight_layout
 activation_arena_size
 attention_workspace_size
 gemm_algorithm_id
 attention_algorithm_id
 stream_dependencies
 kv_cache_write_layout
```

prefill 指标也要拆开。time to first token 包含 tokenizer、权重准备、KV cache 初始化、prefill graph 和 logits/sampler；prompt tokens/s 只描述 prompt 处理吞吐。若报告只写一个“GPU prefill 很快”，但没有说明 prompt 长度、batch、workspace、是否包含权重上传，就无法对标。

**decode GPU plan: 小 batch、KV cache 与同步**

decode 的输入通常是一个 token。即使 GPU 算力强，单 token 路径也可能被固定开销限制。decode GPU plan 要重点处理三件事：

- 减少每 token launch 和同步次数。
- 让 KV cache 读取尽量连续或分块，避免长上下文下显存读流退化。
- 减少 logits 回传，只同步 sampler 真正需要的数据。

**Listing 4.30** decode GPU profiler 应拆分的事件

```
decode_step:
 host_plan_select_ns
 launch_overhead_ns
 rmsnorm_linear_kernel_ns
 kv_append_kernel_ns
 attention_kernel_ns
 mlp_kernel_ns
 logits_staging_ns
 sampler_ns
 total_step_latency_ns
```

如果只看 `attention_kernel_ns`，可能会误以为 GPU 很快；如果 `logits_staging_ns+sampler_ns` 很大，用户感受到的 tokens/s 仍然不高。端到端报告必须包含 `total_step_latency_ns`。

**一个 decode step 的端到端时序**

GPU decode 不能只看 kernel profiler。单 token 生成是一条 host 和 device 共同参与的流水线。即使每个 device kernel 都很快，host 侧计划选择、kernel launch、event wait、logits staging 和 sampler 也可能支配端到端延迟。

**Listing 4.31** GPU decode step 的最小时序

```
host:
```

```

bind current token and position
submit rmsnorm/qkv/rope/attention/mlp kernels
submit logits or top-k staging
wait for sampler boundary
sample next token

device stream:
rmsnorm_qkv_kernel
rope_kv_append_kernel
attention_decode_kernel
mlp_kernel
logits_topk_kernel
copy top-k or logits slice to host

```

这条时序里至少有三个同步边界。第一，KV append 与 attention 之间要有顺序，attention 必须看到当前 token 写入后的 K/V。第二，logits 或 top-k 结果回到 host 前，sampler 不能读。第三，arena 或 workspace 复用前，相关 stream 上的 kernel 必须完成。若这些边界只靠“代码看起来按顺序写”维持，就很容易在异步后端出错。

Listing 4.32 StepTrace 应记录的字段

```

StepTrace:
request_id
step_index
model_position
context_length
selected_plan_id
host_submit_begin_ns
host_submit_end_ns
device_kernel_begin_ns
device_kernel_end_ns
staging_begin_ns
staging_end_ns
sampler_begin_ns
sampler_end_ns
sync_wait_ns
total_step_latency_ns

```

有了这个结构，报告才能回答具体问题：慢在 host 提交、device kernel、staging、wait，还是 sampler。没有这个拆分，只拿 GPU 工具里某个 kernel 的耗时去推断用户感受到的 tokens/s，是不完整的。

### GPU decode 的最小可用优化顺序

GPU 后端优化也要分阶段，不要一开始就写一个巨大的 fused kernel。一个更稳的顺序是：

1. 先让所有权重和 KV cache 常驻 device，消除每步大块 host-device 传输。
2. 把 logits staging 从完整 vocab 缩小到 sampler 需要的最小集合，或把 top-k 放到 device。
3. 合并小算子，优先融合 RMSNorm+Linear、RoPE+KV append 这类语义边界清晰的路径。
4. 对 attention decode 按 context length 分档，短上下文、长上下文、paged KV 使用不同 plan。
5. 最后再做更激进的 persistent kernel、跨层 segment 或多请求 batch decode。

每一级都要同时看 oracle 和 StepTrace。比如把 top-k 放到 GPU 后，logits 全量 oracle 可能不再自然可得，这时 debug plan 要保留可选的 full logits materialization；生产 plan 可以只回传 top-k，但必须声明 sampler 语义没有改变。

## GPU 量化：格式支持比位宽口号更重要

GPU 上的低比特量化并不天然快。关键是目标硬件和库是否支持该格式，以及支持发生在哪一层。如果 int4 权重格式能直接进入高效矩阵乘路径，收益可能明显；如果每次都要先用自定义 kernel 解包成 fp16，再调用 GEMM，收益要重新计算。scale metadata 的布局也会影响 coalesced load。

| 路线                                           | 优点                   | 风险                                  |
|----------------------------------------------|----------------------|-------------------------------------|
| vendor quant kernel<br>自定义 dequant +<br>GEMM | 利用硬件和库优化<br>实现直接，易验证 | 格式受限，descriptor 适配复杂<br>解包和额外写回可能很贵 |
| fused dequant mat-<br>mul                    | 减少中间写回               | kernel 复杂，寄存器/shared<br>memory 压力   |
| KV cache 量化                                  | 降低长上下文显存和带宽          | scale metadata 和 attention 精<br>度风险 |

量化 verifier 要知道 GPU 后端实际支持什么。不能因为 manifest 写了 q4，就假设 GPU plan 可用。若 GPU 不支持该格式，应明确 fallback 到 CPU、fallback 到 dequant path，或拒绝该 plan，并在报告里写出原因。

### fallback：不能悄悄改变执行位置

真实工程中，某些算子或格式 GPU 后端暂时不支持，需要 fallback。fallback 可以是 CPU reference、CPU optimized path、GPU 上的慢路径，或者 vendor 库之外的自定义 kernel。关键是不能悄悄发生。

Listing 4.33 fallback report 字段

```
FallbackEvent:
 operator_id
 reason: unsupported_dtype | unsupported_shape | workspace_limit | ←
 ↔ debug_oracle
 from_backend
 to_backend
 data_transfer_bytes
 added_sync_points
 correctness_profile
```

如果某层 attention fallback 到 CPU，会发生 device-to-host 拷贝、CPU 计算、host-to-device 拷贝和同步，端到端 latency 可能大幅上升。profiler 必须把它显示出来。oracle 也要知道 fallback 路径的误差合同可能不同。

## 多 GPU 和分片：先建立单设备合同

多 GPU 推理涉及权重分片、KV cache 分布、通信、pipeline/tensor parallel 和跨设备同步。工程顺序应该先把单设备合同建立好。原因很简单：如果单 GPU 的 descriptor、stream、workspace、oracle 和 profiler 都不清楚，多 GPU 只会把错误放大。

单设备稳定后，多 GPU 至少要新增这些事实：

- 张量分片轴和每个 shard 的 shape。
- 跨设备通信算子和同步点。
- KV cache 是复制、分片还是按层分布。
- logits 聚合和 sampler 所在设备。
- profiler 能拆分 compute、communication 和 wait。

这些内容和第二册分布式边界、背压、运行时调度有关。它们不是本节主要目标，但要从一开始为 descriptor 和 plan 留出扩展空间。

## GPU 后端验收报告

GPU 后端优化的报告至少要覆盖：

Listing 4.34 GPU 后端验收报告字段

```
hardware:
 gpu_model
 driver_runtime_version
 device_memory
 enabled_backend_features

plan:
 backend_plan_id
 device_layouts
 stream_count
 workspace_bytes
 fallback_events

performance:
 weight_upload_time_ms
 pack_or_convert_time_ms
 prefill_tokens_per_s
 time_to_first_token_ms
 decode_tokens_per_s
 total_step_latency_ms
 launch_time_ms
 staging_time_ms
 device_kernel_breakdown

correctness:
 oracle_profile
 logits_error
 topk_changed
 generated_token_mismatch
```

注意 `launch_time_ms` 和 `staging_time_ms` 必须出现。它们正是很多 GPU decode 路径慢的原因。如果报告只有 device kernel breakdown，就还不是端到端推理报告。

## 硬验收

读完本节，应该能说明 GPU 后端的真实工程边界：

- GPU 后端共享 AI 编译器的语义合同，但拥有自己的 device buffer、stream、event、workspace、staging 和 kernel adapter。
- 数据驻留和 host-device 传输常常比单个 kernel 更影响端到端 latency。
- stream/event 必须进入 executable plan，arena 和 staging buffer 复用要等待异步工作完成。
- kernel launch 是 decode 小算子的固定成本，fusion 和 segment 调度要围绕它设计。
- prefill 关注吞吐和 workspace，decode 关注 launch、KV cache 读流、logits staging 和 sampler 同步。
- GPU 量化要看硬件和库是否真正支持目标格式，fallback 必须显式报告。
- GPU benchmark 必须同时报告 device kernel 时间、launch、staging、同步、端到端 tokens/s 和 oracle。

到这里，CPU 和 GPU 后端有了共同的工程语言：descriptor 表达语义，lowering 选择物理计

划，runtime 绑定状态，oracle 证明正确，profiler 证明性能。两者的差异不是谁更“高级”，而是机器账本不同。

## 4.5 CPU decode 实战：从标量 GEMV 到 packed 低比特微内核

### decode 优化先看形态

7B 推理的 decode 阶段每次只处理一个新 token。对线性层来说，输入是一个 hidden 向量，权重是一个大矩阵，输出是一个新向量。这更像 GEMV，而不是大 batch GEMM。CPU 优化要先承认这个形态：每 token 反复流过大块权重，权重字节和 KV 字节很容易支配延迟。

#### 核心思想

CPU decode 微内核的目标不是写一个“通用最快矩阵乘”，而是为单 token 或小 batch 的固定权重读流建立可验证的 packed layout、累加策略、tail path、线程切分和性能账本。

本节沿同一个 linear 语义推进：标量 fp32 reference, 普通 int8 baseline, packed int8, int4 packed, SIMD 微内核，线程切分，NUMA 和 profiler。每一步都要能回到 oracle。

本节固定一个 decode 形状作为样板，不再泛泛谈“某个矩阵”：

Listing 4.35 CPU decode 样板形状

```
input hidden K = 4096
output channels O = 4096
batch/token T = 1
semantic: y[out] = sum_{k=0}^{4095} W[out,k] * x[k]
weight layout before packing: row-major [O,K]
packed output block: BN = 8 for SIMD kernel, benchmark block field uses 16
```

这个 shape 对应 7B decoder 里的典型 hidden-to-hidden projection。它还不是完整一层，因为没有 RMSNorm、attention、MLP 和 KV cache，但它是 decode 阶段最容易被权重读流支配的核心切片。

### 标量 reference 不是性能 baseline

reference 的职责是正确，不是代表可接受性能。

Listing 4.36 fp32 reference GEMV

```
for out in 0..O:
 acc = 0
 for k in 0..K:
 acc += weight[out, k] * input[k]
 output[out] = acc
```

性能 baseline 可以仍然是标量，但要固定测量条件：连续内存、关闭 debug 检查、固定线程数、固定输入、warmup、重复次数和输出校验。否则 benchmark 比较的是调试开销或缓存偶然性。

### int8 权重 baseline

最简单的 int8 权重实现是边读边反量化：

Listing 4.37 int8 权重 + fp32 输入 baseline

```
for out in 0..O:
 acc = 0
 scale = scales[out]
 for k in 0..K:
```



```
w = float(weight_i8[out, k]) * scale
acc += w * input[k]
output[out] = acc
```

这比 fp32 权重少读字节，但每个权重多了转换和 scale。若 `scale` 是 per-channel，它可以在外层读取一次；若是 per-group，内层每隔 group size 切换 scale。这里必须先有 QuantDesc：

Listing 4.38 int8 QuantDesc 最小字段

```
QuantDesc:
 bit_width = 8
 signed = true
 scale_policy = per_channel | per_group
 group_size
 zero_point_policy
 scale_dtype
 axis
```

没有 descriptor，kernel 无法知道 scale 怎样对应权重，也无法写通用 oracle。

仓库中的 baseline 落在 [books/cpu-volume-3/labs/linux\\_cpu\\_inference/src/int8\\_kernels.cpp](#): `linear_i8` 使用 `int8weight+fp32input+fp32accumulator`。它不是完整整数推理，但它有清晰合同：

Listing 4.39 lcqi 当前 int8 scalar 语义

```
row_base = out * input_size
acc = bias[out]
for k in 0..input_size:
 acc += float(weights[row_base + k]) * scale * input[k]
output[out] = acc
```

对 **4096x4096**，每次调用至少读取约 **16MiB** int8 权重、**16KiB** fp32 input、写 **16KiB** fp32 output。scale 当前是 per-layer 标量，所以它不是主要读流。这个账本解释了为什么 decode GEMV 首先要盯权重读流和 SIMD 转换，而不是盲目增加复杂图优化。

### packed int8: 先改变地址流

原始权重按 `[out, k]` 行主序存储，对单输出行扫描友好。但 SIMD 微内核通常想一次计算多个 `out`，让同一个输入 `input[k]` 广播到多个输出 accumulator。为了让多个输出通道的权重连续，需要把小面板打包。

Listing 4.40 packed int8 panel 示意

```
for out_block in 0..0 step BN:
 for k in 0..K:
 for oi in 0..BN:
 packed.append(weight_i8[out_block + oi, k])
```

打包后，内核的热循环读取 `packed[k, oi]` 是连续的：

Listing 4.41 packed GEMV 热循环

```
for out_block in 0..0 step BN:
 acc[BN] = 0
 for k in 0..K:
```

```

x = input[k]
wv = load packed weights for BN outputs at this k
acc += x * dequant(wv)
store acc

```

packing 的成本应发生在模型 prepare 阶段，而不是每次请求 decode 阶段。plan report 要记录 packed weight 字节数、packing 时间和 layout id。否则上线后可能发现首个请求很慢，却不知道慢在 prepare。

当前 packed layout 使用 `out_block->k->lane` 顺序。原始 row-major 在固定 `k=0` 时，连续 8 个输出通道的地址是：

Listing 4.42 原始 row-major 多输出地址

```

W[0,0] addr = 0
W[1,0] addr = 4096
W[2,0] addr = 8192
...
W[7,0] addr = 28672

```

packed 后，固定 `k=0` 的 8 个输出通道变成连续地址：

Listing 4.43 packed panel 多输出地址

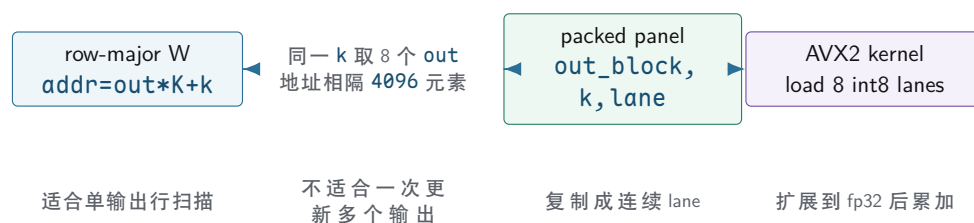
```

packed block out0..7, k=0:
 paddr 0..7 = W[0,0], W[1,0], ..., W[7,0]

packed block out0..7, k=1:
 paddr 8..15 = W[0,1], W[1,1], ..., W[7,1]

```

#### 4096x4096 decode GEMV 的地址流变化



图示：本图要证明的命题是：当前 packed int8 的直接收益是把跨步多输出读取变成连续 lane 读取。

#### int4 packed: 字节减少，解包增加

int4 把两个 4-bit 权重放进一个 byte。常见 signed int4 范围是 `-8..7` 或通过 zero point 表示。解包需要取低四位和高四位，再做符号扩展或减 zero point。

Listing 4.44 int4 解包语义

```

byte b contains two values:
 lo = b & 0x0f
 hi = (b >> 4) & 0x0f

signed value:

```

```

if nibble >= 8:
 value = nibble - 16
else:
 value = nibble

```

int4 的优势是权重字节减半，风险是解包、scale metadata 和误差。group quant 下每 **G** 个权重共享一个 scale。若 **K** 不能整除 **G**，最后一组是 tail group。byte oracle 必须覆盖 nibble 顺序、符号扩展、group tail 和 scale 索引。

### 累加器：低比特不等于低精度累加

低比特权重不代表可以用低精度累加。常见策略有两类：

- 权重低比特，输入 fp16/fp32，反量化后用 fp32 accumulator。
- 权重和激活都量化，用 int32 accumulator，再 requantize 或转回 fp32。

第一类实现简单，适合学习和 CPU baseline；第二类更接近完整整数推理，但需要处理激活 scale、zero point 修正、bias scale 和输出 requantize。第三册前面量化章节已经讲过 zero point 修正，CPU 微内核进入整数路径时必须复用同一合同。

### 常见误区

不要把 int4 权重直接乘成 int8 accumulator。归约维度 **K=4096** 时，误差和溢出风险都不可接受。accumulator dtype 是 kernel descriptor 的核心字段。

### SIMD 微内核的寄存器账本

假设一次计算 **BN=16** 个输出通道。内核维护 16 个 accumulator lane。每个 **k**：

1. 读取一个 `input[k]`。
2. 广播到 SIMD 寄存器。
3. 读取 packed 权重的 16 个值。
4. 转换或解包到可乘格式。
5. 乘加到 accumulator。

Listing 4.45 抽象 SIMD GEMV 微内核

```

acc = zero_vector(BN)

for k in 0..K:
 x = broadcast(input[k])
 w = load_packed_weight_vector(k)
 wf = convert_or_dequant(w, scale_for(k))
 acc = fma(x, wf, acc)

store output block

```

这个账本要和 ISA 绑定。AVX2、AVX-512 的向量宽度、转换指令、整数 dot 能力和 mask tail 都不同。因此代码结构应分成 capability、descriptor、kernel 实现和 fallback，不要在业务调用点写一堆 ISA 分支。

当前 AVX2 实现位于 `books/cpu-volume-3/labs/linux_cpu_inference/src/int8_kernels_avx2.cpp`。它一次处理 8 个输出通道，用一个 `__m256` accumulator 保存 8 个 fp32 lane：

Listing 4.46 当前 AVX2 kernel 的核心语义

```

packed_i8 = load_8_i8(packed_weights + packed_index)
weights_i32 = cvtepi8_epi32(packed_i8)

```

```
weights_f32 = cvtapi32_ps(weights_i32)
input_scaled = broadcast(input[k] * scale)
acc = fmadd(weights_f32, input_scaled, acc)
```

这段代码的价值是把 8 个输出通道的标量乘加合并成 8-lane AVX2 累加账本；它的限制也很明确：它先把 int8 扩展成 fp32，再做 fp32 乘加，并没有使用 VNNI/AMX 这类整数 dot 指令，也没有做 int32 accumulator + requantize。

### tail path: 最后一块经常出错

若输出通道 0 不是 BN 的倍数，最后一个 block 需要 tail。常见策略：

- 单独标量处理 tail，简单但慢一点。
- SIMD mask store，要求 ISA 支持。
- padding packed weight 到 BN 对齐，输出只写有效部分。

若归约维度 K 不是 group size、解包块或向量宽度的倍数，也需要 tail。对 int4，K 为奇数还会出现半个 byte 的语义问题。虽然真实 7B 的很多维度规整，测试必须包含不规整维度，因为模型变体、lm head、adapter 或小 fixture 都可能触发。

### 线程切分：按输出通道还是按层

decode 的单个 linear 可以按输出通道切分：不同线程计算不同 out block，共享同一 input，读取不同权重。优点是没有输出写冲突；缺点是每个线程都读一遍 input，且小层线程开销可能过大。

Listing 4.47 按输出通道切分

```
parallel_for out_block_range:
 run_packed_gemv(input, packed_weight_range, output_range)
```

也可以在线路更高层调度：不同层或不同算子顺序依赖强，不适合随意并行；不同 request 的 decode step 可以并行，但会增加权重竞争和 cache 压力。最稳的起点是单算子内按输出 block 切分，并记录线程数和分区。

| 切分方式            | 优点       | 风险                |
|-----------------|----------|-------------------|
| 按输出通道           | 简单，无输出冲突 | 权重带宽争用、线程开销       |
| 按 batch/session | 可提高吞吐    | 状态复杂，p95 可能变差     |
| 按层流水            | 理论并行     | decoder 层依赖强，收益有限 |

### NUMA 和权重放置

多路 CPU 或 NUMA 机器上，权重放在哪个 NUMA node 会影响带宽。若线程在 node 0，权重页却在 node 1，远端内存访问会增加延迟。最小工程策略：

- prepare 阶段按计划触碰 packed weight，让页分配到目标 node。
- 线程池固定亲和，避免跨 node 随机迁移。
- plan report 记录线程数、亲和策略和 NUMA 策略。
- benchmark 报告硬件拓扑，不把不同机器数字混比。

NUMA 优化不要太早做，但 trace 要保留足够字段。否则后期发现 p95 抖动，只能猜测是调度还是内存放置。

### perf counters: 用证据判断瓶颈

CPU 优化不能只看耗时。至少要观察：

- cycles 和 instructions，判断 IPC。
- cache miss 和 LLC miss，判断权重/KV 读流。
- branch misses，判断 tail 或分支复杂度。

- vector instruction 比例，确认 SIMD path 真的执行。
- memory bandwidth，判断是否接近带宽上限。

如果引入 SIMD 后 instructions 降了但 LLC miss 上升，可能 packing layout 破坏了局部性。如果线程数增加后 tokens/s 不升反降，可能已经带宽饱和或 NUMA 远端访问增加。如果 int4 比 int8 不快，可能解包和 scale 成本抵消了字节收益。

本次环境的 `perfstat` 没有暴露可用 PMU events, `perfstat-ecycles:u,instructions:u` 返回：

**Listing 4.48** 当前实验环境的 perf 边界，不伪造计数器

```
Error:
No supported events found.
The cycles:u event is not supported.
```

这个结果记录在下面的 perf artifact。它不是通过项，而是缺口证据：当前样板章已经有源码、CSV 和反汇编，但还没有 cycles、IPC、cache miss、branch miss。后续必须在有 PMU 权限的实验环境上补齐。

**Listing 4.49** perf artifact

```
books/cpu-volume-3/results/
lcqi-x86-avx2-perf-boundary-2026-06-25.txt
```

### 当前 lab 的真实验收口径

本仓库的 `linux_cpu_inference` 已经把本节的一部分落成代码：同一个 int8 linear 语义下有三个 benchmark backend：

- **scalar**：普通 row-major 标量循环，是 correctness 和耗时基线。
- **packed\_scalar**：使用 output block packing，但仍用标量累加，用来证明 packed layout、padding 和 tail 语义。
- **avx2**：x86-64 目标下编译的 8 输出通道 packed kernel。

benchmark 不再把跨平台源码混成一个数字，而是按 backend 逐行输出：

**Listing 4.50** LCQI kernel benchmark CSV 字段

```
target_arch,input_size,output_size,output_block,repeat,
backend,available,average_us,max_abs_diff,checksum
```

这种格式有两个好处。第一，`available=0` 可以明确说明某个 ISA 后端没有在当前实验环境实测，不能把源码存在当作性能证据。第二，所有 backend 都和同一份 scalar 输出比较 `max_abs_diff`，避免只看速度。

当前 x86-64 Release 样本如下，artifact 为 `books/cpu-volume-3/results/lcqi-x86-avx2-benchmark-2026-06-25.csv`：

| shape     | scalar us | packed scalar us | AVX2 us |
|-----------|-----------|------------------|---------|
| 128x128   | 3.904     | 5.453            | 1.558   |
| 256x256   | 17.843    | 21.381           | 6.043   |
| 512x512   | 81.341    | 87.787           | 23.733  |
| 513x257   | 40.719    | 50.331           | 12.600  |
| 1024x4096 | 1394.170  | 1346.740         | 379.846 |

新增固定 decode 样板 **4096x4096** 后, CSV artifact 是 `books/cpu-volume-3/results/lcqi-x86-avx2-decode4096-2026-06-25.csv`:

| backend       | average us | max abs diff | 解释                         |
|---------------|------------|--------------|----------------------------|
| scalar        | 5870.92    | 0            | row-major 标量基线             |
| packed scalar | 5734.72    | 0            | 证明 layout, 略快于标量基线         |
| AVX2          | 1559.09    | 0            | SIMD 路径, 约 3.77x 快于 scalar |

这些数字的解释要克制。packed scalar 只略快, 说明 packing 本身不是魔法; 它的主要价值是给 SIMD 消费连续 lane。AVX2 变快, 说明连续 lane + 向量扩展/乘加击中了当前瓶颈的一部分。它还没有证明达到了硬件上限, 因为缺少 PMU 计数器、内存带宽、CPU 频率和 NUMA 信息。

### 把 4096x4096 实测换成带宽和吞吐

单次 **4096x4096** int8 GEMV 至少读取 **16MiB** 权重, 输入和输出各约 **16KiB**。忽略很小的 input/output, 按权重读流估算有效带宽:

$$effective\_bandwidth \approx \frac{16MiB}{time}$$

| backend       | average us | 有效权重带宽           | 相对 scalar    |
|---------------|------------|------------------|--------------|
| scalar        | 5870.92    | <b>2.86GB/s</b>  | <b>1.00x</b> |
| packed scalar | 5734.72    | <b>2.93GB/s</b>  | <b>1.02x</b> |
| AVX2          | 1559.09    | <b>10.76GB/s</b> | <b>3.77x</b> |

这个带宽数字明显低于现代 CPU 的内存带宽, 所以当前 micro-kernel 还不是纯 DRAM 带宽上限。瓶颈更可能在标量循环开销、int8 到 fp32 转换链、FMA 指令吞吐、未展开的热循环、编译器调度、cache/TLB 和缺少多线程。换句话说, AVX2 已经证明 SIMD path 生效, 但还没有证明 micro-kernel 接近硬件 roofline。

如果把它放回 7B 账本, 单个 **4096x4096** projection 的 AVX2 延迟约 **1.56ms**。一层 decoder 有 QKV、O 和 MLP 多个 projection, 完整 32 层绝不可能简单乘一个小常数就变成可用 tokens/s。这个样板的价值是拆开一块热路径, 不是宣称完整模型速度。

反汇编 artifact 如下。热循环关键片段随后列出:

Listing 4.51 objdump artifact

```
books/cpu-volume-3/results/
lcqi-x86-avx2-linear-i8-objdump-2026-06-25.txt
```

Listing 4.52 AVX2 objdump 摘要

```
vmulss (%rbx,%rax,4), %xmm3, %xmm0 // input[k] * scale
vpmovsxbd (%rsi,%rax,8), %ymm1 // load 8 int8 and extend
vcvtdq2ps %ymm1, %ymm1 // int32 -> fp32
vbroadcastss %xmm0, %ymm0 // broadcast scaled input
vfmadd231ps %ymm0, %ymm1, %ymm2 // acc += weight * input
```

这段反汇编证明当前路径确实进入 AVX2 向量计算; 也证明它还不是成熟 int8 dot-product kernel。一个更强实现应减少 `vpmovsxbd/vcvtdq2ps/vfmadd` 链路, 尝试 VNNI/AMX 整数 dot、更多 unroll、prefetch、线程分区和 per-channel/per-group scale 的更低开销组织。

### 从反汇编看下一步优化点

当前热循环每个 **k** 至少包含: 读取 input、读取 8 个 int8 权重、符号扩展到 int32、转 fp32、乘 scale、乘 input、累加。它的优点是语义直观, oracle 容易对齐; 缺点是每 8 个权重要走一串转换指



令。

| 优化方向                      | 可能收益                             | 必须补的证据                                                      |
|---------------------------|----------------------------------|-------------------------------------------------------------|
| unroll <b>k</b> 循环        | 减少 loop overhead, 提高调度空间         | objdump、instructions、latency                                |
| 更多输出 block<br>整数 dot path | 提高 input 复用<br>减少 int8->fp32 转换链 | register pressure、spill 检查<br>accumulator 语义、requant oracle |
| prefetch packed weight    | 降低 miss stall                    | LLC miss、bandwidth counter                                  |
| 多线程 out block             | 提高吞吐                             | p50/p95、带宽饱和点、NUMA                                          |
| per-channel scale 外提      | 降低内层 scale 成本                    | scale layout、误差和反汇编                                         |

这些优化不能一起上。下一轮应该只选一个变量, 例如 **k** unroll 或整数 dot path, 保持同一 CSV schema、同一 golden、同一 shape sweep, 再比较耗时、反汇编和 perf counters。否则数字变快了也不知道是哪一项起作用。

### 本章作业：把微内核打成可复查证据

本章作业分三级。不要跳级；每一级都要保留代码、命令、CSV、反汇编和报告。

**A 级：正确性闭环。** 实现或复查 `fp32reference`、`int8scalar`、`packedscalar` 三条路径。固定小 shape **T=2, K=8, N=6**, 手工写出一个输入、一个权重矩阵和期望输出。要求：

- `packedroundtrip` 能把 packed bytes 映射回原始 `W[out, k]`。
- `max_abs_diff` 对 scalar reference 为 0 或在明确阈值内。
- 故意把 **N=6** 放进 **BN=8** 的 tail path, 证明无效 lane 不写输出。
- 报告里画出 `W[0, 0], W[1, 0], ..., W[7, 0]` 从跨步地址变成连续 packed 地址。

**B 级：SIMD 证据。** 接入一个 ISA path, 可以是 AVX2 或 AVX-512。要求：

- capability detection 或构建配置能说明当前机器是否可用。
- 不可用时 `available=0`, 不能把交叉编译或未运行结果写成性能。
- `objdump` 标出目标向量指令, 例如 x86 的向量 load、convert 和 FMA。
- shape sweep 至少覆盖 **128, 256, 512, 1024, 4096, 513x257**。

**C 级：性能解释。** 把 **4096x4096** 实测换算成有效权重带宽, 并和理论内存带宽或外部框架坐标对比。要求：

- 报告 scalar、packed scalar、SIMD 的 average us、p95、checksum、max diff。
- 报告 pack time, 不能把 prepare 成本偷偷藏进或移出 decode。
- 尝试 `perfstat`; 若失败, 保留失败输出并使用替代证据。
- 对比 oneDNN/OpenBLAS/ggml/llama.cpp 至少一个, 并说明 dtype、线程数、shape 是否一致。

### 本章报告模板

Listing 4.53 CPU microkernel report 模板

```
CpuKernelReport:
change:
 kernel_name
 git_commit
 compiler
 flags
 isa
```

```

semantics:
 formula
 input_shape
 weight_shape
 dtype
 quant_desc
 accumulator_dtype

correctness:
 reference_path
 packed_roundtrip
 tested_shapes
 max_abs_diff
 failed_cases

performance:
 warmup
 repeat
 scalar_us
 packed_scalar_us
 simd_us
 p95_us
 effective_weight_bandwidth
 pack_time_us

evidence:
 benchmark_csv
 objdump_file
 perf_file_or_failure_reason
 external_comparison

limits:
 missing_isa_paths
 missing_perf_counters
 unsupported_quant_profiles
 next_single_variable_experiment

```

这份报告写完，读者才真正拥有一个可复查的 kernel 证据。只说“我写了 int8 matmul”不够；必须说明语义、layout、反汇编、计数器、误差、外部对照和未覆盖边界。

### oneDNN 对照：只能比较坐标，不能混淆 dtype

本次还加入了可选 oneDNN 对标目标 `lcqi_onednn_bench`。它在安装 `onednn-devel` 的机器上编译，执行 `1x4096` 乘 `4096x4096` 的 f32 matmul。CSV artifact 是 `books/cpu-volume-3/results/lcqi-onednn-f32-4096-2026-06-23.csv`：

| library       |                          | dtype | shape                   | average us |
|---------------|--------------------------|-------|-------------------------|------------|
| oneDNN matmul |                          | f32   | <b>1x4096*4096x4096</b> | 17187.8    |
| LCQI scalar   | int8 weight + fp32 input |       | <b>4096x4096</b>        | 5868.84    |
| LCQI AVX2     | int8 weight + fp32 input |       | <b>4096x4096</b>        | 1559.09    |

这张表不能读成“LCQI 已经比 oneDNN 强”，因为 dtype、权重表示、primitive 选择和线程策略不同。它只能说明本仓库已经有了外部 primitive 坐标，并暴露下一步对标要求：要么实现同

dtype f32 GEMV/GEMM 与 oneDNN 比，要么找 oneDNN 支持的同量化路径；同时固定线程数、ISA、warmup、输出误差和 perf counters。

### ggml/llama.cpp 对标应比较什么

成熟推理系统的 CPU decode 不是只靠一个 micro-kernel。ggml/llama.cpp 这类项目还包含 tensor descriptor、低比特 block 格式、线程池、NUMA/亲和、KV cache、sampler、模型加载和 prompt/runtime 调度。对标时至少拆三层：

| 层级            | 比较内容                            | 不合格比较               |
|---------------|---------------------------------|---------------------|
| 单算子           | 同 shape、同 dtype、同线程数、同误差        | 拿 int8 对 f32 直接宣布胜负 |
| decoder block | RMSNorm、QKV、RoPE、KV、MLP、lm head | 只比较一个 linear kernel |
| 端到端           | TTFT、decode tok/s、p95、内存峰值、质量   | 只报平均 tokens/s       |

本章当前只完成第一层的一小部分：LCQI int8 scalar/packed/AVX2 与 oneDNN f32 坐标对照。下一步若要对标 ggml/llama.cpp，必须固定模型切片和 prompt，报告加载/prepare/prefill/decode/sampler 分段，并把 logits 或 top-k 差异写出来。没有 correctness 对照的 tokens/s，只是吞吐数字，不是 AI Infra 证据。

### 微内核进入默认路径的验收

一个 CPU decode 微内核进入默认路径前，至少要满足：

#### 验收与评分标准

- descriptor 明确 dtype、layout、group size、accumulator、tail policy 和 ISA。
- packed weight oracle 能把 packed bytes 映射回原始权重。
- operator oracle 覆盖整齐尺寸、输出 tail、归约 tail、非对齐输入和 group tail。
- layer oracle 覆盖至少一个 tiny decoder block。
- performance report 分开 pack time、kernel time、decode step time 和内存峰值。
- unsupported ISA 有明确 fallback 或 plan 阶段拒绝。
- 反汇编或 profiler 证明确实走到目标 SIMD path。

这条门槛比“benchmark 快了”严格，但它能保护项目长期演进。否则下一次改量化、改 layout、接 GPU fallback 时，很容易破坏 CPU 路径而不自知。

### 硬验收

读完本节，应该能把 CPU decode 优化拆成可实现任务：

- 从 fp32 reference 建立正确性基准，再写 int8/int4 baseline。
- 理解 packed layout 改变的是多个输出通道的权重地址流。
- 知道 int4 的 nibble 顺序、符号扩展、group scale 和 tail 为什么必须测试。
- 能说明 accumulator dtype 为什么是 kernel descriptor 的核心字段。
- 能把 SIMD 微内核写成 input broadcast、packed load、dequant、FMA、store 的账本。
- 能设计线程切分、NUMA 策略、perf counters 和默认路径验收。

## 4.6 GPU decode 端到端：常驻权重、KV 布局、sampler 与同步成本

### GPU 快，不代表端到端一定快

GPU 在大 GEMM 上很强，但 decode 阶段常常是小 batch、单 token、长上下文、频繁同步。若只看单个 kernel 时间，很容易误判。端到端 GPU decode 要把权重常驻、KV cache、logits、sampler、copy、stream、event 和 fallback 一起算。

## 核心理想

GPU decode 的核心问题不是“能不能 launch kernel”，而是让权重、KV、activation 和 sampler 尽量留在设备侧，同时把每步同步和 host-device copy 控制到可解释范围。

一个最小 GPU decode step 包括：

**Listing 4.54** GPU decode step 账本

```
input token on host
-> copy token or embedding input to device
-> run layer kernels
-> read/write device KV cache
-> compute logits on device
-> sampler on device or copy logits to host
-> emit token event on host
```

每个箭头都有成本。若每步把所有中间 activation 拷回 CPU 做调试，GPU kernel 再快也没意义。debug oracle 可以按 trace level 打开，但默认路径应尽量减少同步。

## 权重常驻设备

GPU 后端的基本要求是权重 prepare 后常驻设备。每次请求或每个 token 重新上传权重，端到端必然崩溃。

**Listing 4.55** GPU prepare 阶段

```
load model files on host
verify manifest and quant descriptors
pack or convert weights for GPU backend
allocate DeviceBuffer for each weight group
copy weights to device
record device_weight_table in plan report
```

plan report 要记录 device weight 字节、上传时间、quant format 和 fallback。若 GPU 显存不够，应该在 compile/prepare 阶段拒绝或选择 CPU fallback，而不是在请求运行中途失败。

## activation 和 workspace

decode 每层会产生临时 activation。GPU 后端应使用 workspace 或 arena，避免每个 step 调用设备分配器。

**Listing 4.56** GPU workspace 字段

```
GpuWorkspace:
 activation_buffers
 attention_scratch
 logits_buffer
 sampler_temp
 size_bytes
 alignment
 stream_owner
```

workspace 的生命周期通常是 plan 或 session。若 workspace 被多个 session 共享，调度器必须保证不会并发读写同一 buffer。若每个 session 独占 workspace，并发容量要把它算进 admission control。

## Device KV cache

若 layer kernel 在 GPU 上运行, KV cache 最好也在 GPU 上。否则每层 attention 都要在主机和设备之间传 K/V, 成本不可接受。

**Listing 4.57** GPU KV cache 设计点

```
DeviceKvCache:
 layout: contiguous | paged
 dtype: fp16 | int8 | int4
 block_table_optional
 scale_table_optional
 max_context
 resident_device
```

连续 KV 在 GPU 上地址简单, 但并发浪费显存; 分页 KV 更灵活, 但 kernel 要通过 block table 找 K/V。长上下文下, attention kernel 的读取模式决定了 coalescing。若每个 thread 读取不连续地址, 带宽会下降。

## GPU executable plan

GPU 后端也需要 executable plan, 而不是在 runtime 中临时拼 kernel。

**Listing 4.58** GPU segment descriptor

```
GpuSegmentDesc:
 segment_id
 op_or_fused_region
 kernel_name
 grid_shape
 block_shape
 input_buffers
 output_buffers
 state_reads
 state_writes
 workspace_range
 stream_policy
 fallback_segment_optional
```

**state\_reads** 和 **state\_writes** 对 attention 很重要: decode attention 读历史 KV, 同时写当前 K/V。若 plan 不表达状态边, 调度器可能错误重排 segment; 若 workspace range 不明确, 多个 segment 可能覆盖同一临时区域; 若 fallback segment 不记录, 性能报告会失真。

## device residency report

GPU prepare 完成后, 要输出哪些对象在设备上。

**Listing 4.59** DeviceResidencyReport

```
DeviceResidencyReport:
 model_id
 device_id
 weights:
 total_bytes
 quant_profile
 packed_layout
 kv_cache:
```

```

layout
max_reserved_bytes
dtype
workspace:
 bytes
logits:
 device_buffer_bytes
fallback:
 unsupported_segments

```

这份报告能提前发现“显存看似够，实际 packed weight 加 workspace 不够”的问题。它也能解释为什么同一模型在不同 GPU 上 plan 不同：某些设备支持某种低比特 kernel，另一些设备只能 fallback。

### prefill 和 decode 的 GPU 策略不同

prefill 有较大的 token 维度，适合 GEMM 和 batched attention。decode 单 token 时，kernel launch、KV 读流和小矩阵效率更突出。

| 阶段      | GPU 机会                  | GPU 风险                     |
|---------|-------------------------|----------------------------|
| prefill | 大 GEMM、并行 attention、吞吐高 | 显存峰值、长 prompt attention    |
| decode  | 多 session 合批、KV 常驻、低同步步 | 小 batch 低占用、launch/copy 成本 |

因此 GPU 报告必须拆 prefill 和 decode。只报总 tokens/s 会掩盖问题：一个后端可能 prefill 非常快，但单用户 decode 受 launch 成本限制；也可能多 session 合批后吞吐高，但 p95 变差。

### sampler 放设备侧的收益和代价

若 logits 在 GPU 上，CPU sampler 需要把 logits 或 top-k 候选拷回 host。完整词表 logits 可能是几十 KB 到几百 KB，每步还会同步 stream。设备侧 sampler 可以减少同步，但要实现 top-k/top-p、随机数和 stop 检查。

折中做法是 GPU 先选 top-k 候选，只拷回候选 id 和分数给 CPU 最终采样。这样减少拷贝，但仍保留业务逻辑和可复现随机数在 CPU。

| 策略                | 拷贝量 | 实现复杂度 |
|-------------------|-----|-------|
| 拷完整 logits 到 CPU  | 高   | 低     |
| GPU top-k, CPU 采样 | 中低  | 中     |
| 全 GPU sampler     | 低   | 高     |

第一版可以先用 CPU sampler，但必须在 trace 中单独记录 `logits_copy_ns` 和 `sampler_ns`。这样把 sampler 下沉到 GPU 时，收益能被证明。

### stream 和 event

GPU 后端不能只用一个全局同步。最小资源模型：

Listing 4.60 GPU 执行资源

```

GpuExecutionContext:
 device_id
 stream
 events:
 prefill_start

```



```

prefill_end
decode_step_start
decode_step_end
workspace
error_state

```

event 用来测量 GPU 时间，host 计时用来测端到端时间。二者都要记录。若 GPU event 显示 kernel 很快，但 host 端 decode step 慢，问题可能在 copy、sync、sampler、队列或事件等待。

### 同步点要显式列出

GPU 性能问题经常藏在同步点。常见同步来源：

- 拷回 logits 给 CPU sampler。
- debug 模式 materialize 中间张量。
- host 等待 GPU event 才能发 token。
- fallback segment 在 CPU 和 GPU 之间来回搬数据。
- 多 stream 之间的依赖 event。
- device allocator 或临时 buffer 初始化。

每个同步点都要进入 trace。若只是看 kernel profile，可能看不到 host 在等待。端到端优化时，减少一个同步点往往比把某个小 kernel 优化 10% 更有效。

### 合批 decode

GPU decode 要提高吞吐，通常需要合批多个 session 的当前 token。合批后，一个 step 的输入形状从  $[1, H]$  变成  $[B, H]$ ，权重 GEMV 变成小 GEMM，GPU 利用率提高。但合批有代价：请求要等待同批其他 session，KV cache 来自不同 session，stop 和 sampler 也各自独立。

Listing 4.61 GPU decode 合批元数据

```

DecodeBatch:
 batch_size
 session_ids
 model_positions
 kv_block_tables
 input_token_ids
 sampler_states

```

合批不是把 token 拼起来就完事。每个 session 的 `model_position`、KV block table、stop 条件和 sampler state 都不同。kernel 可以共享权重，但不能混淆状态。调度器需要设置最大等待时间，否则为了凑 batch 会拉高 p95。

### GPU oracle 策略

GPU 路径也要有 oracle，但不能每步都拷完整中间张量。可用策略：

- smoke 模式：只比较最终 logits 和 top-k。
- checkpoint 模式：抽样比较某几层输出。
- debug 模式：materialize 指定 segment 的输入输出。
- byte 模式：只比较 packed weight 或量化解包。

trace level 控制 oracle 成本。默认生产路径不应频繁同步中间张量；调试时可以打开指定 segment 的 checkpoint。这样既保留定位能力，又不让 oracle 扭曲性能数据。

### fallback 不是静默降级

GPU 可能不支持某个 dtype、某个 group size、某个上下文长度或某个算子融合。fallback 可以是好策略，但必须显式记录：

Listing 4.62 GpuFallbackEvent 字段

```
GpuFallbackEvent:
 segment_id
 requested_backend
 actual_backend
 reason:
 dtype_unsupported
 group_size_unsupported
 workspace_exceeded
 kernel_missing
 device_memory_exceeded
 performance_impact_estimate
```

静默 fallback 会制造严重误导：用户以为在测 GPU，实际跑 CPU；或者某个 segment 来回拷贝，端到端比纯 CPU 更慢。

### GPU StepTrace

GPU decode 每步至少要记录：

Listing 4.63 GPU StepTrace 字段

```
GpuStepTrace:
 request_id
 step
 context_length
 batch_size
 stream_id
 kernel_time_ns
 copy_h2d_bytes
 copy_d2h_bytes
 copy_time_ns
 sync_time_ns
 sampler_time_ns
 fallback_count
 device_memory_peak
```

这些字段能定位典型问题：kernel 时间低但同步高，说明 host-device 边界有问题；copy 字节突然上升，说明某个 debug 或 fallback materialize 了中间张量；batch size 太小，说明调度器没有把 decode 合并起来。

### 为什么 decode 小 batch 难吃满 GPU

GPU 适合大量独立工作并行推进。prefill 有较大的 token 维度，linear 和 attention 可以形成大 GEMM 或大 batch attention；decode 单用户、单 token 时，很多算子形状退化成  $[1, H]$  乘权重，或者一个 query 读长 KV。问题不是 GPU 算不动，而是并行度、数据复用和固定开销不匹配。

可以把 GPU decode 的一个 step 拆成四类成本：

| 成本            | 来源                                        | 小 batch 风险                    |
|---------------|-------------------------------------------|-------------------------------|
| kernel launch | host 提交 kernel 到 device                   | 每步许多小 kernel，固定开销占比高          |
| global memory | 权重、KV、activation、logits 读写                | 权重/KV 字节大，算术强度低               |
| occupancy     | SM 上活跃 warp 数和资源使用                        | grid 太小或寄存器/shared memory 占用高 |
| sync/copy     | logits、sampler、fallback、debug materialize | host 等待 device，stream 被切断     |

因此“GPU 峰值 TFLOP/s 很高”不能直接推出“单用户 decode 很快”。报告必须拆 prefill 和 decode，还要记录 batch size、context length、kernel 数量、launch/sync/copy 时间和 device 端实际 kernel 时间。

### occupancy 的原理边界

**occupancy**（占用率）粗略表示一个 SM 上同时驻留的 active warps 与硬件上限的比例。它受 block size、每线程寄存器数量、每 block shared memory、线程块数量和硬件限制影响。更高 occupancy 能帮助隐藏 memory latency，但不是越高越快；如果为了提高 occupancy 减少寄存器导致 spill，或让每个线程做更少有用工作，性能可能下降。

decode 小 batch 常见的 occupancy 问题有两类。第一，grid 本身太小：batch=1，某些小算子只发出很少 block，很多 SM 没有工作。第二，单个 kernel 资源占用高：attention kernel 为了处理长 context 使用较多寄存器或 shared memory，每个 SM 能驻留的 block 少，隐藏延迟能力下降。

**Listing 4.64** occupancy 报告要保存的字段

```
kernel_name
grid_dim
block_dim
registers_per_thread
shared_memory_per_block
estimated_active_warps_per_sm
achieved_occupancy_if_available
dram_bytes
elapsed_us
```

occupancy 只能解释“是否有足够并行工作隐藏等待”，不能单独证明瓶颈。若 achieved occupancy 很高但 DRAM bandwidth 接近上限，继续提高 occupancy 不会有太大收益；若 occupancy 低且每步 kernel 很短，可能要做算子融合、合批或 persistent kernel，而不是只调 block size。

### tensor core 为什么更偏爱 prefill

Tensor core 擅长固定 tile 的矩阵乘加，例如 FP16/BF16/TF32/INT8 的矩阵 tile。prefill 的  $A[T, H] \times W[O, H]$  中， $T$  足够大时，可以形成大矩阵 tile，权重和 activation 都能在 tile 内复用。decode batch=1 时， $A[1, H]$  让矩阵的一维太小，tile 利用率低；即使使用 tensor core，很多 lane 可能没有足够有效工作。

| 阶段         | tensor core 机会            | 常见限制                               |
|------------|---------------------------|------------------------------------|
| prefill    | 大 GEMM, tile 饱满, 权重复用高    | activation 峰值和 attention workspace |
| decode 单用户 | 小 GEMM/GEMV, tile 不饱满     | launch、权重流、KV 读、低 batch            |
| decode 合批  | $B \times H$ 变大, 可转小 GEMM | 等待凑 batch, p95/SLO 压力              |

所以 GPU serving 的关键是调度：在不破坏 SLO 的前提下，把多个 session 的 decode step 合

并成更适合 GPU 的小 batch。batch 大了，吞吐可能上升；等待时间也可能上升。报告必须同时写 tokens/s、TTFT、TPOT、p50/p95/p99，而不是只写平均吞吐。

### memory bandwidth 下界

延续 CPU 章节的 7B KV 账本。若 context=4096，fp16 KV 对一个 session 约 512 MiB。GPU attention kernel 不一定每 step 从 DRAM 读完整 512 MiB；cache、分块和并行归约会改变实际流量。但下界分析仍然有用：只要某个 step 的有效 KV 读取接近数百 MiB，latency 就会受到显存带宽约束。

假设有效读流为 **400MiB**，实际可用带宽 **800GB/s**：

$$time_{min} \approx 0.4/800 = 0.0005s = 0.5ms$$

这只是 KV 读取，不含 projection、MLP、lm head、softmax、kernel launch 和 sampler。如果 profiler 显示 kernel 时间低于这个数量级，需要核对实际读字节、是否使用了更低精度 KV、是否只读窗口、是否 batch 共享了某些 prefix、计时边界是否只覆盖了部分 kernel。

### kernel launch overhead 和融合

decode 每个 token 要经过多层，每层又有 RMSNorm、QKV projection、RoPE、attention、output projection、MLP、residual 等操作。如果每个小操作都是独立 kernel，launch overhead 和 stream 同步会变成显著成本。融合的目标不是让代码更炫，而是减少中间内存流和 kernel 数量。

| 融合方向                                 | 可能收益                 | 风险                      |
|--------------------------------------|----------------------|-------------------------|
| RMSNorm + QKV 前处理                    | 减少 activation 读写     | kernel 专用化增加            |
| RoPE + Q/K 写 cache                   | 减少一次 K/Q materialize | position/layout bug 更难查 |
| attention score + softmax + V reduce | 减少中间 score 存储        | 数值稳定和 debug 成本          |
| bias + activation + residual         | 减少小 kernel 和内存流      | tail/广播规则复杂             |

融合前必须有 reference 和 checkpoint oracle。否则 fused kernel 输出错了，很难判断是 RoPE position、softmax 稳定性、KV address、tail mask 还是 dtype 转换出错。

### 小 batch GPU decode 的实验矩阵

GPU decode 报告至少要做 shape sweep，而不是只测一个 prompt：

| 变量             | 建议取值                        | 观察目的               |
|----------------|-----------------------------|--------------------|
| batch size     | 1, 2, 4, 8, 16              | 找到吞吐和 p95 的拐点      |
| context length | 128, 512, 2048, 4096        | 分离 KV 读流影响         |
| kernel fusion  | off/on                      | 判断 launch 和中间内存流成本 |
| sampler 位置     | CPU, GPU top-k, GPU sampler | 判断 copy/sync 成本    |
| KV dtype       | fp16, int8 if available     | 判断容量、带宽和误差权衡       |

每组都要输出 **GpuStepTrace**，至少包含 kernel time、copy time、sync time、sampler time、fallback count、batch wait time 和 p95/p99。只有这样才能回答：GPU 慢是 kernel 本身慢，还是 batch 太小、launch 太多、copy 太多、fallback 静默发生，或者 scheduler 为吞吐牺牲了尾延迟。

### GPU 原理展开模板

本书只把 GPU 作为 AI Infra 后端边界，不把它写成完整 CUDA 教材。但凡本书提到 GPU kernel，都必须按下面模板说明：

- thread/block/warp/SM 如何映射到 tensor 维度。

- global memory 读写字节和 coalescing 条件。
- shared memory 或寄存器保存了什么 tile。
- occupancy 受哪些资源限制。
- 是否使用 tensor core, tile 是否饱满。
- kernel launch、copy、sync 是否进入端到端 trace。
- 小 batch、长 context 和 fallback 的边界是什么。

## 发布前 GPU 检查表

### 验收与评分标准

- prepare 阶段完成权重上传, decode step 不重复上传权重。
- device residency report 记录权重、KV、workspace、logits 和 fallback。
- GPU segment descriptor 记录 kernel、buffer、state read/write、workspace 和 stream。
- StepTrace 同时有 GPU event 时间和 host 端 elapsed 时间。
- logits copy、sampler、sync 和 fallback 单独计时。
- 合批 decode 记录 batch size、等待时间和每个 session 的 position。
- debug oracle 不污染默认性能报告。

## 硬验收

读完本节, 应该能设计 GPU decode 的端到端合同:

- 权重在 prepare 后常驻 device, 并写入 plan report。
- activation 和 workspace 不在每 step 动态分配。
- KV cache 在 device 侧维护, 连续或分页布局都有明确地址合同。
- prefill 和 decode 分开报告, 不用一个 tokens/s 混过。
- sampler、logits copy、stream sync 和 fallback 都进入 StepTrace。
- fallback 必须显式记录原因和实际 backend。
- 用 GPU event 和 host trace 同时解释 kernel 时间与端到端时间。

## 4.7 工业 LLM 专项: 投机解码、MoE、LoRA、多 GPU 与源码对照

### 4.8 为什么需要这一章

本地推理主链路包括 tokenizer、decoder、KV cache、量化、AI compiler、CPU/GPU backend、serving 和验收。工业系统还会遇到一批专项能力: speculative decoding、MoE、LoRA/adapters、复杂低比特格式、CUDA/HIP/Tensor Core、多 GPU 并行, 以及成熟框架源码对照。它们不一定都要在教学项目中实现到生产级, 但必须知道状态、公式、代价和验证边界, 否则进入工业系统时会断层。

本章的原则是: 每个专项都按同一模板展开。

Listing 4.65 工业专项分析模板

```
specialization:
 problem_it_solves
 state_added_to_runtime
 math_or_protocol
 memory_and_compute_cost
 correctness_oracle
 failure_cases
 industrial_reference
 minimal_project_scope
```

## 4.9 Speculative decoding: 用验证换吞吐

投机解码的核心思想是用较小 draft model 先提出多个候选 token，再用 target model 一次性验证这些候选。若候选被接受，就减少 target model 的逐 token 调用次数；若候选被拒绝，就回退到 target model 的分布。它不是简单“开两个模型并行跑”，而是一个带接受率、KV 管理和采样一致性的状态机。

Listing 4.66 投机解码状态机

```
TargetReady(prefix)
-> DraftPropose(k tokens)
-> TargetVerify(prefix + draft_tokens)
-> AcceptPrefix(m tokens)
-> if m < k: SampleCorrection
-> CommitAcceptedTokens
-> UpdateTargetKVAndDraftKV
-> TargetReady(new_prefix)
```

关键账本包括：draft 生成成本、target 验证成本、平均接受 token 数、target KV 如何追加或回滚、draft KV 是否复用、采样分布是否保持正确。若接受率低，投机解码可能更慢；若 KV 管理错，后续 token 会在错误上下文上生成；若采样修正公式错，输出分布改变。

| 字段             | 必须记录                        | 错误后果      |
|----------------|-----------------------------|-----------|
| draft tokens   | 候选 token id 和概率             | 无法复现接受/拒绝 |
| target logits  | 验证位置 logits                 | 接受率不可解释   |
| accepted count | 每轮接受多少                      | 吞吐模型失真    |
| KV mutation    | 追加、回滚、提交边界                  | 状态串扰      |
| sampler state  | RNG、temperature、top-k/top-p | 分布不可复现    |

教学项目的最小范围是：在 tiny target/draft 上实现 greedy 或受限采样投机解码，记录每轮接受数和 KV 状态。生产级实现还要处理 batching、paged KV、不同模型 tokenizer 对齐、draft/target 并发和服务 SLO。

## 4.10 MoE: 算力稀疏，系统不稀疏

MoE 模型用 router 为 token 选择少量 expert。数学上每个 token 只经过 top-k expert，看起来节省计算；系统上却引入路由、分组、负载均衡、expert 权重布局、跨设备通信和尾延迟。MoE 的难点不只是写一个 expert MLP，而是让 token 到 expert 的映射在 batch、GPU 和网络之间可控。

Listing 4.67 MoE 层状态

```
MoELayer:
 router_logits: [tokens, experts]
 selected_experts: [tokens, top_k]
 combine_weights: [tokens, top_k]
 dispatch_plan:
 expert_id
 token_indices
 capacity
 device_owner
 expert_outputs
 combine_output
```

MoE 报告要同时写：router top-k 分布、每个 expert token 数、容量溢出策略、负载均衡 loss 或辅助约束、跨设备 all-to-all 成本、expert 权重驻留位置和尾延迟。若只报告平均 FLOPs，会漏掉



最重要的系统瓶颈：热点 expert 让某个设备成为长尾。

教学项目可以先做单机单设备 MoE trace：固定 8 个 expert、top-2 routing、记录每个 expert 的 token 数和 combine checksum。多 GPU expert parallel 属于更高阶项目，需要通信账本。

### 4.11 LoRA 和 adapter：低秩增量也是 runtime 状态

LoRA 把权重增量表示成低秩矩阵乘积。对一个线性层，原始输出为  $Wx$ ，LoRA 后变成：

$$y = Wx + \alpha B(Ax)$$

这里  $A$  和  $B$  是低秩 adapter 权重， $\alpha$  是缩放。系统难点在于：服务端可能同时加载多个 adapter，不同请求选择不同 adapter；有些场景选择 merge 到主权重，有些场景选择 on-the-fly 叠加；量化主权重和 LoRA 浮点权重混合时，误差和 kernel 路径更复杂。

| 策略            | 优点            | 代价                          |
|---------------|---------------|-----------------------------|
| merge         | 推理路径简单        | 切换 adapter 慢，占用多份权重         |
| unmerge       | 恢复基座方便        | 需要管理数值误差和版本                 |
| on-the-fly    | 多租户灵活         | 每步多一次低秩计算和调度                |
| adapter batch | 多请求共享 adapter | batching 复杂，KV 与 adapter 绑定 |

LoRA 进入 serving 后，request state 必须包含 adapter id、adapter version、rank、dtype、merge state 和 oracle 误差。取消、热更新和多租户隔离都要考虑 adapter 生命周期。不能把 adapter 当成 prompt 附件；它改变的是模型权重路径。

### 4.12 量化算法谱系：格式比论文名更重要

GPTQ、AWQ、SmoothQuant、NF4、K-quants 等名词很多。第三册不应追逐名字，而要训练读者看四件事：实数如何映射到码点，scale/zero point 如何存，kernel 如何解释，oracle 如何判定误差。

| 方向            | 关注点                             | 工程字段                                      |
|---------------|---------------------------------|-------------------------------------------|
| GPTQ 类        | 后训练权重量化，逐层误差补偿                  | group size、order、scale、zero、permute       |
| AWQ 类         | 保护重要 activation/通道              | activation statistics、protected channels  |
| SmoothQuant 类 | 在 activation 和 weight 间迁移 scale | smoothing factor、folded scale             |
| NF4 类         | 非均匀 4-bit codebook              | codebook id、block scale、dequant table     |
| K-quants 类    | 本地推理 block 格式                   | block type、metadata layout、kernel support |

验收不是“支持某个名字”，而是能解析 descriptor，证明 packed bytes、scale metadata、dequant 公式和 reference oracle 一致。若 kernel 只支持某种 group size 或 block type，verifier 要拒绝不支持的资产。

### 4.13 CUDA/HIP 与 Tensor Core：从 kernel 到 plan

GPU 章节已经讲了 warp、block、shared memory、coalescing、occupancy 和 tensor core 原理。工业闭环还要把这些落到 executable plan。一个 GPU backend 不只是一组 kernel，它包括 device buffer、stream、event、workspace、copy、kernel launch、synchronization、fallback 和 profiler。

Listing 4.68 GPU step plan

```
GpuStepPlan:
 stream_id
 resident_weights
 kv_cache_layout
 staging_buffers
 kernels:
```

```

rmsnorm
qkv_gemm
rope
attention
mlp
logits
sampler_or_topk
events:
 h2d_copy
 kernel_time
 d2h_copy
 sync
fallback:
 unsupported_dtype
 unsupported_shape
 workspace_limit

```

Tensor Core 的使用条件必须写成 descriptor: dtype、tile shape、layout、alignment、accumulator type 和 scale 路径。prefill 的矩阵形状通常更容易吃满 tensor core; decode batch 1 常常变成小 GEMV, 利用率差, 需要 batching 或 fused kernels 改善。报告必须分 prefill 和 decode, 不能用 prefill tokens/s 掩盖 decode TPOT。

#### 4.14 多 GPU: 并行不是把模型复制几份

多 GPU 推理至少有三类并行: tensor parallel 把单层矩阵切到多个设备, pipeline parallel 把层切到多个设备, expert parallel 把 MoE expert 分布到多个设备。它们都引入通信和调度状态。

| 并行方式              | 通信形态                                 | 主要风险                    |
|-------------------|--------------------------------------|-------------------------|
| tensor parallel   | all-reduce、all-gather、reduce-scatter | 小 batch 通信占比高           |
| pipeline parallel | stage 间 activation 传递                | bubble、调度复杂、KV 分布       |
| expert parallel   | token dispatch/all-to-all            | 热点 expert、负载不均、尾延迟      |
| KV 分片             | 按层、head、sequence 或 page 分布           | attention gather 和一致性复杂 |

多 GPU 报告要写: 切分维度、每步通信字节、collective 类型、设备拓扑、KV cache owner、失败恢复边界和 SLO 影响。NCCL 或其他 collective 不是魔法黑盒; 它只是实现通信, 不能替你决定模型状态如何分片。

#### 4.15 工业源码对照: 看设计理由, 不贴 logo

源码阅读要围绕问题, 而不是围绕项目名。建议每次只读一个概念:

| 系统             | 阅读问题                                      | 对应轻量实现概念                                  |
|----------------|-------------------------------------------|-------------------------------------------|
| llama.cpp/ggml | tensor、quant block、backend dispatch 如何表达  | descriptor、packed weight、kernel registry  |
| vLLM           | paged KV 和 continuous batching 如何绑定       | block table、sequence state、scheduler      |
| TensorRT-LLM   | engine、plugin、KV cache 和多 GPU 怎样组织        | executable plan、plugin、collective         |
| ONNX Runtime   | graph、execution provider、allocator 如何协作   | typed graph、backend select、memory planner |
| oneDNN         | primitive descriptor 怎样选择 layout 和 kernel | layout propagation、post-op、re-order       |
| MLIR/TVM       | IR/pass/lowering/schedule 如何分层            | verifier、pass pipeline、lowering           |

源码阅读报告必须包含: 读了哪个版本、哪个文件或模块、它如何表达状态、它解决什么问题、它付出什么复杂度、轻量实现借鉴什么、暂时拒绝什么。只写“参考 vLLM”没有意义。

## 源码级阅读报告的字段

工业对标要能落到具体源码入口，但路径会随版本移动。报告必须固定 commit/tag，并用符号搜索确认当前路径，不能把网上旧路径当作永久事实。最低字段如下：

Listing 4.69 工业源码阅读报告模板

```
industrial_source_report:
 system_name:
 version_or_commit:
 question:
 source_files_or_symbols:
 core_state_objects:
 key_transition_functions:
 invariants_checked:
 performance_tradeoff:
 failure_or_fallback_path:
 lesson_adopted:
 lesson_rejected:
```

下面是建议的阅读问题和入口类型。它们不是固定 API 文档，而是给读者一个源码搜索方向：

| 系统             | 版本固定后要找的人口                                                                                | 不应停留的空话              |
|----------------|-------------------------------------------------------------------------------------------|----------------------|
| llama.cpp/ggml | tensor descriptor、GGUF loader、quant block type、backend op dispatch、KV cache 结构            | “它支持 GGUF 和量化”       |
| vLLM           | sequence/request state、block table、KV block manager、scheduler step、continuous batching 决策 | “它有 paged attention” |
| ONNX Runtime   | graph node、execution provider、allocator、kernel registry、memory pattern/planner            | “它是通用 runtime”       |
| TensorRT-LLM   | engine/build plan、plugin、KV cache manager、batch manager、collective plan                   | “它 GPU 很快”           |
| oneDNN         | primitive descriptor、memory descriptor、reorder、post-op、kernel implementation dispatch     | “它有高性能 GEMM”         |
| MLIR/TVM/XLA   | op/type、pass manager、layout/shape rewrite、lowering、schedule/codegen 边界                    | “它是 AI compiler”     |

每个对标都要回答“它为什么这样设计”。例如 vLLM 的重点不是 page table 这个名词，而是 block table 怎样让 KV cache 在请求增长、释放、prefix 复用和 continuous batching 中保持可调度；oneDNN 的重点不是 primitive 名词，而是 memory descriptor 怎样把 layout、reorder 和 kernel 选择变成显式合同；ggml/llama.cpp 的重点不是轻量，而是 tensor/quant/backend 三层如何用较小抽象覆盖多种硬件后端。

## 4.16 专项能力的交付边界

这些专项可以成为独立交付物，但边界要诚实：

- spec decoding：必须有接受率、KV 状态和分布正确性报告。
- MoE：必须有 routing 分布、expert 负载和 combine oracle。
- LoRA：必须有 adapter 生命周期、多 adapter 切换和 logits diff。
- 量化：必须有格式 descriptor、byte oracle、operator oracle 和质量回归。
- GPU：必须有 kernel profiler、copy/sync 分解和 fallback。

- 多 GPU：必须有通信字节账本和 SLO 影响，不只启动多进程。
- 源码对照：必须有设计拆解，不是链接列表。

### 4.17 验收标准

本章不要求一次实现所有工业能力，但每个被声明完成的专项必须满足：

1. 有明确问题和收益模型。
2. 有新增 runtime state 和生命周期。
3. 有数学公式或协议状态机。
4. 有内存、计算或通信账本。
5. 有 correctness oracle 和失败案例。
6. 有至少一个工业系统的源码对照。
7. 有不支持场景和 fallback 说明。

#### 核心思想

工业 LLM 系统不是“Transformer 加几个优化名词”。每个专项都会增加状态、验证、调度和证据负担。能把这些负担讲清楚，才说明真正理解 AI Infra。

### 4.18 工程验收与回归体系：reference、测试、覆盖率、性能门禁和框架对标

#### 为什么“能跑”不是完成

一台本地大模型推理引擎最容易给人错觉：只要 prompt 能生成文字，项目就算跑通。对学习项目来说，这只是第一步；对高质量 AI infra 来说，这远远不够。生成文本看起来正常，不代表 tokenizer 合同正确，不代表量化 scale 正确，不代表 KV cache 没有越界，不代表 SIMD tail path 被覆盖，不代表 GPU fallback 没有偷偷发生，也不代表性能优化真的击中了瓶颈。

工程验收要回答四个问题：

1. 正确性由哪个 reference 定义。
2. 哪些层次的 oracle 已经比较过。
3. 哪些性能指标和硬件环境可复现。
4. 哪些回归测试会在后续改动时保护这些事实。

#### 核心思想

AI infra 的完成标准不是“输出了 token”，而是 reference、oracle、profiler、覆盖率、性能回归和外部框架对标形成闭环。没有闭环，优化无法被信任，也无法长期维护。

#### 联合验收：项目到底能证明什么

联合验收不写空泛能力承诺，只看可复查证据：C++ 热路径是否能追到机器执行，机器执行是否能放进现代计算系统成本模型，这套能力是否真正用于本地大模型推理引擎的正确性、性能和服务化闭环。

| 卷次  | 必须证明的能力                                                           | 对应工程证据                                                             |
|-----|-------------------------------------------------------------------|--------------------------------------------------------------------|
| 第一册 | C++ 对象、ABI、汇编、二进制身                                                | TinyTensor 源码到汇编报告、                                                |
| 第二册 | 份、可信 benchmark<br>cache/TLB/SIMD/线<br>程/NUMA/运行时/SLO 的定<br>量分析    | objdump、GDB、CSV<br>kernel roofline 报告、PMU/替代<br>证据、扩展曲线、trace      |
| 第三册 | tokenizer 到 token、IR 到 plan、<br>CPU/GPU backend、serving 和验收<br>闭环 | MiniLLM-CPU fixture、oracle、<br>plan dump、benchmark、release<br>gate |

联合验收的最终报告应该包含三份主文件：

Listing 4.70 三册联合出口报告

```

volume1-machine-evidence.md:
 source -> object layout -> ABI -> assembly -> benchmark

volume2-systems-performance.md:
 roofline -> PMU/trace -> SIMD/thread/NUMA -> runtime SLO

volume3-ai-infra-release.md:
 tiny decoder -> compiler plan -> CPU kernel -> serving -> comparison

```

这三份报告不替代 CUDA、分布式推理或 MLIR 编译器专项项目；它们证明的是另一件事：底层、性能、AI 推理和工程证据能接成一条可信链路。

### MiniLLM-CPU release gate

第三册阶段性 release 不能只看 [ai-generate](#) 输出文本。MiniLLM-CPU 的 release gate 至少包含下面这些项目：

Listing 4.71 MiniLLM-CPU release gate

```

correctness:
 tokenizer ids match fixture
 embedding/RMSNorm/QKV/RoPE/KV/attention/MLP checkpoints pass
 logits golden max error within profile
 fixed sampler sequence matches golden

compiler:
 manifest-report.json generated
 ir-before.json and ir-after-passes.json generated
 memory-plan.json has no overlapping live values
 executable-plan.json records state reads/writes and backend kernels

cpu backend:
 fp32 reference and int8 scalar baseline pass
 packed weight roundtrip pass
 SIMD path pass or fallback is explicit
 shape sweep CSV generated

serving:
 short request completes
 long prompt reports TTFT
 too-long request rejected before KV allocation
 cancel request releases KV/arena/workspace
 slow client triggers backpressure or cancellation

performance:
 benchmark report records hardware, compiler, flags, threads
 p50/p95 and memory peak recorded
 perf/objdump/trace evidence attached
 comparison against at least one external baseline recorded

```

这份 gate 是推理引擎从“能跑”走向“可维护”的关键。每新增一个 kernel、pass、量化格式或 serving 策略，都必须接入 gate。否则项目会快速堆出功能，但无法长期维护。

## 还不能证明的能力要明确写出

高质量工程材料必须承认边界。若 MiniLLM-CPU 只完成本地 CPU-first 路径，下面这些能力仍然需要单独证据：

- 生产级 CUDA/Tensor Core kernel：还需要 warp、shared memory、occupancy、tensor core、CUDA graph、Nsight 证据。
- 生产级分布式推理：还需要 tensor parallel、pipeline parallel、KV 分片、跨节点通信、故障恢复和集群 SLO。
- 完整 AI compiler 生态：还需要 MLIR/TVM/XLA 级 pass、dialect、codegen、autotuning 或 shape polymorphism 证据。
- 高并发在线服务：还需要 continuous batching、paged KV、prefix cache、多租户隔离、限流、观测和线上事故复盘。
- 质量评估体系：还需要真实模型、真实数据集、量化质量 benchmark 和人工/自动评估。

这些边界应写进 release report。严肃的工程材料会把已证明、未证明和下一步证明方法分开，避免把一个受限项目包装成全栈生产系统。

## reference 分层：先有正确答案

reference 是不追求速度的正确性来源。它应该清晰、可读、可调试。后端优化越激进，reference 越重要。项目中至少需要下面几类 reference：

| reference 层次    | 作用                             | 不追求的东西      |
|-----------------|--------------------------------|-------------|
| 字节/量化 converter | 定义 int4/int8 bytes 如何还原        | SIMD 和吞吐    |
| 单算子 reference   | 定义 RMSNorm/linear/attention 输出 | cache 和线程效率 |
| 层级 reference    | 定义 decoder block 输出            | 融合和内存复用     |
| 端到端 reference   | 定义 logits/生成序列合同               | 生产级速度       |

reference 代码要尽量少依赖后端复杂路径。比如 int4 reference converter 不应该复用 AVX2 解包函数；否则 SIMD 解包错了，reference 也跟着错。单算子 reference 可以用标量循环和 fp32 累加；端到端 reference 可以只跑小模型、小层数或模型切片，保证测试可承受。

## oracle 分层：从字节到生成序列

oracle 消费 reference 和被测实现，判断误差是否在合同内。不同层 oracle 关注不同问题：

**Listing 4.72** oracle 层次与定位能力

```
byte oracle:
 packed bytes -> decoded values
 catches nibble order, zero point, scale, tail

operator oracle:
 optimized kernel output vs scalar reference
 catches SIMD, accumulation, layout, quant

layer oracle:
 decoder block output vs reference block
 catches fusion, arena overwrite, state edge

logits oracle:
 final logits distribution
 catches visible model quality drift

sequence oracle:
```



fixed sampler and seed token sequence  
catches end-to-end behavior changes

误差容忍度必须和 dtype、量化、后端和 reduction 顺序匹配。fp32 reference 对 fp32 scalar 可以要求很严；fp16、bf16、int8、int4、KV cache 量化都需要不同 tolerance profile。oracle 报告里要写清使用的是哪个 profile，不要把阈值埋在测试代码里。

### 测试矩阵：不要只测漂亮形状

AI 后端最容易漏测的是边界形状。测试矩阵要覆盖整齐形状，也要覆盖 tail、非对齐、不同 group size、不同 context length 和不同阶段。

Listing 4.73 后端测试矩阵示例

```
shapes:
 H = 4096
 H = 4097 for tail test
 0 not divisible by tile_n
 prompt length: 1, 7, 32, 512
 context length: 1, 128, 1024, 4096

dtypes:
 f32 reference
 f16/bf16 style
 int8 per-channel
 int4 group-size 32/64/128

layouts:
 contiguous
 padded
 packed CPU panel
 device layout

backends:
 scalar
 CPU SIMD
 CPU threaded
 GPU kernel
 fallback path
```

这些测试不一定全部在每次提交跑。可以分层：快速单元测试常跑，较大的端到端和性能测试在夜间或发布前跑。但测试矩阵必须被设计出来，否则项目只会在“常见 happy path”上看起来稳定。

### coverage：覆盖率要服务风险，不服务数字

C++20 推理项目需要覆盖率意识，但覆盖率不是盲目追数字。高风险路径要优先覆盖：verifier 错误诊断、descriptor 解析、quant converter、tail path、arena liveness、KV cache position、fallback、sampler 和 profiler report。

| 模块             | 高风险分支                                    | 覆盖重点                         |
|----------------|------------------------------------------|------------------------------|
| model/verifier | 坏 shape、坏 dtype、坏 quant                  | 拒绝输入和诊断质量                    |
| quant          | tail、scale shape、packed order            | reference converter 和 oracle |
| memory planner | last use、alignment、debug materialization | offset 不重叠和生命周期              |
| backend CPU    | tail、非对齐、多线程分区                           | 正确性和性能回归                     |
| backend GPU    | fallback、staging、sync                    | 端到端 latency 和资源释放            |
| runtime        | prefill/decode 切换、stop、context limit     | session 状态不串扰                |

如果项目工具链支持覆盖率，核心模块应该接近或超过 90% 的有意义覆盖。若某些 GPU 分支或硬件计数器难以在 CI 中覆盖，要用 fake backend、mock event、small fixture 或本地验收脚本补充。重要的是把不可覆盖原因写清，而不是假装没有风险。

### CI 分层：不是所有测试都每次全跑

AI 推理项目的测试成本差异很大。字节级 quant converter 可以毫秒级完成；7B 模型长上下文 benchmark 可能需要专门机器。工程上要分层，而不是把所有检查都塞进每次提交，也不能因为全量测试太慢就放弃测试。

| 层级 | 触发时机     | 内容                                                          |
|----|----------|-------------------------------------------------------------|
| L0 | 本地开发频繁运行 | 格式化、编译、descriptor/verifier 单测、字节级 quant 测试                  |
| L1 | 每次提交或 PR | 小模型 reference、单算子 oracle、memory planner、CPU scalar/SIMD 小矩阵 |
| L2 | 夜间或合并前   | 模型切片端到端、长 context、paged KV、fallback、覆盖率报告                   |
| L3 | 发布前或专用机器 | 真实模型 benchmark、GPU、框架对标、p50/p95、峰值内存、质量回归                   |

失败报告也要结构化。不要只贴一段日志。最小字段可以是：

Listing 4.74 CI failure 结构

```

CiFailure:
 layer: L0 | L1 | L2 | L3
 component: verifier | quant | backend | runtime | benchmark
 fixture_id
 backend
 expected
 actual
 tolerance_or_gate
 first_bad_artifact
 reproduction_command

```

这样开发者能直接知道问题属于哪层合同。若 byte oracle 失败，先查 nibble order、scale 和 tail；若 layer oracle 失败，查 fusion、arena 和状态边；若 L3 benchmark 回归，先看硬件、热身、频率、prompt set 和 profiler breakdown。CI 的作用不是制造红叉，而是把问题定位到正确工程层。

### fuzz、sanitizer、CI 和性能回归系统

AI infra 项目不能只靠几个 golden case。Golden case 证明固定路径能跑；fuzz 负责攻击输入空间；sanitizer 负责抓 C++ 未定义行为和并发错误；CI 负责把这些检查分层执行；性能回归系统负责防止优化被后续改动悄悄吃掉。这四者合起来，才是长期维护的工程底座。

### fuzz 分层：攻击 descriptor、shape、state 和 scheduler

Fuzz 不是随机喂一堆字节给整个模型。更有效的做法是按合同分层，让每个 fuzz target 都有清楚 oracle 和拒绝边界。

| fuzz 层            | 生成或扰动什么                                        | 必须证明什么                          |
|-------------------|------------------------------------------------|---------------------------------|
| manifest fuzz     | config 字段、tensor 名、offset、bytes、checksum       | verifier 拒绝坏资产，不越界读             |
| shape/stride fuzz | rank、shape、stride、layout、broadcast             | 地址公式不溢出，错误诊断稳定                  |
| quant fuzz        | group size、scale 数量、zero point、packing tail    | converter 不越界，oracle 能定位字节错误    |
| tokenizer fuzz    | 特殊 token、空 prompt、非法 UTF-8、长 prompt            | token ids 不越界，context policy 明确 |
| KV fuzz           | position、slot、page table、prefix、sliding window | 逻辑位置和物理 slot 不串扰                |
| sampler fuzz      | NaN、Inf、极端 logits、top-k/top-p 边界               | 数值稳定，拒绝非法概率                     |
| scheduler fuzz    | admission、cancel、timeout、late completion       | session 状态可回收，不双释放 KV           |

每个 fuzz target 都要避免两个极端：一是只生成合法输入，永远碰不到拒绝路径；二是生成完全无结构的垃圾，让 verifier 只在第一层失败。好的 fuzz 会生成“接近合法”的坏输入，例如 tensor byte range 只差一个 block、q4 scale 少一组、KV slot 合法但 position 复用、tokenizer vocab 与 embedding 差一个特殊 token。这类输入最接近真实转换器和系统事故。

Listing 4.75 fuzz target 的报告字段

```
FuzzCaseReport:
 target: manifest | tensor | quant | tokenizer | kv | sampler | scheduler
 seed
 input_digest
 expected_result: accept | reject | bounded_error
 earliest_contract
 sanitizer_enabled
 minimized_artifact
 reproduction_command
```

Fuzz 失败后，最小化 artifact 必须保存。若一个 30 KB manifest 触发越界，最终报告应尽量保留一个更小的 manifest 片段，能稳定复现同一错误。没有 seed、输入摘要和复现命令，fuzz 只会制造一次性噪声。

### sanitizer 矩阵：正确性工具不是性能工具

Sanitizer 构建必须进入常规验收，但不能拿 sanitizer 下的耗时做性能结论。它们会插入检查代码、改变内存布局和线程调度。正确做法是把 sanitizer 作为 correctness lane。

| 工具    | 能抓什么                               | 不能声称什么              |
|-------|------------------------------------|---------------------|
| ASan  | 越界、use-after-free、栈/堆红区错误          | Release 内存布局 and 性能 |
| UBSan | signed overflow、非法 shift、对齐、枚举等 UB | 程序没有所有逻辑错误          |
| TSan  | 数据竞争、部分同步错误                        | 无锁结构性能或实时行为         |
| MSan  | 未初始化数据传播                           | 所有平台都易部署            |
| LSan  | 泄漏和未释放对象                           | 资源生命周期业务语义全正确       |

AI runtime 特别要跑 ASan/UBSan：packed int4 tail、SIMD load 边界、arena reuse、mmap view、scale metadata、KV cache page table 都容易出现越界或对齐问题。TSan 要用于 scheduler、session、cancel、completion queue、metrics 聚合和 lazy initialization。TSan 变慢很正常，不能据此评价 runtime 吞吐。

**CI lanes：把检查按成本和风险分层**

CI 不是一条命令，而是一组 lane。每条 lane 回答不同问题：

| lane            | 触发           | 内容                                             |
|-----------------|--------------|------------------------------------------------|
| quick           | 每次 push/PR   | configure、build、unit、tiny golden、descriptor 负例 |
| sanitizer       | PR 或 nightly | ASan+UBSan、TSan、关键 fuzz seed 回放                |
| fuzz smoke      | PR 或 nightly | 每个 target 固定 seed + 短时间随机探索                    |
| benchmark smoke | 受控机器或手动      | tiny model、小矩阵、稳定输入的性能冒烟                       |
| nightly         | 夜间           | shape sweep、长 context、scheduler cancel、覆盖率     |
| manual evidence | 发布前          | perf stat、objdump、火焰图、框架对标                     |

一个压缩的 workflow 可以这样组织。真实仓库可以按机器和权限拆得更细：

**Listing 4.76** CI workflow 形态草图

```
jobs:
 quick:
 cmake -S books/cpu-volume-3 -B build/lcqi-ci
 cmake --build build/lcqi-ci
 ctest --test-dir build/lcqi-ci --output-on-failure

 sanitizer:
 cmake -S books/cpu-volume-3 -B build/lcqi-asan \
 -DCMAKE_CXX_FLAGS="-fsanitize=address,undefined"
 cmake --build build/lcqi-asan
 ctest --test-dir build/lcqi-asan --output-on-failure

 fuzz_smoke:
 run fixed seeds for manifest, tensor, quant, kv, sampler

 benchmark_smoke:
 run tiny decoder benchmark and compare with same-host baseline
```

CI 中最容易犯的错误是把不稳定性能数字放进普通共享 runner。性能门禁需要固定机器、频率策略、亲和、输入和历史 baseline；普通 CI 只能跑 benchmark smoke，证明命令还能跑、schema 还稳定，不能证明性能没有 3% 回归。

**性能回归系统：只能和同一硬件身份比较**

性能回归要固定坐标。不同 CPU、不同内核、不同频率策略、不同容器限制、不同 NUMA 绑定之间的数字不能直接作为门禁。最小 baseline key 应包含：

**Listing 4.77** 性能 baseline key

```
PerfBaselineKey:
 hardware_id
 cpu_model
 microcode_or_kernel
 compiler_id
 backend_plan_id
 model_fixture_id
```

```
quant_profile_id
prompt_set_id
thread_count
affinity_policy
context_bucket
```

报告不能只保存平均值。至少保存 raw samples、median、p95、min/max、warmup 次数和异常样本说明。门禁可以采用保守规则：

Listing 4.78 性能回归判定规则示例

```
fail if same_host and:
 median_latency > baseline_median * 1.10
 and p95_latency > baseline_p95 * 1.15
 and two consecutive runs reproduce the regression

warn if:
 peak_memory > baseline_peak * 1.20
 or fallback_count increases
 or pack_time increases while decode improvement is absent
```

这个规则故意要求同一 host 和连续复现，避免一次噪声让 CI 误杀。反过来，若 fallback count 增加，即使 latency 暂时没变，也应该警告，因为某些输入或机器会放大问题。性能门禁的价值不是把数字变成迷信，而是让退化有固定入口。

### 最小仓库 artifact

验收体系不能只停留在文字里，仓库应有对应 artifact。最小集合可以是：

Listing 4.79 验收体系最小 artifact

```
.github/workflows/cpu-books-lcqi.yml
tests/fuzz/manifest_fuzz.cpp
tests/fuzz/tensor_descriptor_fuzz.cpp
tests/fuzz/kv_cache_fuzz.cpp
sanitizer-presets.json
scripts/bench_regression.py
results/perf-baselines/<hardware-id>.json
reports/release-gate/<version>.md
```

这些 artifact 的目标不是一次性做全，而是固定扩展方向。新增 tokenizer、quant format、memory planner pass、CPU kernel、GPU backend 或 serving scheduler 时，都要接入相同的 fuzz seed、sanitizer lane 和性能 baseline 系统。没有统一入口，后续每个优化都会重新发明一套报告，最后无法比较。

### 性能门禁：基线、阈值和报告要稳定

性能回归比功能回归更难测，因为机器负载、频率、温度和系统噪声都会影响数字。工程上不能用单次最好成绩做门禁，而要固定环境、固定输入、固定 warmup 和迭代，并报告分位数。

Listing 4.80 性能门禁报告字段

```
PerformanceGate:
 hardware_id
 backend_plan_id
 model_fixture_id
 quant_profile_id
```

```

prompt_set_id
thread_count_or_device
warmup_iterations
measurement_iterations
baseline_version
current_version
allowed_regression_percent
metrics:
 time_to_first_token
 prefill_tokens_per_s
 decode_tokens_per_s
 p50_step_latency
 p95_step_latency
 peak_memory

```

门禁阈值要分指标。比如 peak memory 回归 20% 可能不能接受；单次 p50 latency 回归 3% 可能在噪声内；p95 latency 回归 15% 需要调查；pack time 变慢但 decode 变快是否可接受，要看目标场景是短请求还是长期服务。性能门禁不是机械判分，而是让退化可见。

### 服务化 SLO：把 kernel 数字换成线上账本

底层 kernel 快，不等于服务能扛业务。推理服务还要处理排队、取消、流式返回、上下文容量、KV cache 内存和 OOM。最小 serving 验收要把单请求 trace 聚合成下面的账本：

Listing 4.81 ServingSloEnvelope

```

ServingSloEnvelope:
 max_waiting_requests
 max_running_requests
 admission_policy: reject | queue | degrade
 ttft_p50_ms
 ttft_p95_ms
 decode_step_p95_ms
 end_to_end_p95_ms
 queue_wait_p95_ms
 kv_cache_reserved_bytes
 kv_cache_used_bytes
 oom_reject_count
 cancel_reclaim_p95_ms

```

用 7B 典型 shape 粗算一次 KV 容量：layers=32, kv\_heads=32, head\_dim=128, fp16 时，每 token KV 字节为：

$$2 \times 32 \times 32 \times 128 \times 2 = 524288 \text{ bytes} = 512 \text{ KiB}$$

一个 4K context session 约 2GiB KV cache。若机器只给 serving 预留 16GiB KV 空间，理论上最多同时容纳 8 个满长 session；实际还要扣除 allocator 碎片、paged metadata、workspace 和安全水位。admission control 因此不能只看请求数，必须看 prompt\_tokens+max\_new\_tokens 对 KV budget 的占用。

调度也要定量。若单请求 decode step p95 是 40ms，同一 worker 串行跑 4 个活跃 session，最坏情况下某个 session 下一个 token 要等另外 3 个 step，队列等待可接近 120ms。若业务要求流式 token 间隔 p95 小于 80ms，就必须降低同 worker 并发、改 continuous batching、或把长请求降级/拒绝。这里的重点不是精确排队论，而是 SLO 要能反推出调度策略。



| 线上现象                 | 应记录的字段                                                                        | 常见根因                           |
|----------------------|-------------------------------------------------------------------------------|--------------------------------|
| TTFT 高<br>token 间隔抖动 | queue wait、load、prefill、pack time<br>decode p95、active sessions、batch<br>size | 冷启动、长 prompt、排队<br>调度过载、同步点、抢占 |
| OOM 或频繁拒绝            | KV reserved/used、workspace、<br>arena peak                                     | admission 不看 token 预算          |
| 取消后内存不降              | cancel reclaim、session state、KV<br>refcount                                   | 资源生命周期错                        |
| p50 正常但 p95 差        | queue wait、page fault、fallback<br>count                                       | 尾部排队、缺页、后端降级                   |

因此服务化验收至少要有三条故障演练：超长上下文被 admission 拒绝并返回 token budget；取消请求后 KV、arena、workspace 在限定时间内释放；强制注入慢请求后，报告能拆出 queue wait 与 kernel time，而不是只看到端到端超时。

### profiler schema：优化证据要可比较

profiler 输出不能每章一个格式。统一 schema 让不同优化可以比较。最小 schema 应该覆盖 stage、segment、backend、context、耗时、内存和可选硬件计数器。

Listing 4.82 统一 profiler event schema

```
ProfilerEvent:
 run_id
 stage: load | prepare | prefill | decode | sample
 segment_id
 layer_id
 backend
 kernel_or_operation
 start_ns
 end_ns
 input_tokens
 context_length
 bytes_read_estimate
 bytes_written_estimate
 arena_peak_bytes
 workspace_bytes
 kv_cache_bytes
 counters_optional
```

同一 schema 可以服务 CPU 和 GPU。CPU 可能填硬件计数器；GPU 可能填 stream event 时间和 staging bytes；reference 路径也可以填 stage 和耗时。这样后续报告不需要人工拼日志。

### 诊断质量：错误要能定位到合同

高质量工程不是永远不出错，而是出错时能定位。诊断应该回到合同，而不是只报底层异常。

Listing 4.83 诊断应该包含的信息

```
Diagnostic:
 severity
 component: manifest | verifier | compiler | backend | runtime
 object: tensor/operator/segment/session
 expected
 actual
```

```
derived_from
suggested_next_check
```

例如 backend 不支持某个 q4 格式时，应该写明 tensor、format、group size、backend capability 和 fallback 选择；KV cache 越界时，应该写明 session position、capacity、max context 和触发的 token；arena 复用冲突时，应该写明两个 value 的生命周期和 offset。这样的诊断能让读者把错误和书中讲过的合同对应起来。

### 上层编排回归：RAG、tool call 和 agent trace

第三册的验收不能只停在 kernel、logits 和 tokens/s。真实 AI 应用常通过 LangChain 类框架、LangGraph 类状态机或 LlamaIndex 类数据编排层进入推理服务。底层引擎没有变，上层 prompt template、retriever、tool schema 或 streaming callback 变了，也足以让质量、延迟和资源预算漂移。因此 release gate 要把“应用编排事实”也纳入回归。

Listing 4.84 上层编排回归 fixture

```
OrchestrationFixture:
 fixture_id
 messages_json
 chat_template_hash
 tokenizer_hash
 retriever_fixture_id
 retrieved_chunk_ids
 citation_map_expected
 tool_schema_hash
 tool_responses
 max_tool_steps
 expected_prompt_token_count
 expected_stop_reason
 expected_stream_event_types
```

这份 fixture 的目标不是测试某个 Python 框架，而是固定跨层合同。若 `messages_json` 相同但 `chat_template_hash` 变了，token ids 可能完全不同；若 retriever 返回同一批 chunk 但 citation map 变了，RAG 答案的可审计性就变了；若 tool schema 从字符串字段改成枚举字段，structured output 的验证路径和错误恢复都会改变。

| 回归层级      | 检查对象                                      | 失败含义              |
|-----------|-------------------------------------------|-------------------|
| prompt    | token ids、token count、template hash       | 模板或 tokenizer 漂移  |
| retrieval | chunk ids、rank、citation span              | RAG 上下文不再可复现      |
| tool call | JSON schema、idempotency key、time-out      | agent loop 事务边界改变 |
| streaming | event 顺序、finish reason、cancel event       | UI/callback 状态机漂移 |
| quality   | answer diff、citation diff、schema validity | 应用输出质量漂移          |
| SLO       | TTFT、TPOT、tool wait、queue wait            | 上层编排改变了资源形状       |

这里的质量检查要分层。纯生成任务可以比较 logits diff、top-k overlap 和固定 seed 下的 token sequence；RAG 任务要检查答案是否引用了允许 chunk，引用 id 是否能回到 prompt span；tool-call 任务要检查 JSON 是否满足 schema，工具是否按 idempotency key 执行一次，取消后是否没有迟到结果写回上下文。若只比较最终文本，很多危险漂移会被自然语言表面的相似性掩盖。

Listing 4.85 上层编排 release gate

```
1. tokenizer gate:
```

```

messages -> prompt bytes -> token ids are unchanged

2. retrieval gate:
 query -> chunk ids -> citation map are reproducible

3. tool gate:
 tool calls validate against schema and step budget

4. trace gate:
 stream events preserve order, parent span and finish reason

5. context budget gate:
 history + retrieval + tool result + output reserve <= max context

6. quality gate:
 answer, citation, structured output and refusal behavior pass fixture

7. SLO gate:
 queue wait, TTFT, TPOT and tool wait remain under thresholds

```

这条门禁把上层框架和底层引擎接在一起。底层 kernel 优化若改变 tokenizer golden 或 stop token 行为，应在 prompt gate 就失败；retriever 改版若让 prompt 多出 2000 token，应在 context budget gate 暴露；tool timeout 策略若让请求排队更久，应在 SLO gate 暴露。AI Infra 的工程边界不是“模型服务 API 返回 200”，而是从输入消息到输出事件的整条链路可复现、可度量、可回滚。

### 上层框架对标：比 API 更重要的是证据合同

对标上层框架时，不能写成“某框架支持 agent，某框架支持 RAG”。这类描述对工程能力帮助很小。真正要比较的是：它如何表达状态，如何保存 checkpoint，如何把文档变成 chunk 和 index，如何处理工具副作用，如何暴露 trace，如何让底层推理服务看到 token budget 和取消。

| 系统类型                   | 值得拆的设计                                            | 本项目需要迁移的合同                                   |
|------------------------|---------------------------------------------------|----------------------------------------------|
| Agent framework        | model/tool/message/middleware 抽象                  | tool schema、step budget、stream event         |
| State graph runtime    | durable step、checkpoint、resume、interrupt          | run id、step id、checkpoint id、重放边界            |
| RAG/data framework     | ingestion、node/chunk、index、retriever、query engine | doc/chunk/embedding/index epoch、citation map |
| Serving engine         | batching、KV、scheduler、streaming                   | TTFT、TPOT、KV budget、cancel/backpressure      |
| Local inference engine | tokenizer、model load、quant、backend                | manifest、tokenizer hash、backend plan、oracle  |

对标报告应固定同一组应用场景，而不是每个框架跑不同 demo。推荐至少四个 fixture：

**Listing 4.86** 上层框架对标 fixture

```

1. plain chat:
 messages -> prompt -> streaming tokens

2. RAG QA:
 documents -> chunks -> retrieve -> rerank -> answer with citations

3. tool call:
 model proposes tool -> schema validate -> execute -> continue

```

**4. durable agent:**

model/tool/checkpoint -> crash or pause -> resume exactly once

每个 fixture 都要报告相同字段:

**Listing 4.87** UpperFrameworkComparisonReport

```
framework_name
fixture_id
template_hash_visible
tokenizer_hash_visible
prompt_token_count
retrieval_trace_available
citation_map_available
tool_schema_enforced
checkpoint_available
cancel_propagates_to_model
cancel_propagates_to_tool
stream_event_schema
ttft_ms
end_to_end_p95_ms
failure_diagnosis_quality
```

这份表能把“好用”拆成工程事实。某框架 prompt 体验很好，但不暴露 token span，本地引擎就难以做 citation oracle；某状态图 runtime checkpoint 完整，但 tool 输出没有 token budget，长结果仍会压垮 context；某 RAG 框架索引方便，但若 index epoch 和 ACL 不进入 trace，多租户审计就不够。对标的目标不是选边站，而是把成熟系统的合同吸收进自己的项目。

**RAG/Agent 回归集的最小目录**

第三册项目如果要展示上层能力，仓库里应有一组可运行 fixture，而不是只在正文描述。最小目录可以这样组织：

**Listing 4.88** 上层回归集目录结构

```
tests/orchestration/
rag/
 docs/
 chunks-golden.jsonl
 embeddings-manifest.json
 retrieval-cases.jsonl
 citation-golden.jsonl
agents/
 tool-schemas/
 tool-fixtures.jsonl
 durable-state-cases.jsonl
 cancellation-cases.jsonl
prompts/
 chat-template-golden.jsonl
 tokenizer-golden.jsonl
reports/
 rag-eval.csv
 agent-eval.csv
```

slo-eval.csv

这套目录不要求一开始支持真实外部向量库。第一版可以用内存向量索引和固定 embedding fixture，但合同要完整：doc id、chunk id、embedding version、index epoch、query id、citation map、tool call id、checkpoint id 都要出现。之后换成真正的向量库、reranker 或 LangChain/LlamaIndex 接入层时，测试对象可以替换，oracle 不需要重写。

框架对标：尊重成熟实现，也要知道差距在哪里

自研引擎需要对标成熟框架，但对标不能只贴一个 tokens/s。成熟框架可能使用不同量化格式、不同 tokenizer、不同线程数、不同 GPU kernel、不同 batch 策略。对标前要固定变量：

- 同一模型或同一模型切片。
- 同一 tokenizer 和 prompt set。
- 同一量化格式或明确说明格式差异。
- 同一线程数、亲和策略或同一 GPU。
- 同样区分加载、prepare、prefill、decode 和 sampler。
- 同样报告正确性或至少 logits/生成差异。

对标表应该能说明差距：

| 指标            | 本项目 | 成熟框架 | 解释                        |
|---------------|-----|------|---------------------------|
| load/prepare  | A   | B    | mmap、packing、上传差异         |
| prefill       | A   | B    | GEMM/attention kernel 差异  |
| decode        | A   | B    | GEMV、KV cache、launch 差异   |
| memory peak   | A   | B    | arena、packed weight、KV 策略 |
| quality drift | A   | B    | 量化和 sampler 差异            |

如果差距很大，也不是失败。高质量工程报告应该定位差距来源：是算法、layout、kernel、线程、设备同步、量化、还是 runtime 调度。只要能定位，就能进入下一轮优化。

发布前验收清单

第三册对应的 C++20 项目在发布一个阶段性版本前，至少应该完成下面清单：

验收与评分标准

- 模型 manifest、tokenizer 合同、shape/dtype/quant verifier 有负例测试。
- reference converter、单算子 reference、层级 reference 和小端到端 reference 可运行。
- CPU scalar、CPU optimized、GPU 或 fallback 后端都通过 oracle。
- activation arena、workspace、packed weight、KV cache 有生命周期和容量测试。
- prefill/decode 分别有 benchmark，报告 time to first token、tokens/s、latency 分位数和内存峰值。
- 量化报告同时包含容量、scale metadata、logits/top-k、生成序列和性能。
- profiler event schema 稳定，报告能追溯到 segment、kernel、backend 和 context length。
- 性能回归门禁记录硬件、线程、prompt、baseline 和允许退化阈值。
- 与成熟框架的对标至少能解释最大差距来源。

这份清单看起来严格，但它正是 AI infra 工程和普通 demo 的区别。demo 证明“能跑”；验收体系证明“能长期改、能定位错、能解释快慢、能对用户负责”。

### 阶段发布报告模板

每个阶段发布时都应该留一份短报告。模板可以固定，内容不需要冗长，但必须能复现。

Listing 4.89 阶段发布报告模板

```
ReleaseReport:
 version
 model_scope
 supported_backends
 supported_quant_profiles
 correctness:
 reference_commit
 oracle_profiles
 failed_or_skipped_cases
 performance:
 hardware
 prompt_set
 prefill_tokens_per_s
 ttft_ms
 decode_tokens_per_s
 p95_step_latency_ms
 memory_peak_bytes
 known_limits:
 max_context
 unsupported_dtypes
 unsupported_shapes
 fallback_paths
 comparison:
 baseline_version
 mature_framework_reference
 largest_gap_explanation
```

这个报告能直接回答三个问题：能跑哪些模型和 profile，正确性证据是什么，性能瓶颈还在哪里。若报告写不出来，说明项目某些合同还没有建立好。

### 验收失败时的处理顺序

失败后不要直接调阈值。处理顺序应该是：

1. 先确认 fixture、模型、prompt、backend plan 和量化 profile 是否一致。
2. 再确认 reference 是否可信，尤其 byte converter 和 tokenizer。
3. 定位最早失败层级：byte、operator、layer、logits、sequence、performance。
4. 若是 correctness，先修实现或合同；若是性能，先确认测量口径和硬件状态。
5. 只有在误差来源被解释后，才调整 tolerance profile。

阈值是合同，不是让测试通过的旋钮。尤其是量化和 GPU fallback，不能因为输出文本看起来还行就放宽 logits/top-k 检查。

### 一次 CPU kernel 优化的发布门禁示例

假设本周把 **linear** 的 CPU 路径从标量 baseline 换成 packed int8 SIMD kernel。一个合格发布门禁不应该只跑一个 benchmark，而要按照证据链通过。

Listing 4.90 CPU int8 kernel 发布门禁



1. byte oracle:  
int8 weights and scale metadata decode to expected values
2. packed weight oracle:  
packed panel maps back to original W[out, in]
3. operator oracle:  
scalar reference vs packed SIMD output
4. layer oracle:  
decoder block output under fixed fixture
5. logits oracle:  
final logits and top-k overlap
6. performance gate:  
decode latency, p95, memory peak and pack time
7. fallback check:  
unsupported ISA falls back to scalar or refuses at plan time

这条门禁能定位失败。例如 byte oracle 失败，说明量化字节解释有错；packed weight oracle 失败，说明 layout 转换有错；operator oracle 失败，说明 SIMD lane、accumulator、tail 或 scale 有错；layer oracle 失败，可能是 arena 复用、fusion 或状态边；performance gate 失败，才进入 cache、线程、NUMA、带宽和反汇编分析。

### 一次 KV cache bug 的定位示例

KV cache bug 很常见，因为它跨 token 存活，且 shape 往往看起来正确。假设长上下文续写时 logits 从第 200 个 token 开始漂移，可以按下面顺序定位：

**Listing 4.91** KV cache 漂移定位流程

```

check tokenizer:
 token ids match reference

check model position:
 each token position is monotonic and not reset by cache slot

check RoPE checkpoint:
 q/k after rope match reference at positions 0, 1, 127, 200

check KV append:
 key/value written at expected logical position

check KV read:
 query head maps to correct kv head
 attention reads positions 0..current_position

check attention probability:
 score and softmax drift starts at which position

```

如果只比较最终生成文本，很难知道问题在哪。把 checkpoint 放在 RoPE、KV append、KV

read 和 attention probability，能把“长文本质量差”拆成具体合同错误。

### 一次性能回归的排查示例

性能回归也要按阶段拆。假设版本 B 比版本 A 的 decode tokens/s 下降 18%，不要直接改 kernel。先确认报告坐标一致：

- 模型、prompt set、quant profile、backend plan 是否相同。
- 线程数、亲和、频率、温度和系统负载是否可比。
- pack time 是否被算进 decode。
- fallback count 是否增加。
- context length 分布是否变化。
- p50 和 p95 是否一起下降，还是只有尾延迟变差。

随后看 profiler breakdown：

| 现象                      | 可能原因                          | 下一步证据                       |
|-------------------------|-------------------------------|-----------------------------|
| 权重 segment 变慢           | packing layout、SIMD path、NUMA | packed oracle、perf counters |
| KV segment 随 context 变差 | KV layout、cache miss、TLB      | context bucket 报告           |
| sampler 时间上升            | logits copy、top-k 实现          | sampler trace               |
| p95 上升但 p50 稳定          | 调度、队列、页错误                     | queue wait、page fault       |
| GPU kernel 快但端到端慢       | copy、sync、launch、fallback     | stream event trace          |

这类排查流程应该写进 release report 的失败处理说明。否则团队会在每次回归时重新发明定位方法。

### 验收报告中的最小可读示例

报告不需要长，但要能回答“这次改动可信吗”。下面是一个压缩示例：

**Listing 4.92** release gate 摘要示例

```
ReleaseGateSummary:
 change: cpu int8 packed linear kernel
 model_fixture: tiny_decoder_v3
 backend_plan: cpu_avx2_int8_pack_v1
 correctness:
 byte_oracle: pass
 packed_weight_oracle: pass
 operator_oracle_max_abs: 0.0061
 logits_topk_overlap: 9/10
 performance:
 decode_tokens_per_s: 18.4 -> 24.7
 p95_step_latency_ms: 68.2 -> 49.5
 pack_time_ms: 143
 memory_peak_mb: 812 -> 856
 risk:
 only group size 64 covered
 avx512 path not implemented
```

这样的摘要比“快了 34%”更有价值。它同时暴露了收益、误差、内存代价和限制。

### 硬验收

读完本节，应该能建立工程闭环：

- reference 定义正确答案，oracle 分层比较误差，profiler 证明性能变化。
- 测试矩阵必须覆盖 tail、非对齐、量化 group、context length、fallback 和不同后端。
- 覆盖率要优先服务 verifier、quant、memory planner、backend、runtime 等高风险路径。
- 性能门禁要固定硬件、输入、warmup、迭代、baseline 和允许退化阈值。
- profiler schema 要统一，让 CPU/GPU/reference/fallback 报告可以比较。
- 诊断要回到 manifest、IR、descriptor、backend plan 和 session 状态这些合同。
- 框架对标要拆开 load、prepare、prefill、decode、memory 和 quality，而不是只报一个总分。

当前仓库已经把这条门禁压成三个最小 artifact：

| artifact           | 证明什么                                                    | 回归入口                     |
|--------------------|---------------------------------------------------------|--------------------------|
| lcqi_trace         | prompt 到 generated token 的 shape/dtype/layout/values 链路 | CTest 检查 generated token |
| lcqi_ledger        | 7B FLOPs、KV 容量、字节流和带宽 tokens/s 上限                       | CTest 检查 int4@100GB/s    |
| lcqi_serving_probe | admission、queue、TTFT、TPOT、取消和拒绝率                        | CTest 检查超长请求拒绝           |

对应命令如下：

Listing 4.93 当前第三册最小证据链命令

```
ctest --test-dir books/cpu-volume-3/build/lcqi-release --output-on-failure

books/cpu-volume-3/build/lcqi-release/labs/linux_cpu_inference/lcqi_trace \
> books/cpu-volume-3/results/lcqi-reference-trace-2026-06-23.csv

books/cpu-volume-3/build/lcqi-release/labs/linux_cpu_inference/lcqi_ledger \
> books/cpu-volume-3/results/lcqi-sevenb-ledger-2026-06-24.csv

books/cpu-volume-3/build/lcqi-release/labs/linux_cpu_inference/↵
↵ lcqi_serving_probe \
> books/cpu-volume-3/results/lcqi-serving-slo-probe-2026-06-24.csv
```

这三个 artifact 是最小验收入口；本章进一步规定 fuzz、sanitizer、CI lane、性能回归、perf stat、火焰图和工业框架对标要接到同一套 release gate。下一步任何新增 kernel、memory planner 或 serving runtime，都应该把输出接到这些口径，而不是重新发明报告格式。

至此，后端与验收部分形成闭环：CPU 和 GPU 后端不是孤立优化技巧，而是以 C++20 资源边界为基础，以编译器 lowering 为入口，以 oracle/profiler/回归门禁为出口的工程系统。

# 实践卷：本地 CPU 推理引擎切片

第三册实践卷入口是：

```
1 books/cpu-volume-3/labs/linux_cpu_inference
```

它不是一个玩具分类器的终点，而是本地 CPU 量化推理引擎的第一条可验证切片：模型 manifest、tokenizer 合同、int8 线性层、reference decoder trace、7B 账本和 serving SLO probe。每个实践都要回答“这个结果由哪个合同保证”，而不是只看能否输出一个 token。

## A.1 零、贯穿主线工程

第三册主线工程是 `linux_cpu_inference`，简称 LCQI。它不是每章换一个小 demo，而是持续推进同一条本地 CPU 推理链路：模型资产怎样解释，prompt 怎样变成 token，int8 kernel 怎样和 reference 对照，decoder trace 怎样定位错误，7B ledger 怎样估算资源，serving probe 怎样把推理放回服务合同。

| 阶段            | 主线代码             | 学到的工程能力                                   |
|---------------|------------------|-------------------------------------------|
| 模型资产          | 模型与 tokenizer    | dtype、shape、offset、词表和模板 hash 必须可校验       |
| 最短闭环          | tiny MLP         | tiny MLP 先跑通加载、linear、logits 和 golden 类别  |
| Kernel 对照     | int8 kernel      | scalar、workspace、AVX2 路径用 max diff 证明语义一致 |
| Decoder trace | reference trace  | 用 checkpoint 定位 embedding 到 logits 的错误边界  |
| 资源和服务         | ledger 与 serving | FLOPs、KV、TTFT、TPOT、拒绝和取消都变成报告字段           |
| 验收网           | LCQI 测试          | 所有优化和新增 shape 必须回到测试和 golden trace        |

## A.2 一、工程入口

主要文件如下：

```
1 include/lcqi/safetensors.hpp
2 include/lcqi/tokenizer.hpp
3 include/lcqi/int8_kernels.hpp
4 include/lcqi/reference_decoder.hpp
5 include/lcqi/inference.hpp
6 include/lcqi/model.hpp
7 tests/lcqi_tests.cpp
8 models/tiny_mlp_i8.txt
9 models/tiny_tokenizer.txt
```

构建和测试命令如下：

Listing A.1 第三册 LCQI 实验构建和测试

```

1 cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -<
 ↪ DCMMAKE_BUILD_TYPE=Debug
2 cmake --build books/cpu-volume-3/build/lcqi-debug
3 ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure

```

### A.3 二、阶段 0: manifest 先说明字节怎样解释

safetensors manifest 是模型资产目录表，说明有哪些张量、每个张量的 dtype、shape 和 data offsets。它不是完整模型执行，也不保证数学结果正确；它只保证“字节应该怎样被解释”。

**SafeTensorEntry** 的字段要逐个理解：**name** 是张量名；**dtype** 是元素类型；**shape** 是维度；**data\_begin/data\_end** 是 payload 中的字节范围；**byte\_size** 是该范围长度。**SafeTensorManifest** 还记录 header 大小、数据起始偏移和 metadata。

阶段化验收：

1. 构造一个包含两个张量的 safetensors fixture；
2. 读取 header，并检查 **header\_size** 和 **data\_start\_offset**；
3. 查找 **model.embed\_tokens.weight**，验证 dtype、shape 和 offset；
4. 构造 payload 太短的坏文件，确认 bad offsets 被拒绝。

如果 manifest 没验收，后面的 tokenizer、decoder、kernel 都是在不可信字节上运行。

### A.4 三、阶段 1: tokenizer 合同固定输入身份

tokenizer 把文本 prompt 变成 token id 序列。合同包括 BOS、EOS、UNK、词表和 chat template hash。BOS 是序列开始标记，EOS 是序列结束标记，UNK 是未知词标记；chat template hash 用来证明对话模板没有被偷偷换掉。

样例 “tok1tok2tok3” 在 **tiny\_tokenizer.txt** 中应编码为 **[0,1,2,3,4]**，其中 0 是 BOS，4 是 EOS。若 tokenizer 没有 UNK，遇到未知 token 必须报错，而不是静默变成 0 或跳过。测试 **test\_tokenizer\_contract** 和 **test\_tokenizer\_rejects\_unknown\_without\_unk** 保护这两个边界。

推理报告里必须记录 tokenizer contract hash。否则同一段文字在不同模板下变成不同 token，decoder trace 的对比就失去意义。

### A.5 四、阶段 2: tiny MLP 是最短闭环

**models/tiny\_mlp\_i8.txt** 和 **run\_inference** 提供最短闭环：加载一个量化 MLP，执行 int8 linear、ReLU、输出 logits，并返回 predicted class。

这个闭环的价值不是模型强，而是把数据路径走通。测试 **test\_tiny\_mlp** 检查输入大小、隐藏层大小、输出大小、hidden 激活、两个 logits 和最终类别。若这里失败，先不要调 decoder；应先查模型加载、scale、bias、矩阵布局和 ReLU。

### A.6 五、阶段 3: int8 kernel 从 scalar 对照到 AVX2

**PackedLinearI8** 把量化线性层重新排布成更适合计算的格式。阶段顺序如下：

1. 用 **make\_deterministic\_i8\_layer** 和 **make\_deterministic\_input** 生成可重复输入；
2. 用基础 **linear\_i8\_packed** 产生 reference 输出；
3. 用 **linear\_i8\_packed\_with\_workspace** 检查 workspace 版本不改语义；
4. 若 **linear\_i8\_packed\_avx2\_available** 为 true，再测 AVX2 路径；
5. 用 **max\_abs\_diff** 比较误差，不能只比较速度。

benchmark 入口如下：

```

1 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_bench 200
2 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_trace
3 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_ledger
4 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_serving_probe

```

报告必须写清当前机器是否支持 AVX2/FMA、benchmark 是 Debug 还是 Release、输入 shape、repeat 次数和 max abs diff。若 AVX2 backend 标为 unavailable，不能把源码里有 AVX2 文件当作性能证据。

## A.7 六、阶段 4: reference decoder trace 定位错误边界

reference decoder 是可信但不追求快的 decoder-only 模型执行路径。它包括 embedding、RMSNorm、Q/K/V、RoPE、KV cache slot、attention、SwiGLU、final norm、logits 和 sampler。

`run_reference_decode_with_trace` 返回 logits 和一组 checkpoint。checkpoint 不是日志装饰，它是定位错误边界的证据。若 logits 不对，按顺序查：tokenizer 输出是否稳定；embedding checkpoint 是否正确；RMSNorm checksum 是否变化；Q/K/V shape 是否对；RoPE 位置是否对；KV cache slot 是否写入正确层和位置；attention 输出是否和前面 token 对齐。

测试中固定了 tiny decoder 的 vocab size、预测 token、logits 数值、trace token count 和 hidden size。新增 shape 时必须同步新增 golden trace，不能只看最终 token。

## A.8 七、阶段 5: 7B 账本不是实测吞吐

`lcqi_ledger` 给出典型 7B decoder-only 模型的 FLOPs、KV 容量和带宽上限估算。ledger 是账本，不是 benchmark。它回答“如果模型参数、上下文长度、并发数固定，大概会消耗多少内存和带宽”，不回答“这台机器真实每秒能出多少 token”。

账本报告至少包含：模型规模、层数、hidden size、head 数、KV dtype、context、batch、KV cache 容量、每 token 粗略 FLOPs、内存带宽约束。若只写一个“7B 可以跑/不能跑”，没有解释价值。

## A.9 八、阶段 6: serving probe 把推理放回服务合同

`lcqi_serving_probe` 固定短请求、RAG 请求、取消请求和超长请求，检查 admission、queue wait、TTFT、TPOT、取消回收和拒绝原因。TTFT 是 time to first token，表示请求到首 token 的时间；TPOT 是 time per output token，表示后续 token 平均间隔。

服务化实践不能只看单请求速度。要回答：超长请求是否被拒绝；取消请求是否释放 KV 预算；RAG 请求是否因上下文更长占用更多资源；queue wait 是否被计入；拒绝原因是否能解释给调用方。

## A.10 九、项目阶段清单

| 阶段 | 代码切片                              | 验收证据                                       |
|----|-----------------------------------|--------------------------------------------|
| 0  | <code>safetensors</code> manifest | dtype、shape、offset 和坏 payload 都被测试         |
| 1  | tokenizer                         | BOS/EOS/UNK、模板 hash 和未知 token 行为固定         |
| 2  | tiny MLP inference                | hidden、logits 和 predicted class 有 golden 值 |
| 3  | int8 packed kernel                | scalar/workspace/AVX2 与 max diff 对照        |
| 4  | reference decoder trace           | 每个 checkpoint 有 shape、dtype、checksum       |
| 5  | 7B ledger                         | FLOPs、KV 容量和带宽上限写清                         |
| 6  | serving probe                     | admission、TTFT、TPOT、取消和拒绝原因可观察             |



大作业阶段 6 是为新增 decode shape 写完整验收。至少包含 safetensors manifest 样例、tokenizer 输入、reference trace golden、kernel max diff、ledger 行和 serving probe 中的内存预算变化。任何一项缺失，都只能算局部实验，不能写成完整推理链路能力。

## A.11 十、每章代码任务矩阵

第三册每章都围绕本地 CPU 推理链路推进。不要把章节学成概念清单；每一章都必须给 LCQI 工程增加或复查一个可运行切片。

| 章节                     | 代码任务                                           | 验收证据                                   |
|------------------------|------------------------------------------------|----------------------------------------|
| 端到端推理链                 | 跑通 <code>run_inference</code> tiny MLP         | hidden、logits、predicted class 有 golden |
| 张量布局地址流                | 为张量写 shape、stride、offset 报告                    | 地址计算不越界，shape 错误会拒绝                    |
| GEMM packing SIMD      | <code>PackedLinearI8</code> scalar 与 packed 对照 | max abs diff 在阈值内                      |
| 量化系统总览                 | 写 scale、zero point、requantize 手算样例             | int8 到 float 路径可解释                     |
| int8 requantize        | 扩展 deterministic layer/input case              | 输出与 reference 对照                       |
| 低 bit KV cache         | 设计 KV dtype 容量账本                               | ledger 能解释内存预算变化                       |
| 量化 C++ verifier        | safetensors dtype/shape/offset 检查              | 坏 dtype、坏 shape、坏 offset 都被拒绝          |
| 7B compute ledger      | 跑 <code>lcqi_ledger</code>                     | FLOPs、KV 容量、带宽上限写成 CSV                 |
| Transformer 与 KV cache | 运行 reference decoder trace                     | 每个 checkpoint 有 shape、stride、checksum  |
| reference decoder      | 固定 prompt、tokenizer、logits golden              | trace 中 token 位置和 layer id 稳定          |
| 采样算法                   | 对 argmax 或 sampler 写 deterministic case        | 相同 logits 得到相同 token                   |
| KV cache 实现细节          | 验证 slot 写入层、位置、head 维度                         | 错位时 trace 能定位边界                        |
| 服务调度业务                 | 跑 <code>lcqi_serving_probe</code>              | TTFT、TPOT、拒绝、取消可观察                     |
| RAG 与 agents           | 设计长上下文请求预算                                     | admission 能解释拒绝原因                      |
| 模型导入 shape verifier    | 加 safetensors fixture                          | tensor name、dtype、shape、offset 全部校验    |
| 真实模型加载闭环               | 说明 tiny 模型和真实模型差距                              | 不把 tiny 成功写成 7B 已完成                    |
| 运行时内存计划                | 写 KV 和权重内存预算表                                  | batch/context 变化会改变预算                  |
| AI 编译器运行时              | 描述 op lowering 到 kernel 的字段                    | 不跳过 dtype、layout、shape                 |
| IR pass lowering       | 写一个 pass 输入输出合同                                | 每个 pass 有前后不变量                         |
| CPU 后端优化               | scalar、workspace、AVX2 可用性检查                    | AVX2 unavailable 时报告真实状态               |
| GPU 硬件基础               | 写 GPU 后端能力边界说明                                 | 当前 CPU 工程不伪装 GPU 实现                    |
| GPU 后端优化               | 设计 host/device 数据移动报告                          | 明确哪些只是设计题                              |
| CPU decode microkernel | 对 decode 热路径写 shape sweep                      | repeat、shape、diff 都记录                  |
| GPU decode 端到端         | 写 CPU/GPU 对照计划                                 | 未实现部分不得写成已完成                           |
| 工业 LLM 专项              | 对比 ggml/llama.cpp/oneDNN 的验证口径                 | 只采用能在本工程复现的证据                          |

工程验收回归                      汇总 tests、trace、ledger、serving probe    缺任一项不得发布推理结论

## A.12 十一、递进大作业：LCQI 推理引擎

| 阶段 | 交付物                     | 通过标准                                                   |
|----|-------------------------|--------------------------------------------------------|
| 0  | 模型资产合同                  | safetensors manifest 和 tokenizer contract 可校验          |
| 1  | tiny MLP 闭环             | 模型加载、int8 linear、logits、分类都有 golden                    |
| 2  | kernel 对照               | scalar、workspace、AVX2 路径语义一致                           |
| 3  | reference decoder trace | checkpoint 能定位 embedding 到 logits 的错误边界                |
| 4  | ledger                  | 7B 规模的 FLOPs、KV、带宽预算可复查                                |
| 5  | serving probe           | admission、TTFT、TPOT、取消、拒绝原因可观察                         |
| 6  | 新 shape 验收              | manifest、tokenizer、trace、kernel、ledger、serving 全链路同步更新 |

## 附录 B

# 完整源码清单：LCQI 本地 CPU 推理引擎

本附录完整收录 `books/cpu-volume-3/labs/linux_cpu_inference` 的主线工程。第三册的代码主线是 LCQI：从模型 manifest、tokenizer、tiny MLP、int8 kernel、reference decoder trace、7B ledger 到 serving probe。读者必须把每个文件看成全链路的一段，而不是孤立 demo。

## B.1 一、CMakeLists.txt

CMake 文件定义 LCQI 库、CLI、trace、ledger、serving probe、benchmark、可选 oneDNN 对标和测试目标。读者应从这里看哪些能力是必构建，哪些是平台可选。

Listing B.1 linux\_cpu\_inference/CMakeLists.txt

```
1 add_library(lcqi STATIC
2 src/inference.cpp
3 src/int8_kernels.cpp
4 src/model.cpp
5 src/reference_decoder.cpp
6 src/safetensors.cpp
7 src/tokenizer.cpp
8)
9
10 if(CMAKE_SYSTEM_PROCESSOR MATCHES "x86_64|AMD64|i[3-6]86")
11 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
12 target_sources(lcqi PRIVATE src/int8_kernels_avx2.cpp)
13 set_source_files_properties(src/int8_kernels_avx2.cpp PROPERTIES ↵
14 ↵ COMPILE_OPTIONS "-mavx2;-mfma")
15 target_compile_definitions(lcqi PRIVATE LCQI_ENABLE_AVX2=1)
16 endif()
17 endif()
18
19 if(NOT CMAKE_SYSTEM_NAME STREQUAL "Linux")
20 message(WARNING "LCQI is written for the book's local Linux CPU target; ↵
21 ↵ current system is ${CMAKE_SYSTEM_NAME}.")
22 endif()
23
24 target_include_directories(lcqi
25 PUBLIC
26 ${CMAKE_CURRENT_SOURCE_DIR}/include
27)
28
29 target_compile_features(lcqi PUBLIC cxx_std_20)
30
31 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
32 target_compile_options(lcqi PRIVATE -Wall -Wextra -Wpedantic)
33 endif()
34
35 add_executable(lcqi_cli
36 src/main.cpp
```

```
35)
36
37 target_link_libraries(lcqi_cli PRIVATE lcqi)
38
39 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
40 target_compile_options(lcqi_cli PRIVATE -Wall -Wextra -Wpedantic)
41 endif()
42
43 add_executable(lcqi_bench
44 src/bench.cpp
45)
46
47 target_link_libraries(lcqi_bench PRIVATE lcqi)
48
49 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
50 target_compile_options(lcqi_bench PRIVATE -Wall -Wextra -Wpedantic)
51 endif()
52
53 add_executable(lcqi_trace
54 src/trace.cpp
55)
56
57 target_link_libraries(lcqi_trace PRIVATE lcqi)
58
59 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
60 target_compile_options(lcqi_trace PRIVATE -Wall -Wextra -Wpedantic)
61 endif()
62
63 add_executable(lcqi_ledger
64 src/ledger.cpp
65)
66
67 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
68 target_compile_options(lcqi_ledger PRIVATE -Wall -Wextra -Wpedantic)
69 endif()
70
71 add_executable(lcqi_serving_probe
72 src/serving_probe.cpp
73)
74
75 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
76 target_compile_options(lcqi_serving_probe PRIVATE -Wall -Wextra -Wpedantic)
77 endif()
78
79 find_package(dnnl QUIET)
80 if(dnnl_FOUND)
81 add_executable(lcqi_onednn_bench
82 src/onednn_bench.cpp
83)
84 target_link_libraries(lcqi_onednn_bench PRIVATE DNNL::dnnl)
85 target_compile_features(lcqi_onednn_bench PRIVATE cxx_std_20)
86 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
```

```

87 target_compile_options(lcqi_onednn_bench PRIVATE -Wall -Wextra -Wpedantic)
88 endif()
89 endif()
90
91 add_executable(lcqi_tests
92 tests/lcqi_tests.cpp
93)
94
95 target_link_libraries(lcqi_tests PRIVATE lcqi)
96
97 if(CMAKE_CXX_COMPILER_ID MATCHES "Clang|GNU")
98 target_compile_options(lcqi_tests PRIVATE -Wall -Wextra -Wpedantic)
99 endif()
100
101 add_test(NAME lcqi_tests COMMAND ${<TARGET_FILE:lcqi_tests>})
102 add_test(NAME lcqi_trace COMMAND ${<TARGET_FILE:lcqi_trace>})
103 set_tests_properties(lcqi_trace PROPERTIES
104 PASS_REGULAR_EXPRESSION "tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4"
105)
106 add_test(NAME lcqi_ledger COMMAND ${<TARGET_FILE:lcqi_ledger>})
107 set_tests_properties(lcqi_ledger PROPERTIES
108 PASS_REGULAR_EXPRESSION "bandwidth_limit,int4_at_100_gbps,23.602,tokens/s"
109)
110 add_test(NAME lcqi_serving_probe COMMAND ${<TARGET_FILE:lcqi_serving_probe>})
111 set_tests_properties(lcqi_serving_probe PROPERTIES
112 PASS_REGULAR_EXPRESSION "request,req_too_long,0,memory_budget_exceeded"
113)

```

## B.2 二、README

README 给出本地构建、测试、CLI、benchmark、trace、ledger 和 serving probe 的命令。它是复现实验的入口，也说明 tiny 模型和真实 7B 模型之间的边界。

**Listing B.2** linux\_cpu\_inference/README.md

```

1 # Linux CPU Quantized Inference
2
3 这是第三册贯穿项目的第一阶段切片：Linux 本地 CPU 量化线性层和最小推理流程。第三
 ↪ 册的贯穿目标不是这个 MLP 本身，而是一台能推理开源 7B 左右 decoder-only 大
 ↪ 模型的本地引擎，并逐步覆盖 tokenizer、模型加载、Transformer 层、KV cache
 ↪ 、CPU/GPU 后端、正确性报告和性能对标。
4
5 目标平台是 x86-64 Linux 实验环境。完整性能报告应记录 CPU 型号、内核版本、编译器
 ↪ 版本、是否能使用 PMU、NUMA 和线程亲和能力。若运行环境缺少 perf、NUMA、线
 ↪ 程亲和性或硬件性能计数器权限，报告必须把相关结论降级为“未验证”，不能把源
 ↪ 码路径存在当作实测性能证据。后续 GPU 后端会作为独立 backend 接入，不改变
 ↪ 当前切片的语义合同。
6
7 当前版本支持：
8
9 - 教学文本模型格式 `LCQI_MODEL_V1`

```

```

10 - 最小 tokenizer 合同格式 `LCQI_TOKENIZER_V1`
11 - safetensors header manifest 解析: metadata、tensor name、dtype、shape、data ←
 ↳ offsets、offset 边界和 dtype/shape 字节数校验
12 - 两层 MLP 教学切片
13 - int8 权重加 per-layer scale
14 - float 输入和激活
15 - 量化线性层、ReLU、分类 argmax
16 - int8 linear scalar baseline、packed layout、AVX2 路径和 shape sweep benchmark
17 - tiny reference decoder: RMSNorm、float linear、RoPE、GQA KV cache、decode ←
 ↳ attention、SwiGLU、lm head
18 - reference trace CSV: 从 prompt/tokenizer 合同到 logits/sampler/token, 逐 ←
 ↳ checkpoint 输出 shape、stride、dtype、layout、checksum、max_abs、values ←
 ↳ 摘要
19 - 7B ledger CSV: 输出典型 7B shape 的 FLOPs、KV 容量、权重/KV 字节和 bandwidth-←
 ↳ only tokens/s 上限
20 - serving SLO probe CSV: 输出 admission、batch state、KV 预算、queue wait、TTFT←
 ↳ 、TPOT、取消回收和拒绝率
21 - CLI 推理和简单 benchmark
22 - CTest 正确性测试
23
24 后续扩展方向:
25
26 - 真实 7B 级模型 metadata 和 tokenizer
27 - 完整 BPE/SentencePiece tokenizer、GGUF/safetensors 权重导入、分片加载和 name ←
 ↳ mapping
28 - 多层 decoder-only Transformer reference path
29 - KV cache 分页、内存规划和可选量化
30 - CPU packed weight、SIMD、线程和 NUMA 优化
31 - GPU backend adapter 和 device kernel
32 - 与现有框架的 prefill/decode/内存/误差对标报告
33
34 构建:
35
36 ```bash
37 cmake -S books/cpu-volume-3 -B books/cpu-volume-3/build/lcqi-debug -↵
 ↳ DCMMAKE_BUILD_TYPE=Debug
38 cmake --build books/cpu-volume-3/build/lcqi-debug
39 ctest --test-dir books/cpu-volume-3/build/lcqi-debug --output-on-failure
40 ```
41
42 运行:
43
44 ```bash
45 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_cli \
46 books/cpu-volume-3/labs/linux_cpu_inference/models/tiny_mlp_i8.txt \
47 1.0 2.0 -1.0 0.5 1000
48 ```
49
50 kernel benchmark:
51
52 ```bash
53 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_bench 200

```



```

54 ```
55
56 输出 CSV 字段:
57
58 ```text
59 target_arch,input_size,output_size,output_block,repeat,backend,available,↵
 ↵ average_us,max_abs_diff,checksum
60 ```
61
62 当前 benchmark 按 backend 逐行输出: `scalar`、`packed_scalar`、`avx2`。x86-64 ↵
 ↵ 构建会编译 AVX2 路径; 不支持 AVX2/FMA 的机器会把该 backend 标为 `↵
 ↵ available=0`, 避免把源码存在误报成当前环境实测性能。
63
64 这还不是生产级高性能 kernel。当前 benchmark 建立 scalar baseline、packed layout↵
 ↵ 、AVX2 基线正确性和 shape sweep 口径。要达到完整 AI Infra 证据, 还必须继↵
 ↵ 续补反汇编分析、AVX-512、perf counter、oneDNN/OpenBLAS/llama.cpp/ggml 对↵
 ↵ 照、sanitizer、fuzz 和 CI。
65
66 safetensors manifest gate:
67
68 `lcqi_tests` 会运行时生成一个最小 safetensors fixture, 验证 header 长度、`↵
 ↵ __metadata__`、tensor name、dtype、shape、data offsets、payload 边界和 ↵
 ↵ dtype/shape 推导出的字节数。它还会生成 offset 超出 payload、dtype/shape ↵
 ↵ 字节数不匹配的坏文件, 确认加载器会拒绝。这个 gate 只覆盖模型资产目录表, ↵
 ↵ 不等于已经支持完整 LLaMA/Qwen 权重映射、分片合并或 mmap 执行; 后续真实模↵
 ↵ 型加载必须在这个 manifest 上继续接 name mapping、shape verifier、quant ↵
 ↵ descriptor 和 first token trace。
69
70 reference decoder trace:
71
72 ```bash
73 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_trace
74 ```
75
76 输出 CSV 字段:
77
78 ```text
79 name,token_position,layer_id,shape,stride,dtype,layout,checksum,max_abs,values
80 ```
81
82 这条 trace 固定 prompt fixture `tok1 tok2 tok3`, tokenizer 输出完整 `tokens↵
 ↵ =[0,1,2,3,4]`, 其中 `0/4` 是 BOS/EOS; decoder trace 会剥离 BOS/EOS 后进入↵
 ↵ tiny decoder。这样报告能同时固定 tokenizer 合同和 decoder 输入合同, 避免↵
 ↵ 把二者混成一个不可解释的 token 数组。随后 trace 把 embedding、RMSNorm、Q/↵
 ↵ K/V、RoPE、KV cache slot、attention、SwiGLU、layer output、final norm、↵
 ↵ logits、argmax sampler 和 generated token 串成可回归检查的硬链路。`↵
 ↵ lcqi_tests` 会检查 tokenizer contract hash、关键 checkpoint 的 shape、↵
 ↵ stride、dtype、layout 和 logits golden, CTest 也会直接运行 `lcqi_trace`。
83
84 当前 tokenizer fixture 的关键输出:
85
86 ```text

```

```

87 tokenizer,0,-1,5,1,i32,contiguous,10,4,0;1;2;3;4
88 tokenizer_contract,0,-1,scalar,scalar,u64,fnv1a,13452902845388333734,0,tiny-↵
 ↵ chat-template-v1
89 ```
90
91 7B ledger:
92
93 ```bash
94 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_ledger \
95 > books/cpu-volume-3/results/lcqi-sevenb-ledger-2026-06-24.csv
96 ```
97
98 输出 CSV 字段:
99
100 ```text
101 section,item,value,unit,notes
102 ```
103
104 这份账本固定 `L=32,H=4096,n_q=32,n_kv=8,head_dim=128,context=4096`, 报告 decode↵
 ↵ FLOPs、KV 容量、fp16/int8/int4 每 token 字节和不同有效带宽下的 tokens/s ↵
 ↵ 上限。CTest 会检查 `int4_at_100_gbps=23.602 tokens/s` 这一行, 防止账本公↵
 ↵ 式漂移。
105
106 serving SLO probe:
107
108 ```bash
109 books/cpu-volume-3/build/lcqi-debug/labs/linux_cpu_inference/lcqi_serving_probe↵
 ↵ \
110 > books/cpu-volume-3/results/lcqi-serving-slo-probe-2026-06-24.csv
111 ```
112
113 输出 CSV 字段:
114
115 ```text
116 row_type,request_id,accepted,rejection_reason,prompt_tokens,reserved_tokens,↵
 ↵ kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,decode_step_p95_ms,↵
 ↵ tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,metric_name,↵
 ↵ metric_value
117 ```
118
119 probe 固定四个请求: 短请求、RAG 请求、取消请求和超长请求。它演示 admission 先按↵
 ↵ `prompt_tokens + max_new_tokens` 预留 KV, 取消请求释放预算, 超长请求返回↵
 ↵ `memory_budget_exceeded`, 并输出 `ttft_p95_ms`、`queue_wait_p95_ms`、`↵
 ↵ rejection_rate` 等服务化指标。

```

### B.3 三、模型公共头

`model.hpp` 定义 tiny MLP 模型结构、权重、bias、scale 和加载入口。它负责说明模型文件被解释成哪些 C++ 对象。

**Listing B.3** `include/lcqi/model.hpp`

```

1 #pragma once
2
3 #include <stdint>
4 #include <filesystem>
5 #include <string>
6 #include <vector>
7
8 namespace lcqi {
9
10 struct QuantizedLinearLayer {
11 std::int32_t input_size = 0;
12 std::int32_t output_size = 0;
13 float scale = 1.0F;
14 std::vector<std::int8_t> weights;
15 std::vector<float> bias;
16 };
17
18 struct TinyMlpModel {
19 std::int32_t input_size = 0;
20 std::int32_t hidden_size = 0;
21 std::int32_t output_size = 0;
22 QuantizedLinearLayer hidden;
23 QuantizedLinearLayer output;
24 };
25
26 TinyMlpModel load_model(const std::filesystem::path& path);
27
28 } // namespace lcqi

```

## B.4 四、推理公共头

`inference.hpp` 定义推理结果和 `run_inference` 入口。读者要看清这里返回 `hidden`、`logits` 和 `predicted class`，因为测试会用这些字段定位闭环错误。

Listing B.4 include/lcqi/inference.hpp

```

1 #pragma once
2
3 #include <stdint>
4 #include
5 #include <vector>
6
7 #include <lcqi/model.hpp>
8
9 namespace lcqi {
10
11 struct InferenceResult {
12 std::vector<float> logits;
13 std::int32_t predicted_class = 0;
14 };
15
16 void linear_i8(const QuantizedLinearLayer& layer,

```

```

17 std::span<const float> input,
18 std::span<float> output);
19
20 void relu_inplace(std::span<float> values);
21
22 InferenceResult run_inference(const TinyMlpModel& model,
23 std::span<const float> input);
24
25 } // namespace lcqi

```

## B.5 五、Reference Decoder 公共头

reference decoder 头文件定义张量、层配置、trace checkpoint、采样结果和 decode 函数。它是定位推理错误的主要合同，不只返回最终 token。

Listing B.5 include/lcqi/reference\_decoder.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include
5 #include <string>
6 #include <vector>
7
8 namespace lcqi {
9
10 struct LinearWeightsF32 {
11 std::int32_t input_size = 0;
12 std::int32_t output_size = 0;
13 std::vector<float> weights;
14 std::vector<float> bias;
15 };
16
17 struct DecoderConfig {
18 std::int32_t hidden_size = 0;
19 std::int32_t query_heads = 0;
20 std::int32_t kv_heads = 0;
21 std::int32_t head_dim = 0;
22 std::int32_t intermediate_size = 0;
23 std::int32_t vocab_size = 0;
24 std::int32_t max_context = 0;
25 float rms_epsilon = 1.0e-5F;
26 float rope_base = 10000.0F;
27 };
28
29 struct DecoderLayerWeightsF32 {
30 std::vector<float> rms_attention_weight;
31 LinearWeightsF32 wq;
32 LinearWeightsF32 wk;
33 LinearWeightsF32 wv;
34 LinearWeightsF32 wo;
35 std::vector<float> rms_mlp_weight;

```

[illegible]

```

88
89 [[nodiscard]] std::int32_t layer_count() const noexcept;
90 [[nodiscard]] std::int32_t filled_tokens() const noexcept;
91
92 private:
93 [[nodiscard]] std::size_t base_offset(std::int32_t layer_id,
94 std::int32_t model_position,
95 std::int32_t kv_head) const;
96 void validate_address(std::int32_t layer_id,
97 std::int32_t model_position,
98 std::int32_t kv_head) const;
99
100 DecoderConfig config_;
101 std::int32_t layer_count_ = 0;
102 std::int32_t filled_tokens_ = 0;
103 std::vector<float> keys_;
104 std::vector<float> values_;
105 std::vector<std::uint8_t> written_;
106 };
107
108 void rms_norm(std::span<const float> input,
109 std::span<const float> weight,
110 float epsilon,
111 std::span<float> output);
112
113 void linear_f32(const LinearWeightsF32& layer,
114 std::span<const float> input,
115 std::span<float> output);
116
117 void apply_rope(std::span<float> heads,
118 std::int32_t head_count,
119 std::int32_t head_dim,
120 std::int32_t model_position,
121 float rope_base);
122
123 void attention_decode(const DecoderConfig& config,
124 const ReferenceKVCache& cache,
125 std::int32_t layer_id,
126 std::int32_t model_position,
127 std::span<const float> query,
128 std::span<float> output);
129
130 ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
131 std::span<const std::int32_t> ↵
132 ↵ token_ids);
133
134 ReferenceDecodeTraceResult run_reference_decode_with_trace(
135 const ReferenceDecoderModel& model,
136 std::span<const std::int32_t> token_ids);
137
138 ReferenceDecoderModel make_tiny_reference_decoder_model();
139

```



```
139 } // namespace lcqi
```

## B.6 六、Int8 Kernel 公共头

int8 kernel 头文件定义 packed linear、deterministic fixture、scalar/workspace/AVX2 路径和误差比较。读者应注意 AVX2 可用性是运行时事实，不能只看源码存在。

**Listing B.6** include/lcqi/int8\_kernels.hpp

```
1 #pragma once
2
3 #include <stdint>
4 #include
5 #include <vector>
6
7 #include <lcqi/model.hpp>
8
9 namespace lcqi {
10
11 struct PackedLinearI8 {
12 std::int32_t input_size = 0;
13 std::int32_t output_size = 0;
14 std::int32_t output_block_size = 0;
15 float scale = 1.0F;
16 std::vector<std::int8_t> packed_weights;
17 std::vector<float> bias;
18 };
19
20 struct KernelBenchmarkCase {
21 std::int32_t input_size = 0;
22 std::int32_t output_size = 0;
23 std::int32_t output_block_size = 0;
24 std::int32_t repeat = 0;
25 };
26
27 struct KernelBackendBenchmark {
28 const char* name = "";
29 bool available = false;
30 double average_us = 0.0;
31 float max_abs_diff = 0.0F;
32 float checksum = 0.0F;
33 };
34
35 struct KernelBenchmarkResult {
36 KernelBenchmarkCase benchmark_case;
37 std::vector<KernelBackendBenchmark> backends;
38 };
39
40 PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
41 std::int32_t output_block_size);
42
43 void linear_i8_packed(const PackedLinearI8& layer,
```

```

44 std::span<const float> input,
45 std::span<float> output);
46
47 void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
48 std::span<const float> input,
49 std::span<float> output,
50 std::span<float> accumulator_workspace);
51
52 bool linear_i8_packed_avx2_available() noexcept;
53
54 void linear_i8_packed_avx2(const PackedLinearI8& layer,
55 std::span<const float> input,
56 std::span<float> output);
57
58 QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
59 std::int32_t output_size,
60 float scale);
61
62 std::vector<float> make_deterministic_input(std::int32_t input_size);
63
64 float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs);
65
66 KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& ←
 ↪ benchmark_case);
67
68 } // namespace lcqi

```

## B.7 七、Tokenizer 公共头

tokenizer 头文件固定 BOS/EOS/UNK、词表和 template hash。读者应重点看未知 token 行为是否明确，因为 prompt 合同不稳定会污染后续所有 trace。

Listing B.7 include/lcqi/tokenizer.hpp

```

1 #pragma once
2
3 #include <cstdint>
4 #include <filesystem>
5 #include <string>
6 #include <string_view>
7 #include <vector>
8
9 namespace lcqi {
10
11 struct TokenEntry {
12 std::string text;
13 std::int32_t id = 0;
14 };
15
16 struct TokenizerModel {
17 std::int32_t bos_id = -1;
18 std::int32_t eos_id = -1;

```

```

19 std::int32_t unk_id = -1;
20 std::string chat_template_hash;
21 std::vector<TokenEntry> vocab;
22 };
23
24 [[nodiscard]] TokenizerModel load_tokenizer(const std::filesystem::path& path);
25
26 [[nodiscard]] std::vector<std::int32_t> encode_prompt(
27 const TokenizerModel& tokenizer,
28 std::string_view prompt);
29
30 [[nodiscard]] std::uint64_t tokenizer_contract_hash(const TokenizerModel& ↵
 ↵ tokenizer);
31
32 } // namespace lcqi

```

## B.8 八、Safetensors 公共头

safetensors 头文件固定 dtype、shape、offset 和 payload 边界。它负责说明模型字节怎样被解释，不能和数学推理逻辑混在一起。

Listing B.8 include/lcqi/safetensors.hpp

```

1 #pragma once
2
3 #include <stdint>
4 #include <filesystem>
5 #include <string>
6 #include <string_view>
7 #include <utility>
8 #include <vector>
9
10 namespace lcqi {
11
12 struct SafeTensorEntry {
13 std::string name;
14 std::string dtype;
15 std::vector<std::int64_t> shape;
16 std::uint64_t data_begin = 0;
17 std::uint64_t data_end = 0;
18
19 [[nodiscard]] std::uint64_t byte_size() const;
20 };
21
22 struct SafeTensorManifest {
23 std::uint64_t header_size = 0;
24 std::uint64_t data_start_offset = 0;
25 std::vector<std::pair<std::string, std::string>> metadata;
26 std::vector<SafeTensorEntry> tensors;
27
28 [[nodiscard]] const SafeTensorEntry* find_tensor(std::string_view name) ↵
 ↵ const;

```

```

29 };
30
31 [[nodiscard]] SafeTensorManifest load_safetensors_manifest(
32 const std::filesystem::path& path);
33
34 } // namespace lcqi

```

## B.9 九、模型加载实现

模型加载实现负责解析 tiny 文本模型、权重、scale、bias 和 shape。读者应检查坏格式、维度和数值解析是否有清晰错误边界。

Listing B.9 src/model.cpp

```

1 #include <lcqi/model.hpp>
2
3 #include <fstream>
4 #include <limits>
5 #include <stdexcept>
6 #include <string>
7
8 namespace lcqi {
9 namespace {
10
11 constexpr std::int32_t LCQI_INT8_MIN_VALUE = -128;
12 constexpr std::int32_t LCQI_INT8_MAX_VALUE = 127;
13
14 std::string read_key(std::istream& input) {
15 std::string key;
16 if (!(input >> key)) {
17 throw std::runtime_error("unexpected end of model file");
18 }
19 return key;
20 }
21
22 void expect_key(std::istream& input, const std::string& expected) {
23 const std::string key = read_key(input);
24 if (key != expected) {
25 throw std::runtime_error("expected key '" + expected + "', got '" + key +
26 ↵ + "'");
27 }
28 }
29
30 std::int32_t read_i32(std::istream& input, const std::string& key) {
31 expect_key(input, key);
32 std::int32_t value = 0;
33 if (!(input >> value)) {
34 throw std::runtime_error("invalid int32 value for key '" + key + "'");
35 }
36 return value;
37 }

```

```

38 float read_float(std::istream& input, const std::string& key) {
39 expect_key(input, key);
40 float value = 0.0F;
41 if (!(input >> value)) {
42 throw std::runtime_error("invalid float value for key '" + key + "'");
43 }
44 return value;
45 }
46
47 std::vector<std::int8_t> read_i8_vector(std::istream& input,
48 const std::string& key,
49 std::int32_t count) {
50 expect_key(input, key);
51 if (count < 0) {
52 throw std::runtime_error("negative vector size for key '" + key + "'");
53 }
54 std::vector<std::int8_t> values(static_cast<std::size_t>(count));
55 for (std::int32_t i = 0; i < count; ++i) {
56 std::int32_t raw = 0;
57 if (!(input >> raw)) {
58 throw std::runtime_error("invalid int8 payload for key '" + key + "'");
59 }
60 if (raw < LCQI_INT8_MIN_VALUE || raw > LCQI_INT8_MAX_VALUE) {
61 throw std::runtime_error("int8 payload out of range for key '" + key + "'");
62 }
63 values[static_cast<std::size_t>(i)] = static_cast<std::int8_t>(raw);
64 }
65 return values;
66 }
67
68 std::vector<float> read_float_vector(std::istream& input,
69 const std::string& key,
70 std::int32_t count) {
71 expect_key(input, key);
72 if (count < 0) {
73 throw std::runtime_error("negative vector size for key '" + key + "'");
74 }
75 std::vector<float> values(static_cast<std::size_t>(count));
76 for (std::int32_t i = 0; i < count; ++i) {
77 if (!(input >> values[static_cast<std::size_t>(i)])) {
78 throw std::runtime_error("invalid float payload for key '" + key + "'");
79 }
80 }
81 return values;
82 }
83
84 void validate_layer(const QuantizedLinearLayer& layer, const std::string& name) {
85 if (layer.input_size <= 0 || layer.output_size <= 0) {

```

```

86 throw std::runtime_error(name + " layer has invalid shape");
87 }
88 const std::size_t expected_weights =
89 static_cast<std::size_t>(layer.input_size) *
90 static_cast<std::size_t>(layer.output_size);
91 if (layer.weights.size() != expected_weights) {
92 throw std::runtime_error(name + " layer weight size mismatch");
93 }
94 if (layer.bias.size() != static_cast<std::size_t>(layer.output_size)) {
95 throw std::runtime_error(name + " layer bias size mismatch");
96 }
97 }
98
99 std::int32_t checked_element_count(std::int32_t rows,
100 std::int32_t columns,
101 const std::string& name) {
102 if (rows <= 0 || columns <= 0) {
103 throw std::runtime_error(name + " layer has invalid shape");
104 }
105 const std::int64_t count =
106 static_cast<std::int64_t>(rows) * static_cast<std::int64_t>(columns);
107 if (count > static_cast<std::int64_t>(std::numeric_limits<std::int32_t>::max()
↵
108 throw std::runtime_error(name + " layer element count is too large");
109 }
110 return static_cast<std::int32_t>(count);
111 }
112
113 } // namespace
114
115 TinyMlpModel load_model(const std::filesystem::path& path) {
116 std::ifstream input(path);
117 if (!input) {
118 throw std::runtime_error("cannot open model file: " + path.string());
119 }
120
121 expect_key(input, "LCQI_MODEL_V1");
122
123 TinyMlpModel model;
124 model.input_size = read_i32(input, "input_size");
125 model.hidden_size = read_i32(input, "hidden_size");
126 model.output_size = read_i32(input, "output_size");
127
128 const std::int32_t hidden_weight_count =
129 checked_element_count(model.input_size, model.hidden_size, "hidden");
130 const std::int32_t output_weight_count =
131 checked_element_count(model.hidden_size, model.output_size, "output");
132
133 model.hidden.input_size = model.input_size;
134 model.hidden.output_size = model.hidden_size;
135 model.hidden.scale = read_float(input, "hidden_scale");
136 model.hidden.weights = read_i8_vector(

```



```

137 input,
138 "hidden_weights",
139 hidden_weight_count);
140 model.hidden.bias = read_float_vector(input, "hidden_bias", model.↵
↵ hidden_size);
141
142 model.output.input_size = model.hidden_size;
143 model.output.output_size = model.output_size;
144 model.output.scale = read_float(input, "output_scale");
145 model.output.weights = read_i8_vector(
146 input,
147 "output_weights",
148 output_weight_count);
149 model.output.bias = read_float_vector(input, "output_bias", model.↵
↵ output_size);
150
151 validate_layer(model.hidden, "hidden");
152 validate_layer(model.output, "output");
153 return model;
154 }
155
156 } // namespace lcqi

```

## B.10 十、Tiny MLP 推理实现

推理实现执行 int8 linear、ReLU、第二层 logits 和 predicted class。它是最短闭环，不是完整 LLM；价值在于把模型加载和 kernel 调用接起来。

Listing B.10 src/inference.cpp

```

1 #include <lcqi/inference.hpp>
2
3 #include <algorithm>
4 #include <cmath>
5 #include <stdexcept>
6
7 namespace lcqi {
8 namespace {
9
10 void validate_input_size(std::span<const float> input, std::int32_t expected) {
11 if (input.size() != static_cast<std::size_t>(expected)) {
12 throw std::runtime_error("input size does not match model shape");
13 }
14 }
15
16 std::int32_t argmax(std::span<const float> values) {
17 if (values.empty()) {
18 throw std::runtime_error("cannot take argmax of empty logits");
19 }
20 std::int32_t best_index = 0;
21 float best_value = values[0];
22 for (std::int32_t i = 1; i < static_cast<std::int32_t>(values.size()); ++i)↵

```

```

 ↪ {
23 const float candidate = values[static_cast<std::size_t>(i)];
24 if (candidate > best_value) {
25 best_value = candidate;
26 best_index = i;
27 }
28 }
29 return best_index;
30 }
31
32 } // namespace
33
34 void linear_i8(const QuantizedLinearLayer& layer,
35 std::span<const float> input,
36 std::span<float> output) {
37 if (input.size() != static_cast<std::size_t>(layer.input_size)) {
38 throw std::runtime_error("linear_i8 input size mismatch");
39 }
40 if (output.size() != static_cast<std::size_t>(layer.output_size)) {
41 throw std::runtime_error("linear_i8 output size mismatch");
42 }
43
44 for (std::int32_t out = 0; out < layer.output_size; ++out) {
45 const std::size_t row_base =
46 static_cast<std::size_t>(out) * static_cast<std::size_t>(layer.↪
 ↪ input_size);
47 float acc = layer.bias[static_cast<std::size_t>(out)];
48 for (std::int32_t in = 0; in < layer.input_size; ++in) {
49 const std::int8_t weight =
50 layer.weights[row_base + static_cast<std::size_t>(in)];
51 acc += static_cast<float>(weight) * layer.scale *
52 input[static_cast<std::size_t>(in)];
53 }
54 output[static_cast<std::size_t>(out)] = acc;
55 }
56 }
57
58 void relu_inplace(std::span<float> values) {
59 for (float& value : values) {
60 value = std::max(0.0F, value);
61 }
62 }
63
64 InferenceResult run_inference(const TinyMlpModel& model,
65 std::span<const float> input) {
66 validate_input_size(input, model.input_size);
67
68 std::vector<float> hidden(static_cast<std::size_t>(model.hidden_size), 0.0F↪
 ↪);
69 std::vector<float> logits(static_cast<std::size_t>(model.output_size), 0.0F↪
 ↪);
70

```

```

71 linear_i8(model.hidden, input, hidden);
72 relu_inplace(hidden);
73 linear_i8(model.output, hidden, logits);
74
75 InferenceResult result;
76 result.predicted_class = argmax(logits);
77 result.logits = std::move(logits);
78 return result;
79 }
80
81 } // namespace lcqi

```

## B.11 十一、命令行入口

`src/main.cpp` 把模型路径和输入接到 `run_inference`, 并打印结果。CLI 应保持薄层, 不复制模型解析和推理逻辑。

Listing B.11 `src/main.cpp`

```

1 #include <lcqi/inference.hpp>
2 #include <lcqi/model.hpp>
3
4 #include <chrono>
5 #include <cstdint>
6 #include <cstdlib>
7 #include <filesystem>
8 #include <iostream>
9 #include
10 #include <stdexcept>
11 #include <string>
12 #include <vector>
13
14 namespace {
15
16 constexpr std::int32_t LCQI_DEFAULT_REPEAT = 1000;
17 constexpr int LCQI_EXPECTED_ARGC_MIN = 2;
18 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
19
20 std::vector<float> parse_input(int argc, char** argv, std::int32_t ↵
 ↵ expected_size) {
21 if (argc < LCQI_EXPECTED_ARGC_MIN + expected_size) {
22 throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats...>↵
 ↵ [repeat]");
23 }
24 std::vector<float> input(static_cast<std::size_t>(expected_size));
25 for (std::int32_t i = 0; i < expected_size; ++i) {
26 input[static_cast<std::size_t>(i)] =
27 std::stof(argv[LCQI_EXPECTED_ARGC_MIN + i]);
28 }
29 return input;
30 }
31

```

```

32 std::int32_t parse_repeat(int argc, char** argv, std::int32_t input_size) {
33 const int repeat_index = LCQI_EXPECTED_ARGC_MIN + input_size;
34 if (argc <= repeat_index) {
35 return LCQI_DEFAULT_REPEAT;
36 }
37 const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
↵ repeat_index]));
38 if (repeat <= 0) {
39 throw std::runtime_error("repeat must be positive");
40 }
41 return repeat;
42 }
43
44 } // namespace
45
46 int main(int argc, char** argv) {
47 try {
48 if (argc < LCQI_EXPECTED_ARGC_MIN) {
49 throw std::runtime_error("usage: lcqi_cli <model.txt> <input floats↵
↵ ...> [repeat]");
50 }
51
52 const std::filesystem::path model_path = argv[1];
53 const lcqi::TinyMlpModel model = lcqi::load_model(model_path);
54 const std::vector<float> input = parse_input(argc, argv, model.↵
↵ input_size);
55 const std::int32_t repeat = parse_repeat(argc, argv, model.input_size);
56
57 const lcqi::InferenceResult result = lcqi::run_inference(model, input);
58 std::cout << "predicted_class " << result.predicted_class << "\n";
59 std::cout << "logits";
60 for (const float value : result.logits) {
61 std::cout << ' ' << value;
62 }
63 std::cout << "\n";
64
65 const auto begin = std::chrono::steady_clock::now();
66 std::int32_t checksum = 0;
67 for (std::int32_t i = 0; i < repeat; ++i) {
68 checksum += lcqi::run_inference(model, input).predicted_class;
69 }
70 const auto end = std::chrono::steady_clock::now();
71 const std::chrono::duration<double> elapsed = end - begin;
72 const double average_us =
73 elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double↵
↵ >(repeat);
74 std::cout << "repeat " << repeat << "\n";
75 std::cout << "average_us " << average_us << "\n";
76 std::cout << "checksum " << checksum << "\n";
77 return EXIT_SUCCESS;
78 } catch (const std::exception& error) {
79 std::cerr << "error: " << error.what() << "\n";

```

```

80 return EXIT_FAILURE;
81 }
82 }

```

## B.12 十二、Reference Decoder 完整实现

这是第三册最重要的长文件。它实现 embedding、RMSNorm、Q/K/V、RoPE、KV cache、attention、SwiGLU、final norm、logits 和 sampler，并在关键位置生成 trace checkpoint。读者应按 checkpoint 顺序读，而不是从最终 logits 倒推。

Listing B.12 src/reference\_decoder.cpp

```

1 #include <lcqi/reference_decoder.hpp>
2
3 #include <algorithm>
4 #include <array>
5 #include <cmath>
6 #include <limits>
7 #include <stdexcept>
8 #include <string>
9
10 namespace lcqi {
11 namespace {
12
13 constexpr float LCQI_SOFTMAX_NEGATIVE_INFINITY = -std::numeric_limits<float>::infinity();
14 constexpr std::int32_t LCQI_ROPE_PAIR_WIDTH = 2;
15 constexpr const char* LCQI_TRACE_DTYPE_F32 = "f32";
16 constexpr const char* LCQI_TRACE_LAYOUT_CONTIGUOUS = "contiguous";
17 constexpr const char* LCQI_TRACE_LAYOUT_ROW_MAJOR = "row_major";
18 constexpr const char* LCQI_TRACE_LAYOUT_KV_CONTIGUOUS = "kv_contiguous";
19
20 void require_positive(std::int32_t value, const char* name) {
21 if (value <= 0) {
22 throw std::runtime_error(std::string(name) + " must be positive");
23 }
24 }
25
26 void validate_config(const DecoderConfig& config) {
27 require_positive(config.hidden_size, "hidden_size");
28 require_positive(config.query_heads, "query_heads");
29 require_positive(config.kv_heads, "kv_heads");
30 require_positive(config.head_dim, "head_dim");
31 require_positive(config.intermediate_size, "intermediate_size");
32 require_positive(config.vocab_size, "vocab_size");
33 require_positive(config.max_context, "max_context");
34 if (config.query_heads % config.kv_heads != 0) {
35 throw std::runtime_error("query_heads must be divisible by kv_heads");
36 }
37 if (config.hidden_size != config.query_heads * config.head_dim) {
38 throw std::runtime_error("hidden_size must equal query_heads * head_dim");
39 }

```

```

39 }
40 if (config.head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
41 throw std::runtime_error("head_dim must be even for RoPE");
42 }
43 if (config.rope_base <= 1.0F) {
44 throw std::runtime_error("rope_base must be greater than 1");
45 }
46 }
47
48 std::size_t checked_size(std::int32_t value, const char* name) {
49 if (value < 0) {
50 throw std::runtime_error(std::string(name) + " cannot be negative");
51 }
52 return static_cast<std::size_t>(value);
53 }
54
55 std::int32_t kv_width(const DecoderConfig& config) {
56 return config.kv_heads * config.head_dim;
57 }
58
59 void validate_linear(const LinearWeightsF32& layer, const char* name) {
60 require_positive(layer.input_size, "linear input_size");
61 require_positive(layer.output_size, "linear output_size");
62 const std::size_t expected_weights =
63 checked_size(layer.input_size, name) * checked_size(layer.output_size,
64 ↪ name);
65 if (layer.weights.size() != expected_weights) {
66 throw std::runtime_error(std::string(name) + " weight size mismatch");
67 }
68 if (!layer.bias.empty() &&
69 layer.bias.size() != checked_size(layer.output_size, name)) {
70 throw std::runtime_error(std::string(name) + " bias size mismatch");
71 }
72 }
73
74 void validate_span_size(std::size_t actual, std::int32_t expected, const char*
75 ↪ name) {
76 if (actual != checked_size(expected, name)) {
77 throw std::runtime_error(std::string(name) + " size mismatch");
78 }
79 }
80
81 std::int32_t argmax(std::span<const float> values) {
82 if (values.empty()) {
83 throw std::runtime_error("cannot take argmax of empty values");
84 }
85 std::int32_t best_index = 0;
86 float best_value = values[0];
87 for (std::int32_t index = 1; index < static_cast<std::int32_t>(values.size()
88 ↪)); ++index) {
89 const float candidate = values[static_cast<std::size_t>(index)];
90 if (candidate > best_value) {

```



```

88 best_value = candidate;
89 best_index = index;
90 }
91 }
92 return best_index;
93 }
94
95 float silu(float value) {
96 return value / (1.0F + std::exp(-value));
97 }
98
99 std::span<const float> row_span(std::span<const float> values,
100 std::int32_t row,
101 std::int32_t row_width) {
102 const std::size_t begin = checked_size(row, "row") * checked_size(row_width,
103 ↪ , "row_width");
104 return values.subspan(begin, checked_size(row_width, "row_width"));
105 }
106
107 void add_inplace(std::span<float> target, std::span<const float> source) {
108 if (target.size() != source.size()) {
109 throw std::runtime_error("residual add size mismatch");
110 }
111 for (std::size_t index = 0; index < target.size(); ++index) {
112 target[index] += source[index];
113 }
114 }
115
116 void swiglu_mlp(const DecoderLayerWeightsF32& layer,
117 std::span<const float> input,
118 std::span<float> output) {
119 std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0F
120 ↪);
121 std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
122 std::vector<float> hidden(gate.size(), 0.0F);
123 linear_f32(layer.w_gate, input, gate);
124 linear_f32(layer.w_up, input, up);
125 if (gate.size() != up.size()) {
126 throw std::runtime_error("SwiGLU gate/up size mismatch");
127 }
128 for (std::size_t index = 0; index < hidden.size(); ++index) {
129 hidden[index] = silu(gate[index]) * up[index];
130 }
131 linear_f32(layer.w_down, hidden, output);
132 }
133
134 float checksum(std::span<const float> values) {
135 float sum = 0.0F;
136 for (const float value : values) {
137 sum += value;
138 }
139 return sum;

```

```

138 }
139
140 float max_abs(std::span<const float> values) {
141 float result = 0.0F;
142 for (const float value : values) {
143 result = std::max(result, std::fabs(value));
144 }
145 return result;
146 }
147
148 std::vector<std::int32_t> contiguous_stride(std::span<const std::int32_t> shape ←
 ↪) {
149 std::vector<std::int32_t> stride(shape.size(), 1);
150 std::int32_t running = 1;
151 for (std::int32_t index = static_cast<std::int32_t>(shape.size()) - 1; ←
 ↪ index >= 0; --index) {
152 stride[checked_size(index, "stride index")] = running;
153 running *= shape[checked_size(index, "shape index")];
154 }
155 return stride;
156 }
157
158 void add_checkpoint(std::vector<ReferenceTensorCheckpoint>* checkpoints,
159 const char* name,
160 std::int32_t token_position,
161 std::int32_t layer_id,
162 std::span<const std::int32_t> shape,
163 const char* layout,
164 std::span<const float> values) {
165 if (checkpoints == nullptr) {
166 return;
167 }
168 ReferenceTensorCheckpoint checkpoint;
169 checkpoint.name = name;
170 checkpoint.token_position = token_position;
171 checkpoint.layer_id = layer_id;
172 checkpoint.shape.assign(shape.begin(), shape.end());
173 checkpoint.stride = contiguous_stride(shape);
174 checkpoint.dtype = LCQI_TRACE_DTYPE_F32;
175 checkpoint.layout = layout;
176 checkpoint.checksum = checksum(values);
177 checkpoint.max_abs = max_abs(values);
178 checkpoint.values.assign(values.begin(), values.end());
179 checkpoints->push_back(std::move(checkpoint));
180 }
181
182 void swiglu_mlp_with_trace(const DecoderLayerWeightsF32& layer,
183 std::span<const float> input,
184 std::span<float> output,
185 std::int32_t token_position,
186 std::int32_t layer_id,
187 std::vector<ReferenceTensorCheckpoint>* checkpoints) ←

```

```

188 ↪ {
189 std::vector<float> gate(checked_size(layer.w_gate.output_size, "gate"), 0.0F);
190 ↪ F);
191 std::vector<float> up(checked_size(layer.w_up.output_size, "up"), 0.0F);
192 std::vector<float> hidden(gate.size(), 0.0F);
193 linear_f32(layer.w_gate, input, gate);
194 linear_f32(layer.w_up, input, up);
195 if (gate.size() != up.size()) {
196 throw std::runtime_error("SviGLU gate/up size mismatch");
197 }
198 const std::array<std::int32_t, 1> intermediate_shape{layer.w_gate.↪
199 ↪ output_size};
200 add_checkpoint(checkpoints,
201 "mlp_gate",
202 token_position,
203 layer_id,
204 intermediate_shape,
205 LCQI_TRACE_LAYOUT_CONTIGUOUS,
206 gate);
207 add_checkpoint(checkpoints,
208 "mlp_up",
209 token_position,
210 layer_id,
211 intermediate_shape,
212 LCQI_TRACE_LAYOUT_CONTIGUOUS,
213 up);
214 for (std::size_t index = 0; index < hidden.size(); ++index) {
215 hidden[index] = silu(gate[index]) * up[index];
216 }
217 add_checkpoint(checkpoints,
218 "mlp_hidden",
219 token_position,
220 layer_id,
221 intermediate_shape,
222 LCQI_TRACE_LAYOUT_CONTIGUOUS,
223 hidden);
224 linear_f32(layer.w_down, hidden, output);
225 }
226
227 void forward_layer(const DecoderConfig& config,
228 const DecoderLayerWeightsF32& layer,
229 std::int32_t layer_id,
230 std::int32_t model_position,
231 ReferenceKVCache& cache,
232 std::span<float> hidden,
233 std::vector<ReferenceTensorCheckpoint*> checkpoints = ↪
234 ↪ nullptr) {
235 validate_span_size(hidden.size(), config.hidden_size, "hidden");
236
237 std::vector<float> normed(checked_size(config.hidden_size, "hidden_size"), ↪
238 ↪ 0.0F);
239 std::vector<float> query(checked_size(config.hidden_size, "hidden_size"), ↪

```

```

↪ 0.0F);
235 std::vector<float> key(checkered_size(kv_width(config), "kv_width"), 0.0F);
236 std::vector<float> value(checkered_size(kv_width(config), "kv_width"), 0.0F);
237 std::vector<float> attention(checkered_size(config.hidden_size, "hidden_size"↪
↪), 0.0F);
238 std::vector<float> attention_projected(checkered_size(config.hidden_size, "↪
↪ hidden_size"), 0.0F);
239 std::vector<float> mlp(checkered_size(config.hidden_size, "hidden_size"), 0.0↪
↪ F);
240
241 const std::array<std::int32_t, 1> hidden_shape{config.hidden_size};
242 const std::array<std::int32_t, 2> query_shape{config.query_heads, config.↪
↪ head_dim};
243 const std::array<std::int32_t, 2> kv_shape{config.kv_heads, config.head_dim↪
↪ };
244 const std::array<std::int32_t, 4> kv_slot_shape{
245 1, 1, config.kv_heads, config.head_dim};
246
247 rms_norm(hidden, layer.rms_attention_weight, config.rms_epsilon, normed);
248 add_checkpoint(checkpoints,
249 "attn_norm",
250 model_position,
251 layer_id,
252 hidden_shape,
253 LCQI_TRACE_LAYOUT_CONTIGUOUS,
254 normed);
255 linear_f32(layer.wq, normed, query);
256 linear_f32(layer.wk, normed, key);
257 linear_f32(layer.wv, normed, value);
258 add_checkpoint(checkpoints,
259 "q_before_rope",
260 model_position,
261 layer_id,
262 query_shape,
263 LCQI_TRACE_LAYOUT_CONTIGUOUS,
264 query);
265 add_checkpoint(checkpoints,
266 "k_before_rope",
267 model_position,
268 layer_id,
269 kv_shape,
270 LCQI_TRACE_LAYOUT_CONTIGUOUS,
271 key);
272 add_checkpoint(checkpoints,
273 "v",
274 model_position,
275 layer_id,
276 kv_shape,
277 LCQI_TRACE_LAYOUT_CONTIGUOUS,
278 value);
279 apply_rope(query, config.query_heads, config.head_dim, model_position, ↪
↪ config.rope_base);

```

```

280 apply_rope(key, config.kv_heads, config.head_dim, model_position, config.↵
↵ rope_base);
281 add_checkpoint(checkpoints,
282 "q_after_rope",
283 model_position,
284 layer_id,
285 query_shape,
286 LCQI_TRACE_LAYOUT_CONTIGUOUS,
287 query);
288 add_checkpoint(checkpoints,
289 "k_after_rope",
290 model_position,
291 layer_id,
292 kv_shape,
293 LCQI_TRACE_LAYOUT_CONTIGUOUS,
294 key);
295
296 cache.append(layer_id, model_position, key, value);
297 add_checkpoint(checkpoints,
298 "kv_cache_key_slot",
299 model_position,
300 layer_id,
301 kv_slot_shape,
302 LCQI_TRACE_LAYOUT_KV_CONTIGUOUS,
303 key);
304 attention_decode(config, cache, layer_id, model_position, query, attention)↵
↵ ;
305 add_checkpoint(checkpoints,
306 "attention_out",
307 model_position,
308 layer_id,
309 hidden_shape,
310 LCQI_TRACE_LAYOUT_CONTIGUOUS,
311 attention);
312 linear_f32(layer.wo, attention, attention_projected);
313 add_inplace(hidden, attention_projected);
314 add_checkpoint(checkpoints,
315 "post_attention_residual",
316 model_position,
317 layer_id,
318 hidden_shape,
319 LCQI_TRACE_LAYOUT_CONTIGUOUS,
320 hidden);
321
322 rms_norm(hidden, layer.rms_mlp_weight, config.rms_epsilon, normed);
323 add_checkpoint(checkpoints,
324 "mlp_norm",
325 model_position,
326 layer_id,
327 hidden_shape,
328 LCQI_TRACE_LAYOUT_CONTIGUOUS,
329 normed);

```

```

330 if (checkpoints == nullptr) {
331 swiglu_mlp(layer, normed, mlp);
332 } else {
333 swiglu_mlp_with_trace(layer, normed, mlp, model_position, layer_id, ↵
↵ checkpoints);
334 }
335 add_inplace(hidden, mlp);
336 add_checkpoint(checkpoints,
337 "layer_out",
338 model_position,
339 layer_id,
340 hidden_shape,
341 LCQI_TRACE_LAYOUT_CONTIGUOUS,
342 hidden);
343 }
344
345 LinearWeightsF32 make_linear(std::int32_t input_size,
346 std::int32_t output_size,
347 std::vector<float> weights,
348 std::vector<float> bias = {}) {
349 LinearWeightsF32 layer;
350 layer.input_size = input_size;
351 layer.output_size = output_size;
352 layer.weights = std::move(weights);
353 layer.bias = std::move(bias);
354 validate_linear(layer, "tiny linear");
355 return layer;
356 }
357
358 std::vector<float> zeros(std::int32_t count) {
359 return std::vector<float>(checked_size(count, "zero count"), 0.0F);
360 }
361
362 } // namespace
363
364 ReferenceDecodeTraceResult run_reference_decode_with_trace(
365 const ReferenceDecoderModel& model,
366 std::span<const std::int32_t> token_ids) {
367 validate_config(model.config);
368 if (token_ids.empty()) {
369 throw std::runtime_error("reference decode needs at least one token");
370 }
371 if (token_ids.size() > checked_size(model.config.max_context, "max_context" ↵
↵)) {
372 throw std::runtime_error("token_ids exceed max_context");
373 }
374 if (model.token_embedding.size() !=
375 checked_size(model.config.vocab_size, "vocab_size") *
376 checked_size(model.config.hidden_size, "hidden_size")) {
377 throw std::runtime_error("token embedding size mismatch");
378 }
379 if (model.final_norm_weight.size() != checked_size(model.config.hidden_size, ↵

```



```

↪ , "hidden_size")) {
380 throw std::runtime_error("final norm weight size mismatch");
381 }
382 validate_linear(model.lm_head, "lm_head");
383
384 ReferenceDecodeTraceResult trace;
385 ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers↪
↪ .size()));
386 std::vector<float> hidden(check_size(model.config.hidden_size, "↪
↪ hidden_size"), 0.0F);
387 const std::array<std::int32_t, 1> hidden_shape{model.config.hidden_size};
388 for (std::int32_t position = 0; position < static_cast<std::int32_t>(↪
↪ token_ids.size());
389 ++position) {
390 const std::int32_t token_id = token_ids[checked_size(position, "↪
↪ position)];
391 if (token_id < 0 || token_id >= model.config.vocab_size) {
392 throw std::runtime_error("token id out of vocabulary");
393 }
394 const std::span<const float> embedding =
395 row_span(model.token_embedding, token_id, model.config.hidden_size)↪
↪ ;
396 std::copy(embedding.begin(), embedding.end(), hidden.begin());
397 add_checkpoint(&trace.checkpoints,
398 "embedding",
399 position,
400 -1,
401 hidden_shape,
402 LCQI_TRACE_LAYOUT_ROW_MAJOR,
403 hidden);
404 for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(↪
↪ model.layers.size());
405 ++layer_id) {
406 forward_layer(model.config,
407 model.layers[checked_size(layer_id, "layer_id")],
408 layer_id,
409 position,
410 cache,
411 hidden,
412 &trace.checkpoints);
413 }
414 }
415
416 std::vector<float> normed(check_size(model.config.hidden_size, "↪
↪ hidden_size"), 0.0F);
417 std::vector<float> logits(check_size(model.config.vocab_size, "vocab_size↪
↪ "), 0.0F);
418 const std::array<std::int32_t, 1> logits_shape{model.config.vocab_size};
419 const std::int32_t last_position = static_cast<std::int32_t>(token_ids.size↪
↪ ()) - 1;
420 rms_norm(hidden, model.final_norm_weight, model.config.rms_epsilon, normed)↪
↪ ;

```

```

421 add_checkpoint(&trace.checkpoints,
422 "final_norm",
423 last_position,
424 -1,
425 hidden_shape,
426 LCQI_TRACE_LAYOUT_CONTIGUOUS,
427 normed);
428 linear_f32(model.lm_head, normed, logits);
429 add_checkpoint(&trace.checkpoints,
430 "logits",
431 last_position,
432 -1,
433 logits_shape,
434 LCQI_TRACE_LAYOUT_CONTIGUOUS,
435 logits);
436
437 trace.result.predicted_token = argmax(logits);
438 trace.result.logits = std::move(logits);
439 return trace;
440 }
441
442 ReferenceKVCache::ReferenceKVCache(const DecoderConfig& config, std::int32_t ←
 ↪ layer_count)
443 : config_(config),
444 layer_count_(layer_count) {
445 validate_config(this->config_);
446 require_positive(this->layer_count_, "layer_count");
447 const std::size_t element_count =
448 checked_size(this->layer_count_, "layer_count") *
449 checked_size(this->config_.max_context, "max_context") *
450 checked_size(this->config_.kv_heads, "kv_heads") *
451 checked_size(this->config_.head_dim, "head_dim");
452 this->keys_.assign(element_count, 0.0F);
453 this->values_.assign(element_count, 0.0F);
454 this->written_.assign(
455 checked_size(this->layer_count_, "layer_count") *
456 checked_size(this->config_.max_context, "max_context"),
457 0);
458 }
459
460 void ReferenceKVCache::append(std::int32_t layer_id,
461 std::int32_t model_position,
462 std::span<const float> key,
463 std::span<const float> value) {
464 validate_span_size(key.size(), kv_width(this->config_), "KV key");
465 validate_span_size(value.size(), kv_width(this->config_), "KV value");
466 this->validate_address(layer_id, model_position, 0);
467 for (std::int32_t kv_head = 0; kv_head < this->config_.kv_heads; ++kv_head) ←
 ↪ {
468 const std::size_t target = this->base_offset(layer_id, model_position, ←
 ↪ kv_head);
469 const std::size_t source =

```

```

470 checked_size(kv_head, "kv_head") * checked_size(this->config_.head_dim, "head_dim");
471 std::copy_n(key.begin() + static_cast<std::ptrdiff_t>(source),
472 this->config_.head_dim,
473 this->keys_.begin() + static_cast<std::ptrdiff_t>(target));
474 std::copy_n(value.begin() + static_cast<std::ptrdiff_t>(source),
475 this->config_.head_dim,
476 this->values_.begin() + static_cast<std::ptrdiff_t>(target));
477 };
478 }
479 const std::size_t written_index =
480 checked_size(layer_id, "layer_id") * checked_size(this->config_.max_context, "max_context") +
481 checked_size(model_position, "model_position");
482 this->written_[written_index] = 1;
483 this->filled_tokens_ = std::max(this->filled_tokens_, model_position + 1);
484 }
485 std::span<const float> ReferenceKVCache::key(std::int32_t layer_id,
486 std::int32_t model_position,
487 std::int32_t kv_head) const {
488 this->validate_address(layer_id, model_position, kv_head);
489 const std::size_t offset = this->base_offset(layer_id, model_position, kv_head);
490 return std::span<const float>(this->keys_.data() + offset,
491 checked_size(this->config_.head_dim, "head_dim"));
492 }
493
494 std::span<const float> ReferenceKVCache::value(std::int32_t layer_id,
495 std::int32_t model_position,
496 std::int32_t kv_head) const {
497 this->validate_address(layer_id, model_position, kv_head);
498 const std::size_t offset = this->base_offset(layer_id, model_position, kv_head);
499 return std::span<const float>(this->values_.data() + offset,
500 checked_size(this->config_.head_dim, "head_dim"));
501 }
502
503 std::int32_t ReferenceKVCache::layer_count() const noexcept {
504 return this->layer_count_;
505 }
506
507 std::int32_t ReferenceKVCache::filled_tokens() const noexcept {
508 return this->filled_tokens_;
509 }
510
511 std::size_t ReferenceKVCache::base_offset(std::int32_t layer_id,
512 std::int32_t model_position,
513 std::int32_t kv_head) const {
514 return (((checked_size(layer_id, "layer_id") *

```

```

515 checked_size(this->config_.max_context, "max_context")) +
516 checked_size(model_position, "model_position")) *
517 checked_size(this->config_.kv_heads, "kv_heads") +
518 checked_size(kv_head, "kv_head")) *
519 checked_size(this->config_.head_dim, "head_dim");
520 }
521
522 void ReferenceKVCache::validate_address(std::int32_t layer_id,
523 std::int32_t model_position,
524 std::int32_t kv_head) const {
525 if (layer_id < 0 || layer_id >= this->layer_count_) {
526 throw std::runtime_error("KV layer_id out of range");
527 }
528 if (model_position < 0 || model_position >= this->config_.max_context) {
529 throw std::runtime_error("KV model_position out of range");
530 }
531 if (kv_head < 0 || kv_head >= this->config_.kv_heads) {
532 throw std::runtime_error("KV head out of range");
533 }
534 }
535
536 void rms_norm(std::span<const float> input,
537 std::span<const float> weight,
538 float epsilon,
539 std::span<float> output) {
540 if (input.size() != weight.size() || input.size() != output.size()) {
541 throw std::runtime_error("RMSNorm size mismatch");
542 }
543 if (input.empty()) {
544 throw std::runtime_error("RMSNorm input is empty");
545 }
546 float sum_square = 0.0F;
547 for (const float value : input) {
548 sum_square += value * value;
549 }
550 const float mean_square = sum_square / static_cast<float>(input.size());
551 const float scale = 1.0F / std::sqrt(mean_square + epsilon);
552 for (std::size_t index = 0; index < input.size(); ++index) {
553 output[index] = input[index] * scale * weight[index];
554 }
555 }
556
557 void linear_f32(const LinearWeightsF32& layer,
558 std::span<const float> input,
559 std::span<float> output) {
560 validate_linear(layer, "linear_f32");
561 validate_span_size(input.size(), layer.input_size, "linear input");
562 validate_span_size(output.size(), layer.output_size, "linear output");
563 for (std::int32_t out = 0; out < layer.output_size; ++out) {
564 const std::size_t row_base =
565 checked_size(out, "out") * checked_size(layer.input_size, "↵
↵ input_size");

```

```

566 float accumulator = layer.bias.empty() ? 0.0F : layer.bias[checked_size←
 ↪ (out, "out")];
567 for (std::int32_t in = 0; in < layer.input_size; ++in) {
568 accumulator += layer.weights[row_base + checked_size(in, "in")] *
569 input[checked_size(in, "in")];
570 }
571 output[checked_size(out, "out")] = accumulator;
572 }
573 }
574
575 void apply_rope(std::span<float> heads,
576 std::int32_t head_count,
577 std::int32_t head_dim,
578 std::int32_t model_position,
579 float rope_base) {
580 require_positive(head_count, "head_count");
581 require_positive(head_dim, "head_dim");
582 validate_span_size(heads.size(), head_count * head_dim, "RoPE heads");
583 if (head_dim % LCQI_ROPE_PAIR_WIDTH != 0) {
584 throw std::runtime_error("RoPE head_dim must be even");
585 }
586 if (model_position < 0) {
587 throw std::runtime_error("RoPE model_position cannot be negative");
588 }
589 if (rope_base <= 1.0F) {
590 throw std::runtime_error("RoPE base must be greater than 1");
591 }
592 for (std::int32_t head = 0; head < head_count; ++head) {
593 for (std::int32_t offset = 0; offset < head_dim; offset += ←
 ↪ LCQI_ROPE_PAIR_WIDTH) {
594 const float exponent = static_cast<float>(offset) / static_cast<←
 ↪ float>(head_dim);
595 const float theta = 1.0F / std::pow(rope_base, exponent);
596 const float angle = static_cast<float>(model_position) * theta;
597 const float cosine = std::cos(angle);
598 const float sine = std::sin(angle);
599 const std::size_t first =
600 checked_size(head, "head") * checked_size(head_dim, "head_dim")←
 ↪ +
601 checked_size(offset, "offset");
602 const float x0 = heads[first];
603 const float x1 = heads[first + 1];
604 heads[first] = x0 * cosine - x1 * sine;
605 heads[first + 1] = x0 * sine + x1 * cosine;
606 }
607 }
608 }
609
610 void attention_decode(const DecoderConfig& config,
611 const ReferenceKVCache& cache,
612 std::int32_t layer_id,
613 std::int32_t model_position,

```

```

614 std::span<const float> query,
615 std::span<float> output) {
616 validate_config(config);
617 validate_span_size(query.size(), config.hidden_size, "attention query");
618 validate_span_size(output.size(), config.hidden_size, "attention output");
619 if (model_position < 0 || model_position >= cache.filled_tokens()) {
620 throw std::runtime_error("attention model_position has not been written↵
↵ ");
621 }
622 std::fill(output.begin(), output.end(), 0.0F);
623
624 const std::int32_t group_size = config.query_heads / config.kv_heads;
625 const float score_scale = 1.0F / std::sqrt(static_cast<float>(config.↵
↵ head_dim));
626 std::vector<float> scores(check_size(model_position + 1, "score count"),
627 LCQI_SOFTMAX_NEGATIVE_INFINITY);
628 std::vector<float> probabilities(scores.size(), 0.0F);
629
630 for (std::int32_t query_head = 0; query_head < config.query_heads; ++↵
↵ query_head) {
631 const std::int32_t kv_head = query_head / group_size;
632 const std::size_t query_base =
633 checked_size(query_head, "query_head") * checked_size(config.↵
↵ head_dim, "head_dim");
634 float max_score = LCQI_SOFTMAX_NEGATIVE_INFINITY;
635 for (std::int32_t position = 0; position <= model_position; ++position)↵
↵ {
636 const std::span<const float> key = cache.key(layer_id, position, ↵
↵ kv_head);
637 float dot = 0.0F;
638 for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
639 dot += query[query_base + checked_size(dim, "dim")] *
640 key[checked_size(dim, "dim")];
641 }
642 const float score = dot * score_scale;
643 scores[checked_size(position, "position")] = score;
644 max_score = std::max(max_score, score);
645 }
646
647 float probability_sum = 0.0F;
648 for (std::int32_t position = 0; position <= model_position; ++position)↵
↵ {
649 const std::size_t index = checked_size(position, "position");
650 const float unnormalized = std::exp(scores[index] - max_score);
651 probabilities[index] = unnormalized;
652 probability_sum += unnormalized;
653 }
654 if (probability_sum <= 0.0F) {
655 throw std::runtime_error("attention probability sum is invalid");
656 }
657 for (std::int32_t position = 0; position <= model_position; ++position)↵
↵ {

```

```

658 const float probability =
659 probabilities[checked_size(position, "position")] / ←
 ↪ probability_sum;
660 const std::span<const float> value = cache.value(layer_id, position↪
 ↪ , kv_head);
661 for (std::int32_t dim = 0; dim < config.head_dim; ++dim) {
662 output[query_base + checked_size(dim, "dim")] +=
663 probability * value[checked_size(dim, "dim")];
664 }
665 }
666 }
667 }
668
669 ReferenceDecodeResult run_reference_decode(const ReferenceDecoderModel& model,
670 std::span<const std::int32_t> ←
 ↪ token_ids) {
671 validate_config(model.config);
672 if (token_ids.empty()) {
673 throw std::runtime_error("reference decode needs at least one token");
674 }
675 if (token_ids.size() > checked_size(model.config.max_context, "max_context"↪
 ↪)) {
676 throw std::runtime_error("token_ids exceed max_context");
677 }
678 if (model.token_embedding.size() !=
679 checked_size(model.config.vocab_size, "vocab_size") *
680 checked_size(model.config.hidden_size, "hidden_size")) {
681 throw std::runtime_error("token embedding size mismatch");
682 }
683 if (model.final_norm_weight.size() != checked_size(model.config.hidden_size↪
 ↪ , "hidden_size")) {
684 throw std::runtime_error("final norm weight size mismatch");
685 }
686 validate_linear(model.lm_head, "lm_head");
687
688 ReferenceKVCache cache(model.config, static_cast<std::int32_t>(model.layers↪
 ↪ .size()));
689 std::vector<float> hidden(checked_size(model.config.hidden_size, "↪
 ↪ hidden_size"), 0.0F);
690 for (std::int32_t position = 0; position < static_cast<std::int32_t>(↪
 ↪ token_ids.size());
691 ++position) {
692 const std::int32_t token_id = token_ids[checked_size(position, "↪
 ↪ position")];
693 if (token_id < 0 || token_id >= model.config.vocab_size) {
694 throw std::runtime_error("token id out of vocabulary");
695 }
696 const std::span<const float> embedding =
697 row_span(model.token_embedding, token_id, model.config.hidden_size)↪
 ↪ ;
698 std::copy(embedding.begin(), embedding.end(), hidden.begin());
699 for (std::int32_t layer_id = 0; layer_id < static_cast<std::int32_t>(↪

```



[illegible]

```

748 0.05F, 0.10F, -0.20F, 0.15F,
749 0.30F, -0.05F, 0.20F, 0.10F,
750 });
751 layer.wk = make_linear(4,
752 2,
753 {
754 0.10F, 0.20F, -0.10F, 0.05F,
755 -0.05F, 0.15F, 0.25F, -0.20F,
756 });
757 layer.wv = make_linear(4,
758 2,
759 {
760 0.25F, -0.05F, 0.10F, 0.15F,
761 -0.10F, 0.30F, 0.05F, 0.20F,
762 });
763 layer.wo = make_linear(4,
764 4,
765 {
766 0.20F, 0.10F, -0.05F, 0.15F,
767 -0.10F, 0.25F, 0.20F, -0.05F,
768 0.05F, -0.20F, 0.30F, 0.10F,
769 0.15F, 0.05F, -0.10F, 0.25F,
770 });
771 layer.rms_mlp_weight = {0.95F, 1.05F, 1.0F, 0.9F};
772 layer.w_gate = make_linear(4,
773 6,
774 {
775 0.20F, -0.10F, 0.05F, 0.10F,
776 -0.05F, 0.15F, 0.20F, -0.10F,
777 0.10F, 0.05F, -0.15F, 0.25F,
778 -0.20F, 0.10F, 0.15F, 0.05F,
779 0.05F, -0.25F, 0.10F, 0.20F,
780 0.15F, 0.05F, 0.05F, -0.15F,
781 });
782 layer.w_up = make_linear(4,
783 6,
784 {
785 0.10F, 0.20F, -0.05F, 0.05F,
786 -0.15F, 0.05F, 0.25F, 0.10F,
787 0.20F, -0.10F, 0.05F, 0.15F,
788 0.05F, 0.10F, -0.20F, 0.25F,
789 -0.05F, 0.15F, 0.10F, -0.10F,
790 0.25F, -0.05F, 0.05F, 0.10F,
791 });
792 layer.w_down = make_linear(6,
793 4,
794 {
795 0.10F, -0.05F, 0.15F, 0.20F, -0.10F, 0.05F,
796 -0.20F, 0.10F, 0.05F, -0.05F, 0.15F, 0.20F,
797 0.05F, 0.15F, -0.10F, 0.10F, 0.20F, -0.05F,
798 0.20F, 0.05F, 0.10F, -0.15F, -0.05F, 0.10F,
799 });

```

```

800 model.layers.push_back(std::move(layer));
801 model.final_norm_weight = {1.0F, 0.95F, 1.05F, 1.0F};
802 model.lm_head = make_linear(4,
803 5,
804 {
805 0.20F, -0.10F, 0.05F, 0.15F,
806 -0.05F, 0.25F, 0.10F, -0.20F,
807 0.15F, 0.05F, -0.25F, 0.10F,
808 -0.10F, 0.20F, 0.15F, 0.05F,
809 0.05F, -0.15F, 0.20F, 0.25F,
810 },
811 zeros(5));
812 return model;
813 }
814
815 } // namespace lcqi

```

### B.13 十三、Int8 Kernel 基础实现

`int8_kernels.cpp` 包含 scalar、packed、workspace、deterministic fixture 和误差比较。它是 AVX2 路径的 reference 对照。

**Listing B.13** `src/int8_kernels.cpp`

```

1 #include <lcqi/int8_kernels.hpp>
2
3 #include <lcqi/inference.hpp>
4
5 #include <algorithm>
6 #include <chrono>
7 #include <cmath>
8 #include <stdexcept>
9 #include <string>
10
11 namespace lcqi {
12 namespace {
13
14 constexpr std::int32_t LCQI_I8_PATTERN_MODULUS = 23;
15 constexpr std::int32_t LCQI_I8_PATTERN_CENTER = 11;
16 constexpr std::int32_t LCQI_INPUT_PATTERN_MODULUS = 17;
17 constexpr std::int32_t LCQI_INPUT_PATTERN_CENTER = 8;
18 constexpr float LCQI_INPUT_PATTERN_SCALE = 0.125F;
19 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
20 constexpr std::int32_t LCQI_SIMD_OUTPUT_BLOCK = 8;
21 constexpr std::size_t LCQI_BENCHMARK_BACKEND_COUNT = 3;
22 constexpr const char* LCQI_BACKEND_SCALAR = "scalar";
23 constexpr const char* LCQI_BACKEND_PACKED_SCALAR = "packed_scalar";
24 constexpr const char* LCQI_BACKEND_AVX2 = "avx2";
25
26 void require_positive(std::int32_t value, const char* name) {
27 if (value <= 0) {
28 throw std::runtime_error(std::string(name) + " must be positive");

```

```

29 }
30 }
31
32 std::size_t checked_size(std::int32_t value, const char* name) {
33 if (value < 0) {
34 throw std::runtime_error(std::string(name) + " cannot be negative");
35 }
36 return static_cast<std::size_t>(value);
37 }
38
39 void validate_quantized_layer(const QuantizedLinearLayer& layer) {
40 require_positive(layer.input_size, "input_size");
41 require_positive(layer.output_size, "output_size");
42 const std::size_t expected_weights =
43 checked_size(layer.input_size, "input_size") *
44 checked_size(layer.output_size, "output_size");
45 if (layer.weights.size() != expected_weights) {
46 throw std::runtime_error("quantized layer weight size mismatch");
47 }
48 if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
49 throw std::runtime_error("quantized layer bias size mismatch");
50 }
51 }
52
53 std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
54 require_positive(value, "value");
55 require_positive(block_size, "block_size");
56 return ((value + block_size - 1) / block_size) * block_size;
57 }
58
59 float checksum(std::span<const float> values) {
60 float sum = 0.0F;
61 for (const float value : values) {
62 sum += value;
63 }
64 return sum;
65 }
66
67 void add_unavailable_backend(KernelBenchmarkResult& result, const char* name) {
68 result.backends.push_back(KernelBackendBenchmark{
69 .name = name,
70 .available = false,
71 .average_us = 0.0,
72 .max_abs_diff = 0.0F,
73 .checksum = 0.0F,
74 });
75 }
76
77 template <typename Callable>
78 double measure_average_us(std::int32_t repeat, Callable&& callable) {
79 require_positive(repeat, "repeat");
80 const auto begin = std::chrono::steady_clock::now();

```

```

81 for (std::int32_t index = 0; index < repeat; ++index) {
82 callable();
83 }
84 const auto end = std::chrono::steady_clock::now();
85 const std::chrono::duration<double> elapsed = end - begin;
86 return elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>←
 ↪ >(repeat);
87 }
88
89 } // namespace
90
91 PackedLinearI8 pack_linear_i8(const QuantizedLinearLayer& layer,
92 std::int32_t output_block_size) {
93 validate_quantized_layer(layer);
94 require_positive(output_block_size, "output_block_size");
95
96 PackedLinearI8 packed;
97 packed.input_size = layer.input_size;
98 packed.output_size = layer.output_size;
99 packed.output_block_size = output_block_size;
100 packed.scale = layer.scale;
101 packed.bias = layer.bias;
102
103 const std::int32_t padded_outputs =
104 round_up_to_block(layer.output_size, output_block_size);
105 packed.packed_weights.assign(
106 checked_size(padded_outputs, "padded_outputs") *
107 checked_size(layer.input_size, "input_size"),
108 0);
109
110 std::size_t packed_index = 0;
111 for (std::int32_t output_block = 0; output_block < padded_outputs;
112 output_block += output_block_size) {
113 for (std::int32_t input_index = 0; input_index < layer.input_size; ++←
 ↪ input_index) {
114 for (std::int32_t offset = 0; offset < output_block_size; ++offset)←
 ↪ {
115 const std::int32_t output_index = output_block + offset;
116 if (output_index < layer.output_size) {
117 const std::size_t source =
118 checked_size(output_index, "output_index") *
119 checked_size(layer.input_size, "input_size") +
120 checked_size(input_index, "input_index");
121 packed.packed_weights[packed_index] = layer.weights[source←
 ↪];
122 }
123 ++packed_index;
124 }
125 }
126 }
127 return packed;
128 }

```

```

129
130 void linear_i8_packed(const PackedLinearI8& layer,
131 std::span<const float> input,
132 std::span<float> output) {
133 std::vector<float> accumulators(
134 checked_size(layer.output_block_size, "output_block_size"),
135 0.0F);
136 linear_i8_packed_with_workspace(layer, input, output, accumulators);
137 }
138
139 void linear_i8_packed_with_workspace(const PackedLinearI8& layer,
140 std::span<const float> input,
141 std::span<float> output,
142 std::span<float> accumulator_workspace) {
143 require_positive(layer.input_size, "packed input_size");
144 require_positive(layer.output_size, "packed output_size");
145 require_positive(layer.output_block_size, "packed output_block_size");
146 if (input.size() != checked_size(layer.input_size, "input_size")) {
147 throw std::runtime_error("linear_i8_packed input size mismatch");
148 }
149 if (output.size() != checked_size(layer.output_size, "output_size")) {
150 throw std::runtime_error("linear_i8_packed output size mismatch");
151 }
152 if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
153 throw std::runtime_error("linear_i8_packed bias size mismatch");
154 }
155
156 const std::int32_t padded_outputs =
157 round_up_to_block(layer.output_size, layer.output_block_size);
158 const std::size_t expected_packed_size =
159 checked_size(padded_outputs, "padded_outputs") *
160 checked_size(layer.input_size, "input_size");
161 if (layer.packed_weights.size() != expected_packed_size) {
162 throw std::runtime_error("linear_i8_packed packed weight size mismatch" ←
163 ↪);
164 }
165 if (accumulator_workspace.size() != checked_size(layer.output_block_size, "←
166 ↪ output_block_size")) {
167 throw std::runtime_error("linear_i8_packed accumulator workspace size ←
168 ↪ mismatch");
169 }
170
171 std::size_t packed_index = 0;
172 for (std::int32_t output_block = 0; output_block < padded_outputs;
173 output_block += layer.output_block_size) {
174 std::fill(accumulator_workspace.begin(), accumulator_workspace.end(), ←
175 ↪ 0.0F);
176 for (std::int32_t offset = 0; offset < layer.output_block_size; ++↪
177 ↪ offset) {
178 const std::int32_t output_index = output_block + offset;
179 if (output_index < layer.output_size) {
180 accumulator_workspace[checked_size(offset, "offset")] =

```

```

176 layer.bias[checked_size(output_index, "output_index")];
177 }
178 }
179
180 for (std::int32_t input_index = 0; input_index < layer.input_size; ++input_index) {
181 const float input_value = input[checked_size(input_index, "input_index")];
182 for (std::int32_t offset = 0; offset < layer.output_block_size; ++offset) {
183 const std::int32_t output_index = output_block + offset;
184 const std::int8_t weight = layer.packed_weights[packed_index + offset];
185 if (output_index < layer.output_size) {
186 accumulator_workspace[checked_size(offset, "offset")] +=
187 static_cast<float>(weight) * layer.scale * input_value;
188 }
189 }
190 }
191
192 for (std::int32_t offset = 0; offset < layer.output_block_size; ++offset) {
193 const std::int32_t output_index = output_block + offset;
194 if (output_index < layer.output_size) {
195 output[checked_size(output_index, "output_index")] =
196 accumulator_workspace[checked_size(offset, "offset")];
197 }
198 }
199 }
200 }
201
202 QuantizedLinearLayer make_deterministic_i8_layer(std::int32_t input_size,
203 std::int32_t output_size,
204 float scale) {
205 require_positive(input_size, "input_size");
206 require_positive(output_size, "output_size");
207 QuantizedLinearLayer layer;
208 layer.input_size = input_size;
209 layer.output_size = output_size;
210 layer.scale = scale;
211 const std::size_t weight_count =
212 checked_size(input_size, "input_size") * checked_size(output_size, "output_size");
213 layer.weights.resize(weight_count);
214 layer.bias.resize(checked_size(output_size, "output_size"));
215 for (std::int32_t output_index = 0; output_index < output_size; ++output_index) {
216 layer.bias[checked_size(output_index, "output_index")] =
217 static_cast<float>((output_index % LCQI_INPUT_PATTERN_MODULUS) -
218 LCQI_INPUT_PATTERN_CENTER) *
219 LCQI_INPUT_PATTERN_SCALE;
220 for (std::int32_t input_index = 0; input_index < input_size; ++input_index) {

```



```

 ↪ input_index) {
221 const std::int32_t raw =
222 ((output_index * input_size + input_index) % ↪
 ↪ LCQI_I8_PATTERN_MODULUS) -
223 LCQI_I8_PATTERN_CENTER;
224 layer.weights[checked_size(output_index, "output_index") *
225 checked_size(input_size, "input_size") +
226 checked_size(input_index, "input_index")] =
227 static_cast<std::int8_t>(raw);
228 }
229 }
230 return layer;
231 }
232
233 std::vector<float> make_deterministic_input(std::int32_t input_size) {
234 require_positive(input_size, "input_size");
235 std::vector<float> input(checked_size(input_size, "input_size"));
236 for (std::int32_t input_index = 0; input_index < input_size; ++input_index)↪
 ↪ {
237 input[checked_size(input_index, "input_index")] =
238 static_cast<float>((input_index % LCQI_INPUT_PATTERN_MODULUS) -
239 LCQI_INPUT_PATTERN_CENTER) *
240 LCQI_INPUT_PATTERN_SCALE;
241 }
242 return input;
243 }
244
245 float max_abs_diff(std::span<const float> lhs, std::span<const float> rhs) {
246 if (lhs.size() != rhs.size()) {
247 throw std::runtime_error("max_abs_diff size mismatch");
248 }
249 float diff = 0.0F;
250 for (std::size_t index = 0; index < lhs.size(); ++index) {
251 diff = std::max(diff, std::fabs(lhs[index] - rhs[index]));
252 }
253 return diff;
254 }
255
256 KernelBenchmarkResult benchmark_linear_i8_case(const KernelBenchmarkCase& ↪
 ↪ benchmark_case) {
257 require_positive(benchmark_case.input_size, "benchmark input_size");
258 require_positive(benchmark_case.output_size, "benchmark output_size");
259 require_positive(benchmark_case.output_block_size, "benchmark ↪
 ↪ output_block_size");
260 require_positive(benchmark_case.repeat, "benchmark repeat");
261
262 const QuantizedLinearLayer layer =
263 make_deterministic_i8_layer(benchmark_case.input_size,
264 benchmark_case.output_size,
265 0.015625F);
266 const PackedLinearI8 packed = pack_linear_i8(layer, benchmark_case.↪
 ↪ output_block_size);

```

```

267 const PackedLinearI8 packed_simd = pack_linear_i8(layer, ↵
↵ LCQI_SIMD_OUTPUT_BLOCK);
268 const std::vector<float> input = make_deterministic_input(benchmark_case.↵
↵ input_size);
269 std::vector<float> scalar_output(checked_size(benchmark_case.output_size, "↵
↵ output_size"),
270 0.0F);
271 std::vector<float> packed_output(scalar_output.size(), 0.0F);
272 std::vector<float> simd_output(scalar_output.size(), 0.0F);
273 std::vector<float> packed_workspace(
274 checked_size(benchmark_case.output_block_size, "output_block_size"),
275 0.0F);
276
277 linear_i8(layer, input, scalar_output);
278 linear_i8_packed_with_workspace(packed, input, packed_output, ↵
↵ packed_workspace);
279
280 KernelBenchmarkResult result;
281 result.benchmark_case = benchmark_case;
282 result.backends.reserve(LCQI_BENCHMARK_BACKEND_COUNT);
283 result.backends.push_back(KernelBackendBenchmark{
284 .name = LCQI_BACKEND_SCALAR,
285 .available = true,
286 .average_us = measure_average_us(benchmark_case.repeat,
287 [&]() { linear_i8(layer, input, ↵
↵ scalar_output); }),
288 .max_abs_diff = 0.0F,
289 .checksum = checksum(scalar_output),
290 });
291 result.backends.push_back(KernelBackendBenchmark{
292 .name = LCQI_BACKEND_PACKED_SCALAR,
293 .available = true,
294 .average_us = measure_average_us(
295 benchmark_case.repeat,
296 [&]() { linear_i8_packed_with_workspace(packed,
297 input,
298 packed_output,
299 packed_workspace); }),
300 .max_abs_diff = max_abs_diff(scalar_output, packed_output),
301 .checksum = checksum(packed_output),
302 });
303 if (linear_i8_packed_avx2_available()) {
304 linear_i8_packed_avx2(packed_simd, input, simd_output);
305 result.backends.push_back(KernelBackendBenchmark{
306 .name = LCQI_BACKEND_AVX2,
307 .available = true,
308 .average_us = measure_average_us(
309 benchmark_case.repeat,
310 [&]() { linear_i8_packed_avx2(packed_simd, input, simd_output); ↵
↵ }),
311 .max_abs_diff = max_abs_diff(scalar_output, simd_output),
312 .checksum = checksum(simd_output),

```

```

313 });
314 } else {
315 add_unavailable_backend(result, LCQI_BACKEND_AVX2);
316 }
317 return result;
318 }
319
320 } // namespace lcqi
321
322 #if !defined(LCQI_ENABLE_AVX2)
323 namespace lcqi {
324
325 bool linear_i8_packed_avx2_available() noexcept {
326 return false;
327 }
328
329 void linear_i8_packed_avx2(const PackedLinearI8&, std::span<const float>, std::span<float>) {
330 throw std::runtime_error("AVX2 packed kernel is not enabled in this build");
331 }
332
333 } // namespace lcqi
334 #endif

```

## B.14 十四、Int8 Kernel AVX2 实现

`int8_kernels_avx2.cpp` 是平台优化路径。读者要先看可用性判断, 再看向量化实现; 如果机器不支持 AVX2, 报告必须写 `unavailable`。

**Listing B.14** `src/int8_kernels_avx2.cpp`

```

1 #include <lcqi/int8_kernels.hpp>
2
3 #if defined(LCQI_ENABLE_AVX2)
4
5 #include <immintrin.h>
6
7 #include <algorithm>
8 #include <cstdint>
9 #include <cstring>
10 #include <stdexcept>
11 #include <string>
12
13 namespace lcqi {
14 namespace {
15
16 constexpr std::int32_t LCQI_AVX2_OUTPUT_BLOCK = 8;
17
18 void require_positive(std::int32_t value, const char* name) {
19 if (value <= 0) {
20 throw std::runtime_error(std::string(name) + " must be positive");

```

```

21 }
22 }
23
24 std::size_t checked_size(std::int32_t value, const char* name) {
25 if (value < 0) {
26 throw std::runtime_error(std::string(name) + " cannot be negative");
27 }
28 return static_cast<std::size_t>(value);
29 }
30
31 std::int32_t round_up_to_block(std::int32_t value, std::int32_t block_size) {
32 require_positive(value, "value");
33 require_positive(block_size, "block_size");
34 return ((value + block_size - 1) / block_size) * block_size;
35 }
36
37 void validate_avx2_contract(const PackedLinearI8& layer,
38 std::span<const float> input,
39 std::span<float> output) {
40 require_positive(layer.input_size, "packed input_size");
41 require_positive(layer.output_size, "packed output_size");
42 if (layer.output_block_size != LCQI_AVX2_OUTPUT_BLOCK) {
43 throw std::runtime_error("AVX2 packed kernel requires output_block_size ←
44 ← == 8");
45 }
46 if (input.size() != checked_size(layer.input_size, "input_size")) {
47 throw std::runtime_error("AVX2 packed kernel input size mismatch");
48 }
49 if (output.size() != checked_size(layer.output_size, "output_size")) {
50 throw std::runtime_error("AVX2 packed kernel output size mismatch");
51 }
52 if (layer.bias.size() != checked_size(layer.output_size, "output_size")) {
53 throw std::runtime_error("AVX2 packed kernel bias size mismatch");
54 }
55 const std::int32_t padded_outputs =
56 round_up_to_block(layer.output_size, layer.output_block_size);
57 const std::size_t expected_packed_size =
58 checked_size(padded_outputs, "padded_outputs") *
59 checked_size(layer.input_size, "input_size");
60 if (layer.packed_weights.size() != expected_packed_size) {
61 throw std::runtime_error("AVX2 packed kernel packed weight size ←
62 ← mismatch");
63 }
64 }
65 // namespace
66
67 bool linear_i8_packed_avx2_available() noexcept {
68 #if defined(__GNUC__) || defined(__clang__)
69 __builtin_cpu_init();
70 return __builtin_cpu_supports("avx2") && __builtin_cpu_supports("fma");
71 #else

```

```

71 return true;
72 #endif
73 }
74
75 void linear_i8_packed_avx2(const PackedLinearI8& layer,
76 std::span<const float> input,
77 std::span<float> output) {
78 if (!linear_i8_packed_avx2_available()) {
79 throw std::runtime_error("AVX2/FMA is not available on this CPU");
80 }
81 validate_avx2_contract(layer, input, output);
82
83 const std::int32_t padded_outputs =
84 round_up_to_block(layer.output_size, layer.output_block_size);
85 std::size_t packed_index = 0;
86 alignas(32) float block_output[LCQI_AVX2_OUTPUT_BLOCK] = {};
87
88 for (std::int32_t output_block = 0; output_block < padded_outputs;
89 output_block += LCQI_AVX2_OUTPUT_BLOCK) {
90 __m256 accumulator = _mm256_setzero_ps();
91
92 for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset) {
93 const std::int32_t output_index = output_block + offset;
94 block_output[checked_size(offset, "offset")] =
95 output_index < layer.output_size
96 ? layer.bias[checked_size(output_index, "output_index")]
97 : 0.0F;
98 }
99 accumulator = _mm256_load_ps(block_output);
100
101 for (std::int32_t input_index = 0; input_index < layer.input_size; ++
102 input_index) {
103 std::uint64_t packed_bytes = 0;
104 std::memcpy(&packed_bytes,
105 layer.packed_weights.data() + packed_index,
106 sizeof(packed_bytes));
107 const __m128i packed_i8 =
108 _mm_cvtsi64_si128(static_cast<long long>(packed_bytes));
109 packed_index += LCQI_AVX2_OUTPUT_BLOCK;
110 const __m256i weights_i32 = _mm256_cvtepi8_epi32(packed_i8);
111 const __m256 weights_f32 = _mm256_cvtepi32_ps(weights_i32);
112 const __m256 input_scaled =
113 _mm256_set1_ps(input[checked_size(input_index, "input_index")]
114 * layer.scale);
115 accumulator = _mm256_fmadd_ps(weights_f32, input_scaled,
116 accumulator);
117 }
118 __mm256_store_ps(block_output, accumulator);
119 for (std::int32_t offset = 0; offset < LCQI_AVX2_OUTPUT_BLOCK; ++offset) {

```

```

118 const std::int32_t output_index = output_block + offset;
119 if (output_index < layer.output_size) {
120 output[checked_size(output_index, "output_index")] =
121 block_output[checked_size(offset, "offset")];
122 }
123 }
124 }
125 }
126
127 } // namespace lcqi
128
129 #endif

```

## B.15 十五、Tokenizer 实现

tokenizer 实现把 prompt 变成 token id, 并固定 unknown token 行为。读者应检查 BOS/EOS 是否稳定插入, UNK 是否按合同处理。

Listing B.15 src/tokenizer.cpp

```

1 #include <lcqi/tokenizer.hpp>
2
3 #include <cstdint>
4 #include <fstream>
5 #include <stdexcept>
6 #include <string>
7 #include <string_view>
8
9 namespace lcqi {
10 namespace {
11
12 constexpr std::uint64_t LCQI_FNV_OFFSET_BASIS = 1469598103934665603ULL;
13 constexpr std::uint64_t LCQI_FNV_PRIME = 1099511628211ULL;
14 constexpr std::int32_t LCQI_INVALID_TOKEN_ID = -1;
15
16 std::string read_key(std::istream& input) {
17 std::string key;
18 if (!(input >> key)) {
19 throw std::runtime_error("unexpected end of tokenizer file");
20 }
21 return key;
22 }
23
24 void expect_key(std::istream& input, const std::string& expected) {
25 const std::string key = read_key(input);
26 if (key != expected) {
27 throw std::runtime_error("expected tokenizer key '" + expected + "', ←
↪ got '" + key + "'");
28 }
29 }
30
31 std::int32_t read_i32(std::istream& input, const std::string& key) {

```

```

32 expect_key(input, key);
33 std::int32_t value = 0;
34 if (!(input >> value)) {
35 throw std::runtime_error("invalid tokenizer int32 value for key '" + ↵
↵ key + "'");
36 }
37 return value;
38 }
39
40 std::string read_string(std::istream& input, const std::string& key) {
41 expect_key(input, key);
42 std::string value;
43 if (!(input >> value)) {
44 throw std::runtime_error("invalid tokenizer string value for key '" + ↵
↵ key + "'");
45 }
46 return value;
47 }
48
49 std::int32_t lookup_token_id(const TokenizerModel& tokenizer, std::string_view ↵
↵ text) {
50 for (const TokenEntry& entry : tokenizer.vocab) {
51 if (entry.text == text) {
52 return entry.id;
53 }
54 }
55 return LCQI_INVALID_TOKEN_ID;
56 }
57
58 void validate_tokenizer(const TokenizerModel& tokenizer) {
59 if (tokenizer.vocab.empty()) {
60 throw std::runtime_error("tokenizer vocab is empty");
61 }
62 for (std::size_t i = 0; i < tokenizer.vocab.size(); ++i) {
63 const TokenEntry& left = tokenizer.vocab[i];
64 if (left.id < 0) {
65 throw std::runtime_error("tokenizer token id must be non-negative")↵
↵ ;
66 }
67 for (std::size_t j = i + 1; j < tokenizer.vocab.size(); ++j) {
68 const TokenEntry& right = tokenizer.vocab[j];
69 if (left.id == right.id) {
70 throw std::runtime_error("duplicate tokenizer token id");
71 }
72 if (left.text == right.text) {
73 throw std::runtime_error("duplicate tokenizer token text");
74 }
75 }
76 }
77 if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID &&
78 lookup_token_id(tokenizer, "<bos>") != tokenizer.bos_id) {
79 throw std::runtime_error("tokenizer BOS id does not match vocab");

```



```

80 }
81 if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID &&
82 lookup_token_id(tokenizer, "<eos>") != tokenizer.eos_id) {
83 throw std::runtime_error("tokenizer EOS id does not match vocab");
84 }
85 }
86
87 std::uint64_t hash_byte(std::uint64_t hash, unsigned char value) {
88 hash ^= static_cast<std::uint64_t>(value);
89 hash *= LCQI_FNV_PRIME;
90 return hash;
91 }
92
93 std::uint64_t hash_string(std::uint64_t hash, std::string_view text) {
94 for (const unsigned char value : text) {
95 hash = hash_byte(hash, value);
96 }
97 return hash;
98 }
99
100 std::uint64_t hash_i32(std::uint64_t hash, std::int32_t value) {
101 const std::uint32_t bits = static_cast<std::uint32_t>(value);
102 for (std::int32_t shift = 0; shift < 32; shift += 8) {
103 hash = hash_byte(hash, static_cast<unsigned char>((bits >> shift) & 0x
104 ↪ xFFU));
105 }
106 return hash;
107 }
108 } // namespace
109
110 TokenizerModel load_tokenizer(const std::filesystem::path& path) {
111 std::ifstream input(path);
112 if (!input) {
113 throw std::runtime_error("cannot open tokenizer file: " + path.string()
114 ↪);
115 }
116
117 expect_key(input, "LCQI_TOKENIZER_V1");
118
119 TokenizerModel tokenizer;
120 tokenizer.bos_id = read_i32(input, "bos_id");
121 tokenizer.eos_id = read_i32(input, "eos_id");
122 tokenizer.unk_id = read_i32(input, "unk_id");
123 tokenizer.chat_template_hash = read_string(input, "chat_template_hash");
124 const std::int32_t vocab_size = read_i32(input, "vocab_size");
125 if (vocab_size <= 0) {
126 throw std::runtime_error("tokenizer vocab_size must be positive");
127 }
128
129 tokenizer.vocab.reserve(static_cast<std::size_t>(vocab_size));
130 expect_key(input, "vocab");

```

```

130 for (std::int32_t i = 0; i < vocab_size; ++i) {
131 TokenEntry entry;
132 if (!(input >> entry.id >> entry.text)) {
133 throw std::runtime_error("invalid tokenizer vocab entry");
134 }
135 tokenizer.vocab.push_back(entry);
136 }
137
138 validate_tokenizer(tokenizer);
139 return tokenizer;
140 }
141
142 std::vector<std::int32_t> encode_prompt(const TokenizerModel& tokenizer,
143 std::string_view prompt) {
144 std::vector<std::int32_t> ids;
145 if (tokenizer.bos_id != LCQI_INVALID_TOKEN_ID) {
146 ids.push_back(tokenizer.bos_id);
147 }
148
149 std::size_t begin = 0;
150 while (begin < prompt.size()) {
151 while (begin < prompt.size() && prompt[begin] == ' ') {
152 ++begin;
153 }
154 if (begin >= prompt.size()) {
155 break;
156 }
157 std::size_t end = begin;
158 while (end < prompt.size() && prompt[end] != ' ') {
159 ++end;
160 }
161 const std::string_view token = prompt.substr(begin, end - begin);
162 const std::int32_t token_id = lookup_token_id(tokenizer, token);
163 if (token_id == LCQI_INVALID_TOKEN_ID) {
164 if (tokenizer.unk_id == LCQI_INVALID_TOKEN_ID) {
165 throw std::runtime_error("tokenizer encountered unknown token")↵
166 ↵ ;
167 }
168 ids.push_back(tokenizer.unk_id);
169 } else {
170 ids.push_back(token_id);
171 }
172 begin = end;
173 }
174
175 if (tokenizer.eos_id != LCQI_INVALID_TOKEN_ID) {
176 ids.push_back(tokenizer.eos_id);
177 }
178 return ids;
179 }
180
181 std::uint64_t tokenizer_contract_hash(const TokenizerModel& tokenizer) {

```

```

181 std::uint64_t hash = LCQI_FNV_OFFSET_BASIS;
182 hash = hash_string(hash, tokenizer.chat_template_hash);
183 hash = hash_i32(hash, tokenizer.bos_id);
184 hash = hash_i32(hash, tokenizer.eos_id);
185 hash = hash_i32(hash, tokenizer.unk_id);
186 for (const TokenEntry& entry : tokenizer.vocab) {
187 hash = hash_string(hash, entry.text);
188 hash = hash_i32(hash, entry.id);
189 }
190 return hash;
191 }
192
193 } // namespace lcqi

```

## B.16 十六、Safetensors 实现

safetensors 实现解析 header、metadata、dtype、shape 和 data offsets。坏 dtype、坏 shape、坏 offset 和 payload 太短都应该被拒绝。

Listing B.16 src/safetensors.cpp

```

1 #include <lcqi/safetensors.hpp>
2
3 #include <algorithm>
4 #include <cctype>
5 #include <cstdint>
6 #include <fstream>
7 #include <limits>
8 #include <stdexcept>
9 #include <string>
10 #include <string_view>
11
12 namespace lcqi {
13 namespace {
14
15 constexpr std::size_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
16 constexpr std::uint64_t LCQI_SAFETENSORS_MAX_HEADER_BYTES = 16U * 1024U * 1024U↵
 ↵ ;
17 constexpr std::size_t LCQI_SAFETENSORS_OFFSET_COUNT = 2;
18 constexpr std::uint64_t LCQI_BYTE_BITS = 8;
19 constexpr std::uint64_t LCQI_DECIMAL_BASE = 10;
20 constexpr std::uint64_t LCQI_BYTE_MASK = 0xFFU;
21 constexpr unsigned char LCQI_JSON_CONTROL_CHAR_LIMIT = 0x20U;
22 constexpr std::uint64_t LCQI_DTYPE_BYTES_F64 = 8;
23 constexpr std::uint64_t LCQI_DTYPE_BYTES_F32 = 4;
24 constexpr std::uint64_t LCQI_DTYPE_BYTES_F16 = 2;
25 constexpr std::uint64_t LCQI_DTYPE_BYTES_I64 = 8;
26 constexpr std::uint64_t LCQI_DTYPE_BYTES_I32 = 4;
27 constexpr std::uint64_t LCQI_DTYPE_BYTES_I16 = 2;
28 constexpr std::uint64_t LCQI_DTYPE_BYTES_I8 = 1;
29 constexpr std::uint64_t LCQI_DTYPE_BYTES_U8 = 1;
30

```

```

31 class HeaderCursor {
32 public:
33 explicit HeaderCursor(std::string_view text) : text_(text) {}
34
35 void expect(char expected) {
36 this->skip_ws();
37 if (this->eof() || this->text_[this->position_] != expected) {
38 throw std::runtime_error("invalid safetensors header syntax");
39 }
40 ++this->position_;
41 }
42
43 [[nodiscard]] bool consume(char expected) {
44 this->skip_ws();
45 if (!this->eof() && this->text_[this->position_] == expected) {
46 ++this->position_;
47 return true;
48 }
49 return false;
50 }
51
52 [[nodiscard]] std::string parse_string() {
53 this->skip_ws();
54 this->expect('"');
55 std::string result;
56 while (!this->eof()) {
57 const char ch = this->text_[this->position_++];
58 if (ch == '"') {
59 return result;
60 }
61 if (static_cast<unsigned char>(ch) < LCQI_JSON_CONTROL_CHAR_LIMIT) ↵
↵ {
62 throw std::runtime_error("control character in safetensors ↵
↵ string");
63 }
64 if (ch == '\\') {
65 result.push_back(this->parse_escape());
66 } else {
67 result.push_back(ch);
68 }
69 }
70 throw std::runtime_error("unterminated safetensors string");
71 }
72
73 [[nodiscard]] std::uint64_t parse_u64() {
74 this->skip_ws();
75 if (this->eof() ||
76 !std::isdigit(static_cast<unsigned char>(this->text_[this->↵
↵ position_]))) {
77 throw std::runtime_error("expected unsigned integer in safetensors ↵
↵ header");
78 }

```

```

79 std::uint64_t value = 0;
80 while (!this->eof() &&
81 std::isdigit(static_cast<unsigned char>(this->text_[this->↵
↵ position_]))) {
82 const std::uint64_t digit =
83 static_cast<std::uint64_t>(this->text_[this->position_] - '0');
84 if (value >
85 (std::numeric_limits<std::uint64_t>::max() - digit) / ↵
↵ LCQI_DECIMAL_BASE) {
86 throw std::runtime_error("safetensors integer overflow");
87 }
88 value = value * LCQI_DECIMAL_BASE + digit;
89 ++this->position_;
90 }
91 return value;
92 }
93
94 [[nodiscard]] std::vector<std::int64_t> parse_i64_array() {
95 std::vector<std::int64_t> values;
96 this->expect('[');
97 if (this->consume(' ')) {
98 return values;
99 }
100 while (true) {
101 const std::uint64_t raw = this->parse_u64();
102 if (raw > static_cast<std::uint64_t>(std::numeric_limits<std::↵
↵ int64_t>::max())) {
103 throw std::runtime_error("safetensors shape value is too large"↵
↵);
104 }
105 values.push_back(static_cast<std::int64_t>(raw));
106 if (this->consume(' ')) {
107 return values;
108 }
109 this->expect(',');
110 }
111 }
112
113 [[nodiscard]] std::vector<std::uint64_t> parse_u64_array() {
114 std::vector<std::uint64_t> values;
115 this->expect('[');
116 if (this->consume(' ')) {
117 return values;
118 }
119 while (true) {
120 values.push_back(this->parse_u64());
121 if (this->consume(' ')) {
122 return values;
123 }
124 this->expect(',');
125 }
126 }

```

```

127
128 [[nodiscard]] bool eof_after_ws() {
129 this->skip_ws();
130 return this->eof();
131 }
132
133 private:
134 std::string_view text_;
135 std::size_t position_ = 0;
136
137 [[nodiscard]] bool eof() const {
138 return this->position_ >= this->text_.size();
139 }
140
141 void skip_ws() {
142 while (!this->eof() &&
143 std::isspace(static_cast<unsigned char>(this->text_[this->↵
↵ position_]))) {
144 ++this->position_;
145 }
146 }
147
148 [[nodiscard]] char parse_escape() {
149 if (this->eof()) {
150 throw std::runtime_error("unterminated safetensors escape");
151 }
152 const char escaped = this->text_[this->position_++];
153 switch (escaped) {
154 case '"':
155 case '\\':
156 case '/':
157 return escaped;
158 case 'b':
159 return '\b';
160 case 'f':
161 return '\f';
162 case 'n':
163 return '\n';
164 case 'r':
165 return '\r';
166 case 't':
167 return '\t';
168 default:
169 throw std::runtime_error("unsupported safetensors string escape↵
↵ ");
170 }
171 }
172 };
173
174 std::uint64_t read_le_u64(const std::string& bytes) {
175 if (bytes.size() < LCQI_SAFETENSORS_PREFIX_BYTES) {
176 throw std::runtime_error("safetensors file is too small");

```

```

177 }
178 std::uint64_t value = 0;
179 for (std::size_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
180 value |= static_cast<std::uint64_t>(
181 static_cast<unsigned char>(bytes[i]))
182 << (LCQI_BYTE_BITS * i);
183 }
184 return value;
185 }
186
187 std::string read_file_bytes(const std::filesystem::path& path) {
188 std::ifstream input(path, std::ios::binary);
189 if (!input) {
190 throw std::runtime_error("cannot open safetensors file: " + path.string()
191 ↵ <());
192 }
193 input.seekg(0, std::ios::end);
194 const std::streamoff size = input.tellg();
195 if (size < 0) {
196 throw std::runtime_error("cannot determine safetensors file size");
197 }
198 input.seekg(0, std::ios::beg);
199 std::string bytes(static_cast<std::size_t>(size), '\0');
200 if (!bytes.empty() && !input.read(bytes.data(), static_cast<std::streamsize>
201 ↵ < >(bytes.size())) {
202 throw std::runtime_error("cannot read safetensors file");
203 }
204 return bytes;
205 }
206
207 void parse_metadata_value(HeaderCursor& cursor,
208 SafeTensorManifest& manifest,
209 const std::string& key) {
210 if (key == "__metadata__") {
211 cursor.expect('{');
212 if (cursor.consume('}')) {
213 return;
214 }
215 while (true) {
216 const std::string metadata_key = cursor.parse_string();
217 cursor.expect(':');
218 const std::string metadata_value = cursor.parse_string();
219 manifest.metadata.emplace_back(metadata_key, metadata_value);
220 if (cursor.consume('}')) {
221 return;
222 }
223 cursor.expect(',');
224 }
225 }
226 throw std::runtime_error("unexpected safetensors metadata value");
227 }

```



```

227 SafeTensorEntry parse_tensor_entry(HeaderCursor& cursor, const std::string& ↵
 ↵ name) {
228 SafeTensorEntry entry;
229 entry.name = name;
230 bool saw_dtype = false;
231 bool saw_shape = false;
232 bool saw_offsets = false;
233
234 cursor.expect('{');
235 if (cursor.consume('}')) {
236 throw std::runtime_error("empty safetensors tensor entry");
237 }
238 while (true) {
239 const std::string field = cursor.parse_string();
240 cursor.expect(':');
241 if (field == "dtype") {
242 entry.dtype = cursor.parse_string();
243 saw_dtype = true;
244 } else if (field == "shape") {
245 entry.shape = cursor.parse_i64_array();
246 saw_shape = true;
247 } else if (field == "data_offsets") {
248 const std::vector<std::uint64_t> offsets = cursor.parse_u64_array()↵
 ↵ ;
249 if (offsets.size() != LCQI_SAFETENSORS_OFFSET_COUNT) {
250 throw std::runtime_error("safetensors data_offsets must contain↵
 ↵ two values");
251 }
252 entry.data_begin = offsets[0];
253 entry.data_end = offsets[1];
254 saw_offsets = true;
255 } else {
256 throw std::runtime_error("unsupported safetensors tensor field: " +↵
 ↵ field);
257 }
258 if (cursor.consume('}')) {
259 break;
260 }
261 cursor.expect(',');
262 }
263
264 if (!saw_dtype || !saw_shape || !saw_offsets) {
265 throw std::runtime_error("safetensors tensor entry is missing required ↵
 ↵ fields");
266 }
267 if (entry.data_end < entry.data_begin) {
268 throw std::runtime_error("safetensors tensor offset range is invalid");
269 }
270 return entry;
271 }
272
273 void parse_header(std::string_view header, SafeTensorManifest& manifest) {

```

```

274 HeaderCursor cursor(header);
275 cursor.expect('{');
276 if (cursor.consume('}')) {
277 throw std::runtime_error("safetensors header is empty");
278 }
279
280 while (true) {
281 const std::string key = cursor.parse_string();
282 cursor.expect(':');
283 if (key == "__metadata__") {
284 parse_metadata_value(cursor, manifest, key);
285 } else {
286 manifest.tensors.push_back(parse_tensor_entry(cursor, key));
287 }
288 if (cursor.consume('}')) {
289 break;
290 }
291 cursor.expect(',');
292 }
293
294 if (!cursor.eof_after_ws()) {
295 throw std::runtime_error("trailing data after safetensors header");
296 }
297 }
298
299 std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry);
300
301 void validate_manifest(const SafeTensorManifest& manifest,
302 std::uint64_t file_size) {
303 const std::uint64_t payload_size = file_size - manifest.data_start_offset;
304 for (std::size_t i = 0; i < manifest.tensors.size(); ++i) {
305 const SafeTensorEntry& left = manifest.tensors[i];
306 if (left.name.empty() || left.dtype.empty()) {
307 throw std::runtime_error("safetensors tensor has empty name or ↵
↵ dtype");
308 }
309 if (left.data_end > payload_size) {
310 throw std::runtime_error("safetensors tensor data_offsets exceed ↵
↵ payload size");
311 }
312 const std::uint64_t expected_bytes = expected_tensor_bytes(left);
313 if (left.byte_size() != expected_bytes) {
314 throw std::runtime_error("safetensors tensor byte size does not ↵
↵ match dtype and shape");
315 }
316 for (const std::int64_t dim : left.shape) {
317 if (dim < 0) {
318 throw std::runtime_error("safetensors shape dimension is ↵
↵ negative");
319 }
320 }
321 for (std::size_t j = i + 1; j < manifest.tensors.size(); ++j) {

```

```

322 const SafeTensorEntry& right = manifest.tensors[j];
323 if (left.name == right.name) {
324 throw std::runtime_error("duplicate safetensors tensor name");
325 }
326 const bool overlaps =
327 left.data_begin < right.data_end && right.data_begin < left.↵
↵ data_end;
328 if (overlaps) {
329 throw std::runtime_error("overlapping safetensors tensor data ↵
↵ range");
330 }
331 }
332 }
333 }
334
335 std::uint64_t dtype_byte_size(std::string_view dtype) {
336 if (dtype == "F64") {
337 return LCQI_DTYPE_BYTES_F64;
338 }
339 if (dtype == "F32") {
340 return LCQI_DTYPE_BYTES_F32;
341 }
342 if (dtype == "F16" || dtype == "BF16") {
343 return LCQI_DTYPE_BYTES_F16;
344 }
345 if (dtype == "I64" || dtype == "U64") {
346 return LCQI_DTYPE_BYTES_I64;
347 }
348 if (dtype == "I32" || dtype == "U32") {
349 return LCQI_DTYPE_BYTES_I32;
350 }
351 if (dtype == "I16" || dtype == "U16") {
352 return LCQI_DTYPE_BYTES_I16;
353 }
354 if (dtype == "I8") {
355 return LCQI_DTYPE_BYTES_I8;
356 }
357 if (dtype == "U8" || dtype == "BOOL") {
358 return LCQI_DTYPE_BYTES_U8;
359 }
360 throw std::runtime_error("unsupported safetensors dtype: " + std::string(↵
↵ dtype));
361 }
362
363 std::uint64_t expected_tensor_bytes(const SafeTensorEntry& entry) {
364 std::uint64_t element_count = 1;
365 for (const std::int64_t dim : entry.shape) {
366 if (dim < 0) {
367 throw std::runtime_error("safetensors shape dimension is negative")↵
↵ ;
368 }
369 const std::uint64_t extent = static_cast<std::uint64_t>(dim);

```

```

370 if (extent != 0 &&
371 element_count > std::numeric_limits<std::uint64_t>::max() / extent)↵
↵ {
372 throw std::runtime_error("safetensors shape element count overflow"↵
↵);
373 }
374 element_count *= extent;
375 }
376 const std::uint64_t dtype_bytes = dtype_byte_size(entry.dtype);
377 if (dtype_bytes != 0 &&
378 element_count > std::numeric_limits<std::uint64_t>::max() / dtype_bytes)↵
↵) {
379 throw std::runtime_error("safetensors tensor byte size overflow");
380 }
381 return element_count * dtype_bytes;
382 }
383
384 } // namespace
385
386 std::uint64_t SafeTensorEntry::byte_size() const {
387 return this->data_end - this->data_begin;
388 }
389
390 const SafeTensorEntry* SafeTensorManifest::find_tensor(std::string_view name) ↵
↵ const {
391 const auto found = std::find_if(
392 this->tensors.begin(),
393 this->tensors.end(),
394 [name](const SafeTensorEntry& entry) {
395 return entry.name == name;
396 });
397 if (found == this->tensors.end()) {
398 return nullptr;
399 }
400 return &(*found);
401 }
402
403 SafeTensorManifest load_safetensors_manifest(const std::filesystem::path& path)↵
↵ {
404 const std::string bytes = read_file_bytes(path);
405 const std::uint64_t header_size = read_le_u64(bytes);
406 if (header_size == 0 || header_size > LCQI_SAFETENSORS_MAX_HEADER_BYTES) {
407 throw std::runtime_error("safetensors header size is invalid");
408 }
409 const std::uint64_t data_start_offset =
410 static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) + header_size↵
↵ ;
411 if (data_start_offset > bytes.size()) {
412 throw std::runtime_error("safetensors header extends beyond file size")↵
↵ ;
413 }
414

```

```

415 SafeTensorManifest manifest;
416 manifest.header_size = header_size;
417 manifest.data_start_offset = data_start_offset;
418 const std::string_view header(
419 bytes.data() + LCQI_SAFETENSORS_PREFIX_BYTES,
420 static_cast<std::size_t>(header_size));
421 parse_header(header, manifest);
422 validate_manifest(manifest, static_cast<std::uint64_t>(bytes.size()));
423 return manifest;
424 }
425
426 } // namespace lcqi

```

## B.17 十七、Benchmark 程序

`bench.cpp` 输出 int8 kernel benchmark。它服务性能观察，但必须和 max diff 对照一起看。

Listing B.17 src/bench.cpp

```

1 #include <lcqi/int8_kernels.hpp>
2
3 #include <algorithm>
4 #include <cstdlib>
5 #include <iostream>
6 #include <stdexcept>
7 #include <string>
8 #include <vector>
9
10 namespace {
11
12 constexpr const char* LCQI_TARGET_ARCH_X86_64 = "x86_64";
13 constexpr const char* LCQI_TARGET_ARCH_I386 = "i386";
14 constexpr const char* LCQI_TARGET_ARCH_UNKNOWN = "unknown";
15 constexpr std::int32_t LCQI_DEFAULT_REPEAT = 200;
16 constexpr int LCQI_REPEAT_ARG_INDEX = 1;
17 constexpr std::int32_t LCQI_LARGE_CASE_REPEAT_DIVISOR = 4;
18 constexpr std::int32_t LCQI_MIN_LARGE_CASE_REPEAT = 1;
19 constexpr std::int32_t LCQI_DECODE_SHAPE_SIZE = 4096;
20
21 const char* target_arch() noexcept {
22 #if defined(__x86_64__) || defined(_M_X64)
23 return LCQI_TARGET_ARCH_X86_64;
24 #elif defined(__i386__) || defined(_M_IX86)
25 return LCQI_TARGET_ARCH_I386;
26 #else
27 return LCQI_TARGET_ARCH_UNKNOWN;
28 #endif
29 }
30
31 std::int32_t parse_repeat(int argc, char** argv) {
32 if (argc <= LCQI_REPEAT_ARG_INDEX) {
33 return LCQI_DEFAULT_REPEAT;

```

```

34 }
35 const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
↵ LCQI_REPEAT_ARG_INDEX]));
36 if (repeat <= 0) {
37 throw std::runtime_error("repeat must be positive");
38 }
39 return repeat;
40 }
41
42 std::vector<lcqi::KernelBenchmarkCase> make_cases(std::int32_t repeat) {
43 return {
44 {128, 128, 16, repeat},
45 {256, 256, 16, repeat},
46 {512, 512, 16, repeat},
47 {513, 257, 16, repeat},
48 {1024, 4096, 16, std::max(LCQI_MIN_LARGE_CASE_REPEAT,
49 repeat / LCQI_LARGE_CASE_REPEAT_DIVISOR)},
50 {LCQI_DECODE_SHAPE_SIZE, LCQI_DECODE_SHAPE_SIZE, 16,
51 std::max(LCQI_MIN_LARGE_CASE_REPEAT, repeat / ↵
↵ LCQI_LARGE_CASE_REPEAT_DIVISOR)},
52 };
53 }
54
55 } // namespace
56
57 int main(int argc, char** argv) {
58 try {
59 const std::int32_t repeat = parse_repeat(argc, argv);
60 std::cout
61 << "target_arch,input_size,output_size,output_block,repeat,backend,↵
↵ available,"
62 << "average_us,max_abs_diff,checksum\n";
63 for (const lcqi::KernelBenchmarkCase& benchmark_case : make_cases(↵
↵ repeat)) {
64 const lcqi::KernelBenchmarkResult result =
65 lcqi::benchmark_linear_i8_case(benchmark_case);
66 for (const lcqi::KernelBackendBenchmark& backend : result.backends)↵
↵ {
67 std::cout << target_arch() << ','
68 << result.benchmark_case.input_size << ','
69 << result.benchmark_case.output_size << ','
70 << result.benchmark_case.output_block_size << ','
71 << result.benchmark_case.repeat << ','
72 << backend.name << ','
73 << (backend.available ? 1 : 0) << ','
74 << backend.average_us << ','
75 << backend.max_abs_diff << ','
76 << backend.checksum << '\n';
77 }
78 }
79 return EXIT_SUCCESS;
80 } catch (const std::exception& error) {

```

```

81 std::cerr << "error: " << error.what() << '\n';
82 return EXIT_FAILURE;
83 }
84 }

```

## B.18 十八、Trace 程序

`trace.cpp` 输出 reference decoder checkpoint。它用于定位错误边界，不应该被简化成只打印最终 token。

Listing B.18 src/trace.cpp

```

1 #include <lcqi/reference_decoder.hpp>
2 #include <lcqi/tokenizer.hpp>
3
4 #include <algorithm>
5 #include <array>
6 #include <cstdlib>
7 #include <exception>
8 #include <filesystem>
9 #include <iostream>
10 #include
11 #include <sstream>
12 #include <string>
13 #include <string_view>
14
15 namespace {
16
17 constexpr std::array<std::int32_t, 3> LCQI_TRACE_TOKEN_IDS{1, 2, 3};
18 constexpr std::string_view LCQI_TRACE_PROMPT = "tok1 tok2 tok3";
19 constexpr std::int32_t LCQI_TRACE_METADATA_LAYER = -1;
20 constexpr std::int32_t LCQI_TRACE_BOS_ID = 0;
21 constexpr std::int32_t LCQI_TRACE_EOS_ID = 4;
22 constexpr std::size_t LCQI_TRACE_VALUE_LIMIT = 8;
23
24 std::filesystem::path tokenizer_fixture_path() {
25 return std::filesystem::path(__FILE__).parent_path().parent_path() /
26 "models" / "tiny_tokenizer.txt";
27 }
28
29 std::vector<std::int32_t> decoder_tokens_from_prompt(const lcqi::TokenizerModel& ←
 ↪ & tokenizer,
30 std::string_view prompt) {
31 std::vector<std::int32_t> full_tokens = lcqi::encode_prompt(tokenizer, ←
 ↪ prompt);
32 if (full_tokens.size() != LCQI_TRACE_TOKEN_IDS.size() + 2 ||
33 full_tokens.front() != LCQI_TRACE_BOS_ID ||
34 full_tokens.back() != LCQI_TRACE_EOS_ID) {
35 throw std::runtime_error("unexpected tokenizer fixture ids");
36 }
37
38 std::vector<std::int32_t> decoder_tokens(

```



```

39 full_tokens.begin() + 1,
40 full_tokens.end() - 1);
41 if (!std::equal(decoder_tokens.begin(),
42 decoder_tokens.end(),
43 LCQI_TRACE_TOKEN_IDS.begin(),
44 LCQI_TRACE_TOKEN_IDS.end())) {
45 throw std::runtime_error("unexpected decoder token ids");
46 }
47 return decoder_tokens;
48 }
49
50 std::string join_ints(std::span<const std::int32_t> values) {
51 std::ostringstream stream;
52 for (std::size_t index = 0; index < values.size(); ++index) {
53 if (index != 0) {
54 stream << 'x';
55 }
56 stream << values[index];
57 }
58 return stream.str();
59 }
60
61 std::string join_int_values(std::span<const std::int32_t> values) {
62 std::ostringstream stream;
63 for (std::size_t index = 0; index < values.size(); ++index) {
64 if (index != 0) {
65 stream << ';';
66 }
67 stream << values[index];
68 }
69 return stream.str();
70 }
71
72 std::int32_t checksum_ints(std::span<const std::int32_t> values) {
73 std::int32_t result = 0;
74 for (const std::int32_t value : values) {
75 result += value;
76 }
77 return result;
78 }
79
80 std::int32_t max_abs_ints(std::span<const std::int32_t> values) {
81 std::int32_t result = 0;
82 for (const std::int32_t value : values) {
83 result = std::max(result, std::abs(value));
84 }
85 return result;
86 }
87
88 std::string join_floats(std::span<const float> values) {
89 std::ostringstream stream;
90 const std::size_t count = std::min(values.size(), LCQI_TRACE_VALUE_LIMIT);

```

```

91 for (std::size_t index = 0; index < count; ++index) {
92 if (index != 0) {
93 stream << ',';
94 }
95 stream << values[index];
96 }
97 if (values.size() > LCQI_TRACE_VALUE_LIMIT) {
98 stream << "...";
99 }
100 return stream.str();
101 }
102
103 void print_trace_csv(const lcqi::ReferenceDecodeTraceResult& trace,
104 const lcqi::TokenizerModel& tokenizer,
105 std::span<const std::int32_t> full_tokens) {
106 std::cout << "name,token_position,layer_id,shape,stride,dtype,layout,↵
↵ checksum,max_abs,values\n";
107 std::cout << "prompt,0," << LCQI_TRACE_METADATA_LAYER
108 << ",scalar,scalar,utf8,text,0,0," << LCQI_TRACE_PROMPT << '\n';
109 std::cout << "tokenizer,0," << LCQI_TRACE_METADATA_LAYER
110 << ',' << full_tokens.size() << ",1,i32,contiguous,"
111 << checksum_ints(full_tokens) << ','
112 << max_abs_ints(full_tokens) << ','
113 << join_int_values(full_tokens) << '\n';
114 std::cout << "tokenizer_contract,0," << LCQI_TRACE_METADATA_LAYER
115 << ",scalar,scalar,u64,fnv1a,"
116 << tokenizer_contract_hash(tokenizer) << ",0,"
117 << tokenizer.chat_template_hash << '\n';
118 for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints)↵
↵ {
119 std::cout << checkpoint.name << ','
120 << checkpoint.token_position << ','
121 << checkpoint.layer_id << ','
122 << join_ints(checkpoint.shape) << ','
123 << join_ints(checkpoint.stride) << ','
124 << checkpoint.dtype << ','
125 << checkpoint.layout << ','
126 << checkpoint.checksum << ','
127 << checkpoint.max_abs << ','
128 << join_floats(checkpoint.values) << '\n';
129 }
130 std::cout << "sampler_argmax,"
131 << LCQI_TRACE_TOKEN_IDS.size() - 1 << ','
132 << LCQI_TRACE_METADATA_LAYER
133 << ',' << trace.result.logits.size() << ",1,f32,argmax,"
134 << "0,0,token=" << trace.result.predicted_token << '\n';
135 std::cout << "generated_token,"
136 << LCQI_TRACE_TOKEN_IDS.size() << ','
137 << LCQI_TRACE_METADATA_LAYER
138 << ",scalar,scalar,i32,generated,"
139 << trace.result.predicted_token << ','
140 << trace.result.predicted_token << ','

```

```

141 << trace.result.predicted_token << '\n';
142 }
143
144 } // namespace
145
146 int main() {
147 try {
148 const lcqi::TokenizerModel tokenizer =
149 lcqi::load_tokenizer(tokenizer_fixture_path());
150 const std::vector<std::int32_t> full_tokens =
151 lcqi::encode_prompt(tokenizer, LCQI_TRACE_PROMPT);
152 const std::vector<std::int32_t> decoder_tokens =
153 decoder_tokens_from_prompt(tokenizer, LCQI_TRACE_PROMPT);
154 const lcqi::ReferenceDecoderModel model = lcqi::↵
↵ make_tiny_reference_decoder_model();
155 const lcqi::ReferenceDecodeTraceResult trace =
156 lcqi::run_reference_decode_with_trace(model, decoder_tokens);
157 print_trace_csv(trace, tokenizer, full_tokens);
158 return EXIT_SUCCESS;
159 } catch (const std::exception& error) {
160 std::cerr << "error: " << error.what() << '\n';
161 return EXIT_FAILURE;
162 }
163 }

```

## B.19 十九、Ledger 程序

`ledger.cpp` 输出 7B 规模账本。它是估算，不是实测吞吐；读者要区分 FLOPs、KV 容量和带宽约束。

**Listing B.19** src/ledger.cpp

```

1 #include <cstdlib>
2 #include <cstdint>
3 #include <exception>
4 #include <iomanip>
5 #include <iostream>
6 #include <string_view>
7
8 namespace {
9
10 constexpr double LCQI_BYTES_PER_GB = 1000.0 * 1000.0 * 1000.0;
11 constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
12 constexpr double LCQI_FLOPS_PER_MAC = 2.0;
13 constexpr std::int32_t LCQI_SEVENB_LAYER_COUNT = 32;
14 constexpr std::int32_t LCQI_SEVENB_HIDDEN_SIZE = 4096;
15 constexpr std::int32_t LCQI_SEVENB_QUERY_HEADS = 32;
16 constexpr std::int32_t LCQI_SEVENB_KV_HEADS = 8;
17 constexpr std::int32_t LCQI_SEVENB_HEAD_DIM = 128;
18 constexpr std::int32_t LCQI_SEVENB_INTERMEDIATE_SIZE = 11008;
19 constexpr std::int32_t LCQI_SEVENB_CONTEXT_LENGTH = 4096;
20

```

```

21 struct DTypeLedger {
22 std::string_view name;
23 double weight_gb_per_token = 0.0;
24 double kv_element_bytes = 0.0;
25 };
26
27 constexpr DTypeLedger LCQI_DTYPE_LEDGERS[] = {
28 {"fp16", 14.0, 2.0},
29 {"int8", 7.0, 2.0},
30 {"int4", 3.7, 2.0},
31 };
32
33 constexpr double LCQI_BANDWIDTH_GB_PER_S[] = {
34 50.0,
35 100.0,
36 200.0,
37 600.0,
38 1000.0,
39 1600.0,
40 };
41
42 double linear_macs_per_layer() noexcept {
43 const double hidden = LCQI_SEVENB_HIDDEN_SIZE;
44 const double kv_width = LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM;
45 const double qkv = hidden * (hidden + 2.0 * kv_width);
46 const double output_projection = hidden * hidden;
47 const double mlp = 3.0 * hidden * LCQI_SEVENB_INTERMEDIATE_SIZE;
48 return qkv + output_projection + mlp;
49 }
50
51 double attention_macs_per_layer() noexcept {
52 return 2.0 * LCQI_SEVENB_QUERY_HEADS * LCQI_SEVENB_CONTEXT_LENGTH *
53 LCQI_SEVENB_HEAD_DIM;
54 }
55
56 double kv_bytes_per_token_all_layers(double element_bytes) noexcept {
57 return static_cast<double>(LCQI_SEVENB_LAYER_COUNT) * 2.0 *
58 LCQI_SEVENB_KV_HEADS * LCQI_SEVENB_HEAD_DIM * element_bytes;
59 }
60
61 double kv_read_gb_per_decode_token(double element_bytes) noexcept {
62 const double bytes = static_cast<double>(LCQI_SEVENB_CONTEXT_LENGTH) *
63 kv_bytes_per_token_all_layers(element_bytes);
64 return bytes / LCQI_BYTES_PER_GB;
65 }
66
67 double kv_capacity_mib(double element_bytes, std::int32_t context_length) ↵
68 ↵ noexcept {
69 const double bytes = static_cast<double>(context_length) *
70 kv_bytes_per_token_all_layers(element_bytes);
71 return bytes / LCQI_BYTES_PER_MIB;
72 }

```

```

72
73 void print_model_rows() {
74 const double linear_macs = linear_macs_per_layer();
75 const double attention_macs = attention_macs_per_layer();
76 const double total_flops =
77 (linear_macs + attention_macs) * LCQI_SEVENB_LAYER_COUNT * ←
 ↪ LCQI_FLOPS_PER_MAC;
78 std::cout << "section,item,value,unit,notes\n";
79 std::cout << "model,layers," << LCQI_SEVENB_LAYER_COUNT << ",count,7B ←
 ↪ fixture\n";
80 std::cout << "model,hidden," << LCQI_SEVENB_HIDDEN_SIZE << ",elements,H\n";
81 std::cout << "model,query_heads," << LCQI_SEVENB_QUERY_HEADS << ",count,GQA ←
 ↪ query heads\n";
82 std::cout << "model,kv_heads," << LCQI_SEVENB_KV_HEADS << ",count,GQA KV ←
 ↪ heads\n";
83 std::cout << "model,head_dim," << LCQI_SEVENB_HEAD_DIM << ",elements,per ←
 ↪ head\n";
84 std::cout << "compute,linear_macs_per_layer," << linear_macs << ",macs, ←
 ↪ decode token\n";
85 std::cout << "compute,attention_macs_per_layer," << attention_macs
86 << ",macs,context=4096\n";
87 std::cout << "compute,total_decode_flops_per_token," << total_flops
88 << ",flops,linear plus attention\n";
89 }
90
91 void print_kv_rows() {
92 for (const std::int32_t context : {1024, 2048, 4096, 8192}) {
93 std::cout << "kv,fp16_capacity_context_" << context << ', '
94 << kv_capacity_mib(2.0, context)
95 << ",MiB,single session\n";
96 }
97 }
98
99 void print_dtype_rows() {
100 for (const DTypeLedger& dtype : LCQI_DTYPE_LEDGERS) {
101 const double kv_read_gb = kv_read_gb_per_decode_token(dtype. ←
 ↪ kv_element_bytes);
102 const double total_gb = dtype.weight_gb_per_token + kv_read_gb;
103 std::cout << "decode_bytes," << dtype.name << "_weight_gb,"
104 << dtype.weight_gb_per_token << ",GB/token,weight stream\n";
105 std::cout << "decode_bytes," << dtype.name << "_kv_read_gb,"
106 << kv_read_gb << ",GB/token,context=4096\n";
107 std::cout << "decode_bytes," << dtype.name << "_total_gb,"
108 << total_gb << ",GB/token,weight plus KV\n";
109 for (const double bandwidth : LCQI_BANDWIDTH_GB_PER_S) {
110 std::cout << "bandwidth_limit," << dtype.name << "_at_"
111 << static_cast<std::int32_t>(bandwidth) << "_gbps,"
112 << bandwidth / total_gb
113 << ",tokens/s,bandwidth-only upper bound\n";
114 }
115 }
116 }

```

```

117
118 } // namespace
119
120 int main() {
121 try {
122 std::cout << std::fixed << std::setprecision(3);
123 print_model_rows();
124 print_kv_rows();
125 print_dtype_rows();
126 return EXIT_SUCCESS;
127 } catch (const std::exception& error) {
128 std::cerr << "error: " << error.what() << '\n';
129 return EXIT_FAILURE;
130 }
131 }

```

## B.20 二十、Serving Probe 程序

`serving_probe.cpp` 固定短请求、RAG 请求、取消请求和超长请求。它把 TTFT、TPOT、拒绝和取消回收变成可观察字段。

**Listing B.20** `src/serving_probe.cpp`

```

1 #include <algorithm>
2 #include <cstdlib>
3 #include <stdint>
4 #include <exception>
5 #include <cmath>
6 #include <iomanip>
7 #include <iostream>
8 #include <string_view>
9 #include <vector>
10
11 namespace {
12
13 constexpr double LCQI_BYTES_PER_MIB = 1024.0 * 1024.0;
14 constexpr std::int32_t LCQI_PROBE_LAYER_COUNT = 32;
15 constexpr std::int32_t LCQI_PROBE_KV_HEADS = 8;
16 constexpr std::int32_t LCQI_PROBE_HEAD_DIM = 128;
17 constexpr double LCQI_PROBE_KV_ELEMENT_BYTES = 2.0;
18 constexpr double LCQI_PROBE_KV_BUDGET_MIB = 512.0;
19 constexpr double LCQI_PROBE_WORKSPACE_MIB = 64.0;
20 constexpr double LCQI_PROBE_ARENA_MIB = 32.0;
21 constexpr double LCQI_PROBE_TRACE_MIB = 4.0;
22 constexpr double LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS = 8.0;
23 constexpr double LCQI_PROBE_PREFILL_MS_PER_TOKEN = 0.08;
24 constexpr double LCQI_PROBE_DECODE_STEP_MS = 14.0;
25 constexpr double LCQI_PROBE_CANCEL_RECLAIM_MS = 2.0;
26 constexpr std::size_t LCQI_PERCENTILE_MIN_INDEX = 1;
27
28 struct ProbeRequest {
29 std::string_view request_id;

```

```

30 std::int32_t prompt_tokens = 0;
31 std::int32_t max_new_tokens = 0;
32 bool cancel_after_prefill = false;
33 };
34
35 struct ProbeResult {
36 std::string_view request_id;
37 std::int32_t accepted = 0;
38 std::string_view rejection_reason;
39 std::int32_t prompt_tokens = 0;
40 std::int32_t reserved_tokens = 0;
41 double kv_reserved_mib = 0.0;
42 double queue_wait_ms = 0.0;
43 double prefill_ms = 0.0;
44 double ttft_ms = 0.0;
45 double decode_step_p95_ms = 0.0;
46 double tpot_ms = 0.0;
47 std::int32_t completion_tokens = 0;
48 std::string_view finish_reason;
49 double cancel_reclaim_ms = 0.0;
50 };
51
52 const std::vector<ProbeRequest> LCQI_PROBE_REQUESTS{
53 {"req_short", 32, 4, false},
54 {"req_rag", 512, 8, false},
55 {"req_cancel", 128, 16, true},
56 {"req_too_long", 4096, 512, false},
57 };
58
59 double kv_mib_for_tokens(std::int32_t tokens) noexcept {
60 const double bytes = static_cast<double>(tokens) * LCQI_PROBE_LAYER_COUNT *
 ↪ 2.0 *
61 LCQI_PROBE_KV_HEADS * LCQI_PROBE_HEAD_DIM *
62 LCQI_PROBE_KV_ELEMENT_BYTES;
63 return bytes / LCQI_BYTES_PER_MIB;
64 }
65
66 double percentile(std::vector<double> values, double fraction) {
67 if (values.empty()) {
68 return 0.0;
69 }
70 std::sort(values.begin(), values.end());
71 const std::size_t max_index = values.size() - LCQI_PERCENTILE_MIN_INDEX;
72 const std::size_t index =
73 std::min(max_index,
74 static_cast<std::size_t>(
75 std::ceil(fraction * static_cast<double>(max_index))));
76 return values[index];
77 }
78
79 std::vector<ProbeResult> run_probe() {
80 std::vector<ProbeResult> results;

```



```

81 double reserved_kv_mib = 0.0;
82 std::int32_t accepted_before = 0;
83 for (const ProbeRequest& request : LCQI_PROBE_REQUESTS) {
84 ProbeResult result;
85 result.request_id = request.request_id;
86 result.prompt_tokens = request.prompt_tokens;
87 result.reserved_tokens = request.prompt_tokens + request.max_new_tokens;
88 result.kv_reserved_mib = kv_mib_for_tokens(result.reserved_tokens);
89 const double static_session_mib =
90 result.kv_reserved_mib + LCQI_PROBE_WORKSPACE_MIB +
91 LCQI_PROBE_ARENA_MIB + LCQI_PROBE_TRACE_MIB;
92 if (reserved_kv_mib + result.kv_reserved_mib > LCQI_PROBE_KV_BUDGET_MIB)
93 {
94 result.accepted = 0;
95 result.rejection_reason = "memory_budget_exceeded";
96 result.finish_reason = "rejected";
97 results.push_back(result);
98 continue;
99 }
100 reserved_kv_mib += result.kv_reserved_mib;
101 result.accepted = 1;
102 result.rejection_reason = "none";
103 result.queue_wait_ms = static_cast<double>(accepted_before) *
104 LCQI_PROBE_QUEUE_WAIT_PER_REQUEST_MS;
105 result.prefill_ms = static_cast<double>(request.prompt_tokens) *
106 LCQI_PROBE_PREFILL_MS_PER_TOKEN;
107 result.ttft_ms = result.queue_wait_ms + result.prefill_ms +
108 LCQI_PROBE_DECODE_STEP_MS;
109 result.decode_step_p95_ms = LCQI_PROBE_DECODE_STEP_MS +
110 static_session_mib / 512.0;
111 result.tpot_ms = result.decode_step_p95_ms;
112 if (request.cancel_after_prefill) {
113 result.completion_tokens = 0;
114 result.finish_reason = "cancelled";
115 result.cancel_reclaim_ms = LCQI_PROBE_CANCEL_RECLAIM_MS;
116 reserved_kv_mib -= result.kv_reserved_mib;
117 } else {
118 result.completion_tokens = request.max_new_tokens;
119 result.finish_reason = "max_tokens";
120 }
121 ++accepted_before;
122 results.push_back(result);
123 }
124 return results;
125 }
126 void print_request_rows(const std::vector<ProbeResult>& results) {
127 std::cout << "row_type,request_id,accepted,rejection_reason,prompt_tokens,"
128 << "reserved_tokens,"
129 << "kv_reserved_mib,queue_wait_ms,prefill_ms,ttft_ms,"

```

```

129 decode_step_p95_ms,"
 "tpot_ms,completion_tokens,finish_reason,cancel_reclaim_ms,←
130 metric_name,"
 "metric_value\n";
131 for (const ProbeResult& result : results) {
132 std::cout << "request," << result.request_id << ','
133 << result.accepted << ','
134 << result.rejection_reason << ','
135 << result.prompt_tokens << ','
136 << result.reserved_tokens << ','
137 << result.kv_reserved_mib << ','
138 << result.queue_wait_ms << ','
139 << result.prefill_ms << ','
140 << result.ttft_ms << ','
141 << result.decode_step_p95_ms << ','
142 << result.tpot_ms << ','
143 << result.completion_tokens << ','
144 << result.finish_reason << ','
145 << result.cancel_reclaim_ms << ",,\n";
146 }
147 }
148
149 void print_summary_row(const std::vector<ProbeResult>& results) {
150 std::vector<double> ttft_values;
151 std::vector<double> queue_values;
152 double accepted = 0.0;
153 double rejected = 0.0;
154 double cancelled = 0.0;
155 double reserved_peak_mib = 0.0;
156 double running_reserved_mib = 0.0;
157 for (const ProbeResult& result : results) {
158 if (result.accepted == 0) {
159 rejected += 1.0;
160 continue;
161 }
162 accepted += 1.0;
163 ttft_values.push_back(result.ttft_ms);
164 queue_values.push_back(result.queue_wait_ms);
165 running_reserved_mib += result.kv_reserved_mib;
166 reserved_peak_mib = std::max(reserved_peak_mib, running_reserved_mib);
167 if (result.finish_reason == "cancelled") {
168 cancelled += 1.0;
169 running_reserved_mib -= result.kv_reserved_mib;
170 }
171 }
172 const double total = accepted + rejected;
173 const double rejection_rate = total == 0.0 ? 0.0 : rejected / total;
174 std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,0,"
175 << "accepted_count," << accepted << '\n';
176 std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,0,summary,"
177 << LCQI_PROBE_CANCEL_RECLAIM_MS << ",cancel_count," << cancelled << ←
 << '\n';

```

```

178 std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,summary,0,"
179 << "rejected_count," << rejected << '\n';
180 std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,summary,0,"
181 << "rejection_rate," << rejection_rate << '\n';
182 std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,summary,0,"
183 << "kv_reserved_peak_mib," << reserved_peak_mib << '\n';
184 std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,summary,0,"
185 << "queue_wait_p95_ms," << percentile(queue_values, 0.95) << '\n'↵
↵ ;
186 std::cout << "summary,all,0,summary,0,0,0,0,0,0,0,0,summary,0,"
187 << "ttft_p95_ms," << percentile(ttft_values, 0.95) << '\n';
188 }
189
190 } // namespace
191
192 int main() {
193 try {
194 std::cout << std::fixed << std::setprecision(3);
195 const std::vector<ProbeResult> results = run_probe();
196 print_request_rows(results);
197 print_summary_row(results);
198 return EXIT_SUCCESS;
199 } catch (const std::exception& error) {
200 std::cerr << "error: " << error.what() << '\n';
201 return EXIT_FAILURE;
202 }
203 }

```

## B.21 二十一、oneDNN 对标程序

`onednn_bench.cpp` 是可选外部库对标入口。它只在环境安装 oneDNN 时构建，不能把不可用平台写成已验证性能。

Listing B.21 src/onednn\_bench.cpp

```

1 #include <dnnl.hpp>
2
3 #include <chrono>
4 #include <cstdint>
5 #include <cstdlib>
6 #include <iostream>
7 #include <numeric>
8 #include <stdexcept>
9 #include <string>
10 #include <unordered_map>
11 #include <vector>
12
13 namespace {
14
15 constexpr std::int64_t LCQI_ONEDNN_BATCH = 1;
16 constexpr std::int64_t LCQI_ONEDNN_K = 4096;
17 constexpr std::int64_t LCQI_ONEDNN_O = 4096;

```

```

18 constexpr std::int32_t LCQI_ONEDNN_DEFAULT_REPEAT = 20;
19 constexpr std::int32_t LCQI_ONEDNN_WARMUP = 3;
20 constexpr int LCQI_REPEAT_ARG_INDEX = 1;
21 constexpr double LCQI_SECONDS_TO_MICROSECONDS = 1000000.0;
22 constexpr float LCQI_INPUT_SCALE = 0.125F;
23 constexpr std::int32_t LCQI_INPUT_MODULUS = 17;
24 constexpr std::int32_t LCQI_INPUT_CENTER = 8;
25 constexpr std::int32_t LCQI_WEIGHT_MODULUS = 23;
26 constexpr std::int32_t LCQI_WEIGHT_CENTER = 11;
27 constexpr float LCQI_WEIGHT_SCALE = 0.03125F;
28
29 std::int32_t parse_repeat(int argc, char** argv) {
30 if (argc <= LCQI_REPEAT_ARG_INDEX) {
31 return LCQI_ONEDNN_DEFAULT_REPEAT;
32 }
33 const std::int32_t repeat = static_cast<std::int32_t>(std::stoi(argv[↵
↵ LCQI_REPEAT_ARG_INDEX]));
34 if (repeat <= 0) {
35 throw std::runtime_error("repeat must be positive");
36 }
37 return repeat;
38 }
39
40 std::vector<float> make_input() {
41 std::vector<float> input(static_cast<std::size_t>(LCQI_ONEDNN_K));
42 for (std::int64_t index = 0; index < LCQI_ONEDNN_K; ++index) {
43 input[static_cast<std::size_t>(index)] =
44 static_cast<float>((index % LCQI_INPUT_MODULUS) - LCQI_INPUT_CENTER↵
↵) *
45 LCQI_INPUT_SCALE;
46 }
47 return input;
48 }
49
50 std::vector<float> make_weights_kn() {
51 std::vector<float> weights(static_cast<std::size_t>(LCQI_ONEDNN_K * ↵
↵ LCQI_ONEDNN_0));
52 for (std::int64_t k = 0; k < LCQI_ONEDNN_K; ++k) {
53 for (std::int64_t out = 0; out < LCQI_ONEDNN_0; ++out) {
54 const std::int64_t row_major_out_k = out * LCQI_ONEDNN_K + k;
55 const float value =
56 static_cast<float>((row_major_out_k % LCQI_WEIGHT_MODULUS) -
57 LCQI_WEIGHT_CENTER) *
58 LCQI_WEIGHT_SCALE;
59 weights[static_cast<std::size_t>(k * LCQI_ONEDNN_0 + out)] = value;
60 }
61 }
62 return weights;
63 }
64
65 float checksum(const std::vector<float>& values) {
66 return std::accumulate(values.begin(), values.end(), 0.0F);

```

```

67 }
68
69 } // namespace
70
71 int main(int argc, char** argv) {
72 try {
73 const std::int32_t repeat = parse_repeat(argc, argv);
74 std::vector<float> input = make_input();
75 std::vector<float> weights = make_weights_kn();
76 std::vector<float> output(static_cast<std::size_t>(LCQI_ONEDNN_BATCH * ↵
↵ LCQI_ONEDNN_O),
77 0.0F);
78
79 dnnl::engine engine(dnnl::engine::kind::cpu, 0);
80 dnnl::stream stream(engine);
81
82 const dnnl::memory::dims source_dims = {LCQI_ONEDNN_BATCH, ↵
↵ LCQI_ONEDNN_K};
83 const dnnl::memory::dims weight_dims = {LCQI_ONEDNN_K, LCQI_ONEDNN_O};
84 const dnnl::memory::dims output_dims = {LCQI_ONEDNN_BATCH, ↵
↵ LCQI_ONEDNN_O};
85
86 const dnnl::memory::desc source_desc(
87 source_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↵
↵ ::ab);
88 const dnnl::memory::desc weight_desc(
89 weight_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↵
↵ ::ab);
90 const dnnl::memory::desc output_desc(
91 output_dims, dnnl::memory::data_type::f32, dnnl::memory::format_tag↵
↵ ::ab);
92
93 dnnl::memory source_memory(source_desc, engine, input.data());
94 dnnl::memory weight_memory(weight_desc, engine, weights.data());
95 dnnl::memory output_memory(output_desc, engine, output.data());
96
97 dnnl::matmul::primitive_desc matmul_desc(engine, source_desc, ↵
↵ weight_desc, output_desc);
98 dnnl::matmul matmul(matmul_desc);
99 const std::unordered_map<int, dnnl::memory> arguments = {
100 {DNNL_ARG_SRC, source_memory},
101 {DNNL_ARG_WEIGHTS, weight_memory},
102 {DNNL_ARG_DST, output_memory},
103 };
104
105 for (std::int32_t index = 0; index < LCQI_ONEDNN_WARMUP; ++index) {
106 matmul.execute(stream, arguments);
107 stream.wait();
108 }
109
110 const auto begin = std::chrono::steady_clock::now();
111 for (std::int32_t index = 0; index < repeat; ++index) {

```

```

112 matmul.execute(stream, arguments);
113 stream.wait();
114 }
115 const auto end = std::chrono::steady_clock::now();
116 const std::chrono::duration<double> elapsed = end - begin;
117 const double average_us =
118 elapsed.count() * LCQI_SECONDS_TO_MICROSECONDS / static_cast<double>
119 ↪ >(repeat);
120
121 std::cout << "library,input_size,output_size,dtype,repeat,average_us,
122 ↪ checksum\n";
123 std::cout << "onednn," << LCQI_ONEDNN_K << ',' << LCQI_ONEDNN_0
124 << ",f32," << repeat << ',' << average_us << ','
125 << checksum(output) << '\n';
126 return EXIT_SUCCESS;
127 } catch (const std::exception& error) {
128 std::cerr << "error: " << error.what() << '\n';
129 return EXIT_FAILURE;
130 }

```

## B.22 二十二、Tiny MLP Fixture

`tiny_mlp_i8.txt` 是 tiny MLP 的模型资产。它决定 shape、权重、scale 和 bias，测试中的 logits golden 依赖它。

**Listing B.22** models/tiny\_mlp\_i8.txt

```

1 LCQI_MODEL_V1
2 input_size 4
3 hidden_size 3
4 output_size 2
5 hidden_scale 0.1
6 hidden_weights
7 5 -3 2 1
8 -4 6 1 -2
9 3 2 -5 4
10 hidden_bias
11 0.1 -0.2 0.0
12 output_scale 0.05
13 output_weights
14 4 -6 2
15 -3 5 8
16 output_bias
17 -0.5 0.4

```

## B.23 二十三、Tiny Tokenizer Fixture

`tiny_tokenizer.txt` 决定词表、BOS/EOS/UNK 和模板 hash。prompt 变 token 的合同从这里开始。

**Listing B.23** models/tiny\_tokenizer.txt

```

1 LCQI_TOKENIZER_V1
2 bos_id 0
3 eos_id 4
4 unk_id -1
5 chat_template_hash tiny-chat-template-v1
6 vocab_size 5
7 vocab
8 0 <bos>
9 1 tok1
10 2 tok2
11 3 tok3
12 4 <eos>

```

## B.24 二十四、完整测试

测试文件把 safetensors、tokenizer、tiny MLP、int8 kernel、reference decoder、trace、ledger 和 serving probe 串成验收网。新增优化或 shape 时,必须先扩展这里的 golden 和边界样例。

Listing B.24 tests/lcqi\_tests.cpp

```

1 #include <lcqi/inference.hpp>
2 #include <lcqi/int8_kernels.hpp>
3 #include <lcqi/model.hpp>
4 #include <lcqi/reference_decoder.hpp>
5 #include <lcqi/safetensors.hpp>
6 #include <lcqi/tokenizer.hpp>
7
8 #include <array>
9 #include <cmath>
10 #include <cstdlib>
11 #include <filesystem>
12 #include <fstream>
13 #include <iostream>
14 #include
15 #include <stdexcept>
16 #include <string>
17 #include <vector>
18
19 namespace {
20
21 constexpr float LCQI_TEST_EPSILON = 1.0e-5F;
22 constexpr std::int32_t LCQI_TEST_INPUT_SIZE = 4;
23 constexpr std::int32_t LCQI_TEST_HIDDEN_SIZE = 3;
24 constexpr std::int32_t LCQI_TEST_OUTPUT_SIZE = 2;
25 constexpr std::int32_t LCQI_TEST_PREDICTED_CLASS = 1;
26 constexpr float LCQI_TEST_LOGIT_0 = -0.48F;
27 constexpr float LCQI_TEST_LOGIT_1 = 1.06F;
28 constexpr float LCQI_TEST_HIDDEN_0 = 0.0F;
29 constexpr float LCQI_TEST_HIDDEN_1 = 0.4F;
30 constexpr float LCQI_TEST_HIDDEN_2 = 1.4F;
31 constexpr float LCQI_RMS_EXPECTED_0 = 0.848528F;

```



```

32 constexpr float LCQI_RMS_EXPECTED_1 = 0.565685F;
33 constexpr float LCQI_ATTENTION_EXPECTED_0 = 2.0F;
34 constexpr float LCQI_ATTENTION_EXPECTED_1 = 3.0F;
35 constexpr float LCQI_KERNEL_MAX_DIFF = 1.0e-5F;
36 constexpr std::int32_t LCQI_SIMD_TEST_BLOCK = 8;
37 constexpr std::int32_t LCQI_REFERENCE_DECODER_PREDICTED_TOKEN = 0;
38 constexpr std::size_t LCQI_REFERENCE_DECODER_VOCAB_SIZE = 5;
39 constexpr std::array<float, LCQI_REFERENCE_DECODER_VOCAB_SIZE> ↵
 ↵ LCQI_REFERENCE_DECODER_LOGITS{
40 0.352516979F,
41 -0.126884326F,
42 0.0797837451F,
43 -0.29797405F,
44 0.127030417F,
45 };
46 constexpr std::int32_t LCQI_REFERENCE_TRACE_TOKEN_COUNT = 3;
47 constexpr std::int32_t LCQI_REFERENCE_TRACE_HIDDEN_SIZE = 4;
48 constexpr std::int32_t LCQI_REFERENCE_TRACE_FIRST_TOKEN = 0;
49 constexpr std::uint64_t LCQI_TOKENIZER_HASH_EXPECTED = 13452902845388333734ULL;
50 constexpr std::int32_t LCQI_SAFETENSORS_PREFIX_BYTES = 8;
51 constexpr std::int32_t LCQI_TEST_BYTE_BITS = 8;
52 constexpr std::uint64_t LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES = 48;
53 constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_BEGIN = 0;
54 constexpr std::uint64_t LCQI_SAFETENSORS_EMBED_END = 24;
55 constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_BEGIN = 24;
56 constexpr std::uint64_t LCQI_SAFETENSORS_QPROJ_END = 48;
57 constexpr unsigned int LCQI_BYTE_MASK = 0xFFU;
58
59 struct KernelShapeCase {
60 std::int32_t input_size = 0;
61 std::int32_t output_size = 0;
62 std::int32_t output_block_size = 0;
63 };
64
65 constexpr std::array<KernelShapeCase, 4> LCQI_KERNEL_SHAPE_CASES{{
66 {17, 19, 8},
67 {33, 7, 8},
68 {64, 32, 16},
69 {65, 31, 16},
70 }};
71
72 void require(bool condition, const char* message) {
73 if (!condition) {
74 throw std::runtime_error(message);
75 }
76 }
77
78 void require_close(float actual, float expected, const char* message) {
79 if (std::fabs(actual - expected) > LCQI_TEST_EPSILON) {
80 throw std::runtime_error(message);
81 }
82 }

```

```

83
84 std::filesystem::path test_model_path() {
85 return std::filesystem::path(__FILE__).parent_path().parent_path() /
86 "models" / "tiny_mlp_i8.txt";
87 }
88
89 std::filesystem::path test_tokenizer_path() {
90 return std::filesystem::path(__FILE__).parent_path().parent_path() /
91 "models" / "tiny_tokenizer.txt";
92 }
93
94 std::filesystem::path temp_file_path(const std::string& name) {
95 return std::filesystem::temp_directory_path() / name;
96 }
97
98 void write_le_u64(std::ofstream& output, std::uint64_t value) {
99 for (std::int32_t i = 0; i < LCQI_SAFETENSORS_PREFIX_BYTES; ++i) {
100 const char byte = static_cast<char>(
101 (value >> (LCQI_TEST_BYTE_BITS * i)) & LCQI_BYTE_MASK);
102 output.write(&byte, 1);
103 }
104 }
105
106 void write_safetensors_fixture(const std::filesystem::path& path,
107 const std::string& header,
108 std::uint64_t payload_bytes) {
109 std::ofstream output(path, std::ios::binary);
110 if (!output) {
111 throw std::runtime_error("cannot create safetensors test fixture");
112 }
113 write_le_u64(output, static_cast<std::uint64_t>(header.size()));
114 output.write(header.data(), static_cast<std::streamsize>(header.size()));
115 for (std::uint64_t i = 0; i < payload_bytes; ++i) {
116 const char byte = static_cast<char>(i & 0xFFU);
117 output.write(&byte, 1);
118 }
119 }
120
121 void test_tiny_mlp() {
122 const lcqi::TinyMlpModel model = lcqi::load_model(test_model_path());
123 require(model.input_size == LCQI_TEST_INPUT_SIZE, "input size mismatch");
124 require(model.hidden_size == LCQI_TEST_HIDDEN_SIZE, "hidden size mismatch")↵
125 ↵ ;
126 require(model.output_size == LCQI_TEST_OUTPUT_SIZE, "output size mismatch")↵
127 ↵ ;
128
129 const std::vector<float> input{1.0F, 2.0F, -1.0F, 0.5F};
130 const lcqi::InferenceResult result = lcqi::run_inference(model, input);
131
132 require(result.predicted_class == LCQI_TEST_PREDICTED_CLASS, "unexpected ↵
133 ↵ predicted class");
134 require(result.logits.size() == static_cast<std::size_t>(↵

```

```

 ↪ LCQI_TEST_OUTPUT_SIZE),
132 "unexpected logits size");
133 require_close(result.logits[0], LCQI_TEST_LOGIT_0, "logit 0 mismatch");
134 require_close(result.logits[1], LCQI_TEST_LOGIT_1, "logit 1 mismatch");
135
136 std::vector<float> hidden(static_cast<std::size_t>(LCQI_TEST_HIDDEN_SIZE), ↪
 ↪ 0.0F);
137 lcqi::linear_i8(model.hidden, input, hidden);
138 lcqi::relu_inplace(hidden);
139 require_close(hidden[0], LCQI_TEST_HIDDEN_0, "hidden 0 mismatch");
140 require_close(hidden[1], LCQI_TEST_HIDDEN_1, "hidden 1 mismatch");
141 require_close(hidden[2], LCQI_TEST_HIDDEN_2, "hidden 2 mismatch");
142 }
143
144 void test_tokenizer_contract() {
145 const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(↪
 ↪ test_tokenizer_path());
146 require(tokenizer.bos_id == 0, "tokenizer BOS id mismatch");
147 require(tokenizer.eos_id == 4, "tokenizer EOS id mismatch");
148 require(tokenizer.unk_id == -1, "tokenizer UNK id mismatch");
149 require(tokenizer.chat_template_hash == "tiny-chat-template-v1",
150 "tokenizer template hash mismatch");
151
152 const std::vector<std::int32_t> token_ids =
153 lcqi::encode_prompt(tokenizer, "tok1 tok2 tok3");
154 const std::vector<std::int32_t> expected{0, 1, 2, 3, 4};
155 require(token_ids == expected, "tokenizer prompt ids mismatch");
156 require(lcqi::tokenizer_contract_hash(tokenizer) == ↪
 ↪ LCQI_TOKENIZER_HASH_EXPECTED,
157 "tokenizer contract hash mismatch");
158 }
159
160 void test_tokenizer_rejects_unknown_without_unk() {
161 const lcqi::TokenizerModel tokenizer = lcqi::load_tokenizer(↪
 ↪ test_tokenizer_path());
162 bool threw = false;
163 try {
164 static_cast<void>(lcqi::encode_prompt(tokenizer, "tok1 missing_token"))↪
 ↪ ;
165 } catch (const std::runtime_error&) {
166 threw = true;
167 }
168 require(threw, "tokenizer should reject unknown token when unk id is absent↪
 ↪ ");
169 }
170
171 void test_safetensors_manifest_contract() {
172 const std::filesystem::path path =
173 temp_file_path("lcqi_safetensors_manifest_contract.safetensors");
174 const std::string header =
175 R("{"__metadata__":{"format":"lcqi-fixture"},"model.embed_tokens.weight↪
 ↪ {"dtype":"F32","shape":[2,3],"data_offsets":[0,24]},"model.layers.0.↪

```

```

175 ↪ attn.q_proj.weight":{"dtype":"F16","shape":[4,3],"data_offsets"↪
176 ↪ :[24,48]}}}");
177 write_safetensors_fixture(path, header, ↪
178 ↪ LCQI_SAFETENSORS_VALID_PAYLOAD_BYTES);
179
180
181 const lcqi::SafeTensorManifest manifest = lcqi::load_safetensors_manifest(↪
182 ↪ path);
183 std::filesystem::remove(path);
184
185 require(manifest.header_size == static_cast<std::uint64_t>(header.size()),
186 "safetensors header size mismatch");
187 require(manifest.data_start_offset ==
188 static_cast<std::uint64_t>(LCQI_SAFETENSORS_PREFIX_BYTES) +
189 static_cast<std::uint64_t>(header.size()),
190 "safetensors data start mismatch");
191 require(manifest.metadata.size() == 1, "safetensors metadata count mismatch↪
192 ↪ ");
193 require(manifest.metadata[0].first == "format" &&
194 manifest.metadata[0].second == "lcqi-fixture",
195 "safetensors metadata mismatch");
196 require(manifest.tensors.size() == 2, "safetensors tensor count mismatch");
197
198 const lcqi::SafeTensorEntry* embedding =
199 manifest.find_tensor("model.embed_tokens.weight");
200 require(embedding != nullptr, "safetensors embedding tensor missing");
201 require(embedding->dtype == "F32", "safetensors embedding dtype mismatch");
202 require(embedding->shape == std::vector<std::int64_t>({2, 3}),
203 "safetensors embedding shape mismatch");
204 require(embedding->data_begin == LCQI_SAFETENSORS_EMBED_BEGIN &&
205 embedding->data_end == LCQI_SAFETENSORS_EMBED_END,
206 "safetensors embedding offset mismatch");
207 require(embedding->byte_size() ==
208 LCQI_SAFETENSORS_EMBED_END - LCQI_SAFETENSORS_EMBED_BEGIN,
209 "safetensors embedding byte size mismatch");
210
211 const lcqi::SafeTensorEntry* q_proj =
212 manifest.find_tensor("model.layers.0.attn.q_proj.weight");
213 require(q_proj != nullptr, "safetensors q projection tensor missing");
214 require(q_proj->dtype == "F16", "safetensors q projection dtype mismatch");
215 require(q_proj->data_begin == LCQI_SAFETENSORS_QPROJ_BEGIN &&
216 q_proj->data_end == LCQI_SAFETENSORS_QPROJ_END,
217 "safetensors q projection offset mismatch");
218 }
219
220 void test_safetensors_rejects_bad_offsets() {
221 const std::filesystem::path path =
222 temp_file_path("lcqi_safetensors_bad_offsets.safetensors");
223 const std::string header =
224 R("{'tensor':{'dtype':'F32','shape':[4],'data_offsets':[0,32]}}}");
225 write_safetensors_fixture(path, header, 4);
226
227 bool threw = false;

```

```

223 try {
224 static_cast<void>(lcqi::load_safetensors_manifest(path));
225 } catch (const std::runtime_error&) {
226 threw = true;
227 }
228 std::filesystem::remove(path);
229 require(threw, "safetensors parser should reject offsets beyond payload");
230 }
231
232 void test_safetensors_rejects_bad_dtype_shape_size() {
233 const std::filesystem::path path =
234 temp_file_path("lcqi_safetensors_bad_dtype_shape_size.safetensors");
235 const std::string header =
236 R("{\"tensor\":{\"dtype\":\"F32\",\"shape\":[4],\"data_offsets\":[0,8]}}");
237 write_safetensors_fixture(path, header, 8);
238
239 bool threw = false;
240 try {
241 static_cast<void>(lcqi::load_safetensors_manifest(path));
242 } catch (const std::runtime_error&) {
243 threw = true;
244 }
245 std::filesystem::remove(path);
246 require(threw, "safetensors parser should reject dtype/shape byte mismatch" ←
 ↪);
247 }
248
249 void test_reference_rms_norm() {
250 const std::vector<float> input{3.0F, 4.0F};
251 const std::vector<float> weight{1.0F, 0.5F};
252 std::vector<float> output(2, 0.0F);
253 lcqi::rms_norm(input, weight, 0.0F, output);
254 require_close(output[0], LCQI_RMS_EXPECTED_0, "RMSNorm output 0 mismatch");
255 require_close(output[1], LCQI_RMS_EXPECTED_1, "RMSNorm output 1 mismatch");
256 }
257
258 void test_reference_rope() {
259 std::vector<float> heads{1.0F, 0.0F, 0.0F, 1.0F};
260 lcqi::apply_rope(heads, 1, 4, 1, 10000.0F);
261 require_close(heads[0], std::cos(1.0F), "RoPE pair 0 cosine mismatch");
262 require_close(heads[1], std::sin(1.0F), "RoPE pair 0 sine mismatch");
263 require_close(heads[2], -std::sin(0.01F), "RoPE pair 1 negative sine ←
 ↪ mismatch");
264 require_close(heads[3], std::cos(0.01F), "RoPE pair 1 cosine mismatch");
265 }
266
267 void test_reference_kv_attention() {
268 lcqi::DecoderConfig config;
269 config.hidden_size = 4;
270 config.query_heads = 2;
271 config.kv_heads = 1;
272 config.head_dim = 2;

```

```

273 config.intermediate_size = 4;
274 config.vocab_size = 4;
275 config.max_context = 4;
276
277 lcqi::ReferenceKVCache cache(config, 1);
278 const std::vector<float> key{1.0F, 0.0F};
279 const std::vector<float> value{LCQI_ATTENTION_EXPECTED_0, ←
 ↪ LCQI_ATTENTION_EXPECTED_1};
280 cache.append(0, 0, key, value);
281 require(cache.filled_tokens() == 1, "KV filled token count mismatch");
282 require_close(cache.key(0, 0, 0)[0], 1.0F, "KV key read mismatch");
283
284 const std::vector<float> query{1.0F, 0.0F, 0.0F, 1.0F};
285 std::vector<float> output(4, 0.0F);
286 lcqi::attention_decode(config, cache, 0, 0, query, output);
287 require_close(output[0], LCQI_ATTENTION_EXPECTED_0, "attention head 0 dim 0←
 ↪ mismatch");
288 require_close(output[1], LCQI_ATTENTION_EXPECTED_1, "attention head 0 dim 1←
 ↪ mismatch");
289 require_close(output[2], LCQI_ATTENTION_EXPECTED_0, "attention head 1 dim 0←
 ↪ mismatch");
290 require_close(output[3], LCQI_ATTENTION_EXPECTED_1, "attention head 1 dim 1←
 ↪ mismatch");
291 }
292
293 void test_reference_decoder_end_to_end() {
294 const lcqi::ReferenceDecoderModel model = lcqi::←
 ↪ make_tiny_reference_decoder_model();
295 const std::vector<std::int32_t> tokens{1, 2, 3};
296 const lcqi::ReferenceDecodeResult result = lcqi::run_reference_decode(model←
 ↪ , tokens);
297 require(result.logits.size() == LCQI_REFERENCE_DECODER_VOCAB_SIZE,
298 "reference decoder logits size mismatch");
299 require(result.predicted_token == LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,
300 "reference decoder predicted token mismatch");
301 for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.size(); ←
 ↪ ++index) {
302 require_close(result.logits[index],
303 LCQI_REFERENCE_DECODER_LOGITS[index],
304 "reference decoder golden logit mismatch");
305 }
306 }
307
308 void test_reference_decoder_trace_contract() {
309 const lcqi::ReferenceDecoderModel model = lcqi::←
 ↪ make_tiny_reference_decoder_model();
310 const std::vector<std::int32_t> tokens{1, 2, 3};
311 const lcqi::ReferenceDecodeTraceResult trace =
312 lcqi::run_reference_decode_with_trace(model, tokens);
313
314 require(trace.result.predicted_token == ←
 ↪ LCQI_REFERENCE_DECODER_PREDICTED_TOKEN,

```

```

315 "reference trace predicted token mismatch");
316 require(!trace.checkpoints.empty(), "reference trace checkpoints missing");
317
318 const lcqi::ReferenceTensorCheckpoint& first = trace.checkpoints.front();
319 require(first.name == "embedding", "first checkpoint should be embedding");
320 require(first.token_position == LCQI_REFERENCE_TRACE_FIRST_TOKEN,
321 "first checkpoint token position mismatch");
322 require(first.dtype == "f32", "first checkpoint dtype mismatch");
323 require(first.layout == "row_major", "first checkpoint layout mismatch");
324 require(first.shape.size() == 1 &&
325 first.shape[0] == LCQI_REFERENCE_TRACE_HIDDEN_SIZE,
326 "first checkpoint shape mismatch");
327 require(first.stride.size() == 1 && first.stride[0] == 1,
328 "first checkpoint stride mismatch");
329
330 bool saw_logits = false;
331 bool saw_kv_slot = false;
332 for (const lcqi::ReferenceTensorCheckpoint& checkpoint : trace.checkpoints) {
333 if (checkpoint.name == "kv_cache_key_slot") {
334 saw_kv_slot = true;
335 require(checkpoint.shape.size() == 4, "KV checkpoint rank mismatch");
336 require(checkpoint.layout == "kv_contiguous", "KV checkpoint layout
337 mismatch");
338 }
339 if (checkpoint.name == "logits") {
340 saw_logits = true;
341 require(checkpoint.token_position ==
342 LCQI_REFERENCE_TRACE_TOKEN_COUNT - 1,
343 "logits checkpoint token position mismatch");
344 require(checkpoint.shape.size() == 1 &&
345 checkpoint.shape[0] ==
346 static_cast<std::int32_t>(
347 LCQI_REFERENCE_DECODER_VOCAB_SIZE),
348 "logits checkpoint shape mismatch");
349 require(checkpoint.values.size() == LCQI_REFERENCE_DECODER_LOGITS.
350 size(),
351 "logits checkpoint value size mismatch");
352 for (std::size_t index = 0; index < LCQI_REFERENCE_DECODER_LOGITS.
353 size(); ++index) {
354 require_close(checkpoint.values[index],
355 LCQI_REFERENCE_DECODER_LOGITS[index],
356 "trace logits checkpoint mismatch");
357 }
358 }
359 }
360 require(saw_kv_slot, "KV checkpoint missing");
361 require(saw_logits, "logits checkpoint missing");
362 }
363
364 void test_packed_i8_kernel_matches_scalar() {

```



```

360 for (const KernelShapeCase& shape_case : LCQI_KERNEL_SHAPE_CASES) {
361 const lcqi::QuantizedLinearLayer layer =
362 lcqi::make_deterministic_i8_layer(shape_case.input_size,
363 shape_case.output_size,
364 0.03125F);
365 const lcqi::PackedLinearI8 packed =
366 lcqi::pack_linear_i8(layer, shape_case.output_block_size);
367 const std::vector<float> input = lcqi::make_deterministic_input(layer.↵
↵ input_size);
368 std::vector<float> scalar_output(static_cast<std::size_t>(layer.↵
↵ output_size), 0.0F);
369 std::vector<float> packed_output(static_cast<std::size_t>(layer.↵
↵ output_size), 0.0F);
370 lcqi::linear_i8(layer, input, scalar_output);
371 lcqi::linear_i8_packed(packed, input, packed_output);
372 require(lcqi::max_abs_diff(scalar_output, packed_output) <= ↵
↵ LCQI_KERNEL_MAX_DIFF,
373 "packed int8 kernel output differs from scalar");
374
375 const lcqi::PackedLinearI8 simd_packed =
376 lcqi::pack_linear_i8(layer, LCQI_SIMD_TEST_BLOCK);
377 if (lcqi::linear_i8_packed_avx2_available()) {
378 std::vector<float> avx2_output(static_cast<std::size_t>(layer.↵
↵ output_size), 0.0F);
379 lcqi::linear_i8_packed_avx2(simd_packed, input, avx2_output);
380 require(lcqi::max_abs_diff(scalar_output, avx2_output) <= ↵
↵ LCQI_KERNEL_MAX_DIFF,
381 "AVX2 int8 kernel output differs from scalar");
382 }
383 }
384 }
385
386 } // namespace
387
388 int main() {
389 try {
390 test_tiny_mlp();
391 test_tokenizer_contract();
392 test_tokenizer_rejects_unknown_without_unk();
393 test_safetensors_manifest_contract();
394 test_safetensors_rejects_bad_offsets();
395 test_safetensors_rejects_bad_dtype_shape_size();
396 test_reference_rms_norm();
397 test_reference_rope();
398 test_reference_kv_attention();
399 test_reference_decoder_end_to_end();
400 test_reference_decoder_trace_contract();
401 test_packed_i8_kernel_matches_scalar();
402
403 std::cout << "lcqi tests passed\n";
404 return EXIT_SUCCESS;
405 } catch (const std::exception& error) {

```

```
406 std::cerr << "lcqi test failure: " << error.what() << "\n";
407 return EXIT_FAILURE;
408 }
409 }
```

## 第三册能力清单

第三册的出口是能把本地大模型推理拆成模型资产、张量、算子、KV cache、量化、compiler/runtime、后端、serving 和应用编排合同，并能用 oracle、trace 和 benchmark 证明关键判断。

### 能实现

- 实现 tiny decoder reference, 覆盖 embedding、RMSNorm、Q/K/V、RoPE、GQA attention、KV append/read、SwiGLU、lm head 和 sampler。
- 实现模型导入闭环: tokenizer fixture、GGUF/safetensors 或等价 manifest、tensor name mapping、shape/dtype/quant verifier 和首 token trace。
- 实现 int8/int4 至少一种量化路径的 descriptor、packing、operator oracle、误差报告和容量报告。
- 实现 serving probe: admission、KV budget、prefill/decode 区分、TTFT、TPOT、p95、取消、backpressure 和拒绝原因。

### 能诊断

- 诊断 tokenizer/chat template/vocab/embedding 不一致导致的 token drift 或 logits drift。
- 诊断 KV cache 的 layer、position、kv\_head、dim、physical slot、prefix 和 sliding window 错误。
- 诊断 decode bytes/token、GQA/MQA 节省量、context length 斜率、paged KV 和 continuous batching 的资源影响。
- 诊断 AI compiler 中 shape inference、layout propagation、fusion legality、alias/liveness、buffer reuse 和 stateful op 调度约束。

### 能解释

- 解释 Transformer 数据流中 RMSNorm、RoPE、softmax、SwiGLU、sampling、低精度误差和数值稳定性的边界。
- 解释 GEMM/GEMV、prefill/decode、packing、blocking、register tile、cache tile、SIMD tail 和 GPU launch/copy/sync 的不同成本。
- 解释 llama.cpp/ggml、oneDNN、ONNX Runtime、vLLM、TensorRT-LLM、TVM/MLIR/XLA 等成熟系统应从哪些源码对象和设计合同拆解。
- 解释 LangChain/LangGraph/LlamaIndex 类上层框架如何影响 token budget、RAG trace、tool side effect、checkpoint 和本地推理服务接口。

### 不能夸大的部分

第三册主线是本地推理引擎和 AI Infra 骨架。多 GPU 训练级别通信、生产级 CUDA/HIP kernel 库、完整分布式 serving 平台和所有模型格式方言仍需要专项项目继续扩展。

## 术语表

**张量** 带元素类型、形状、步长和生命周期的数据视图。

**算子** 有输入合同、输出合同和错误边界的计算单元。

**CPU** Central Processing Unit, 中央处理器。本册 CPU 后端通常指本地内存、SIMD、多线程和 cache/TLB 下的实现。

**GPU** Graphics Processing Unit, 图形处理器或通用并行加速器。本册 GPU 后端关注 kernel、stream、显存和同步边界。

**ISA** Instruction Set Architecture, 指令集架构, 规定软件可见的指令、寄存器和状态。

**API** Application Programming Interface, 源码层调用接口。

**ABI** Application Binary Interface, 二进制层调用、对象布局 and 链接合同。

**SDK** Software Development Kit, 软件开发工具包, 通常包含头文件、库、工具、示例和文档。

**GDB** GNU Debugger, GNU 调试器, 用于断点、栈帧、变量、反汇编和崩溃现场分析。

**C++20** C++ 标准版本名。本册默认使用的语言和标准库能力边界。

**SIMD** Single Instruction, Multiple Data, 一条指令处理多个数据 lane。

**AVX/AVX2** x86 向量指令扩展, 常用于 CPU 后端热循环。

**AVX-512** x86 512 位向量扩展, 提供更宽寄存器和 mask 机制, 但要验证频率、tail 和平台覆盖。

**FMA** Fused Multiply-Add, 一条指令完成乘加。

**VNNI** Vector Neural Network Instructions, 面向低精度 dot-product 的向量指令能力。

**AMX** Advanced Matrix Extensions, x86 tile 级矩阵扩展, 适合足够大的矩阵块计算。

**PMU** Performance Monitoring Unit, 硬件性能计数器来源。

**IPC** Instructions Per Cycle, 每周期执行指令数, 需要结合内存、分支和前后端瓶颈解释。

**TLB** Translation Lookaside Buffer, 地址翻译缓存。

**L2/L3** CPU 缓存层级。L2 通常比 L1 大且更慢, L3 通常更大并可能被多个核心共享。

**LLC** Last-Level Cache, 最后一级缓存。

**DRAM** Dynamic Random-Access Memory, 主存。

**HBM** High Bandwidth Memory, 高带宽显存, 常见于数据中心 GPU。

**NUMA** Non-Uniform Memory Access, 非统一内存访问, 不同节点访问成本不同。

**JSON** JavaScript Object Notation, 结构化文本格式, 常用于 config、tokenizer、manifest 和报告。

**CSV** Comma-Separated Values, 逗号分隔文本表格, 常用于 benchmark 和 trace 摘要。

**CI** Continuous Integration, 持续集成, 用自动测试和构建保护回归。

**I/O** Input/Output, 输入输出, 模型加载、日志、trace 和文件映射都会涉及。

**Unicode** 统一字符集合, tokenizer 和文本规范化必须明确它和字节编码的关系。

**UTF-8** Unicode 的常见变长字节编码。prompt、stop string 和 golden case 要保存原始字节边界。

**KB/MB/GB** 存储容量单位。模型文件、权重、KV cache 和 arena 报告必须统一单位口径。

**KiB/MiB/GiB** 二进制容量单位, 分别按 1024 进位。报告内存、显存和 mmap 区间时, 混用 GB 与 GiB 会造成可见误差。

**mmap** 把文件或匿名内存映射到进程虚拟地址空间的系统接口。模型加载常用它减少显式读拷贝, 但仍要处理页故障、生命周期和文件截断风险。

**perf** Linux 性能分析工具, 用采样和 PMU 事件观察热点、cache miss、分支、调度和调用栈。

**objdump** 目标文件和可执行文件查看/反汇编工具, 用来确认后端是否生成预期 SIMD、VNNI 或 AMX 指令。

**BM/BN/BK** GEMM tile 记号。BM 是 batch 行块, BN 是输出通道块, BK 是归约维度块。

**TILE\_N/TILE\_K** 后端 schedule 常量, 分别表示 N 方向列块和 K 方向归约块大小。

**量化** 用较低位宽的整数近似表示实数值, 并用 scale 和 zero point 记录映射关系。

**校准** 为量化选择数值范围和 scale 的过程。权重可以离线扫描，activation 和 KV cache 通常需要代表性输入或运行时策略。

**Requantize** 把较宽累加结果重新映射到目标低位宽输出的步骤，通常包含固定点乘法、舍入、加 zero point 和 clamp。

**Group Quantization** 按一组连续元素共享 scale 的量化方式，常用于 int4/int8 大模型权重；它降低误差但要求 kernel 正确读取 group scale。

**QuantDescriptor** 描述量化格式的结构化元数据，包括 bit width、scale dtype、zero point policy、group size、axis、packed order 和 tail policy。

**CalibrationRecord** 记录量化校准来源和结果的工程对象，包括校准集、算法、范围、饱和率、scale 摘要和工具版本。

**Reference Converter** 用清晰标量实现把低比特字节、scale 和 zero point 还原为参考浮点值的组件，用来验证 packed order、tail policy 和 metadata 解释。

**Reference** 用来定义正确结果的清晰实现，不以速度为目标。

**Oracle** 判定实现输出是否满足 reference 和误差合同的比较器。

**Kernel** 某个算子在特定 dtype、layout 和目标机器上的具体实现。

**推理图** 算子节点和张量边组成的有向计算结构。

**AI 编译器** 把模型图、张量合同、量化元数据和后端能力转换成可执行计划的编译流水线。

**编译前端** 把外部输入转换成已检查内部表示的阶段。在 AI 推理引擎里，它读取模型文件、tokenizer、权重索引和量化 metadata，输出 manifest 与 typed graph。

**IR** 中间表示。AI 编译器中常分为表达算子依赖的 graph IR、表达 shape/layout/dtype 的 tensor IR，以及表达 tile/vector/thread 的 loop IR。

**SSA** Static Single Assignment，静态单赋值形式，每个虚拟值只定义一次，便于数据流和优化分析。

**Pass** 读取一种 IR 并产出改写后 IR 或分析事实的编译步骤，例如融合 pass、layout pass、liveness pass 和 lowering pass。

**Analysis Pass** 不改写 IR、只收集事实的 pass，例如 use-def、liveness、alias、shape、quant 和 cost analysis。

**Transform Pass** 根据已有分析事实改写 IR 的 pass，例如融合、消除冗余 reshape、量化 lowering 和 layout 转换。

**Lowering** 把较高层表示逐步降到更接近目标后端的表示，例如从算子图降到 CPU SIMD kernel 或 GPU device kernel 调用。

**后端** 把已验证的 IR 或 executable plan 落到具体硬件和运行环境的实现层，例如 CPU SIMD/多线程后端或 GPU stream/kernel 后端。

**Executable Plan** 编译器输出给 runtime 的可执行计划，包含 segment、kernel 选择、arena offset、外部状态绑定点和后端同步要求。

**内存规划** 根据张量生命周期、大小、对齐和后端位置安排 arena、workspace、packed weight 和 KV cache 的过程。

**Manifest** 模型导入后形成的机器可读清单，记录 config、tokenizer、张量位置、shape、dtype、layout 和量化元数据。

**Verifier** 检查 IR、shape、dtype、量化和状态边是否满足合同的组件；失败时应给出可定位诊断，而不是让后端继续执行。

**Schema Resolution** 把不同模型格式或命名方言映射到引擎内部 canonical role 的过程。

**Activation Arena** 根据中间张量生命周期预先规划的一块或多块临时内存，用 offset 复用短生命周期 activation，避免热路径反复分配。

**Packed Weight** 后端准备阶段从逻辑权重转换出的物理布局，通常包含 tile 重排、对齐、padding、量化 scale 布局和 kernel 读取合同。

**KV Cache Quantization** 对推理过程中动态增长的 K/V 状态做低比特存储的策略。它不同于静态权重量化，必须同时考虑写入成本、metadata、长上下文误差和 decode attention 热路径。

**KV cache** Key/Value cache，Transformer 解码时保存历史 token 的 K/V 张量，避免每一步重算全部历史。



**QKV** Query、Key、Value。attention 用 Q 查询历史 K，再用权重加权 V。

**QH/KVH/KVA** Query head 数和 key/value head 数的形状变量。GQA/MQA 下二者关系必须被 verifier 检查。

**BN** Batch 或 block 相关维度记号，具体含义要看所在公式或 fixture。

**MLP** Multi-Layer Perceptron，多层感知机。Transformer 中通常指 attention 后的前馈网络，由升维、激活/门控和降维投影组成。

**GEMM** General Matrix-Matrix Multiplication，矩阵乘矩阵。prefill 阶段常更接近 GEMM。

**GEMV** General Matrix-Vector Multiplication，矩阵乘向量。decode 阶段很多投影更接近 GEMV。

**FLOP/FLOPS/TFLOP** FLOP 是一次浮点运算；FLOPS 是每秒浮点运算次数；TFLOP 表示每秒万亿次浮点运算。指标必须和 dtype、batch、内存带宽一起解释。

**MFLOP/GFLOP** 百万/十亿级浮点运算量或吞吐单位，报告中要区分总量和每秒速率。

**FP16/BF16/TF32/INT8** 常见低精度数据类型。FP16 是 16 位浮点；BF16 保留更多指数位；TF32 是 NVIDIA Tensor Core 常见计算格式；INT8 是 8 位整数推理常见格式。

**MAC** Multiply-Accumulate，乘加次数。很多模型账本会先数 MAC，再按约定换算 FLOP。

**RMSNorm** Root Mean Square Layer Normalization，用均方根缩放 hidden state 的归一化方式。

**RoPE** Rotary Position Embedding，旋转位置编码，把位置信息注入 Q/K。

**SwiGLU** Swish-Gated Linear Unit，一种门控前馈网络结构。

**MHA** Multi-Head Attention，多头注意力，每个 query head 有独立 K/V head。

**GQA** Grouped-Query Attention，多个 query head 共享一组 KV head，降低 KV cache 体积。

**MQA** Multi-Query Attention，所有 query head 共享一组 KV head，进一步降低 KV cache。

**QK** Attention 中 query 与 key 的乘积，用来得到注意力分数。

**PV** Attention 中概率权重与 value 的乘积，用来得到上下文表示。

**FlashAttention** 分块 attention 思路，通过在线 softmax 和 tile 复用减少中间矩阵写回。

**Segment** executable plan 中比单个算子更粗的调度单元，绑定 arena offset、外部状态和一段 kernel 序列，用来降低运行时解释开销。

**Time to First Token** 从请求进入到第一个生成 token 可用之间的延迟，主要受 tokenizer、prefill、KV cache 初始化和首轮 logits 影响。

**TTFT** Time To First Token，首 token 延迟。

**TPOT** Time Per Output Token，生成阶段每个输出 token 的平均时间。

**p50/p95/p99** 延迟或资源占用分位数。p50 描述典型请求，p95/p99 描述尾部请求，模型服务验收必须同时报告。

**SLO** Service Level Objective，服务等级目标，例如 TTFT、TPOT 或 p95 延迟阈值。

**FIFO** First In First Out，先进先出。服务调度中 FIFO 简单但可能让长请求阻塞短请求。

**FAQ** Frequently Asked Questions，常见问题。RAG chunking 要避免把 FAQ 的问题和答案拆散。

**DDIA** Designing Data-Intensive Applications，常被用作数据密集系统设计材料简称。本册提到它时，关注 artifact、调度、日志和可恢复系统思维。

**RAG** Retrieval-Augmented Generation，检索增强生成，先检索外部资料再把资料放入 prompt。

**BM25** 经典稀疏关键词检索打分方法，RAG 中常和 dense vector 检索互补。

**HTML** HyperText Markup Language，网页标记语言。RAG ingestion 需要把它解析成安全文本和元数据。

**LLM** Large Language Model，大语言模型。

**GGUF** llama.cpp 生态常见模型文件格式，打包权重、量化信息和模型元数据。

**ggml** llama.cpp 生态中的张量和推理基础库。阅读它时重点看 tensor layout、量化格式、kernel dispatch 和内存规划。

**llama.cpp** C/C++ 本地 LLM 推理项目，常作为 CPU/GPU 后端、GGUF、tokenizer 和量化工程的源码阅读样本。

**vLLM** 高吞吐 LLM 服务系统，常用于对照 continuous batching、KV cache 管理、调度和服务 SLO。

**oneDNN** CPU 深度学习算子库，提供卷积、矩阵乘、归一化等优化 primitive，可作为手写 kernel

的正确性和性能对照。

**ONNX** Open Neural Network Exchange, 开放神经网络交换格式, 用图、算子、张量和权重描述模型。

**BPE** Byte Pair Encoding, 一类 tokenizer 子词合并算法。

**BOS/EOS/PAD** BOS 是句首 token, EOS 是句尾 token, PAD 是填充 token。

**组合缩写** 斜杠并列的缩写组合, 例如 **GEMM/GEMV** 或 **MLIR/TVM**。它提示这些概念在当前段落被放在一起比较, 不代表新术语。

**UNK** Unknown token, 未知 token, 用来表示 tokenizer 无法映射到已知词表项的输入。

**OOV** Out-of-vocabulary, 词表外输入。BPE 等子词 tokenizer 会降低 OOV 概率, 但词表、normalization 和 byte fallback 仍要验证。

**MoE** Mixture of Experts, 专家混合结构。

**LoRA** Low-Rank Adaptation, 低秩适配, 用低秩增量调整模型权重。

**GPTQ** 一类训练后权重量化方法, 常用近似二阶信息逐层降低量化误差。

**AWQ** Activation-aware Weight Quantization, 激活感知权重量化, 常通过保护重要通道降低低比特权重量化损失。

**NF4** NormalFloat 4-bit, 一类非均匀 4-bit codebook 量化格式, 通常要同时保存 codebook、block scale 和 dequant 合同。

**LCQI** Local CPU Quantized Inference, 本书本地 CPU 量化推理实验线的项目代号。

**LCQI\_TOKENIZER\_V1** LCQI 教学工程里的 tokenizer 合同版本名, 用来固定 vocab、special token、normalization 和 byte fallback 解释。

**AST** Abstract Syntax Tree, 抽象语法树, 传统编译器前端常用的源码结构表示。

**MLIR/TVM/XLA** 工业编译器或编译框架。本书用它们比较 IR、pass、schedule 和 lowering 边界。

**CUDA/HIP** CUDA 是 NVIDIA GPU 编程接口, HIP 常用于 AMD GPU 生态。

**NCCL** NVIDIA Collective Communications Library, 多 GPU collective 通信库, 常见于 all-reduce、broadcast 等训练或服务并行场景。

**ACL** Arm Compute Library, Arm 生态常见计算库, 可作为 CPU/GPU 后端算子实现来源。

**PCIe** Peripheral Component Interconnect Express, 主机和设备之间常见高速互连。

**OOM** Out Of Memory, 内存不足。

**RNG** Random Number Generator, 随机数生成器, 采样时必须纳入可复现性边界。

**NaN** Not-a-Number, 浮点数中的非数字值, 常提示数值计算已经失控。

**KL/MSE** KL divergence 和 mean squared error, 量化校准常用的分布差异或误差指标。

**MMR** Maximal Marginal Relevance, 最大边际相关性, 检索中用于在相关性和多样性之间折中。

**MRR** Mean Reciprocal Rank, 平均倒数排名, 用于评价第一个相关结果排得是否靠前。

**PII** Personally Identifiable Information, 个人可识别信息。RAG 和日志系统必须避免把它无意写入 prompt、trace 或训练样本。

**UB** Undefined Behavior, C++ 未定义行为。推理后端中的越界、悬空指针、错误对齐和数据竞争都可能触发 UB。

**Micro-kernel** 面向特定 dtype、layout、tile 和指令集的小型核心计算内核, 例如一个 AVX2 int8 GEMV tile。它通常由外层 packing、线程调度和尾部路径包围。

**RAII** C++ 中用对象生命周期管理资源的惯用法。推理后端中, 文件映射、aligned buffer、device buffer、stream 和 profiler event 都应由 RAII 对象管理。

**Baseline** 优化前可重复测量的基线实现。它可以不快, 但必须正确、可构建、可 benchmark, 并能作为后续优化的比较对象。

**Tail Path** 处理维度不能整除 SIMD 宽度、tile 大小或 group size 时的边界路径。它必须被 descriptor、verifier、kernel 和测试共同覆盖。

**ThreadPlan** CPU 后端对任务划分、最小任务粒度、barrier、scratch、输出写策略和亲和策略的描述。



**DeviceBuffer** GPU 后端中拥有显存生命周期的 RAII 对象, 避免裸 device pointer 在计划和运行时之间泄漏所有权。

**SM/CU** GPU 上的基本执行资源组。NVIDIA 常称 SM, AMD 常称 CU; 它们包含调度、寄存器、执行单元、shared memory 和矩阵计算单元等资源。

**HBM/GDDR** GPU 常见显存类型。HBM 带宽高、封装近; GDDR 更常见于消费级 GPU。

**Warp/Wavefront** GPU 中一组 lockstep 执行的线程。它类似放大的 SIMD lane 组, 喜欢规则分支和连续访存。

**Shared Memory** GPU SM/CU 内部的快速小容量内存, 常用于 tile 复用; 容量和使用量会影响 occupancy。

**Tensor Core** GPU 中面向矩阵块乘加的专用计算单元, 通常对 dtype、shape、layout 和对齐有要求。

**Occupancy** 一个 SM/CU 上可同时驻留的 warp/block 相对上限的比例。它影响隐藏延迟的能力, 但不能单独代表性能。

**Stream** GPU 后端的异步执行队列。kernel launch、拷贝和 event 通常挂在 stream 上, runtime 必须用它表达同步边界。

**StagingBuffer** Host 与 device 之间传输 logits、token id 或小 metadata 的中转缓冲。它的大小、同步和复用会影响 decode latency。

**Performance Gate** 性能回归门禁, 记录硬件、输入、baseline、阈值和指标, 用来判断一次改动是否让关键路径退化。

**ProfilerEvent** 统一性能事件记录, 通常包含 stage、segment、backend、kernel、时间、context length、内存和可选硬件计数器。

**UI** User Interface, 用户界面。服务和工具章节提到 UI 时, 重点是状态机、取消和回调顺序。

**PR** Pull Request, 代码合并请求。验收章节用它表示进入审查和门禁的变更单位。

**PDF** Portable Document Format, 固定版式文档格式。RAG ingestion 读取 PDF 时要处理页眉页脚、断行、表格和页码噪声。

**QA** Quality Assurance 或 Question Answering。工程验收章节多指质量保证; RAG/应用章节多指问答任务。

# 索引

instruction  
    add, 12  
    max, 13  
    mul, 12  
IR, 195  
manifest, 161  
MiniLLM-CPU, iii  
occupancy, 257

packing, 19  
Pass, 198  
  
SIMD, 20  
  
TTFT, 7  
  
可执行计划, 4  
因果语言建模, 82